

SECOND EDITION

PROGRAMMING IN C++



ASHOK N. KAMTHANE



ALWAYS LEARNING

PEARSON

Programming in C++

Second Edition

Ashok Namdev Kamthane

Associate Professor

Department of Electronics and Telecommunication Engineering

Shri Guru Gobind Singhji

Institute of Engineering and Technology

Nanded, Maharashtra

PEARSON

Delhi • Chennai

Copyright © 2013 Dorling Kindersley (India) Pvt. Ltd.

Licensees of Pearson Education in South Asia

No part of this eBook may be used or reproduced in any manner whatsoever without the publisher's prior written consent.

This eBook may or may not include all assets that were part of the print version. The publisher reserves the right to remove any material in this eBook at any time.

ISBN 9788131791448

eISBN 9789332520288

Head Office: A-8(A), Sector 62, Knowledge Boulevard, 7th Floor, NOIDA 201 309, India

Registered Office: 11 Local Shopping Centre, Panchsheel Park, New Delhi 110 017, India

Brief Contents

PREFACE	XV
ABOUT THE AUTHOR	XVII
1. INTRODUCTION TO C++	1
2. BASICS OF C++	21
3. INPUT AND OUTPUT IN C++	33
4. C++ DECLARATIONS	99
5. DECISION STATEMENTS	161
6. CONTROL LOOP STRUCTURES	187
7. FUNCTIONS IN C++	203
8. CLASSES AND OBJECTS	257
9. CONSTRUCTORS AND DESTRUCTORS	345
10. OPERATOR OVERLOADING AND TYPE CONVERSION	395
11. INHERITANCE	441
12. ARRAYS	511

13. POINTERS	531
14. C++ AND MEMORY MODELS	577
15. BINDING, POLYMORPHISMS, AND VIRTUAL FUNCTIONS	603
16. APPLICATIONS WITH FILES	639
17. GENERIC PROGRAMMING WITH TEMPLATES	697
18. WORKING WITH STRINGS	731
19. EXCEPTION HANDLING	759
20. OVERVIEW OF STANDARD TEMPLATE LIBRARY	785
21. ADDITIONAL INFORMATION ABOUT ANSI AND TURBO-C++	817
22. C++ GRAPHICS	843
APPENDICES	869
INDEX	879

Contents

Preface	xv
About the Author	xvii
1. INTRODUCTION TO C++	1
1.1 Differences between C and C++	1
1.2 Evolution of C++	2
1.3 The ANSI Standard	2
1.4 The Object Oriented Technology	2
1.5 Disadvantage of Conventional Programming	4
1.6 Programming Paradigms	5
1.7 Preface to Object Oriented Programming	6
1.8 Key Concepts of Object Oriented Programming	7
1.9 Advantages of OOP	15
1.10 Object Oriented Languages	16
1.11 Usage of OOP	17
1.12 Usage of C++	18
<i>Summary</i>	18
<i>Exercises</i>	19
2. BASICS OF C++	21
2.1 Introduction	21
2.2 Steps to Create and Execute a C++ Program	21
2.3 Flowchart for Creating a Source File, Compiling, Linking and Executing in C++	22
2.4 C++ Environments	23
2.5 Typical C++ Environment (Borland C++)	24
2.6 Structure of a C++ Program	27
2.7 Illustrative Simple Program in C++ without Class	28
2.8 Header Files and Libraries	29
<i>Summary</i>	30
<i>Exercises</i>	31

3. INPUT AND OUTPUT IN C++	33
3.1 Introduction	33
3.2 Streams in C++ and Stream Classes	34
3.3 Pre-defined Streams	34
3.4 Buffering	35
3.5 Stream Classes	36
3.6 Formatted and Unformatted Data	37
3.7 Unformatted Console I/O Operations	38
3.8 Type Casting with the <code>cout</code> Statement	44
3.9 Member Functions of the <code>istream</code> Class	56
3.10 Formatted Console I/O Operations	59
3.11 Bit Fields	67
3.12 Flags without Bit Fields	70
3.13 Manipulators	71
3.14 User-defined Manipulators	74
3.15 Manipulator with One Parameter	76
3.16 Manipulators with Multiple Parameters	77
3.17 More Programs	79
Summary	87
Exercises	88
4. C++ DECLARATIONS	99
4.1 Introduction	99
4.2 Tokens	100
4.3 Variable Declaration and Initialization	108
4.4 Data Types in C++	115
4.5 Operators in C and C++	129
4.6 Scope Access Operator	133
4.7 Namespace	133
4.8 Memory Management Operators	137
4.9 Comments	141
4.10 Comma Operator	142
4.11 Comma in Place of Curly Braces	143
4.12 More Programs	145
Summary	155
Exercises	156
5. DECISION STATEMENTS	161
5.1 Introduction	161
5.2 The <code>if</code> Statement	162
5.3 Multiple <code>ifs</code>	165
5.4 The <code>if-else</code> Statement	167
5.5 Nested <code>if-else</code> Statements	169
5.6 The <code>else-if</code> Ladder	171
5.7 Unconditional Control Transfer Statements	175

- 5.8 The `switch` Statement 177
- 5.9 Nested `switch case` 182
 - Summary* 183
 - Exercises* 184

6. CONTROL LOOP STRUCTURES

187

- 6.1 Introduction 187
- 6.2 What Is a Loop? 187
- 6.3 The `for` Loop 188
- 6.4 Nested `for` Loops 191
- 6.5 The `while` Loop 192
- 6.6 The `do-while` Loop 195
- 6.7 The `do-while` Statement with `while` Loop 196
- 6.8 More Programs 197
 - Summary* 199
 - Exercises* 199

7. FUNCTIONS IN C++

203

- 7.1 Introduction 203
- 7.2 Parts of a Function 205
- 7.3 Passing Arguments 209
- 7.4 Lvalues and Rvalues 215
- 7.5 Return by Reference 216
- 7.6 Returning More Values by Reference 217
- 7.7 Default Arguments 218
- 7.8 `const` Arguments 222
- 7.9 Inputting Default Arguments 224
- 7.10 Inline Functions 225
- 7.11 Function Overloading 228
- 7.12 Principles of Function Overloading 230
- 7.13 Precautions with Function Overloading 234
- 7.14 Recursion 235
- 7.15 Library Functions 237
- 7.16 More Programs 241
 - Summary* 252
 - Exercises* 252

8. CLASSES AND OBJECTS

257

- 8.1 Introduction 258
- 8.2 Structure in C 259
- 8.3 Structure in C++ 261
- 8.4 Classes in C++ 262
- 8.5 Declaring Objects 263
- 8.6 The `public` Keyword 264

8.7	The <code>private</code> Keyword	265
8.8	The <code>protected</code> Keyword	266
8.9	Access Specifiers and Their Scope	267
8.10	Defining Member Functions	268
8.11	Characteristics of Member Functions	272
8.12	Outside Member Function as Inline	272
8.13	Rules for Inline Functions	274
8.14	Data Hiding or Encapsulation	274
8.15	Classes, Objects, and Memory	277
8.16	<code>static</code> Member Variables	280
8.17	<code>static</code> Member Functions	286
8.18	<code>static</code> Object	289
8.19	Array of Objects	290
8.20	Objects as Function Arguments	292
8.21	<code>friend</code> Functions	295
8.22	The <code>const</code> Member Functions	304
8.23	The Volatile Member Function	305
8.24	Recursive Member Function	306
8.25	Local Classes	307
8.26	<code>empty</code> , <code>static</code> , and <code>const</code> Classes	310
8.27	Member Function and Non-member Function	310
8.28	The <code>main()</code> Function as a Member Function	311
8.29	Overloading Member Functions	312
8.30	Overloading <code>main()</code> Functions	313
8.31	The <code>main()</code> , Member Function, and Indirect Recursion	314
8.32	Bit Fields and Classes	317
8.33	Nested Class	319
8.34	More Programs	320
	<i>Summary</i>	339
	<i>Exercises</i>	340

9. CONSTRUCTORS AND DESTRUCTORS

345

9.1	Introduction	346
9.2	Constructors and Destructors	348
9.3	Characteristics of Constructors and Destructors	349
9.4	Applications with Constructors	350
9.5	Constructors with Arguments (Parameterized Constructor)	353
9.6	Overloading Constructors (Multiple Constructors)	355
9.7	Array of Objects Using Constructors	359
9.8	Constructors with Default Arguments	360
9.9	Copy Constructors	361
9.10	The <code>const</code> Objects	363
9.11	Destructors	364
9.12	Calling Constructors and Destructors	367
9.13	Qualifier and Nested Classes	370

9.14	Anonymous Objects	372
9.15	Private Constructors and Destructors	374
9.16	Dynamic Initialization Using Constructors	375
9.17	Dynamic Operators and Constructors	377
9.18	<code>main()</code> as a Constructor and Destructor	379
9.19	Recursive Constructors	380
9.20	Program Execution Before <code>main()</code>	381
9.21	Constructor and Destructor with Static Members	383
9.22	Local Versus Global Object	384
9.23	More Programs	385
	<i>Summary</i>	392
	<i>Exercises</i>	392

10. OPERATOR OVERLOADING AND TYPE CONVERSION

395

10.1	Introduction	395
10.2	The Keyword Operator	398
10.3	Overloading Unary Operators	400
10.4	Operator Return Type	403
10.5	Constraint on Increment and Decrement Operators	404
10.6	Overloading Binary Operators	405
10.7	Overloading with <code>friend</code> Function	409
10.8	Overloading Assignment Operator (<code>=</code>)	412
10.9	Type Conversion	414
10.10	Rules for Overloading Operators	422
10.11	One-Argument Constructor and Operator Function	424
10.12	Overloading Stream Operators	425
10.13	More Programs	427
	<i>Summary</i>	438
	<i>Exercises</i>	439

11. INHERITANCE

441

11.1	Introduction	442
11.2	Reusability	442
11.3	Access Specifiers and Simple Inheritance	442
11.4	Protected Data with Private Inheritance	449
11.5	Types of Inheritance	454
11.6	Single Inheritance	456
11.7	Multilevel Inheritance	457
11.8	Multiple Inheritance	459
11.9	Hierarchical Inheritance	460
11.10	Hybrid Inheritance	462
11.11	Multipath Inheritance	465
11.12	Virtual Base Classes	466
11.13	Constructors, Destructors, and Inheritance	468
11.14	Object as a Class Member	483
11.15	Abstract Classes	489

- 11.16 Qualifier Classes and Inheritance 489
- 11.17 Constructors in Derived Class 490
- 11.18 Pointers and Inheritance 491
- 11.19 Overloading Member Function 492
- 11.20 Advantages of Inheritance 494
- 11.21 Disadvantages of Inheritance 494
- 11.22 More Programs 494
 - Summary* 505
 - Exercises* 506

12. ARRAYS

511

- 12.1 Introduction 511
- 12.2 One-dimensional Array Declaration and Initialization 511
- 12.3 Characteristics of Arrays 512
- 12.4 Accessing Array Elements Through Pointers 515
- 12.5 Arrays of Pointers 516
- 12.6 Passing Array Elements to a Function 517
- 12.7 Passing Complete Array Elements to a Function 518
- 12.8 Initialization of Arrays Using Functions 519
- 12.9 Two-dimensional Arrays 520
- 12.10 Pointers and Two-dimensional Arrays 523
- 12.11 Three- or Multi-dimensional Arrays 524
- 12.12 Arrays of Classes 525
 - Summary* 528
 - Exercises* 529

13. POINTERS

531

- 13.1 Introduction 531
- 13.2 Features of Pointers 532
- 13.3 Pointer Declaration 533
- 13.4 Arithmetic Operations with Pointers 536
- 13.5 Pointer to Pointer 538
- 13.6 void Pointers 539
- 13.7 wild Pointers 540
- 13.8 Pointer to Class 542
- 13.9 Pointer to Object 544
- 13.10 The this Pointer 546
- 13.11 Pointer to Derived Classes and Base Class 550
- 13.12 Pointer to Members 554
- 13.13 Accessing Private Members with Pointers 561
- 13.14 Direct Access to Private Members 562
- 13.15 Addresses of Objects and void Pointers 564
- 13.16 More Programs 565
 - Summary* 573
 - Exercises* 573

14. C++ AND MEMORY MODELS	577
14.1 Introduction	577
14.2 Memory Models	577
14.3 Dynamic Memory Allocation	581
14.4 The new and delete Operators	582
14.5 Heap Consumption	586
14.6 Overloading new and delete Operators	588
14.7 Overloading new and delete in Classes	592
14.8 Execution Sequence of Constructor and Destructor	595
14.9 Specifying Address of an Object	597
14.10 Dynamic Objects	598
14.11 Calling Convention	599
<i>Summary</i>	600
<i>Exercises</i>	601
15. BINDING, POLYMORPHISMS, AND VIRTUAL FUNCTIONS	603
15.1 Introduction	603
15.2 Binding in C++	604
15.3 Pointer to Base and Derived Class Objects	609
15.4 Virtual Functions	612
15.5 Rules for Virtual Functions	613
15.6 Array of Pointers	615
15.7 Pure Virtual Functions	618
15.8 Abstract Classes	619
15.9 Working of Virtual Functions	621
15.10 Virtual Functions in Derived Classes	627
15.11 Object Slicing	629
15.12 Constructors and Virtual Functions	631
15.13 Virtual Destructors	632
15.14 Destructors and Virtual Functions	634
<i>Summary</i>	635
<i>Exercises</i>	635
16. APPLICATIONS WITH FILES	639
16.1 Introduction	639
16.2 File Stream Classes	641
16.3 Steps of File Operations	643
16.4 Checking for Errors	650
16.5 Finding End of a File	653
16.6 File Opening Modes	655
16.7 File Pointers and Manipulators	657
16.8 Manipulators with Arguments	661
16.9 Sequential Access Files	664
16.10 Binary and ASCII Files	666

16.11	Random Access Operation	670
16.12	Error Handling Functions	674
16.13	Command-Line Arguments	679
16.14	Strstreams	680
16.15	Sending Output to Devices	682
16.16	More Programs	684
	<i>Summary</i>	691
	<i>Exercises</i>	692

17. GENERIC PROGRAMMING WITH TEMPLATES

697

17.1	Introduction	698
17.2	Need for Templates	698
17.3	Definition of Class Templates	698
17.4	Normal Function Templates	701
17.5	Working of Function Templates	704
17.6	Class Templates with More Parameters	704
17.7	Function Templates with More Arguments	705
17.8	Overloading of Template Functions	708
17.9	Member Function Templates	709
17.10	Recursion with Template Functions	710
17.11	Class Templates with Overloaded Operators	711
17.12	Class Templates Revisited	713
17.13	Class Templates and Inheritance	715
17.14	Bubble Sort Using Function Templates	717
17.15	Guidelines for Templates	718
17.16	Differences Between Templates and Macros	719
17.17	Linked Lists with Templates	720
17.18	More Programs	722
	<i>Summary</i>	727
	<i>Exercises</i>	728

18. WORKING WITH STRINGS

731

18.1	Introduction	731
18.2	Moving From C String to C++ String	734
18.3	Declaring and Initializing String Objects	735
18.4	Relational Operators	738
18.5	Handling String Objects	740
18.6	String Attributes	743
18.7	Accessing Elements of Strings	747
18.8	Comparing and Exchanging	750
18.9	Miscellaneous Functions	752
18.10	More Programs	754
	<i>Summary</i>	757
	<i>Exercises</i>	757

19. EXCEPTION HANDLING	759
19.1 Introduction	759
19.2 Principles of Exception Handling	760
19.3 The Keywords <code>try</code> , <code>throw</code> , and <code>catch</code>	760
19.4 Guidelines for Exception Handling	761
19.5 Multiple <code>catch</code> Statements	765
19.6 Catching Multiple Exceptions	767
19.7 Re-throwing Exception	768
19.8 Specifying Exceptions	769
19.9 Exceptions in Constructors and Destructors	771
19.10 Controlling Uncaught Exceptions	773
19.11 Exceptions and Operator Overloading	774
19.12 Exceptions and Inheritance	776
19.13 Class Templates with Exception Handling	777
19.14 Guidelines for Exception Handling	778
19.15 More Programs	779
<i>Summary</i>	781
<i>Exercises</i>	782
20. OVERVIEW OF STANDARD TEMPLATE LIBRARY	785
20.1 Introduction to STL	785
20.2 STL Programing Model	786
20.3 Containers	786
20.4 Sequence Containers	787
20.5 Associative Containers	789
20.6 Algorithms	789
20.7 Iterators	794
20.8 Vectors	795
20.9 Lists	801
20.10 Maps	808
20.11 Function Objects	811
<i>Summary</i>	814
<i>Exercises</i>	815
21. ADDITIONAL INFORMATION ABOUT ANSI AND TURBO-C++	817
21.1 Introduction	817
21.2 Innovative Data Types	818
21.3 New Type-casting Operators	821
21.4 The Keyword <code>explicit</code>	826
21.5 The Keyword <code>mutable</code>	827
21.6 Namespace Scope	828
21.7 Nested Namespaces	829
21.8 Anonymous Namespaces	830
21.9 The Keyword <code>using</code>	830

21.10	Namespace Alias	835
21.11	The Standard Namespace <code>std</code>	835
21.12	ANSI and Turbo-C++ Keywords	836
21.13	ANSI and Turbo-C++ Header Files	839
21.14	C++ Operator Keywords	840
	<i>Summary</i>	840
	<i>Exercises</i>	841

22. C++ GRAPHICS

843

22.1	Introduction	843
22.2	Computer Display Modes	844
22.3	Video Display and Display Adapters	844
22.4	Initialisation of Graphics	844
22.5	Few Additional Graphics Functions	846
22.6	Programs Using Library Functions	848
22.7	Working with Texts	856
22.8	Filling Patterns with Different Colors and Styles	858
22.9	Mouse Programming	862
22.10	Drawing Noncommon Figures	866
	<i>Summary</i>	867
	<i>Exercises</i>	867

Appendices

869

Index

879

Preface

Programming in C++ is meant for students pursuing various disciplines such as engineering, science, computer application, and diploma courses. Students who are learning object-oriented programming (OOP) can also refer to this book. It covers the subject of C++ as per the syllabi prescribed by various branches of Indian universities, state board of technical educations, and the Department of Electronics and Accreditation of Computer Courses. Hence, this book can be adopted in engineering, degree as well as other courses that deal with this subject. Written for beginners and for those who have some knowledge of C, the book will be especially useful for students who are learning object-oriented programming.

Designed to bridge the gap between theory and practice, the chapters have been presented using a systematic and lucid approach. Each topic is explained in an easy-to-understand manner with ample worked-out examples and programs. The book abounds with about 700 solved programs and as many as 300 exercises for the student's benefit. The programmer can run the solved programs, see the output and appreciate the concepts of C++.

The programs have been fully tested and executed with Borland's Turbo C++ compiler version 3.00 and Visual C++ 6.0 compiler. The programs in Chapter 22 have been tested and compiled with Java Compiler version 2.

This second edition has been thoroughly revised based on comments and responses received for the previous edition. All programming examples have been tested, compiled and executed. While utmost care has been taken at the time of writing this book, it is possible that errors and omissions may have inadvertently crept in. Such incongruities, if any, may please be pointed out. Suggestions and feedback are most welcome and may be directed to my e-mail address: ankamthane@gmail.com.

ACKNOWLEDGEMENTS

I express my gratitude to the Minister for Technical and Medical Education of Maharashtra State, Honorable Shri D. P. Sawant, who complimented and felicitated me for writing technical books, while he was invited to our institute recently in the month of May 2013. He also encouraged me to write more books.

I am grateful to Prof. B. M. Naik, Former Principal of S.G.G.S. Institute of Engineering and Technology, Nanded, who has always acted as a source of encouragement. His dynamism and leadership led me to write this book and I will never forget his support and words of wisdom.

Special thanks are due to the members of the board of governors of S.G.G.S. Institute of Engineering and Technology, who motivated me to write this book. I thank eminent industrialist Shri B. N. Kalyani, head of the Kalyani Group of Companies, for inspiring and complimenting me for authoring technical books. I am also obliged to Dr L. M. Waghmare, Director of our institute, for supporting and appreciating me while writing this book.

I am indebted to all my colleagues, friends and students who helped me at the time of preparing the manuscript for this book. I acknowledge the support rendered by Dr S. V. Bonde, Dr P. D. Jadhav, Dr P. Pramanik, Dr P. S. Charpe, Dr B. M. Dabde, Dr Y. V. Joshi, Dr U. V. Kulkarni, Dr S. P. Kallurkar, Dr V. M. Nandedkar, Dr A. U. Digraskar, Dr R. S. Holambe, Dr B. M. Patre, Prof. R. K. Chavan, Dr J. V. L. Venkatesh, Prof. P. S. Nalawade, S. S. Hatkar, Dr R. C. Thool, Dr V. R. Thool, Dr D. D. Doye, Milind Bhalerao, Dr A. B. Gonde, Dr A. Chakraborti, Dr A. V. Nandedkar, Dr P. B. Londhe, Dr R. R. Manthalkar, Dr S. S. Gajre, Prof. N. G. Megde, Dr A. S. Sontakke, Dr R. H. Chille, Dr L. M. Waikar and A. I. Tamboli of S.G.G.S. Institute of Engineering and Technology.

I thank my friends Principal Prof. S. L. Kotgire and Prof. Balaji Bacchewar for their words of encouragement. M. M. Jahagirdar, L. M. Buddhewar, K. M. Buddhewar, D. V. Deshpande, S. R. Kokne, M. G. Damkondawar, Yeramwar, Pampatwar, D. R. Yerawar, S. R. Tumma, and S. R. Mana also helped me to a great extent while writing this book.

I thank Pearson Education for publishing this book. In particular, I thank Ms V. Pavithra and Ms Neha Goomer for their help in seeing this book through production. In addition, I also thank Thomas Rajesh for his constant encouragement.

I appreciate the help and support provided by all the students, faculty, and non-teaching staff of this institute as well as that of other friends who have directly or indirectly added value to this book.

Thanks are also due to my wife, Surekha, who persistently supported me at all times. I am indebted to my sons Amol and Amit, daughter Sangita, as also my daughter-in-law Swarupa, who all played a key role in bringing this book to fruition. Swarupa was of immense help in drawing the figures in several chapters. I thank all of them.

Ashok Namdev Kamthane

About the Author

Ashok Namdev Kamthane obtained his M.E. (Electronics) degree from S.G.G.S. Institute of Engineering and Technology, Nanded. A meritorious student throughout his career, he has bagged a number of prizes including cash and medals for his distinct work in the academics. For his M. E. (Electronics) dissertation, he worked at the Bhabha Atomic Research Center, Mumbai.

Associated with the teaching profession for the past 30 years, Kamthane was instrumental in the development of hardware and software using the 8051 (8-bit microcontroller) on Acoustic Transceiver System required in submarines. He has also worked at Melton as an executive. Currently Associate Professor at the Department of Electronics and Telecommunication Engineering in S.G.G.S Institute of Engineering and Technology, he has guided a number of undergraduate and postgraduate students in their projects and published a number of technical papers at both national and international conferences.

This page is intentionally left blank.

Introduction to C++

1

CHAPTER

CHAPTER OUTLINE

- 1.1 Differences between C and C++
- 1.2 Evolution of C++
- 1.3 The ANSI Standard
- 1.4 The Object Oriented Technology
- 1.5 Disadvantage of Conventional Programming
- 1.6 Programming Paradigms
- 1.7 Preface to Object Oriented Programming
- 1.8 Key Concept of Object Oriented Programming
- 1.9 Advantages of OOP
- 1.10 Object Oriented Languages
- 1.11 Usage of OOP
- 1.12 Usage of C++

1.1 DIFFERENCES BETWEEN C AND C++

Some differences between C and C++ are as follows:

- (1) C is a procedure/function-oriented language and C++ language is driven by a procedure/object.
- (2) Data is not protected in C, whereas data is secured in C++. Data hiding concept is absent in C.
- (3) C uses a top down approach while C++ uses a bottom up approach. The program is prepared step by step in C, and in C++ base elements are prepared first.
- (4) In C, we cannot give the same name to two functions in a program, whereas due to the function overloading feature, the above concept is possible in C++. One can initialize a number of functions with the same name, but with different arguments. The polymorphism feature is built in C++, which supports this concept.

- (5) C uses `printf()` and `scanf()` functions to write and read the data respectively, while C++ uses `cout` and `cin` objects for output and input operations, respectively. Further, the `cout` uses `<<` (insertion operator) and `cin` uses `>>` (extraction operator).
- (6) C uses `stdio.h` file for input and output functions, whereas C++ uses `iostream.h` for these functions.
- (7) Constructor and destructors are absent in C and the same are provided in C++.
- (8) Inline functions are supported by C++, and the same are absent in C. Inline functions can be used as macros. They are stated by a word 'inline'.

1.2 EVOLUTION OF C++

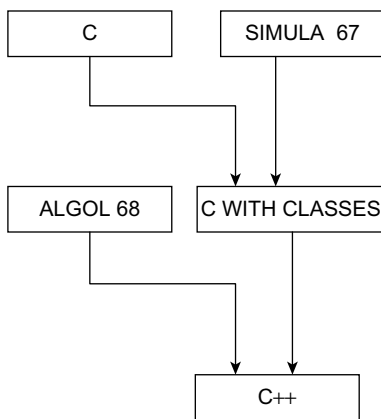


Fig. 1.1 Evolution of C++

C++ is an object oriented programming language and also considered as an extension of C. Bjarne Stroustrup at AT&T Bell Laboratories in Murray Hill, New Jersey (USA) developed this language in the early 1980s. Stroustrup, a master of *SIMULA67* and C, wanted to combine the features of both the languages and he developed a powerful language that supports object-oriented programming with features of C. The outcome was C++ as per Fig. 1.1. Various features were derived from *SIMULA67* and *ALGOL68*. Stroustrup called the new language '*C with classes*'. However, in 1983, the name was changed to C++.

The thought of C++ came from the C increment operator ++. Rick Mascitti coined the term C++ in 1983. Therefore, C++ is an extension of C. C++ is a superset of C. All the concepts of C are applicable to C++ also.

For developing complicated applications, object oriented language such as C++ is the most convenient and easy.

Hence, a programmer must be aware of its features.

1.3 THE ANSI STANDARD

The ANSI stands for American National Standards Institute. This Institute was founded in **1918**. The main goal for establishing this Institute was to suggest, reform, recommend, and publish standards for data processing in the USA. This committee sets up the standard in the computer industry.

The recognized council working under the procedure of the American National Standards Institute (ANSI) has made an international standard for C++. The C++ standard is also referred to as ISO (International Standards Organization) standard. The process of standardization is gradual and the first draft of the planned ANSI standard was made on **25 January 1994**. This book will continue to refer ANSI standard code, which is more commonly used. The ANSI standard is an attempt to ensure that the C++ is portable.

1.4 THE OBJECT ORIENTED TECHNOLOGY

Nature is composed of various objects. Living beings can be categorized into different objects.

Let us consider an example of a teaching institute which has two different working sections – teaching and non-teaching. Further sub-grouping of teaching and non-teaching can be made for

the coordination of management. The various departments of any organization can be thought of as objects working for certain goals and objectives.

Usually an institute has faculty of different departments. The Director/Principal is a must for the overall management of the institute. The Academic Dean is responsible for the academics of the institute. The Dean for Planning should have the future plans of the institute and he/she must see how the infrastructure is utilized effectively. The Dean R&D should see research activities run in the institute forever.

Besides teaching staff there must be laboratory staff for assistance in conducting practical sessions, and a site development section for beautification of the campus. The accounts department is also required for handling monetary transactions and salaries of the employees. The Sports section is entrusted the responsibility of sports activities. The Registrar for Administration and staff for dealing with administrative matters of the institute are also required. Each department has an in-charge who carries clear-cut given responsibilities. Every department has its own work as stated above. When an institute's work is distributed into departments as shown in Fig. 1.1, it is comfortable to accomplish goals and objectives. The activities are carried on smoothly. The burden of one particular department has to be shared among different departments with personnel. The staff in the department is controlled properly and act according to the instructions laid down by the management. The faculty performs activities related to teaching. If the higher authority needs to know the details regarding the theory, practical, seminar and project loads of individuals of the department, then a person from the department furnishes the same to the higher authority. This way some responsible person from the department accesses the data and provides the higher authority with the requisite information. It is also good to think that no unconnected person from another department reads the data or attempts to make any alteration that might corrupt the data.

As shown in Fig. 1.2, an institute is divided into different departments such as library, classroom, computer laboratory, etc. Each department performs its own activities in association with the other departments. Each department may be considered as a module and it contains class and object in C++ language. This theory of class and object can be extended to every walk of life and can be implemented with software. In general, objects are in terms of entities.

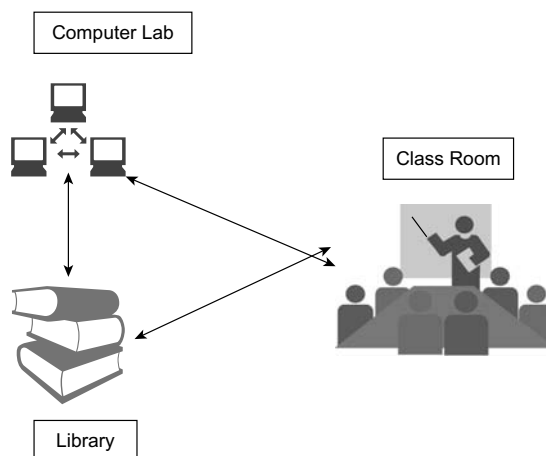


Fig. 1.2 Relationship between different sections

In a nutshell, in object oriented programming objects of a program interact by sending messages to each other.

1.5 DISADVANTAGE OF CONVENTIONAL PROGRAMMING

Traditional programming languages such as COBOL, FORTRAN, C etc. are commonly known as procedure oriented languages. The program written in these languages consists of a sequence of instructions that tells the compiler or interpreter to perform a given task. Numerous functions are initiated by the user to perform a task. When a program code is large, it becomes inconvenient to manage it. To overcome this problem, procedures or subroutines were adopted to make a program more understandable to the programmers. A program is divided into many functions.

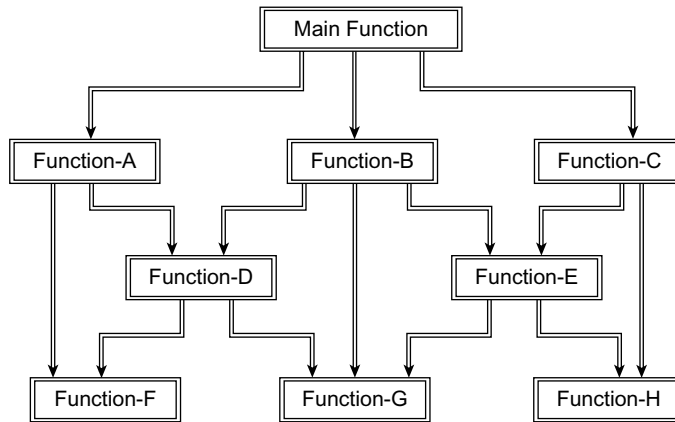


Fig. 1.3 Flow of functions in non-OOP languages

Each function can call another function, as shown in Fig. 1.3. Each function has its own task. If the program is too large the function also creates problems. In many programs, important data variables are declared as global. In case of programs containing several functions, every function can access the global data as per the simulation in Fig. 1.4. In huge programs it is difficult to know what data is used by which function. Due to this the program may contain several logical errors.

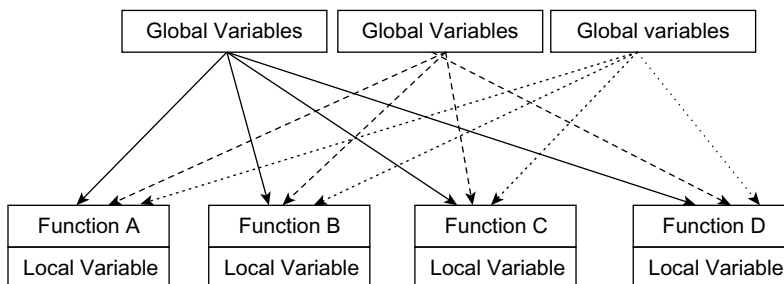


Fig. 1.4 Sharing of data by functions in non-OOP languages

The following are the drawbacks observed in monolithic, procedure, and structured programming languages:

- (1) Huge programs are divided into smaller programs known as functions. These functions can call one another. Hence security is not provided.
- (2) No importance is given to security of data and importance is laid on doing things.
- (3) Data passes globally from function to function.
- (4) Most function accesses global data.

1.6 PROGRAMMING PARADIGMS

(1) Monolithic Programming:

- (A) In these types of programming languages, the program is written with a single function. A program is not divided into parts; hence it is named as monolithic programming. It is also called single thread execution.
- (B) When the program size increases it leads to difficulty.
- (C) In monolithic programming languages such as basic and assembly language, the data variables declared are global and the statements are written in sequence.
- (D) The program contains jump statements such as `goto` that transfers control to any statement as specified in it. Fig. 1.5 shows a program of monolithic type. The global data can be accessed from any portion of the program. Due to this reason the data is not fully protected.
- (E) The concept of sub-programs does not exist, and hence is useful for small programs.

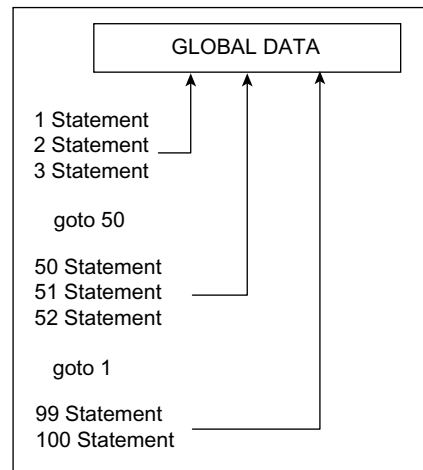


Fig. 1.5 Program in monolithic programming

(2) Procedural/Structured Programming

- (A) Sometimes known as modular programming.
- (B) Programs written are more efficient and easier to understand and modify.
- (C) The procedural/structured programming languages are similar to solving a problem by human. In a nut shell, humans attempt a problem by adopting a sequence of operations.
- (D) It makes use of a top-down design model in which a program developer maps out the overall program structure into separate subsections.
- (E) Large size programs can be developed in structured programming such as Pascal and C. Programs are divided in multiple sub-modules.
- (F) Procedural/Structured programming languages such as FORTRAN, BASIC, ALGOL, COBOL, C, etc., are divided into a number of segments called as subprograms. There is a main function and it invokes subprograms. Thus, it focuses on functions apart from data. Figure 1.6 describes a program of procedural/structured type. It shows different sub-programs accessing the same global data. Here also the programmer can observe the lack of secrecy.

- (G) The control of program can be transferred using unsafe `goto` statement.
- (H) This type of programming language uses different control structures that are as follows.
 - Decision/selection control statements
 - Iteration control statements
 - Jump control statements
- (I) Data are global and all the sub-programs share the same data, i.e. data are globally accessible to all functions. Thus, any function operates on the global data and this directs to loosing some vital information. We can conclude here that a module represents a function.
- (J) Procedural structured/programming languages permit data transfer through messages by means of functions.
- (K) Least importance is given to the data in procedural/structured programming languages.
- (L) These languages are used for developing medium-sized software applications.
- (M) Compiler and interpreter construction are easy and simple with this type of programming language. Furthermore, these compilers and interpreters need low memory to run on the computers.
- (N) It is difficult to implement simultaneous processes/parallelization.

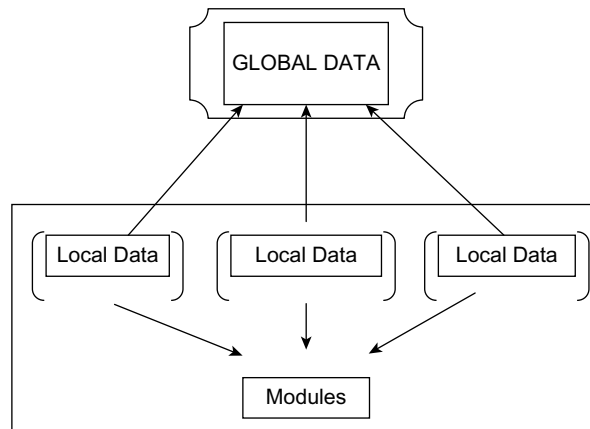


Fig. 1.6 Program in procedural/structured programming

1.7 PREFACE TO OBJECT ORIENTED PROGRAMMING

The prime factor in the design of object oriented programming approach is to rectify some of the faults observed in the procedure oriented languages. OOP acts with data as a critical component in program development. It does not allow the data to flow freely around the systems. It ties data to the functions that operate on it and prevents it from accidental change due to external functions. OOP permits us to analyze a problem into a number of items called objects and then assembles data and functions around these items as per Fig. 1.7. Following are the impressive characteristics of object-oriented programming:

- (1) OOP pays more importance to data rather than function.
- (2) Programs are divided into classes and their member functions.
- (3) OOP follows a bottom-up approach.
- (4) New data items and functions can be comfortably added whenever essential.
- (5) Data is private and prevented from accessing external functions.
- (6) Objects communicate with each other through functions.

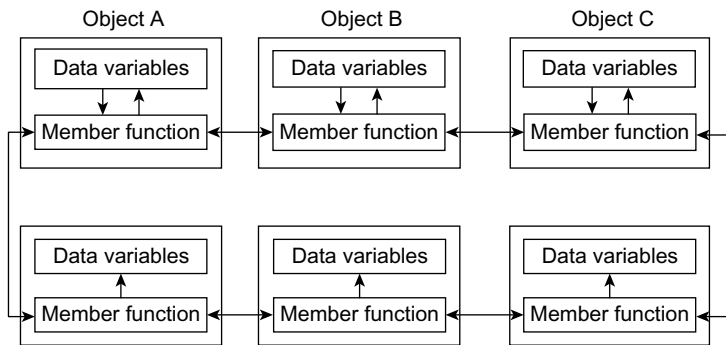


Fig. 1.7 Relation between data and member function in OOP

1.8 KEY CONCEPTS OF OBJECT ORIENTED PROGRAMMING

Object oriented programming language is a feature that allows a mode of modularizing programs by forming any area for data as well as functions that is used as object for making copies of modules as per requirement.

There are several fundamental concepts in object oriented programming. They are shown in Fig. 1.8 and are discussed as follows.

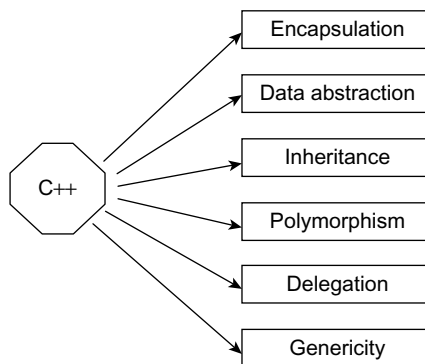


Fig. 1.8 Concepts/elements of object oriented paradigm

(1) Objects

Objects are primary run-time entities in object oriented programming.

Objects are primary run-time entities in object-oriented programming. They may stand for a thing that makes sense in a specific application. Some examples are a spot, a person, any data item related to the program including user-defined data types. Programming issues are analyzed in terms of object and the type of transmission between them. Program objects must be selected like the items equivalent to actual world objects. Objects occupy space in memory. Every object has its own properties or features that illustrate what the object can do. An object is a specimen of a class. It can be singly recognized by its name. It declares the state that is shown by the data values of its characteristic at a specific time. The state of the object varies according to the procedure used with it. It is called as the action of the object. The action of the object depends upon the member function defined within its class.

Fig. 1.9 (a) shows some of the objects that we use in our daily life.

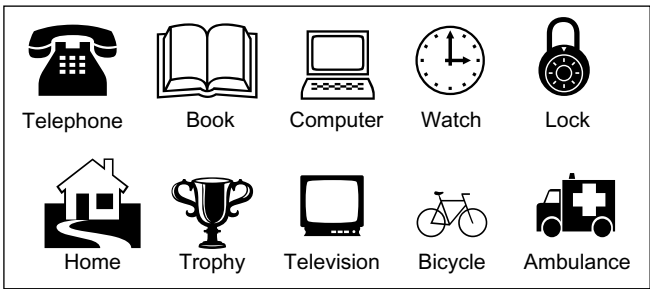


Fig. 1.9 (a) Commonly available objects

Object: City
Data: Name_of_city Population Area
Functions: Average age Literacy_rate Display

Fig. 1.9 (b) Representation of an object

Objects communicate with each other by sending messages. An object can be represented by Fig 1.9 (b).

The name of the above object is city. Its data members are name_of_city, population, and area. The various functions associated with the city are average age, literacy_rate, and display.

(2) Classes

A class is grouping of objects that have the identical properties, common behavior, and shared relationship. A class binds the data and its related functions together.

A class is the accomplishment of abstract data type. It defines the nature and methods that act on the data structure and abstract data type, respectively. Specimens are also called as objects. In other words, a class is a grouping of objects that have identical properties, common behavior, and shared relationship. For example, Tata’s Swift, Maruti’s Alto etc. are the members of a class car.

The entire group of data and code of an object can be built as a user-defined data type using class. Objects are nothing but variables of type class. Once a class has been declared, the programmer can create a number of objects associated with that class. The syntax used to create

an object is similar to the syntax used to create an integer variable in C. A class is a model and not a true specimen of the object. Every object has its own value for each of its member variables. However, it shares the property names or operations with other instances of the class. Thus, classes define the characteristic and action of objects.

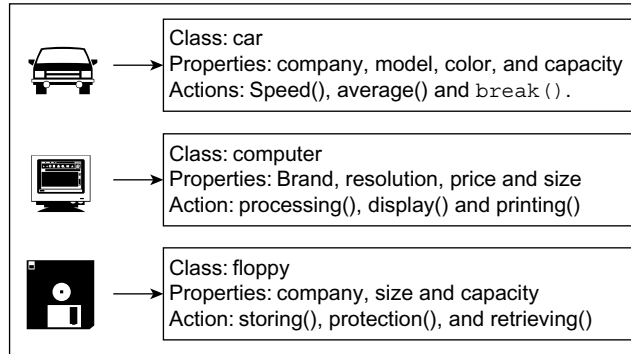


Fig. 1.10 Objects and their properties

In Fig. 1.10 commonly useful classes are described with their properties and action they perform. For example, a car has properties like company, model and color with which a car can be identified among a number of cars of other properties, and the actions like speed () and average () describes its working.

```

Class home
{
    Telephone
    Book
    Computer
    Watch
    Lock
}
Class vehicle

```

```

{     }

```

```

Class entertainment

```

```

{     }

```

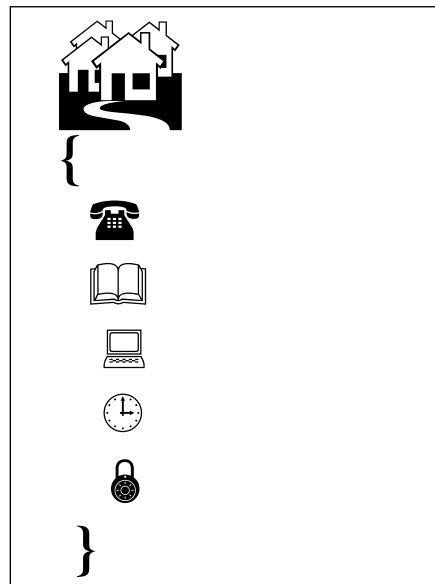


Fig. 1.11 Classes and their members

Figure 1.11 describes the classes and related data that come under the different classes.

(3) Method

An operation required for an object or entity when coded in a class is called a method.

An operation required for an object or entity when coded in a class is called a method. The operations that are required for an object are to be defined in the class. All objects in a class carry out certain common actions or operations. Each action needs an object that becomes a function in the class that defines it and is referred to as a method.

In Fig. 1.12 the class and its associated data members and functions are shown in different styles. It is a frequent style of writing a class in the program.

The `class A` contains private data members and public methods or member functions. Usually, the data members are declared private and methods or member functions are declared as public and they are available outside the class. The data member of any class uses its member functions or methods to perform operations.

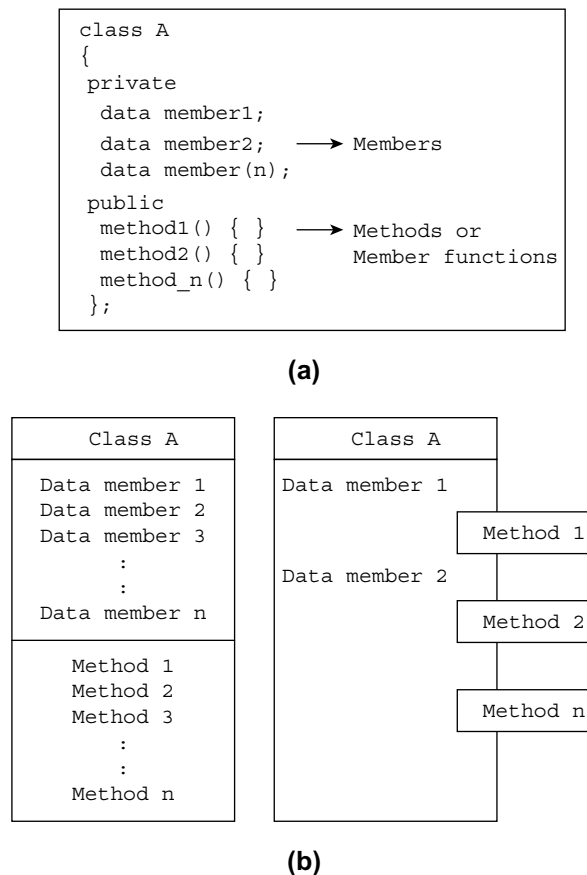


Fig. 1.12 (a) and (b) Representation of methods in different manners

(4) Data Abstraction

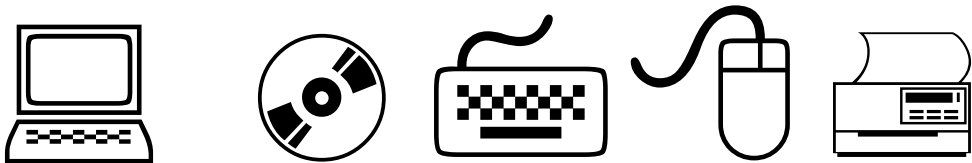
Abstraction refers to the procedure of representing essential features without including the background details.

Abstraction refers to the procedure of representing essential features without including the background details. Classes use the theory of abstraction and are defined as a list of abstract properties such as size, cost, height, and few functions to operate on these properties. Data abstraction is the procedure of identifying properties and method related to a specific entity as applicable to the application.

A powerful method to achieve abstraction is through the manipulation of hierarchical classifications. It permits us the breaking of semantics of multiple systems into layers by separating them into multiple controllable parts. For example, a computer as shown in Fig. 1.13 is made of various parts such as CPU, keyboard, and so on. We think it as a single unit, but the single unit has several sub-units. They together do the single task. By assembling sub-parts we can build a single system.

Hierarchical abstraction of complicated systems can also be used in computer software. The data from conventional procedure oriented programs can be converted by abstraction mechanism into its partial objects. A series of operation steps may develop a set of messages between these objects. Each object shows its own attributes.

Data abstraction is used to define a data type available in the programming language, called as abstract data type (ADT). It consists of a set of values and a set of operations.



(a) Computer as single unit

(b) Different components of computer

Fig 1.13 Computer and its parts

(5) Encapsulation

The packing of data and functions into a single component is known as encapsulation.

C++ supports the features of encapsulation using classes. The packing of data and functions into a single component is known as encapsulation. The data is not reachable by the outside functions. Only those functions that are able to access the data are defined within the class. These functions prepare the interface between the object's data and the program. With encapsulation we can accomplish data hiding. Data hiding is an important feature using which an object can be used without the user knowing how it works internally.

In C++ the fundamental of encapsulation is class. A class defines the structure of data and member functions. It is common to all its objects. Class is a logical structure whereas object is

a physical actuality. The goal of the class is to encapsulate complication. The class also has a mechanism for hiding the data. Each member in the class may be private or public. Any non-member function cannot access the data of the class. The public section of the class must be mindfully coded not to expose the inner details of the class. Figure 1.14 explains sections of encapsulation.

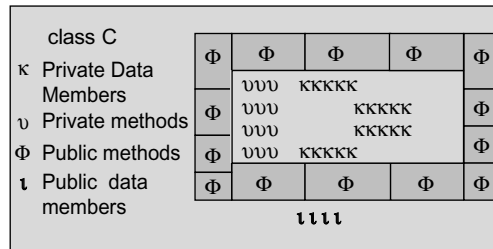


Fig. 1.14 Encapsulation: Private and public sections

(6) Inheritance

Inheritance is the method by which objects of one class get the properties of objects of another class.

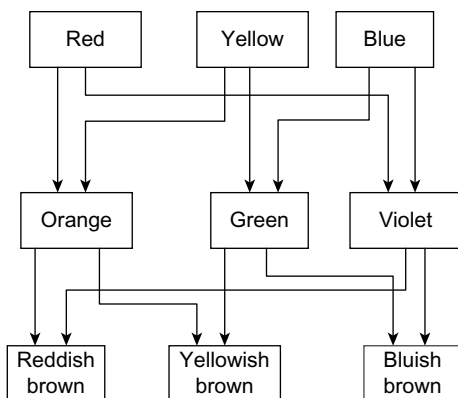


Fig. 1.15 Inheritance

Inheritance is the method by which objects of one class get the properties of objects of another class. In object oriented programming inheritance provides the thought of reusability. The programmer can add new properties to the existing class without changing it. This can be achieved by deriving a new class from the existing one. The new class will possess features of both the classes. The actual power of the inheritance is that it permits the programmer to reuse a class that is nearly, but not precisely, what he wants, and to tailor the class in such a manner that it does not bring any unwanted incidental result into the rest of the class. Thus, inheritance is the feature that permits the reuse of an existing class to make a new class. Figure 1.15 shows an example of inheritance.

In Fig. 1.15, red, yellow and blue are the main colors. The orange is created from the combination of red and yellow, green is created from yellow and blue and violet is created from red and blue. The orange color has attributes of both red and yellow, which produces a new effect. Thus, many combinations are possible.

(7) Polymorphism

Polymorphism allows the same function to act in a different way in different classes.

Polymorphism makes it possible for the same functions to act differently on different classes as shown in Fig. 1.16. It is an important feature of OOP concept. It holds an ability to take more than one form. Polymorphism accomplishes an important part in allowing objects of different classes to share the same external interface. It is possible to code a non-specific (generi(c) interface to a set of associated actions.

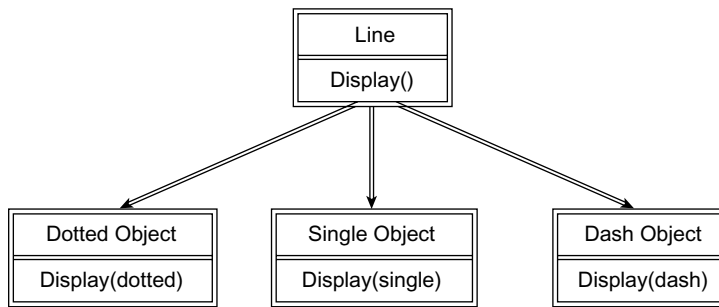


Fig. 1.16 Polymorphism in OOP

(8) Dynamic Binding

Binding means connecting one program to another program that is to be executed in reply to the call.

Binding means connecting one program to another program that is to be executed in reply to the call. Dynamic binding is also known as *late binding*. The code present in the specified program is unknown till it is executed. It is analogous polymorphism.

In Fig. 1.16 polymorphism allows the single object to invoke similar function from different classes. The program action is different in all the classes. At execution time, the code analogous to the object under the present reference will be executed. The reader is advised to refer polymorphism chapter for more details.

(9) Message passing

An object oriented programming includes objects. The objects communicate with one another. The programming with these objects should be followed with following steps.

- (1) Declaring classes that define objects and their actions.
- (2) Declaring objects from classes.
- (3) Implementing relation between objects.

Data is transferred from one object to another object. A message for an object is the demand for implementation of the process. Message passing consists of indicating the name of the object, function, and required data elements (Fig. 1.17). Objects can be created, released, and interacted with each other. An object is workable, as long as it is active. In object oriented programming there is a panorama of independent objects that communicate with each other by swapping messages. Objects invoke member functions. They also negate if the calling object is not a member of

the same class. Thus a message is a solicitation to an object to call one of its member functions. A message contains name of the member function and parameters of the function. Execution of member function is just a response generated due to receipt of a message. It is possible when the function and the object are of the same class.

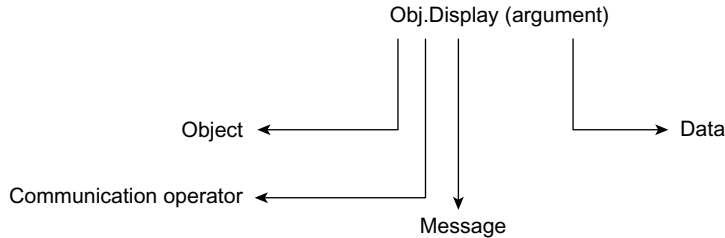


Fig. 1.17 Message Passing

(10) Reusability

Object oriented technology allows reusability of the classes by extending them to other classes using inheritance.

Object oriented technology allows reusability of the classes by extending them to other classes using inheritance. Once a class is defined, the other programmer can also use it in their programs. The programmer can also add new feature to the derived classes. The verified and checked qualities of base classes need not to be redefined. Thus, the reusability saves the time. In Fig. 1.18, class A is reused and class B is created. Again class B is reused and class C is created.

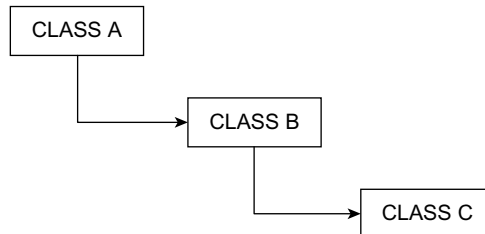


Fig. 1.18 Reusability

(11) Delegation

In OOP, two classes can be joined in two ways: (a) Inheritance, (b) Delegation. Both these ways provide reusability of the class.

In Inheritance one class can be derived from the other class. The relationship between these two classes is called as *kind of relationship*. For example if *class Y* is derived from *class X*, then *class Y* is known as kind of *X*. Figure 1.19 (a) explains this point.

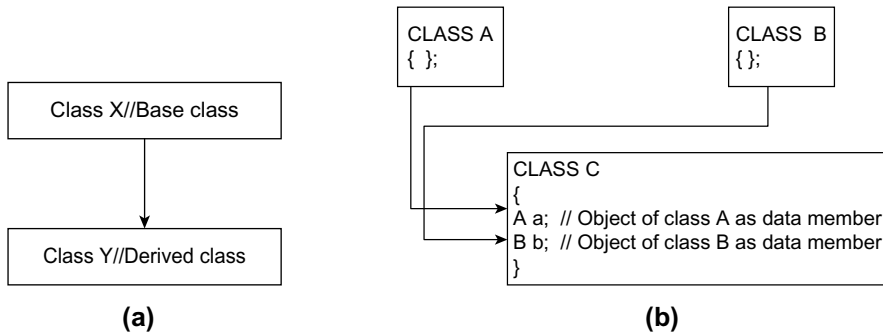


Fig. 1.19 Relationships between two classes: (a) Kind of relationship (Inheritance) (b) has a relationship (Delegation)

The second type of relationship is **has a relationship**. When object of one class is used as data member in the other class, such composition of objects is known as delegation. As shown in Fig. 1.19 (b) class C has two data members. These two data members are objects of class A and B, such relationship between the classes is known as **has a relationship**.

(12) Genericity

The software components of a program have more than one version depending on the data types of arguments. This feature allows declaration of variables without specifying exact data type. The compiler identifies the data type at run time. The programmer can create a function that can be used for any type of data. The template feature in C++ allows generic programming.

1.9 ADVANTAGES OF OOP

Object oriented technology provides many advantages to the programmer and the user. This technology solves many problems related to software development, provides improved quality and low cost software.

- (1) Object oriented programs can be comfortably upgraded.
- (2) Using inheritance, we can eliminate redundant program code and continue the use of previously defined classes.
- (3) The technology of data hiding facilitates the programmer to design and develop safe programs that do not disturb code in other parts of the program.
- (4) The encapsulation feature provided by OOP languages allows programmer to define the class with many functions and characteristics and only few functions are exposed to the user.
- (5) All object oriented programming languages allows creating extended and reusable parts of programs.
- (6) Object oriented programming changes the way of thinking of a programmer. This results in rapid development of new software in a short time.
- (7) Objects communicate with each other and pass messages.

1.10 OBJECT ORIENTED LANGUAGES

There are many languages which support object oriented programming. Tables 1.1 and 1.2 describe the OOP languages and features supported by them.

Table 1.1 Properties of pure OOP and object based languages

	Pure Object Oriented Languages					Object Based Languages
Properties	Java	Simula	Smalltalk	Eiffel	Java	Ada
Encapsulation	✓	✓	✓	✓	✓	✓
Inheritance	✓	✓	✓	✓	✓	No
Multiple inheritance	✗	✗	✓	✓	✗	No
Polymorphism	✓	✓	✓	✓	✓	✓
Binding (Early and late)	Both	Both	Late binding	Early binding	Both	Early binding
Genericity	✗	✗	✗	✓	✗	✓
Class libraries	✓	✓	✓	✓	✓	Few
Garbage collection	✓	✓	✓	✓	✓	✗
Persistence	✓	✗	Promised	Less	✓	Same as 3GL
Concurrency	✓	✓	Less	Promised	✓	Hard

Table 1.2 Properties of extended traditional languages

	Extended traditional languages				
Properties	Objective C	C++	Charm ++	Objective Pascal	Turbo Pascal
Encapsulation	✓	✓	✓	✓	✓
Inheritance	✓	✓	✓	✓	✓
Multiple inheritance	✓	✓	✓	---	---
Polymorphism	✓	✓	✓	✓	✓
Binding (Early and late)	Both	Both	Both	Late	Early
Genericity	✗	✓	✓	✗	✗
Class libraries	✓	✓	✓	✓	✓
Garbage collection	✓	✗	✗	✓	✓
Persistence	✗	✗	✗	✗	✗
Concurrency	Poor	Poor	✓	✗	✗

The following are the object-oriented languages, which are widely accepted by the programmer.

- C++
- Smalltalk
- Charm ++
- Java

SMALLTALK

Smalltalk is a pure object oriented language. C++ makes few compromises to ensure quick performance and small code size. Smalltalk uses run-time binding. Smalltalk programs are considered to be faster than the C++. Smalltalk needs longer time to learn than C++. Smalltalk programs are written using Smalltalk browser. Smalltalk uses dynamic objects and memory is allocated from free store. It also provides automatic garbage collection and memory is released when object is no longer in use.

CHARM++

Charm ++ is also an object oriented programming language. It is a portable. The language provides features such as inheritance, strict type checking, overloading, and reusability. It is designed in order to work efficiently with different parallel systems together with shared memory systems, and networking.

JAVA

Java was developed by Patrick Naughton, James Gosling, Chris Warth, Mike Sheridan and Ed Frank at Sun Microsystems. Java is an object oriented programming language and supports maximum OOP characteristics exist. Its statement structure is like C and C++, but it easier than C++. Java excludes few confusing and unsafe features of C and C++ like pointers.

Java allows client/server programming. Java is used for Internet programming. The Java programs are downloaded by the client machines and executed on different types of hardware. This portability is achieved by translation of Java program to machine code using compiler and interpreter.

The Java compiler converts the source program to JVM (Java virtual machine). The JVM is a dummy CPU. The compiler Java program is called as byte code. The Java interpreter translates byte code into the object code. Compiler and interpreter do the conversion of Java program to object code.

1.11 USAGE OF OOP

Object oriented technology is changing the style of software engineers to think, analyze, plan, and implement the software. The software developed using OOP technology is more efficient and easy to update. OOP languages have standard class library. The users in their program can reuse the standard class library. Thus, it saves lot of time and coding work.

The most popular application of object oriented programming is windows. There are sev-

eral windowing software based on OOP technology. Following are the areas for which OOP is considered.

- (1) Simulation
- (2) Object oriented DBMS
- (3) Office automation software
- (4) Artificial Intelligence and expert systems
- (5) CAD/CAM software
- (6) Network programming & Internet applications
- (7) System software

1.12 USAGE OF C++

C++ is a flexible language. Lengthy programs can be easily controlled by the use of C++. It permits us to create hierarchy -associated objects and libraries that can be useful to other programmers. C++ helps the programmer to write bug-free program, which are easy to maintain.

SUMMARY

- (1) C++ is an object oriented programming language invented by Bjarne Stroustrup at AT&T Bell Laboratories.
- (2) The recognized council working under the procedure of the American National Standard Institute (ANSI) has made an international standard for C++.
- (3) The disadvantage in conventional programming language is that the program written in these languages consists of a sequence of instructions that tells the compiler or interpreter to perform a given task. When program code is large then it becomes inconvenient to manage.
- (4) The prime factor in the design of object oriented programming approach is that to get back some of the faults found in the procedure oriented languages.
- (5) In monolithic programming languages such as basic and assembly language, the data variables declared are global and the statements are written in sequence.
- (6) In the procedural programming languages such as FORTRAN and COBOL, programs are divided into number of segments called as subprograms. Thus it focuses on functions apart from data.
- (7) Larger programs are developed in structured programming such as Pascal and C. Programs are divided in multiple sub modules and procedures.
- (8) OOP acts with data as a critical component in the program development and does not let the data to flow freely around the systems.
- (9) Object: Objects are primary run-time entities in an object-oriented programming. They may stand for a thing that makes sense in a specific application.
- (10) Class: A class is grouping of objects that have the identical properties, common behavior, and shared relationship. The entire group of data and code of an object can be built as a user-defined data type using class.
- (11) Method: An operation required of an object or entity when coded in a class is called a method.
- (12) Data Abstraction: Abstraction directs to the procedure of representing essential features without including the background details.
- (13) Encapsulation: C++ supports the features of encapsulation using classes. The packing of data and functions into a single component is known as encapsulations.
- (14) Inheritance: Inheritance is the method by

- which objects of one class get the properties of objects of another class. In object oriented programming inheritance provides the thought of reusability.
- (15) Polymorphism: Polymorphism makes possible the same functions to act differently on different classes. It is an important feature of OOP concept.
 - (16) Object oriented technology allows reusability of the classes by extending them to other classes using inheritance.
 - (17) The languages C++, Smalltalk, Eiffel and Java are widely used OOP languages.
 - (18) Object oriented technology is changing the style of software engineers to think, analyze, plan and implements the software. The software developed using OOP technology is more efficient and easy to update.
 - (19) C++ is a flexible language. Lengthy programs can be easily controlled by the use of C++.

EXERCISES

(A) Answer the following questions.

- (1) Compare C and C++.
- (2) What is object oriented programming?
- (3) Explain the key concepts of OOP.
- (4) What is ANSI standard?
- (5) What are the disadvantages of conventional programming languages?
- (6) Explain the characteristics of monolithic programming languages.
- (7) Compare structured and procedural languages.
- (8) List the disadvantages of procedural programming languages.
- (9) Explain evolution of C++.
- (10) List the names of popular OOP languages.
- (11) List the unique features of an OOP paradigm.
- (12) What is an object and class?
- (13) How is data secured in OOP languages?
- (14) Compare and contrast OOP languages with procedure oriented languages.
- (15) Mention the types of relationships between two classes.
- (16) What is structured oriented programming? Discuss its pros and cons.
- (17) How is global data shared in procedural programming?
- (18) Describe any two object oriented programming languages.
- (19) What are the differences between Java and C++?
- (20) Mention the advantages of OOP languages?
- (21) What do you mean by message passing?
- (22) Distinguish between inheritance and delegation.
- (23) Can we define member functions in private section?
- (24) Can we declare data member in the public section?

(B) Answer the following by selecting the appropriate option.

- (1) Data hiding concept is supported by language
 - (a) C
 - (b) Basic
 - (c) Fortran
 - (d) C++
- (2) Function overloading means
 - (a) different functions with different names
 - (b) function names are same but same number of arguments
 - (c) function names are same but different number of arguments
 - (d) none of the above
- (3) C++ language was invented by
 - (a) Bjarne Stroustrup
 - (b) Dennis Ritchie
 - (c) Ken Thompson
 - (d) none of the above
- (4) The languages COBOL and BASIC are commonly known as

- (a) **procedure oriented languages**
 - (b) object oriented languages
 - (c) low level languages
 - (d) none of the above
- (5) A program with only one function is observed in
- (a) **monolithic programming languages.**
 - (b) object oriented languages
 - (c) structured programming languages
 - (d) none of the above
- (6) The packing of data and functions into a single component is known as
- (a) **encapsulation**
 - (b) polymorphism
 - (c) abstraction
 - (d) none of the above
- (7) The method by which objects of one class get the properties of objects of another class is known as
- (a) **inheritance**
 - (b) encapsulations
 - (c) abstraction
 - (d) none of the above
- (8) The mechanism that allows same functions to act differently on different classes is known as
- (a) **polymorphism**
 - (b) encapsulations
 - (c) inheritance
 - (d) none of the above
- (9) The existing class can be reused by
- (a) **inheritance**
 - (b) polymorphism
 - (c) dynamic binding
 - (d) abstraction
- (10) Composition of objects in a class is known as
- (a) **delegation**
 - (b) inheritance
 - (c) polymorphism
 - (d) none of the above
- (11) A class
- (a) **binds the data and its related functions together**
 - (b) data and their addresses
 - (c) contains only functions
 - (d) none of the above
- (12) The major drawback of procedural programming languages is
- (a) frequently invoking functions from `main()`
 - (b) **non security of data**
 - (c) non security of methods
 - (d) none of the above

Basics of C++

2

CHAPTER

CHAPTER OUTLINE

- 2.1 Introduction
- 2.2 Steps to Create and Execute a C++ Program
- 2.3 Flowchart for Creating a Source File, Compiling, Linking and Executing in C++
- 2.4 C++ Environments
- 2.5 Typical C++ Environment (Borland C++)
- 2.6 Structure of a C++ Program
- 2.7 Illustrative Simple C++ Program without Class
- 2.8 Header Files and Libraries

2.1 INTRODUCTION

A programmer must know rules and steps to develop programs with C++. Programs given in this book are developed using Borland's C++ IDE compiler version 3.0. Let us try to follow the steps for creating, compiling and running programs under Borland's C++ IDE compiler.

2.2 STEPS TO CREATE AND EXECUTE A C++ PROGRAM

For creating and executing a C++ program, one must follow various phases. The C++ program is created with an editor and saved with extension `.cpp`. Following this step, compilation and execution operations are to be performed. The following phases are essential in 'C++' when a program is to be executed in MS-DOS mode.

- (1) Create
- (2) Save
- (3) Compile
- (4) Execute.

We would now see the steps in each phase.

- (1) Steps for creation of a C++ program
 - (a) Open the Turbo C++ editor. In this book, the path selected for opening the C++ editor is C:\TURBOC3\TC\BIN. Executable file TC is to be clicked in BIN file to open the window of C++.
 - (b) In the `Edit` menu section with `alt+f`, select the `New` option. A new file is selected with this procedure.
 - (c) The program should be written in 'C++' editor.
- (2) Saving a C++ program
 - (a) Save the program using the `Save` option in the file menu with extension `.cpp`. The reader may create his/her own folder where programs are to be stored. The file name does not necessarily include extension `".cpp."` The default extension is `".cpp."` The user can also specify his/her own extension. The C++ program includes preprocessor directives.
- (3) Compilation of a C++ program
 - (a) The source program contains statements that are to be translated into object codes. These object codes are used for execution by the computer. So, compile the source code with `alt+c` keys.
 - (b) If the program contains errors, the programmer should correct it using C++ editor.
 - (c) If there is no error in the program, compilation proceeds and the translated program is stored in another file with the same file name and with extension `".obj."` This object file is stored on a secondary storage device such as the disk.
- (4) Linking and running/execution of a C++ program
 - (a) The linking is also an essential process. It puts together all the program files and functions that are required by the program. For example, if the programmer is using `pow()` function, then the object code of this function should be brought from `math.h` library of the system and linked to the `main()` program. After linking, the program code is stored on the disk. The program code generated is called the executable code.
 - (b) Run the executable code file with `alt+r`, in case of no errors. In case there are errors, correct the program, and follow steps (1), (2), (3), and (4).

2.3 FLOWCHART FOR CREATING A SOURCE FILE, COMPILING, LINKING AND EXECUTING IN C++

The following steps are necessary for execution of a program in C++.

- Write the code for a program and save it
- Compile the program with a compiler
- Link the program with library functions
- Run the program.

A flowchart for implementing the aforesaid steps is given in Figure 2.1

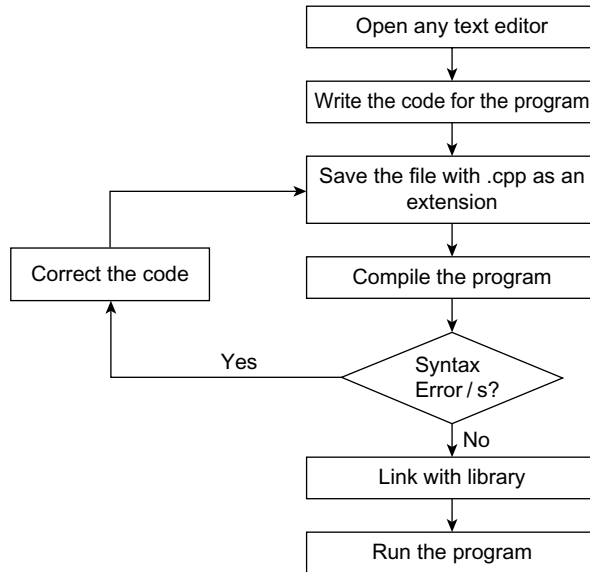


Figure 2.1 Flowchart for creating a C++ program, compiling and linking

The process of compiling, linking and running a program in C++ is elaborated in the following section.

2.4 C++ ENVIRONMENTS

C++ programs can be implemented on different operating systems such as UNIX, Linux, Windows, and DOS, etc. C++ programs executed under DOS environment can also run successfully under Windows, because we have to follow the same programming rules on both platforms. Only the C++ execution environments are different.

Various types of C++ compilers are described below.

- (1) **GNU C++:** The GNU C++ is a compiler and it is available in the source code form. It is available in different versions.
- (2) **NDP C++:** NDP C++ is also a C++ compiler, i.e. compatible with AT&T C++ 2.0 for DOS, UNIX, Sun OS, etc.
- (3) **Oregon C++:** Oregon C++ is a C++ compiler compatible with AT&T C++ 2.0 for a variety of UNIX systems.
- (4) **Zortech C++ Version 2.1:** This compiler has been developed for MS-DOS, OS/2, and UNIX.
- (5) **Borland C++:** Borland C++ compiler is a combination of two compilers. They are ANSI standard C compiler and C++ compiler. It runs on MS-DOS. Borland C++ includes Turbo Assembler, Turbo Debugger, and Turbo Profiler.

In the following section, brief information for creating, compiling, and executing C++ programs are described using Borland C++ compiler.

2.5 TYPICAL C++ ENVIRONMENT (BORLAND C++)

In this section, brief information is given about the different platforms for C++ implementation, and execution of C++ programs using Borland C++ compiler under the Windows platform is described.

STEP 1: OPEN ANY TEXT EDITOR

Unix/Linux platform:

Use text editors such as Gedit, Kwrite, Medit, ed, VI, etc.

Windows platform:

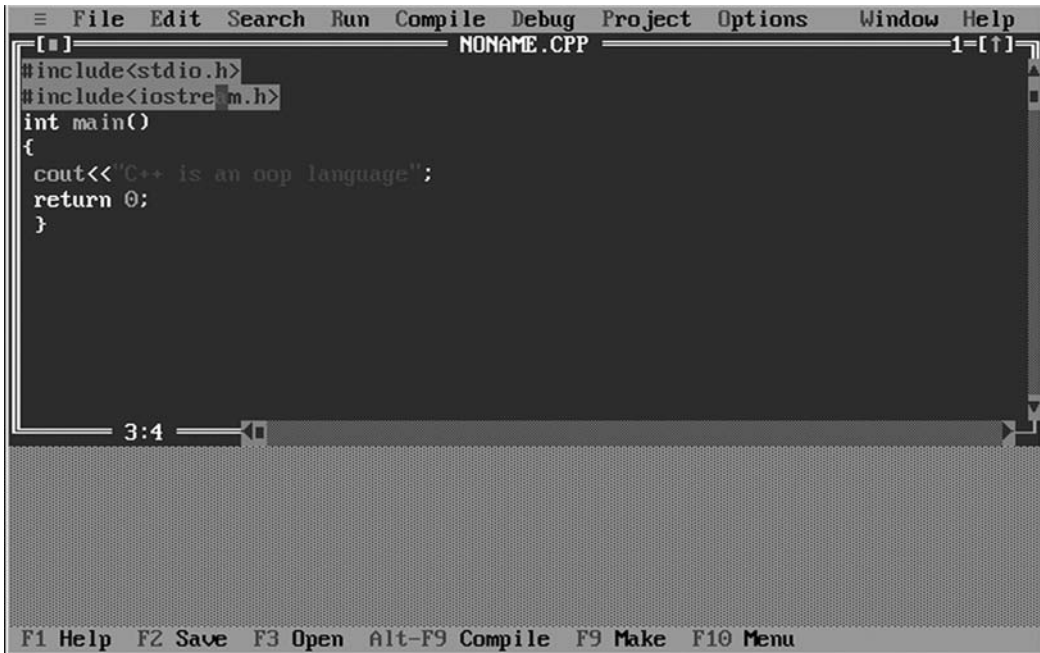
Use text editors such as Notepad, Wordpad, etc.

Alternately, one can use integrated Borland C++ compiler, which supports its own text editor. Once Turbo C++ is installed, it automatically creates a directory `turboc3` and subdirectory `bin`. Follow the path `c:\turboc3` and click the executable file `tc`. One can use desktop shortcut for `turboc3`. Turbo C++ screen appears, which is blank initially and has a menu bar at the top, which is shown as follows.



STEP 2: WRITE THE CODE FOR THE PROGRAM

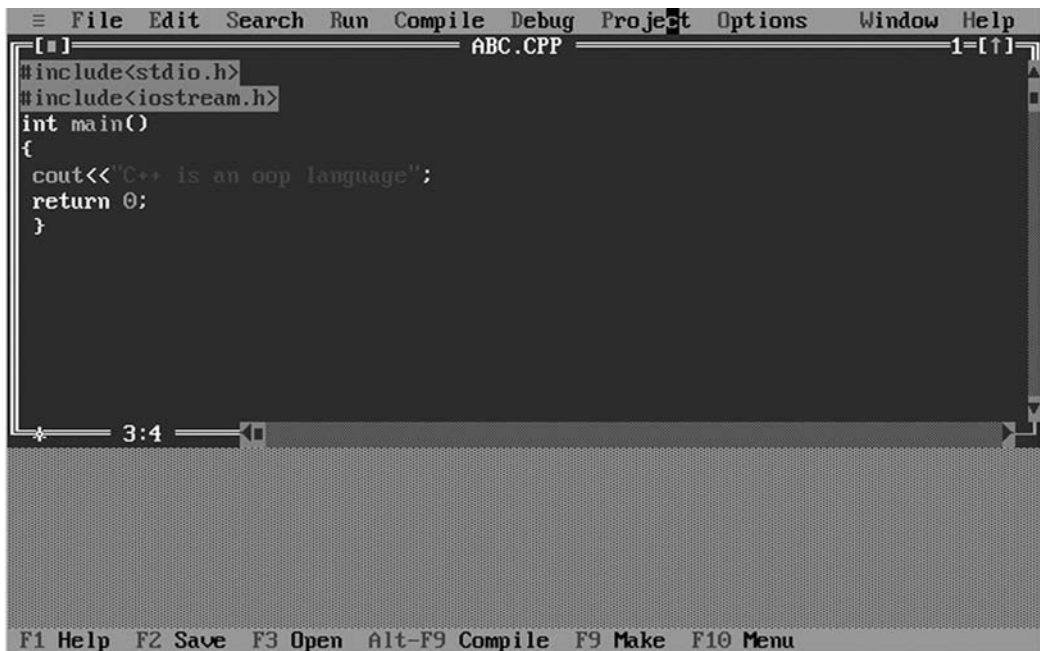
Write the code to fulfill the requirements of the program, if possible using the proper syntax and indentation using an editor.



```
File Edit Search Run Compile Debug Project Options Window Help
NONAME.CPP
#include<stdio.h>
#include<iostream.h>
int main()
{
    cout<<"C++ is an oop language";
    return 0;
}
3:4
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu
```

Alternatively, create a source program using integrated Turbo C++ editor. Go to the File menu and select New. The file name NONAME00 . CPP appears on the screen at the centre of the window. Check whether your source program is typed correctly. If mistakes are found, correct them.

STEP 3: SAVE THE FILE WITH .CPP AS AN EXTENSION



```
File Edit Search Run Compile Debug Project Options Window Help
ABC.CPP
#include<stdio.h>
#include<iostream.h>
int main()
{
    cout<<"C++ is an oop language";
    return 0;
}
3:4
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu
```

Now press F2 to save your program or use Save option with .cpp extension. You can change the file name and save it using the Save option in File menu. In the following window the file is saved with ABC.CPP.

STEP 4: COMPILE THE PROGRAM

Unix/Linux platform:

Open the terminal window; change the working directory to the directory where the program file is stored.

Compile the program using the command: `g++ filename.cpp`

It will display the errors and warnings. If there are any errors and warnings, correct them and save the corrected file again.

After successful compilation, file a.out will be created in the same directory.

Windows platform:

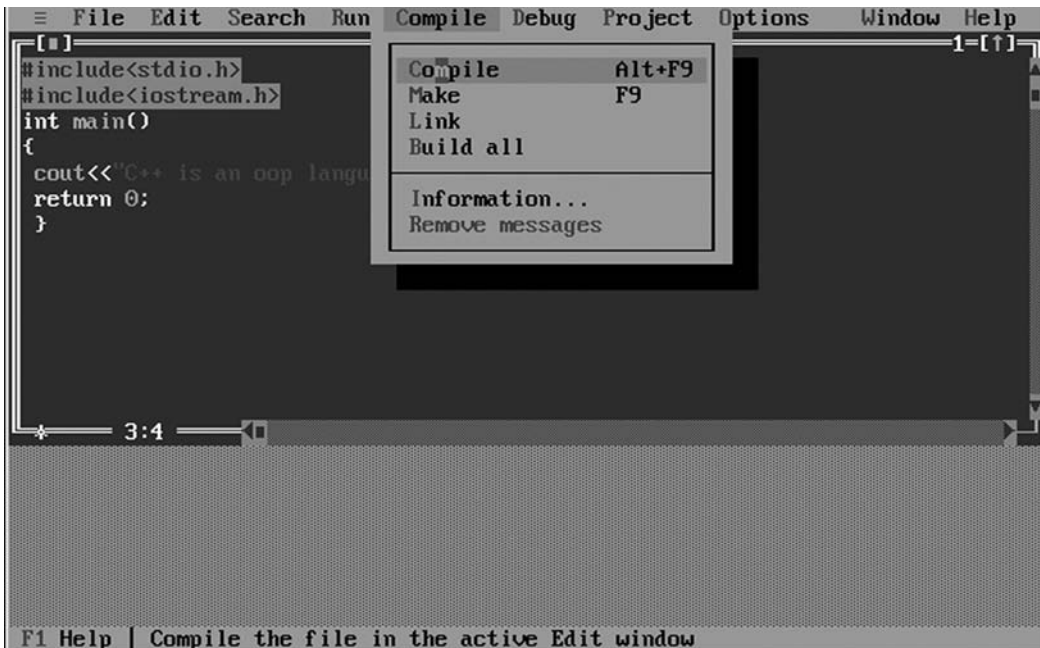
Use IDE such as Turbo C++.

From the File menu, open the program file and compile it using 'Compile' option in Compile menu or use Alt+F9.

It will display the errors and warnings. If there are any errors or warnings, correct them and save the corrected file again, compile again. If program is correct, then the window shows:

- Warnings: 0
- Errors: 0

After successful compilation, a file filename.obj will be created in the same directory. The following window shows the compilation of a program named ABC.CPP.



STEP 5: RUN THE PROGRAM

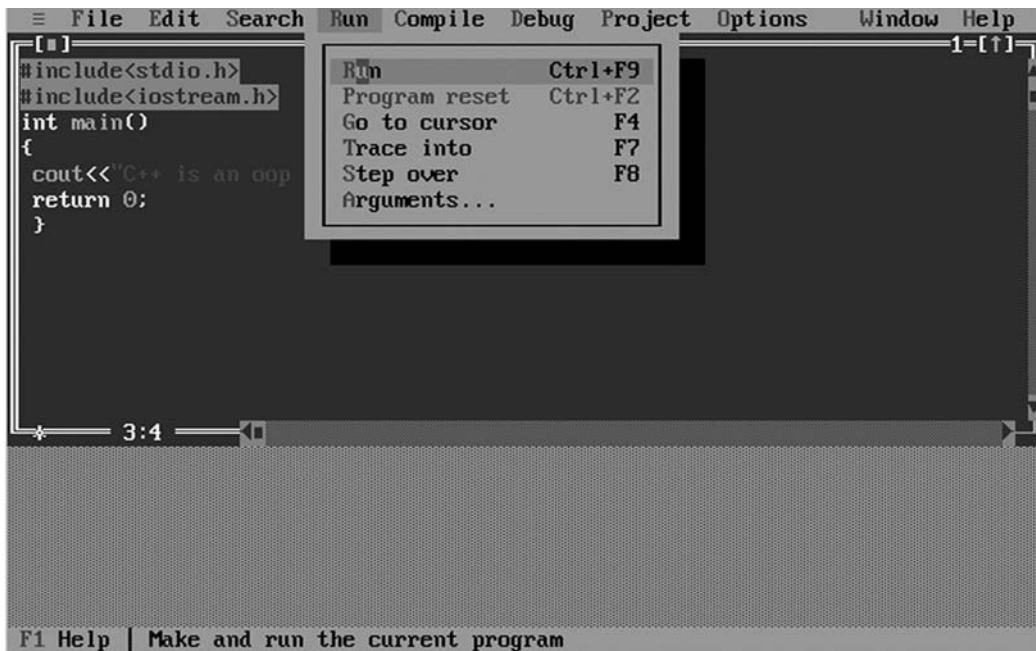
Unix/Linux platform:

Use command `./a.out` to run the program.

Windows platform:

Use 'Run' option from Run menu of Turbo C++ IDE or use `ctrl+F9` and see the output as per program.

The following window shows execution of ABC.CPP program.



2.6 STRUCTURE OF A C++ PROGRAM

C++ programs consist of objects, classes, functions, variables, and other statements. Figure 2.2 shows four different parts of a C++ program.

(a) Preprocessor directives (Include header files section):

Preprocessing refers to the first step in compiling a C++ file from a source language into machine language. One of the most important features of C++ language is to offer preprocessor directives. The preprocessor directives are to be included at the beginning of the program before the `main()`. It begins with symbol `#` (hash). It can be placed anywhere, but quite often, it is declared at the beginning before the `main()` function or any particular function. In traditional C, `#` (hash) must begin at the first column.

Preprocessor directives include header files
Class declaration or definition
Class function definitions
The <code>main()</code> function and program

Fig. 2.2 Parts of C++ program

The following is a preprocessor directive that can be used in C++ programs besides many more.

The #define directive

For example

```
#define PI 3.14
```

This statement defines macro templates. During preprocessing, the preprocessor replaces every occurrence of PI (identifier) with 3.14 (substitute value). Here, PI is a macro template and 3.14 is its macro expansion. The macro templates are generally declared with capital letters for quick identification. One can also define macros with small letters. The macro templates and its expansions must be separated with at least one blank space. It is not necessary to provide space between # and define. It is optional to the programmer. To increase readability, the programmer should provide space.

Include header files section

One can write modular programs by making use of the #include directives. It is used to read the content of file that is included at the beginning of the program. Another file can be included at the start of the program with #include name_of_the_file.cpp.

The C++ program also depends on some header files for function prototypes. They are used in the program. Each header file is to be extended with a .h extension. The file should be included using #include directive as per the format given below.

Example: #include<iostream.h> or #include"iostream.h"

In this example, <iostream.h> header file is included at the beginning by the programmer. This preprocessor directive is essential, which uses input/output statements such as cin and cout. All the definitions and prototypes of the function defined in this file are available in the current program. This file also gets compiled with the original program.

Besides the <iostream.h> header file, numerous other header files are used in programs in different chapters. Details of such header files are not given, which is beyond the scope of this discussion.

- (b) **Class declaration or definition:** Declaration of class is done in this section. In class definition, prototype or definitions of function are also declared.
- (c) **Class function definitions:** This part contains definition of functions. The definition of function can also be written outside the class but before main(). The outside definitions of function should need class name and scope access operator before the function name.
- (d) **The main() function:** Like C, C++ programs also start with the main() function. The execution of every program starts with the main() function. The programming rules for C++ are the same as C. However, there are a few differences that are discussed at appropriate places.

2.7 ILLUSTRATIVE SIMPLE PROGRAM IN C++ WITHOUT CLASS

A simple C++ program is as follows:

```
#include<iostream.h>
#include<conio.h>

// First Simple program
```



```
int main()
{
    clrscr();
    cout<<"\n Hello! Welcome to the world of C++";
    return 0;
}
```

- Every C++ program executes from the `main()`.
- Opening and closing braces are used for the beginning and ending of the program, respectively. Braces must be balanced. The number of opening braces should be the same as that of closing braces.
- The flow of the program can be followed by proper documentation. One can use comments in the program. A comment statement starts with two successive back slashes with no space between them, i.e. it begins with a double slash symbol(`//`) in the program statement as shown in the above program. A comment can be placed anywhere in the line and it does not end with any symbol.

Comment symbols similar to those used in C++ can also be used. In C, comment symbols `/*` and `*/` are used in programs. These symbols are also permitted in C++ programs.

- Header files are needed at the beginning of the program; they include the definition of the functions that are used in the program. In the above program `#include<conio.h>` header file is used, which takes care of `clrscr()` and the header file `#include<iostream.h>` helps stream programming feature such as `cout` and `cin`.
- The statement `cout<<"\n Hello! Welcome to the world of C++";` displays the string enclosed between the quotation marks and `\n` escape sequence is used to print the string on the next line.
- The `cout` (C out) and `<<` (insertion operator) are the identifiers used in C++.

In C++, `int main()` returns an integer value to operating system. Operating system checks whether the program is executed successfully. The return value of the function `int main()` is used for successful execution of the program. If the returned value is zero (0), it means that the program is executed successfully. A non-zero value indicates that the program execution was unsuccessful.

The prototype of `main()` function in C++ is as follows. The return statement is used in the program at the end of the `main()` function to return a value. If there is no return statement, the compiler displays a warning message "Function should return a value". The default `int` is to be returned to the `main()` in case `int` is not specified in the `main()`.

```
int main();
int main(int argc, char *argv[]);
```

C++ programs are coupled with class. A class encapsulates data and associated member functions. A detailed description is given in the classes and objects chapter in this book.

2.8 HEADER FILES AND LIBRARIES

This part describes library functions that are provided with the C++ compiler. The source code for these library functions are not provided, but they are in a compiled form. We have to link the

code of the library functions with the compiled programmer's code to form the entire program. The library functions perform operations such as managing memory, reading and writing disk files, input/output, mathematical operations, etc.

Numerous library functions are supported by different files. For example, `math.h` supports functions such as `abs()`, `pow()`, `sqrt()`, `floor()`, `ceil()`, etc. The `conio.h` supports functions such as `getch()`, `getche()`, `clrscr()`, `gotoxy()`, `import()`, etc. These files should be included while library functions are to be used in the programs. They are provided with C++ compiler. These files have the extension `.h` and they are called as header files. Table 2.1 gives a list of some of the header files and functions supported by them. The programmer is advised to refer library functions from the C++ compiler.

Table 2.1 The header files and library functions

Name of the header file	Function of the header file	List of the few functions supported by the header files
<code>alloc.h</code>	Memory management functions	<code>calloc()</code> , <code>malloc()</code> , <code>free()</code> , <code>realloc()</code> , etc.
<code>complex.h</code>	Complex math functions	<code>asin()</code> , <code>atan()</code> , <code>arg()</code> , <code>tanh()</code> , etc.
<code>ctype.h</code>	Automatic type conversion functions	<code>toupper()</code> , <code>tolower()</code> , <code>islower()</code> , <code>isupper()</code> , etc.
<code>dos.h</code>	Perform DOS functions	<code>getdate()</code> , <code>gatetime()</code> , <code>int 86()</code> , <code>sleep()</code> , etc.
<code>graphics.h</code>	Graphics functions	<code>arc()</code> , <code>bar()</code> , <code>circle()</code> , <code>getx()</code> , etc.
<code>process.h</code>	Process functions	<code>exit()</code> , <code>abort()</code> , <code>system()</code> , etc.
<code>stdio.h</code>	Standard input/output functions	<code>puts()</code> , <code>gets()</code> , <code>fopen()</code> , <code>fclose()</code> , <code>printf()</code> , <code>getchar()</code> , <code>putchar()</code> , etc.
<code>stdlib.h</code>	Standard library functions	<code>atoi()</code> , <code>atol()</code> , <code>time()</code> , <code>abort()</code> , etc.
<code>string.h</code>	Manipulates various string operations with different functions	<code>strcpy()</code> , <code>strcat()</code> , <code>strlen()</code> , etc.
<code>time.h</code>	Declare functions that manipulate time and date	<code>time()</code> , <code>stime()</code> , <code>difftime()</code> , <code>localtime()</code> , etc.

SUMMARY

- (1) Steps to develop programs with C++.
- (2) Phases of execution of a C++ program.
 - (i) Create
 - (ii) Save
 - (iii) Compile
 - (iv) Execute
- (3) Flowchart for creating a source file, compiling, linking and executing in C++. Following four steps are described briefly
 - Step 1: Open any text editor.
 - Step 2: Write the code for the program.

Step 3: Save the file with .cpp as an extension.

Step 4: Compile the program.

Step 5: Run the program.

(4) Structure of C++ program.

(5) Header and library files.

EXERCISES

(A) Answer the following questions

- (1) Explain the steps for execution of a C++ program using Borland C++ compiler.
- (2) Show and explain the flowchart for creating, compiling, linking and executing a C++ program.
- (3) What are the roles of header files?
- (4) Compare `main()` functions of C with C++.
- (5) Why do we use comment statements in the program. Write your answer to the point.
- (6) Elaborate the structure of C++ program.
- (7) Name the various C++ compilers that exist in the market other than the Borland C++ compiler.
- (8) What do you mean by namespace?

(B) Answer the following by selecting the appropriate options

- (1) To open a file in C++ with Turbo C++ editor, the key combination used is
 - (a) **alt+f**
 - (b) alt+o
 - (c) alt+n
- (2) To save a file in C++ with Turbo C++ editor, the short key to be used is
 - (a) **F2**
 - (b) F3
 - (c) F4
- (3) To compile a file in C++ with Turbo C++ editor, the key combination used is
 - (a) **alt+c**
 - (b) alt+k
 - (c) alt+i
- (4) To run a program in C++ with Turbo C++ editor, the key combination used is
 - (a) **alt+F9**
 - (b) ctrl+F9
 - (c) ctrl+F10
- (5) Comment line to be used in C++ starts with
 - (a) \
 - (b) **//**
 - (c) **
- (6) The following code is acceptable by machines/computers for the execution of a program
 - (a) source code
 - (b) **object code**
 - (c) none of the above
- (7) Every program in C++ must start with
 - (a) {
 - (b) }
 - (c) **main()**
- (8) A simple C++ program executes
 - (a) **with pre-processor directives**
 - (b) without pre-processor directives
- (9) In C++ the function `main` has a default return type
 - (a) float
 - (b) **integer**
 - (c) character
- (10) The scope of the variable/s is/are confined
 - (a) **to a block**
 - (b) out of block
 - (c) none of the above
- (11) `#include` instructs the
 - (a) **compiler**
 - (b) assembler
 - (c) none of the above
- (12) `#define` is a
 - (a) instruction
 - (b) **directive**
 - (c) decision statement

-
- (13) The compiler translates source code into
- (a) octal code
 - (b) **object code**
 - (c) decimal
- (14) Borland C++ consists of
- (a) ANSI standard C compiler
 - (b) C++ compiler
 - (c) **both**
 - (d) none of these
- (15) The `iostream.h` header file supports
- (a) **input/output statements such as `cin` and `cout`**
 - (b) process management
 - (c) memory allocation
- (16) The `exit()` function is supported by the which of the following header files
- (a) `math.h`
 - (b) **`process.h`**
 - (c) `graphics.h`
 - (d) `dos.h`
- (17) The `circle()` function is supported by the which of the following header files
- (a) `alloc.h`
 - (b) `math.h`
 - (c) **`graphics.h`**
 - (d) `ctype.h`

Input and Output in C++

3 CHAPTER

CHAPTER OUTLINE

- 3.1 Introduction
- 3.2 Streams in C++ and Stream Classes
- 3.3 Pre-defined Streams
- 3.4 Buffering
- 3.5 Stream Classes
- 3.6 Formatted and Unformatted Data
- 3.7 Unformatted Console I/O Operations
- 3.8 Type Casting with the `cout` Statement
- 3.9 Member Functions of the `istream` Class
- 3.10 Formatted Console I/O Operations
- 3.11 Bit Fields
- 3.12 Flags without Bit Field
- 3.13 Manipulators
- 3.14 User-defined Manipulators
- 3.15 Manipulator with One Parameter
- 3.16 Manipulators with Multiple Parameters
- 3.17 More Programs

3.1 INTRODUCTION

Computer applications generally involve tremendous amount of data to be read from input devices and sending them to the output devices. Hence, to control such operations, every programming language provides a set of inbuilt functions. C++ supports all input/output (I/O) functions of C. C++ also has library functions. A library is a set of `.obj` files, which is linked to the program and gives additional beneficial support with its functions. The programmer can use the library functions in the programs. The library is also called as `iostream` library. The difference

between C and C++ I/O function is that C functions do not support object-oriented platform whereas C++ supports object-oriented platform.

3.2 STREAMS IN C++ AND STREAM CLASSES

The C++ supports a number of I/O operations to perform read and write operations. These C++ I/O functions help the user to work with different types of devices such as keyboard, disk,

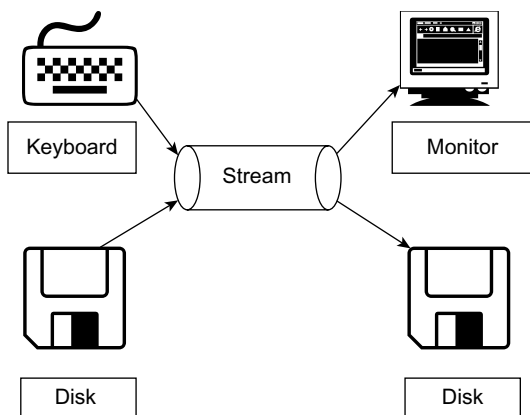


Fig. 3.1 Streams and I/O devices

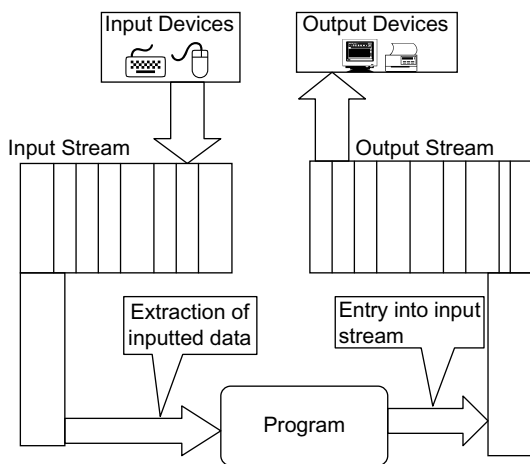


Fig. 3.2 C++ input and output streams

tape drivers, etc. Stream is an inter-mediator between I/O devices and the user. The standard C++ library contains the I/O stream functions. The I/O functions are part of the standard library that provides portability to the language itself. A library is a set of .obj files connected to the user's program.

The stream is a flow of data, measured in bytes, in sequence. If data is received from input devices in sequence, then it is called as source stream, and when the data is passed to output devices, then it is called as destination stream. This process of flow of data is also known as encapsulation through streams. The data is received from the keyboard or disk and can be passed to the monitor or to the disk. Figure 3.1 describes the concept of stream with input and output devices.

The data in source stream can be used as input data by the program. So, the source stream is also called as input stream. The destination stream that collects output data from the program is known as the output stream. The mechanism of the input and output stream is illustrated in Figure 3.2.

As discussed earlier, the stream is an inter-mediator between I/O devices and the user. The input stream receives data from keyboard or storage devices such as hard disk, floppy disk, etc. The data present in output stream is passed on to the output devices such as monitor or printer according to the user's choice.

3.3 PRE-DEFINED STREAMS

C++ has a number of pre-defined streams. These pre-defined streams are also called as standard I/O objects. These streams are automatically activated when program execution starts. Table 3.1 describes the pre-defined streams.

Table 3.1 Pre-defined C++ Stream or I/O Global Objects

<code>cin</code>	Standard input, usually keyboard, corresponding to <code>stdin</code> in C. It handles input from input devices usually from keyboard.
<code>cout</code>	Standard output, usually screen, corresponding to <code>stdout</code> in C. It passes data to output devices such as monitors and printers. Thus, it controls output.
<code>clog</code>	A fully buffered version of <code>cerr</code> (no C equivalent). It controls error messages that are passed from buffer to the standard error device.
<code>cerr</code>	Standard error output, usually screen, corresponding to <code>stderr</code> in C. It controls the un-buffered output data. It catches the errors and passes to standard error device monitor.

3.1 Write a program to display a message using pre-defined objects.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout<<"\nSTREAMS";
    cerr<<"\nSTREAMS";
    clog<<"\nSTREAMS";
    return 0;
}
```

OUTPUT

```
STREAMS
STREAMS
STREAMS
```

Explanation: In this program the pre-defined objects `cout`, `cerr`, and `clog` are used to display the message “STREAMS” on the screen.

3.4 BUFFERING

It is a block of memory used to hold data temporarily. It is always located between a peripheral device and a faster computer. Buffers are used to pass data between computer and devices. The buffer increases the performance of the computer by allowing read/write operation in larger chunks. For example, if the size of the buffer is N , the buffer can hold N number of bytes of data.

Writing data to the disk is very costly. The read/write operation with disk takes a long time. When these operations are carried out, the execution of the program will be slow for few seconds. If the program involves many read and write operations, the program execution speed will be slower. To avoid this problem, the stream provides buffer. Buffer holds the data temporarily. The input data is passed to the stream buffer. The data is not written to the disk immediately till the buffer fills. When the buffer is completely filled, the data is written to the disk. This is explained with an example in Figure 3.3.

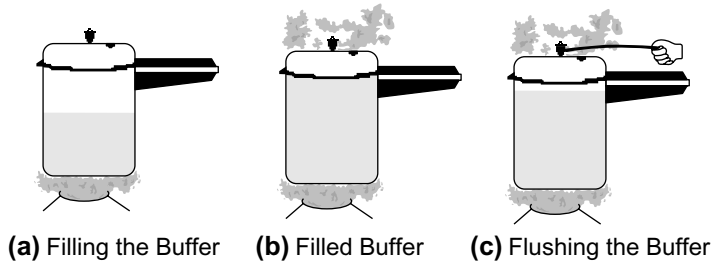


Fig. 3.3 Working of buffer

In Figure 3.3a, the level of the steam in the pressure cooker is increasing. The steam is not full, hence the whistle will not blow, and the steam will not be released from the pressure cooker.

In Figure 3.3b, the pressure cooker is completely filled. The level of the steam reaches to the top. Due to high-pressure the whistle blows automatically and the steam comes out of the pressure cooker.

After the steam comes out the whistle falls down. Even when the cooker is not completely filled with steam, the steam can be released manually by pulling up the whistle. It is just like flushing the buffer even when the buffer is not completely filled.

3.5 STREAM CLASSES

C++ streams are based on class and object theory. C++ has number of classes that work with console and file operations. These classes are known as stream classes. Figure 3.4 shows the stream classes. All these classes are declared in the header file `iostream.h`. The file `iostream.h` must be included in the program, if we are using the functions of these classes.

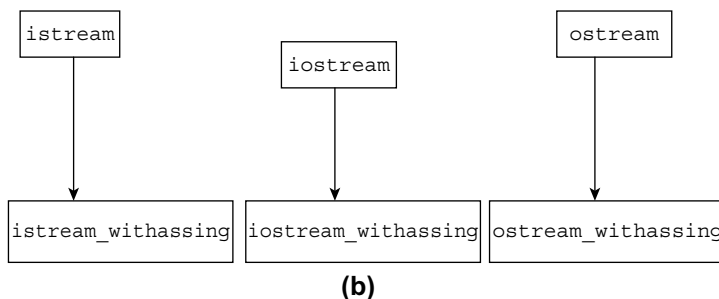
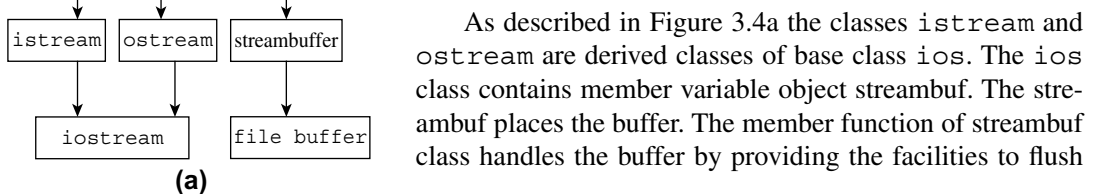


Fig. 3.4 Hierarchy of stream classes

clear and pour the buffer. The class `iostream` is derived from the classes `istream` and `ostream` by using multiple inheritance. The `ios` class is a virtual class and it is present to avoid ambiguity that frequently appears in multiple inheritance. Chapter 11 describes multiple inheritance and virtual classes.

The `ios` class has an ability to handle formatted and unformatted I/O operations. The `istream` class gives support for both formatted and unformatted data. The `ostream` classes handle the formatting of output data. The `iostream` contains functions of both `istream` and `ostream` classes. The classes `istream_withassign`, `ostream_withassign`, and `iostream_withassign` append appropriate assignment operators as shown in Figure 3.4b. Table 3.2 describes functions/contents of C++ stream classes.

Table 3.2 Functions/Contents of C++ Stream Classes

Class	Function/Contents
<code>ios</code>	(1) It is an input and output stream class. (2) It is used to implement a buffer, i.e. it is pointer to a buffer <code>streambuf</code>. (3) <code>ios</code> maintains the information on the state of <code>streambuf</code>, i.e. good, bad, eof, etc.
<code>istream</code>	(1) <code>istream</code> provides formatted input. (2) It is used to handle formatted as well as unformatted conversion of character from a <code>streambuf</code>. (3) The properties of <code>ios</code> are inherited in <code>istream</code> class. (4) The instance of class does not carry out the actual input. (5) <code>istream</code> declares functions such as <code>peek()</code>, <code>tellg()</code>, <code>seekg()</code>, <code>getline()</code>, <code>read()</code>, etc. (6) <code>istream</code> class overloads the <code>'>>'</code> operator.
<code>ostream</code>	(1) It is used for general-purpose output. (2) It is used to declare the output functions such as <code>tellp()</code>, <code>put()</code>, <code>write()</code>, <code>seekp()</code>, etc. (3) It is the parent of all output stream. (4) <code>ostream</code> overloads the <code>'<<'</code> operator.
<code>iostream</code>	(1) It is used to handle both input and output streams.
<code>istream_withassign</code>	(1) It is derived from <code>istream</code>. (2) It is used for <code>cin</code> input.
<code>iostream_withassign</code>	(1) It is a bidirectional stream.

3.6 FORMATTED AND UNFORMATTED DATA

Formatting means representation of data with different settings as per the requirement of the user. The various settings that can be done are number format, field width, decimal points, etc.

The data accepted or printed with default setting by the I/O function of the language is known as unformatted data. For example, when the `cin` statement is executed, it asks for a number. The user enters a number in decimal. For entering decimal number or displaying the number in decimal using `cout` statement, the user will not need to apply any external setting. By default, the I/O function represents the number in decimal format. Thus, the data handled in such a way is called as unformatted data.

If the user needs to accept or display data in hexadecimal format, manipulators with I/O functions should be used. The data obtained or represented with these manipulators are known as

formatted data. For example, if the user needs to display the data in hexadecimal format, then the manipulator can be used as follows.

```
cout<<hex<<15;
```

The above statement converts decimal 15 to hexadecimal F.

3.7 UNFORMATTED CONSOLE I/O OPERATIONS

Input and Output Streams

The input stream uses `cin` object to read data and the output stream uses `cout` object to display the data on the screen. The `cin` and `cout` are pre-defined streams for input and output data. The data type is identified by these functions using operator overloading of the operators `<<` (insertion operator) and `>>` (extraction operator). The operator `<<` is overloaded in `ostream` class and the operator `>>` is overloaded in `istream` class. Figure 3.5 shows the flow of input and output stream.

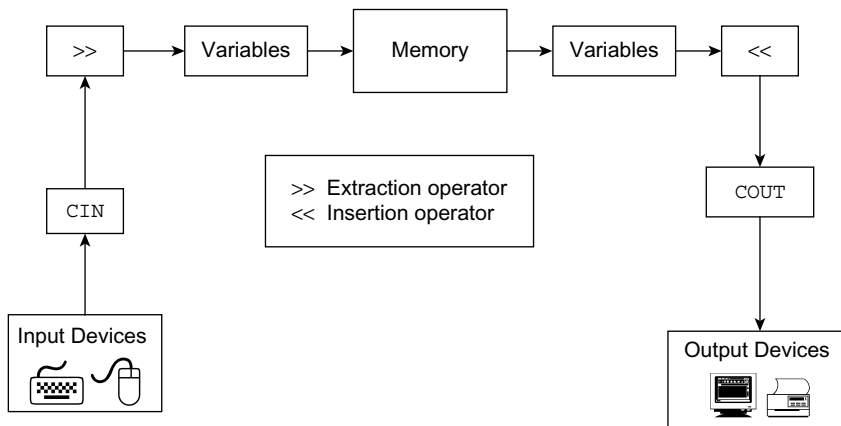


Fig. 3.5 Working of `cin` and `cout` statements

Input Stream

The input stream reads operation through keyboard. It uses `cin` as object. The `cin` statement uses `>>` (extraction operator) before a variable name. The `cin` statement is used to read data through the input device. Its syntax and example are as follows.

Syntax:

```
cin>>variable;
```

Example:

```
int v1;
```

```
float v2;
```

```
char v3;
```

```
cin>> v1>> v2>> v3...>> vn;
```

where `v1`, `v2`, and `v3` are variable names. The response of the user to this statement would be as shown below.

```
2 5.4 A // input data
```

If the user enters data in the manner `2 5.4 A`, then the operator will assign 2 to `v1`, 5.4 to `v2` and `A` to `v3`. If the entered data is greater than the variable, it remains in the input stream. While entering string, blank spaces are not allowed. More than one variable can be used in `cin` statement to input data. Such operations are known as cascaded input operations.

For example,

```
cin>>v1>>v3;
```

where `v1` and `v2` are variables.

The operator `>>` accepts the data and assigns it to the memory location of the variables. Each variable requires `>>` operator. Both these statements should not be included in the bracket. The enter data is separated by space, tab, or enter. Similar to `scanf()` statement, `cin` does not require control strings such as `%d` for integer, `%f` for float, etc.

More examples

```
int weight;  
cin>>weight // Reads integer value  
float height;  
cin>>height; // Reads float value  
double volume;  
cin>>volume; // Reads double value  
char result[10];  
cin>>result; // Reads char string
```

Output Streams

The output streams manage output of the stream, that is, display contents of variables on the screen. It uses `<<` insertion operator before the variable name. It uses the `cout` object to perform console write operation. The syntax and example of `cout` statement are as follows.

Syntax:

```
cout<<variable
```

Example:

```
cout<<v1 <<v2 <<v3 ... <<vn;
```

where, `v1`, `v2`, and `v3` are variables. The above statement displays the contents of these variables on the screen. The syntax rules are similar to `cin`. Here, the `cout` statement uses the insertion operator `<<`. The `cout` statement does not use the format specification such as `%d`, `%f` as used in C, etc. The `cout` statement allows us to use all 'C' escape sequences such as `'\n'`, `'\t'`, etc.

More examples

```
int weight;  
cout<<weight // Displays integer value  
float height;
```

```
cout<<height; // Displays float value
double volume;
cout<<volume; // Displays double value
char result[10];
cout<<result; // Reads char string
```

The table given below illustrates comparative programs on cout statements.

PROGRAM 3.2	PROGRAM 3.3	PROGRAM 3.4
<pre>#include <iostream.h> int main() { cout<<" C PLUS PLUS"; return 0; }</pre>	<pre>#include <iostream.h> int main() { char *n="Hello"; cout<<n; return 0; }</pre>	<pre>#include <iostream.h> int main() { int x=10; float f=3.14; cout<<x; cout<<"\t"; cout<<f; return 0; }</pre>
OUTPUT	OUTPUT	OUTPUT
C PLUS PLUS	Hello	10 3.14

In this program, the cout statement displays the message "C PLUS PLUS" on the screen.

In this program, the string is first assigned to character pointer n. The cout statement displays the contents of variable n.

In this program, integer and float values are assigned to variables x and f. The cout statement displays the values of x and f.

3.5 Write a program to accept a string through the keyboard and display it on the screen. Use cin and cout statements.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char name[15];
    clrscr();
    cout<<"Enter Your Name :";
    cin>>name;
    cout<<"Your name is"<<name;
```

{coderipe}

```
    return 0;  
}
```

OUTPUT

Enter Your Name :Amit
Your name is Amit

Explanation: In the above program, `cout` statement displays a given string on the screen. It is similar to `printf()` statement. The `cin` statement reads data through the keyboard. It is similar to `scanf()` statement.

3.6 Write a program to read two integers and display them. Use `cin` and `cout` statements.

```
#include<iostream.h>  
#include<conio.h>  
  
int main()  
{  
    int num,num1;  
    clrscr();  
    cout<<"Enter Two numbers :";  
    cin>>num>>num1;  
    cout<<"Entered Numbers are :";  
    cout<<num<<"\t"<<num1;  
    return 0;  
}
```



OUTPUT

Enter Two numbers : 8 9
Entered Numbers are : 8 9

Explanation: In the above program, the `cin` statement reads two integers in variables `num` and `num1`. The `cout` statement displays the read numbers. The escape sequence `'\t'` is used to insert tab between two numbers.

3.7 Write a program to display data using `cout` statements.

```
#include<iostream.h>  
#include<conio.h>  
  
int main()  
{
```

```

clrscr();
cout<<"=====";
cout<<endl;
cout<<"Hello"; // prints hello
cout<<"\n";
cout<<123; // prints 123
cout<<endl<<3.145; // prints 3.145
cout<<endl<<"NUMBER:"<<"\t"<<452; // prints string and value
cout<<"\n==== The end =====";
return 0;
}

```

OUTPUT

```

=====
Hello
123
3.145
NUMBER : 452
==== The end =====

```

Explanation: Explanation of each of the cout statement is as follows.

- (a) `cout<<"====="` – Displays line of on the screen.
- (b) `cout<<endl` – Breaks a line.
- (c) `cout<<"Hello"` – Display string "Hello" on the screen.
- (d) `cout<<"\n"` – Breaks a line.
- (e) `cout<<123;` – Displays integer 123.
- (f) `cout<<endl<<3.145` – Breaks a line and displays float number 3.145.
- (g) `cout<<endl<<"NUMBER: "<<"\t"<<452` – Breaks a line, displays the string "number", inserts a tab and integer number 452.
- (h) `cout<<"\n==== The end ====="` – Displays the line and string.

3.8 Write a program to display int, float, char, and string using cout statement.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x=5;
    float y=5.2;
    char z='z';
}

```

```
char city[]="NANDED";
cout<<"x = "<<x <<" y ="<<y <<" z = "<<z <<endl;
cout<<"City = "<<city;
return 0;
}
```

OUTPUT

X = 5 y =5.2 z = z
City = NANDED

Explanation: In the above program, the statement `cout<<"x = "<<x <<" y ="<<y <<" z = "<<z <<endl` displays values of x, y, and z. The statement `cout<<"City = "<<city` displays the contents of character array city.

3.9 Write a program to input int, float, char, and string using cin statement and display using cout statement.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x;
    float y;
    char z;
    char city[15];
    cout<<"\n Enter integer, float and char";
    cin>>x>>y>>z;
    cout<<"\n Enter a string :";
    cin>>city;
    cout<<"x = "<<x <<" y ="<<y <<" z = "<<z <<endl;
    cout<<"City = "<<city;
    return 0;
}
```

OUTPUT

Enter integer, float and char 12 1.2 H
Enter a string : NAGPUR
X = 12 y =1.2 z = H
City = NAGPUR

Explanation: Explanation of the program is as follows.

- (a) `cout<<"\n Enter integer, float, and char"` – Prompts message “Enter integer, float, and char”
- (b) `cin>>x>>y>>z` – Accepts integer, float, and char and stores in x, y, and z.
- (c) `cout<<"\n Enter a string: "` – Displays message “Enter a string: ”.
- (d) `cin>>city` – Reads string through the keyboard and stores in the array `city[15]`.
- (e) `cout<<"City = "<<city;` – Displays contents of the array `city`.

3.8 TYPE CASTING WITH THE `cout` STATEMENT

Type casting refers to conversion data of one basic type to another by applying external use of data type keywords. The description of type casting is explained in Chapter 3. Programs on type casting are as follows.

3.10 Write a program to use different formats of type casting and display the converted values.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int a=66;
    float f=2.5;
    double d=85.22;
    char c='K';
    clrscr();
    cout<<"\n int in char format : "<<(char)a;
    cout<<"\n float in int format : "<<(int)f;
    cout<<"\n double in char format : "<<(char)d;
    cout<<"\n char in int format : "<<(int)c;
    return 0;
}
```



OUTPUT

```
int in char format : B
float in int format : 2
double in char format : U
char in int format : 75
```

Explanation: In the above program, the variables of `int`, `float`, `double`, and `char` type are declared and initialized. The variable `a` is initialized with 66, `f` with 2.5, `d` with 85.22, and `c` with character ‘K’.

The first `cout` statement converts integer value to the corresponding character according to the ASCII character set and the character B is displayed.

The second `cout` statement converts float value to integer. The value displayed is 2 and not 2.5. When type cast format (`int`) is used, the decimal portion of float value is removed and only the integer part is considered.

In the third statement, the double value is converted to character. The number 85.22 is converted to integer and then to character. The `char` data type is nothing else than `int` data type. The only difference is that the `char` data type has range from -128 to 127, which requires one byte in the memory.

The last statement converts character to `int`. The value of 'K' is 75 when printed as an integer. The format (`int`) converts `char` to `int`.

In Figure 3.6, 65 is an integer and it is converted to character A by using type casting format (`char`). Table 3.3 describes various type casting formats and their output results.

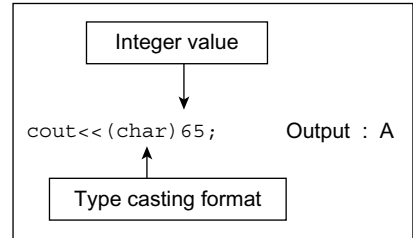


Fig. 3.6 Typecasting integer

Table 3.3 Type Casting Formats

Type Casting Formats	Outputs	Conversion
<code>cout << (char) 65;</code>	A	int to char
<code>cout << (int) 'A';</code>	65	char to int
<code>cout << (int) 5.22;</code>	5	float to int
<code>cout << (char) 78.33;</code>	N	float to char
<code>cout << (double) 123445338.33;</code>	1.234453e+08	float to double
<code>cout << (unsigned) -1;</code>	65535	signed to unsigned

3.11 Write a program to display data using type casting.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x=77;
    float y=5.1252;
    char z='A';
    char city[15];
    cout<<" x = "<<(char)x<<endl;
    cout<<" y = "<<(int)y <<endl;
    cout<<" z = "<<(int)z;
  
```



```
    return 0;
}
```

OUTPUT

```
x = M
y = 5
z = 65
```

Explanation: Consider the following statements.

`int x=77` – Declares integer variable `x` and initializes it with 77.

`float y=5.1252` – Declares float variable `y` and initializes it with 5.1252.

`char z='A'` ; – Declares character variable `z` and initializes it with character 'A'.

The following statements are used to display the contents on the screen.

`cout<<" x = "<<(char) x<<endl` – Variable `x` is an integer but value displayed will be 'M' because the statement `(char)` converts integer to the corresponding *ASCII* character.

`cout<<" y = "<<(int)y <<endl` – Variable `y` is a float but before printing the value 5.1252, it is converted to integer and the output will be 5.

`cout<<" z = "<<(int)z` – Variable `z` is of character type. The character value is converted to integer and the output displayed will be 65.

3.12 Write a program to display A to Z alphabets using ASCII values.

```
#include<iostream.h>
#include<conio.h>
#include<stdio.h>

int main()
{
    clrscr();
    int j;
    for (j=65;j<91;j++)
        cout<<(char)j<<" ";
    cout<<endl;
    for (j=65;j<91;j++)
        printf ("%c ",j);
    return 0;
}
```

OUTPUT

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

Explanation: In the above program, A to Z alphabets are displayed using `cout()` and `printf()` statements. In `cout()` statement before printing, type casting is done. The integer is converted to corresponding char type symbol and displayed. In the `printf()` statement, the control string `%c` performs this task. The quotation mark (“ ”) inserts space between two successive characters.

3.13 Write a program to display addresses of variables in hexadecimal and unsigned integer formats.



```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x=77;
    float y=5.1252;
    int z=78;
    cout<<" Address of x = "<<&x<<endl;
    cout<<" Address of y = "<<&y <<endl;
    cout<<" Address of z = "<<(unsigned)&z <<endl;
    return 0;
}
```

OUTPUT

Address of x = 0x887ffff4

Address of y = 0x887ffff0

Address of z = 65518

Explanation: The `cout` statement displays the address of a variable in hexadecimal format. Using type casting, syntax `(unsigned)` converts hexadecimal to unsigned integer (decimal). The output shows addresses in both hexadecimal and unsigned integer (decimal) formats.

The `&` (ampersand) operator is used to display the address of the variable. The address operator is preceded by the variable name. The address is always represented as an unsigned integer. The `cout` statement displays the address in hexadecimal format. To convert the hexadecimal address to an unsigned integer, type casting is used.

3.14 Write a program to display string using `&` and `*` operators with `cout` statements.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    char *name="c plus plus";
    cout<<name<<"\n";
}
```

```
    cout<<&name[0]<<"\n";  
    cout<<*&name);  
    return 0;  
}
```

OUTPUT

C plus plus

C plus plus

C plus plus

Explanation: In the above program, the character pointer name is assigned to the string “c plus plus”. The first cout statement displays the string using variable name. The statement `cout<<&name[0]<<"\n"` displays the string. Here, & operator is used and the 0 (zero) points to the base address of the string. In the second statement, if the base address is not specified, then it will display the address. The third statement uses the pointer notation to display the string.

3.15 Write a program to display a string using different syntaxes using the operators * and & with cout statement.

```
#include<iostream.h>  
#include<conio.h>  
  
int main()  
{  
    clrscr();  
    char *name="c plus plus";  
    cout<<*&(name)<<"\n";  
    cout<<&(*name)<<"\n";  
    cout<<*&name<<"\n";  
    return 0;  
}
```

OUTPUT

C plus plus

C plus plus

C plus plus

Explanation: In the first statement, the operators * and & are used one after another and the variable name is inside the parentheses. In the second statement, operator & is outside and the operator * and variable name are inside the parentheses. In the third statement, parentheses are not used. All the three statements display the output “c plus plus.”

3.8.1 Difference of Using C and C++ I/O Functions

The `printf()` and `scanf()` of C language needs format string. For example, to read and display an integer, the `scanf` and `printf()` statement can be written as follows.

```
int x;
scanf ("%d", &x)
printf ("%d", x);
```

In the above statements, `%d` is used to tell the I/O functions to treat the data as integer.

If the integer `x` is changed to long integer, the programmer needs to change every occurrence of `%d` in the program with `%ld`.

The C++ statement reads and displays the same data as follows.

```
int x;
cin>>x;
cout<<x;
```

Here, the `cin` and `cout` statements do not require any format string. If the type of `x` is changed to long integer, the user need not specify the type of data or any correction in the statement. The `cin` and `cout` statement identifies the data type. The format of `cin` and `cout` statement is the same for all types of variables.

get () and put () functions

get () function

The single character input and output operations in C++ can be done using `put()` and `get()` functions. The classes `istream` and `ostream` provide the two member functions `put()` and `get()`. The `get()` is used to read a character and `put()` is used to display the character on the screen.

The `get()` function has two syntaxes:

- (a) `get(char*);`
- (b) `get(void);`

If syntax (a) is used, the `get()` function assigns the read data to its argument, whereas when the statement (b) is used, the `get()` function returns the data read. The data is assigned to the variable present on the left-hand side of the assignment operator. These functions are members of I/O stream classes and can be called using object.

put () function

The `put()` function is used to display the string on the screen. It is a member of `ostream` class. The syntax of `put()` is as follows:

- (a) `cout.put ('A');`
- (b) `cout.put (x);`

The statement (a) displays the character 'A' on the screen and the statement (b) displays the contents of variable `x` on the screen. If an integer is used as an argument, its corresponding *ASCII* value is displayed. Few examples are illustrated below.

3.16 Write a program to display the character on the screen using `put()` function.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.put('C');
    cout.put ('+');
    cout.put ('+');
    return 0;
}
```

OUTPUT**C++**

Explanation: The `cout.put()` statement displays one character at a time on the screen. In this program, three characters are displayed on the screen using `cout.put()` statement.

3.17 Write program to use escape sequence with `cout.put()` statement.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.put('1');
    cout.put ('\n');
    cout.put ('2');
    return 0;
}
```

OUTPUT**1**
2

Explanation: The escape sequences such as `'\n'`, `'\t'`, etc., can be used with `cout.put()` statement. The escape sequences are combinations of two characters. The output of the program displays the number 1 and 2 in two separate lines. The statement `cout.put('\n')` splits a line.

3.18 Write a program to use multiple put () statements with single cout object and display the characters.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.put('C').put('+').put('+');
    return 0;
}
```

OUTPUT**C++**

Explanation: In the above program, the single object cout is used followed by sequence of put () statement. The put () statements are separated by dot operators. In this way, multiple statements can be combined.

3.19 Write a program to read character using get () and display it using put ().

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char ch;
    clrscr();
    cout<<"\n Enter a character :";
    cin.get(ch);
    cout<<"\n Entered character was :";
    cout.put(ch);
    return 0;
}
```

OUTPUT**Enter a character : C****Entered character was : C**

Explanation: In the above program, character variable ch is declared. The first cout statement displays message "Enter a character:" on the screen. The cin.get () function activates input stream and character entered by the user is stored in the variable ch. The cout . put () statement displays the character on the screen.

3.20 Write a program to read characters using a sequence of `get()` statements and display the characters read using a sequence of `put()` statements.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char ch[3];
    clrscr();
    cout<<"\n Enter characters :";
    cin.get(ch[0]).get(ch[1]).get(ch[2]);
    cout<<"\n Characters Entered :";
    cout.put(ch[0]).put(ch[1]).put(ch[2]);
    return 0;
}
```

OUTPUT**Enter characters : cpp****Characters entered : cpp**

Explanation: In the above program, a character array `ch[3]` is declared. The sequence of `get()` and `put()` functions are used to read and display the characters. The `get()` function reads characters and stores in array `ch[3]`. The `put()` function displays the same on the screen.

3.21 Write a program to read and display the string. Use `get()` and `put()` functions.

```
#include<iostream.h>
#include<conio.h>
#include<stdio.h>

int main()
{
    clrscr();
    char j=0;
    char x[20];
    cout<<"\n Enter a string :";
    while (x[j++]!='\n')
        cin.get(x[j]);
    j=0;
    cout<<"\n Entered string :";
    while (x[j++]!='\n')
        cout.put(x[j]);
    return 0 ;
}
```



OUTPUT

Enter a string : Programming

Entered string : Programming

Explanation: In the above program, the character array `x[]` is declared. The first `while` loop reads characters using `cin.get()` function through the keyboard. When a user presses the enter key, the `while` loop terminates. The second `while` loop displays the characters read using the function `cout.put()`. The output of the program is as shown above.

getline() and write() functions

getline() function

The `getline()` and `write()` functions are useful in string input and output. The `getline()` functions read the string including white space. The `cin()` function does not allow to enter the string with blank spaces. The input reading is terminated when a user presses the enter key. The new line character is accepted but not saved and replaced with the null character. The object `cin` calls the function as follows.

```
cin.getline (variable, size);
```

where the variable name may be any character array name and the size is the size of the array.

write() function

The `write()` function is used to display the string on the screen. Its format is similar to `getline()` function, but the function is exactly the opposite. The syntax is as follows.

```
cout.write (variable, size);
```

where the variable name may be a character type and size is the size of the character arrays. The `cout.write()` statement displays only a specified number of characters given in the second argument, though the actual string may be more in length. If the size of the array is larger than the actual string length, then the size argument contains the actual size of the array. In this case, the `getline()` displays blank spaces for the remaining unfilled elements. The following program illustrates both the functions.

3.22 Write a program to display a string using `cout.write()` statements.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.write ("INDIA",6);
    cout.write ("IS",3);
```



```
    cout.write ("GREAT", 5);  
    return 0;  
}
```

OUTPUT

INDIA IS GREAT

Explanation: In this program, the first statement displays “INDIA” followed by one blank space. This is because the argument value is greater by one than the actual string length.

Similarly, the second statement displays “IS” followed by one blank space. The reason is same.

The last statement displays “GREAT.” Here, the argument value and the string lengths are same. Hence, no blank spaces are displayed.

In case the argument value is less than the actual string length, the complete string will not be displayed. The number of characters of the string displayed depends on the value of the given argument.

3.23 Write a program to show the effect if less argument is given than the actual string length in the `cout.write()` statement.

```
#include<iostream.h>  
#include<conio.h>  
  
int main()  
{  
    clrscr();  
    cout.write ("SUNDAY", 3);  
    return 0;  
}
```

OUTPUT

SUN

Explanation: In the above program, the `cout.write()` will not display the complete string. The statement displays the characters according to the value of the second argument. The value of the second argument is three. Hence, instead of six characters only three characters are displayed and the remaining characters are skipped.

3.24 Write a program to read a string using `getline()` function and display it using the `write()` statement.

```
#include<iostream.h>  
#include<conio.h>  
#include<string.h>
```

```
int main()
{
    clrscr();
    char x[30];
    cout<<"\n Enter any string :";
    cin.getline(x,30);
    cout<<"\n Entered string : " <<x;
    cout<<"\n Entered string :";
    cout.write(x,30);
    cout<<"\n Entered string :";
    cout.write(x,strlen(x));
    return 0 ;
}
```

OUTPUT

```
Enter any string : C++ is advanced C
Entered string : C++ is advanced C
Entered string : C++ is advanced C h >& )
Entered string : C++ is advanced C
```

Explanation: In the above program, the `x[]` is a character array. The `getline()` function reads the string through the keyboard. The `getline()` function accepts the string including spaces. The `cout()` statement displays the string including white spaces. The first `write()` statement displays the string with garbage values. This is because the string length is less than the actual size of the array. In the second `write()` statement, `strlen()` function calculates the length of the string and that length is used as an argument in the `write()` statement. This statement displays the entered string without any garbage collection.

The `cin()` statement cannot accept a string including spaces. It accepts only a single word. The `cout()` statement displays the string read through `cin()` and `getline()` functions, that is, it can display the string with or without blank spaces. The `write()` statement displays the string according to the specified size. It displays the string with or without blank spaces. The `write()` does not support any escape sequence.

3.25 Write a program to display the string using different arguments in `write()` statement.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
```

```
int main()
{
```



```

clrscr();
char a,x[30];
cout<<"\n Enter any string :";
cin.getline(x,30);
cout<<"\n Entered string : \n";
for (a=0;a<=strlen(x);a++)
{
cout<<"\n";
cout.write(x,a);
}
return 0 ;
}

```

OUTPUT

Enter any string : C Plus Plus

Entered string :

C
C
C P
C Pl
C Plu
C Plus
C Plus
C Plus P
C Plus Pl
C Plus Plu
C Plus Plus

Explanation: In the above program, two strings are entered in character arrays `x[]` and `a[]`. The successive `write()` statement in one line displays the string one after another. Thus, we can use multiple `write()` statements followed by a single `cout` object.

3.9 MEMBER FUNCTIONS OF THE `istream` CLASS

The `istream` contains the following functions that can be called using `cin` object.

peek(): It returns the succeeding character without extraction. For example,

```
cin.peek()=='#';
```

where `cin` is an object and `'#'` is the symbol that is to be caught in the stream.

ignore(): The member function `ignore()` has two arguments, maximum number of characters to avoid and the termination character. For example,

```
cin.ignore (1, '#');
```

The statement ignores character 1 till character `'#'` is found.

3.26 Write a program to demonstrate the use of `peek()` and `ignore()` functions.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char c;
    clrscr();
    cout<<"enter text (press F6 to end :";
    while (cin.get(c))
    {

cout<<c;
        while (cin.peek()=='#')
        {
            cin.ignore(1, '#');
        }
    }
    return 0;
}
```

OUTPUT

enter text (press F6 to end : ABCDEFG###HIJK
ABCDEFGHIJK

Explanation: In the above program, the `cin.get()` function continuously reads characters through the keyboard till the user presses F6. The `cout` statement inside the loop displays the contents of variable `c` on the console. The `cin.peek()` statement checks the variable `c`. If the variable `c` contains '#', it is ignored from the stream and not displayed on the screen.

putback(): The `putback()` replaces the given character into the input stream. For example,

`cin.putback ('*');`

where `cin` is an object and '*' is the symbol which is replaced in the stream.

3.27 Write a program to demonstrate the use of `putback()` function.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char c;
    clrscr();
```

```
    cout<<"enter text (press F6 to end :";
    while (cin.get(c))
    {
        if (c=='s')
            cin.putback('S');
        cout.put(c);
    }
    return 0;
}
```

OUTPUT

enter text (press F6 to end :

c plus plus^Z

c plusS plusS

Explanation: In the above program, the `cin.get()` function continuously reads characters through the keyboard till the user presses F6. The `cout` statement inside the loop displays the contents of the variable `c` on the console. The `if` statement checks the contents of the variable `c`. If variable `c` contains a small letter 's', the `putback()` statement sends capital 'S' in the stream. The small 's' is replaced with capital 'S'. The contents displayed will be with capital 'S'.

gcount(): This function returns the number of unformatted characters extracted from the input stream. The last statement should be `get()`, `getline()`, or `read()`.

3.28 Write a program to demonstrate the use of `gcount()` function.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char text[20];
    int len;
    clrscr();
    cout<<"Enter text :";
    cin.getline(text,20);
    len=cin.gcount();
    cout<<"The numbers of characters extracted are:"<<len;
    return 0;
}
```

OUTPUT

Enter text : Virtual

The numbers of characters extracted are : 8

Explanation: In the above program, the `getline()` function reads text through the keyboard. The `gcount()` function returns the number of characters extracted from stream to variable `len`. It also counts the null character. The `cout` statement displays the value of variable `len` on the screen.

3.29 Write a program to perform the operation with `peek()` and `putback()`.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char c;
    clrscr();
    cout<<"enter text (press F6 to end :";
    while (cin.get(c))
    {
        if (c==' ')
            cin.putback ('.');
        else
            cout<<c;
        while (cin.peek()=='#') cin.ignore(1,'#');
    }
    return 0;
}
```

OUTPUT

Enter text (press F6 to end: One! Two! Three! Four!
One..Two..Three..Four.

Explanation: In the above program, the `cin.get()` function reads data through the keyboard using first while loop. The if condition checks for blank space in the entered text. If space is found, the `putback()` statement replaces space dot (.). The `putback()` sends the given character in the input stream. The `peek()` also checks the '#' symbol in the entered text, if it is found, the `ignore()` statement ignores the character and the character will not be displayed on the screen. The program is terminated when the key F6 or `ctrl+z` is pressed.

3.10 FORMATTED CONSOLE I/O OPERATIONS

C++ provides various formatted console I/O functions for formatting the output. They are of three types.

- (1) Ios class function and flags
- (2) Manipulators
- (3) User-defined output functions

The `ios` grants operations common to both input and output. The classes (`istream`, `ostream`, and `iostream`) derived from `ios` are special I/O with high-level formatting operations: The `iostream` class is automatically loaded in the program by the compiler. Figure 3.4 describes the hierarchy of stream classes.

- (a) `istream` performs formatted input.
- (b) `ostream` performs formatted output.
- (c) `iostream` performs formatted input and output.

`streambuf` allows an abstraction for connecting to a physical device. Classes derived from it work with files, memory, etc. The `ios` communicates to a `streambuf`. It keeps information on the state of the `streambuf` (good, bad, eof, etc.) and saves flags for use by `istream` and `ostream`.

- The `streambuf` class controls the buffer and its related member functions. It allows the ability to fill, flush, and empty the buffer.
- The `streambuf` is an object of `ios` class. The base class of input and output stream classes is `ios` class.
- The `istream` and `ostream` classes are derived classes of `ios` class and control input and output streams.
- The `iostream` class is a derived class of `istream` and `ostream`. It provides input and output functions to control console operations.

Table 3.4 describes the functions of `ios` class in brief.

Table 3.4 `ios` Class Function

Function	Working
<code>width()</code>	To set the required field width. The output will be displayed in given width.
<code>precision()</code>	To set number of decimal point to a <code>float</code> value.
<code>fill()</code>	To set a character to fill in the blank space of a field.
<code>setf()</code>	To set various flags for formatting output.
<code>unsetf()</code>	To remove the flag setting.

(1) `ios::width` (member functions)

The `width()` function can be declared in two ways.

- (a) `int width();`
- (b) `int width(int);`

`int width();`

If this function is declared as given above, it returns the present width setting.

`int width(int);`

If this function is declared as given above, it sets the width as per the given integer and returns the previous width setting. The setting should be reset for each input or output value if a width other than the default is desired.

3.30 Write a program to set column width and display the characters at specified position.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout . width(5);
    cout<<"A";
    cout . width(15);
    cout<<"B";
    return 0;
}
```

OUTPUT**A B**

Explanation: The first `cout.width()` statement sets the width 5. The `cout` statement sets the column width position at 5 and the `cout` statement displays the character "A" at column 5. Similarly, the column width is set 15 and the character 'B' is displayed at column 15.

3.31 Write a program to use both the formats of `width()` function and display the result.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.width(50);
    int x=cout.width(1);
    cout<<x;
    return 0;
}
```

OUTPUT**50**

Explanation: In the above program, the `width()` function call sets the width at column 50. The second `width()` function call sets the width at column 1 and returns the previous

setting, that is 50. The integer `x` collects this value. The `cout()` statement displays the number 50 at column 1.

(2) `ios::precision`

This function can be declared in two ways.

(a) `int precision (int);`

(b) `int precision();`

`int precision (int);`

If the function is declared as given above, it sets the floating-point precision and returns the previous setting. The precision should be reset for every value being output if we want a precision result other than the default.

`int precision();`

If the function is used as given above, it returns the current setting of floating-point precision.

3.32 Write a program to set precision to two and display the float number

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.precision(2);
    cout<<3.1452;
    return 0;
}
```

OUTPUT

3.14

Explanation: In the above program, the `cout.precision()` statement sets float point precision to 2. The `cout` statement displays 3.14 instead of 3.1452.

3.33 Write a program to set number of precision points. Display the results of 22/7 in different precision settings.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x=0;
    float pi;
```

```

    for (x=10;x>=1;x--)
    {
        pi=(float)22/7;
        cout.precision(x);
        cout<<"\n"<<pi;
    }
    x=cout.precision(1);
    cout<<"\n\n Previous Setting : "<<x;
    return 0;
}

```

OUTPUT

3.1428570747

3.142857075

3.14285707

3.1428571

3.142857

3.14286

3.1429

3.143

3.14

3.1

Previous Setting : 1

Explanation: In the above program, the for loop executes from 10 to 1 in reverse order. Each time the equation $22/7$ is calculated and the result is stored in the variable `pi`. The precision is set according to the value of variable `x`. The output of the program is as given above.

(3) `ios::fill`

This function can be declared in two ways.

- (a) `char fill();`
- (b) `char fill(char);`

`char fill();`

If the function is used as given above, it returns the current setting of fill character.

`char fill(char);`

If the function is used as given above, it resets the fill character and returns the previous setting.

3.34 Write a program to fill empty spaces in a line with a specific symbol (*).

```

#include<iostream.h>
#include<conio.h>

int main()

```



```

{
    clrscr();
    cout.fill('*');
    cout.width(10);
    cout<<"H";
    return 0;
}

```

OUTPUT

*****H

Explanation: In the above program, the statement `cout.fill('*');` fills the blank spaces with * symbols. The `cout.width()` sets the width to 10 and the character 'H' is displayed at column 10. The column 1 to 9 remains blank. The symbol '*' is displayed instead of blank spaces. This effect is due to `cout.fill()` statement as shown below.

*	*	*	*	*	*	*	*	*	H
---	---	---	---	---	---	---	---	---	---

3.35 Write a program to fill blank spaces with different symbols.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.fill('/');
    cout.width(20);
    cout<<"WEL"<<endl;
    cout.fill('-');
    cout.width(10);
    cout<<"DONE";
    return 0;
}

```

OUTPUT

//////////WEL
-----DONE

Explanation: This program is similar to the last one. Here, two different characters are used to fill the blank spaces in the specified width. The width is specified using `cout.width()` statement. The output can be observed as follows.

/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	/	W	E	L
-	-	-	-	-	-	-	D	O	N	E										

**TIP**

The character '\' (back slash) is not allowed in `cout.fill()` statement to fill the blanks. This is because it is used with escape sequences.

3.36 Write a program to show the effect `cout.fill()`.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout<<"Begin :";
    cout.width(15);
    cout<<"ABC";
    cout<<"\n";
    cout<<"Begin :";
    cout.width(15);
    cout.fill('#');
    cout<<"ABC";
    return 0;
}
```

OUTPUT

Begin : **ABC**

Begin : #####ABC

Explanation: The first line displays ABC following blank spaces because the `fill()` is not used. In the second statement, the symbol '#' is displayed because the fill character is set to '#'.

Begin							A	B	C
Begin	#	#	#	#	#	#	A	B	C

3.37 Write a program to fill the unused filled area with '*'

```
#include<iostream.h>
#include<conio.h>

int main()
```

```

{
    clrscr();
    int x=0,j=0;
    float pi;
    cout.fill('*');
    for (x=10;x>=1;x--)
    {
        pi=(float)22/7;
        cout.width(j++);
        cout.precision(x);
        cout<<"\n"<<pi;
    }
    cout.width(j++);
    cout.precision(x);
    cout<<"\n"<<pi;
    x=cout.fill();
    cout<<"\n\n Fill setting : "<<(char)x;
    return 0;
}

```

OUTPUT

```

3.1428570747
3.142857075*
3.14285707**
3.1428571***
3.142857****
3.14286*****
3.1429*****
3.143*****
3.14*****
3.1*****
3.142857
Fill setting :*

```

Explanation: In the above program, the precision setting and display of floating point number using precision is the same as in the previous example. In the `for` loop, each time `width()` is set to the value of variable `j`. The value of `j` increases at each iteration. The floating precision number decreases from top to bottom, and blank spaces remain in the specified field area by the statement `width()`. The unused area is filled with `'*'`. The setting of fill character is done before the `for` loop statement. The fill character may be set to any character. The `fill()` statement after `for` loop body displays the character used for filling.

(4) `ios::setf`

This function can be declared in two ways.

- (a) `long setf (long sb, long f);`
- (b) `long setf (long);`
`long setf (long sb, long f);`

The bits according to those marked in variable `f` are removed in the data member `x_flags`, and then reset to be those marked in variable `sb`. Using the constants in the formatting flag enumeration of class `ios` can specify the value of variable `sb`.

`long setf (long);`

If the above declaration is used in the program, it sets the flags according to those marked in the given `long`. The flags are set in the data member `x_flags` of class `ios`. Using the constants in the formatting flags can specify the long enumeration of class `ios`. It returns the previous settings.

3.11 BIT FIELDS

The `ios` class contains the `setf()` member function. The flag indicates the format design. The syntax of the `unsetf()` function is used to clear the flags. The syntaxes of the function are as follows.

Syntax: `cout.setf (v1, v2);`

Syntax: `cout.unsetf(v1);`

where variable `v1` and `v2` are two flags. Table 3.5 describes the flag, and bit-field setting can be used with this function. The `unsetf()` accepts setting of `v1` and `v2` arguments.

Table 3.5 Flags and Bits

Format	Flag (V1)	Bit Field (V2)
Left justification	<code>ios::left</code>	<code>ios::adjustfield</code>
Right justification	<code>ios::right</code>	<code>ios::adjustfield</code>
Padding after sign and base	<code>ios::internal</code>	<code>ios::adjustfield</code>
Scientific notation	<code>ios::scientific</code>	<code>ios::floatfield</code>
Fixed point notation	<code>ios::fixed</code>	<code>ios::floatfield</code>
Decimal base	<code>ios::dec</code>	<code>ios::basefield</code>
Octal base	<code>ios::oct</code>	<code>ios::basefield</code>
Hexadecimal base	<code>ios::hex</code>	<code>ios::basefield</code>

(a) `ios::adjustfield`

It is a data member and used with `setf()` function to arrange padding to the left or right, or for internal fill. It can be declared as follows.

`static const long adjustfield;`

(b) `ios::floatfield`

It is a data member and used with `setf()` function to set the float point notation to scientific or fixed. It is declared as follows.

`static const long floatfield;`

(c) `ios::basefield`

It is a data member and used with `setf()` function and used with `setf()` function to set the notation to a decimal, octal, or hexadecimal base. It is declared as follows.

```
static const long basefield;
```

3.38 Write a program to display the message left and right justified.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.fill('=');
    cout.setf(ios::right, ios::adjustfield);
    cout.width(20);
    cout<<"Figure" << "\n";
    cout.setf(ios::left, ios::adjustfield);
    cout.width(20);
    cout<<"Figure" << "\n";
    return 0;
}
```

OUTPUT

```
=====Figure
Figure =====
```

Explanation: In the above program, the fill character is set to sign "=". The `setf()` is set to right justified. The column width is set to 20. The message "Figure" appears at the 20th column. Before the message the blank space is filled with the sign "=". Again the justification property is set to left in `setf()` function. The text appears left justified. The output can be observed as follows.

=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	F	i	g	u	r	e
F	i	g	u	r	e	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=

3.39 Write a program to display the number in scientific format with sign.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.fill('=');
```

```

    cout.setf(ios::internal, ios::adjustfield);
    cout.setf(ios::scientific, ios::floatfield);
    cout.width(15);
    cout<<- 3.121;
    return 0;
}

```

OUTPUT

- ===== 3.121e+00

Explanation: In the above program, the fill character is set to sign “=”. The `setf()` properties are set to `internal` and `scientific`. The scientific properties display the number in scientific (e) format. The internal properties display the sign before blank spaces. If the setting were removed, the output would be as given below.

Effect of statement

```
cout.setf (ios::internal, ios::adjustfield);
```

-	=	=	=	=	=	3	.	1	2	1	e	+	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The output without this statement would be as follows.

=	=	=	=	=	-	3	.	1	2	1	e	+	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

The property `ios::fixed` displays the float number without scientific format, though the number is big. If the floating-point number is more than 6, then the extras are ignored. This format displays floating point up to 6 only.

3.40 Write a program to convert a decimal number to hexadecimal and octal format.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.setf(ios::hex, ios::basefield);
    cout<<"\n Hexadecimal of 184 is :"<<184;
    cout.setf(ios::oct, ios::basefield);
    cout<<"\n Octal of 15 is :"<<15;
    cout.setf(ios::dec, ios::basefield);
    cout<<"\n Decimal of 0xfe is :"<<0xfe;
    return 0;
}

```


OUTPUT

Hexadecimal of 184 is : b8

Octal of 15 is : 17

Decimal of 0xfe is : 254

Explanation: In the above program, the properties of `setf()` function are set to `ios::hex`, `ios::oct`, and `ios::dec`. After this setting, the decimal number given in `cout()` statement will convert to its hexadecimal and octal equivalent, respectively. The hexadecimal number 0xfe is converted to its decimal equivalent.

3.12 FLAGS WITHOUT BIT FIELDS

The flags described in Table 3.6 have no corresponding bit fields. The programs on these functions are illustrated as follows.

Table 3.6 Flags without Bit Fields

Flag	Working
<code>ios::showbase</code>	Uses base indicator on output
<code>ios::showpos</code>	Displays + preceding positive number
<code>ios::showpoint</code>	Shows trailing decimal point and zeros
<code>ios::uppercase</code>	Uses capital case for hex output
<code>ios::skipws</code>	Skips white space on input
<code>ios::unitbuf</code>	Flushes all streams after insertion
<code>ios::stdio</code>	Flushes <code>stdout</code> and <code>stderr</code> later insertion
<code>ios::uppercase</code>	Uses capital characters for scientific and hexadecimal values
<code>ios::stdio</code>	Adjusts the stream with C standard input and output
<code>ios::boolalpha</code>	Converts Boolean values to text ("true" or "false")

3.41 Write a program to use the various settings given in the table.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.setf(ios::showpos);
    cout<<1254;
    cout.setf(ios::showpoint);
    cout<<"\n"<<1254.524;
    cout.setf(ios::hex,ios::basefield);
    cout<<"\n"<<184;
    cout.setf(ios::uppercase);
    cout<<"\n"<<184;
```

```
    return 0;
}
```

OUTPUT

```
+1254
+1254.524000
0xb8
0XB8
```

Explanation: In the above program, the setting `ios::showpos` displays the + sign before the number 1254. The setting `ios::showpoint` displays the trailing zeros after the number 1254.524. The setting `ios::hex` converts the decimal number to hexadecimal. It uses small letters. The setting `ios::uppercase` displays the hexadecimal number in uppercase.

3.13 MANIPULATORS

The output formats can be controlled using manipulators. The header file `iomanip.h` has a set of functions. The effect of these manipulators is similar to `ios` class member functions. Every `ios` member function has two formats. The first format is used for setting and the second format is used to understand the previous setting. But the manipulator does not return the previous setting. The manipulator can be used with `cout()` statement as follows.

```
cout<<m1 <<m2 <<v1;
```

Here, `m1` and `m2` are two manipulators and `v1` is any valid C++ variable.

Table 3.7 describes the most useful manipulators. The manipulators `hex`, `dec`, `oct`, `ws`, `endl`, and `flush` are defined in `iostream.h`. The manipulators `setbase`, `width()`, `fill()`, etc., that require an argument are defined in `iomanip.h`.

Table 3.7 Pre-defined Manipulators

Manipulator	Function
<code>setw (int n)</code>	The field width is fixed to n
<code>setbase</code>	Sets the base of the number system
<code>setprecision(int p)</code>	The precision point is fixed to p
<code>setfill (char f)</code>	The fill character is set to the character stored in variable f
<code>setiosflags(long l)</code>	Format flag is set to l
<code>resetiosflags(long l)</code>	Removes the flags indicated by l
<code>endl</code>	Splits a new line
<code>skipws</code>	Omits white space on input
<code>noskipws</code>	Does not omit white space on input
<code>ends</code>	Adds null character to close an output string
<code>flush</code>	Flushes the buffer stream
<code>lock</code>	Locks the file associated with the file handle
<code>ws</code>	Omits the leading white spaces present before the first field
<code>hex, oct, dec</code>	Displays the number in hexadecimal, octal, and decimal format

3.42 Write a program to display formatted output using manipulators.

```
#include<iostream.h>
#include<iomanip.h>
#include<conio.h>

int main()
{
    clrscr();
    cout<<setw(10) <<"Hello"<<endl;
    cout<<setw(15) <<setprecision(2) <<2.5555;
    cout<<setiosflags(ios::hex);
    cout<<endl<<"Hexadecimal of 84 is :"<<84;
    return 0;
}
```

OUTPUT**Hello****2.56****Hexadecimal of 84 is : 54**

Explanation: In the above program, the manipulator `setw (10)` sets the field width to 10. The message "hello" is displayed at 10th column. The `endl` inserts a new line. In the second `cout ()` statement, the `setprecision (2)` manipulator sets the decimal point to 2. The number 2.5555 will be displayed as 2.56 at column 15. The manipulator `setiosflag` sets the hexadecimal setting for the display of number. The last `cout ()` statement displays the equivalent hexadecimal of 84, that is, 54.

3.43 Write a program to display the given decimal number in hexadecimal and octal format.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

int main()
{
    clrscr();
    int x=84;
    cout<<"\n Hexadecimal Number : "<<hex<<x;
    cout<<"\n Octal Number : "<<oct<<x;
    return 0;
}
```

OUTPUT**Hexadecimal Number : 54****Octal Number : 124**

Explanation: In the above program, the integer variable x is declared and initialized with 84. The first cout statement displays the decimal number to its equivalent hexadecimal number. The manipulator hex converts a decimal number to its hexadecimal equivalent. The second cout statement converts a decimal number to its equivalent octal number.

3.44 Write a program to read a number in hexadecimal format using cin statement.

Display the number in decimal format.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x;
    cout<<"\n Enter Hexadecimal Number : ";
    cin>>hex>>x;
    cout<<"\n Decimal Number : "<<dec<<x;
    return 0;
}
```

OUTPUT

Enter Hexadecimal Number : 31

Decimal Number : 25

Explanation: In the above program, the cin statement reads a number in hex format. The cout statement displays its equivalent decimal number with the use of dec manipulator.

3.45 Write a program to demonstrate the use of endl manipulator.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout<<" Demo of endl";
    endl(cout);
    cout<<" It splits a line" ;
    return 0;
}
```

OUTPUT

Demo of endl

It splits a line

Explanation: In the above program, endl manipulator is used to split the line. The two strings are displayed on two separate lines.

3.46 Write a program to demonstrate the use of flush () statement.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char text[20];
    clrscr();
    cout<<"Enter text :";
    cin.getline(text,20);
    cout<<"Text entered :"<<text;
    flush(cout);
    return 0;
}
```

OUTPUT

Enter text : Buffer
Text entered : Buffer

Explanation: In the above program, the statement flush (cout) flushes the buffer. This statement can be used as cout<<flush. For more information on buffer, read Section 3.4.

3.14 USER-DEFINED MANIPULATORS

The programmer can also define his or her own manipulator according to the requirement of the program. The syntax for defining manipulator is as follows.

```
ostream & m_name ( ostream & o )
{
    statement1;
    statement2;
    return o;
}
```

The m_name is the name of the manipulator.

3.47 Write a program to create manipulator equivalent to '\t'. Use it in the program and format the output.

```
#include<iostream.h>
#include<iomanip.h>
#include<conio.h>
```

```
ostream & tab (ostream & o)
{
    o <<"\t";
    return o;
}
void main()
{
    clrscr();
    cout<<1<<tab<<2 <<tab<<3;
}
```

OUTPUT

1 2 3

Explanation: Figure 3.7 illustrates the working of the program.

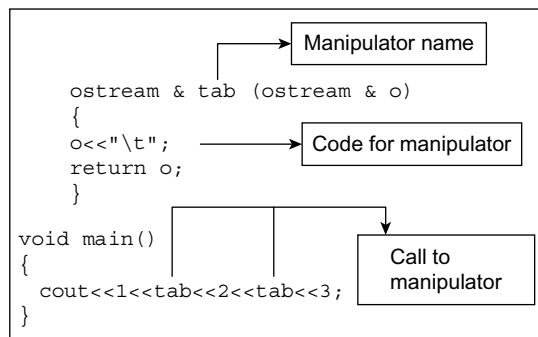


Fig. 3.7 Working of tab manipulator

In the above program, tab named manipulator is defined. The definition of tab manipulator contains the escape sequence '\t'. Whenever we call the tab manipulator, the '\t' is executed and we get the effect of tab.

3.48 Write a program to display a message using manipulator.

```
#include<iostream.h>
#include<iomanip.h>
#include<conio.h>
ostream & N (ostream & o)
{
    o <<"Negative Number";
    return o;
}
ostream & P (ostream & o)
{
```

```

    o <<"Positive Number";
    return o;
}
void main()
{
    int x;
    clrscr();
    cout<<"\n Enter a Number :";
    cin>>x;
    if (x<0)
        cout<<x <<" is" <<N;
    else
        cout<<x <<" is" <<P;
}

```

OUTPUT

Enter a Number : -15
-15 is Negative Number

Explanation: In the above program, two manipulators N and P are created. When called, N displays the message "Negative number" and P displays the message "Positive number" .

3.15 MANIPULATOR WITH ONE PARAMETER

The prototype declared in the `iomanip.h` as described earlier allows us to define individual set of macros. The manipulator can be created without the use of `int` or `long` argument.

3.49 Write a program with one parameter to display the string in the middle of the line.

```

#include<iostream.h>
#include<iomanip.h>
#include<string.h>
#include<conio.h>
ostream& fc (ostream& ost, int iw)
{
    for (int k=0;k<((75-iw)/2);k++)
        ost <<" ";
    return (ost);
}
OMANIP (int) middle (int iw)
{
    return OMANIP (int) (fc,iw);
}

```

```
int main()
{
    clrscr();
    char *m=" * Well come *";
    cout<<middle (strlen(m)) <<m;
    return 0;
}
```

OUTPUT

**** WEL COME ****

Explanation: In the above program, `middle` is a user-defined parameterized manipulator. It receives a value `strlen (m)`, that is, string length. The header file `IOMANIO.H` defines a macro, `OMANIP (int)`. It is expanded to `class_OMANIP_int`. Due to this, the definition consists of a constructor and an overloaded `ostream` insertion operator. When `middle()` function is added into the stream, it invokes the constructor which generates and returns an `OMANIP_int` object. The `fc()` function is called by the object constructor.

3.16 MANIPULATORS WITH MULTIPLE PARAMETERS

In this type of manipulators, multiple arguments are passed to the manipulator. The following program explains it.

3.50 Write a program to create a manipulator with two arguments.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
#include<math.h>

struct w_p
{
    int w;
    int p;
};

IOMANIPdeclare (w_p);
static ostream& ff(ostream& os, w_p w_p)
{
    os.width (w_p.w);
    os.precision(w_p.p);
    os.setf(ios::fixed);
    return os;
}

OMANIP (w_p) column (int w, int p)
{
```



```

    w_p w_p;
    w_p.w=w;
    w_p.p=p;
    return OMANIP (w_p) (ff,w_p);
}
int main()
{
    clrscr();
    double n,sq,sqr;
    cout<<"number\t" <<"square\t" <<"\tsquare root\n";
    cout<<"===== \n";
    n=1.0;
    for (int j=1;j<16;j++)
    {
        sq=n*n;
        sqr=sqrt(n);
        cout.fill('0');
        cout<<column(3,0)<<n <<"\t";
        cout<<column(7,1) <<sq <<"\t\t";
        cout<<column(7,6) <<sqr <<endl;
        n=n+1.0;
    }
    return 0;
}

```

OUTPUT

number	square	square root
001	0000001	0000001
002	0000004	1.414214
003	0000009	1.732051
004	0000016	0000002
005	0000025	2.236068
006	0000036	2.44949
007	0000049	2.645751
008	0000064	2.828427
009	0000081	0000003
010	0000100	3.162278
011	0000121	3.316625
012	0000144	3.464102
013	0000169	3.605551
014	0000196	3.741657
015	0000225	3.872983

Explanation: The above program is similar to the previous one. The only difference is that here two parameters are used. The user-defined manipulator is assigned two integer values. The first argument decides the number of spaces and the second argument decides the number of decimal places. After initializing the `w_p` structure, the constructor is called. The constructor creates and returns a `_OMANIP` object. The output of the program is shown above.

3.17 MORE PROGRAMS

3.51 Write a program to calculate the simple interest and total amount, input principal amount, period, and rate of interest.

```
#include<iostream.h>
#include<conio.h>
int main()
{
    clrscr();
    int p_amount; // principle amount
    int period; // period in years
    int i_rate; // interest rate
    int interest; // interest
    int t_amount; // total amount
    cout<<endl<<"Enter Principle amount :";
    cin>>p_amount;
    cout<<"Enter period (years) :";
    cin>>period;
    cout<<"Enter interest rate :";
    cin>>i_rate;
    interest=(p_amount*period*i_rate) / 100;
    cout<<"Interest :"<<interest;
    t_amount=p_amount+interest;
    cout<<endl<<"Total Amount : " <<t_amount;
    return 0;
}
```



OUTPUT

```
Enter Principle amount : 10000
Enter period (years) : 3
Enter interest rate : 5
Interest : 189
Total Amount : 10189
```

Explanation: In the above program, integer variables `p_amount`, `period`, `i_rate`, `interest`, and `t_amount` are declared. The principal amount, period in years, and rate of interest are read through the keyboard using respective variable with `cin` statement.

The interest is calculated by the statement `interest=(p_amount*period*i_rate)/100` and the interest calculated is assigned to variable `interest`. The total amount is calculated by adding interest in the principle amount.

3.52 Write a program to set width and display the integer number.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.width(7);
    cout<<20123<<endl;
    cout.width(10);
    cout<<12541;
    return 0;
}
```

Explanation: In the above program, the `cout.width(7)` statement sets width to 7. The `cout` statement displays the five digits. The total number of digits are less than the total width. Hence, two blank spaces are displayed at the beginning as shown below.

		2	0	2	1	3
--	--	---	---	---	---	---

					2	0	2	1	3
--	--	--	--	--	---	---	---	---	---

3.53 Write a program to demonstrate use of `showpos` and `showpoint` flags.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
int main()
{
    clrscr();
    cout.setf(ios::showpos);
    cout.setf(ios::showpoint);
    cout.setf(ios::internal,ios::adjustfield);
    cout.precision(2);
    cout.width(9);
    cout<<587.4;
    return 0;
}
```

Explanation: The output of the program can be understood from the following table.

+			5	8	7	.	4	0
---	--	--	---	---	---	---	---	---

The `showpos` flag displays the positive sign at the beginning. The `showpoint` option displays decimal point and trailing zeros. The `internal` and `adjustfield` flags add blank spaces between the sign and the number.

3.54 Write a program to display results in right justification.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

int main()
{
    clrscr();
    int k=0;
    long double n=1.00,f=1.00;
    cout.precision(0);
    cout.setf(ios::fixed);
    while (k<15)
    {
        f*=n++;
        cout.width(15);
        cout<<f<<"\n";
        k++;
    }
    return 0;
}
```

OUTPUT

```

      1
      2
      6
     24
    120
    720
   5040
  40320
 362880
3628800
39916800
479001600
6227020800
87178291200
1307674368000
```

Explanation: In the above program, the integer variable `k` is initialized to 0 and long double variables `n` and `f` are initialized to 1.00. The precision is set to 0. The `setf()` function sets the precision to fixed position. The statement `cout.width(15)` fixes the field width. Thus, using while loop repetitive operations are performed. The output of the program is given above.

3.55 Write a program to display square and cube of numbers from 1.5 to 16.5 in table format.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
#include<math.h>

int main()
{
    clrscr();
    int k=0;
    float num=1.5,sq,cb;
    cout<< "Number\t\t" <<"Square\t\t" <<"Cube\n";
    cout<< " =====\n";
    cout.setf(ios::fixed);
    while (k<16)
    {
        sq=pow(num,2);
        cb=pow(num,3);
        cout.fill('*');
        cout.width(4);
        cout.precision(0);
        cout<<num <<"\t\t";
        cout.width(6);
        cout.precision(2);
        cout<<sq<<"\t\t";
        cout.width(8);
        cout.precision(2);
        cout<<cb<<endl;
        num+=1;
        k++;
    }
    return 0;
}
```

OUTPUT

Number	Square	Cube
=====	=====	=====
*1.5	**2.25	***3.38

*2.5	**6.25	***15.63
*3.5	*12.25	***42.87
*4.5	*20.25	***91.12
*5.5	*30.25	**166.38
*6.5	*42.25	**274.62
*7.5	*56.25	**421.88
*8.5	*72.25	**614.13
*9.5	*90.25	**857.37
10.5	110.25	*1157.63
11.5	132.25	*1520.88
12.5	156.25	*1953.12
13.5	182.25	*2460.37
14.5	210.25	*3048.63
15.5	240.25	*3723.88
16.5	272.25	*4492.12

Explanation: In the above program the variable num is initialized to 1.5. The while loop is executed repetitively and square and cube of the number are calculated using the pow() function. The functions cout.width() and cout.precision() set the field width and the number of floating points. The output of the program is shown above.

3.56 Write a program to display octal and hexadecimal equivalents of decimal numbers 100 to 200 with a difference of 10.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
#include<math.h>

int main()
{
    clrscr();
    int k=100;
    cout<< "Number\t" <<"Octal\t" <<"Hexadecimal\n";
    cout<<"=====\n";
    while (k<201)
    {
        cout.setf(ios::dec, ios::basefield);
        cout<<k <<"\t";
        cout.setf(ios::oct, ios::basefield);
        cout<<k<<"\t";
        cout.setf(ios::hex, ios::basefield);
        cout<<k<<endl;
        k+=10;
    }
}
```

```

    }
    return 0;
}

```

OUTPUT

Number	Octal	Hexadecimal
=====	=====	=====
100	144	64
110	156	6e
120	170	78
130	202	82
140	214	8c
150	226	96
160	240	a0
170	252	aa
180	264	b4
190	276	be
200	310	c8

Explanation: In the above program, ios base fields such as oct, hex, and dec are set using setf() function. After this setting the decimal number given in cout() before printing is converted to the above formats. The while loop repetitively performs this task and the result is displayed as above.

3.57 Write a program to create manipulator. Use setf() function.

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>
ostream & rs ( ostream & o)
{
    o << "Rs.";
    return o;
}
ostream & arg (ostream & o)
{
    o.setf(ios::showpos);
    o.setf(ios::showpoint);
    o.fill(' ');
    o.precision(2);
    o<<setiosflags (ios::fixed) <<setw(7);
    return o;
}

```

```
int main()
{
    clrscr();
    cout<<rs<<arg<<132.58;
    return 0;
}
```

OUTPUT

Rs.+132.58

Explanation: In the above program, `rs` and `arg` are two user-defined manipulators. The `arg` manipulator contains various member functions of `ios` class that are used to manage the screen.

3.58 Write a program to define macro for %d and <<. Use them in cout and printf statements.

```
#include<iostream.h>
#include<conio.h>
#include<stdio.h>
#define d <<j
#define i "\n%d"

int main()
{
    clrscr();
    int j=1;
    cout d ;
    printf (i,j);
    return 0;
}
```

OUTPUT

1
1

Explanation: The operator `<<` and variable `j` together are assigned to macro `d` and the format string `%d` is assigned to macro `i`. Both are used in `cout` and `printf` statements, respectively.

3.59 Write a program to display trailing decimal zeros using formatted functions.

```
#include<iostream.h>
#include<conio.h>
int main()
{
```



```

clrscr();
cout.setf(ios::showpoint);
cout.precision(2);
cout<<44.74<<endl;
cout<<23.30<<endl;
cout<<20.00;
return 0;
}

```

Explanation: By default, the `cout` statement omits the trailing decimal zeros. The trailing zeros can be displayed using a setting as shown in the above program. The result of the above program is shown as follows in tabular format for the sake of understanding.

4	4	.	7	4
2	3	.	3	0
2	0	.	0	0

3.60 Write a program to display contents of an array using `width()` function. Fill the blank space with ‘*’.

```

#include<iostream.h>
#include<conio.h>
int main()
{
    clrscr();
    int i,j;
    float amount;
    float item[3][3]= {111,45,75.25,112,50,100.15,113,45,101.75};
    cout.precision(2);
    cout<<" Code No."<<" Qty" <<"\t Rate"<<endl;
    cout.fill('*');
    for ( i=0;i<3;i++)
    {
        cout<<endl;
        for (j=0;j<3;j++)
        {
            cout.width(8);
            cout<<item[i][j];
        }
    }
    return 0;
}

```

Explanation: In the above program, an array `item[3][3]` is declared and initialized with float numbers. The following formatting statements are used.

`cout.precision(2)` – Sets decimal point limit to two digits.
`cout.fill('*')` – Fills '*' in blank space of field.
`cout.width(8)` – Sets field width to 8.

The first and second `for` loops are used to read and display the contents of the array on the screen. The output of the program is as follows.

CODE NO.							QTY								RATE								
*	*	*	*	*	1	1	1	*	*	*	*	*	*	4	5	*	*	*	7	5	.	2	5
*	*	*	*	*	1	1	2	*	*	*	*	*	*	5	0	*	*	1	0	0	.	1	5
*	*	*	*	*	1	1	3	*	*	*	*	*	*	4	5	*	*	1	0	1	.	7	5

Here, each field has width 8. The numbers are displayed with right justification. The blank spaces are filled with '*' symbol.

SUMMARY

- (1) The input/output function of C++ works with different physical devices. It also acts as an interface between the user and the device.
- (2) A stream is a series of bytes and acts as source and destination for data. The source stream is called as input stream and the destination stream is called as output stream.
- (3) The `cin`, `cout`, `cerr`, and `clog` are pre-defined streams.
- (4) The header file `iostream.h` files must be `#include` when we use `cin` and `cout` functions.
- (5) The `istream` and `ostream` are derived class from `ios` base class. Figure 3.4 displays all the derived classes.
- (6) The formatting of output can be effectively done with member function of `ios` class. The member function `width()`, `precision()`, `fill()`, and `setf()` allows the user to design and display the output in formatted form.
- (7) Table 3.6 describes the list of functions without bit fields. These functions are also used for formatting the output
- (8) The `putback()` replaces the given character into the input stream. The member function `ignore` ignores the number of given character till it finds the termination character.
- (9) Manipulators also help the user in formatting the output. The programmer can also create his or her manipulators.
- (10) The header file `iomanip.h` contains pre-defined manipulators. Table 3.7 describes these manipulators.

EXERCISES

(A) Answer the following questions

- (1) List the names of pre-defined streams with their 'C' equivalents?
- (2) What are formatted and unformatted I/O functions?
- (3) Distinguish between
 - (a) `cin()` and `scanf()`
 - (b) `cout()` and `printf()`
 - (c) `ios::fixed` and `cout.precision()`
- (4) What are the uses of `put()` and `get()` functions?
- (5) What is the use of `getline()` function? Which two arguments does it require?
- (6) Describe the bit fields required in `setf()` function.
- (7) List the flags without bit fields with their working.
- (8) What is the role of `iostream.h` and `iomanip.h` header files?
- (9) Write the statement for concatenation of two strings using `cout.write()` statement.
- (10) Describe the procedure for designing manipulator.
- (11) What is the function of `peek()` and `ignore()` functions?
- (12) What are single and multiple parameter manipulators?
- (13) In which format does the `out` statement display the address of a variable? How can it be converted to unsigned?
- (14) In which situation is the `putback()` function useful?
- (15) What is the use of `ignore()` function?
- (16) What do you mean by formatted and unformatted data?

(B) Answer the following by selecting the appropriate option

- (1) The `cin` and `cout` functions require the header file to include
 - (a) **`iostream.h`**
 - (b) `stdio.h`
 - (c) `iomanip.h`
 - (d) none of the above
- (2) The `set.precision()` is used to set
 - (a) number of digits
 - (b) **decimal places**
 - (b) field width
 - (d) none of the above
- (3) To fill unused sections of the field, the character is set by the function
 - (a) **`fill()`**
 - (b) `width()`
 - (c) `precision()`
 - (d) none of the above
- (4) The manipulator `<<endl` is equivalent to
 - (a) **'\n'**
 - (b) **'\t'**
 - (c) **'\b'**
 - (d) none of the above
- (5) This function accepts the string with blank spaces
 - (a) `cin`
 - (b) `scanf()`
 - (c) **`getline()`**
 - (d) none of the above
- (6) The streams is a
 - (a) flow of integers
 - (b) **flow of data**
 - (c) flow of statements
 - (d) none of the above
- (7) The statement `cout<<hex<<15;` gives the data in
 - (a) **hexadecimal format**
 - (b) octal format
 - (c) binary format
 - (d) decimal format
- (8) The `gcoun()` function counts the
 - (a) inserted character

- (b) **unformatted extracted character**
 (c) both (a) and (b)
 (d) none of the above
- (9) The buffer is used to move data between
 (a) input and output devices
 (b) **i/O devices and computer**
- (c) input devices to storage devices
 (d) none of the above
- (10) The buffer is a
 (a) **block of memory**
 (b) part of ram
 (c) part of hard disk
 (d) none of the above

(C) Attempt the following programs

- (1) Write a program to read five float numbers with six decimal places.
- (2) Write a program to display the hexadecimal and octal equivalent of 85, 25, 152, 251, and 458 numbers.
- (3) Write a program to display decimal equivalent of hexadecimal numbers 0x52, 0x98, 0x101, 0x524, and 0x421.
- (4) Write a program to read ten records of a student with the information Name, Age, and Date of Birth. Arrange the information (output) in the following format. Fill
- the unused field with dots.
- | Sr. no. | Name | Age | Date of Birth |
|---------|------|-----|---------------|
| 01 | Ajay | 20 | 01/01/1991 |
- (5) Write a program to accept string using `get()` function. Display the string using `write()` and `put()` function. Mention the difference between `write()` and `put()` functions.
- (6) Write a program to design the following manipulators.

Manipulator Name	Function (Escape sequence)
<<bkp (backspace)	'\b' or '\r'
<<newl	'\n'
<<bell	'\a'
<<plus	'+' (displays + sign)
<<dollar	\$ (displays \$ sign)

- (7) Write a program to read item code, quantity, and price and calculate the amount. Display the data in the following format.

Note: (a) Sr.no. and Item code are left justified.
 (b) The price and amount are right justified.
 (c) Three-digit precision for amount field.

Sr.no.	Item code	Quantity	Price	Amount
01	0101	10	55.15	551.501

- (8) Write a program to enter text up to 100 characters through the keyboard. Ignore? symbol and replace it with *. Display dot between two words.
- (9) Given $x = 5$, $y = 8$, and $z = 12$, what will be the value of the variables on the left side of the following equations?
 (a) $a = x/y * z$
 (b) $b = y/z * x$
- (c) $c = z * y/z$
 (d) $d = x * z/y$
 (e) $d = (x/y) * z$
- (10) If $p = 0.5$, $q = -2.0$, $r = 7.3$, $s = 10.4$, $m = 2$, $n = -3$, what will be the value of the variables on the left side of the following equations.
 (a) $x = 5.0 * p + q - (r + s*3.0)$
 (b) $y = 3.0 * p + q + s*2$

- (c) $z = p * s + q * r - s * q / 5.0$
 (d) $a = \text{sqrt}(4.0 - r + 9.0 * q)$
 (11) Solve the following algebraic equations.
 (a) $k = 4.5 \log_{10} x + 2xy + x^5$

- (b) $j = (bx + x) / (bx - b)$
 (c) $z = (x)2 + (x + 1)2 + (x + 2)2$
 (d) $u = a * b / (c + d * g + k) + e$

- (12) Write a program to generate the following outputs.

-	#	#	#	7	8	4	.	5	0							
*	4	5	.	4	8	\$	\$	\$	2	8	.	3	4	*	*	5
	7	8	.	4	9			7	9	.	4					9
-	%	%	5	8	9	.	4	8	0							

- (13) Write a program to generate the following output.

```

C
CL
CLA
CLASS
CLASS
CPLUSPLUS
CPLUSPL
CPLUS
CPL
C

```

(D) What will be the output of the following programs?

- (1) `cout.setf(ios::left, ios::adjustfield);`
`cout.fill('*');`
`cout.precision(2);`
`cout.width(7);`
`cout<<345.54;`
`cout.width(8);`
`cout<<78;`
- (2) `cout.setf(ios::internal, ios::adjustfield);`
`cout.fill('$');`
`cout.precision(4);`
`cout.width(11);`
`cout<<-543.453;`
- (3) `cout.setf(ios::showpos);`
`cout.setf(ios::showpoint);`
`cout.setf(ios::internal, ios::adjustfield);`
`cout.precision(4);`
`cout.width(12);`
`cout<<2342.34;`
- (4) `cout.precision(2);`
`cout<<4.57<<endl;`
`cout<<7.1453<<endl;`
`cout<<4.5132<<endl;`
`cout<<8.004<<endl;`
- (5) `cout.fill('$');`
`cout.precision(4);`
`cout.setf(ios::internal, ios::adjustfield);`
`cout.setf(ios::scientific, ios::floatfield);`
`cout.width(13);`
`cout<<-78.47854;`
- (6) `cout.fill('=');`
`cout.setf(ios::internal, ios::adjustfield);`
`cout.setf(ios::scientific, ios::floatfield);`
`cout.unsetf(ios::scientific);`
`cout.width(15);`

```

    cout<<- 3.121;
(7) cout.width (7);
    cout<<123<56;

```

(E) Additional programs

- (1) Program to read integer, character, float, and double integer and display them.
- ```

#include<iostream.h>
#include<conio.h>
int main()
{
 clrscr();
 int a=10;
 float b=10.4;
 char c='b';
 double d=12345.4567;
 cout<<"\nValue of a is
 :"<<a;
 cout<<"\nValue of b is
 :"<<b;
 cout<<"\nCharacter assigned
 to c is :"<<c;
 cout<<"\n The double value d
 is: :"<<d;
 return(0);
}

```

#### OUTPUT

**Value of a is :10**  
**Value of b is :10.4**  
**Character assigned to c is :b**  
**The double value d is: :12345.4567**

- (2) Program to use different formats of typecasting and display the converted values.
- ```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int a=97;
    float b=100.4;
    char c='z';
    double d=12345.4567;
    cout<<"\n Integer
    in character format is
    :"<<char(a);

```

```

    cout<<"\n Float in
    integer format is
    :"<<int(b);
    cout<<"\n Character
    in integer format is
    :"<<int(c);
    cout<<"\n Double in
    integer format is
    :"<<int(d);
    return(0);
}

```

OUTPUT

Integer in character format is :a
Float in integer format is :100
Character in integer format is :122
Double in integer format is :12345

- (3) Program to display HAI! using, C++ using the put () statement.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    cout.put('H').
    put('A').put('I').
    put('!');
    cout<<endl;
    cout.put('C').
    put('+').put('+');
    return (0);
}

```

OUTPUT

HAI!
C++

- (4) Program to display lower- and uppercase characters from a to z.
- ```

#include<iostream.h>
#include<conio.h>
#include<stdio.h>

int main()
{
 clrscr();
 int j;

```

```

cout<<"Lowercase characters:";
for(j=122;j>96;j--)
cout<<(char)j<<" ";
cout<<endl;
cout<<"Uppercase characters:";
for(j=90;j>64;j--)
printf("% c",j);
return (0);
}

```

### OUTPUT

**Lowercase characters: z y x w v u t s r  
q p o n m l k j i h g f e d c b a  
Uppercase characters: Z Y X W V U T  
S R Q P O N M L K J I H G F E D C  
B A**

- (5) Program to read characters using a sequence of `get()` statements and display the characters read using a sequence of `put()` statements.

```

#include<iostream.h>
#include<conio.h>

int main()
{
clrscr();
int age;
float num;
char sex;
char*msg="C++ is an object
 oriented lan-
 guage";

sex='F';
age=25;
num=1004.5;
cout<<"Sex:"<<sex<<endl;
cout<<"Age:"<<age<<endl;
cout<<"Entered float number
 is:"<<num<<endl;
cout<<"Message entered
is:"<<msg<<endl;
cout<<10<<" "<<20<<"
"<<30<<endl;
cout<<"Modified float number
is:";

```

```

cout<<num+5;
return(0);
}

```

### OUTPUT

**Sex:F  
Age:25  
Entered float number is:1004.5  
Message entered is: C++ is an object  
oriented language  
10 20 30  
Modified float number is:1009.5**

- (6) Program to display the addresses of variables in hex and unsigned integer format.

```

#include<iostream.h>
#include<conio.h>

int main()
{
clrscr();
int a=77,c=78;
float b=5.1252;
cout<<"Address of
a="<<&a<<endl;
cout<<"Address of
b="<<&b<<endl;
cout<<"Address of
c="<<(unsigned)&c<<endl;
return(0);
}

```

### OUTPUT

**Address of a=0x8f69fff4  
Address of b=0x8f69ffec  
Address of c=65522**

- (7) Program to read characters using `read()` and display it by `write()` function.

```

#include<iostream.h>
#include<conio.h>

int main()
{
clrscr();
char name[6];
cout<<"\n Enter a string:";
cin.read(name,5);

```

```
cout<<endl;
cout<<"Entered string is:";
cout.write(name,5);
return(0);
}
```

**OUTPUT**

**Enter a string:Amit**  
**Entered string is:Amit**

- (8) Program to read a string using getline() function and display it using write() statement.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>

int main()
{
 char a[7];
 clrscr();
 cout<<"\n Enter any
 string:";
 cin.getline(a,7);
 cout<<"\n Entered
 string:"<a;
 cout<<"\n Entered
 string:";
 cout.write(a,7);
 cout<<"\n Entered
 string;";
 cout.
 write(a,strlen(a));
 return(0);
}
```

**OUTPUT**

**Enter any string:Mumbai**  
**Entered string: Mumbai**  
**Entered string: Mumbai**  
**Entered string: Mumbai**

- (9) Program to generate the following output.
- ```
C
C+
C++
C++e
C++ea
C++eas
```

```
C++easy
C++easy
C++easy
```

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
```

```
int main()
{
    clrscr();
    char p,b[8];
    cout<<"\n Enter any
    string:";
    cin.getline(b,8);
    cout<<"\n Entered
    string:";

    for(p=0;p<=strlen(b);p++)
    {
        cout.write(b,p);
        cout<<"\n";
    }

    for(p=8;p>=strlen(b);p--)
    {
        cout.write(b,p);
        cout<<"\n";
    }

    return(0);
}
```

OUTPUT

Enter any string:C++easy
Entered string:
C
C+
C++
C++e
C++ea
C++eas
C++easy
C++easy
C++easy

- (10) Program to generate the following output.
- ```
Decimal.....64
Hexadecimal.....40
Octal.....100
```



```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

int main()
{
 clrscr();
 int a=64;
 cout<<setfill('.')<<endl;
 cout<<setiosflags(ios:
:left);
 cout<<setw(16)
<<"Decimal";
 cout<<resetiosflags
(ios::left);
 cout<<setw(6)
<<dec<<a<< endl;
 cout<<setiosflags
(ios::left);
 cout<<setw(15)
<<"Hexadecimal";
 cout<<resetiosflags
(ios::left);
 cout<<setw(6)
<<hex<<a<<endl;
 cout<<setiosflags
(ios::left);
 cout<<setw (15)<<"Oc-
tal";
 cout<<resetiosflags
(ios::left);
 cout<<setw(6)
<<oct<<a<<endl;
 return(0);
}

```

### OUTPUT

```

Decimal.....64
Hexadecimal.....40
Octal.....100

```

- (11) Program to generate the following output.

```

c
c+
c++
c++e
c++ea
c++eas
c++easy

```

```

c++easy
c++eas
c++ea
c++e
c++
c+
c
#include<iostream.h>
#include<conio.h>
#include<string.h>

int main()
{
 clrscr();
 char p,b[9];
 cout<<"\n Enter any
string";
 cin.getline(b,9);
 cout<<"\n Entered
string:";

 for(p=0;p<=strlen(b);p++)
 {
 cout.write(b,p);
 cout<<"\n";
 }

 for(p=strlen(b);p>=0;p--)
 {
 cout.write(b,p);
 cout<<"\n";
 }
 return(0);
}

```

### OUTPUT

```

Enter any stringc++easy
Entered string:
c
c+
c++
c++e
c++ea
c++eas
c++easy
c++easy
c++eas
c++ea
c++e

```

```

c++
c+
c

(12) Program to generate output using flag bits
such as showpos, showpoint, hex, and up-
percase of given number.

#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 cout.setf
 (ios::showpos);
 cout<<1254;
 cout.setf
 (ios::showpoint);
 cout<<"\n"<<1254.524;
 cout.setf(ios::hex,
 ios::basefield);
 cout<<"\n"<<184;
 cout.setf
 (ios::uppercase);
 cout<<"\n"<<184;
 return(0);
}

```

**OUTPUT**

```

+1254
+1234.524000
b8
B8

```

```

(13) Program to fill the trailing digits of num-
ber by '$' symbol.

#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 int s=0,j=0;
 float pi;
 cout.fill('$');
 for(s=4;s>=1;s--)
 {
 pi=(float)22/7;
 cout.width(j++);

```

```

 cout.precision(s);
 cout<<"\n"<<pi;
 }
 cout.width(j++);
 cout.precision(s);
 cout<<"\n"<<pi;
 s=cout.fill();
 cout<<"\n previous
 setting : "<<(char)s;
 return(0);
}

```

**OUTPUT**

```

3.1429
3.143$
3.14$$
3.1$$$
3.142857
previous setting :

```

```

(14) Program to print "*****ABC*****".

#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 cout.fill('*');
 cout.width(10);
 cout<<"ABC";
 cout.fill('*');
 cout.width(7);
 cout<<"*";
 return(0);
}

```

**OUTPUT**

```

*****ABC*****

```

```

(15) Program to use different formats of
width() and display the results.

#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 cout.width(20);
 int x=cout.width(20);

```

```

 cout<<x;
 return(0);
 }

```

## OUTPUT

20

- (16) Program to use skipws and stdio flags.

```

#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 char x[5];
 cout<<"Enter array of
 characters:";
 cin>>x;
 cout.setf
 (ios::skipws);
 cout<<"Entered array
 of characters:"<<x;
 cin>>x;
 cout.
 setf(ios::skipws);
 cout<<"\nEn-
 tered array of
 characters:"<<x;
 cin>>x;
 cout.setf
 (ios::stdio);
 cout<<"\nEn-
 tered array of
 characters:"<<x;
 return(0);
}

```

## OUTPUT

**Enter array of characters: as h ok**  
**Entered array of characters: as**  
**Entered array of characters: h**  
**Entered array of characters: ok**

- (17) Program to convert decimal to hexadecimal and octal number.

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

int main()

```

```

{
 clrscr();
 int a;
 cout<<"Enter the
 number :";
 cin>>a;
 cout<<"\n Hexadecimal
 number :"<<hex<<a;
 cout<<"\n Octal number
 :"<<oct<<a;
 return(0);
}

```

## OUTPUT

**Enter the number : 156**  
**Hexadecimal number : 9c**  
**Octal number : 234**

- (18) Program for converting hexadecimal number to decimal number.

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

int main()
{
 clrscr();
 int a;
 cout<<"\n Enter hexa-
 decimal number:";
 cin>>hex>>a;
 cout<<"\n Equiva-
 lent decimal number
 is:"<<dec<<a;
 return(0);
}

```

## OUTPUT

**Enter hexadecimal number is : f**  
**Equivalent decimal number is : 15**

- (19) Program to convert octal number to hexadecimal and decimal number.

```

#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

int main()
{
 clrscr();

```

```

int a;
cout<<"\n Enter octal
number :";
cin>>oct>>a;
cout<<"\n Equivalent
hexadecimal number is
:"<<hex<<a;
cout<<"\n Equivalent
decimal number is
:"<<dec<<a;
return(0);
}

```

**OUTPUT**

**Enter octal number : 124**  
**Equivalent hexadecimal number is : 54**  
**Equivalent decimal number is : 84**

- (20) Program to generate the following output.

```

****1000.2
*100.45

```

```

#include<iostream.h>
#include<conio.h>
int main()
{
 clrscr();
 cout.fill('*');
 cout.precision(3);
 cout.width(10);
 cout<<1000.2<<endl;
 cout.fill('*');
 cout.precision(2);
 cout.width(7);
 cout<<100.4532;
 return(0);
}

```

**OUTPUT**

```

****1000.2
*100.45

```

*This page is intentionally left blank.*

# C++ Declarations

# 4

## CHAPTER

### CHAPTER OUTLINE

- 4.1 Introduction
- 4.2 Tokens
- 4.3 Variable Declaration and Initialization
- 4.4 Data Types in C++
- 4.5 Operators in C and C++
- 4.6 Scope Access Operator
- 4.7 Namespace
- 4.8 Memory Management Operators
- 4.9 Comments
- 4.10 Comma Operator
- 4.11 Comma in Place of Curly Braces
- 4.12 More Programs

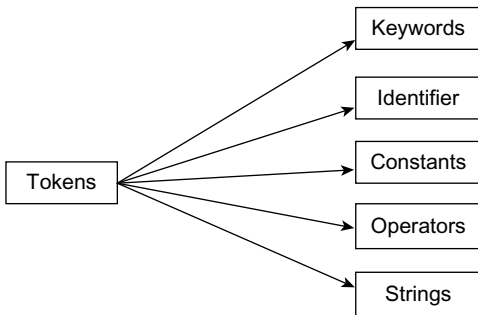
## 4.1 INTRODUCTION

To write a C++ program, it is necessary to know the syntaxes of the language. This enables a programmer to write error-free programs. If the programmer is not aware of the basic rules of the language, he/she may fail to write program, even though the logic is clear.

A few basic concepts of C++ language, such as syntaxes, data types, and keywords, are discussed in this chapter to help the programmer to develop programs. As the object-oriented programming is a new technology, the programmer will find a notable amount of new concepts to learn and master in it.

Most of the C++ programs in this chapter are implemented with former C language. Few rules and regulations of C programs are valid in C++ too. Intact the C++ is upward compatible to C. Hence the programs with C can be executed with C++. C++ allows us to use variable declaration anywhere in the program, and new operators such as reference and scope access operator are introduced. All these concepts are illustrated with figures and examples.

## 4.2 TOKENS



**Fig. 4.1** Types of tokens

The smallest individual unit in a program is known as ‘**token**’. C++ programs contain various components. The compiler identifies them as tokens. Tokens are classified into the following types and are also indicated in Figure 4.1.

- (1) Keywords – Reserved by the compiler.
- (2) Identifiers – Names of variables, arrays, classes, functions, etc. are identifiers.
- (3) Constants – Constants are fixed values.
- (4) Operators – Operators are different types and are used in expressions.
- (5) String – Sequence of characters.

The keywords are reserved set of words with fixed meanings. The variables are used to hold data temporarily. The operators are used to perform different operations such as arithmetic and logical. Values such as 1, 5, 2.5, etc. are known as constants. The operators such as #, ?, and ~ are known as special characters. The # is used for preprocessor directive; ? is a conditional operator; and ~ is used for bitwise operation. The detailed descriptions of these tokens are described in following sections.

### 4.2.1 Keywords

The ‘C++’ keywords are reserved words by the compiler and are assigned fixed meanings. All ‘C’ keywords are valid in C++. The programmer may not apply them in the programs for defining variable names; however, few ‘C++’ compilers permit to declare variable names that exactly match with the keywords.

The common keywords between C and C++ are listed in Table 4.1 and the additional keywords of C++ are listed in Table 4.2. These keywords are used with classes, templates, and exception handling, etc. Table 4.3 contains keywords added by ANSI committee. Table 4.4 contains additional keywords provided by Turbo C++ compiler.

**Table 4.1** C and C++ Common Keywords

|          |        |          |          |
|----------|--------|----------|----------|
| auto     | double | int      | struct   |
| break    | else   | long     | switch   |
| case     | enum   | register | typedef  |
| char     | extern | return   | union    |
| const    | float  | short    | unsigned |
| continue | for    | signed   | void     |
| default  | goto   | sizeof   | volatile |
| do       | if     | static   | while    |

**Table 4.2** Additional C++ Keywords

|          |           |
|----------|-----------|
| asm      | private   |
| catch    | protected |
| class    | public    |
| delete   | template  |
| friend   | this      |
| inline   | throw     |
| new      | try       |
| operator | virtual   |

**Table 4.3** Keywords Added by ANSI Committee

| Keywords Added by Ansi Committee |            |                  |             |        |
|----------------------------------|------------|------------------|-------------|--------|
| bool                             | const_cast | dynamic_cast     | explicit    | true   |
| mutable                          | namespace  | reinterpret_cast | static_cast | false  |
| typename                         | using      | wchar_t          | export      | typeid |

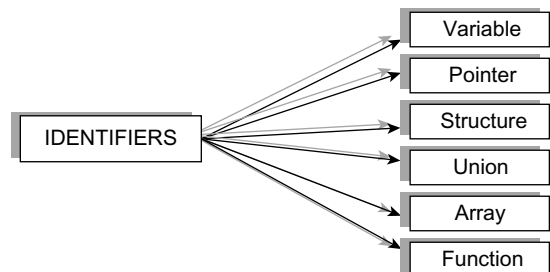
**Table 4.4** Additional Keywords in Turbo C++

| Additional keywords in Turbo C++ |           |           |        |      |
|----------------------------------|-----------|-----------|--------|------|
| cdecl                            | _cs       | _ds       | _es    | far  |
| _fastcall                        | huge      | interrupt | _loads | near |
| pascal                           | _saveregs | _seg      | _ss    |      |

## 4.2.2 Identifiers

Each variable is represented by a symbol using identifiers. Identifiers are names of variables, functions, arrays, etc. In other words, identifiers refer to variety of entities such as structures, unions, enumeration, constants, typedef names, functions, and objects (refer Figure 4.2).

C++ identifier always starts with an alphabet and it is a plain sequence of alphabets and/or digits. C identifier does not allow blank spaces, punctuation, signs, etc.



**Fig. 4.2** Types of tokens



Identifiers are user-defined names and are generally defined in lower case letters. However, the upper case letters are also permitted. The underscore (`_`) symbol can be used as an identifier. In general, underscore is used to link two words for the long identifiers. Valid identifiers are as follows:

length, area, volume, sUM, Average, etc.

Invalid identifiers are as follows:

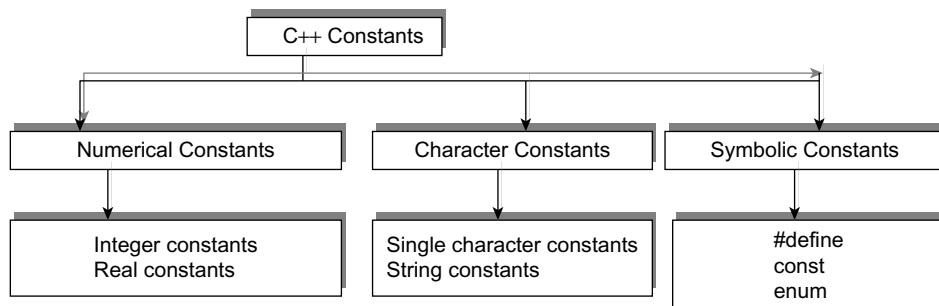
Length of line, S+um, year's, etc.

Rules for defining identifiers are as follows:

- (1) The identifier name must begin with a character or underscore and should not start with digit. No spaces are allowed between the characters in the variable but underscore is allowed.
- (2) The identifier name should not be a 'C++' keyword.
- (3) The identifier name can be a combination of upper and lower characters. For example, the variables suM, sum, and SuM are not same.

### 4.2.3 Constants

The constants in 'C++' are applicable to the values that do not change during execution of a program. There are several types of constants in 'C++'. They are classified into the following groups as shown in Figure 4.3 and Table 4.5.



**Fig. 4.3** 'C++' constants

**Table 4.5** Constant Types

| Example | Constant Type                |
|---------|------------------------------|
| 542     | Integer constant             |
| 35.254  | Floating point constant      |
| 0x54    | Hexadecimal integer constant |
| 0171    | Octal integer constant       |
| 'C'     | Character constant           |
| "cpp"   | String constant              |

## Numerical Constants

### *Integer Constants:*

Integer constants are represented with whole numbers. It requires minimum two bytes and maximum four bytes of memory.

The following concepts are essential for the numerical constants:

- (a) The numerical constants are represented with numerical numbers. At least one digit is needed for representing the numerical number.
- (b) The decimal point or fractional part or any other symbols are not permitted. Neither even blank spaces nor commas are permitted.
- (c) Integer constant could be either positive or negative or may be zero.
- (d) The number without a sign is assumed to be positive.

Valid Examples: 10, 20, +30, -15, etc.

Invalid integer constants: 2.3, .235, \$76, 3\*^6, etc.

Integers can be represented in octal or hexadecimal based on the requirement, besides decimals.

Octal has base 8 and hexadecimal 16. The octal numbers are 0, 1, 2, 3, 4, 5, 6, 7, and hexadecimal numbers are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F.

The representation of octal numbers in C++ would be done with leading digit 0 and for hexadecimal with leading 0X.

Following are few examples of octal and hexadecimal numbers:

027, 037, 072 – octal numbers

0X9, 0Xab, 0X4 – hexadecimal

Various constants are given in Table 4.5.

### *Real Constants:*

Real constants are often known as floating point constants and can be represented in exponential or fractional form. Integer constants are unfit to represent many quantities. Many parameters or quantities are defined not only in integers but also in real numbers. For example, length, height, prize, distance are also measured in real numbers.

The following concepts are essential for real numbers.

- (a) Decimal point is permitted.
- (b) Neither blank spaces nor commas are permitted.
- (c) Real numbers could be either positive or negative.
- (d) Number without a sign is assumed to be positive.

Examples of real numbers are as follows:

2.5, 5.521, 3.14.

The real constants can be written in exponential notation, which contains fractional and exponential parts. For example, the value 2456.123 can be written as  $2.4561 \times e^{+3}$ .

The part that precedes 'e' is called as mantissa and following it is an exponent.

In this example 2.4561 is mantissa and +3 is exponent.

Following points must be noted while constructing a real number in exponential form:

- (a) The real number contains mantissa and exponent.

- (b) The letter 'e' separates the mantissa and exponent and it can be written either in upper or in lower case.
- (c) The mantissa is either a real number represented either as decimal or as an integer.
- (d) The mantissa may be either positive or negative.
- (e) The exponent is an integer which may be either positive or negative.

Valid examples are 5.2e2, -2, 5.0e-5, 0.5e-3, etc.

Also in `double` type, the real numbers can be expressed with mantissa and exponent parts.

## Character Constants

### *Single-character constants:*

A character constant in C is represented with a single character and it is enclosed within single quotes. Each character constant has a particular integer value associated it.

Each character is represented with different ASCII value (the values are listed in Appendix 1). A character can also be represented with single digit or single special symbol or white space, all enclosed within a pair of single quote marks.

Example: 'a', '8', ' ', '&', etc.

### *String constant:*

String constants are sequence of characters enclosed within double quote marks. The string may be combination of all types of symbols.

Example: "Hello", "India", "444", "a".

In short, the various constants are shown in Table 4.5.

A programming example for various constants is as follows:

#### 4.1 Write a program to demonstrate various constants.

```
#include<iostream.h>
#include<conio.h>
int main()
{
 int x;
 float y;
 char z;
 double p;
 clrscr();
 x=20;
 y=2e1;
 z='a';
 p=3.2e20;
 cout<<"x="<<x<<endl;
 cout<<"y="<<y<<endl;
 cout<<"z="<<z<<endl;
 cout<<"p="<<p;
 return 0;
}
```



**OUTPUT**

```
x=20
y=20
z=a
p=3.2e+20
```

**Explanation:** In this programming example, various constants are assigned to different variables.

**Symbolic Constants**

A symbolic constant is defined in the same way as a variable. However, once the constant is initialized, the assigned value cannot be altered.

The constant can be defined in three ways: `#define`, `const` keyword, `enum` keyword

**#define preprocessor directive:** The `#define` preprocessor directive can be used for defining constant. The symbols defined using `#define` are called **macros**. The syntax of `#define` is as follows.

**#define name constant\_value**

**Explanation:** `#define price 152`

In the above example, `price` symbolic constant contains 152 and data type, such as `int`, `float`, or `char`, is not mentioned. Every time when the preprocessor finds the word `price`, it substitutes with 152.

**4.2 Write a program to find the radius and circumference of the circle using #define directive.**

```
#include<iostream.h>
#include<conio.h>
#define pi 3.14
int main()
{
 float area, radius, circum;
 clrscr();
 cout<<"\nEnter the radius of the circle:";
 cin>>radius;
 cout<<"\n Entered radius of the circle is "<<radius;
 area=pi*radius*radius;
 cout<<"\nArea of circle is "<<area;
 circum=2*pi*radius;
 cout<<"\n Circumference of the circle <<circum;
 return 0;
}
```

**OUTPUT**

```
Enter the radius of the circle:1
Entered radius of the circle is 1
Area of circle is = 3.14
Circumference of the circle is = 6.28
```

**Explanation:** In the above program, the value of pi is replaced by 3.14 during program execution. In the program instead of writing the value of pi as 3.14 we define directly the value pi as 3.14. The term pi is used for calculating the area and circumference of the circle.

Constant using const keyword:

*Example:*

```
const int price=152;
```

The above declaration defines a constant of type int. Here, data type is strictly maintained while execution and the value of constant cannot be changed during run-time.

#### 4.3 Program on const keyword.

```
#include<iostream.h>
#include<conio.h>

int main()
{
 const int x=20;
 clrscr();
 cout<<"x="<<x<<endl;
 // cout<<"x="<<x++; cannot modify the constant
 return 0;
}
```

#### OUTPUT

x=20

**Explanation:** A constant value is assigned to the variable x and the same is displayed. If attempts are made to modify constant value, compiler shows an error (cannot modify the constant).

### Constant Pointers

C++ allows us to create both constant pointer and pointer to constant. Consider the following example.

- (1) **Constant pointer:** It is not possible to modify the address of the constant pointer.

*Example:*

```
char * const str="Constant";
```

In the above example, it is not possible to modify the address of the pointer str. Thus, the following operations will generate error.

```
// str="san"; // cannot modify a constant object
// ++str; // cannot modify a constant object
```

- (2) **Pointer to constant:** If pointer is declared to constant, only the value can be changed using actual variable which is not possible using pointer.

*Example:*

```
int const * pm=&k;
```

In the above example, pm is declared as pointer to constant. The following examples are possible and invalid operations.

```
// k=5; // possible
// *pm=5 // cannot modify the constant object
// pm++; // possible
// ++ *pm; // cannot modify a constant object
// *pm=5; // cannot modify a constant object
```

### (3) Pointer and variable both constants:

*Example:*

```
const char * const p="ABC";
```

In the above example, both the pointer and variable are constants. Hence, it is not possible to change the value and address of the pointer.

#### 4.4 Write a program to define constant pointer and pointer to constant. Perform the possible operations.

```
#include<iostream.h>
#include<conio.h>
#include<process.h>
void main()
{
 clrscr();
 // constant pointer //
 char * const str="Constant"; // Declaration of constant
 //pointer

 cout<<str;
 // str="san"; // can not modify a constant object
 //++str; // can not modify a constant object
 // pointer to constant //
 int k=1;
 int const * pm=&k; // pointer points to k
 // k=5; // possible
 //*pm=5 // can not modify the constant object
 //pm++; // possible
 //++*pm; // can not modify a constant object
 // *pm=5; // can not modify a constant object
 cout<<"\n"<<*pm;
 // pointer and variable both constant
 const char * const p="ABC";
 // p++; // can not modify a constant object
```

```
// p="xxx"; // can not modify a constant object
cout<<"\n"<<p;
}
```

## OUTPUT

**Constant**

**1**

**ABC**

### (4) Constants can be defined using enum as follows:

*Example:*

```
enum {a,b,c};
```

where *a*, *b*, and *c* are declared as integer constants with values 0, 1, and 2, respectively.

We can also assign new values to *a*, *b*, and *c*.

```
enum [a=5,b=10,c=15];
```

where *a*, *b*, and *c* are declared as integer constants with values 5,10,and 15, respectively.

## 4.2.4 Operators

C++ supports all the operators of 'C'. In addition, C++ introduces few more operators. The new operators are <<, >>, ::, ::\*, ->\*,\*, **delete**, **new**, etc.

These operators are discussed in detail at the end of this chapter.

## 4.2.5 String Constants

String is represented by an array of characters. String constants are enclosed in double quotes. Following are the valid examples of string constants.

```
"Pearson", "Delhi", "India", "Singapore", "Popular", "publish-
er", etc.
```

## 4.3 VARIABLE DECLARATION AND INITIALIZATION

### 4.3.1 Variable

A variable is used to store values. Each variable is stored in memory location(s). It can be of any data type such as integer, char, float, etc. A variable holds a single value at a time of its type; that is, values of variable vary, and the programmer can change the value of the variable.

### 4.3.2 Variable Declaration

The variable must be declared before they are used in the program. Once declared (1) the compiler obtains the variable name and (2) the compiler is notified about the data type of the variable being declared and helps in allocation of the memory. The variable can also be declared before `main()` and such variables are called external variables.

In C, all the variables must be declared in the declaration part. Hence, each time when one needs to declare a variable the programmer should go back to the beginning of the program.

C++ permits declaration of variables anywhere in the program. This makes the programmer more comfortable to declare the variables and need not go back to the beginning of the program. The declaration of variable consists of name of data type and variable list as follows.

The syntax of declaring a variable is as follows:

```
Data_type variable_name;
```

*Example:*

```
int age;
char m;
float s;
double k;
int a,b,c;
```

The `int`, `char`, `float`, and `double` are keywords to represent data types. Commas are used as separators for multiple variables.

### 4.3.3 Initialization

When a variable is declared, appropriate number of bytes is reserved for that particular variable in the random access memory. Bytes are filled with garbage values if the user does not assign them a value. If the user performs operations without initializing the variable, the result will be unexpected. Hence, before using a variable it is essential to initialize it. Assigning a value to the variable is called as initialization. When a value is assigned to the variable, the garbage value is removed and replaced with the given value.

*Example:*

```
int x;
```

Here, the integer variable `x` is declared and not initialized. When the variables are not initialized, neither they are set to zero by the compiler nor they remain empty. The variables contain garbage values. The user cannot predict the garbage value, which is system dependent. The garbage values differ for each system.

If you try to print the value of `x` in C, the statement `printf("%d", x)` will display the garbage value (-29281 on my system). The `cout<<x` in C++ will display the value 0. If you use both these statements one after another, the value displayed will be 0.

### 4.5 Write a program to display value of uninitialized variable using `cout` statement.

```
#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 int x;
```



```

 cout<<x<<endl;
 cout<<x+5<<endl;
 cout<<5+(++x);
 return 0;
}

```

### OUTPUT

```

0
5
6

```

**Explanation:** In the above program, the first `cout` statement displays the value of `x` zero; the second statement displays the value of `x` as 5; and the third statement displays the value 6. In the third statement `x` is incremented first and then added to 5. This program is compiled under compact memory model. If the program is compiled and executed under any other memory model the resultant value will be garbage.

All the operations performed in the above program are directly put in the `cout` statement. If the above program is executed with the `printf()` statement, the program will display the garbage values.

The following program displays garbage value of the variables:

#### 4.6 Write a program to display garbage value of a variable.

```

#include<iostream.h>
#include<conio.h>
#include<stdio.h>

int main()
{
 int x,y;
 clrscr();
 y=5(++x);
 cout<<"Value of Y:"<<y;
 return 0;
}

```

### OUTPUT

```

Value of Y : 1134

```

**Explanation:** In the above program, 5 is added to variable `x` and assigned to `y`. One may expect that the value of `y` to be 6. Here, the value of `y` is displayed as a garbage value. From the above program it is clear that if the variable is not initialized, it will be assigned to a garbage value.

In the afore-mentioned program, the statement `y=5(++x);` it uses variables `x` and `y`. The variables `x` and `y` are not initialized. The variable `y` has no effect on the operation because it is at

the left-hand side and the result of the expression is assigned on the right-hand side. The variable *x* holds a garbage value. Integer 5 is added to the garbage value of *x* and stored in the variable *y*. Hence the value of *y* displayed is 1134.

Initialization of variable can be done at the place where it is declared or anywhere in the program before their use. Variables declared can be assigned or initialized using assignment operator '='. The declaration and initialization can also be done in the same line.

*Syntax:* variable\_name = constant;  
or  
data\_type variable\_name= constant;

*Example:*

*x*=2; where *x* is an integer variable.

*Example:*

int *y*=2;

Figure 4.4 shows an example of initialization of a variable.

*Syntax:*

data-type variable\_name = value;

*Example:*

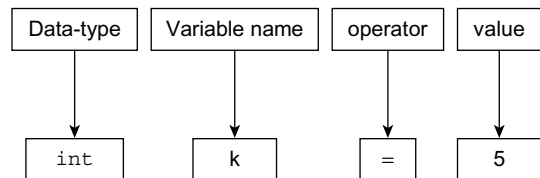
int *k* = 5;

In the above example, variable *k* of integer type is declared. The value 5 is stored in it.

int *a, b, c, d*;

*a*=*b*=*c*=*d*=5;

In the above example all the variables *a*, *b*, *c*, and *d* are initialized to 5. An example is illustrated as follows:



**Fig. 4.4** Initialization of variable

#### 4.7 Write a program to initialize more than one variable at a time.

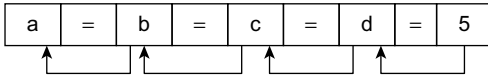
```
#include<conio.h>
#include<iostream.h>

int main()
{
 int a,b,c,d;
 a=b=c=d=5;
 clrscr();
 cout<<"\n a="<<a <<"b="<<b <<"c="<<c <<"d="<<d;
 return 0;
}
```

#### OUTPUT

**a=5 b=5 c=5 d=5**

**Explanation:** In the above program, variables, a, b, c, and d are declared. In next statement, all four variables are initialized with a value 5. First the value 5 assigned to variable d, then the value of d is assigned to c, c is assigned to b, and finally b is assigned to a.



**Fig. 4.5** Initialization of variables

Here the variables are local and they are stored in the stack. Stack is the portion of the random access memory (RAM). Generally it is located at end of RAM to avoid overlapping of program and data memory. Figure 4.5 simulates this assignment.

#### 4.3.4 Dynamic Initialization

The declaration and initialization of variable in a single statement at any place in the program is called as dynamic initialization. The dynamic initialization is always accomplished at run-time. Run-time means execution of program. Dynamic refers to a process carried out at run-time, for example dynamic initialization, dynamic memory allocation etc. The C++ compiler allows declaration and initialization of a variable at any point in the program. In C, initialization of variable can be done at any point but the variable must be declared at the beginning of the program illustrated as follows:

#### 4.8 Write a program in C to demonstrate declaration and initialization of a variable.

```
#include<conio.h>
#include<stdio.h>

void main()
{
 int r;
 float area;
 clrscr();
 printf ("\n Enter radius : ");
 scanf ("%d",&r);
 area=3.14*r*r;
 printf ("\n Area =%g",area);
}
```

#### OUTPUT

**Enter radius : 4**

**Area =50.24**

**Explanation:** The above program is executed with C compiler. The variables area and r are declared at the beginning because declaration in C is compulsorily done at the beginning. The multiplication of 3.14 and the variable r is assigned to the variable area. This assignment is done within the program. Thus, from the above program it is demonstrated that in C variable declaration is done at the beginning and initialization can be done at any point in the program.

**4.9 Write a program in C++ to demonstrate dynamic initialization.**

```
#include<conio.h>
#include<iostream.h>

int main()
{
 clrscr();
 cout<<"\n Enter radius:";
 int r;
 cin>>r;
 float area=3.14*r*r;
 cout<<"\n Area ="<<area;
 return 0;
}
```

**OUTPUT**

**Enter radius : 3**

**Area =28.26**

**Explanation:** In the above program, the variables r and area are declared within the program. The declaration and initialization of a variable area is done in the single statement within the program. Consider the following statement.

```
float area=3.14*r*r;
```

In the above statement, float variable area is declared and product of  $3.14*r*r$  is assigned to the variable area. The assignment is carried out at run-time. Such type of declaration and initialization of a variable is called as dynamic initialization.

**4.10 Write a program to read two integers through the keyboard. Declare the variables in C++ style.**

```
#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 cout<<"Enter Two numbers:";
 int num;
 cin>>num;
 int num1;
 cin>>num1;
```

```
 cout<<"Entered Numbers are :";
 cout<<num<<"\t"<<num1;
 return 0;
}
```

### OUTPUT

**Enter Two numbers : 8 9**

**Entered Numbers are : 8 9**

**Explanation:** In the above program, the variables num and num1 are declared within the program and not at the beginning of the program. Once declared, they are used with cin statement that reads two integers and stores them in the variables num and num1. The cout statement displays entered values on the screen.

#### 4.11 Use dynamic initialization and display an alphabet.

```
#include<iostream.h>
#include<conio.h>

int main()
{
 int z=102;
 clrscr();
 char a = z;
 cout<<a;
 return 0;
}
```

### OUTPUT

**f**

**Explanation:** In the above program, variable z is initialized with 102. The ASCII value of 102 is f; hence, the output is f.

#### 4.12 Write a program to calculate length of the string. Use run-time declaration and initialization of variables.

```
#include<iostream.h>
#include<string.h>
#include<conio.h>

int main()
{
 clrscr();
 char name[15];
```



```

cout<<"Enter Your Name : ";
cin>>name;
int len=strlen(name);
cout<<"The length of the string is : "<<len;
return 0;
}

```

## OUTPUT

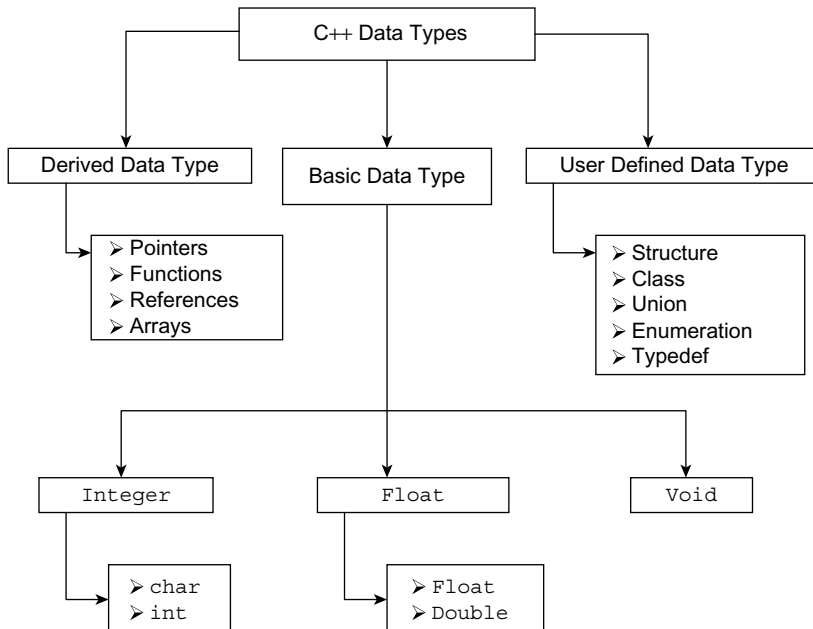
**Enter Your Name : Santosh**

**The length of the string is :7**

**Explanation:** In the above program, the `cin` statement reads the string through the keyboard. The `strlen()` function is used to determine the length of the given string. The `strlen()` function calculates the length of the string and returns the length to variable `len`. The variable `len` is declared and value returned by `strlen()` is assigned to variable `len`. To avoid separate statements for declarations and initializations, both statements are combined as one statement. Declaration and initialization are carried out in one statement `int len=strlen (name) .`

## 4.4 DATA TYPES IN C++

Data are collections of characters, digits, symbols, etc. The data are used to represent information. The data are classified into various types. Figure 4.6 indicates all data types. C++ data types can be classified as basic data type, derived data type and user-defined data type.



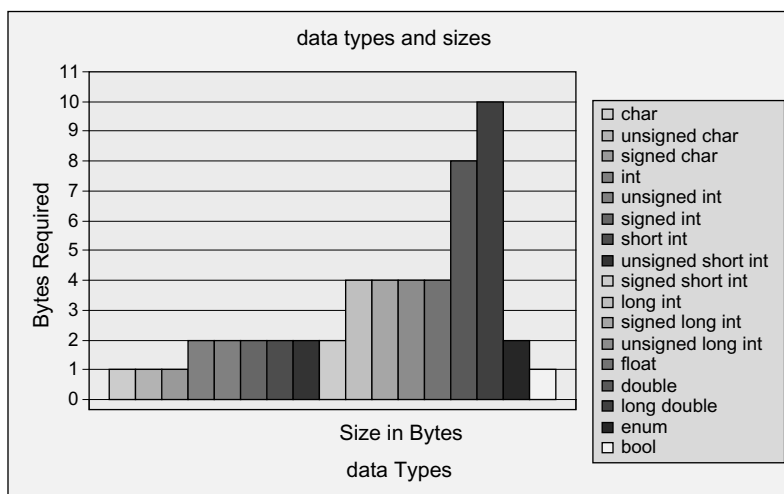
**Fig. 4.6** C++ data types

### 4.4.1 Basic Data Type

The basic data types supported by C++ are described with their size in bytes and ranges in Table 4.6. Sizes of data types supported by C++ are shown in Figure 4.7.

**Table 4.6** C++ Basic Data Types with Size and Range

| Data Type          | Size in Bytes | Range                     |
|--------------------|---------------|---------------------------|
| char               | 1             | –128 to 127               |
| unsigned char      | 1             | 0 to 255                  |
| signed char        | 1             | –128 to 127               |
| int                | 2             | –32768 to 32767           |
| unsigned int       | 2             | 0 to 65535                |
| signed int         | 2             | –32768 to 32767           |
| short int          | 2             | –32768 to 32767           |
| unsigned short int | 2             | 0 to 65535                |
| signed short int   | 2             | –32768 to 32767           |
| long int           | 4             | –2147483648 to 2147483647 |
| signed long int    | 4             | –2147483648 to 2147483647 |
| unsigned long int  | 4             | 0 to 4294967295           |
| float              | 4             | 3.4e-38 to 3.4e+38        |
| double             | 8             | 1.7e-308 to 1.7e+308      |
| long double        | 10            | 3.4e-4932 to 1.1e+4932    |
| enum               | 2             | –32768 to 32767           |
| bool               | 1             | true/false                |



**Fig. 4.7** Data types and their sizes

## Type Modifiers

The keywords `signed`, `unsigned`, `short`, and `long` are type modifiers. A type modifier changes the meaning of the base data type to produce a new data type. Each of these type modifiers is applicable to the base type `int`. The modifiers `signed` and `unsigned` are also applicable to the base type `char`. In addition, `long` can be applied to double data type. When the base type is absent from a declaration, `int` is supposed.

Examples:

```
long l; // int is implied
unsigned char c;
signed int s; // signed is default
unsigned long int u; // int OK, not necessary
```

## The void Data Type

The type `void` is empty data type. It can be used in three ways:

- (a) When specified as a function return type, `void` means that the function does not return a value.

```
void message (char *name)
{
 cout<<"Hello."<< name;
}
```

Here, `message()` is a void function. The keyword `void` is preceded by the function name. This function when executed displays only message and does not return any value to the calling function.

- (b) The `void` keyword also uses as argument for function. When found in a function heading, `void` means the function does not take any arguments.

```
int fun(void)
{
 return 1;
}
```

Here, the function is `fun()` and does not require any argument. It returns an integer value.

- (c) When specified as a function return type and in function heading, that is, the function neither returns a value nor requires any argument.

```
void fun (void);
```

The above function neither returns a value nor requires any argument.

### 4.4.2 Derived Data Type

The derived data types are pointers, functions, arrays, and references.



## Pointers

A pointer is a memory variable that stores a memory address. Pointer can have any name that is legal for other variable and it is declared in the same fashion like other variable but it is always denoted by '\*' operator.

```
int *x;
float *f;
char *y;
```

In the first statement 'x' is an integer pointer and it tells the compiler that it holds the address of any integer variable. In the same way 'f' is a float pointer that stores the address of any float variable and 'y' is a character pointer that stores the address of any character variable.

### 4.13 Write a program to use pointers.

```
#include<conio.h>
#include<iostream.h>

int main()
{
 clrscr();
 int x=2,*p;
 cout<<"\n Address of x ="<<(unsigned)&x;
 p=&x;
 cout<<"\n Value of x="<<*p;
 return 0;
}
```

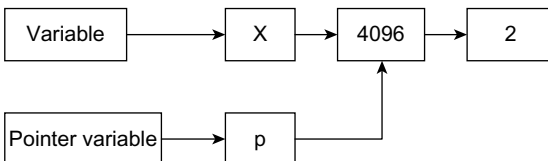
### OUTPUT

Address of x = 4096

Value of x=2

**Explanation:** In this program, x is an integer variable and \*p is an integer pointer. The first cout statement displays the address of variable x. The address of x is assigned to pointer p.

The pointer variables are always used to store address of another variable. The second statement displays the value of x using pointer p. Figure 4.8 explains pointers.

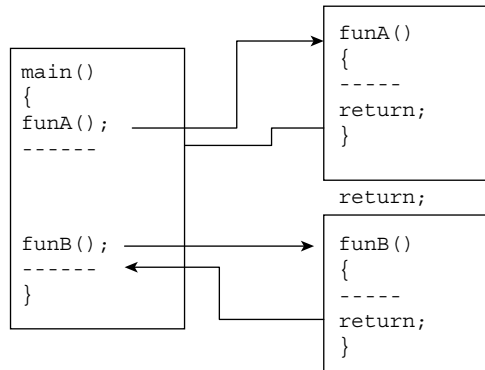


**Fig. 4.8** Pointers

## Functions

A function is a self-contained block or a sub-program of one or more statements that perform a special task when called. The

C++ functions are more civilized than C. It is possible to use the same name with multiple definitions known as function overloading. Figure 4.9 illustrates the functions.

**Fig. 4.9** Functions

A simple program on function is described as follows:

#### 4.14 Write a program to demonstrate user-defined functions.

```
#include<conio.h>
#include<iostream.h>

int main()
{
 clrscr();
 void show (void);
 show();
 return 0;
}

void show()
{
 cout<<"\n In function show()";
}
```

#### OUTPUT

**In function show()**

**Explanation:** In the above program, the function `show()` is defined. The function body contains only one `cout` statement. When the function is executed, a message is displayed. Functions are discussed in detail in Chapter 7 (Functions in C++).

### Arrays

Array is a collection of elements of similar data type in which each element is located in separate memory location, for example

```
int b[4];
```

The above statement declares an array `b[]` which can hold four integer values. The following program illustrates use of array.

#### 4.15 Write a program to declare, initialize an array. Display the elements of an array.

```
#include<conio.h>
#include<iostream.h>

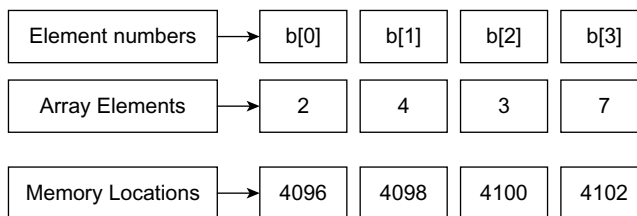
int main()
{
 clrscr();
 int b[4]={2,4,3,7};
 cout<<"b[0]="<<b[0]<<endl;
 cout<<"b[1]="<<b[1]<<endl;
 cout<<"b[2]="<<b[2]<<endl;
 cout<<"b[3]="<<b[3];
 return 0;
}
```



#### OUTPUT

```
b[0]=2
b[1]=4
b[2]=3
b[3]=7
```

**Explanation:** In the above program, an array `b[4]` is declared and initialized. The `cout` statement displays the array elements. `b[0]` refers to first element, `b[1]` refers to second element, and so on. Figure 4.10 describes the elements in an array and their memory locations.



**Fig. 4.10** Array elements and their memory locations

## References

### Referencing (&) and dereferencing (\*) operators

The `&` and `*` operators are used for referencing and dereferencing. The `&` symbol is also used in C++ to define reference types, and as a bitwise AND operator. We can also use the asterisk (`*`) as an operator to dereference a pointer and as a multiplication operator.

- (a) *Referencing operator (&)*: The referencing operator is used to define referencing variable. A reference variable prepares an alternative (alias) name for previously defined variable. The syntax of the referencing operator is as follows.

*Syntax:*

Data-type & reference variable name = variable name;

*Example:*

```
int qty=10;
int & qt=qty;
```

Here, `qty` is already declared and initialized. The second statement defines an alternative variable name, that is, `qt` to variable `qty`. If both variables printed displays the same value. Any change made in one of the variable causes the change in both the variables.

```
qt=qt*2;
```

Now, contents of `qt` and `qty` will be 20.

Note that the token **&** is not an address operator. The declaration `int &` indicates reference to data type `int`.

Principles for declaring reference variable are as follows:

- (1) A reference variable should be initialized.
- (2) Once a reference variable is declared, it should not refer to any other variable. Once the actual variable and reference variable are connected, they are tied jointly congenitally.
- (3) The reference variable can be created referring to pointer variable. The declaration is as follows:

```
char * h="C++" ;
char *&q=h;
```

- (4) A variable can contain various references. Modifying the value of one of them will result a change in all other variables.
- (5) Array of references is not allowed.

- (b) *Dereferencing operator (\*)*: The asterisk (\*) in a variable expression is used to declare a pointer to a given type. In the example,

```
int *x; Where, x is a pointer of integer type.
```

if the operand is a "pointer to function," the result is a function designator. If the operand is a pointer to an object, the result is an lvalue indicating that object. In the following conditions, the result of indirection is undefined.

- (1) The expression is a null pointer.
- (2) The expression is the address of an automatic variable and execution and it is out of scope.

### ***Difference between & and \* operator***

The **&** is a reference operator. It displays the address of a variable in the RAM. To display the address of the variable, it should be preceded by the variable name.



```

qt++;
cout<<qt<<" "<<(unsigned)&qt<<" "<<qty<<" "<<(unsigned
 (&qt)<<endl;

qt--;
cout<<qt<<" "<<(unsigned)&qt<<" "<<qty<<" "<<(unsigned
 (&qt)<<endl;

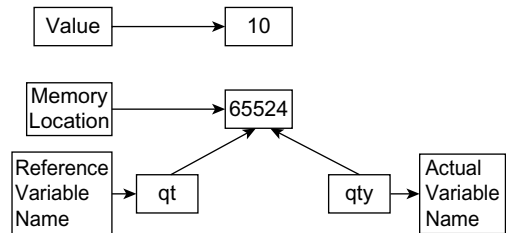
return 0;
}

```

## OUTPUT

| qty | Location | qt | Location |
|-----|----------|----|----------|
| 10  | 65524    | 10 | 65524    |
| 11  | 65524    | 11 | 65524    |
| 10  | 65524    | 10 | 65524    |

**Explanation:** In the above program, the variable `qty` is declared as integer variable and initialized with 10. The variable `qt` is declared as reference variable for variable `qty`. We can use variable `qt` to access the value of `qty`. Any change made in one of the variable changes the contents of both the variables. The contents of both variables and their addresses are always same. The variables `qt` and `qty` are modified using increment and decrement operator. But the contents and address printed of both variables are the same. Figure 4.12 explains the reference variable with respect to other variable.



**Fig. 4.12** Reference variable

## 4.4.3 User-Defined Data Type

User-defined data types are structure and classes, union, and enumerated

### Structure and Classes

- (1) **keyword struct:** *struct* is a keyword and used to combine variables of different data types into a single record.

Syntax:

```

struct < struct name >
{
 <data-type> <variable-name1, variable-name, 2>;
 <data-type> <variable-name3, variable-name, 4>;
} <structure variable declarations>;

```

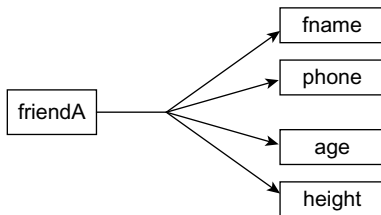
**struct name:** An optional tag name that defines the structure type.

**structure variables:** These are member variables and hold data

Though struct name and structure variables are noncompulsory, one of them should be present. Member variables in the structure are declared by naming a <data-type>, followed by one or more <variable-name> separated by commas.

A semicolon can separate variables of different data types.

```
Example:
 struct my_friend
 {
 char fname [80], phone[80];
 int age, height;
 } friendA ;
```



The structure my\_friend defines a variable containing two strings (fname and phone) and two integers (age and height)(Fig 4.13).

To access elements in a structure, record selector (.) called as dot operator is used. For example

```
strcpy (friendA.fname, "Sachin");
```

**Fig. 4.13** Structure and its elements

#### 4.17 Write a program to declare struct object, initialize it and display the contents.

```
#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 struct my_friend
 {
 char *fname;
 int phone, age;
 float height;
 } A ;
 A.fname="Bharat";
 A.phone=26251;
 A.age=22;
 A.height=4.5;
 cout<<"\n Contents of object A";
 cout<<"\nName:"<<A.fname;
 cout<<"\nPhone:"<<A.phone;
 cout<<"\nAge:"<<A.age;
 cout<<"\nHeight:"<<A.height;
```

```
}
```

## OUTPUT

### Contents of object A

**Name : Bharat**

**Phone : 26251**

**Age : 22**

**Height : 4.5**

**Explanation:** In the above program, struct `my_friend` is defined with four member variables. Identifier `A` is an object of struct `my_friend`. The initialization of data members of struct is done using dot operator with object `A`. The `cout` statements display contents of object `A`.

- (2) **keyword class:** The class is a new keyword introduced in C++. Its use is the same as struct keyword. To declare a class, following syntax is used.

Syntax:

```
<classkeyword> <class- name> [<:baseclasslist>]
{<member variable list>}
```

**Class keyword:** It is one of the keywords `class`, `struct`, or `union`.

**Class-name:** It can be any unique name inside its scope.

**Base classlist:** If the class is derived class, then it follows the list of the base class(es). It is optional.

**Member variable list:** Defines the data member variables and member functions.

Example:

```
class circle
{
 int radius; // data member
 int area (void); // member function
};
```

Reader may refer chapter 8 on class for more details.

## Union

A union is same as compared to a struct, but the only difference is that it allows the user to declare variables that share the same memory space.

The union requires bytes that are equal to the number of bytes required for the largest members. For example, if the union contains `char`, `integer`, and `long integer`, then the number of bytes reserved in the memory for the union is 4 bytes

Syntax:

```
union [<union name>]
{
 <data-type> <variable names>;
} [<union variables name>;]
```



```

Example
union charorint
{
 char c;
 int i;
} number;

```

C++ will allocate sufficient storage in union variable `number` to hold the largest element in the union. The union member variables `number.c` and `number.i` use the same memory location. In this way, writing into one will replace the other. Member variables of a union are accessed in the same way as a struct.

#### 4.18 Program to access variable of different data types in union.

```

#include<stdio.h>
#include<conio.h>
#include<iostream.h>

union ABC
{
 char a;
 int num;
 float f1;
}x;

int main()
{
 clrscr();
 x.a='d';
 cout<<"\nCharacter = "<<x.a;
 x.num=2;
 cout<<"\nNumber = "<<x.num;
 x.f1=4.55;
 cout<<"\nfloat number = "<<x.f1;
 return 0;
}

```

#### OUTPUT

```

Character = d
Number = 2
float number = 4.55

```

**Explanation:** In the above program, ABC is the name of the union. a, num, f1 are the variables of char, int, and float, respectively. X is the variable associated with the union type. Dot (.) operator is used to access the variable of any data types.

## Anonymous Unions

An anonymous union does not contain tag name. Elements of such union can be accessed without using tag name. Consider the following example.

```
Union
{
 int k;
 float j;
};
```

Both the member variables of union have the same memory location. They can be accessed as follows:

```
K = 20;
j = 2.2;
```

The declaration should not declare a variable of the above union type. Following program illustrates the use of anonymous union.

### 4.19 Write a program to declare anonymous union and access its elements.

```
#include<stdio.h>
#include<iostream.h>
#include<conio.h>
int main()
{
 clrscr();
 union
 {
 int k;
 float f;
 };
 f=3.1;
 k=2;
 cout<<"\n k ="<<k;
 printf ("\n f = %.1f",f);
 return 0;
}
```

### OUTPUT

```
k = 2
f = 3.1
```

**Explanation:** In the above program, **anonymous union** is declared. The union has two data member variables k as integer variable and f as a float variable. The union has no tag name; hence, it is called as anonymous union. The member variable of such a union can be accessed directly like normal variable. Both the variables hold the same memory location.

## Enumerated Data Type

The `enum` is a keyword. It is used for declaring enumeration data types. The programmer can declare new data type and define the variables of these data types that can hold. For example, the user can define the material as new data type. Its variable may be solid, liquid, and gas. Thus the three values are restricted for this new data type. These enumeration data types are useful in `switch()` case statement.

The syntax for enumerated data type is as follows and it uses a keyword **enum**.

```
enum logical { false, true};
enum logical {true=2, false=4};
enum components{solid, liquid, gas};
```

This statement declares a user-defined data type. The keyword **enum** is followed by the tag name `logical`. The enumerators are the identifiers `false` and `true`. Their values are constant unsigned integers and start from 0. The identifier `false` refers to 0 and `true` to 1. The identifiers are not to be enclosed with quotation marks. Also note that integer constants are not permitted and we can start the constants as given in the second statement. In the second statement, `true` refers to 2 and `false` refers to 4. In the third statement, the `components` is the user-defined data type and the variables attached to it are `solid`, `liquid`, and `gas`.

The *ANSI C++* and *Turbo C++* allow us to declare variables of `enum` type.

```
logical =N // N is of the type logical
logical F=false // valid
logical TT=1 // invalid in C++
logical TT=(logical) 1 // valid
int k=true; // valid
```

We can also define `enum` without tag name. Consider the following example.

```
enum{yes, no};
```

Here, `yes` is 0 and `no` is 1 and can be used as `int answer=yes`.

### 4.20 Write a program to declare `enum` data type and display their values.

```
#include<iostream.h>
#include<conio.h>
int main()
{
 clrscr();
 enum logical {false, true};
 cout<<"true:"<<true <<"false : "<<false;
 return 0;
}
```

### OUTPUT

```
true : 1 false : 0
```

**Explanation:** In the above program, enum data type logical is declared with two values, that is, false and true. The false contains value 0 and true contains the value 1. The `cout` statement displays the contents of true and false. True means 1 and false means 0.

## 4.5 OPERATORS IN C AND C++

Operator is an instruction to the compiler or interpreter specified by a single or double symbol to perform certain operation with constants.

### Example:

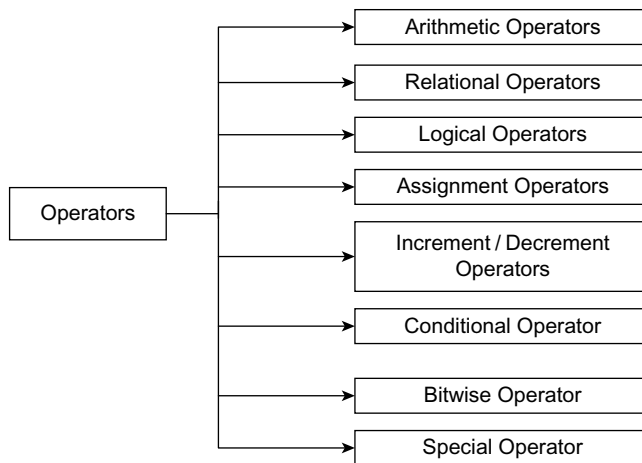
```
5+10
```

Here, 5 and 10 are constants. The symbol '+' is an operator that indicates the operation to be performed. The '+' performs addition of numbers. The '+' is a single operator.

```
int x = 5;
++x;
```

In the above example, the operator ++ is an increment operator. This operator adds one to the value of operand.

The types of operators (Fig. 4.14) with their symbols are described in Table 4.7.



**Fig. 4.14** Types of operators

**Table 4.7** Types of Operators

| Type of Operators                 | Symbolic Representation                 |
|-----------------------------------|-----------------------------------------|
| Arithmetic operators              | +, -, *, /, and %                       |
| Relational operators              | >, <, =, >=, <=, and !=                 |
| Logical operators                 | &&,   , and !                           |
| Increment and decrement operators | ++ and --                               |
| Assignment operators              | =, +=, -=, *=, /=, <<=, >>=, &=,  =, ^= |
| Bit wise operators                | &,  , ^, >>, <<, and ~                  |
| Special operator                  | ,                                       |
| Conditional operators             | ? and :                                 |

C++ supports all operators of 'C' and C++ introduces new additional operators. The new operators are listed in Table 4.8.

**Table 4.8** Operators in C++

| Operators | Description                                     |
|-----------|-------------------------------------------------|
| <<        | Insertion operator                              |
| >>        | Extraction operator                             |
| ::        | Scope access (or resolution) operator           |
| ::*       | Pointer to member decelerator                   |
| ->*       | Deference pointers to pointers to class members |
| .*        | Deference pointers to class members             |
| delete    | Memory release operator                         |
| new       | Memory allocation operator                      |

The scope access (or resolution) operator **::** (two colons) allows you to access a global (or file duration) name even if a local hides it redecoration of that name.

The **.\*** and **->\*** operators represents deference pointers to class members and deference pointers to pointers to class members, respectively.

#### 4.5.1 Precedence of Operators in C++

In the following operator precedence Table 4.9, the C++ operators are classified into 13 groups. The #1 group has the top (highest) precedence, group #2 (unary operators) takes second precedence, and so on to the comma operator, which has lowest precedence. The precedence of operator in C++ is shown in Table 4.9.

The operators within each category have equal precedence.

The unary (group #2), conditional (group #11), and assignment (group #12) operators associate right-to-left; all other operators associate left-to-right.

**Table 4.9** Precedence of Operators in C++

| # Group  | Operator                     | Operation                                                                                                                                             |
|----------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Top   | ()<br>[]<br>-><br>::<br>.    | Function call<br>Array subscript<br>C++ indirect component selector<br>C++ scope access/resolution<br>C++ direct component selector                   |
| 2. Unary | !<br>~<br>+<br>-<br>++<br>-- | Logical negation (NOT)<br>Bitwise (1's) complement<br>Unary plus<br>Unary minus<br>Pre-increment or post-increment<br>Pre-decrement or post-decrement |

(Table 4.9 Contd.)

(Table 4.9 Contd.)

|                   |        |                                         |
|-------------------|--------|-----------------------------------------|
|                   | &      | Address                                 |
|                   | *      | Indirection                             |
|                   | sizeof | Returns size of operand, in bytes       |
|                   | new    | Dynamically allocates C++ storage       |
|                   | delete | Dynamically de-allocates C++ storage    |
| 3. Multiplicative | *      | Multiply                                |
|                   | /      | Divide                                  |
|                   | %      | Remainder (modulus)                     |
| 4. Member Access  | .*     | C++ dereference                         |
|                   | ->*    | C++ deference                           |
| 5. Additive       | +      | Binary plus                             |
|                   | -      | Binary minus                            |
| 6. Shift          | <<     | Shift left                              |
|                   | >>     | Shift right                             |
| 7. Relational     | <      | Less than                               |
|                   | <=     | Less than or equal to                   |
|                   | >      | Greater than                            |
|                   | >=     | Greater than or equal to                |
| 8. Equality       | ==     | Equal to                                |
|                   | !=     | Not equal to                            |
| 9. Bitwise        | &      | Bitwise AND                             |
|                   | ^      | Bitwise XOR                             |
|                   |        | Bitwise OR                              |
| 10. Logical       | &&     | Logical AND                             |
|                   |        | Logical OR                              |
| 11. Conditional   | ?:     | (a ? x : y means "if a then x, else y") |
| 12. Assignment    | =      | Simple assignment                       |
|                   | *=     | Assign product                          |
|                   | /=     | Assign quotient                         |
|                   | %=     | Assign remainder (modulus)              |
|                   | +=     | Assign sum                              |
|                   | -=     | Assign difference                       |
|                   | &=     | Assign bitwise AND                      |
|                   | ^=     | Assign bitwise XOR                      |
|                   | =      | Assign bitwise OR                       |
|                   | <<=    | Assign left shift                       |
|                   | >>=    | Assign right shift                      |
| 13. Comma         | ,      | Evaluate                                |

All of the operators in this table can be overloaded except the following:

```
. C++ direct component selector
.* C++ dereference
:: C++ scope access/resolution
?: Conditional
```

#### 4.5.2 Precedence of \* and [ ] Operators

In C++, the statements `* x[4]` and `(* x) [4]` are not same because the `*` operator having lower precedence than the `[ ]` operator. Consider the following examples.

```
int *arr[5];
```

The above statement declares an array of five pointers and the following operation is invalid because the array name itself is an address and it is a constant. Hence, cannot be changed.

```
arr++; or ++arr;
int (* arr)[5]
```

The above declaration declares a pointer to an array of five elements. Hence, the operations such as `arr++` and `++arr` are not supported. The following program explains both these conditions.

#### 4.21 Write a program declare a pointer to array and display the elements.

```
#include<iostream.h>
#include<conio.h>
int main()
{
 void display (int[] [3],int);
 clrscr();
 int a[3][3]={ {11,22,33},
 {44,55,66},
 {77,88,99}
 };
 display (a,3);
 return 0;
}
void display (int x[] [3], int k)
{
 int (*d) [3];
 d=x;
 for (int g=0;g<k;g++)
 {
 for (int h=0;h<3;h++)
 cout<<d[g][h] <<"\t";
 cout<<"\n";
 }
}
```



#### OUTPUT

```
11 22 33
44 55 66
77 88 99
```

**Explanation:** In the above program, an integer array `a[3][3]` is declared and initialized. The base address of array and number of rows are passed to the function `display()`. In function `display()`, `d` is a pointer. The base address received by the variable `x` is assigned to pointer `d`. Using nested for loops, the elements of array are displayed.

## 4.6 SCOPE ACCESS OPERATOR

Like C, the variables declared in 'C++' programs are totally different from other languages. We can use the same variable names in the 'C++' program in separate blocks. The two declarations of the same variable refer different memory locations. When we declare a variable, it is available only to specific part or block of the program. Remaining block or other functions cannot access the variable. The area or block of the 'C++' program from where the variable can be accessed is known as the scope of variables.

The scope access (or resolution) operator `::` (two colons) allows programmer to access a global (or file duration) name even if it is hidden by a local re-declaration of that name.

### 4.22 Write a program to use scope access operator. Display the various values of the same variable declared at different scope levels.

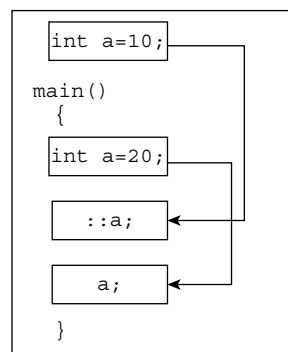
```
#include<iostream.h>
#include<conio.h>

int a=10;
int main()
{
 clrscr();
 int a=20;
 cout<<"::a=" <<::a;
 cout<<" a=" <<a;
 return 0;
}
```

### OUTPUT

**::a=10 a=20**

**Explanation:** In the above program, the integer variable 'a' is declared before `main()` and initialized with 10. It is a global variable. In the function `main()`, re-declaration of 'a' is done and this time the variable 'a' is initialized with 20. The first `cout` statement displays the global value of variable 'a', that is, 10. In this statement **scope access operator** is used to access the global value of the variable 'a'. The second `cout` statement displays the local value of the variable 'a', that is, 20. Figure 4.15 shows the use of scope access operator for accessing global variable.



## 4.7 NAMESPACE

Namespace is one of the new features introduced by the ANSI C++.

**Fig. 4.15** Scope access operator



It is used by programmers to avoid name clashes when programmer uses more than one library. If duplicate name with a matching scope is found in two parts of the program then name clash occurs.

All the items within the namespace have public visibility and the items declared within the namespace must include the C++ standard library and directives like `using namespace std;`.

Syntax of namespace is very similar to the classes or struct. The keyword `namespace` is followed by a namespace name, an opening curly brace, and terminated with closing brace without semicolon.

Syntax of the namespace is as follows-

```
Namespace name_of_the_namespace
{
.....
.....
.....
}
```

#### 4.23 Write a program to use namespace keyword and access variables by using scope resolution operator.

**Note:** Following program is executed using visual c++.

```
#include<iostream.h>
#include<conio.h>
namespace window
{
 int v=20;
 float f=30;
}
int main()
{
 cout<<window::v<<endl;
 cout<<window::f;
 return 0;
}
```

#### OUTPUT

20

30

**Explanation:** In the above program, variables `v` and `f` are declared within the namespace and are accessed by using scope resolution operator (`::`).

**4.24 Write a program to perform few arithmetic operations between the variables declared under two namespaces.**

**Note:** Following program is executed using visual c++.

```
#include<iostream.h>
namespace window
{
 int v=20;
 float f=2.5;
}
namespace wind
{
 int v=20;
 float f=2.5; e
}
int main()
{
 cout<<" Additon of two numbers:"<<endl;
 cout<<window::v+wind::v<<endl;
 cout<<"Multiplication of two numbers"<<endl;
 cout<<window::f*wind::f<<endl;
 return 0;
}
```

### OUTPUT

**Additon of two numbers:**

**40**

**Multiplication of two numbers**

**6.25**

**Explanation:** In the above program, two different namespaces are used. Variables declared within them are used for performing the mathematical operations such as addition and multiplication.

**4.25 Write a program to illustrate nested namespaces. Also use the unnamed namespace and display the variable declare and initialized in it.**

**Note:** Following program is executed using visual c++.

```
#include<iostream.h>
namespace xyz
{

```

```
int a=2;
char b='p';
namespace amit
{

int a=5;

char b='u';
}
}
namespace //unnamed namespace
{
 int g=100;
}
int main()
{
 cout<<"a="<<xyz::a<<endl;
 cout<<"b="<<xyz::b<<endl;
 cout<<"a="<<xyz::amit::a<<endl;
 cout<<"b="<<xyz::amit::b<<endl;
 cout<<"g="<<g;
 return 0;
}
```

## OUTPUT

```
a=2
b=p
a=5
b=u
g=100
```

**Explanation:** In this program nesting of namespaces is used and the variables declared under it are accessed. Unnamed namespace is used and variable within it is assigned a value which is displayed.

### 4.26 Write a program to declare a function using namespace and perform different operations.

```
#include<iostream.h>
namespace xyz
{
 int mul(int x,int y)
```

```
{
 return(x*y);
}
int div(int x,int y);
}
int xyz::div(int x,int y)
{
 return(x/y);
}
int main()
{
 cout<<"Multiplication ="<<xyz::mul(10,20)<<endl;
 cout<<"Division ="<<xyz::div(20,10)<<endl;
 return 0;
}
```

## OUTPUT

**Multiplication =200**

**Division =2**

**Explanation:** Two functions named as mul and div are taken in namespace. Arguments to the functions are passed from the main () and functions' results are displayed.

## 4.8 MEMORY MANAGEMENT OPERATORS

In 'C' language, we have studied the function malloc(), calloc(), and realloc() to allocate memory dynamically at run-time in the program. The free() function is used to release the resources allocated by these functions. C++ allows us to use these functions. In addition, C++ provides operators which help us to allocate and release the memory in easy way than these functions. These new operators are new and delete. The new operator creates an object and delete destroys the object. These operators are easy in writing as compared to malloc() and calloc(). The syntaxes for new and delete are illustrated with suitable programs.

Following are the advantages of new operator over the function malloc():

- (1) The new operator itself calculates the size of the object without the use of sizeof() operator.
- (2) It returns the pointer type. The programmers need not take care of its type casting.
- (3) The new operator allocates memory and initializes the object at once.
- (4) The new and delete operators are simple in syntax. They can be overloaded.

### 4.8.1 new Operator

Format of new operator is as follows:

```
pointer memory variable = new operator - data type[size];
```

Here, pointer memory variable is a pointer to the data type. The new operator allocates memory of specified type and returns back the starting address to the pointer memory variable. Here, the element size is optional and used when we allocation of memory space is required for user-defined data types such as arrays, classes, and structures. If the new operator fails to allocate the memory it returns *NULL*, which can be used to detect failure or success of new operator.

*Examples:*

- (a) `pv = new int;`
- (b) `int *pv = new int (50);`
- (c) `*p = new int [3]`

In example (a), pv is a pointer variable of integer type. Once allocated, the pv contains the starting address. In example (b), 50 is assigned to pointer variable pv. In example (3), memory for 3 integers, that is, 6 bytes are assigned to pointer variable p. The example of new operator with arrays are as follows:

```
pv= new int [5] [2]; // valid
pv= new int [8] [k] [2] // invalid
pv= new int [] [2] [2] // invalid
```

#### 4.27 Program to illustrate the simple example of new operator.

```
#include<conio.h>
#include<iostream.h>

int main()
{
 int *x;
 clrscr();
 x=new int [10];
 if (x==NULL)
 cout<<"\nMemory is not allocated";
 else
 cout<<"\nMemory is allocated";
 return 0;
}
```

#### OUTPUT

**Memory is allocated**

**Explanation:** In the above program, x is an integer pointer variable. The new operator allocates memory required for 10 integers, i.e. 20 bytes to pointer x as each integer occupies 2 bytes. new operator is used to replace the `sizeof()` operator.

### 4.8.2 delete Operator

The delete operator frees the memory allocated by the new operator. This operator is used when the memory allocated is no longer used in the program. The following syntax is used for the delete operator.

| Syntax                                             | Example                         |
|----------------------------------------------------|---------------------------------|
| a) delete <pointer memory variable>                | a) delete p;                    |
| b) delete [element size] <pointer memory variable> | b) delete [5 ]p or delete [ ]p; |

In example (a), the delete operator releases the memory allocated to pointer p. Example (b) is advantageous when we want to free the dynamically allocated memory of array. The new C++ compilers do not require element size.

#### 4.28 Program to illustrate the simple example of new operator.

```
#include<conio.h>
#include<iostream.h>
int main()
{
 int *x;
 clrscr();
 x=new int[10];
 if(x==NULL)
 cout<<"\nMemory is not allocated";
 else
 cout<<"\nMemory is allocated";
 delete x;
 return 0;
}
```

#### OUTPUT

**Memory is allocated**

**Explanation:** The above program shows allocation of memory by using new operator. The delete operator frees the memory allocated by the new operator.

#### 4.39 Write a program to allocate memory using new operator.

```
#include<iostream.h>
#include<conio.h>

void main()
```

```

{
 clrscr();
 int *p= new int[3],k; // Memory allocation for 3 integers
 for (k=0;k<3;k++)
 {
 cout<<"\nEnter a Number :";
 cin>>*p;
 p++; // Pointing to next location
 }
 p-=3; // Back to starting location
 cout<<"\n Entered numbers with their address are :\n";
 for (k=0;k<3;k++)
 {
 cout<<"\n\t"<<*p <<"\t"<<(unsigned)p; // type casting
 p++;
 }
 p-=3;
 delete p;
}

```

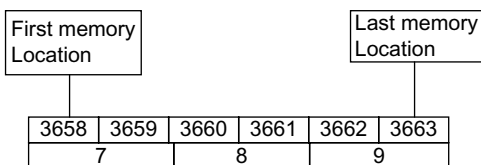
## OUTPUT

```

Enter a Number : 7
Enter a Number : 9
Enter a Number : 8
Entered numbers with their address are :
7 3658
9 3660
8 3662

```

**Explanation:** In the above program, `p` is an integer pointer variable. The `new` operator allocates memory required for three integers, i.e. 6 bytes to pointer `p`. The first `for` loop reads integer through the keyboard and stores the number at memory location pointed by `p` as shown in Figure 4.16. Each time, pointer `p` is incremented and it shows the next location of its type. The second `for` loop displays the number by applying the same logic. Before that the pointer is again set to the starting location by decrementing by 3. The `delete` operator releases the memory allocated by the `new` operator. Figure 4.16 shows the memory allocation of the different variable.



**Fig. 4.16** Memory allocated by the different variables

Before that the pointer is again set to the starting location by decrementing by 3. The `delete` operator releases the memory allocated by the `new` operator. Figure 4.16 shows the memory allocation of the different variable.

In the output of the program, only starting memory location numbers are displayed.

### 4.8.3 sizeof()

The `sizeof()` operator is used to return size occupied in bytes in memory by the variable. The `sizeof()` operator in C++ displays different values as compared to C. The following program illustrates this.

#### 4.30 Write a program to display number of bytes occupied by char data type.

```
#include<iostream.h>
#include<conio.h>

void main()
{
 clrscr();
 cout<<sizeof('a');
}
```

#### OUTPUT

1

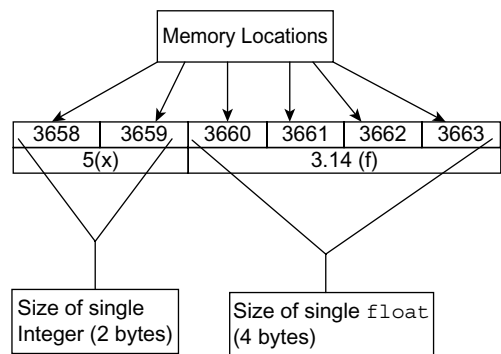
**Explanation:** In the above program, a character constant is used, `sizeof()` operator. The size determined is 1 byte. The same program 'C' will display the size 2 bytes, because C and C++ reacts differently with data types. In C, 'a' character is considered as integer. Hence the size displayed is 2 whereas in C++, it is considered as a character.

The size is the space occupied in memory in bytes by the variable. It depends upon the data type of variable. Figure 4.17 describes the space occupied in the memory by the variable integer and float.

*Example:*

```
int x = 5;
float f = 3.14;
```

The integer variable occupies two bytes and float variable occupies four bytes in memory.



**Fig. 4.17** Space occupied by the int and float

## 4.9 COMMENTS

In C++ a symbol `//` (double slash) is used as a comment symbol and there is no termination symbol. Comment begin with `//` symbol. A comment can be inserted anywhere in the line and whatever follows till the end of line is ignored. The C comment symbols `/*` and `*/` are also valid in C++.

Examples:

```
// This is a C++ comment style
/* The C comment style is also valid in C++. */
```



## 4.10 COMMA OPERATOR

In C, every statement of the program is terminated by semicolon (;). In C++ it is applicable and in addition C++ allows us to terminate a statement using comma operator after satisfying the following rules.

- (1) The variable declaration statements should be terminated by semicolon.
- (2) The statements followed by declaration statements like `clrscr()`, `cin`, and `cout` can be terminated by comma operator.
- (3) C++ permits declaration of variables at any point in program, but such declaration is not allowed in between the statements terminated by comma operator. The initialization of previously declared variables can be done.
- (4) The last statement of the program must be terminated by semicolon.

Consider the following programs.

### 4.31 Write a program to use comma operator in place of semi-colon.

```
#include<iostream.h>
#include<conio.h>

void main()
{
 clrscr(),
 cout<<"\nUse of comma operator",
 cout<<endl;
}
```

#### OUTPUT

**Use of comma operator**

**Explanation:** In function `main()`, the first two statements are terminated by comma operator and the last statement is terminated by semicolon.

All the above points are noticed in the Turbo C++ compiler. The reader is advised to follow his/her own observations.

### 4.32 Write a program to declare an integer, initialize it and display it. Terminate the statements using comma operator.

```
#include<iostream.h>
#include<conio.h>
void main()
{
 int x;
 clrscr(),
```

```
x=10,
cout<<"\n x = "<<x,
cout<<endl;
}
```

**OUTPUT**

**x = 10**

**Explanation:** In the above program, the first and last statements are terminated by semicolon. Comma operator terminates the statements in between these two statements.

**4.11 COMMA IN PLACE OF CURLY BRACES**

The curly braces ({} ) are used to define the body of function and scope of the control statements. The opening curly brace ({} ) indicates starting of the scope and closing curly ({} ) brace indicates the end of the scope. It is also possible to use comma operator in condition and loop statement in place of {} to indicate the scope of the statement. The use of comma operator is not allowed in definition of function. The following declaration is invalid.

```
main(
,
,
```

The following program explains the use of comma operator with conditional and loop statements.

**4.33 Write a program to use comma operator in if-else structure as scope indicator.**

```
#include<iostream.h>
#include<conio.h>

void main()
{
 int x;
 clrscr(),
 x=10;
 if (x==10) // if block
 cout<<x<<endl,
 cout<<x+1<<endl,
 cout<<x+2<<endl,
 cout<<"end of if block"; // end of if block terminated by //
 semi-colon
 else
 cout<<"False", // else block
```

```
 cout<<"\nEnd"; // end of else block terminated by semi-colon
}
```

### OUTPUT

```
10
11
12
end of if block
```

**Explanation:** In the above program, integer variable x is declared and initialized with 10. The if statement checks the value of x and executes respective blocks. The statements of if and else blocks are terminated by comma. The last statements of if block and else block are terminated by a semicolon to indicate the end of scopes. In this program, the if block is executed.

#### 4.34 Write a program to use comma operator in for loop to indicate the scope.

```
#include<iostream.h>
#include<conio.h>

void main()
{
 int x;
 clrscr(),
 x=5;
 for (;x>0;x--)
 cout<<"\nx = "<<x, // {
 cout<<" In loop",
 cout<<" In loop"; // }
 cout<<" Out of loop"; // executed after end of for loop
}
```

### OUTPUT

```
x = 5 In loop In loop
x = 4 In loop In loop
x = 3 In loop In loop
x = 2 In loop In loop
x = 1 In loop In loop Out of loop
```

**Explanation:** In the above program, the statements terminated by comma are included in the scope of for loop. The first two statements are terminated by comma and the last statement is terminated by semicolon, that is end of scope of for loop.

## 4.12 MORE PROGRAMS

### 4.35 Write a program to show the use of scope access operator.

```
#include<stdio.h>
#include<iostream.h>
#include<conio.h>

int j=11;
int main()
{
 clrscr();
 int j=12;
 cout<<"\n Local j = "<<j <<" Global j = "<<::j;
 ::j=21;
 cout<<"\n Local j = "<<j <<" Global j = "<<::j;
 return 0;
}
```

#### OUTPUT

**Local j = 12 Global j = 11**

**Local j = 12 Global j = 21**

**Explanation:** In the above program, variable `j` is declared in two scopes, that is, local and global. The variable `j` declared before `main()` is a global variable and the variable `j` is declared inside the `main()` function is local. Both global and local variables are initialized with 11 and 12, respectively. The local variable `j` can be accessed directly. The local variable hides the existence of global variable. The global variable can be accessed using `::` the scope access operator. The scope access operator prefixed with variable name displays the value of global scope and similarly we can also change the value. The output shows the values of local and global variables.



#### TIP

Whenever there is a tie between local and global variables, the local variable gets the preference.

### 4.36 Write a program to define three variables in different scopes and access them with and without a scope access operator.

```
#include<stdio.h>
#include<iostream.h>
#include<conio.h>
int j=10;
int main()
{
```

```

clrscr();
int j=20;
cout<<"\nj = "<<j <<" j = "<<::j;
{

int j=30;

cout<<"\nj = "<<j <<" j = "<<::j;
}
cout<<"\nj = "<<j <<" j = "<<::j;
return 0;
}

```

### OUTPUT

```

j = 20 j = 10
j = 30 j = 10
j = 20 j = 10

```

**Explanation:** In the above program, the variable `j` is declared in three different scopes. First is defined in before `main()`, second is inside the `main()`, and the third one is also inside the `main()` but within another block. The first `cout` statement displays the values of variable `j` of local and global scope. The second `cout` statement that is inside the block displays the value of variable `j` local to the same block and the global variable declared before `main()`. The third statement produces the same output as the first.

### 4.37 Write a program to demonstrate the use of reference variable.

```

#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 int x=15;
 int &y=x;
 cout<<endl<<"x = "<<x <<" y = "<<y;
 y=25;
 cout<<endl<<"x = "<<x <<" y = "<<y;
 x=x+y;
 cout<<endl<<"x = "<<x <<" y = "<<y;
 x--;
 cout<<endl<<"x = "<<x <<" y = "<<y;
 y--;
}

```



```

 cout<<endl<<"x = "<<x <<" y = "<<y;
 cout<<endl <<"Address of x=" <<(unsigned)&x <<"\nAddress of
y="<< (unsigned) &y;
 return 0;
}

```

## OUTPUT

```

x = 15 y = 15
x = 25 y = 25
x = 50 y = 50
x = 49 y = 49
x = 48 y = 48
Address of x=65524
Address of y=65524

```

**Explanation:** In the above program integer variable x is initialized to 15 and y is declared reference to variable x. The variable x and y refers to the same memory location and change in one variable can affects other. In the above program values of variable x and y are modified and values and address are displayed. From the output we notice that value and address of the variable x and y are displayed same.

### 4.38 Write a program to pass structure variable into function using c and c++ style.

```

#include<iostream.h>
#include<conio.h>
#include<string.h>

struct boys
{
 char name[20];
 int age ;
};

int main()
{
 clrscr();
 void cstyle(boys *);
 void cppstyle(boys &);
 boys b1={"Ankit",15};
 boys b2={"Saurav",10};
 cstyle(&b1);
 cout<<"\n Name : "<<b1.name<<" Age : "<<b1.age;
 cppstyle(b2);
 cout<<"\n Name : "<<b2.name<<" Age : "<<b2.age;
}

```

```

 return 0;
}
void cstyle (boys *b)
{
 strcpy (b->name, "Suraj");
 b->age=11;
}
void cppstyle (boys &b)
{
 strcpy (b.name, "Harsha");
 b.age=5;
}

```

## OUTPUT

**Name : Suraj Age : 11**

**Name : Harsha Age : 5**

**Explanation:** In the above program, the structure boy is declared before main() with two data members name and age of char and int data type, respectively. Two user-defined functions are defined as cstyle() and cppstyle(). In function main(), b1 and b2 are two variables of the struct boy type and they are initialized. The address of b1 is passed to the cstyle() function whereas cppstyle() needs only variable name. The cstyle() function receives address and stores it in the pointer. The cppstyle() function accept reference of the actual variable and stores it in the format variable. The code inside these functions directly replaces the old values with new ones. The result is displayed as per the output.

### 4.39 Write a program to return an object by reference.

```

#include<iostream.h>
#include<conio.h>
#include<string.h>

struct boys
{
 char name[20];
 int age ;
};

boys b1={"Ankit",15};
boys b2={"Saurav",10};
int main()
{
 clrscr();
 boys & show();
 show()=b2;
 cout<<"\n Name : "<<b1.name<<" Age : "<<b1.age;

```



```
 return 0;
}
boys & show()
{
 cout<<"\n Name : "<<b1.name<<" Age : "<<b1.age;
 return (b1);
}
```

### OUTPUT

**Name : Ankit Age : 15**

**Name : Saurav Age : 10**

**Explanation:** In the above program, structure boys is declared. The objects b1 and b2 are declared of structure boys type. Both the structure variables are initialized. The function show() is declared and defined. The statement show()=b2; passes contents of b2 into function show(). The contents of variable b1 are displayed that is similar to b2. The function returns object b1 and again content of b1 is displayed.

#### 4.40 Write a program to declare constant variable and calculate area of a circle.

```
#include<iostream.h>
#include<conio.h>
#include<math.h>

int main()
{
 float r, area;
 const float pi=3.14;
 clrscr();
 cout<<"\n Enter radius : ";
 cin>>r;
 area= pow(r,2)*pi;
 cout<<"\n Area of circle : "<<area;
 return 0;
}
```

### OUTPUT

**Enter radius : 2**

**Area of circle : 12.5**

**Explanation:** In the above program, variable r and area are of float type. The variable pi is constant and initialized to 3.14. The value of r is entered through the keyboard. Area of circle is calculated using the formula. The pow() function calculates the square of the variable r. The cout statement displays the area of the circle. The variable pi is constant and cannot be changed during the program execution.



**4.41 Write a program to declare reference variable to const variable.**

```
#include<iostream.h>
#include<conio.h>
#include<math.h>

int main()
{
 clrscr();
 const int x=15;
 int & y=x;
 cout<<" x = "<<x <<" y = "<<y;
 y=25;
 cout<<"\n x = "<<x <<" y = "<<y;
 return 0;
}
```

**OUTPUT**

**x = 15 y = 15**

**x = 15 y = 25**

**Explanation:** In the above program, integer x is a constant variable and initialized with 15. The variable y is a reference variable to variable x. We cannot change the value of x as it is a constant variable. But we can change the value of its reference variable. The change in reference variable does not change the value of constant variable. The output shows the values of both the variables.

**4.43 Write a program to declare a constant variable.**

```
#include<iostream.h>
#include<conio.h>
#include<math.h>

int main()
{
 clrscr();
 int x=15;
 const int & y=x;
 cout<<" x = "<<x <<" y = "<<y;
 x=25;
 cout<<"\n x = "<<x <<" y = "<<y;
 return 0;
}
```

**OUTPUT**

**x = 15 y = 15**

**x = 25 y = 25**

**Explanation:** The above program is the same as previous one. In this program the reference is declared as constant. We cannot change the value of the reference variable. A change made in actual variable changes the value of the reference variable.

#### 4.44 Write a program to declare reference variable to character pointer and display the strings.

```
#include<iostream.h>
#include<conio.h>
#include<math.h>

int main()
{
 clrscr();
 char *txt1="Eye";
 char *& txt2=txt1;
 cout<<"\n"<<txt1 <<" "<<txt2;
 *txt1='B';
 cout<<"\n"<<txt1 <<" "<<txt2;
 *txt2='d';
 cout<<"\n"<<txt1 <<" "<<txt2;
 return 0;
}
```

#### OUTPUT

Eye Eye  
Bye Bye  
dye dye

**Explanation:** In the above program, txt1 is a character pointer and initialized with the string "Eye". The pointer txt2 is a reference variable to the character pointer txt1. The statement \*txt1='B'; replaces the first character of the string with character 'B' and contents of both the pointers are displayed. The statement \*txt2='d'; replaces the first character of the string with variable 'd' through the reference variable.

#### 4.45 Write a program to return a constant reference.

```
#include<iostream.h>
#include<conio.h>
#include<math.h>
int main()
{
 clrscr();
 const char *show();
 const char *s;
```

```

 s=show();
 // *s='W'; can not modify the constant object
 cout<<s;
 return 0;
}
const char *show() { return "Hello"; }

```

## OUTPUT

**Hello**

**Explanation:** In the above program, the function `show()` and pointer `s` are declared as constants. It is not possible to assign any data to pointer `s` because it is a constant. The function `show()` is invoked and it returns string. The returned string is assigned to pointer `s`. The content displayed is "Hello".

### 4.46 Write a program to use global and local variable and their pointer of same name and calculate their sum.

```

#include<iostream.h>
#include<conio.h>
int s=15;
int *sp=&s;
int main()
{
 clrscr();
 int s=20;
 int *sp=&::s;
 cout<<"Local Variable s = "<<s<<endl;
 cout<<"Global Variable ::s = "<<::s<<endl;
 cout<<"Local pointer sp = "<<*sp<<endl;
 cout<<"Global pointer ::sp = "<<*::sp<<endl;
 cout<<"::s + s = " <<::s+s<<endl;
 cout<<"*::sp+ *sp = "<<*::sp+*sp;
 return 0;
}

```

## OUTPUT

**Local Variable s = 20**

**Global Variable ::s = 15**

**Local pointer sp = 15**

**Global pointer ::sp = 15**

**::s + s = 35**

**\*::sp+ \*sp = 30**

**Explanation:** In the above program, variable `s` and pointer `sp` are declared in two scopes, that is, `global` and `local`. The address of global variables is assigned to both the local and global pointers `sp` using the following statements:

- (a) `int *sp=&s.;`
- (b) `int *sp=&::s;`

In statement (a), the address of global variable `s` is assigned to global pointer `sp`. In statement (b), the scope access operator is used to access the global variable and ampersand pre-fixed assigns address of global variable to local pointer `sp`. Thus, both the local and global pointers point to the same variable. The output indicates contents of individual variables and pointers and sum of local and global variables. The last sum result is 30 since both pointers point to the same variable, that is global variable.

#### 4.47 Write a program to perform division of local and global variable of same name.

```
#include<iostream.h>
#include<conio.h>

#define G ::
int loop=110;
int main()
{
 clrscr();
 register loop=2;
 for (;loop<10;loop+=2)
 cout<<endl<<G loop/loop;
 return 0;
}
```

#### OUTPUT

```
55
27
18
13
```

**Explanation:** In the above program, `G` is a macro initialized with scope access operator. We can use `G` in place of `::` scope access operator. The variable `loop` is declared and initialized in two scopes (local and global). Using `for loop` repetitive division of global and local `loop` variable is performed and result is displayed.

#### 4.48 Write a program to create variable with reference to pointer variable. Initialize and display the values.

```
#include<iostream.h>
#include<conio.h>
```

```

int main()
{
 clrscr();
 int a=175;
 int *x=&a;
 int &j=*x;
 cout<<"a = "<<a <<"j = "<<j <<"*x = "<<*x <<endl;
 int q=147;
 x=&q;
 q=155;
 cout<<"a = "<<a <<" j="<<j <<"*x = "<<*x<<endl;
 return 0;
}

```

### OUTPUT

**a= 175 j = 175 \*x = 175**

**a = 175 j=175 \*x = 155**

**Explanation:** In the above program, integer a is declared and initialized with value 175. The pointer variable x is declared and initialized with address of variable a. The reference variable j is declared and value pointer x is assigned to it. The cout statement displays the values of a, j, and x. The value printed is 175. The integer variable q is declared and initialized with 147. Its address is assigned to pointer x and again q is initialized with 155. This time the values of a, j, and x displayed are 175, 175, and 155, respectively. The value of pointer is altered but values of actual and reference variables are unaffected.

#### 4.49 Write a program to use type casting and convert values from one to another data type.

```

#include<conio.h>
#include<iostream.h>
int main()
{
 clrscr();
 int x;
 float y=2.1;
 cout<<"int (15.2) = "<<int(15.2) <<endl;
 cout<<"int (85.13)= "<<int(85.13) <<endl;
 cout<<"y = "<<y<<endl;
 x=int(y);
 cout<<"x="<<x<<endl;
 y=float(x)+1.5;
 cout<<"y = float(x)+1.5 = "<<y;
}

```

```
 return 0;
}
```

## OUTPUT

```
int(15.2) = 15
int(85.13) = 85
y = 2.1
x = 2
y = float(x) + 1.5 = 3.5
```

**Explanation:** In the above program, x is an integer variable and y is a float variable. The variable y is initialized with value 2.1. The values 15.2 and 85.13 are displayed by converting them to integer using type casting syntaxes.

The value of float y is assigned to integer variable x using type casting. The values of x and y are displayed on the screen. Again variable y is assigned with x + 1.5. The result will be 3.5 and not 3.6 because by applying float type cast we cannot convert an integer value to float.

## SUMMARY

- (1) C++ programs consist of objects, classes, functions, variables, and other statements.
- (2) The 'C++' keywords are reserved words by the compiler. All 'C' language keywords are valid in C++ and a few additional keywords by ANSI committee are also valid.
- (3) Identifiers are names of variables, functions, and arrays. They are user-defined names, consisting of a sequence of letters and digits, with a letter as a first character.
- (4) The constants in 'C++' are applicable to the values, which do not change during the execution of a program. The two types of constants are literal and symbolic.
- (5) C++ supports all data types of C.
- (6) The keywords `signed`, `unsigned`, `short`, and `long` are type modifiers. A type modifier changes the meaning of the base data type to produce a new data type.
- (7) The `void` type is an empty data type.
- (8) The `enum` is a keyword. It is used for declaring enumeration data types. C++ allows us to declare a variable of `enum` data type and `enum` without tag name.
- (9) Variables are used to store constants, that is, information. A variable is a sequence of memory locations, which are used to store the assigned constant.
- (10) C++ permits declaration of variables anywhere in the program.
- (11) The initialization of variable at run-time is called as dynamic initialization.
- (12) C++ supports all the operators of 'C'. The list of newly introduced operators is described in this chapter.
- (13) The referencing operator is used to define referencing variable. A reference variable prepares an alternative (alias) name for a previously defined variable.
- (14) The asterisk (\*) in a variable expression is used to declare a pointer to a given type.

- (15) The scope access (or resolution) operator `::` (two colons) allows a programmer to access a global (or file duration) name even if it is hidden by a local re-declaration of that name.
- (16) The `new` operator creates an object and `delete` destroys the object. These operators are easy in writing as compared to `malloc()` and `calloc()`.
- (17) C++ reference types, declared with `&` operator, are nearly identical but not exactly the same as the pointer types. They declare aliases for object variables and allow the programmer to pass arguments by reference to functions.
- (18) In C++, a symbol `//` (double slash) is used as a comment symbol and there is no termination symbol.

## EXERCISES

### (A) Answer the following questions

- (1) Describe different parts of C++ programs.
- (2) List the new keywords in C++ with their functions.
- (3) What are identifiers, variables, and constants?
- (4) What are the two types of constants? Describe them with suitable examples.
- (5) Describe the statements for creating constants. Explain with examples.
- (6) Describe the use of keyword `void`? In how many ways can it be used with a function?
- (7) What is the difference between variable declaration in C and C++?
- (8) What is dynamic initialization? Is it possible in 'C'?
- (9) Describe the use of scope access operator (`::`) and reference operator (`&`).
- (10) What are the advantages of `new` operator over `malloc()` function?
- (11) What are type modifiers? Why they are essential in C++?
- (12) Describe the types of derived data type.
- (13) Describe the following terms:
  - a) Precedence of operators in C++
  - b) Type modifiers
  - c) Constant pointers
- (14) What do you mean by namespace?
- (15) Explain the nesting of namespace with a suitable program.
- (16) What is the difference between namespace and class?
- (17) Write a short note on comma operator.

### (B) Answer the following by selecting the appropriate option

- (1) In C++, the symbol used for writing comments is
  - (a) `//`
  - (b) `/* */`
  - (c) `*/ */`
  - (d) none of the above
- (2) The `::` is known as
  - (a) **scope access operator**
  - (b) two colons
  - (c) both (a) and (b)
  - (d) none of the above
- (3) The scope resolution operator (`::`) is used to access the
  - (a) local variables
  - (b) **global variables**
  - (c) both local and global variable
  - (d) none of the above
- (4) The `delete` operator is used
  - (a) **to delete object**
  - (b) to delete file
  - (c) both (a) and (b)
  - (d) none of the above

- (5) The new and delete are  
 (a) operators  
**(b) keywords**  
 (c) both (a) and (b)  
 (d) none of the above
- (6) The new operator  
**(a) allocates memory**  
 (b) releases memory  
 (c) both (a) and (b)  
 (d) none of the above
- (7) The union declared without tag name is called as  
**(a) anonymous union**  
 (b) nameless union  
 (c) unknown union  
 (d) void union
- (8) The symbol defined using #define are called  
 (a) micro  
 (b) small  
**(c) macro**  
 (d) marco
- (9) What will be the output of the following program?
- ```
#include<iostream.h>
int main()
{
    char *n;
    cout<<sizeof(n);
    return 0;
}
```
- (a) 2**
 (b) 1
 (c) 4
 (d) none of the above
- (10) What will be the value of c after execution of the following program?
- ```
#include<iostream.h>
void main()
{
 int *p, c=0;
 p=new int[4];

 for (int x=0;x<2;x++)
 c=c+sizeof((p+x));
 cout<<c;
}
```
- (a) 4**                      (b) 8  
 (c) 2                      (d) 0
- (11) What will be the output of the following program?
- ```
#include<iostream.h>
void main()
{
    {
        for ( int i=0;i<5;i++)
        { cout<<i; }
    }
    cout<<" i ="<<i;
}
```
- (a) 01234 i =5**
 (b) undefined symbol i
 (c) 012345
 (d) none of the above
- (12) What will be the output of the following program?
- ```
#include<iostream.h>
#include<conio.h>
union aa
{
 int a;
 char b;
 float c;
 unsigned int d;
 double e;
}z;
int main()
{
 clrscr();
 cout<<sizeof(z);
 return 0;
}
```
- (a) 8**  
 (b) 4  
 (c) 17  
 (d) none of the above
- (13) What will be the output of the following program?
- ```
#include<iostream.h>
#include<conio.h>
struct aa
{
```


- ```

 int a;
 char b;
 float c;
 unsigned int d;
 double e;
 }z;
int main()
{
 clrscr();
 cout<<sizeof(z);
 return 0;
}

```
- (a) 8  
(b) 4  
**(c) 17**  
(d) none of the above
- (14) What will be the output of the following program?
- ```

#include<iostream.h>
namespace aaa
{
    int b=10;
}
int main()
{
    cout<<b;
}

```
- (a) garbage value
(b) error
(c) 10
(d) none of the above
- (15) What will be the output of the following program?
- ```

#include<iostream.h>
namespace
{
 char b='a';
}
int main()
{
 cout<<b;
}

```
- (a) Error  
**(b) a**  
(c) garbage value  
(d) none of the above
- (16) What will be the output of the following program?
- ```

#include<conio.h>
#include<iostream.h>

void main()
{
    clrscr();
    float x=5.5,y=10.0,*p,*q,r;
    p=&x;
    q=&y;
    r=*p++*++*q/2;
    cout<<"\n Value of r="<<r;
    getch();
}

```
- (a) 45.67
(b) 30.25
(c) 0
(d) error
- (17) What will be the output of the following program?
- ```

#include<iostream.h>
#include<constream.h>
const int tax=1000;
void main()
{
 clrscr();
 int value=10000;
 int cost;
 cost=tax+value;
 cout<<"\n"<<cost;
 getch();
}

```
- (a) **11000**  
(b) 1000  
(c) 11  
(d) 110
- (18) What will be the output of the following program?
- ```

#include<iostream.h>
#include<conio.h>
void main()
{
    clrscr();
    int *price= new

```

```

int [1], var;
*price=500, price++;
price-=1;
cout<<"Price :";
cout<<" "<<*

```

```

price, delete price;
}
(a) 50          (b) 500
(c) 5000        (d) 5

```

(C) Attempt the following programs

- (1) Write a program to allocate memory using new operator for 10 integers. Read and display the integers.
- (2) Write a program to evaluate the following series.
 - (a) $x = x^2 + x^3 + \dots - x^n$
 - (b) $y = 2^x + 2^*(x^3 - 10)$
 - (c) $z = x - y$ (use (a) and (b))
 - (d) Display square root of z .
- (3) Write a program to display A to Z characters using while loop.
- (4) Write a program to draw a square box. Use for loop.
- (5) Write a program to declare and initialize a variable. Create a reference variable. Display the value of actual variable using reference variable.
- (6) Given the Van der Waals constants x and y for a gas, calculate the critical temperature, pressure, and volume using the following formulas.

$$C_t = 8x/27Rb$$

$$p = x/27b^2$$

$$v = 3b$$

$$R = 0.0821 \text{ dm}^3\text{atm/mol/k}$$
 Read values of x and y and calculate and print the values of C_t , p , and v .
- (7) The sum of the square of the first n natural numbers is calculated by the formula $\text{sum} = n(n+1)(2n+1)/6$. Read value of n through the keyboard and calculate the sum of square of first n natural numbers.
- (8) Write a program to calculate
 - (a) Area of circle ($\text{area} = 3.14 * r^2$)
 - (b) Circumference of the circle ($c = 2 * 3.14 * r$)
 - (c) Volume of the cylinder ($v = 3.14 * r^2 * h$)
 - (d) Surface area of the closed cylinder ($s = 2 * 3.14 * r * h + 2 * 3.14 * r^2$)
 - (e) Volume of sphere ($v = 4/3 * 3.14 * r^3$)
- (9) A company gives the following rates of commission for the monthly sales of the product.

Below Rs. 15000	No commission
150001–20000	5%
200001–30000	10%
Above 30000	12%

Write a program to read the sales and display the commission.
- (10) Write a program to display the sum of odd numbers between 1 and 150.
- (11) A worker takes a job for 31 days. His pay for the first day is Rs. 20. His pay for the second day is Rs. 40. Pay for each day is twice what he gets the previous day. What will be the total pay for 31 days?
- (12) Write a program to find the range of the given numeric data.
(Range: smallest number – largest number)
- (13) Write a program to find the factorial of entered name using namespace.

This page is intentionally left blank.

Decision Statements

5

CHAPTER

CHAPTER OUTLINE

- 5.1 Introduction
- 5.2 The `if` Statement
- 5.3 Multiple `ifs`
- 5.4 The `if-else` Statement
- 5.5 Nested `if-else` Statements
- 5.6 The `else-if` Ladder
- 5.7 Unconditional Control Transfer Statements
- 5.8 The `switch` Statement
- 5.9 Nested `switch case`

5.1 INTRODUCTION

A program is nothing but the execution of a sequence of one or more instructions. In a monolithic program, the sequences of instructions are executed in the same order as they appear in the program. Quite often, it is desirable to alter the sequence of the statements in the program depending upon certain circumstances. In practical applications, there are a number of situations where one has to change the decisions based on the requirements/conditions. This involves taking a decision to see whether a particular condition is satisfied

Decision-making statements in a programming language help the programmer to transfer the control from one part of the program to another part. Thus, these decision-making statements enable the programmer to determine the flow of control.

On the basis of applications it is essential to:

- Alter the flow of a program
- Test the logical conditions and
- Control the flow of execution as per the selection.

These conditions can be placed in the program using decision-making statements. C++ language supports the decision making-statements as listed below.

- The `if` statement
- The `if-else` statement
- The nested `if-else` statements.
- The `else-if` ladder
- The `switch case` statement.
- The `break` statement
- The `default` keyword

The decision-making statement checks the given condition and then executes its sub-block. The decision statement decides the statement to be executed after the success or failure of a given condition.

5.2 THE `if` STATEMENT

Among the decision-making statements in C++, the `if` statement is the simplest one. C++ uses the keyword `if` to execute a set of command lines or a command line when the logical condition is true. It has only one option. The syntax for the simplest `if` statement is as shown in Figure 5.1.

```
Syntax for the simplest if statement:  
if (expression) /* no semi-colon */  
    Statement;  
  
Syntax for the simplest if statement:  
if (expression) /* no semi-colon */  
{  
    Statement 1;  
    Statement 2;  
    -----  
    -----  
}
```

Fig. 5.1 The `if` statement

The `if` statement contains an expression. The expression is always enclosed within a pair of parentheses. The conditional statements should not be terminated with semicolons (;). The statements following the `if` statement are normally enclosed within curly braces. The curly braces indicate the scope of the `if` statement. The default scope is one statement. But it is a good practice to use curly braces even with a single statement.

The `if` keyword is followed by an expression in parentheses. The expression is evaluated. If the expression is true, it returns 1, otherwise 0. The value 1 or any non-zero value is considered as true and 0 as false. In C++, the values (1) true and (0) false are known `bool` data type. The `bool` data type occupies one byte in memory. If the given expression in the `if` statement is true, the following statement or block of statements are executed; otherwise, the statement that appears

immediately after the if block (true block) is executed as given in Figure 5.2.

As shown in Figure 5.2, the expression is always evaluated to true or false. When the expression is evaluated as true, the statements within the if block are executed and the program continues from the next statement. When the expression is false, the if block is skipped and statement 3 is executed.

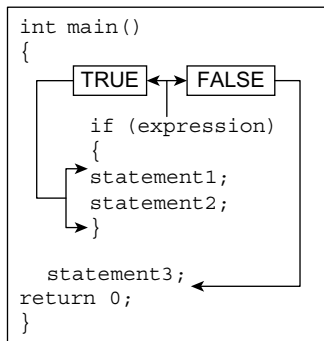


Fig. 5.2 The simple if statement

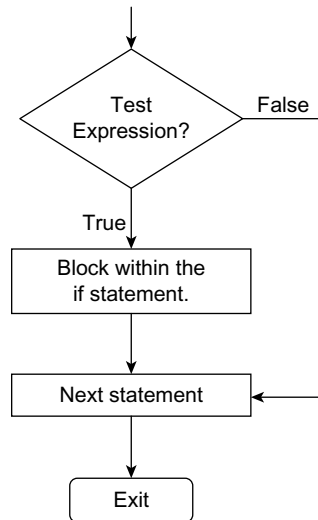


Fig. 5.3 Flowchart for if statement

The expression given in the if statement may not be always true or false. For example, if (1) or if (0). When such a statement is encountered, the compiler will display a warning message “condition is always true” or “condition is always false”. Instead of the expression, we can also use the function that returns 0 or 1 return values. The following programs illustrates all the above discussed points.

Flowchart for if statement is as shown in Figure 5.3.

Example of a simple if statement.

5.1 Write a program to declare the price of the book with if statement and check its price. If its price is less than or equal to 600, print the output with some comment, else terminate.

Algorithm:

Step 1: Start.

Step 2: Declare the price of a book with some variable.

Step 3: Check the condition with if statement i.e. price<=600.

Step 4: If 'yes' print the output with some comment, otherwise terminate.

Step 5: End.

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
int main()
```

```
{
```

```
    int price;
```

```
    clrscr();
```

```
    cout<<"\nEnter the price of the book:";
```

```
    cin>>price;
```

```
    if(price<=600)
```

```
{
    cout<<"\n Hurry up buy the book!!!!!";
}
return 0;
}
```

OUTPUT

Enter the price of the book: 345

Hurry up buy the book!!!!!

Explanation: In the above program, the price of the book is entered through the keyboard. If entered price is less than or equal to 600, the message displayed is **“Hurry up buy the book!!!!!”**.

5.2 Write a program to check the equivalence of two numbers.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int m,n;
    clrscr();    cout<<"\nEnter two numbers:";
    cin>>m>>n;
    if(m-n==0)
        cout<<"\n Two numbers are equal.";
    return 0;
}
```

{coderipe}

OUTPUT

Enter two numbers: 5 5

Two numbers are equal.

Explanation: The two numbers are entered. They are checked with if statement. If the difference of two numbers is zero, it prints the message “Two numbers are equal”.

5.3 Write a program to use library function `strlen()` and check for a string. If the string is non-zero, then display some message.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>

int main()
```

```
{
    static char nm[]="Hello";
    clrscr();
    if (strlen(nm))
    {
        cout<<"The string is not empty.";
    }
    return 0;
}
```

OUTPUT

The string is not empty.

Explanation: In the above program, the character array nm[] is initialized with the string "Hello". The strlen() function is used in the if statement. The strlen() function calculates the length of the string and returns it. If the string is empty, it returns (0) false; otherwise, it returns a non-zero value (string length). The non-zero value is considered as true. The strlen() function returns non-zero value (true). The if statement displays the message "The string is not empty." Here, instead of an expression, a library function is used.

5.3 MULTIPLE ifs

In our daily life also we take alternate decisions to satisfy conditions such as should we go for purchasing a laptop, desktop, or tablet based on the budget/specifications of a device/application. In C++ language, we take the help of multiple ifs to tackle such a problem.

The syntax of multiple ifs is shown in Figure 5.4. Programs on multiple ifs are as follows.

Syntax for the multiple ifs:

```
if (expression) /* no semi-colon */
    Statement 1;
if (expression) /* no semi-colon */
    Statement 2;
if (expression) /* no semi-colon */
    Statement 3;
```

Fig. 5.4 Multiple ifs

5.4 Write a program to enter the percentage of marks obtained by a student and display the class obtained.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    float per;
    clrscr();
    cout<<"\nEnter the percentage:";
    cin>>per;
```



```
if(per>=90 && per<=100)
    cout<<"\nDistinction";
if(per>=70 && per<90)
    cout<<"\nFirst class";
if(per>=50 && per<70)
    cout<<"\nSecond class";
if(per>=40 && per<50)
    cout<<"\nPass";
if(per<40)
    cout<<"\nFail";
return 0;
}
```

OUTPUT

Enter the percentage: 95
Distinction.

5.5 Program to display whether an entered character is in upper case, lower case, or not an alphabet.

```
#include<iostream.h>
#include<conio.h>
int main()
{
    char c;
    clrscr();
    cout<<"\nEnter the alphabet:";
    cin>>c;
    if(c>=65 && c<=90)
        cout<<"\nEntered alphabet is in Upper case";
    if(c>=97 && c<=122)
        cout<<"\nEntered alphabet is in Lower case ";
    if(c>=48 && c<=57)
        cout<<"It is not an alphabet.";
    return 0;
}
```



OUTPUT

Enter the alphabet: 7
It is not an alphabet.

Explanation: The program prompts the user to enter an alphabet. In case an uppercase letter is entered, the first if statement is executed, otherwise, for lowercase letters the second if statement is executed. For digit entry the third if is executed.

5.4 THE if-else STATEMENT

In the if-else statement, if the expression/condition is true, the body of the if statement is executed; otherwise, the body of the else statement is executed. The else keyword is used when the expression is not true. The format/syntax of if-else statement is given in Figures 5.5, 5.6 and 5.7.

As shown in Figure 5.7, both if and else blocks contain statement/statements. When the expression is true, the if block is executed, otherwise, the else block is executed.

```
if ( expression)
execute the statement1;
else
execute the statement2;
```

Fig. 5.5 The if-else statement

OR

```
Syntax of if-else statement can be given as follows.
if (expression is true) // if block
{
    statement1;
    statement 2;
}
else // else block
{
    statement 3;
    statement 4;
}
```

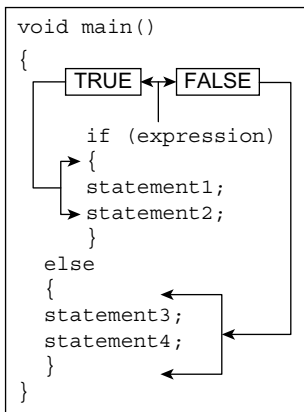


Fig. 5.6 The if-else statement

Fig. 5.7 The if-else statement

Flowchart for if-else is as shown in Figure 5.8.

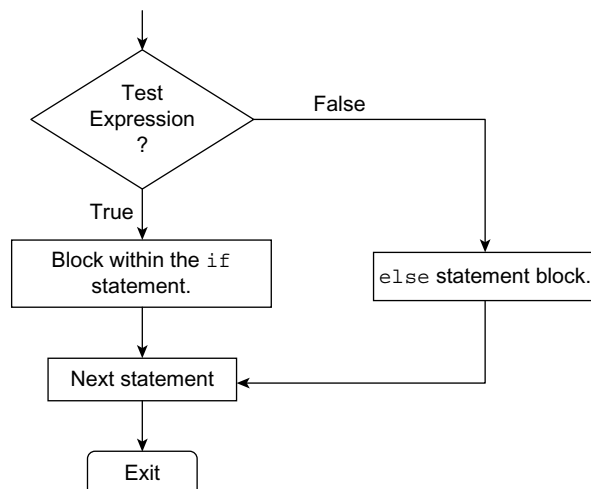


Fig. 5.8 The if-else flow-chart

Consider the following programs for understanding the `if-else` statement.

5.6 Write a program to enter age and display a message whether the user is eligible for voting or not. Use `if-else` statement.

```
#include<iostream.h>
#include<constream.h>
#include<string.h>

int main()
{
    clrscr();
    int age;
    cout<<"Enter Your Age:";
    cin>>age;
    if (age>=18)
    {
        cout<<"You are eligible for voting.";
    }
    else
    {
        cout<<"You are not eligible for voting"<<endl;
        cout<< "Wait for"<<18-age<<"year(s).";
    }
    return 0;
}
```

OUTPUT

Enter Your Age: 17
You are not eligible for voting
Wait for 1 year(s).

Explanation: In the above program, the user enters his/her age. The integer variable `age` is used to store the value. The `if` statement checks the value of `age`. If the age is greater than or equal to 18, then the `if` block of the statement is executed. Otherwise, the `else` block of statement is executed.

5.7 Write a program with simple `if-else` statement. If the entered cricket score is less than 50 display one message otherwise another message.

```
#include<iostream.h>
```

```
#include<conio.h>

int main()
{
    int score;
    clrscr();
    cout<<"Enter the run scored of batsman: ";
    cin>>score;
    if(score>=50)
        cout<<"\nCongrats !! You scored a half / more than half
                century " ;
    else
        if(score<50)
            cout<<"\nCome on !! You can too score a half century " ;
    return 0;
}
```

OUTPUT

Enter the run scored of batsman: 45
Come on!! You can too score a half century

Explanation: The program prompts the user to enter the score of the batsman. If the score entered is greater than 50 or equal to 50, then it shows a particular message. Otherwise, for a score less than 50, the other message displayed.

5.5 NESTED if-else STATEMENTS

In this kind of statement, a number of logical conditions are tested for taking decisions. Here, the `if` keyword followed by an expression is evaluated. If it is true, the compiler executes the block following the `if` condition; otherwise, it skips this block and executes the `else` block. It uses the `if` statement nested inside an `if-else` statement, which is nested inside another `if-else` statement. This kind of nesting can be limitless. The syntax of a simple nested `if-else` statement is shown in Figure 5.9.

The flowchart for nesting an `if-else` statement is shown in Figure 5.10.

```
if (expression1)
{
    if (expression2)
        statement1;
    else
        statement2;
}

else
{
    if (expression3)
        statement3;
    else
        statement4;
}

next statement5;
```

Fig. 5.9 The nested if-else

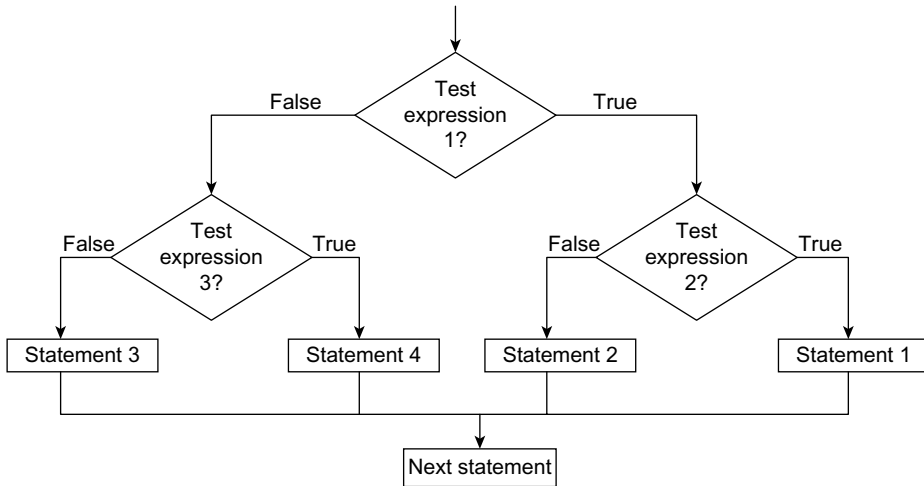


Fig. 5.10 The nested if-else flow-chart

Programs on nested if-else statements are as follows.

5.8 Program to print the largest of three numbers.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    int a,b,c;
    clrscr();
    cout<<"\nEnter three numbers:";
    cin>>a>>b>>c;
    if(a>b)
    {
        if(a>c)
            cout<<"\n a is largest ";
        else
            cout<<"\n c is largest ";
    }
    else
    {
        if(b>c)
            cout<<"\n b is largest ";
        else

```



```
        cout<<"\n c is largest ";
    }
    return 0;
}
```

OUTPUT

Enter three numbers: 3 6 89

c is largest.

Explanation: Three numbers are entered. They are compared as per the theory illustrated above and the largest number is obtained and displayed

5.6 THE else-if LADDER

A common programming construct is the else-if ladder, sometimes called the if-else-if staircase because of its appearance. In the program one can write a ladder of else-if. The program goes down the ladder of else-if, in anticipation of one of the expressions being true.

The general syntax of else-if ladder is as shown in Figure 5.11.

```
/*Syntax of else-if statement can be given as follows.*/
if(condition)
{
    statement 1; /* if block*/
    statement 2;
}
else if(condition)
{
    statement 3; /* else if block*/
    statement 4;
}
else
{
    statement 5; /* last else block */
    statement 6;
}
```

Fig. 5.11 The else-if ladder

The conditions are evaluated from top to bottom. As soon as a true condition is met, the associated statement block gets executed and the rest of the ladder is bypassed. If none of the conditions are met, then the final else block is executed. If this else is not present and none of the if statements evaluate to true, then the entire ladder is bypassed

Programs on else if ladder are explained as follows.

5.9 Write a program to calculate energy bill. Read the starting and ending meter reading. The charges are as follows.

No. of Units Consumed	Rates in (Rs.)
200 - 500	3.5
100 - 200	2.50
Less than 100	1.50

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
int main()
```

```
{
```

```
    int previous,current,consumed;
```

```
    float total;
```

```
    clrscr();
```

```
    cout<<"\nInitial & Final Readings:";
```

```
    cin>>previous>>current;
```

```
    consumed = current-previous;
```

```
    if(consumed>=200 && consumed<=500)
```

```
        total=consumed *3.50;
```

```
    else if(consumed>=100 && consumed<=199)
```

```
        total=consumed *2.50;
```

```
    else if(consumed<100)
```

```
        total=consumed*1.50;
```

```
    cout<<"\n Total no of units consumed: "<<consumed;
```

```
    cout<<"\n Electric Bill for units "<< consumed<< "is" <<total;
```

```
    return 0;
```

```
}
```

OUTPUT

Initial & final readings: 100 235

Total no of units consumed: 135

Electric bill for units 135 is 337.5

Explanation: Previous and current electrical readings are entered through the keyboard. Their difference is the total energy consumed. As per the table, rates are applied and the total bill based on consumption of energy is calculated

5.10 Write a program to calculate gross salary for the conditions given below.

Basic Salary (Rs.)	DA(Rs.)	HRA(Rs.)	Conveyance(Rs.)
bs>=10000	110% of Basic	20% of Basic	500

bs>=5000 && bs<=1000	100% of Basic	15% of Basic	400
bs<5000	90% of Basic	10% of Basic	300

```
#include<iostream.h>
#include<stdio.h>
#include<conio.h>

int main()
{
    float bs,hra,da,cv,ts;
    clrscr();
    cout<<"\n Enter Basic Salary:";
    cin>>bs;
    if(bs>=10000)
    {
        hra=20*bs/100;
        da= 110*bs/100;
        cv=500;
    }
    else
    if(bs>=5000 && bs<10000)
    {
        hra=15*bs/100;
        da=100*bs/100;
        cv=400;
    }
    else
    {
        if(bs<5000)
        hra=10*bs/100;
        da= 90*bs/100;
        cv=300; }
    ts=bs+hra+da+cv;
    cout<<"\nBasic Salary: "<<bs;
    cout<<"\nHRA: "<<hra;
    cout<<"\nDA: "<<da;
    cout<<"\nConveyance: "<<cv;
    cout<<"\nGross Salary: " <<ts;
    return 0;
}
```

OUTPUT

Enter Basic Salary: 7500

Basic Salary: 7500

HRA: 1125
DA: 7500
Conveyance: 400
Gross Salary: 16525.

Explanation: In the above program, the basic salary of an employee is entered through the keyboard. This entered figure is checked with different conditions as cited in the problem. The else-if statements are used. Based on the conditions, gross salary is calculated and displayed.

5.11 Write a program to simulate tariff charges for reaching different destinations by bus.

```
#include<conio.h>
#include<iostream.h>

int main()
{
    int cost,ch;
    clrscr();
    cout<<"\n.....NANDED BUS STATION.....\n";
    cout<<".....Menu.....";
    cout<<"\n1.Bombay";
    cout<<"\n2.Nagpur";
    cout<<"\n3.Pune";
    cout<<"\n4.Amravati";
    cout<<"\n5.Aurangabad";
    cout<<"\n6.Enter your destination:";
    cin>>ch;
    if(ch==1)
        cost=1000;
    else if(ch==2)
        cost=700;
    else if(ch==3)
        cost=500;
    else if(ch==4)
        cost=600;
    else if(ch==3)
        cost=400;
    else
        cost=0;
    if(cost!=0)
    {
        cout<<"\n\nThe ticket cost is: Rs "<<cost;
```

```

        cout<<"\nPay the amount to get book the ticket";
    }
    else
        cout<<"Sorry there's no bus to the desired destination";
    cin.get();
    return 0;
}

```

OUTPUT

.....NANDED BUS STATION.....

.....Menu.....

1.Bombay

2.Nagpur

3.Pune

4.Amravati

5.Aurangabad

Enter your destination: 4

The ticket cost is: Rs 600

Pay the amount to book the ticket.

Explanation: As can be seen, the program simulates a bus station. We use an if-else-if ladder to find out the cost of a ticket to a particular station, if there was a bus to that station. There was no bus to that station that had cost=0. Finally, if the cost is non-zero, it is printed. The user is prompted to enter the choice. Depending upon the choice, the fare of destination station is displayed. In the above example, the choice 4 is given. The result displayed is

“The ticket cost is : Rs 600

Pay the amount to book the ticket.”

5.7 UNCONDITIONAL CONTROL TRANSFER STATEMENTS

C/C++ has four statements that perform an unconditional control transfer. They are return, goto, break and continue. Of these, return is used only in functions. The goto and return may be used anywhere in the program but continue and break statements may be used only in conjunction with a loop statement. The break is used most frequently in switch case.

5.7.1 The goto statement

This statement does not require any condition. This statement passes control anywhere in the program without considering any condition. The general format for this statement is shown in Figure 5.12.

Here, a label is any valid label either before or after goto. The ‘label’ must start with any character and can be constructed with rules used for forming identifiers. Avoid using the goto statement.

```

goto label;
-----
-----
-----
label:

```

Fig. 5.12 The goto statement

5.12 Write a program to demonstrate the use of goto statement.

```
#include<iostream.h>
#include<constream.h>

void main()
{
    int x;
    clrscr();
    cout<<"Enter a Number:";
    cin>>x;
    if (x%2==0)
        goto even;
    else
        goto odd;

    even: cout<<x<<" is an Even Number.";
    return;
    odd: cout<<x<<" is an Odd Number.";
}
```

OUTPUT

Enter a Number: 5
5 is an Odd Number.

Explanation: In the above program a number is entered. The number is checked to see if it is even or odd with modulus division operator. When the number is even, the goto statement transfers the control to the label even. Similarly, when the number is odd, the goto statement transfers the control to the label odd and the respective message will be displayed.

5.7.2 The break Statement

The break statement allows the programmer to terminate the loop. The break skips from the loop or the block in which it is defined. The control then automatically passes on to the first statement after the loop or the block. The break can be associated with all the conditional statements (especially switch case). We can also use break statements in the nested loops. If we use break statement in the innermost loop, then the control of the program is terminated from that loop only and resumes at the next statement following that loop. The widest use of this statement is in the switch case where it is used to avoid flow of control from one 'case' to the other. Programming examples on break are given in switch topic.

5.7.3 The continue Statement

The continue statement works somewhat like the break statement. Instead of forcing the control to the end of the loop (as it is in case of break), the continue case causes the control

to pass on to the beginning of the block/loop. In the case of `for` loop, the `continue` case initiates the testing condition and increment on steps has to be executed (while rest of the statement following the `continue` are neglected). For `while` and `do-while`, the `continue` case causes control to pass on to conditional tests. It is useful in a programming situation where it is required that particular iterations occur only up to some extent or when some part of the code has to be neglected. The programs on `continue` are explained in the control loop chapter.

5.8 THE `switch` STATEMENT

The `switch` statement is a multi-way branch statement and an alternative to `if-else-if` ladder in many situations. The expression of `switch` contains only one argument, which is then checked with a number of `switch` cases. The `switch` statement evaluates the expression and then looks for its value among the case constants. If the value is matched with a particular case constant, then those case statements are executed until a `break` statement is found or until the end of `switch` block is reached. If not, then simply the default (if present) is executed (if a default is not present, then the control flows out of the `switch` block). The default is normally present at the bottom of the `switch` case structure. But we can also define default statement anywhere in the `switch` structure. The default block must not be empty. Every case statement terminates with a `:` (colon). The `break` statement is used to stop the execution of succeeding cases and pass the control to the end of the `switch` block.

```
switch(variable or expression)
{
    case constant A:
        statement;
        break;

    case constant B:
        statement;
        break;

    default:
        statement;
}
```

Fig. 5.13 The switch case.

The syntax of the `switch` case statement is shown in Figure 5.13.

Note the following for `switch` case.

- (a) **The `switch` expression:** In the block the variable or expression can be a character or an integer. The integer expression following the keyword `switch` will yield an integer value only. The integer may be any value 1, 2, 3, etc. In case of character constant, the values may be with alphabets such as 'x', 'y', 'z', etc.
- (b) **The `switch` organisation:** The `switch` expression should neither be terminated with a semicolon (;) nor with any other symbol. The entire case structure following the `switch` should be enclosed within curly braces. The keyword `case` is followed by a constant. Every constant terminates with a colon (:). Each case statement must contain different constant values. Any number of case statements can be provided. If the case structure contains multiple statements, they need not be enclosed within curly braces. Here, the keyword `case` & `break` performs, respectively, the job of opening and closing curly braces.
- (c) **The `switch` execution:** When one of the cases is satisfied, the statements following it are executed. In case there is no match, the default case is executed.

(d) The `break` statement used in `switch` passes control outside the `switch` block. By mistake if no `break` statements are given, all the cases following it are executed. The flowchart for the `switch case` is as shown in Figure 5.14.

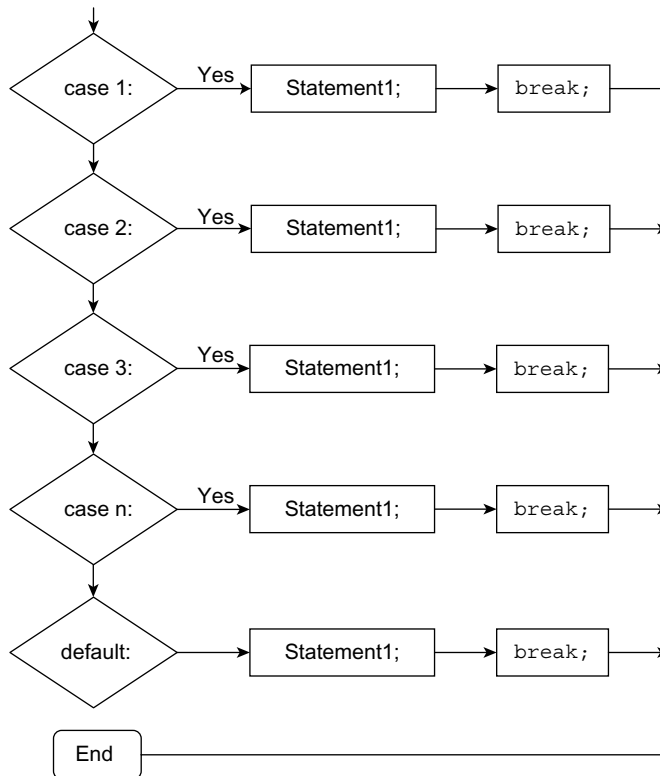


Fig. 5.14 Flowchart for `switch case`.

Programs on `switch case` are as follows.

5.13 Write a program to print lines by selecting the choice.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    int ch;
    clrscr();
    cout<<"\n[1] .....";
    cout<<"\n[2] _____";
    cout<<"\n[3] *****";
}

```

```

cout<<"\n[4] =====";
cout<<"\n[5] EXIT";
cout<<"\n\n ENTER YOUR CHOICE:";
cin>>ch;
switch(ch)
{
    case 1:
        cout<<"\n .....";
        break;
    case 2:
        cout<<"\n _____";
        break;
    case 3:
        cout<<"\n *****";
        break;
    case 4:
        cout<<"\n =====";
        break;
    case 5:
        cout<<"\n Terminated by choice";
        break;
    default:
        cout<<"\n Invalid Choice";
}
return 0;
}

```

OUTPUT

```

[1] .....
[2] _____
[3] *****
[4] =====
[5] EXIT

```

ENTER YOUR CHOICE: 4

```
=====
```

Explanation: In the above program a menu appears with five options and it requests the user to enter his/her choice. The choice entered by the user is then passed to switch statement. In the switch statement the value is checked with all the case constants. The matched case statement is executed in which the line is printed according to the user's choice. If the user enters a non-listed value, then no match occurs and the default is executed. The default warns the user with a message "Invalid Choice."

5.14 Write a program to perform the multiple arithmetic operations as 1. Addition, 2. Subtraction, 3. Multiplication, and 4. Division using switch case.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int a,b,c,ch;
    clrscr();
    cout<<"\t      =====";
    cout<<"\n\t MENU";
    cout<<"\n\t =====";
    cout<<"\n\t [1]  ADDITION";
    cout<<"\n\t [2]  SUBTRACTION";
    cout<<"\n\t [3]  MULTIPLICATION";
    cout<<"\n\t [4]  DIVISION";
    cout<<"\n\t =====";
    cout<<"\n\n\t ENTER YOUR CHOICE:";
    cin>>ch;
    if (ch<=4 & ch>0)
    {
        cout<<"Enter Two Numbers:";
        cin>>a>>b;
    }
    switch (ch)
    {
        case 1 :
            c=a+b;
            cout<<"\n Addition:"<<c;
            break;
        case 2 :
            c=a-b;
            cout<<"\n Subtraction:"<<c;
            break;
        case 3 :
            c=a*b;
            cout<<"\n Multiplication:"<<c;
            break;
        case 4 :
            c=a/b;
            cout<<"\n Division:"<<c;
            c=a%b;
            cout<<"\n Remainder:"<<c;
            break;
```

```
        default:
            cout<<"\nInvalid choice:";
            break;
    }
    return 0;
}
```

OUTPUT

```
=====
          MENU
=====
[1] ADDITION
[2] SUBTRACTION
[3] MULTIPLICATION
[4] DIVISION
=====

Enter your choice: 4
Enter two numbers: 20 10

Division :2
Remainder: 0
```

Explanation: The above program is used to perform various arithmetic operations. It requests the user to enter the choice. The choice entered by the user is checked with the `if` statement. If it is between 1 and 4, the `if` block is executed, which prompts the user to enter two numbers. After this the choice entered by the user is passed to the `switch` statement and it performs the relevant operation.

5.15 Write a program to enter a month number of year 2011 and display number of days present in the month.

```
#include<iostream.h>
#include<conio.h>

void main()
{
    clrscr();
    int month, day;
    cout<<"Enter a month of year 2011:";
    cin>>month;
    switch(month)
    {
        case 1:
        case 3:
```



```

        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
        day=31;
        break;
        case 2:
        day=28;
        break;
        case 4:
        case 6:
        case 9:
        case 11:
        day=30;
        break;
    }
    cout<<"\nNumber of days in this month are "<<day;
}

```

OUTPUT

Enter a month of year 2011: 2
 Number of days in this month are 28.

5.9 NESTED switch case

C/C++ supports the nesting of switch case. The inner switch can be part of an outer switch. The inner and the outer switch case constants may be the same. No conflict arises even if they are same. The example below demonstrates this concept.

5.16 Write a program to demonstrate the nested switch case statement.

```

#include<iostream.h>
#include<constream.h>
int main()
{
    int x;
    clrscr();
    cout<<"\nEnter a number:";
    cin>>x;
    switch(x)
    {
        case 0:
            cout<<"\nThe number is 0";
            break;

```



```
default:
int y;
y=x%2;
switch(y)
{
    case 0:
        cout<<"\nThe number is even";
        break;
    case 1:
        cout<<"\nThe number is odd";
        break;
}
return 0;}
```

OUTPUT

Enter a number: 5
The number is odd

Explanation: The above program aims at identifying whether the input number is zero or an even or an odd number. The first switch case finds out whether the number is zero or non-zero. If a non-zero value is entered, a default statement of first switch case statement is executed, which executes another nested switch case. The nested switch case statement determines whether the number is even or odd and displays the respective messages.

Summary

- (1) Decision-making statements in a programming language help the programmer to transfer the control from one part of the program to another part.
- (2) C++ language supports the following decision-making statements: if statement, if-else statement, nested if-else statements, else-if ladder, switch case statement, break statement, and the default keyword.
- (3) C++ uses the keyword if to execute a set of command lines or a command line when the logical condition is true.
- (4) Multiple ifs are used in C++ to tackle alternate decisions.
- (5) In the if-else statement, if the expression/condition is true, the body of the if statement is executed; otherwise, the body of the else statement is executed. The else keyword is used when the expression is not true.
- (6) In nested if-else statements, a number of logical conditions are tested for taking decisions. Here, the if keyword followed by an expression is evaluated. If it is true, the compiler executes the block following the if condition; otherwise, it skips this block and executes the else block. It uses the if statement nested inside an if-else statement, which is nested in-

- side another `if-else` statement. This kind of nesting can be limitless.
- (7) A common programming construct is the `else-if` ladder, sometimes called the `if-else-if` staircase because of its appearance. In the program one can write a ladder of `else-if`. The program goes down the ladder of `else-if`, in anticipation of one of the expressions being true.
 - (8) C/C++ has four statements that perform an unconditional control transfer. They are `return`, `goto`, `break` and `continue`. Of these, `return` is used only in functions. The `goto` and `return` may be used anywhere in the program but `continue` and `break` statements may be used only in conjunction with a loop statement. The `break` is used most frequently in `switch case`.
 - (9) The `goto` statement does not require any condition. This statement passes control anywhere in the program without considering any condition.
 - (10) The `break` statement allows the programmer to terminate the loop. It is widely used in the `switch case` where it is used to avoid the flow of control from one “case”
 - (11) The `continue` statement works somewhat like the `break` statement. Instead of forcing the control to the end of the loop (as it is in case of `break`), the `continue` case causes the control to pass on to the beginning of the block/loop. It is useful in a programming situation where it is required that particular iterations occur only up to some extent or when some part of the code has to be neglected.
 - (12) The `switch` statement is a multi-way branch statement and an alternative to `if-else-if` ladder in many situations.
 - (13) C/C++ supports the nesting of `switch case`. The inner `switch` can be part of an outer `switch`. The inner and the outer `switch case` constants may be the same. No conflict arises even if they are same.

EXERCISES

(A) Answer the following questions

- (1) Give the syntax and flowchart of the `if` decision making statement.
- (2) Why is it important to use multiple `ifs` in a program?
- (3) Is it possible to use multiple `else` with an `if` statement?
- (4) Explain the use of `if-else` statement.
- (5) Explain the use of `switch case` statement.
- (6) Explain the use of keyword `default`.
- (7) Is it possible to use multiple `default` statements in a `switch` statement?
- (8) Write the use of `else` and `default` statements in `if-else` and `switch` statements, respectively.
- (9) Is it possible to use `else` statement in place of `default` or vice versa?
- (10) Can we put `default` statement anywhere in the `switch case` structure?
- (11) What are the limitations of `switch case` statement?
- (12) What is sequential execution?
- (13) What is transfer of control? Explain the control transfer keywords used in C++.
- (14) Explain nested `ifs`.
- (15) Explain nested `switch case` statement.

(B) Answer the following by selecting the appropriate option

- (1) The switch statement is used to
 - (a) **switch between functions in a program.**
 - (b) switch from one variable to another variable.
 - (c) choose from multiple possibilities which may arise due to different values of a single variable.
 - (d) use switching variables.
- (2) The default statement is executed when
 - (a) **all the case statements are false**
 - (b) one of the case is true
 - (c) one of the case is false
 - (d) none of the above
- (3) Each case statement in switch is separated by
 - (a) **break**
 - (b) continue
 - (c) exit()
 - (d) goto
- (4) The keyword else can be used with
 - (a) **if statement**
 - (b) switch statement
 - (c) do..while() statement
 - (d) none of the above
- (5) Conditional if statement always returns
 - (a) **0 or 1**
 - (b) 1 or 2
 - (c) -1 or 0
 - (d) none of the above
- (6) The meaning of if (1) is
 - (a) always false
 - (b) **always true**
 - (c) both (a) and (b)
 - (d) none of the above
- (7) In case curly braces are absent, following the if statements, then what is the scope of if block?
 - (a) **one statement**
 - (b) two statements
 - (c) four statements
 - (d) none of the above
- (8) In nested loop
 - (a) **the inner most loop is completed first**
 - (b) the outer most loop is completed first
 - (c) both (a) and (b)

- (d) none of the above
- (9) What will be the output of the following program?


```
#include<iostream.h>
#include<conio.h>

int main()
{
    char x='H';
    clrscr();
    switch(x)
    {
        case 'H': cout<<"H";
        case 'E': cout<<"E";
        case 'L': cout<<"L";
        case 'L': cout<<"L";
        case 'O': cout<<"O";
    }
    return 0;
}
```

 - (a) HELLO
 - (b) HELlo
 - (c) H
 - (d) **none of the above**
- (10) What will be the output of the following program?


```
#include<iostream.h>
#include<conio.h>

int main()
{
    char x='G';
    clrscr();
    switch(x)
    {
        if( x=='B')
        {
            case 'd': cout<<"o";
            case 'B': cout<<"Bad";
        }
        else
            case 'G': cout<<"Good";
            break;
        default : cout<<" Boy";
    }
    return 0;
}
```

 - (a) Good Boy

(b) **Good**

(c) Boy

(d) none of the above

- (11) What will be the output of the following program?

```
#include<iostream.h>
#include<conio.h>

int main()
{
    char x='d';
    clrscr();
    switch(x)
    {
        case 'b':
```

```
cout<<"0 1 001";
break;
default :
cout<<"1 2 3";
break;
case 'R':
cout<<"I II III";
}
return 0;
}
```

(a) **1 2 3**

(b) 0 1 001

(c) I II III

(d) none of the above

(C) Attempt the following programs

- (1) Write a program to check whether the blood donor is eligible for donating blood. The conditions laid down are as under. Use *if* statement.
 - (a) Age should be greater than 18 years but not more than 55 years.
 - (b) Weight should be more than 45 kg.
- (2) Write a program to check whether the voter is eligible for voting. If his/her age is equal to or greater than 18, display the message "*Eligible*" otherwise "*Non-eligible*". Use *if* statement.
- (3) Write a program to calculate the bill of a job work done as follows. Use *if else* statement.
 - (a) Rate of typing 3 Rs./ page.
 - (b) Printing of first copy Rs. 5/ page and the rate of every copy taken after that as Rs. 3/ page.

The user should enter the number of pages and printout copies he/she wants.
- (4) Write a program to calculate the amount of the bill for the following jobs.
 - (a) Scanning and hardcopy of a passport photo to Rs. 5.
 - (b) Scanning and hardcopies (more than 10) Rs. 3.
- (5) Write a program to calculate bill of Internet browsing. The conditions are given below.
 - (a) 1 Hour Rs. 20.
 - (b) ½ Hour Rs. 10.
 - (c) Hours Rs. 90.

The owner should enter the number of hours spent by the customer.

- (6) Write a program to enter a character through keyboard. Use *switch case* structure and print appropriate message. Recognize whether the entered character is a vowel, a consonant, or a symbol?
- (7) The table given below is a list of gases, liquids and solids. By entering substances one by one through the keyboard, recognize their state (gas, liquid and solid).

WATER	OZONE
OXYGEN	PETROL
IRON	ICE
GOLD	MERCURY

- (8) Write a program to display the days of week using *switch case*.
- (9) Write a program to calculate telephone bill based on the following .

(a) calls 0 to 100	Rs. 1/call.
(b) calls 101 >=500	Rs. 0.75/call.
(c) more than 500 calls	Rs. 0.50/call.
- (10) Write a program to find the incentive to be offered to a salesman based on the sales of number of items/turnover. Use *else if* ladder.
- (11) Write a program to display grades obtained by a student in the MCA CET examination. Assume suitable data.

Control Loop Structures

CHAPTER OUTLINE

- 6.1 Introduction
- 6.2 What is a Loop?
- 6.3 The `for` Loop
- 6.4 The Nested `for` Loops
- 6.5 The `while` Loop
- 6.6 The `do-while` Loop
- 6.7 The `do-while` Statement with `while` Loop
- 6.8 More Programs

6.1 INTRODUCTION

In our day to day life in numerous applications we need to perform tasks that are repetitive in nature. It is a tedious process to perform such a kind of task repeatedly with pen/pencil and paper. But with computer programming languages and packages, the task becomes easy, accurate and fast. For example, the salary of laborers of a factory can be calculated by a formula (number of hours worked $\times$ wage rate per hour). The accounts department of each organization is involved in performing this type of calculation every month/week. Such a type of repetitive deeds can be easily done using a loop or iterative structure in a program.

6.2 WHAT IS A LOOP?

A loop is defined as a block of statements that are repeatedly executed for certain number of times even though those block statements appear only once in the program.

Loops are of two types and they are as follows:

- (1) Counter-controlled repetition
- (2) Sentinel-controlled repetition

- (1) **Counter-controlled repetition:** This is also called definite repetitive actions, because in advance the number of iterations to be performed is defined in the program itself. The steps for performing counter-controlled repetitions are given below.

Steps in loop

Loop Variable: It is a variable used in the loop.

Initialization: It is the first step in which starting and final value is assigned to the loop variable. Each time the updated value is checked by the loop itself.

Incrimination/Decrimination: It is the numerical value added or subtracted to the variable in each round of the loop. The updated value is compared with the final value and if it is found to be less than final value the steps in the loop are executed.

The above steps are implemented in numerous programs in this chapter.

- (2) **Sentinel-controlled repetition:** This is also called indefinite repetitive actions. One cannot estimate the number of iterations to be performed. In this type loop, termination happens based on certain conditions using decision-making statement.

The 'C++' language supports three types of loop control statements. Their syntaxes are described as given in Fig. 6.1.

for	while	do-while
<pre>for (expression 1; expression 2; expression 3) statement ;</pre>	<pre>expression 1; while (expression 2) { Statement; expression 3; }</pre>	<pre>expression 1; do { statement; expression 2; } while (expression 3);</pre>

Fig. 6.1 Syntaxes of various loops.

The `for` loop comprises three actions. The three actions are placed in the `for` statement itself. The three actions *initialize counter*, *test condition* and *re-evaluation parameters* are included in one statement. The expressions are separated by semi-colons (;). This leads to the programmer to visualize the parameters easily. The `for` statement is equivalent to the `while` and `do-while` statements. The only difference between `for` and `while` is later checks the logical condition and then executes the body of the loop, whereas in the `for` statement test is always performed at the beginning of the loop. The body of the loop may not be executed at all times if the condition fails at the beginning. The `do-while` loop executes the body of the loop at least once regardless of the logical condition.

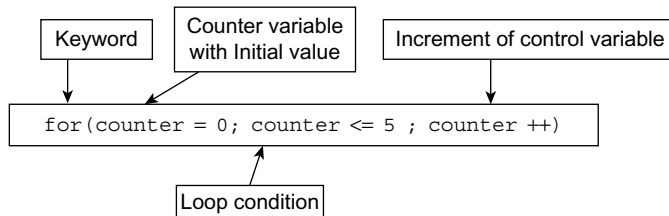
6.3 THE `for` LOOP

The `for` loop allows one to execute a set of the instructions until a condition is met. The condition may be predefined or open-ended. Although all programming languages provide `for` loops still the power and flexibility provided by C/C++ is worth mentioning. The general syntax for the `for` loop is as follows Fig. 6.2.

```
Syntax of for loop
for (initialization; test-condition; increment/
decrement)
{
    Statement;
}
```

Fig. 6.2 for loop

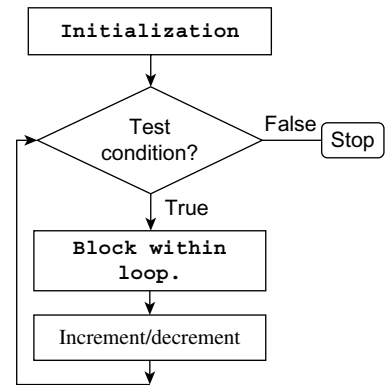
An example of the for loop is as shown in Fig 6.3.

**Fig. 6.3** The for loop

Although many variations of for loop are allowed but the simplest form is as shown in Fig. 6.3.

The ‘initialization’ is an assignment statement that is used to set the loop control variable(s). The ‘condition’ is a relational expression that determines the number of the iterations desired or the condition to exit the loop. The ‘increment’ or the re-evaluation parameter decides how to change the state of the variable(s) (quite often increase or decrease so as to approach the limit). These three sections must be separated by semi-colon (;). The body of the loop may consist of a block of statements (which have to be enclosed in braces) or a single statement (enclosure within braces is not compulsory but advised).

Flow chart of for loop is as shown in Fig. 6.4.
The following programs serve as examples for the for loop.

**Fig. 6.4** Flow chart of for loop**6.1**

```
#include<iostream.h>
#include<conio.h>

int main()
{
```

6.2

```
#include<iostream.h>
#include<conio.h>

int main()
{
```

6.3

```
#include <iostream.h>
#include<conio.h>

int main()
{
```


<pre>clrscr(); int j; for (j=1; j<11;j++) cout<<" "<<j; return 0; }</pre>	<pre>clrscr(); int j=0; for (;j<11;j++) cout<<" "<<j; return 0; }</pre>	<pre>clrscr(); int j=0; for (;j<11;) { cout<<" "<<j; ++j; } return 0; }</pre>
---	---	---

OUTPUT**1 2 3 4 5 6 7 8 9 10****OUTPUT****0 1 2 3 4 5 6 7 8 9 10****OUTPUT****0 1 2 3 4 5 6 7 8 9 10**

Explanation: In this program initialization, condition and increment is done in single parenthesis.

Explanation: In this program initialization is done before for statement. In for statement only test condition and increment operations are done.

Explanation: In this program initialization is done before for statement. Increment is done inside the loop. The for loop contains only test condition.

6.4 Write a program to display all leap years from 2000 to 2012.

```
#include<conio.h>
#include<iostream.h>
int main()
{
int i=1999;
clrscr();
cout<<"\nProgram to print all the leap years from 2000 to 2012 \n\n";
for(;i++<=2012;)
{
if(i%4==0&& i%100!=0)
cout<<i<<" ";
else if(i%100==0&& i%400==0)
cout<<i<<" ";
}
return 0;
}
```

{coderipe}**OUTPUT**

Program to print all the leap years from 2000 to 2012
2000 2004 2008 2012

Explanation: One must be aware that only those years that are

- (a) divisible by 4 and not 100,
- (b) divisible by 100 and also 400

alone are qualified to be called as leap years. The above program checks for these conditions for all years from 2000 to 2012 and then prints the year if it is leap. Stress is on 'for' loop construct and use of 'if' and 'else - if' in the body.

6.4 NESTED for LOOPS

We can also nest for loop to gain more advantage in some situations. The nesting level is not restricted at all. In body of a 'for' loop any number of sub - 'for' loop(s) may exist.

6.5 Write a program to demonstrate nested for loops.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int a,b,c;
    clrscr();
    for (a=1;a<=2;a++) // outer loop
    {
        for (b=1;b<=2;b++) // middle loop
        {
            for (c=1;c<=2;c++) //inner loop
            cout<<"\n a="<<a <<" + b="<<b <<" + c="<<c <<":" <<a+b+c;
            cout<<"\n Inner  Loop Over.";
        }
        cout<<"\n Middle Loop Over.";
    }
    cout<<"\n Outer Loop Over.";
    return 0;
}
```

Explanation: The above program will be executed in sequence as per Table 6.1. The total numbers of iterations are equal to $2*2*2 = 8$. The final output provides 8 results.

Table 6.1 Working and output of program 6.5

Values of loop variables		
Outer	Middle	Inner
(1) a=1	b=1	c=1
(2) a=1	b=1	c=2

(Table 6.1 Contd.)

(Table 6.1 Contd.)

Inner Loop Over		
(3) a=1	b=2	c=1
(4) a=1	b=2	c=2
Inner Loop Over		
Middle Loop Over		
(5) a=2	b=1	c=1
(6) a=2	b=1	c=2
Inner Loop Over		
(7) a=2	b=2	c=1
(8) a=2	b=2	c=2
Inner Loop Over		
Middle Loop Over		
Outer Loop Over		

6.5 THE while LOOP

Another kind of loop structure in C/C++ is the while loop. Its format is as given in Fig. 6.5.

```
Syntax:
while (test condition)
{
    body of the loop
}
```

The test condition may be any expression. The loop statements will be executed till the condition is true, i.e. the test condition is evaluated and if the condition is true, then the body of the loop is executed. When the condition becomes false the execution will be out of the loop.

The execution of the while loop can be followed by Fig. 6.6.

Fig. 6.5 The while loop

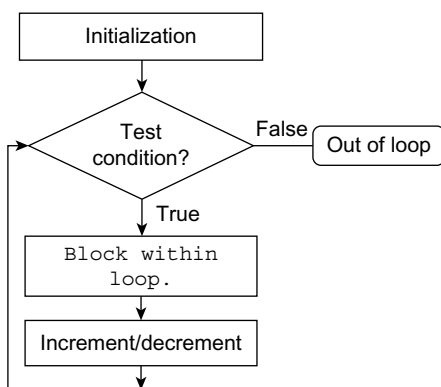


Fig. 6.6 Flow chart for while-loop

Here, the block of the loop may either contain single statement or a number of statements. The same block can be repeated. The braces are needed only if the body of the loop contains more than one statement. However, it is a good practice to use braces even if the body of the loop contains only one statement.

Steps of while loop are as follows.

- (1) The test condition is evaluated, and if it is true, the body of the loop is executed.
- (2) On execution of the body, and updating the variable test condition is repeatedly checked. If it is true the body is executed.
- (3) In case the test condition fails the control transfer out of the loop or stop.

Given below are a few example problems on while loop.

6.6 Write a program to add 10 consecutive numbers starting from 1. Use the while loop.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int a=1,sum=0;
    clrscr();
    while (a<=10)
    {
        cout<<" "<<a;
        sum=sum+a;
        a++;
    }
    cout<<"\n Sum of 10 numbers : "<<sum;
    return 0;
}
```



OUTPUT

1 2 3 4 5 6 7 8 9 10

Sum of 10 numbers : 55

Explanation: In the above program integer variable 'a' is initialized to 1 and variable 'sum' to 0. The while loop checks the condition for a<=10. The variable 'a' is added to variable 'sum' and each time 'a' is incremented by 1. In each while loop 'a' is incremented and added to 'sum'. When the value of 'a' reaches 11, the condition given in while loop becomes false. At last the loop is terminated. The sum of the number is displayed.

6.7 Write a program to calculate the sum of the individual digits of an entered number.

```
#include<iostream.h>
#include<conio.h>
int main()
{
    int num,t;
    clrscr();
    cout<<"\n Enter number : ";
    cin>>num;
    int sum=0;
    while(num!=0) //condition is true until num!=0
    {
        sum=sum+num%10;
```

```

    num=num/10;
}
cout<<"\n Sum of the individual digits of the number"
    <<t<<" is = "<<sum;
    return 0;
}

```

OUTPUT

Enter number : 234

Sum of the individual digits of the number 234 is = 9

Explanation: The logic behind the program is to extract each time the LSD(lower significant digit). Each time divide the number by 10 so as to shift it by one place). To extract the LSD we use the fact that $\text{num \% 10}$ = the remainder on dividing num by 10.

6.8 Write a program to check whether the entered number is a palindrome or not.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    int num;
    clrscr();
    cout<<"\n Enter the number :";
    cin>>num;
    int b=0,a=num;
    while(a)
    {
        b=b*10+a%10;
        a=a/10;
    }
    if(num==b)
    cout<<"\n The given number is palindrome";
    else
    cout<<"\n The given number is not palindrome";
    return 0;
}

```



OUTPUT

Enter the number: 121

The given number is palindrome

Explanation: Palindrome number is a number that is equal to its reverse (e.g. 51715). The above program first reverses the number and stores it in 'b'. Then it compares 'b' with original number to tell you whether or not the number is a palindrome. See that in each iteration the LSD of 'a' is being made MSD of 'b'. The 'a' is being shifted left and 'b' is shifted right by one digits in every iteration. Iterations are being done until 'a' exhausts or becomes equal to 0.

6.6 THE do-while LOOP

The format of do-while loop in C/C++ is as follows (Fig 6.7)

The difference between the while and do-while loop is the place/position where the condition is to be tested. In the while loops the condition is tested following the while statement and then the body gets executed. Whereas in the do-while the condition is checked at the end of the loop. The do-while loop will execute at least one time even if the condition is false initially. The do-while loop executes until the condition becomes true. Table 6.2 shows the comparison between the while and do-while loops.

```
do
{
statement/s;
}
while (condition);
```

Fig. 6.7 do-while loop

Table 6.2 Comparison of while and do-while loop.

Sr No	while-loop	do-while
1	Condition is specified at the top	Condition is mentioned at the bottom
2	Body statement/s is/are executed when condition is satisfied	Body statement/s execute even when condition is false
3	No brackets for a single statement	Brackets are essential even when a single statement exit
4	It is entry controlled loop	It is exit controlled loop

Programs on do-while loop are as follows.

6.9 Write a program using do-while loop to print numbers and their cubes up to 10.

```
#include<iostream.h>
#include<conio.h>
#include<math.h>

int main()
{
    int y,x=1;
    clrscr();
    cout<<"\n\tNumbers and their cubes \n";
    do
    {
        y=pow(x,3);
        cout<<"\t"<<x<<"\t\t"<<y<<"\n";
        x++;
    }
}
```

```

    while (x<=10);
}

```

OUTPUT

Numbers and their cubes

1	1
2	8
3	27
4	64
5	125
6	216
7	343
8	512
9	729
10	1000

Explanation: Here, the mathematical function $\text{pow}(x,3)$ is used from the library. It's meaning is to calculate the third power of x . With this function we get the value of $y=x^3$. For use of the $\text{pow}()$ function we have to include `math.h` header file.

6.7 THE do-while STATEMENT WITH while LOOP

The format of do-while with multiple while statements loop is as shown in Fig. 6.8.

```

do while(condition)
{
    statement/s;
}
while (condition);

```

Fig. 6.8 Syntax of do-while with multiple while statement

6.10 Write a program to add numbers from 0 to 5 using while statement in do-while loop.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    int sum=0,x=0;
    clrscr();
    do while (x<5)
    {
        x++;
    }
}

```

```
        sum=sum+x;
    }
    while(x>5);
    cout<<"\nSum of numbers from 0 to 5:"<<sum;
    return 0;
}
```

OUTPUT

Sum of numbers from 0 to 5: 15

Explanation: Initially the value of x and sum are 0. Value of x is incremented in and added with the sum in each iteration. With do-while loop the value of x is incremented and when it is greater than 5 the outer while loop is executed.

6.8 MORE PROGRAMS

6.11 Write a program to use break statement and terminate the loop.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int x=1;
    clrscr();
    while (1)
    {
        cout<<" "<<x;
        if (x==8)
            break;
        x++;
    }
    return 0;
}
```

OUTPUT

1 2 3 4 5 6 7 8

Explanation: In the above program the while loop is an infinite loop. The integer variable x is initialization to 1. In each iteration the value of x is displayed and incremented. The if condition checks the value of x with 8. If the value of x is equal to 8 the break statement is executed and the infinite while loop terminates. Thus, the break statement is used to exit from the loop as per shown in Fig. 6.9

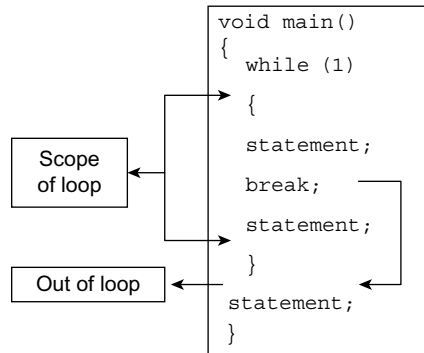


Fig. 6.9 Working of break statement

6.12 Write a program to demonstrate the use of continue statement.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    int x=1;
    clrscr();
    for(;x<=50;x++)
    {
        if (x%2==0)
            continue;
        cout<<" "<<x;
    }
    return 0;
}

```

OUTPUT

```

1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41
43 45 47 49

```

Explanation: In the above program the `for` loop executes from 1 to 50. The `if` condition check whether the remainder of number is 0 or non-zero. If the remainder is zero, the `continue` statement is executed and the statement after `continue` statement will not be executed. The `continue` statement transfers the control at the beginning of the loop. Fig. 6.10 simulates the working of `continue` statement.

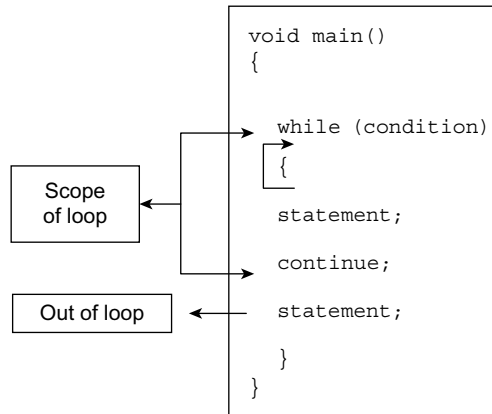


Fig. 6.10 Working of continue statement

SUMMARY

- (1) This chapter discusses the basics of control structure of C++ language.
- (2) The C++ control structure covers certain operations that are performed repetitively called as loop statements.
- (3) C/C++ has four statements that perform an unconditional control transfer. These are return, goto, and break and continue. Of these, return is used only in functions. goto and return may be used anywhere in the program but continue and break statements may be used only in conjunction with a loop statement.
- (4) The for loop comprises three actions. The three actions are placed in the for state-

ment itself. The three actions *initialize counter*, *test condition* and *re-evaluation parameters* are included in one statement. The for statement is equivalent to the while and do-while statements. The only difference between for and while is that the latter checks the logical condition and then executes the body of the loop, whereas the for statement test is always performed at the beginning of the loop. The body of the loop may not be executed at all times if the condition fails at the beginning. The do-while loop executes the body of the loop at least once regardless of the logical condition.

EXERCISES

(A) Answer the following questions

- (1) Explain the need of control structure in C++.
- (2) What are the differences between break and continue?

- (3) What are the differences between `while` and `do-while` loop statements?
- (4) What is an infinite loop?
- (5) What happens if you create a loop that never ends?
- (6) Is it possible to create a `for` loop that is never executed?
- (7) What is a loop? Why it is necessary in the program?
- (8) Is it possible to nest `while` loop within `for` loops?
- (9) What is the difference between `(!0)` and `(!1)`. How does the `while` loop work with these values?
- (10) Is it possible to use multiple `while` statements with the `do` statement?

(B) Answer the following by selecting the appropriate option

- (1) The following loop statement uses two keywords
 - (a) **do-while loop**
 - (b) `for` loop
 - (c) `while` loop
 - (d) none of the above
- (2) The correct syntax of '`for`' loop is
 - (a) `for (i=0; i<n; i++) {----}`
 - (b) `for (i=0, j=0; i<n && j<n; i++, j++) {-----}`
 - (c) `for (i=0; i++<n;) {-----}`
 - (d) **All of these**
- (3) This loop statement is terminated by semi-colon
 - (a) **do-while loop**
 - (b) `for` loop
 - (c) `while` loop
 - (d) none of the above
- (4) Every expression always returns
 - (a) **0 or 1**
 - (b) 1 or 2
 - (c) -1 or 0
 - (d) none of the above
- (5) In a nested loop
 - (a) **the innermost loop is completed first**
 - (b) the outermost loop is completed first
 - (c) both (a) and (b)
 - (d) none of the above

(C) Attempt the following programs

- (1) Write a program to display numbers from 10 to 1 using `for` loop.
- (2) Write a program to calculate the factorial of a given number.
- (3) Write a program to display only even numbers between 1 and 150.
- (4) Write a program to solve the series $x = 1/2! + 1/4! + 1/n!$.
- (5) Write a program to calculate the sum of numbers between 1 to N numbers. The user enters the value of N.
- (6) Write a program to use `break` and `continue` statement.
- (7) Write a program to display alphabets A to Z using `while` loop.
- (8) Write a program to display alphabets as given below.
`az by cx dw ev fu gt hs Ir jq kp lo mn nm ol
pk qj ri sh tg uf ve wd xc yb za.`
- (9) Write a program to display count values from 0 to 100 and flash each digit for one second. Reset the counter after it reaches to hundred. The procedure is to be repeated. Use `for` loop.
- (10) Develop a program to simulate a seconds clock. Put 60 dots on the circle with equal distance between each other and mark them 0 to 59. A seconds pointer is to be shown with any symbol. Also print the total number of revolutions made by the second's pointer.
- (11) Write a program to simulate an analog watch (numbers 1 to 12 to be arranged in circular fashion with all the three pointers for seconds, minutes, and hours) on the screen. Use nested `for` loops.
 Use `(.)` dot for seconds pointer.
 Use `(*)` star for minutes pointer.
 Use `(#)` hash for hours pointer.
- (12) Write a program to calculate the sum of first and last number from 1 to 10 (Example $1 + 10, 2 + 9, 3 + 8$; sums should be always 11).

- (13) Write a program to find the total number of votes in favor of persons 'A' and 'B'. Assume 100 voters will be casting their votes to these persons. Count the number of votes gained by 'A' and 'B'. The user can enter his/her choices by pressing only 'A' or 'B'.
- (14) Write a program to pass the resolution in a meeting comprising of five members. If three or more votes are obtained the resolution is passed otherwise rejected.
- (15) Assume that there are 99 voters voting to a person for selecting chairman's candidature. If he secures more than 2/3 votes he should be declared as chairman otherwise his candidature will be rejected.
- (16) Write a program to display the numbers of a series 1, 3, 9, 27, 81.... n, by using for loop.
- (17) Write a program to check that entered input data for the following. Whenever input is non-zero or positive display numbers from 1 to that number otherwise display message "Negative or zero". The program is to be performed for 10 numbers.
- (18) Write a program to check entered data types for 10 times. If a character is entered print "Character is entered" otherwise "Numeric is entered" for numerical values.
- (19) Write a program to find the sum of the first hundred natural numbers ($1 + 2 + 3 + \dots + 100$).
- (20) Write a program to display numbers 11, 22, 33,..., 99 using ASCII values from 48 to 57 in loops.
- (21) Create an infinite for loop. Check each value of the for loop. If the value is odd, display it otherwise continue with iterations. Print even numbers from 1 to 100. Use break statement to terminate the program.
- (22) Write a program to display the alphabets as a rectangle as shown below.
- (23) Write a program to read ten numbers through the keyboard and count the number of positive, negative and zero numbers.
- (24) Write a nested for loop that prints a 5 X 10 pattern of 0s.
- (25) Is it possible to create a loop using goto statement? If yes write the code for it.
- (26) Write a program to find the triangular number of a given integer. For example triangular of 5 is ($1 + 2 + 3 + 4 + 5$) 15. Use do-while loop.
- (27) Write a program to display all ASCII numbers and their equivalent character numbers and symbols. Use do-while loop. User should prompt every time to press 'Y' or 'N'. If the user presses 'Y' display next alphabet otherwise terminate the program.
- (28) Accept any five one-digit numbers. If the first number is smaller than the second, then display the sum of squares; otherwise, display the sum of cubes of all.
- (29) Evaluate the following series. Use do-while loop.
 - (a) $1 + 3 + 5 + 7 + \dots + n$
 - (b) $1 + 4 + 25 + 36 + \dots + n$
 - (c) $x + x^2/2! + x^3/3! + \dots + n$
 - (d) $1 + x + x^2 + x^3 + \dots + x^n$
- (30) Write a program to display the following using do-while loop.
 $a + 1 + b + 2 + c + 3 + \dots + n$, where n is an integer.
 $z + y + x + \dots + a$.
 $za + yb + xc + \dots + az$.
- (31) Enter the 10 numbers through the keyboard and sort them in ascending and descending orders, using do-while loop.
- (32) Enter text through the keyboard and display it in reverse order. Use do-while loop.
- (33) Print multiplication of digits of any number. For example number 235, multiplication to be $5 * 3 * 2 = 30$. Use do-while loop.
- (34) Print square roots of each digit of any number. Consider each digit as a perfect square. For example, for the number 494 the square roots to be printed should be 2 3 2.
- (35) Write a program to read any positive/negative integer number 'n' and generate the

```

      Z
    ZYZ
  XYZYX
RXYZYXR
XYZYX
  ZYZ
    Z

```

numbers in the following way. For example, if the entered number is 3, the output will be 9 4 1 0 1 4 9, else for negative (i.e., -3) the output will be 9 4 1 0 1 2 3.

- (36) Write a program to enter two integer values through the keyboard. Using `while` loop, perform the product of two integers. In case product is zero (0), loop should be terminated otherwise loop will continue.
- (37) Write a program to enter a single character either in lower or capital case. Display its corresponding ASCII equivalent number. Use the `while` loop for checking ASCII equivalent numbers for different characters. When capital 'E' is pressed, the program should be terminated.
- (38) Write a program to read a positive integer number 'n' and generate the numbers in the following way. If the entered number is 4 the output will be as follows. OUTPUT: 4! 3! 2! 1! 0 1! 2! 3! 4! 5!.
- (39) Write a program to read a positive integer number 'n' and generate the numbers in the different ways as given below.
 - If n = 10, generate 2 4 6 8 10.
 - If n = 11, generate 1 3 5 7 11.
- (40) Write a program to enter a multiple digit number and find its largest and smallest digits.
- (41) Write a program to enter an integer number and find the largest and smallest digits of the number.

Functions in C++

7

CHAPTER

CHAPTER OUTLINE

- 7.1 Introduction
- 7.2 Parts of a Function
- 7.3 Passing Arguments
- 7.4 Lvalues and Rvalues
- 7.5 Return by Reference
- 7.6 Returning More Values by Reference
- 7.7 Default Arguments
- 7.8 `const` Arguments
- 7.9 Inputting Default Arguments
- 7.10 Inline Functions
- 7.11 Function Overloading
- 7.12 Principles of Function Overloading
- 7.13 Precautions with Function Overloading
- 7.14 Recursion
- 7.15 Library Functions
- 7.16 More Programs

7.1 INTRODUCTION

In our daily life, most of the complex works are done by a group of people. This results in better productivity and quality. Work is done at a great speed. Normally, big work is distributed among many people. For example, to declare the result of examination in an autonomous institute, marks/score of all subjects, practical, seminar, projects, etc., are collected from different departments through professors and the result is compiled, declared, and displayed.

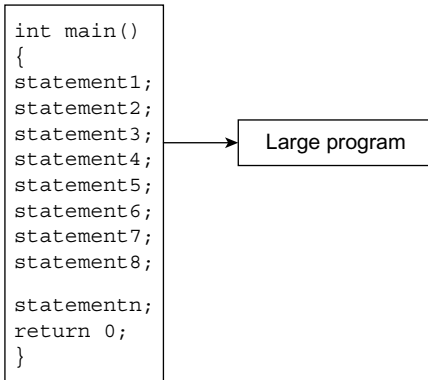


Fig. 7.1 Program without functions

The same concept is applied while developing complex programs/software applications using programming languages. In case the size of the program is large, it becomes difficult to maintain it and it is also hard to identify the flow of data. It is preferred to divide the large program into small modules called functions. Each function takes the data that are provided by the `main()` function and carries out operations as per the requirement, and results can be written to the calling function. Thus, programs are made more efficient.

When the concept of function or sub-program was not introduced, the program sizes were large and few codes get repeated as shown in the Figure 7.1.

It was very difficult to debug and update these large programs because many bugs get encountered. When the functions and sub-programs are introduced, the programs are divided into a number of segments and code duplication is avoided as shown in Figure 7.2. Bugs in small programs can be searched easily and this step leads to the development of complex programs with functions.

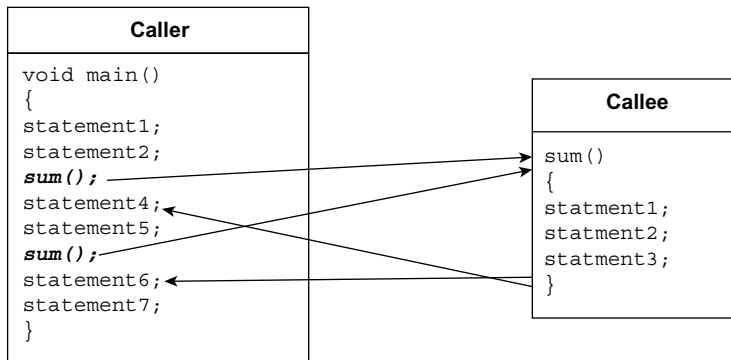


Fig. 7.2 Program with function

One of the features of C/C++ language is that a large program can be divided into smaller sub-programs. The smaller programs can be written in the form of functions. The process of dividing a large program into tiny and handy sub-programs and manipulating them independently is known as **modular programming**. This technique is similar to divide-and-conquer technique. Dividing a large program into smaller functions provides advantages to the programmer. Testing and debugging a program with functions is easier. A function can be called repeatedly based on the application and thus the size of the program can be reduced. The message passing between a caller (calling function) and callee (called function) takes place using arguments. The concept of modular approach of C++ is obtained from a function. Advantages of the functions are as follows:

- **Reusability:** A function once written can be invoked again and again, thus helping us to reuse the code and removing data redundancy.

- **Modularity:** Functions can help us in breaking a large, hard to manage problem into smaller manageable sub-problems. It is easier to understand the logic of sub-programs.
- **Reduced Program Size:** Functions can reduce the size of the program by removing data redundancy.
- **Easy Debugging:** Using functions, debugging of a program becomes very easy, as it is easier to locate and rectify the bug in the program if functions are used.
- **Easy Updating:** If we need to update some code in the program, then it is much more easier in case we have used functions, as the changes need to be made in one place only (in function).

The programs written in 'C++' language are highly depending on functions. The 'C++' program is nothing but a combination of one or more functions. Execution of every 'C++' program starts with user defined function `main()`. The method of using functions is slightly changed and enhanced in C++ as compared to C. The C++ language adds few new features to function like overloading of functions, default arguments etc. These features are described in ahead.

C++ functions are classified in two categories. They are (1) library functions and (2) user-defined functions. The library functions can be used in any program by including respective header files. The header files must be included using `#include` pre-processor directive. For example, a mathematical function uses `math.h` header file.

The programmer can also define and use his/her own functions for performing some specific tasks. Such functions are called *user-defined functions*. Every C++ program consists of `main()` function. This is a user-defined function in which program statements are written by the programmer according to the problem definition.

7.2 PARTS OF A FUNCTION

Parts of a function are as follows.

- (a) Function prototype declaration
- (b) Function call
- (c) Definition of a function
- (d) Actual and formal arguments
- (e) Return statement

7.2.1 Function Prototype Declaration

We use many built-in functions. The prototypes of these functions are given in the respective header files, and we include them using `#include` directive. In C++, while defining user-defined functions, it is unavoidable to declare its prototype. A prototype statement helps the compiler to check the return and argument types of the function.

A function prototype declaration consists of function's return type, name, and arguments list. It tells the compiler

- (a) Name of the function
- (b) Type of value returned
- (c) The type and number of arguments

When the programmer defines the function, the definition of function must be same like its prototype declaration. If programmer makes mistake, the compiler flags an error message. The function

prototype declaration statement is always terminated with semicolon. The statements given below are the examples of function prototypes.

```
(A) void show (void);
(B) float sum (float, int);
(C) float sum (float x, int y);
```

In example (A) the return type is void, that is the function does not return any value. The void functions are always without return statement. The void argument, that is the function, does not require any argument. By default, every function returns an integer value. To return a non-integer value, the data type should be mentioned in function prototype and definition. While writing definition of function, the return type must be preceded by the function name and it is optional if return type is default (int).

In statement (B), the prototype of function sum() is declared. Its return type is float and arguments are float and integer type, respectively. It is shown in Figure 7.3.

In example (C) with argument type, argument names are also declared. It is optional and also not compulsory to use the same variable name in the program.

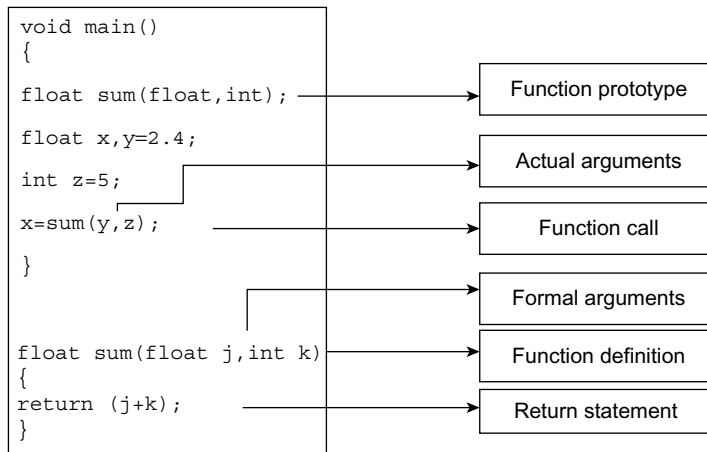


Fig. 7.3 Parts of a function

7.2.2 Function Call

A function is a latent body. It gets activated only when a call to a function is invoked. A function must be called by its name followed by argument, or without argument, list enclosed in parenthesis and terminated by semicolon.

Syntax of function call is as follows:

```
function-name(with/without argument list);
```

In the above statement, function-name is the name of the function, arguments are within the bracket and arguments are separated by comma. If arguments are absent one can write void within the bracket.

7.2.3 Function Definition

The first line is called **function definition** and function body follows it. The function definition and function prototype should match with each other. The function body is enclosed within curly braces. The function can be defined anywhere. If the function is defined before its caller, then its prototype declaration is optional.

Syntax of function call is as follows:

```
return_data_type function-name(argument/parameter list);  
{  
    variable declarations  
    function statements  
}
```

Return data type specifies the type of value returned by the function.

The function-name is the name of the function being defined.

The argument/parameter list specifies types and names of the arguments/parameters (also called formal arguments).

7.2.4 Actual and Formal Argument

The arguments declared in caller function and given in the function call are called **actual arguments**. The arguments declared in the function definition are known as **formal arguments**.

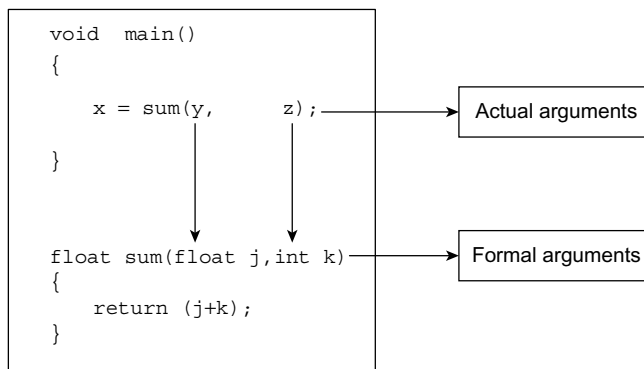


Fig. 7.4 Actual and formal arguments

As shown in Figure 7.4, variables `y` and `z` are actual arguments and variables `j` and `k` are formal arguments. The values of `y` and `z` are stored in `j` and `k`, respectively. The values of actual arguments are assigned to formal arguments. The function uses formal arguments for computing.

7.2.5 The return Statement

The return statement is used to return value to the caller function. The return statement returns only one value at a time. When a return statement is encountered, compiler transfers the control of the program to caller function. The syntax of return statement is as follows:

```
return (variable name); or return variable name;
```

The parenthesis is optional.

Few programs are as follows:

7.1 Write a program to declare prototype of function `sum()`. Define the function `sum()` exactly similar to its prototype.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    float sum (int, float ); // function prototype
    int a=20;
    float s,b=2.5;
    s=sum(a,b);
    cout<<"Sum=" <<s;
    return 0;
}

float sum (int x, float y)
{
    return (x+y);
}
```

OUTPUT

Sum = 22.5

Explanation: In the above program, the prototype of function `sum()` is declared. The prototype instructs the compiler that the function `sum()` should return `float` value. The types of arguments used by function `sum()` are `int` and `float`, respectively. The values of variables `a` and `b` are passed to function `sum()`. These values are assigned to formal arguments `x` and `y`. The sum of `x` and `y` is calculated and returned. The return value of function is assigned to `float` variable `s`.

7.2 Write a program to declare and define void function.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    void show (void);
    show();
    return 0;
}

void show()
{
    // ...
}
```

```
    cout<<"\n In show() ";  
}
```

OUTPUT

In show()

Explanation: In the above program, the prototype of function `show()` is given preceding `void` keyword. The statement `show()` invokes the `show()` function that displays the message “In show()”.

7.3 PASSING ARGUMENTS

The main objective of passing argument to function is message passing. The message passing is also known as communication between two functions, that is between caller and called functions. There are three methods by which we can pass values to the function. These methods are as follows:

- (a) Call by value (pass by value)
- (b) Call by address (pass by address)
- (c) Call by reference (pass by reference)

C++ supports all these three types of passing values to the function, whereas C supports the first two types only. The arguments used to send values to function are known as input arguments. The arguments used to return result are known as output arguments. The arguments used to send as well as return results are known as input-output arguments. While passing values to the function, the following conditions should be fulfilled.

The data type and number of actual and formal arguments should be same both in caller and callee functions. Extra arguments are discarded if they are declared. If the formal arguments are more than the actual arguments, then the extra arguments appear as garbage. Any mismatch in the data type will produce the unexpected result.

7.3.1 Call by Value

In this type, values of actual arguments are passed to the formal arguments and operation is done on the formal arguments. Any change in the formal arguments does not effect to the actual arguments because formal arguments are photocopy of actual arguments. Hence, when function is called by call by value method, it does not affect the actual contents of actual arguments. Changes made in the formal arguments are local to the block of called function. Once control returns back to the calling function, the changes made will vanish. The advantage of this call by value is that actual parameters are fully protected because their values are not changed when control is returned to the calling function. The following example illustrates the use of call by value.

7.3 Write a program to demonstrate call by value.

```
#include<iostream.h>  
#include<conio.h>
```

```

int main()
{
    clrscr();
    int x,y;
    void swap (int, int);
    cout<<"\n Enter Values of X & Y:";
    cin>>x>>y;
    swap(x,y);
    cout<<"\n\n In function main() ";
    cout<<"\n Values X="<<x <<" and Y= "<<y;
    cout<<"\nAddress X="<<(unsigned) &x <<" and Y= "<<(unsigned) &y;
    return 0;
}
void swap(int a, int b)
{
    int k; k=a; a=b; b=k;
    cout<<"\n In function swap() ";
    cout<<"\n Values X="<<a <<" and Y= "<<b;
    cout<<"\nAddress X="<<(unsigned) &a <<" and Y=
"<<(unsigned) &b;
}

```

OUTPUT

Enter Values of X & Y :5 4

In function swap()

Values X=4 and Y= 5

Address X=4090 and Y= 4092

In function main()

Values X=5 and Y= 4

Address X=4096 and Y= 4094

Explanation: In the above program we are passing values of actual arguments 'x' and 'y' to function swap(). The formal arguments 'a' and 'b' of function swap() receive these values. The values are exchanged i.e. value of 'a' is assigned to 'b' and vice-versa. They are displayed on the screen. When the control returns back to the main(), the changes made in function swap() vanish, because a and b are local variables of function swap(). In the main() the values of 'x' and 'y' are printed as they read from the keyboard. In call by value method the formal argument acts as duplicate of the actual argument. The addresses of actual and formal arguments are different. Thus, changes made with the variables are temporary.

7.3.2 Call by Address

In this type, instead of passing values, addresses of actual parameters are passed to the function by using pointers. Function operates on addresses rather than values. Here the formal arguments

are pointers to the actual arguments. In this type, formal arguments are pointers to actual argument. Because of this, when the values of formal arguments are changed, the values of actual parameters also change. Hence changes made in the argument are permanent. The following example illustrates passing the arguments to the function using call by address method.

7.4 Write a program to demonstrate pass by address.



```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x,y;
    void swap (int *, int *);
    cout<<"\n Enter Values of X & Y:";
    cin>>x>>y;
    swap(&x, &y);
    cout<<"\n In main()";
    cout<<"\n Values X="<<x <<" and Y="<<y;
    cout<<"\n Address X="<<(unsigned) &x <<" and Y="<<(unsigned) &y;
    return 0;
}

void swap(int *a, int *b)
{
    int *k;
    *k=*a;
    *a=*b;
    *b=*k;
    cout<<"\n In swap()";
    cout<<"\n Values X="<<*a <<" and Y="<<*b;
    cout<<"\n Address X="<<(unsigned) a <<" and Y="<<(unsigned) b;
}
```

OUTPUT

```
Enter Values of X & Y :5 4
In swap()
Values X=4 and Y=5
Address X=4096 and Y=4094
In main()
Values X=4 and Y=5
Address X=4096 and Y=4094
```

Explanation: In the above example we are passing addresses of actual arguments to the function swap (). The formal arguments are declared as pointers in the function declaration of swap ().

The formal arguments receive addresses of actual arguments i.e. formal arguments points to the actual argument. Here the `swap()` function operates on the addresses of actual argument through pointers. The addresses of actual arguments and formal arguments are same. Hence, the changes made in the values are permanent.

7.3.3 Call by Reference

C passes arguments by value and address. In C++ it is possible to pass arguments by value, address, and reference. C++ reference types, declared with `'&'` operator, are nearly identical but not exactly the same as pointer types. They declare aliases for objects variables and allow the programmer to pass arguments by reference to functions. The reference decelerator (`&`) can be used to declare references outside functions.

```
int k = 0;
int &kk = k; // kk is an alias for k
kk = 2; // same effect as k = 2
```

This creates the lvalue `kk` as an alias (assumed name) for `k`, provided that the initializer is the same type as the reference. Any operations on `kk` have exactly the same result as operations on `k`.

```
Example,
kk = 5 // assigns 5 to kk and
&kk // returns the address of kk.
```

The reference decelerator can also be used to declare reference type parameters within a function:

```
void funcA (int i);
void funcB (int &kk); // kk is type "reference to int"
int s=4;
funcA(s); // s passed by value
funcB(s); // s passed by reference
```

The `s` argument passed by reference can be modified directly by `funcB`; whereas the function `funcA` receives a duplicate copy (not actual) of the `s` argument (passed by value). Due to this reason, variable `s` itself cannot be modified by `funcA`. When an actual variable `s` is not appearing in the above example, argument `s` is passed by value, and the matching formal argument in the function obtains a copy of `s`. Any alterations to this copy inside the scope of the function body are not thrown back in the value of `s` itself. Absolutely, the function can return a value that could be used later to change `s`, but the function cannot directly alter a parameter passed by value. The conventional C method for changing `s` operates on the address of actual argument (`&s`), the address of `s`, rather than `s` itself. Although `&s` is passed by value, the function can access `s` through the copy of `&s` it receives. Even though the function does not need to alter `s`, it is still useful (though subject to possibly risky secondary results) to pass `&s`, especially if `s` is a large data structure. Passing `s` directly by value contains the useless copying of the data structure.

7.5 Write a program to pass the value of variable by value, reference, and address and display the result.

```
#include<iostream.h>
#include<conio.h>
#include<process.h>

int main()
{
    clrscr();
    void funA (int s);
    void funB (int & );
    void funC (int *);
    int s=4; // initial value
    funA(s); // s passed by value
    cout<<"\nValue of s= "<<s <<" Address of s: "
        <<unsigned(&s);
    funB(s); // s passed by reference
    cout<<"\nValue of s= "<<s <<" Address of s: "
        <<unsigned(&s);
    funC(&s); // s passed by reference (C style )
    cout<<"\nValue of s= "<<s <<" Address of s: "
        <<unsigned(&s);
    return 0;
}

void funA ( int i)
{ i++;}
void funB (int &k)
{ k++;}
void funC(int *j)
{ ++*j;}
```

OUTPUT

Value of s= 4 Address of s : 4096

Value of s= 5 Address of s : 4096

Value of s= 6 Address of s : 4096

Explanation: In the above example, funA(), funB(), and funC() functions are declared. The integer variable s is declared and initialized to 4. The funA() is invoked and value of s is passed by value. In function funA() the value of s is incremented. After execution of funA() the value of s printed in main() is the same as the previous one. Thus, passing variable by value cannot change the contents of the variable.

The value of s is once again passed to funB(). The function funB() receives the value of s by reference. The value of s is received by the variable k by reference, that is variable k is

an alias for variable *s* and both have the same memory location. The variable *k* is incremented. Thus, any change made in reference variable affects the actual variable. Thus, after execution of `funB()` the printed value of *s* is 5.

The third function `funC()` uses conventional C style. Here, address of the variable *s* is passed to `funC()`. The formal argument `*j` is pointer to the actual argument. Thus, any change made through pointer *j* also reflects on the actual variable. Thus, we can change the value of variable using call by reference. Figure 7.5 explains the methods of passing arguments to function.

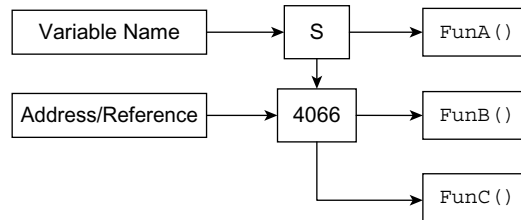


Fig. 7.5 Argument passing methods

7.6 Write a program to demonstrate call by reference.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int x,y;
    void swap (int &, int &);
    cout<<"\n Enter Values of X & Y:";
    cin>>x>>y;
    swap(x,y);
    cout<<"\n In main() ";
    cout<<"\n Values X="<<x <<" and Y="<<y;
    cout<<"\n Addresses X="<<(unsigned)&x <<" Y
        ="<<(unsigned)&y;
    return 0;
}

void swap(int &a, int &b)
{
    int k; k=a; a=b; b=k;
    cout<<"\n In swap() ";
    cout<<"\n Values X="<<a <<" and Y="<<b;
    cout<<"\n Addresses X="<<(unsigned)&a <<" Y
        ="<<(unsigned)&b;
}
  
```

OUTPUT**Enter Values of X & Y :2 3****In swap ()****Values X=3 and Y=2****Addresses X=65524 Y =65522****In main ()****Values X=3 and Y=2****Addresses X=65524 Y =65522**

Explanation: In the above program, the function `swap ()` is invoked and actual arguments, variable `x` and `y`, are passed. The formal arguments of function `swap ()`, `a` and `b`, are reference variables of actual arguments `x` and `y`. The addresses of actual and formal arguments are same. Any change made in `a` and `b` leads to change in the values of `x` and `y`. To pass variables by reference, the function call is like call by value. In function declarator and function prototype declaration, the formal arguments are preceded by `&` operator. The reference type variable can be used in the same manner like other ordinary variables. The following points are noted concerning reference parameters:

- (a) A reference may not be null. It should always refer to a legal variable.
- (b) Once declared, a reference should not be altered to refer to another object.
- (c) A reference variable does not need any explicit address manipulation to access the actual value of the variable.

7.4 LVALUES AND RVALUES**7.4.1 Lvalues (Left Values)**

An expression that indicates the location is referred as **lvalue** and the expression that indicates the value is referred as **rvalue**. On left-hand side of the assignment operator (`=`), lvalues expression appears. Lvalues are quite often identifiers; whereas, on the right-hand side of the assignment operator (`=`), rvalue expression appears, which is a value to be stored at some address in memory. The set of rvalues is the subset of set of lvalues.

An lvalue is an object locator. It is an expression that points an object. An example of an lvalue expression is `*k` that results to a non-null pointer. A changeable lvalue is an identifier or expression that relates to an object that can be accessed and suitably modified in computer memory. A `const` pointer to a constant is not a changeable lvalue. A pointer to a constant can be altered (its de-referenced value cannot be altered). An lvalue could suitably stand on the left (the assignment side) of an assignment statement. Now, only changeable lvalue can legally stand on the left of an assignment statement. For example, suppose `x` and `y` are non-constant integer identifiers with appropriate allocated memory. Their lvalues are changeable. The following expressions are legal:

```
x = 1;  
y = x + y are legal expressions.
```

7.4.2 Rvalues (Right Values)

The statement $x + y$ is not an lvalue, $x + y = z$ is invalid because the expression on the left is not related to a variable. Such expressions are often called *rvalues*.

7.5 RETURN BY REFERENCE

We have studied the reference variable and its functioning. A reference allows creating alias for the pre-existing variable. A reference can also be returned by the function. A function that returns reference variable is in fact an alias for referred variable. This technique of returning reference is used to establish cascade of member functions calls in operator overloading. Consider the following example:

The below given program illustrates return by reference.

7.7 Write a program to return a value by reference.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int & min ( int &j, int &k);
    int a=18,b=11,c;
    c=min (a,b);
    cout<<"Minimum Value = "<<c;
    return 0;
}

int & min (int &j, int &k)
{
    if (k<j ) return k;
    else
    return j;
}
```

OUTPUT

Minimum Value = 11

Explanation: In the above program, the statement `int & min (int &j, int &k)` declares prototype of function `min()`. The '&' reference operator is used because the function returns reference to int and also receives arguments as reference. The function `min()` receives two integers as reference and returns minimum value out of two by reference.

7.8 Write a program to demonstrate return by reference.

```
#include<iostream.h>
#include<conio.h>
```

```

int & large (int & p, int & q);
int main()
{ clrscr();
  int l,k;
  cout<<"\n Enter values of l and k: ";
  cin>>l>>k;
  large (l,k)=120;
  cout<<" l= "<<l << " k="<<k;
  return 0;
}
int & large (int & p, int & q)
{
  if (p>q) return p;
  else return q;
}

```

OUTPUT

Enter values of l and k : 4 8

l= 4 k=120

Enter values of l and k : 9 2

l= 120 k=2

Explanation: In function `main()`, the statement `large (l, k) = 120` calls the function `large()`. It returns reference to the variable containing larger value and assigns the value 120 to it. The return type of function `large()` is `int '&'` (reference), it indicates that the call to function `large()` can be written on the left side of the assignment operator. Consequently, the statement `large(l,k) = 120` is legal and assigns 120 to the variable containing larger value.

7.6 RETURNING MORE VALUES BY REFERENCE

The `return()` statement has one big limitation that it can return only one value at a time. The `return()` statement is only used when values are passed by value to functions. Call by address and call by reference, accesses memory location directly. When user wants to return more than one value from function, he/she should pass values by address or by reference method. In C for returning more values, call by address is used. In C++ we have one additional method, that is call by reference. Consider the following program.

7.9 Write a program to return more than one value from function by reference.

```

#include<iostream.h>
#include<conio.h>

int main()
{
  int sq,cb,n;

```

```

void more (int &, int &, int);
cout<<"\n Enter a Number: ";
cin>>n;
more (sq,cb,n);
cout<<"\n Square ="<<sq;
cout<<"\n Cube ="<<cb;
return 0;
}
void more (int & s, int & c,int j)
{
    s=j*j;
    c=j*j*j;
}

```

OUTPUT

Enter a Number : 2

Square =4

Cube =8

Explanation: In the above program the function `more()` is used to perform square and cube of an integer passed. The three variables `sq`, `cb`, and `n` of integer type are declared. The number entered by the user is stored in variable `n`. The variables `sq`, `cb`, and `n` are passed to function `more()`.

In function `more()`, `s` and `j` are reference variables and `j` is a local variable of function `more()`. The variable `s` is reference variable of `sq` and variable `c` is reference variable of `cb`. The variables `s` and `sq`, and `c` and `cb` have the same memory locations. Thus, any value assigned to `s` and `c` can be accessed by `sq` and `cb`. The variable `s` is assigned the square of `j(n)` and variable `cb` is assigned with cube of `j(n)`.

7.7 DEFAULT ARGUMENTS

Usually, a function is called with all the arguments as declared in function prototype declaration and function definition. C++ compiler allows the programmer to assign default values in the function prototype declaration/function declarator of the function. When the function is called with less parameter or without parameters, the default values are used for the operations.

It is not allowed to assign default value to any variable, which is in between the variable list. Variables lacking default values are written first followed by the variables containing default values. This is so because C++ convention of storing variables on the stack is from right to left.

The default values can be specified in function prototype declaration or function declarator. Normally, the default values are placed in function prototype declaration. In case the function is defined before caller function, then there is no need to declare function prototype. Hence, the default arguments are placed in function declarator. The compiler checks for the default values in function prototype and function declarator and provides these values to the arguments that are omitted in function call.

The default arguments are useful while making a function call if we do not want to take effort for passing arguments that are always same. It is also useful when we update an old function by adding more arguments to it. Using default arguments, the function calls can continue to use previous arguments with the new arguments.

The following example declares the default arguments:

Example

```
int sum(int a, int b=10, int c=15, int d=20)
```

In this example function `sum()` has four arguments. They are `a`, `b`, `c`, and `d` of integer types. The variable `b`, `c`, and `d` are initialized with 10, 15, and 20, respectively. These values are used when the function `sum()` is called with fewer arguments.

7.10 Write a program to define function sum with default arguments.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int sum (int a,int b=10,int c=15,int d=20 ); // Function
                                                    prototype

    int a=2;
    int b=3;
    int c=4;
    int d=5;
    cout<<"Sum=" <<sum(a,b,c,d);
    cout<<"\nSum=" <<sum(a,b,c);
    cout<<"\nSum=" <<sum(a,b);
    cout<<"\nSum=" <<sum(a);
    cout<<"\nSum=" <<sum(b,c,d);
    return 0;
}

sum (int j, int k, int l, int m)
{
    return (j+k+l+m);
}
```

OUTPUT

```
Sum=14
Sum=29
Sum=40
Sum=47
Sum=32
```

Explanation: In the above example, the prototype of variable `sum()` is declared. The function `sum()` has four arguments of integer types **a**, **b**, **c**, and **d**. The variables `b`, `c`, and `d` are initialized with default values 10, 15, and 20, respectively, in function prototype declaration. The variable "a" is not initialized. The function `sum()` is called five times. Each time the result has different values.

In the first `cout` statement, the function `sum()` is called with four arguments. In this function call, no default values are used, that is actual values of variables initialized are considered. Hence, the result is 14.

In the second `cout` statement the function `sum()` is called with three arguments (one less argument). In this function call actual values of starting three variables `a`, `b`, `c` and default value of forth variable "d" are considered. Hence, the result is 29.

In the third `cout` statement, the function `sum()` is called with two arguments (two arguments less). In this function call actual values of starting two variables `a`, `b` and default values of last two variables `c`, `d` is considered. Hence, the result is 40.

In the fourth `cout` statement the function `sum()` is called with one argument. In this function call, the actual value of variable `a` and default values of variables `b`, `c`, and `d`, 10, 15, and 20, respectively, are taken into account. The result displayed is 47.

In the last `cout` statement the function `sum()` is called with last three arguments. The actual values of `b`, `c`, and `d` variables are 3, 4, and 5 and last default value 20 is together taken into account. Hence the result is 32.

7.11 Write a program to place default values in function declarator. Execute the function with default values.

```
#include<iostream.h>
#include<conio.h>

sum (int j, int k=10, int l=15, int m=20)
{
    return (j+k+l+m);
}

int main()
{
    clrscr();
    int a=2;
    int b=3;
    int c=4;
    int d=5;
    cout<<"Sum=" <<sum(a,b,c,d);
    cout<<"\nSum=" <<sum(a,b,c);
    cout<<"\nSum=" <<sum(a,b);
    cout<<"\nSum=" <<sum(a);
    cout<<"\nSum=" <<sum(b,c,d);
    return 0;
}
```

OUTPUT

Sum=14
 Sum=29
 Sum=40
 Sum=47
 Sum=32

Explanation: In the above example, the prototype of variable `sum()` is declared. The function `sum()` has four arguments of integer types **a**, **b**, **c**, and **d**. The variable **b**, **c**, and **d** are initialized with default values 10, 15, and 20, respectively, in function prototype declaration. The variable "a" is not initialized. The function `sum()` is called five times. Each time the result sum has different values.

In the first `cout` statement, the function `sum()` is called with four arguments. In this function call no default values are used, that is actual values of variables initialized are considered. Hence, the result is 14.

In the second `cout` statement the function `sum()` is called with three arguments (one less argument). In this function call actual values of starting three variables **a**, **b**, **c** and default value of forth variable "d" are considered. Hence, the result is 29.

In the third `cout` statement, the function `sum()` is called with two arguments (two arguments less). In this function call actual values of starting two variables **a**, **b** and default values of last two variables **c**, **d** are considered. Hence, the result is 40.

In the fourth `cout` statement the function `sum()` is called with one argument. In this function call the actual value of variable **a** and default values of variables **b**, **c**, and **d** are 10, 15, and 20, respectively, are taken into account. The result displayed is 47.

In the last `cout` statement the function `sum()` is called with last three arguments. The actual values of **b**, **c**, and **d** variables are 3, 4, and 5 and last default value 20 is together taken into account. Hence the result is 32.

7.12 Write a program to find area of triangle by using default values and actual values.

The formula for area of a triangle is $\frac{1}{2} * \text{base} * \text{height}$.

```
#include<iostream.h>
#include<conio.h>
```

```
int main()
{
```

```
    clrscr();
    float area (int base=3,int height=5 ); // Function prototype
    int base=2;
    int height=7;
    cout<<"Area of Triangle: " << area(base,height);
    cout<<"\nArea of Triangle: " << area (base);
    cout<<"\nArea of Triangle: " <<area(height);
    return 0;
```




```

}
float area (int j, int k)
{ return (.5*j*k); }

```

OUTPUT

Area of Triangle : 7

Area of Triangle : 5

Area of Triangle : 17.5

Explanation: In the above program, the prototype of function `area()` is declared with two default values 3 and 5 for variables `base` and `height`, respectively. In function `main()` the variable `base` and `height` are defined and initialized with 2 and 7, respectively. In the first call of the function `area()` function is called with both the arguments, that is `base` and `height`. In this call no default arguments are used. In the second call the function `area()` is called with one argument (one less argument). In this call one actual and one default value are used. The third call is the same as second one.

7.8 const ARGUMENTS

The constant variable can be declared using `const` keyword. The `const` keyword makes variable value stable. The constant variable should be initialized while declaring.

Syntax:

```

(a) const <variable name> = <value>;
(b) <function name> (const <type>*<variable name>;)
(c) int const x // invalid
(d) int const x =5 // valid

```

In statement (a), the `const` modifier enables to assign an initial value to a variable that cannot be changed later by the program.

For example,

```
const age = 40;
```

Any attempt to change the contents of `const` variable `age` will produce a compiler error. Using pointer one can indirectly modify a `const` variable as per the following:

```
*(int *)&age = 45;
```

When the `const` variable is used with a pointer argument in a function's parameter list, the function cannot modify the variable that the pointer points to. For example,

```
int cube(const int *x, ...);
```

Here the `cube` function is prohibited from altering the integer variable.

7.13 Write a program to declare constant variable.

```

#include<iostream.h>
#include<conio.h>

```

```

int main()
{
    clrscr();
    int min (int const a=8, int b=20);
    int a=12,b=45;
    b=min (a);
    cout<<"\n a= "<<a <<" b= "<<b;
    return 0;
}
int min (int const j, int k)
{
    // j++; // can not modify a constant variable
    // k++; // valid because b is not constant
    cout<<"\n j= "<<j <<" k ="<<k;
    if (k<j ) return k;
    else return j;
}

```

OUTPUT

j= 12 k =20
a= 12 b= 12

Explanation: In the above program, the prototype of function `min()` declares two arguments. Out of two arguments the first argument `a` is constant and its default value is 8. The second variable is `b` and default value is 20. In function `main()` we can change the value of variable `a`, because in `main()` it is not declared as a constant. When function `min()` is invoked, the first argument `j` is a constant according to prototype of function. Any attempt made to change the value of variable `j` will cause an error message “cannot modify a constant variable.”

7.14 Write a program to change the value of a constant variable using pointer.

```

#include<iostream.h>
#include<conio.h>
#include<process.h>

int main()
{
    clrscr();
    int const x=1;
    // x++; invalid operation
    // cin>>x; possible, but no change in constant value
    ++(*(int *)&x);
    cout<<"x = "<<x <<"\t\t Address: "<<(unsigned)&x;
    cout<<"\n*(&x) = "<<*(&x)<<"\t\t Address: " <<(unsigned)&x;
    return 0;
}

```

```
}

```

OUTPUT

```
x = 1 Address : 65524
*(&x) = 2 Address : 65524
```

Explanation: In the above program, the variable `x` is a constant integer variable. In C it is possible to change the value of a constant variable using `scanf()` statement, that is run time assignment. In C++, though compiler accepts this statement, the value is not changed. We can change the value of constant variable using a pointer. The statement `++(* (int *) &x);` points to the address of the variable `x` and increase is done in the value stored at the location, that is indirectly to variable `x`. Though it is possible to change the value of a variable using pointer, the following statements are invalid.

```
int const x=1;
int const *p=&x;
++*p;
```

It is not possible to change the value of a variable using pointer `*p`. From the above program we learned that *the value of a constant could be changed using pointer*.

7.9 INPUTTING DEFAULT ARGUMENTS

Default arguments are used when function is invoked with fewer arguments than declared. The default values are placed in function prototype or function declarator. It is also possible to directly invoke user-defined function in function prototype or function declarator. The return value of function is considered as default value. The following program illustrates this point:

7.15 Write a program to enter default values using function.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int input(void);
    void display (int = input());
    display(20); // first call with argument
    display(); // second call with default argument
    return 0;
}

void display (int j)
{
```

```
    cout<<"\nInteger is: "<<j;
}
input()
{
    int k;
    cout<<"\n Enter an integer: ";
    cin>>k;
    return k;
}
```

OUTPUT

Integer is : 20

Enter an integer : 12

Integer is : 12

Explanation: In the above program, user-defined functions `display()` and `input()` are declared and defined. The `display()` function is used to display passed integer on the screen. The `input()` function when executed asks for an integer. The function `input()` is used as default value for function `display()`. When a function `display()` is invoked without argument, the `input()` function assigned as default argument in function prototype is executed. User enters a number. The entered number is then passed on to `display()` function and displayed on the screen.

7.10 INLINE FUNCTIONS

One of the prime factors behind using function is that code duplication in program is avoided and memory space is saved. When a function is defined and invoked, one set of instruction is created in the memory of the system. At each call, the control passes to the subroutine at a specified address in the memory. The CPU stores the memory address of instruction following the function calls in to the stack and also pushes the arguments in the stack area when call is made to the function. The compiler runs the function, stores the return values in a memory location or register, and when execution completes, the control returns to the calling function. After this, execution resumes immediately to the next line following the function call statement. If the same function is called several times, each time the control is passed to the function, that is the compiler uses the same set of instructions at each call. Due to this passing of control in between caller and callee functions, execution speed of program decreases. By preventing the repetitive calls in a program, execution speed can be increased.

C++ provides a mechanism called `inline` function. When a function is declared as `inline`, the compiler copies the code of the function in the calling function, that is function body is inserted in place of function call during compilation. Passing of control between caller and callee functions is avoided. If the function is very large, in such a case `inline` function is not used because the compiler copies the contents in the called function that reduces the program execution speed. The `inline` function is mostly useful when calling function is small. It is advisable to use the `inline` function for only small functions.

Inline mechanism increases execution and performance in terms of speed. The overhead of repetitive function calls and returning values are removed. On the other hand, the program using inline functions needs more memory space, since the inline function are copied at every point where the function is invoked.

For defining an inline function, the `inline` keyword is preceded by the function name. The inline functions are declared as follows:

```
inline function - name
{
    statement1;
    statement2; // Function body
}
```

```
Example:
inline float square (float k)
{
    return (k*k);
}
```

The above example can be executed as follows:

```
j = square(2.5);
k = square(1.1 + 1.4);
```

After the execution of above statements, the values of `j` and `k` will be 6.25. The `inline` keyword just makes an appeal to the compiler. The compiler may neglect this request if the function defined is too big in size or too convoluted. In such a case the function is treated as normal function.

Following are few situations where inline functions may not work:

- (1) The function should not be recursive.
- (2) Function should not contain static variables.
- (3) Function containing control structure statements such as `switch`, `if`, `for` loop, etc.
- (4) The function `main()` cannot be used as inline.

The inline functions are similar to macros of C. The main limitation of macros is that they are not functions, and errors are not checked at the time of compilation. The function offers better type testing and does not contain limitations as present in macros. Consider the following example:

7.16 Write a program to calculate square using inline functions and macros.

```
#include<iostream.h>
#include<conio.h>
#define SQUARE(v) v * v

inline float square (float j)
{
```



```

    return (j*j);
}
int main()
{
    clrscr();
    int p=3, q=3,r,s;
    r=SQUARE(++p);
    s=square(++q);
    cout<<" r= "<<r<<"\n"<<" s= "<<s;
    return 0;
}

```

OUTPUT

r= 25

s= 16

Explanation: In the above program, the function `square()` is declared as an inline function. The macro `square()` expanded into `r = ++v * ++v`. The variable `p` is incremented only once but in macros expansion is incremented twice. Therefore, it gives wrong result.

7.17 Write a program to define an inline function and obtain the result.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int sum (int,int,int,int ); // Function prototype
    cout<<"Sum=" <<sum(5,4,3,1);
    return 0;
}

inline sum (int j, int k, int l, int m)
{
    return (j+k+l+m);
}

```

OUTPUT

Sum = 13

Explanation: In the above program, the function `sum()` calculates the sum of given integers. The function is defined as inline. The compiler copies the code of the function in the calling function and executes it.

7.18 Write a program to define function cube() as inline for calculating cube.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int cube (int);
    int j,k,v=5;
    j = cube (3) ;
    k = cube (v);
    cout<<"\n Cube of j="<<j;
    cout<<"\n Cube of k="<<k;
    return 0;
}

inline int cube (int h)
{
    return (h*h*h);
}
```

OUTPUT**Cube of j=27****Cube of k=125**

Explanation: In the above program, the function cube () is declared as inline. The function cube () calculates cube of passed number. The function is declared as inline, that is the statements of function cube () are inserted at the point of function call.

7.11 FUNCTION OVERLOADING

It is possible in C++ to use the same function name for number of times and for different intentions. Defining multiple functions with same name is known as function overloading or function polymorphism. Polymorphism means one function having many forms. The overloaded function must be different in their argument list and with different data types. The following are examples of overloaded functions. All the functions defined should be equivalent to their prototypes.

```
int  sqr(int);
float  sqr(float);
long  sqr(long);
```

7.19 Write a program to calculate square of an integer and float number. Define function sqr(). Use function-overloading concept.

```
#include<iostream.h>
#include<conio.h>
```

```

int sqr (int);
float sqr (float);

int main()
{
    clrscr();
    int a=15;
    float b=2.5;
    cout<<"Square = "<<sqr(a) <<"\n";
    cout<<"Square = "<< sqr(b) <<"\n";
    return 0 ;
}

int sqr (int s)
{
    return (s*s);
}

float sqr (float j)
{
    return (j*j);
}

```

OUTPUT

Square = 225

Square =6.25

Explanation: In the above function, `sqr()` is overloaded for integer and `float`. In the first call of the function `sqr()`, an integer 15 is passed. The compiler executes integer version of the function and returns result 225. In the second call a `float` value 2.5 is passed to function `sqr()`. In this call, the compiler executed the `float` version of the function and returns the result 6.25. The selection of which function to execute is decided at run time by the compiler according to the data type of variable passed.

7.20 Write a program to find the area of rectangle, triangle and surface area of sphere. Use function overloading.

```

#include<iostream.h>
#include<conio.h>
#define pi 3.142857142857142857
int calcarea(int length,int breadth);
float calcarea(double base,double height);
float calcarea(double radius);

int main()
{
    int area1;
    float area2;

```




```

double area3;
area1=calcare(10,20);
area2=calcare(4.5,2.1);
area3=calcare(3.12145);
clrscr();
cout<<"Area of rectangle is: "<<area1<<endl;
cout<<"Area of triangle is: "<<area2<<endl;
cout<<"Surface Area of sphere is: "<<area3<<endl;
return 0;
}
int calcare(int length,int breadth)
{
    return (length*breadth);
}
float calcare(double base,double height)
{
    return ((0.5)*base*height);
}
float calcare(double radius)
{
    return (4*pi*radius*radius);
}

```

OUTPUT

Area of rectangle is : 200

Area of triangle is : 4.725

Surface Area of sphere is : 122.489087

Explanation: In the above program, function **calcare()** is overloaded. It is used for finding the area of a rectangle, triangle, and sphere. Explanation is same as detailed in previous problem.

7.12 PRINCIPLES OF FUNCTION OVERLOADING

- (1) If two functions have the similar type and number of arguments (data type), the function cannot be overloaded. The return type may be similar or void, but argument data type or number of arguments must be different. For example,

(a) `sum(int, int, int);`
`sum(int, int);`

Here, the above function can be overloaded. Though the data type of arguments in both the functions are similar, number of arguments are different.

(b) `sum(int, int, int);`
`sum(float, float, float);`

In the above example, number of arguments in both the functions are same, but data types are different. Hence, the above function can be overloaded.

- (2) Passing constant values directly instead of variables also results in ambiguity. For example,
- ```
int sum(int, int);
float sum(float, float, float);
```
- Here, `sum()` is an overloaded function for integer and float values. Values are passed as follows:

```
sum(2, 3);
sum(1.1, 2.3, 4.3);
```

The compiler will flag an error because the compiler cannot distinguish between these two functions. Here, internal conversion of float to int is possible. Hence, in both the above calls integer version of function `sum()` is executed. To overcome this problem, the user needs to do the following things.

- (1) Declare prototypes of all the overloaded functions before function `main()`.
- (2) Pass argument using variables as follows:

```
sum(a,b); // a and b are integer variables
sum(e,r,t,y); // e,r,t, and y are float variables
```

Refer program number 7.22 for confirmation. Also try this program with direct values and by declaring function prototype inside `main()`.

- (3) The compiler attempts to find an accurate function definition that matches in types and number of arguments and invokes that function. The arguments passed are checked with all declared function. If matching function is found then that function gets executed.
- (4) If there are no accurate match found, compiler makes the implicit conversion of actual argument. For example, `char` is converted to `int` and `float` is converted to `double`. If all the above steps have failed, then compiler performs user-built functions.

The following example illustrates this point:

**7.21 Write a program to overload a function and create the situation that the compiler does integral conversion.**

```
#include<iostream.h>
#include<conio.h>

int sqr (int);
float sqr (double);
int main()
{
 clrscr();
 int a=15;
 float b=3.5;
 cout<<"Square = "<<sqr('A') <<"\n";
 cout<<"Square = "<< sqr(b) <<"\n";
 return 0 ;
}

int sqr (int s) { return (s*s); }
float sqr (double j) { return (j*j); }
```

**OUTPUT**

**Square = 4225**  
**Square = 12.25**

**Explanation:** In the above program, the function square is overloaded. The first prototype of function `sqr(int)` is proposed for integer variable and second for `double` variable. In function `main()` the "A" character data type value is passed to the function `sqr()`. The compiler invokes the integer version of the function `sqr()` because the `char` data type is compatible with integer. In the second call, a `float` value is passed to the function `sqr()`. This time the compiler invokes the `double` data type version of function `sqr()`.

When accurate match is not found the compiler makes internal conversion as seen in the above example. C++ automatically attempts to convert the arguments used to call a function into the type of arguments expected by the function.

- (5) If internal conversion fails, user-defined conversion is carried out with implicit conversion and integral promotion. The user-defined conversion is used with class objects.
- (6) Sometimes while making internal conversion, ambiguity is created if one data type is compatible with two or more data types. If such a situation occurs, the compiler displays an error message.

Consider the following example. Following are two versions of `sqr()` function, one is for long data type and other for double data type.

```
long sqr(long);
double sqr(double);
sqr(10); // function call
```

If an integer value is passed to the function `sqr()`, the compiler will have confusion of which version is to be executed and will result in an error message "**Ambiguity between 'sqr(long)' and 'sqr(double)'**".

- (7) The program has two versions of function, that is one for float and second for double data type. In such a situation if we pass a float number, the double version of function is selected for execution. But the same is not applicable with integer and long integer.

**7.22 Write a program to define overloaded function `add()` for integer and float and perform the addition.**

```
#include<iostream.h>
#include<conio.h>

int add (int, int, int);
float add (float, float, float);
int main()
{
 clrscr();
```

```

float fa,fb,fc,fd;
int ia,ib,ic,id;
cout<<"\n Enter values integer values for ia,ib and ic: ";
cin>>ia >>ib >>ic;
cout<<"\n Enter float values for fa,fb and fc: ";
cin>>fa >>fb >>fc;
id=add (ia,ib,ic);
cout<<"\n Addition: " <<id;
fd= add (fa,fb,fc);
cout<<"\n Addition: " <<fd;
return 0;
}
add (int j, int k, int l)
{
 return (j+k+l);
}
float add (float a, float b, float c)
{ return (a+b+c); }

```

## OUTPUT

**Enter values integer values for ia,ib and ic : 1 2 4**

**Enter float values for fa,fb and fc : 2.2 3.1 4.5**

**Addition : 7**

**Addition : 9.8**

**Explanation:** In the above program, two versions of function `add()` are declared. One for integer values and the other for float values. Three integers and float values are entered through the keyboard. According to the data type, the compiler executes appropriate function. The prototype of functions should be written before `main()` function. If we write the prototype inside the `main()` function, the float version of the function will not be executed.

### 7.23 Write a program to define overloaded function. Invoke the overloaded function through another overloaded function.

```

#include<iostream.h>
#include<conio.h>

int add (void);
float add (float, float, float);
float fa,fb,fc,fd;
int id;
int main()
{
 clrscr();

```

```

 cout<<"\n Enter float values for fa,fb and fc: ";
 cin>>fa >>fb >>fc;
 id=add();
 cout<<"\n Addition: " <<id;
 fd= add (fa,fb,fc);
 cout<<"\n Addition: " <<fd;
 return 0;
}
add ()
{
 int x;
 x=add(fa,fb,fc);
 return (x);
}
float add (float a, float b, float c)
{
 return (a+b+c);
}

```

## OUTPUT

**Enter float values for fa,fb and fc : 5.5 2.4 8.2**

**Addition : 16**

**Addition : 16.1**

**Explanation:** In the above program, the function `add()` is overloaded. The `add(void)` function does not require any argument but returns integer. The `float add()` function requires three arguments of `float` type. Here, the arguments are `fa`, `fb`, `fc`, `fd`, and `id` and are declared as global so that any function can call them. After entering three float values, the `add(void)` function is called. The `add(void)` function calls the `float` type `add()`, the `float` type `add()` function calculates addition and returns results to function `add(void)`. The `add(void)` function then returns addition as an integer. In the second call, the compiler calls the `float add()` function; this function returns addition of float values. The output of the program is shown above.

## 7.13 PRECAUTIONS WITH FUNCTION OVERLOADING

Function overloading is a powerful feature of C++. But, this facility should not be overused. Otherwise it becomes an additional overhead in terms of readability and maintenance. Following precautions must be taken:

- (1) Only those functions that basically do the same task, on different sets of data, should be overloaded. The overloading of function with identical name but for different purposes should be avoided.
- (2) In function overloading, more than one function has to be actually defined and each of these occupy memory.

- (3) Instead of function overloading, using default arguments may make more sense and fewer overheads.
- (4) Declare function prototypes before `main()` and pass variables instead of passing constant directly. This will avoid ambiguity that frequently occurs while overloading functions.

## 7.14 RECURSION

In programming there might be a situation where a function needs to invoke itself. The C++ language supports recursive feature, that is a function calls itself repetitively. In recursion a function calls itself and the control goes to the same function and it executes repeatedly until some condition is satisfied. In this type of recursive calls a function starts with a new value every time.

### 7.14.1 Rules for Recursive Function

- (1) In recursion, it is essential to call a function by itself; otherwise recursion would not take place.
- (2) Only the user-defined function can be involved in the recursion. Library function cannot involve in recursion because their source code cannot be viewed.
- (3) A recursive function can be invoked by itself or by other function. It saves return address with the intention to return at proper location when return to a calling statement is made.

The last-in-first-out nature of recursion indicates that stack data structure can be used to implement it.

- (4) Recursion is turning out to be increasingly important in non-numeric applications and symbolic manipulations.
- (5) To stop the recursive function, it is necessary to base the recursion on test condition, and proper terminating statement such as `exit()` or `return()` must be written using the `if()` statement.

#### 7.24 Program to find the factorial of a entered number.

```
#include<conio.h>
#include<iostream.h>

int main()
{
 unsigned long int fact(int);
 int f,x;
 clrscr();
 cout<<"\nEnter a Number:";
 cin>>x;
 f=fact(x);
 cout<<"\nFactorial of " <<x<<" is "<<f;
 return 0;
}

unsigned long int fact(int a)
{
 unsigned long factorial;
```

```
 if(a==1)
 return 1;
 else
 factorial=a*fact(a-1);
 return factorial;
}
```

## OUTPUT

**Enter a Number:6**  
**Factorial of 6 is 720**

### 7.25 Program to calculate the gcd of two numbers.

```
#include<conio.h>
#include<iostream.h>
int gcd(int,int);

int main()
{
 clrscr();
 int a,b,z;
 cout<<"\nEnter two numbers:";
 cin>>a>>b;
 z=gcd(a,b);
 cout<<z;
 return 0;
}

int gcd(int a,int b)
{
 if(a!=b)
 {
 if(a>b)
 gcd(a-b,b);
 else
 gcd(a,b-a);
 }
 else
 return(a);
}
```

{coderipe}

## OUTPUT

**Enter two numbers:45 60**  
**GCD of 45 and 60 is 15**

## 7.15 LIBRARY FUNCTIONS

### 7.15.1 Ceil, ceil, floor, floorl

The functions `ceil` and `ceil` round up the given float number, whereas the functions `floor` and `floorl` round down the float number. They are defined in `math.h` header file. Their declarations are as given below.

| Declaration                                                                                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------|
| <pre>Double ceil(double n); Double floor(double n); Long double ceil(long double (n)); Long double floorl(long double (n));</pre> |

The following program illustrates the working of these functions.

#### 7.26 Write a program to round down and up the given float number.

```
#include<math.h>
#include<iostream.h>
#include<conio.h>

int main(void)
{
 clrscr();
 float num = 3.12;
 float d, u;
 d = floor(num);
 u = ceil(num);
 cout<<"\n Original number is: "<<num;
 cout<<"\n Number rounded up is: "<<u;
 cout<<"\n Number rounded down is: "<<d;
 return 0;
}
```

#### OUTPUT

**Original number is : 3.12**

**Number rounded up is : 4**

**Number rounded down is : 3**

**Explanation:** In the above program, the float variable `num` has a value `3.12`. The `floor()` function converts it to the nearest and small integer number than variable `num` and returns it to variable `d`, whereas the `ceil()` function converts the float number to the nearest and greater number than `num`. The output of the program is shown above.



### 7.15.2 modf and modfl

The function `modf` breaks double into integer and fraction elements, and the function `modfl` breaks long double into integer and fraction elements. These functions return the fractional elements of a given number. They are declared as given below.

|                                                                                                      |
|------------------------------------------------------------------------------------------------------|
| Declaration                                                                                          |
| <pre>double modf(double n, double *ip); long double modfl(long double (n), long double *(ip));</pre> |

#### 7.27 Write a program to separate double number into integer and fractional parts.

```
#include<math.h>
#include<conio.h>
#include<iostream.h>

int main()
{
 clrscr();
 double f, i;
 double num = 211.57;
 f = modf(num, &i);
 cout<<"\n The Complete Number: "<<num;
 cout<<"\n The Integer elements: "<<i;
 cout<<"\n Fractional Elements: "<<f;
 return 0;
}
```

#### OUTPUT

**The Complete Number : 211.57**

**The Integer elements : 211**

**Fractional Elements : 0.57**

**Explanation:** In the above program, the `modf()` function separates a double number into separate integer and fractional parts. The second argument of the function is passed by reference that after execution holds the integer part of the number.

### 7.15.3 abs, fabs, and labs

The function `abs()` returns the absolute value of an integer. The `fabs()` returns the absolute value of a floating-point number, and `labs()` returns the absolute value of a long number.

|                                                                                           |
|-------------------------------------------------------------------------------------------|
| <pre>Declaration: int abs(int n); double fabs(double n); long int labs(long int n);</pre> |
|-------------------------------------------------------------------------------------------|

**abs :** If **abs** is invoked using *STDLIB.H* , it is executed as a macro that expands to inline code. If we need to use the original **abs** function then undefine the macro using statement **#undef abs** in the program, after the **#include<stdlib.h>**.

The following program illustrates the working of these functions.

### 7.28 Write a program to display the absolute number of an integer, float, and long integer.

```
#include<iostream.h>
#include<math.h>
#include<conio.h>

int main()
{
 clrscr();
 int num = -1874;
 cout<<"\n Number: "<<num <<"\t\t Absolute value: "
 <<abs (num) ;

 {
 float num = -111.11;
 cout<<"\n Number: "<<num <<"\t Absolute value: "<<fabs (num) ;
 }
 long ans;
 long n = -12252111L;
 ans= labs (n) ;
 cout<<"\n Number: "<<n <<"\t Absolute value: "<<ans;
 return 0;
}
```

### OUTPUT

```
Number : -1874 Absolute value : 1874
Number : -111.110001 Absolute value : 111.110001
Number : -12252111 Absolute value : 12252111
```

**Explanation:** In the above program, the function **abs ( )** returns the absolute value of a negative number **-1874** . The **fabs ( )** function returns the absolute value of a floating-point number. The **labs ( )** function returns the absolute value of long number. In the statement **long n = -12252111L**, the **L** tells the compiler that the number is of long data type and provides storage equal to and as required for long data type.

### 7.15.4 norm

This function is defined in **complex.h** header file and it is used to calculate the square of the absolute value.

Syntax: double norm(complex n) ;

### 7.29 Write a program to calculate the square of the complex number using norm() function.

```
#include<iostream.h>
#include<complex.h>
#include<conio.h>

int main()
{
 clrscr();
 double x=-12.5;
 cout<<norm(x);
 return 0;
}
```

#### OUTPUT

6.25

**Explanation:** In the above program the norm() function calculates the square of complex negative number.

### 7.15.5 complex(), real(), imag(), and conj()

**complex():** This function is defined in complex.h header file and it creates complex numbers from given real and imaginary parts. The imaginary part is supposed to be 0 if not given. Complex is the constructor for C++ class complex. This function returns the complex number with the given real and imaginary parts.

**real():** Returns real part of the complex number.

**imag():** Returns the imaginary part of the complex number.

**conj():** Returns complex conjugate of a complex number.

```
Syntax: complex complex(double real, double imag);
Syntax: double real(complex x);
Syntax: double imag(complex x);
Syntax: double conj(complex x);
```

### 7.30 Write a program to return real, imaginary part and conjugate number.

```
#include<conio.h>
#include<iostream.h>
#include<complex.h>

int main(void)
{
 double a = 4.5, b = 7.4;
 clrscr();
```

```

 complex c = complex(a,b);
 cout<< "c = "<< c << "\n";
 cout<< " and imaginary real part = " << imag(c) << "\n";
 cout<< "c has complex conjugate = " << conj(c) << " \n";
 return 0;
}

```

## OUTPUT

**C = (4.5, 7.4)**  
**and imaginary real part = 7.4**  
**c has complex conjugate = (4.5, -7.4)**

**Explanation:** In the above program variable a and b of double data type are declared and initialized with 4.5 and 7.4. By applying the `complex()`, `imag()` and `conj()` functions results are obtained. The results are shown in the output.

## 7.16 MORE PROGRAMS

**7.31 Write a program to convert a float value to integer and char. Invoke overloaded function `show()` as per the data type.**

```

#include<iostream.h>
#include<conio.h>
void show (int);
void show (float);
void show (char);

int main()
{
 clrscr();
 float x;
 int i;
 char c;
 for (x=65;x<=70;x+=0.5)
 {
 show(x);
 i=(int)x;
 show(i);
 c=(char)x;
 show(c);
 cout<<endl;
 }
 return 0;
}

```

```
void show (int j) {cout<<"\tint = "<<j <<"\t";}
void show (float k) {cout<<'\\t'<<" float = " <<k;}
void show (char c) {cout<<'\\t' <<" char = " <<c;}
```

## OUTPUT

```
float = 65 int = 65 char = A
float = 65.5 int = 65 char = A
float = 66 int = 66 char = B
float = 66.5 int = 66 char = B
float = 67 int = 67 char = C
float = 67.5 int = 67 char = C
float = 68 int = 68 char = D
float = 68.5 int = 68 char = D
float = 69 int = 69 char = E
float = 69.5 int = 69 char = E
float = 70 int = 70 char = F
```

**Explanation:** In the above program, the function `show()` is overloaded for `float`, `int`, and `char` data types, respectively. In `for` loop, conversion from `float` to `int` and `float` to `char` data type is carried out and converted values are assigned to variables `i` and `c`, respectively. After each conversion `show()` function is invoked. The compiler invokes the appropriate function that exactly matches the data type argument. Thus, output displays `float`, `int`, and corresponding *ASCII* character.

### 7.32 Write a program to use function as default arguments.

```
#include<iostream.h>
#include<conio.h>

int main()
{
 clrscr();
 int in();
 int sum (int a,int b=in(),int c=in(),
 int d=in()); // Function prototype

 int a=2;
 int b=3;
 int c=4;
 int d=5;
 cout<<"Sum=" <<sum(a,b,c,d) ;
 cout<<"\nSum=" <<sum(a,b,c) ;
 cout<<"\nSum=" <<sum(a,b) ;
 cout<<"\nSum=" <<sum(a) ;
 return 0;
}
```

```

sum (int j, int k, int l, int m)
{
 return (j+k+l+m);
}
in()
{
 static int j=1;
 int k;
 cout<<"\n (call "<<j++<<") Enter integer number: ";
 cin>> k;
 return (k);
}

```

## OUTPUT

```

Sum=14
(call 1) Enter integer number : 2
Sum=11
(call 2) Enter integer number : 3
(call 3) Enter integer number : 4
Sum=12
(call 4) Enter integer number : 5
(call 5) Enter integer number : 4
(call 6) Enter integer number : 5
Sum=16

```

**Explanation:** In the above program, the function `in()` is used to read an integer value through the keyboard. The function `in()` is used in the prototype of function `sum()` as a default argument. Thus, when `sum()` function is called with less arguments, the function `in()` is executed and user can enter a value to pass in the `sum()` function. The execution of `in()` function depends upon the number of arguments passed to the function `sum()`. In this program, the function `sum()` is invoked with decreasing arguments from 4 to 1.

First time the function `sum()` is called with four arguments. Hence, there is no need of default argument and function `in()` will not be invoked. In second call, three arguments are passed, that is with one less argument. This time `sum()` needs one default argument and `in()` function supplies this value. Thus, in the above successive calls the function `in()` is called based on passed arguments.

### 7.33 Write a program to use global variable as default argument.

```

#include<iostream.h>
#include<conio.h>
int v=2;

int main()

```

```

{
 clrscr();
 int show (int d=v+5);
 int a=2;
 cout<<"\n a++ =" <<show(a);
 cout<<"\n a++ =" <<show();
 v++;
 cout<<"\n a++ =" <<show();
 return 0;
}
show (int j)
{
 j++;
 return (j);
}

```

### OUTPUT

```

a++ =3
a++ =8
a++ =9

```

**Explanation:** In the above program, function `show()` is used to increase the value passed to it. The function `show()` returns increased value. The prototype of function `show()` contains default arguments `(v + 5)`, that is value of variable `v` is added to 5 and the result is used as a default argument for the function `show()`. In the first call, the function `show()` is called with the single argument and it returns value 3. In second call, `show()` is called without argument. This time function `show()` takes default argument, that is `v + 5 = 7` and returns the result 8. The third time the function `show()` is again called without argument. This time it returns the result 9 because the global variable `v` is incremented by 1. Thus, the use of global variable as a default argument changes the default value in different calls. The use of local variable as a default argument is not allowed. If such attempt is made, the compiler shows an error message “*Default expression may not use local variables.*”

### 7.34 Write a program to use function as an argument for another function.

```

#include<iostream.h>
#include<conio.h>
int v=2;

int main()
{
 clrscr();
 int show (int d=v+5);
 cout<<"\n ++ =" <<show(show());

```

```
 return 0;
 }
 show (int j)
 {
 j++;
 return (j);
 }
}
```

## OUTPUT

**++=9**

**Explanation:** In the above program the function `show()` is used as an argument. When function is used as an argument, the argument function is called first and its return value is used as an argument for the first function. In the first call the function uses the default value 7 and it increments by one. The return value 8 is again passed to the function `show()`. The return value again increases by one and finally the `show()` function returns the value 9.

### 7.35 Write a program to overload a function `display()`. Print the integer, double and string using overloaded `show()` function.

```
#include<conio.h>
#include<iostream.h>
void display (int d) { cout<<endl<<" d= "<<d; }
void display (double d) { cout<<endl<<" d= "<<d; }
void display (char *d) { cout<<endl<<" d= "<<d; }

int main()
{
 clrscr();
 display (50);
 display(3.14521);
 display("Hello");
 return 0;
}
```

## OUTPUT

**d= 50**

**d= 3.14521**

**d= Hello**

**Explanation:** In the above program the function `display()` is overloaded for displaying integer, double and string. In function `main()`, integer, double and string are passed to function `display()`. The compiler executed appropriate function `display()` and displays the passed value on the screen.



**7.36 Write a program to declare a function with default arguments. Execute the function without argument.**

```
#include<conio.h>
#include<iostream.h>
void display (char *s="Hearty Welcome\n")
{
 cout<<endl<<s;
}
int main()
{
 clrscr();
 display("Hello");
 display();
 return 0;
}
```

**OUTPUT**

**Hello**  
**Hearty Welcome**

**Explanation:** In the above program character pointer s is initialized with string “ Hearty Welcome” and it is declared as default arguments for function display(). The default argument is used when the function is invoked without argument. Here, in the first call a string “Hello” is passed and it is displayed on the screen by the function display(). In second call, the function display() is called without argument. In this call the default argument is used and the string displayed will be “ Hearty Welcome”.

**7.37 Write a program to draw line. Create line function with default arguments.**

```
#include<conio.h>
#include<iostream.h>
int main()
{
 void line (char = '\-', int = 50);
 clrscr();
 line(); // without arguments
 line ('\+', 50);
 line ('\/', 50);
 line ('\S', 50);
 line(); // without arguments
 return 0;
}
```



```
 cout<<"\nDivision of given numbers is "<<e;
 cout<<"\nModulus of given numbers is "<<f;
 return(0);
}
float add (float x,float y)
{
 return(x+y);
}
float sub(float x,float y)
{
 return(x-y);
}
float mul (float x,float y)
{
 return(x*y);
}
float div(float x,float y)
{
 return(x/y);
}
int mod(int p,int q)
{
 return(p%q);
}
```

## OUTPUT

**Enter numbers 5 2**

**Addition of given numbers is 7**

**Subtraction of given numbers is 3**

**Multiplication of given numbers is 10**

**Division of given numbers is 2.5**

**Modulus of given numbers is 1**

### 7.39 Program to display function show() with default parameters

```
#include<iostream.h>
#include<conio.h>
void show(int=1,float=7.5,long=9);
void main()
{
 clrscr();
 show();
}
```

```

 show(5);
 show(3,2.4);
 show(1,7.7,31);
}
void show(int first,float second,long third)
{
 cout<<"\nFirst = "<<first;
 cout<<" , second = "<<second;
 cout<<" , third = "<<third;
}

```

## OUTPUT

```

First = 1,second = 7.5,third = 9
First = 5,second = 7.5,third = 9
First = 3, second = 2.4, third = 9
First = 1, second = 7.7, third = 3

```

### 7.40 Program to find addition and multiplication of digits of any number.

```

#include<iostream.h>
#include<conio.h>

int main()
{
 int n,p,q;
 int sum(int);
 int mul(int);
 clrscr();
 cout<<"Enter any number ";
 cin>>n;
 p= sum(n);
 q= mul(n);
 cout<<"Sum of digits "<<p;
 cout<<"\nMultiplication of digits "<<q;
 return 0;
}

int sum(int s)
{
 int a,sum=0,rem;
 for(a=s;a>0;a/=10)
 {
 rem=a%10;
 sum=sum+rem;
 }
}

```

```
 }
 return(sum);
}
int mul(int m)
{
 int e,res=1,rem;
 for(e=m;e>0;e/=10)
 {
 rem=e%10;
 res*=rem;
 }
 return (res);
}
```

## OUTPUT

**Enter any number 123**

**Sum of digits 6**

**Multiplication of digits 6**

### 7.41 Program to convert binary number to decimal.

```
#include<iostream.h>
#include<conio.h>

int main()
{
 int b;
 clrscr();
 cout<<"Enter any number in binary:";
 cin>>b;
 void decimal(int);
 decimal(b);
 return 0;
}

void decimal(int n)
{
 int k,rem,d,dec=0,a=1;
 for(k=n;k>0;k/=10)
 {
 rem=k%10;
 d=rem*a;
 dec+=d;
 a*=2;
 }
}
```



```
 cout<<" Binary number= " <<n<<"\nDecimal number= "<<dec;
}
```

### OUTPUT

**Enter any number in binary:1111**  
**Binary number= 1111**  
**Decimal number= 15**

### 7.42 Program to overload function addition of integers and floats.

```
#include<iostream.h>
#include<conio.h>
int add (int,int);
float add(float,float,float);

int main()
{
 int a,b,i;
 float p,q,r,f;
 clrscr();
 cout<<"Enter integers: a&b ";
 cin>>a>>b;
 cout<<"Enter floats: p,q & r ";
 cin>>p>>q>>r;
 i=add(a,b);
 f=add(p,q,r);
 cout<<"\nAddition of integers= "<<i;
 cout<<"\nAddition of floats= "<<f;
 return 0;
}

int add(int x,int y)
{
 return(x+y);
}

float add(float m,float n,float o)
{
 return(m+n+o);
}
```

### OUTPUT

**Enter integers : a&b 2 7**  
**Enter floats: p,q & r 2.4 5.6 7.6**  
**Addition of integers= 9**  
**Addition of floats= 15.6**

## SUMMARY

- (1) Like “C” language, the programs written in “C++” language highly depend on functions. The “C++” program is nothing but a combination of one or more functions.
- (2) A function prototype declaration consists of function’s return type, name, and arguments list. When the programmer defines that function, the definition must be the same as its prototype declaration.
- (3) In C++ it is possible to pass arguments by value or by reference. C++ reference type is declared with & operator and it is nearly identical but not exactly the same as pointer types. They declare aliases for objects variables and allow the programmer to pass arguments by reference to functions.
- (4) An *lvalue* is an object locator and an expression that indicates an object.
- (5) The statement  $x + y$  is not an *lvalue*;  $x + y = z$  is invalid because the expression on the left is not related to a variable. Such expressions are often called *rvalues*.
- (6) C++ allows the programmer to assign default values in the function prototype declaration of the function. When the function is called with lesser parameters, then the default values are used for the operations.
- (7) The constant variable can be declared using `const` keyword. The `const` keyword makes variable value stable. The constant variable should be initialized at the time of declaration.
- (8) C++ provides a mechanism called *inline* function. When a function is declared as *inline*, the compiler copies the code of the function in the calling function.
- (9) C++ makes it possible for the programmer to use the same function name for various times for different intentions. This is called function overloading or function polymorphism.
- (10) The function `modf()` breaks double into integer and fraction elements, and the function `modfl()` breaks long double into integer and fraction elements.
- (11) The functions `ceil()` and `ceilf()` round up the given float number, whereas the functions `floor()` and `floorf()` round down the float number.
- (12) The function `abs()` returns the absolute value of an integer. The `fabs()` returns the absolute value of a floating-point number and `labs()` returns the absolute value of a long number.

## EXERCISES

### (A) Answer the following questions

- (1) What are the differences between C and C++ functions?
- (2) Describe different parts of function.
- (3) What are void functions?
- (4) What does it mean by function prototype? Is it compulsory?
- (5) When is the function prototype declaration not necessary?
- (6) What are default arguments?
- (7) Where are the default arguments assigned?
- (8) How are the default arguments entered at run time?
- (9) What are inline functions? Discuss its advantages and disadvantages.
- (10) What are the *rvalue* and *lvalue* in an expression? Explain with examples.
- (11) What is the difference between call by value and call by reference? Illustrate them with examples.
- (12) What are constant arguments?

- (13) How is the value of a constant variable changed?
- (14) What is function overloading?
- (15) What are the rules for defining overloaded functions?
- (16) What precautions should we take while overloading function?
- (17) What is the difference between pointer and reference variable?
- (18) What is the difference between normal function and inline function?
- (19) What are actual and formal arguments?
- (20) How can a return statement pass more than a single value from a function?
- (21) What is recursion?
- (22) Explain the rules of recursion?

**(B) Answer the following by selecting the appropriate option**

- (1) The concept of declaring same function name with multiple definitions is
  - (a) **function overloading**
  - (b) operating overloading
  - (c) both (a) and (b)
  - (d) none of the above
- (2) The default arguments are used when
  - (a) **function is called with less arguments**
  - (b) function is void
  - (c) when arguments are passed by reference
  - (d) none of the above
- (3) The constant function
  - (a) **cannot alter values of a variable**
  - (b) can alter values of a constant variable
  - (c) makes its local variable constant
  - (d) none of the above
- (4) The function `abs()` returns
  - (a) **absolute value**
  - (b) negative value
  - (c) both (a) and (b)
  - (d) none of the above
- (5) The use of parentheses is optional for one of the following statement:
  - (a) `return`
  - (b) `main`
  - (c) **`clrscr`**
  - (d) `exit`
- (6) The following program will generate an error message
 

```
void main()
{
 return 0;
}
```

  - (a) **main() cannot return value**
  - (b) void keyword is not allowed in `main()`
  - (c) function should return a non-zero value
  - (d) return statement is not allowed
- (7) Which of the following statements are true?
  - (1) A return type for void specifies that no value be returned
  - (2) Functions by default return `int` value
  - (3) The return type can only be `int`, `char`, or `double`
  - (a) **(1) and (2)**
  - (b) (1), (2), and (3)
  - (c) (1) and (3)
  - (d) none of the above
- (8) C++ provides inline functions to reduce function call overhead, mainly for
  - (a) **small functions**
  - (b) large functions
  - (c) member function
  - (d) none of the above
- (9) To \_\_\_\_ an inline function, we must change it to an outline function.
  - (a) debug
  - (b) edit
  - (c) **remove**
  - (d) comment out
- (10) Identify which of the following function calls are allowed. The prototype declaration is `int sum(int j, int k = 4, int l = 3)`
  - (1) `sum(2)`
  - (2) `sum(2, 3)`
  - (3) `sum()`
  - (4) `sum(3, 4, 5)`
  - (a) **1, 2, and 4**



- (b) 2, 3, and 4
  - (c) 1 and 5
  - (d) 2 and 4
- (11) It is possible to overload functions
- (a) when they have similar name and return type
  - (b) when they have similar name, return type, and different arguments
  - (c) when they have similar name, different argument type, and different return type
  - (d) **when they have similar name, different type of argument, or different number of arguments and return type does not matter**
- (12) What will be the output of the following program?
- ```
int sub(int q)
{
    int m;
    m = sub(q - 1);
    return(m);
}
void main()
{
    int r = sub(7);
    cout<<r;
}
```
- (a) 0
 - (b) **stack overflow**
 - (c) compilation error
 - (d) -5
- (13) Compiler may reject the request for inline function if
- (a) it is small
 - (b) the return type is void
 - (c) **it is a loop, switch, or goto exists**
 - (d) never
- (14) The default return type of main function is
- (a) **int**
 - (b) double
 - (c) long int
 - (d) void
- (15) Advantage of inline function is
- (a) **faster execution**
 - (b) saves memory
 - (c) both (a) and (b)
 - (d) none
- (16) Default arguments are defined when functions are
- (a) defined
 - (b) **declared**
 - (c) both (a) and (b)
 - (d) never

(C) Attempt the following programs

- (1) Write a program to define a function with default argument. Whenever the function needs default values of arguments, it should prompt the user to enter a default value. Also display the default values.
- (2) Write a program to define a constant variable. Change the value of the variable using pointer. Display the default and the newly assigned values along with memory locations.
- (3) Write a program to calculate the power of a given number. Define user-defined function `power()`. Make it inline.
- (4) Write a program to accept a double number. Separate its integer and fractional part.
- (5) Write a program to calculate the absolute value of long and float number.
- (6) Write a program to round down and round up the floating-point number.
- (7) Write a program to overload the function `uabs()`. The function should return absolute value of the given number for data type `int` and `float`.
- (8) Write a program to enter quantity and rate through the keyboard. Then calculate the amount by multiplying quantity and rate. If the fractional part of the amount is greater than or equal to **.50** then round up the number, otherwise round down the number. Enter a minimum of 10 records. Calculate the total fractional parts of the

- amount rounded up and rounded down separately.
- (9) Write a program to display only the integer portion of the given `float` numbers without type casting.
 - (10) Write a program to accept a `float` number through the keyboard. Then calculate the square of the number. Separate the `float` number into integer and fractional part. Individually calculate the square of an integer and fractional parts and add them in another variable. Compare the two squares obtained. Write your observation regarding the squares obtained. (*Note: set precision to 2*).
 - (11) Write a program to calculate the square root of 1 to 10 numbers. Display the sum of integer parts and fractional parts of the square roots obtained. (*Note: set precision to 2*.)
 - (12) Write a program using function overload to convert an integer number to an ASCII character and `float` to ASCII string.
 - (13) Write an inline function to display lines of different patterns.
 - (14) Write a program to display the Fibonacci series up to 21 using recursion.
 - (15) Write a program to display the sum of first 100 odd numbers using recursion.
 - (16) Write a program to find the power of integer number using `pow()` library function.
 - (17) Write a program to return more than one values from function using call by reference method.

This page is intentionally left blank.

Classes and Objects

8

CHAPTER

CHAPTER OUTLINE

- 8.1 Introduction
- 8.2 Structure in C
- 8.3 Structure in C++
- 8.4 Classes in C++
- 8.5 Declaring Objects
- 8.6 The `public` Keyword
- 8.7 The `private` Keyword
- 8.8 The `protected` Keyword
- 8.9 Access Specifiers and Their Scope
- 8.10 Defining Member Functions
- 8.11 Characteristics of Member Functions
- 8.12 Outside Member Function as Inline
- 8.13 Rules for Inline Functions
- 8.14 Data Hiding or Encapsulation
- 8.15 Classes, Objects, and Memory
- 8.16 `static` Member Variables
- 8.17 `static` Member Functions
- 8.18 `static` Object
- 8.19 Array of Objects
- 8.20 Objects as Function Arguments
- 8.21 `friend` Functions
- 8.22 The `const` Member Functions
- 8.23 The Volatile Member Function
- 8.24 Recursive Member Function
- 8.25 Local Classes
- 8.26 `empty`, `static`, and `const` Classes

- 8.27 Member Function and Non-member Function
- 8.28 The `main()` Function as a Member Function
- 8.29 Overloading Member Functions
- 8.30 Overloading `main()` Functions
- 8.31 The `main()`, Member Function, and Indirect Recursion
- 8.32 Bit Fields and Classes
- 8.33 Nested `class`
- 8.34 More Programs

8.1 INTRODUCTION

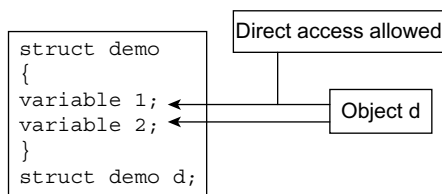


Fig. 8.1 Structures in C

In C, structures are used to define custom or user-defined data type. Structure is a combination of same or different data types. Only variables are declared inside a structure. The initialization of member variables inside the structure is not permitted. Objects can directly access the data members of the structures.

Functions are not permitted as members in structure. Outside functions are also able to access the data members of the structure through the object. Thus, there is no security to data members declared inside

the structure in C as shown in Figure 8.1.

In C++, structure also combines function with data members. C++ introduces new keyword `class`. A `class` in C++ is similar to structure. Using `class` or structure, a programmer can merge one or more dissimilar data types and a new custom data type can be created. A `class` is nothing but grouping of variables of different data types with functions. Each variable of a `class` is called as member variable. Functions are called as member functions or methods.

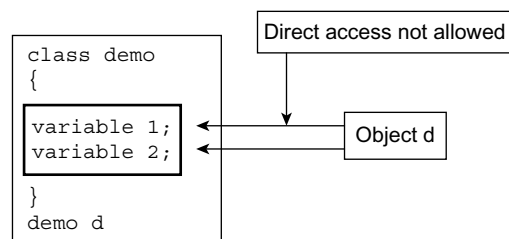


Fig. 8.2 class in C++

In C++, it is possible to access data members directly by objects, which is not possible in C. It is always the programmer's choice to allow or disallow the direct access of data members. The mechanism of restricting access of data outside the `class` is called as data hiding or encapsulation. In such a case, only the member functions can access the data. If `class` is used in place of `struct`, it restricts the access of data members as shown in Figure 8.2.

8.2 STRUCTURE IN C

In C, it is possible to combine dissimilar data types using structure but it has the following limitations:

- a) Functions are not allowed as members of structure.
- b) Direct access to data members is possible. So, by default all the structure members are public. Hence, security to data or data hiding is not provided.
- c) The `struct` data type is not treated as built-in type; that is the use of `struct` keyword is necessary to declare objects.
- d) The data members cannot be initialized inside the structure.

The syntax of structure declaration is as follows:

```
syntax:
struct <struct name>
{
    Variable1;
    Variable2;
};
```

```
Example:
struct item
{
    int codeno;
    float prize;
    int qty;
};
```

In the above example, `item` is a structure name. The data members are `codeno`, `prize`, and `qty`. Thus, a custom data type is created by combination of one or more data members.

The object of structure `item` can be declared as follows:

```
struct item a, *b
```

The object declaration is same as the declaration of variables of built-in data types. The object `a` and the pointer `*b` are variables of type `item` and each of them has three data members. The `a` and `*b` can access the data members of `struct item`. Use of keyword `struct` is necessary.

Access to structure members: The data members of a structure are accessed by using object name and operators such as dot (`.`) or arrow (`->`). The dot (`.`) or arrow (`->`) operators are known as referencing operators. The dot operator is used when simple object is declared and arrow operator is used when object is pointer to structure. The access of members can be accomplished as given in the following syntax:

[Object name][Operator][Member variable name]

When an object is a simple variable, access of members is done as follows:

Object name dot (`.`) member variable name

```
a.codeno
a.price
a.qty
```

When an object is a pointer to structure then members are accessed as follows:

```
a->codeno  
a->price  
a->qty
```

Note: A structure declaration must have a name or identifier followed by keyword `struct`, else the compiler will display an error. This is because it will not recognize the type of object declared. Hence, there is no possibility of accessing the structure member variables.

The following program illustrates the above discussion.

8.1 Write a program to access data members of a structure.

```
#include<stdio.h>  
#include<conio.h>  
struct item // struct declaration  
{  
    int codeno; // codeno=200 not possible as per 8.2 section  
    float prize;  
    int qty;  
};  
void main()  
{  
    struct item a,*b; // object declaration  
    clrscr();  
    a.codeno=123; // direct access & initialization of member variables  
    a.prize=150.75;  
    a.qty= 150;  
    printf ("\n With simple variable");  
    printf ("\n Codeno : %d ",a.codeno);  
    printf ("\n Prize : %d",a.prize);  
    printf ("\n Qty : %d",a.qty);  
    b->codeno=124; // direct access & initialization of member vari-  
                  ables  
    b->prize=200.75;  
    b->qty= 75;  
    printf ("\n\n With pointer variable");  
    printf ("\n Codeno : %d ",b->codeno);  
    printf ("\n Prize : %d",b->prize);  
    printf ("\n Qty : %d",b->qty);  
}
```

OUTPUT

With simple variable

Codeno : 123

Prize : 150.75

Qty : 150

With pointer to structure**Codeno : 124****Prize : 200.75****Qty : 75**

Explanation: The above program is compiled with C compiler. The following discussion is according to C compiler. In the above program, the structure item is declared with three member variables. The initialization of member variables inside the struct is not permitted. The declaration of member variables is enclosed within the curly braces. The struct declaration is terminated with a semicolon.

In function `main()`, the objects `a` and `b` are declared. Consider the following statement.

```
struct item a, *b; // object declaration
```

The struct must be preceded by structure name. The member variables can be accessed and initialization is done directly by object. The dot and arrow operators are used to access the member variables.

8.3 STRUCTURE IN C++

No doubt, C++ has made various improvements in structure. To know the improvements made in C++, the last program is compiled and executed with C++ compiler. The explanation followed by this program discusses the various improvements.

8.2 Write a program to declare struct. Initialize and display contents of data members.

```
#include<iostream.h>
#include<conio.h>
struct item // struct declaration
{
    int codeno; // codeno=200 not possible
    float prize;
    int qty;
};
int main()
{
    item a, *b; // object declaration
    clrscr();
    a.codeno=123; //direct access & initialization of member variables
    a.prize=150.75;
    a.qty= 150;
    cout<<"\n With simple variable";
    cout<<"\n Codeno : "<<a.codeno;
    cout<<"\n Prize : "<<a.prize;
    cout<<"\n Qty : "<<a.qty;
    b->codeno=124; // direct access & initialization of member variables
    b->prize=200.75;
```



```

b->qty= 75;
cout<<"\n\n With pointer to structure";
cout<<"\n Codeno : "<<b->codeno;
cout<<"\n Prize : "<<b->prize;
cout<<"\n Qty : "<<b->qty;
return 0;
}

```

Explanation: The above program is the same as previous one. The output is also same, hence not shown. Consider the following statement.

```
item a, *b; // object declaration in c++
```

While declaring an object, the keyword `struct` is omitted, which is compulsory in C. The structure item is a user-defined data type. C++ itself behaves as a structure data type which is built-in type and allows variable declaration.

C++ introduces new keyword `class`, which is similar to structure. The other improvements are discussed with the use of `class` in the following section.

8.4 CLASSES IN C++

A `class` is used to pack data members and member function together. The `class` has a mechanism to prevent direct access to its members, which is the central idea of object-oriented programming. The whole declaration of `class` is given in Table 8.1. The `class` declaration is also known as formation of new abstract data type. The abstract data type can be used as basic data type such as `int`, `float`, etc.

Table 8.1 Syntax and an Example of `class`

Syntax of <code>class</code> Declaration	Example of <code>class</code>
<pre> class <name of class> { private: declaration of variables; prototype declaration of function; public: declaration of variables; prototype declaration of function; }; </pre>	<pre> class item // class declaration { private: int codeno; float prize; int qty; void values(); public: void show(); }; </pre>

The `class` is a keyword. The `class` declaration is same as `struct` declaration. The declaration of a `class` is enclosed with curly braces and terminated with a semicolon. The data members and member functions can be declared in two sections, that is `private` and `public`. The `private` and `public` keywords are terminated with a colon (:). The object cannot directly access the data members and member functions declared in `private` section, but it can access the data member and member functions declared in `public` section. The `private` members of a `class` can only be accessed by a `public` member function of the same `class`. Different sections of `class` are illustrated with examples in the following sections.

It is also possible to access `private` member variables directly like `public` member variables provided that the `class` should have at least one `public` member variable. Both the `private` and `public` member variables are stored in consecutive memory locations in the memory. A pointer

to member variable provides address of member variable. By applying increment (++) and decrement (--) operations on pointer, we can access all private and public member variables of the class. The object of a class contains address of the first member variable. It can be also used to access the private or public data.

8.5 DECLARING OBJECTS

A class declaration only builds the structure of an object. The member variables and functions are combined in the class. The declaration of objects is same as declaration of variables of basic data types. Defining objects of class data type is known as class *instantiation*. Only when objects are created, memory is allocated to them.

Consider the following examples.

```
a) int x,y,z; // Declaration of integer variables
b) char a,b,c; // Declaration of character variables
c) item a,b, *c; // Declaration of object or class type variables
```

In the example (a), three variables x, y, and z of int types are declared. In the example (b), three variables a, b, and c of char type are declared. In the same fashion the third example declares the three objects a, b, and c of class item. The object *c is pointer to class item.

An object is an abstract unit with the following properties:

- a) It is individual.
- b) It points to a thing, either physical or logical that is identifiable by the user.
- c) It holds data as well as operation method that handle data.
- d) Its scope is limited to the block in which it is defined.

Access to class members: The object can access the public data member and member functions of a class by using dot (.) and arrow (->) operators. The syntax is as follows:

[Object name] [Operator] [Member name]

To access data members of class the statement would be as follows:

```
a.show();
```

where a is an object and show() is a member function. The dot operator is used because a is a simple object.

In statement

```
c->show();
```

*c is pointer to class item; therefore, the arrow operator is used to access the member.

Consider the given example.

```
class item // class declaration
{
    int codeno;
    float prize;
    int qty;
};
```

We replaced the struct keyword with class. If programs 8.1 and 8.2 are executed with class, they will not work. For example,

```
void main()
{
    item a,*b; // object declaration
    clrscr();
    a.codeno=123; // Direct access is not allowed
    a.prize=150.75;
    a.qty= 150;
}
```

The above program will generate error messages such as “‘item::codeno’ is not accessible”. This is because the object cannot directly access the member variables of a class, which is possible with structure. Hence, we can say that the difference between class and struct is that the member variables of struct can be accessed directly by the object, whereas the member variables of class cannot be accessed directly by the object.

8.6 THE `public` KEYWORD

In Section 8.3, we noticed that the object directly accesses the member variables of structure, whereas the same is not possible with class members. The keyword `public` can be used to allow objects to access the member variables of a class directly like structure. The `public` keyword is written inside the class. It is terminated with a colon (:). The member variables and functions declared followed by the keyword `public` can be accessed directly by the object. The declaration can be done as follows:

```
class item // class declaration
{
    public: //public section begins
    int codeno;
    float prize;
    int qty;
};
```

Consider the following program that illustrates the use of `public` keyword with class.

8.3 Write a program to declare all members of a class as public. Access the elements using object.

```
#include<iostream.h>
#include<conio.h>
class item
{
    public: // public section begins
    int codeno;
    float prize;
    int qty;
}; // end of class
```

```
int main()
{
    clrscr();
    item one; // object declaration
    one.codeno=123; // member initialization
    one.prize=123.45;
    one.qty=150;
    cout<<"\n Codeno = "<<one.codeno;
    cout<<"\n Prize = "<<one.prize;
    cout<<"\n Quantity = "<<one.qty;
    return 0;
}
```

OUTPUT

Codeno = 123

Prize =123.449997

Quantity =150

Explanation: In the above program, the members of class item are declared followed by keyword public. The object one of class item accesses the member variables directly. The member variables are initialized and values are displayed on the screen.

8.7 THE private KEYWORD

The private keyword is used to prevent direct access of member variables or function by the object. The class by default produces this effect.

The structure variables by default are public. To prevent member variables and functions of struct from direct access, the private keyword is used. The syntax of private keyword is same as public. The private keyword is terminated with a colon. Consider the following example.

```
struct item
{
    private: // private section begins
    int codeno;
    float prize;
    int qty;
}; // end of class
int main()
{
    clrscr();
    item one; // object declaration
    one.codeno=123; // member initialization
    one.price=123.45;
    one.qty=150;
}
```

As soon as the above program is compiled, the compiler will display the following error message:

```
'item::codeno' is not accessible
'item::prize' is not accessible
'item::qty' is not accessible
'item::codeno' is not accessible
'item::prize' is not accessible
```

From the above discussion, we noticed that by default (without applying `public` or `private` keyword) the `class` members are `private` (not accessible) whereas the `struct` members are `public` (accessible).

The `private` members are not accessible by the object directly. Then the question is how will they be accessed? To access the `private` members of a `class`, member functions of the same `class` are used. The member function must be declared in the `class` in `public` section. A program on accessing `private` members is given in the forthcoming sections. Through the `public` member function, an object can access the `private` members.

8.8 THE `protected` KEYWORD

The access mechanism of `protected` keyword is same as `private` keyword. We cannot access `protected` section members from outside the `class` by any object.

The following example clears the above concept:

8.4 Write a program using `class` to declare member variable and function under `protected` section and make an attempt to access them using object.

{coderipe}

```
#include<iostream.h>
#include<conio.h>
class A
{
    protected:
    int num;
    void display()
    {
        cout<<num;
    }
};
int main()
{
    A a;
    a.num=100;
    a.display();
    return 0;
}
```

Note: The above program gives an error **`A::num`** and **`A::display()`** are not accessible.

The protected keyword is frequently used in inheritance of classes. Its detailed description is given Chapter 11.

8.9 ACCESS SPECIFIERS AND THEIR SCOPE

As described in Table 8.2, the `class` object can access a public member of the `class` directly without the use of member function. The private and protected mechanisms do not allow an object to access data directly. The object can access private or protected members only through public member functions of the same `class`.

Table 8.2 Access Limits of `class` Members

Access Specifiers	Access Permission	
	class Members	class Object
Public	Allowed	Allowed
Private	Allowed	Disallowed
Protected	Allowed	Disallowed

The following program explains the working of the above keywords:

8.5 Write a program using `class` to declare member variable and functions private, public and protected section and make an attempt to access them using object.

```
#include<iostream.h>
#include<conio.h>
class sample
{
    private:
        int num;
        void show1 ()
        {
            cout<<"Inside the private section";
            cout<<"\nEnter a number:";
            cin>>num;
            cout<<"Number is:"<<num;
        }
    public:
        int num1;
        void show ()
        {
            show1 ();
            cout<<"\n\nInside the public section";
            cout<<"\nEnter a number:";
            cin>>num1;
            cout<<"Number is:"<<num1;
            show2 ();
        }
}
```

```
protected:
    int num2;
    void show2 ()
{
    cout<<"\n\nInside the protected section";
    cout<<"\nEnter a number:";
    cin>>num2;
    cout<<"Number is:"<<num2;
}
};
int main()
{
    sample s;
    s.show();
    return 0;
}
```

OUTPUT

Inside the private section
Enter a number:4
Number is:4
Inside the public section
Enter a number:5
Number is:5
Inside the protected section
Enter a number:6
Number is:6

8.10 DEFINING MEMBER FUNCTIONS

The member function must be declared inside the class. They can be defined as (a) private or public section and (b) inside or outside the class. The member functions defined inside the class are treated as inline function. If the member function is small then it should be defined inside the class. Otherwise, it should be defined outside the class.

If function is defined outside the class, its prototype declaration must be done inside the class. While defining the function, scope access operator and class name should precede the function name. The following programs illustrate all about member function and how to access the private member of the class.

8.10.1 Member Function Inside the class

Member function inside the class can be declared in public or private section. The following program illustrates the use of a member function inside the class in public section.

8.6 Write a program to access private members of a class using member function.

```
#include<iostream.h>
#include<conio.h>
class item
{
    private: // private section starts
        int codeno;
        float price;
        int qty;
    public: // public section starts
        void show() // member function
        {
            codeno=125; // access to private members
            price=195;
            qty=200;
            cout<<"\n Codeno ="<<codeno;
            cout<<"\n Price ="<<price;
            cout<<"\n Quantity="<<qty;
        }
};
int main()
{
    clrscr();
    item one; // object declaration
    one.show(); // call to member function
    return 0;
}
```

OUTPUT**Codeno =125****Price =195****Quantity=200**

Explanation: In the above program, the member function `show()` is defined inside the `class` in public section. In function `main()`, object `one` is declared. We know that an object has a permission to access the public members of the `class`. The object `one` invokes the public member function `show()`. The public member function can access the private members of the same `class`. The function `show()` initializes the private member variables and displays the contents on the console. For the sake of understanding only one function is defined.

In the above program the member function is defined inside the `class` in public section. Now the following program explains how to define private member function inside the `class`.

8.10.2 Private Member Function

In the last section, we learned how to access private data of a `class` using public member function. It is also possible to declare a function in private section like data variables. To execute private member function, it must be invoked by public member function of the same `class`. A member function of a `class` can invoke any other member function of its own `class`. This method of invoking function is known as nesting of member function. When one member function invokes other member function, the frequent method of calling function is not used. The member function can be invoked by its name terminated with a semicolon only like normal function. The following program illustrates this point.

8.7 Write a program to declare private member function and access it using public member function.

```
#include<iostream.h>
#include<conio.h>
struct item
{
    private: // private section starts
        int codeno;
        float price;
        int qty;
        void values() // private member function
        {
            codeno=125;
            price=195;
            qty=200;
        }
        public: // public section starts
        void show() // public member function
        {
            values(); // call to private member functions
            cout<<"\n Codeno ="<<codeno;
            cout<<"\n Price ="<<price;
            cout<<"\n Quantity="<<qty;
        }
};
int main()
{
    clrscr();
    item one; // object declaration

    // one.values(); // not accessible

    one.show(); // call to public member function
```

```
    return 0;
}
```

OUTPUT

Codeno =125

Price =195

Quantity=200

Explanation: In the above program, the private section of a class item contains one-member function `values()`. The function `show()` is defined in public section. In function `main()`, one is an object of class `item`. The object one cannot access the private member function. In order to execute the private member function, the private function must be invoked using public member function. In this example, the public member function `show()` invokes the private member function `values()`. In the invocation of function `values()`, object name and operator are not used.

8.10.3 Member Function Outside the class

In the previous examples, we observed that the member functions are defined inside the `class`. The function prototype is also not declared. The functions defined inside the `class` are considered as inline functions. If a function is small, it should be defined inside the `class` and if large it must be defined outside the `class`. To define a function outside the `class`, following care must be taken.

- a) The prototype of function must be declared inside the `class`.
- b) The function name must be preceded by `class` name and its return type separated by scope access operator.

The following example illustrates the function defined outside the `class`.

8.8 Write a program to define member function of class outside the class.

```
#include<iostream.h>
#include<conio.h>
class item
{
    private: // private section starts
    int codeno; // member data variables
    float price;
    int qty;
    public: // public section starts
    void show (void); // prototype declaration
}; // end of class
void item:: show() // definition outside the class
{
    codeno=101;
    price=2342;
    qty=122;
```

```
    cout<<"\n Codeno ="<<codeno;
    cout<<"\n Price ="<<price;
    cout<<"\n Quantity="<<qty;
}
int main()
{
    clrscr();
    item one; // object declaration
    one.show(); // call to public member function
    return 0;
}
```

OUTPUT

Codeno =101

Price =2342

Quantity=122

Explanation: In the above program, the prototype of function `show()` is declared inside the `class` terminated by `class` definition. The body of function `show()` is defined inside the `class`. The `class` name that it belongs to and its return type precede the function name. The function declarator of function `show()` is as follows:

```
void item:: show()
```

where `void` is return type; that is function is not returning a value. The `item` is a `class` name. Scope access operator separates the `class` name and function name, followed by the body of function that is defined.

8.11 CHARACTERISTICS OF MEMBER FUNCTIONS

- (1) The difference between member and normal function is that the formal function can be invoked freely, whereas the latter function only by using an object of the same `class`.
- (2) The same function can be used in any number of classes. This is possible because the scope of the function is limited to their classes and cannot overlap one another.
- (3) The private data or private function can be accessed by public member function. Other functions have no access permission.
- (4) The member function can invoke one another without using any object or dot operator.

8.12 OUTSIDE MEMBER FUNCTION AS INLINE

In Chapter 7, we learned how inline mechanism is useful for small function. It is good practice to declare function prototype inside the `class` and definition outside the `class`. The inline mechanism reduces overhead relating to access the member function. It provides better efficiency and allows quick execution of functions. An inline member function is similar to macros. Call to inline function in the program, puts the function code in the caller program. This is known as inline expansion. Inline functions are also called as open subroutines because

their code is replaced at the place of function call in the caller function. The normal functions are known as closed subroutines because when such functions are called, the control passes to the function.

By default, all member functions defined inside the `class` are inline function. The member function defined outside the `class` can be made inline by prefixing the keyword `inline` to function declarator as shown in Figure 8.3.

<code>inline</code>	<code>return type</code>	<code>class name</code>	<code>::</code>	<code>function name (signature)</code>
---------------------	--------------------------	-------------------------	-----------------	--

Fig. 8.3 Inline function outside the `class`

The `inline` is a keyword and acts as function qualifier. The `return type` is functions return type; that is the function returns values of this type. The `class name` is the name of `class` that the function belongs to. Scope access operator separates `class name` and function name. The signature means argument list passed function. The following program illustrates inline function outside the `class`.

8.9 Write a program to make an outside function as inline.

```
#include<iostream.h>
#include<conio.h>
class item
{
    private: // private section starts
        int codeno; // member data variables
        float price;
        int qty;
    public: // public section starts
        void show (void); // prototype declaration
}; // end of class
inline void item:: show() // outside inline function
{
    codeno=213;
    price=2022;
    qty=150;
    cout<<"\n Codeno ="<<codeno;
    cout<<"\n Price ="<<price;
    cout<<"\n Quantity="<<qty;
}
int main()
{
    clrscr();
    item one; // object declaration
```

```
one.show(); // call to public member function (inline)
return 0;
}
```

OUTPUT

Codeno =213

Price =2022

Quantity=150

Explanation: The above program is same as last one. The only difference is that the function `show()` is defined as inline outside the `class`. The function declarator is as follows `inline void item::show()`.

8.13 RULES FOR INLINE FUNCTIONS

- (1) Inline function should be used rarely. It should be applied only at appropriate circumstances.
- (2) Inline function can be used when the member function contains few statements. For example,

```
inline int item::square (int x)
{
    return (x*x);
}
```

- (3) If function takes more time to execute, then it must be declared as inline. The following inline function cannot be expanded as inline.

```
inline void item:: show()
{
    cout<<"\n Codeno ="<<codeno;
    cout<<"\n Price ="<<price;
    cout<<"\n Quantity="<<qty;
}
```

The member function that performs input and output operation requires more times. Inline functions have one drawback, the entire code of the function is placed at the point of call in caller function and it must be noted at compile time. Therefore, inline functions cannot be placed in standard library or run time library.

8.14 DATA HIDING OR ENCAPSULATION

Data hiding is also known as encapsulation. It is a procedure of forming objects. An encapsulated object is often called as an abstract data type. We need to encapsulate data, because programmer often makes various mistakes and the data get changed accidentally. Thus, to protect

data, we need to construct a secure and impassable wall to protect the data. Data hiding is nothing but making data variable of the `class` or `struct` `private`. Thus, private data cannot be accessed directly by the object. The objects using public member functions of the same `class` can access the private data of the `class`. The keywords `private` and `protected` are used for hiding the data. Table 8.2 shows the brief description of access specifiers. The following program explains data hiding:

8.10 Write a program to calculate simple interest. Hide the data elements of the `class` using `private` keyword.



```
#include<iostream.h>
#include<conio.h>
class interest
{
    private :
        float p_amount; // principle amount
        float rate; // rate of interest
        float period; // numbers of years
        float interest;
        float t_amount; // total amount

    public :
        void in()
        {
            cout<<" Principle Amount : "; cin>>p_amount;
            cout<<" Rate of Interest : "; cin>>rate;
            cout<<" Number of years : "; cin>>period;
            interest=(p_amount*period*rate)/100;
            t_amount=interest+p_amount;
        }
        void show()
        {
            cout<<"\n Principle Amount : "<<p_amount;
            cout<<"\n Rate of Interest : "<<rate;
            cout<<"\n Number of years : "<<period;
            cout<<"\n Interest : "<<interest;
            cout<<"\n Total Amount : "<<t_amount;
        }
};
int main()
{
    clrscr();
    interest r;
```

```

    r.in();
    r.show();
    return 0;
}

```

OUTPUT

Principle Amount : 5000

Rate of Interest : 2

Number of years : 3

Principle Amount : 5000

Rate of Interest : 2

Number of years : 3

Interest : 300

Total Amount : 5300

Explanation: In the above program, the class `interest` is declared with the data members `p_amount`, `float rate`, `period`, `4`, and `t_amount` of `float` type. These entire data elements are declared in private section; hence, it is hidden (encapsulated) and cannot be directly accessed by the object. Here, member functions `in()` is used to read data through the keyboard. The function `show()` is used to display the contents of the variables.

8.11 Write a program to declare class with private, public and private sections. Declare object and access data elements of these different sections.

```

#include<iostream.h>
#include<conio.h>
class access
{
    private :
    int p;
    void getp()
    {
        cout<<" In getp() enter value of p : ";
        cin>>p;
    }
    public:
    int h;
    void geth()
    {
        cout<<" In geth() : " <<endl;
        getp();
        getm();
    }
}

```

```

    cout<<" p = "<<p <<" h = "<<h <<" m = "<<m;
}
protected :
int m;
void getm()
{
    cout<<" In mget () enter value of m : ";
    cin>>m;
}
};
int main()
{
    clrscr();
    access a; // object declaration
    // a.p=2; // access to private member is not possible.
    // a.pget () // -----" -----
    // a.m=5; // access to protected member is not possible
    // a.mget () ; // -----" -----
    a.h=4; // direct access to public member is possible
    a.geth();
    return 0;
}

```

OUTPUT

In geth() :

In pget () enter value of p:7

In mget () enter value of m: 4

p = 7 h = 4 m = 4

Explanation: In the above program, the class access is declared with private, protected, and public sections. Each section holds one integer variable and one member function. The object cannot directly access the data variable and member function of the private and protected sections. The object can access only the public section of the class and through public section it can access the private or protected section. Here, the object accesses the public variable h and initializes it with 4 whereas the private and protected variables are accessed using member functions. The function getp() and getm() are invoked by the public member function geth(). The geth() also displays the contents of the variables on the screen.

8.15 CLASSES, OBJECTS, AND MEMORY

Objects are the identifiers declared for class data type. Object is a composition of one more variables declared inside the class. Each object has its own copy of public and private data members.

An object can access to its own copy of data members and have no access to data members of other objects.

Only declaration of a class does not allocate memory to the class data members. When an object is declared, memory is reserved for only data member and not for member functions.

Consider the following program.

8.12 Write a program to declare objects and display their contents.

```
#include<iostream.h>
#include<constream.h>
class month
{
    public:
    char *name;
    int days;
}; // end of class
int main()
{
    clrscr();
    month M1,M3; // object declaration
    M1.name="January";
    M1.days=31;
    M3.name="March";
    M3.days=31;
    cout<<"\n Object M1 ";
    cout<<"\n Month name : "<<M1.name <<" Address : "<<(unsigned) &M1.
    name;
    cout<<"\n Days : " <<M1.days <<"\t\t Address : "<<(unsigned) &M1.
    days;
    cout<<"\n\n Object M3 ";
    cout<<"\nMonthname : "<<M3.name<<"\tAddress : "<<(unsigned) &M3.
    name;
    cout<<"\n Days : " <<M3.days <<"\t\t Address : "<<(unsigned) &M3.
    days;
    return 0;
}
```

OUTPUT

Object M1

Month name : January Address :65522

Days : 31 Address : 65524

Object M3

Month name : March Address : 65518

Days : 31 Address : 65520

Explanation: M1 and M3 are objects of class month. Separate memory is allocated to each object. The contents and address of the member variables are displayed in the output. Figure 8.4 shows it more visibly.

From the last program it is clear that memory is allocated for data members. What about functions? Member functions are created and memory is allocated to them only once when a class is declared. All objects of a class access the same memory location where member functions are stored. Hence, separate copies of member functions are not present in every object like member variables, which is illustrated in the following program and Figure 8.5.

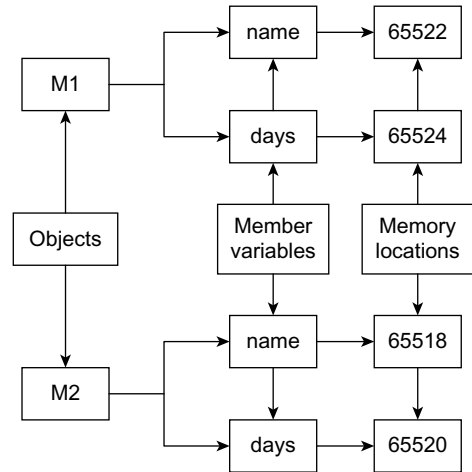


Fig. 8.4 Memory occupied by objects

8.13 Write a program to display the size of the objects.

```

#include<iostream.h>
#include<constream.h>
class data
{
    long i; // By default private
    float f;
    char c;
};
int main()
{
    clrscr();
    data d1,d2;
    cout<<endl<<" Size of object d1 = "<<sizeof(d1);
    cout<<endl<<" Size of object d2 = "<<sizeof(d2);
    cout<<endl<<" Size of class ="<<sizeof(data);
    return 0;
}

```

OUTPUT

Size of object d1 = 9

Size of object d2 = 9

Size of class = 9

Explanation: In the above program, the class data has three member variables of long, float, and char type. The d1 and d2 are objects of the class data. The sizeof operator displays the size of objects. The size of any object is equal to the sum of sizes of all the data members of the class. In the class data, the data type long occupies 4 bytes, float occupies 4 bytes, and char occupies 1 byte. Their sum is 9, which equals the size of an individual object.

The member functions are not considered in the size of the object. All the objects of a class use the same member functions. Only one copy of member function is created and stored in the memory whereas each object has its own set of data members.

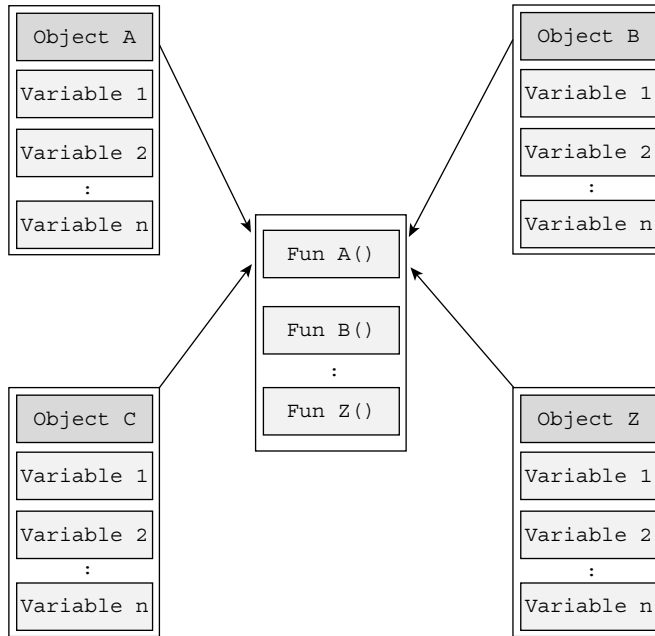


Fig. 8.5 Data members and member functions in memory

8.16 static MEMBER VARIABLES

We noticed that each object has a separate set of data member variables in memory. The member functions are created only once and all objects share the functions. No separate copy of function of each object is created in the memory like data member variables.

It is possible to create common member variables like function using the `static` keyword. Once a data member variable is declared as `static`, only one copy of that member is created for the whole class. The `static` is a keyword that is used to preserve the value of a variable. When a variable is declared as `static` it is initialized to zero. A `static` function or data element is only recognized inside the scope of the present compile.

In earlier version of Turbo C++, it was not necessary to explicitly define `static` data members. It was linker's responsibility to find undefined `static` data. The linker would implicitly define the `static` data and allocate the required memory without showing error message. In new versions of Turbo C++, it is necessary to explicitly define `static` members.

```
Syntax:  
static <variable definition> ;  
static <function definition>;
```

If a local variable is declared with `static` keyword, it preserves the last value of the variable. A static data item is helpful when all the objects of the class should share a common data. The static data variable is accessible within the class, but its value remains in the memory throughout the whole program (Figure 8.6).

```
Examples:  
static int c;  
static void display() {}  
a) int sumnum :: c=0;
```

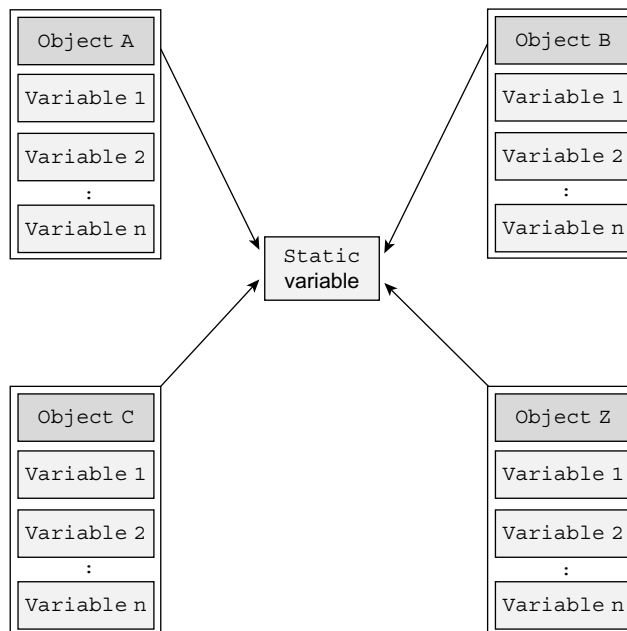


Fig. 8.6 static members in memory

The class and scope of the static member variable is defined outside the class declaration as per the statement (a). The reasons are as follows:

- (1) The static data members are associated with the class and not with any object.
- (2) The static data members are stored individually rather than an element of an object.
- (3) The static data member variable must be initialized otherwise the linker will generate an error.

- (4) The memory for static data is allocated only once.
- (5) Only one copy of static member variable is created for the whole class for any number of objects. All the objects have common static data member.

8.14 Write a program to declare static data member. Display the value of static data member

```
#include<iostream.h>
#include<constream.h>
class number
{
    static int c;

public:
    void count ()
    {
        ++c;
        cout<<"\n c="<<c;
    }
};
int number :: c=0; // initialization of static member variable
int main()
{
    number a,b,c;
    clrscr();
    a.count();
    b.count();
    c.count();
    return 0;
}
```

OUTPUT

```
c=1
c=2
c=3
```

Explanation: In the above program, the class number has one static data variable c. The count () is a member functions' increment value of static member variable c by one when called. The statement int number :: c=0 initializes the static member with 0. It is also possible to initialize the static data members with other values. In the function main () a, b, and c are three objects of class number. Each object calls the function count (). At each call to the function count (), the variable c gets incremented and the cout statement displays the value of the variable c. The objects a, b, and c share the same copy of static data member c.

8.15 Write a program to show the difference between static and non-static member variables.

```
#include<iostream.h>
#include<conio.h>
class number
{
    static int c; // static variable
    int k; // non-static variable
public:
    void zero()
    {
        k=0;
    }
    void count()
    {
        ++c;
        ++k;
        cout<<"\n Value of c = "<<c <<" Address of c = "<<(unsigned) &c;
        cout<<"\n Value of k = "<<k <<" Address of k = "<<(unsigned) &k;
    }
};
int number :: c=0; // initialization of static member variable
int main()
{
    number A,B,C;
    clrscr();
    A.zero();
    B.zero();
    C.zero();
    A.count();
    B.count();
    C.count();
    return 0;
}
```

OUTPUT

```
Value of c = 1 Address of c = 11138
Value of k = 1 Address of k = 85524
Value of c = 2 Address of c = 11138
Value of k = 1 Address of k = 65522
Value of c = 3 Address of c = 11138
Value of k = 1 Address of k = 65520
```

Explanation: This program compares between static and non-static member variables. The class `number` has two member variables `c` and `k`. The variable `c` is declared as static and `k` as normal variable. The function `zero()` is used to initialize the variable `k` with zero. The static member variable `c` is initialized with zero as follows:

```
int number::c=0; // initialization of static member variable
```

The function `count()` is used to increment values of `c` and `k`. In function `main()`, `A`, `B`, and `C` are objects of class `number`. The function `zero()` is invoked three times by object `A`, `B`, and `C`. Each object has its own copy of variable `k`, and, hence, each object invokes the function `zero()` to initialize its copy of `k`. The static member variable `c` is common among the object `A`, `B`, and `C`. Hence, it is initialized only once. Figure 8.7 shows the object and member variables in memory.

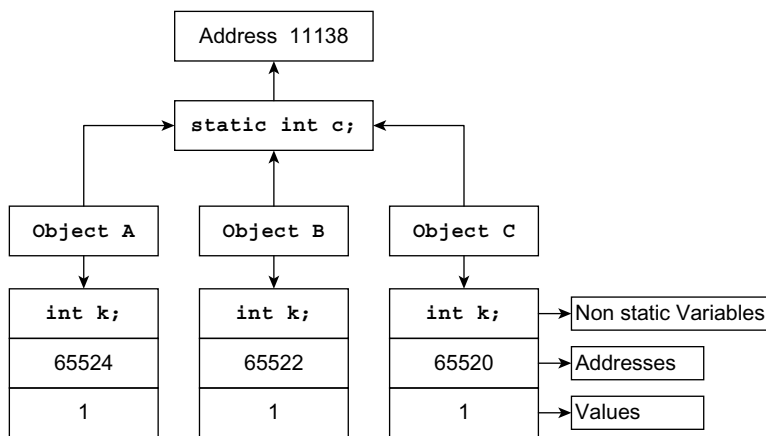


Fig. 8.7 static and non-static members

8.16 Write a program to enter a number. Count the total number of digits from 0 to 9 occurring from 1 to entered number.

```
#include<iostream.h>
#include<conio.h>
class digit
{
    static int num[10];
public:
    void check(int n);
    void show();
    void input();
    void ini();
};
int digit::num[]={0,0,0,0,0,0,0,0,0,0};
void digit::show()
```

{coderipe}

```
{
    for (int j=0;j<10;j++)
    {
        if (num[j]==0) continue;
        cout<<"\nNumber "<<j <<" occurs " <<num[j] <<" times";
    }
}
void digit::ini()
{
    for (int k=0;k<10;k++)
        num[k]=0;
}
void digit::input()
{
    int x,y;
    cout<<endl<<"\n Enter a Number : ";
    cin>>y;
    check(y);
}
void digit::check (int u)
{
    int m;
    while (u!=0)
    {
        m=u%10;
        num[m]++;
        u=u/10;
    }
}
int main()
{
    clrscr();
    digit d;
    // d.ini();
    d.input();
    d.show();
    return 0;
}
```

OUTPUT

Enter Numbers Number : 22151
Number 1 occurs 2 times
Number 2 occurs 2 times
Number 5 occurs 1 times

Explanation: In the above program, the class `digit` is declared with one static array member `num[10]` and four member functions `check()`, `show()`, `input()`, and `ini()`. The function `input()` reads an integer through the keyboard. The entered number is passed to function `check()`. The function `check()` is invoked by function `input()`. The function `check()` separates individual digits of the entered number using repetitive modular division and division operation. The separated digits are counted and the count value is stored in the array `num[10]` according to the element number. The function `show()` displays the contents of array `num[]`. The function `ini()` is declared and when called initializes all array elements with zero. In case the array is not declared as `static`, this function is useful. Here, in this program the array is `static`, hence we initialized it with the statement `int digit::num[]={0,0,0,0,0,0,0,0,0,0}`. If this statement is removed, we need to call the function `ini()`.

8.17 static MEMBER FUNCTIONS

Like member variables, function can also be declared as `static`. When a function is defined as `static`, it can access only `static` member variables and functions of the same class. The not-`static` members are not available to these functions. The `static` member function declared in public section can be invoked using its class name without using its objects. The `static` keyword makes the function free from the individual object of the class and its scope is global in the class without creating any side effect for other part of the program. The programmer must follow the following points while declaring `static` function:

- 1) Just one copy of a `static` member is created in the memory for entire class. All objects of the class share the same copy of `static` member.
- 2) `static` member function can access only `static` data members or functions.
- 3) `static` member function can be invoked using class name.
- 4) It is also possible to invoke `static` member functions using objects.
- 5) When one of the objects changes the value of data member variables, the effect is visible to all the object of the class.

8.17 Write a program to declare static member functions and call them from the `main()` function.

```
#include<iostream.h>
#include<conio.h>
class bita
{
    private :
    static int c;
    public :
    static void count() { c++; }
    static void display()
    {
        cout<<"\nValue of c : "<<c;
    }
};
```

```
int bita::c=0;
int main()
{
    clrscr();
    bita::display();
    bita::count();
    bita::count();
    bita::display();
    return 0;
}
```

OUTPUT

Value of c : 0

Value of c : 2

Explanation: In the above program, the member variables `c` and functions of class `bita` are static. The function `count()` when called increases the value of static variable `c`. The function `display()` prints the current value of the variable `c`. The static function can be called using class name and scope access operator from the following statements:

```
bita::count(); // invokes count() function
bita::display(); // invokes display() function
```

8.17.1 static Private Member Function

A static member function can also be declared in private section. The private static function must be invoked using static public function. The following program illustrates the point.

8.18 Write a program to define private static member function and invoke it.

```
#include<iostream.h>
#include<conio.h>
class bita
{
    private :
        static int c;
        static void count() { c++; }
    public:
        static void display()
        {
            count(); // Call to private static member function
            cout<<"\nValue of c : "<<c;
        }
};
int bita::c=0;
int main()
```

```

{
    clrscr();
    bita::display();
    bita::display();
    return 0;
}

```

OUTPUT

Value of c : 1

Value of c : 2

Explanation: In the above program, `count()` is a private static member function. The public static function `display()` invokes the private static function `count()`. The function `display()` also displays the value of static variable `c`.

8.17.2 static Public Member Variable

The static public member variable can also be initialized in function `main()` like other variables. The static member variable using class name and scope access operator can be accessed. The scope access operator is also used when variables of same name are declared in global and local scope, which is illustrated in the following program:

8.19 Write a program to declare static public member variable, global and local variable with the same name. Initialize and display their contents.

```

#include<iostream.h>
#include<constream.h>
int c=11; // global variable
class bita
{
    public:
    static int c;
};
int bita::c=22; // class member variable
int main()
{
    clrscr();
    int c=33; // local variable
    cout<<"\nClass member c = "<<bita::c;
    cout<<"\nGlobal variable c = "<<::c;
    cout<<"\nLocal variable c = "<<c;
    return 0;
}

```

OUTPUT**Class member c = 22****Global variable c = 11****Local variable c = 33**

Explanation: In the above program, the variable `c` is declared and initialized in three different scopes such as global, local, and inside the `class`. The variable `c` declared inside is `static` variable and initialized to 22. The global variable `c` is initialized to 11 and local variable `c` is initialized to 33.

static member variable: The value of `static` variable is displayed using variable name preceded by `class` name and scope access operator as per the statement `cout<<"\nClass member c = "<<bita::c;`.

Global variable: The global variable can be access using variable name preceded by scope access operator from the statement `cout<<"\nGlobal variable c = "<<::c;`.

Local variable: The local variable can be access only by putting its name as per the statement `cout<<"\nLocal variable c = "<<c;`.

8.18 static OBJECT

In C, it is common to declare variable `static` and it gets initialized to zero. The object is a composition of one or more member variables. There is a mechanism called constructor to initialize member variables of the object to desired values. The constructors are explained in the next chapter. The `static` keyword can be used to initialize all `class` data member variables to zero. Declaring object itself as `static` can do this. Thus all its associated members get initialized to zero. The following program illustrates the working of `static` object.

8.20 Write a program to declare static object. Display its contents.

```
#include<iostream.h>
#include<constream.h>
class bita
{
    private:
        int c;
        int k;
    public :
        void plus()
        {
            c+=2;
            k+=2;
        }
        void show()
        {
            cout<<" c= "<<c<<"\n";
```

```
        cout<<" k= "<<k;
    }
};
int main()
{
    clrscr();
    static bita A;
    A.plus();
    A.show();
    return 0;
}
```

OUTPUT

```
c= 2
k= 2
```

Explanation: The class bita has two member variables c and k and two member functions plus() and show(). In function main(), the object A is declared. It is also declared as static. The data members of object A are initialized to zero. The function plus() is invoked, which adds 2 to the values of c and k. The function displays values of c and k. Declare object static does not mean that entire class is static including member function. The declaration of static object removes garbage of its data members and initializes them to zero.

8.19 ARRAY OF OBJECTS

Arrays are collection of similar data types. Arrays can be of any data type including user-defined data type created using struct, class, and typedef declarations. We can also create an array of objects. The array elements are stored in continuous memory locations as shown in Figure 8.8. Consider the following example.

```
class player
{
    private:
        char name [20];
        int age;
    public:
        void input (void);
        void display (void);
};
```

The player is a user-defined data type and can be used to declare an array of object of type player. Each object of an array has its own set of data variables.

```
player cricket [5];
player football [5];
player hockey [5];
```

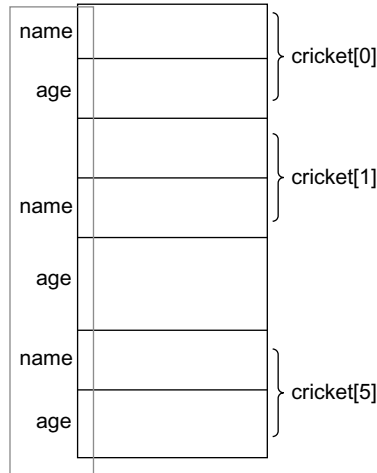


Fig. 8.8 Arrays of objects

As shown above, arrays of object of type `player` are created. The array `cricket[5]` contains name and age information for five objects. The next two declarations can maintain the same information for other player in arrays `hockey[5]` and `football[5]`. These arrays can be initialized or accessed like an ordinary array. The following program describes the working of array of objects.

8.21 Write a program to declare the array of objects. Initialize and display the contents of arrays.

```
#include<iostream.h>
#include<constream.h>
class player
{
    private:
        char name [20];
        int age;
    public:
        void input (void);
        void display (void);
};
void player :: input ()
{
    cout<<"\n Enter Palyer name : ";
    cin>>name;
    cout<<" Age : ";
    cin>>age;
}
void player :: display()
```

```
{
    cout<<"\n Player name : "<<name;
    cout<<"\n Age : "<<age;
}
int main()
{
    clrscr();
    player cricket[3]; // array of objects
    cout<<"\n Enter Name and age of 3 players ";
    for (int i=0;i<3;i++)
        cricket[i].input();
    for (i=0;i<3;i++)
        cricket[i].display();
    return 0;
}
```

OUTPUT

Enter Name and age of 3 players

Enter Palyer name : Sachin

Age : 29

Enter Palyer name : Rahul

Age : 28

Enter Palyer name : Saurav

Age : 30

Player name : Sachin

Age : 29

Player name : Rahul

Age : 28

Player name : Saurav

Age : 30

Explanation: In the above program, the member function `input()` reads information of players. The `display()` function displays information on the screen. In function `main()` the statement `player cricket[3];` creates an array `cricket[3]` of three objects of type `player`. The `for` loops are used to invoke member function `input()` and `display()` using array of objects.

8.20 OBJECTS AS FUNCTION ARGUMENTS

Similar to variables, object can be passed to functions. The following are the three methods to pass argument to a function:

- a) **Pass-by-value** – A copy of object (actual object) is sent to function and assigned to the object of callee function (formal object). Both actual and formal copies of objects are

stored at different memory locations. Hence, changes made in formal object are not reflected to actual object.

- b) Pass-by-reference – Address of object is implicitly sent to function.
- c) Pass-by-address – Address of the object is explicitly sent to function.

In pass-by-reference and pass-by-address methods, an address of actual object is passed to the function. The formal argument is reference/pointer to the actual object. Hence, changes made in the object are reflected to actual object. These two methods are useful because an address is passed to the function and duplicating of object is prevented.

The following examples illustrate both the methods of passing objects to the function as an argument.

8.22 Write a program to pass objects to the function by pass-by-value method.

```
#include<iostream.h>
#include<conio.h>
class life
{
    int mfgyr;
    int expyr;
    int yr;
public :
    void gettyrs ()
    {
        cout<<"\nManufacture Year : ";
        cin>>mfgyr;
        cout<<"\n Expiry Year : ";
        cin>>expyr;
    }
    void period ( life);
};
void life :: period (life y1)
{
    yr=y1.expyr-y1.mfgyr;
    cout<<"Life of the product : " <<yr <<" Years";
}
int main()
{
    clrscr();
    life a1;
    a1.gettyrs();
    a1.period(a1);
    return 0;
}
```

{coderipe}

OUTPUT**Manufacture Year : 1999****Expiry Year : 2002****Life of the product : 3 Years**

Explanation: In the above program, the class `life` is declared with three member integer variables. The function `getyrs()` reads the integers through the keyboard. The function `period()` calculates the difference between the two integers entered. In the function `main()`, `a1` is an object to the class `life`. The object `a1` calls the function `getyrs()`. Immediately after this, the same object (`a1`) is passed to the function `period()`. The function `period()` calculates the difference between two integers (dates) using the two data members of the same class. Thus, an object can be passed to the function. To pass an object by reference, the prototype of function `period()` should be as follows:

```
void period ( life & );
```

8.23 Write a program to pass objects to the function pass-by-address method.

```
#include<iostream.h>
#include<conio.h>
class life
{
    int mfgyr;
    int expyr;
    int yr;
public :

    void getyrs ()
    {
        cout<<"\nManufacture Year : ";
        cin>>mfgyr;
        cout<<"\n Expiary Year : ";
        cin>>expyr;
    }
    void period ( life* );
};
void life :: period (life *y1)
{
    yr=y1->expyr-y1->mfgyr;
    cout<<"Life of the product : " <<yr;
}
int main()
{
    clrscr();
    life a1;
```

```
    a1.getyrs();  
    a1.period(&a1);  
    return 0;  
}
```

OUTPUT

Manufacture Year : 1999

Expiry Year : 2002

Life of the product : 3 Years

Explanation: The above program is same as previous one. In this program, the object a1 is passed by address. Consider the following statements.

```
a) void period ( life*);  
b) a1.period (&a1);  
c) Yr=y1->expyr-y1->mfgyr;
```

The statement (a) is the prototype of the function period(). In this statement, the deference operator (*) indicates that the function will accept address of the actual argument. The statement (b) is used to pass the address of the argument to the function period(). The statement (c) is used to access the member variables of the class. When an object is a pointer to the class members, then its elements are accessed by using arrow (->) operator. In this case, use of a dot operator (.) is invalid.

8.21 friend FUNCTIONS

The central idea of encapsulation and data hiding concept is that any non-member function has no access permission to the private data of the class. The private members of the class are accessed only from member functions of that class.

C++ allows a mechanism, in which a non-member function has access permission to the private members of the class. This can be done by declaring a non-member function friend to the class whose private data is to be accessed. The friend is a keyword. Consider the following example.

```
class ac  
{ private:  
    char name [15];  
    int acno;  
    float bal;  
    public:  
    void read();  
    friend void show-  
bal();  
};
```

The keyword `friend` must precede the function declaration, whereas function declarator must not. The function can be defined at anyplace in the program like normal function. The function can be declared as friend function in one or more classes. The keyword `friend` or scope access operator must not precede the definition of the friend function. The declaration of friend function is done inside the `class` in private or public part and a function can be declared as friend function in any number of classes. These functions use objects as arguments. Thus the following statement is wrong.

```
a) friend void :: showbal (ac a) // Wrong function definition
{
    statement1;
    statement2;
}
```

The above declaration of function is wrong because the function declarator precedes the keyword `friend`.

The friend functions have the following properties:

- a) There is no scope restriction for the friend function; hence, they can be called directly without using objects.
- b) Unlike member functions of `class`, the friend cannot access the member directly. On the other hand it uses `object` and `dot` operator to access the private and public member variables of the `class`.
- c) By default, friendship is not shared (mutual). For example, if `class X` is declared as friend of `Y`, this does not mean that `Y` has privileges to access private members of `class X`.
- d) Use of friend functions is rare, since it violates the rule of encapsulation and data hiding.
- e) The function can be declared in public or private sections without changing its meaning.

8.24 Write a program to access private data using non-member function. Use friend function.

```
#include<iostream.h>
#include<conio.h>
class ac
{
    private:
        char name[15];
        int acno;
        float bal;
    public:
        void read()
        {
            cout<<"\nName : " ;
```

```

    cin>>>name;
    cout<<"\nA/c No. :";
    cin>>>acno;
    cout<<"\n Balance :";
    cin>>>bal;
}
friend void showbal (ac ); // friend function declaration
};
void showbal (ac a)
{
    cout<<"\n Balance of A/c no. " <<a.acno <<" is Rs." <<a.bal;
}
int main()
{
    ac k;
    k.read();
    showbal(k); // call to friend function
    return 0;
}

```

OUTPUT

Name :Manoj
A/c No. :474
Balance :40000
Balance of A/c no. 474 is Rs.40000

Explanation: In the above program, class ac is declared. It has three member variables and one member function. Also, inside the class ac, showbal () is a function, which is declared as friend of the class ac. Once the outside function is declared as friend to any class, it gets an authority to access the private data of that class. The function read() reads the data through the keyboard such as name, account number, and balance. The friend function showbal () display the balance and acno.

8.25 Write a program to declare friend function in two classes. Calculate the sum of integers of both the classes using friend sum() function.

```

#include<iostream.h>
#include<conio.h>
class first;
class second
{
    int s;
    public :

```

```

    void getvalue()
    {
        cout<<"\nEnter a number : ";
        cin>>>s;
    }
    friend void sum (second, first);
};
class first
{
    int f;
    public :
    void getvalue()
    {
        cout<<"\nEnter a number : " ;
        cin>>>f;
    }
    friend void sum (second , first);
};
void sum (second d, first t)
{
    cout<<"\n Sum of two numbers : " <<t.f + d.s;
}
int main()
{
    clrscr();
    first a;
    second b;
    a.getvalue();
    b.getvalue();
    sum(b,a);
}

```

OUTPUT

Enter a number: 7

Enter a number: 8

Sum of two numbers: 15

Explanation: In the above program, two classes `first` and `second` are declared with one integer and one member function in each. The member function `getvalue()` of both classes reads integers through the keyboard. In both the classes the function `sum()` is declared as `friend`. Hence, this function has an access to the members of both the classes. Using `sum()`, function addition of integers is calculated and displayed.

8.26 Write a program to exchange values between two classes. Use friend functions.

```
#include<iostream.h>
#include<conio.h>
class second;
class first
{
    int j;
public:
    void input ()
    {
        cout<<"Enter value of j : ";
        cin>>>j;
    }
    void show (void)
    {
        cout<<"\n Value of J = ";
        cout<<j <<"\n";
    }
    friend void change (first &, second &);
};
class second
{
    int k;
public :
    void input ()
    {
        cout<<"\nEnter value of k : ";
        cin>>>k ;
    }
    void show (void)
    {
        cout<<" Value of K = ";
        cout<<k ;
    }
    friend void change (first & , second &);
};
void change ( first &x, second &y)
{
    int tmp=x.j;
    x.j=y.k;
    y.k=tmp;
}
```

```

main()
{
    clrscr();
    first c1;
    second c2;
    c1.input();
    c2.input();
    change (c1,c2);
    cout<<"\nAfter change values are" <<"\n";
    c1.show();
    c2.show();
    return 0;
}

```

OUTPUT

```

Enter value of j : 4
Enter value of k : 8
After change values are
Value of J = 8
Value of K = 4

```

Explanation: In the above program, two classes `first` and `second` are defined. Each class contains one integer variable and two member functions. The function `input()` is used to read an integer through the keyboard. The function `show()` is used to display the integer on the screen. The function `change()` is declared as friend function for both the classes. Passing values by reference of member variables of both the classes, values are exchanged.

8.27 Write a program to declare three classes. Declare integer array as data member in each class. Perform addition of two data member array into array of third class. Use friend function.

```

#include<iostream.h>
#include<conio.h>
class B;
class C;
class A
{
    int a[5];
    public :
    void input();
    friend C sum (A,B,C);
};
void A::input()
{

```

```
    int k;
    cout<<"\n Enter five integers : ";
    for (k=0;k<5;k++)
        cin>>a[k];
}
class B
{
    int b[5];
    public :
    void input();
    friend C sum (A,B,C);
};
void B::input()
{
    int k;
    cout<<"\n Enter five integers : ";
    for (k=0;k<5;k++)
        cin>>b[k];
}
class C
{
    int c[5];
    public :
    void show();
    friend C sum (A,B,C);
};
void C::show()
{
    cout<<"\n\t Addition : ";
    for (int k=0;k<5;k++)
        cout<<" " <<c[k];
}
C sum (A a1 ,B b1, C c1)
{
    for (int k=0;k<5;k++)
        c1.c[k]=a1.a[k]+b1.b[k];
    return c1;
}
int main()
{
    clrscr();
    A a;
    B b;
    C c;
```



```

a.input();
b.input();
c=sum(a,b,c);
c.show();
}

```

OUTPUT

Enter five integers : 5 4 8 7 5

Enter five integers : 2 4 1 2 3

Addition : 7 8 9 9 8

Explanation: In the above program, three classes A, B, and C are declared. Each class contains single integer arrays as data member `a[5]`, `b[5]`, and `c[5]`, respectively. The class A and B contains member function `input()` to read integers. The function `sum()` is declared as friend in all the three classes. This function performs addition of arrays of class A and B and stores results in the array of class C. The result obtained is returned in `main()` where the return value is assigned to object `c`. In `main()` `a`, `b`, and `c` are objects of classes A, B, and C, respectively. The member function `show()` of class C displays the contents of object `c`.

8.21.1 friend Classes

It is possible to declare one or more functions as friend functions or an entire class can also be declared as friend class. When all the functions need to access another class in such a situation we can declare an entire class as friend class. The friend is not transferable or inheritable from one class to another. Declaring class A to be a friend of class B does not mean class B a friend of class A; that is, friendship is not exchangeable. The friend classes are applicable when we want to make available private data of a class to another class.

8.28 Write a program to declare friend classes and access the private data.

```

#include<iostream.h>
#include<constream.h>
class B;
class A
{
    private :
        int a;
    public :
        void aset() {a=30;}
        void show (B);
};
class B
{
    private :

```

```
    int b;
    public :
    void bset () { b=40 ; };
    friend void A :: show (B bb) ;
};
void A :: show (B b)
{
    cout<<"\n a = "<<a;
    cout<<"\n b = "<<b.b;
}
int main()
{
    clrscr();
    A a1;
    a1.aset();
    B b1;
    b1.bset();
    a1.show(b1);
}
```

OUTPUT

a = 30

b = 40

Explanation: In the above program, two classes A and B are declared. The class A is friend of class B. The member function of class A can access the data of class B. Thus, the show() function displays the values of data members of both the classes.

8.29 Write a program to demonstrate friend classes.

```
#include<iostream.h>
#include<conio.h>
class CPP;
class C
{
    private :
    int j;
    public :
    void set () { j=22; }
    friend CPP;
};
class CPP
```

```

{
    public :
    void joy(C a)
    {
        cout<<endl<<" j = "<<a.j;
    }
    void joya(C o)
    {
        cout<<endl<<" j = "<<o.j;
    }
};
int main()
{
    clrscr();
    C x;
    CPP y;
    x.set();
    y.joy(x);
    y.joya(x);
    return 0;
}

```

OUTPUT

```

j = 22
j = 22

```

Explanation: In the above program, class C and class CPP are declared as friend. The class CPP is declared as friend class of class C. Member function of class CPP can access the private data variables of the class C.

8.22 THE `const` MEMBER FUNCTIONS

The member functions of a class can also be declared as constant using `const` keyword. The constant functions cannot modify any data in the class. The `const` keyword is suffixed to the function prototype as well as in function definition. If these functions attempt to change the data, compiler will generate an error message.

8.30 Write a program to declare `const` member function and attempt any operation within it.

```

#include<iostream.h>
#include<conio.h>

class A
{

```

```

    int c;
    public :
    void add (int a,int b) const
    {
        // c=a+b; // invalid statement
        a+b;
        cout<<"a+b = "<<_AX ;
    }
};
int main()
{
    clrscr();
    A a;
    a .add (5,7) ;
    return 0;
}

```

OUTPUT

a+b = 12

Explanation: In the above program, the class A is declared with one member variable (c) and one constant member function add (). The add () function is invoked with two integers. The constant member function cannot perform any operation. Hence, the expression c=a+b will generate an error. The expression a+b is valid and cannot alter any value. The result obtained from the equation a+b is displayed using CPU register.

8.23 THE VOLATILE MEMBER FUNCTION

In C++, one can declare a member function with volatile specifies. This step leads to call safely the volatile object. Calling volatile member function with volatile object is safe. This concept is supported with the following programming example.

8.31 Write a program to call a volatile member function from a volatile object.

```

#include<iostream.h>
#include<conio.h>
class A
{
    private:
    int x;
    public:
    void f() volatile // The volatile member function
    {
        int x=10;
        cout<<"Value of x:"<<++x;
    }
}

```

```
};
int main()
{
    clrscr();
    volatile A c; // The c is a volatile object
    c.f(); // Call a volatile member function safely
    return 0;
}
```

OUTPUT

Value of x:11

Explanation: The `volatile` member function `f()` is called from a `volatile` object `c` and the value of `x` is initialized to 10. Its value is increased by one and displayed on the screen.

8.24 RECURSIVE MEMBER FUNCTION

Like C, C++ also supports recursive feature; that is, a function is called repetitively by itself. The recursion can be used directly or indirectly. The direct recursion function calls to itself until the condition is true. In indirect recursion, a function calls to another function and then the called function calls to the calling function. Here, the recursion with member function is illustrated in the following program:

8.32 Write a program to calculate triangular number by creating a member function. Call it recursively.

```
#include<iostream.h>
#include<conio.h>
class num
{
    public :
    tri_num(int m)
    { int f=0;
      if (m==0)
        return(f);
      else
        f=f+m+tri_num(m-1);
      return (f);
    }
};
int main()
{
    clrscr();
    num a;
    int x;
```



```

    cout<<"\n Enter a number : ";
    cin>>x;
    cout<<"Triangular number of "<<x<<" is : "<<a.tri_num(x);
    return 0;
}

```

OUTPUT

Enter a number : 5

Triangular number of 5 is : 15

Explanation: In the above program, class num is declared with one member function `tri_num()`. This function is used to calculate the triangular number of the entered number. The triangular number is nothing but the sum from 1 to the number entered. In function `main()`, a number is read through the keyboard and it is passed to function `tri_num()`, which is invoked by the object `a` of class `num`. The `tri_num()` is invoked and `tri_num()` invokes itself repetitively till the value of `m` becomes 0. The variable `f` holds the cumulative total of successive numbers and `return()` statement returns value of `f` in function `main()`, where it displays triangular number on the screen.

8.25 LOCAL CLASSES

When classes are declared inside the function then such classes are called as local classes. The local classes have access permission to global variables as well as `static` variables. The global variables need to be accessed using scope access operator when the class itself contains member variable with same name as global variable. The local classes should not have `static` data member and `static` member functions. If at all they are declared, the compiler provides an error message. The following programs illustrate the local classes.

8.33 Write a program to define classes inside and outside `main()` function and access the elements.

```

#include<iostream.h>
#include<conio.h>
class A
{
    private :
    int a;
    public :
    void get ()
    {
        cout<<"\n Enter value for a : ";
        cin>>>a;
    }
    void show()
    { cout<<endl<<" a = " <<a; }
};

```

```
main()
{
    clrscr();
    class B
    {
        int b;
    public :
        void get()
        {
            cout<<"\n Enter value for b : ";
            cin>>b;
        }
        void show()
        {
            cout<<" b = " <<b;
        }
    };
    A j;
    B k;
    j.get();
    k.get();
    j.show();
    k.show();
    return 0;
}
```

OUTPUT

Enter value for a : 8

Enter value for b : 9

a = 8 b = 9

Explanation: In the above program, class A is declared before main() function as usual. The class B is declared inside the main() function. Both the functions have two member functions get() and show(). The get() function reads integers through the keyboard. The show() functions display the values of data members on the screen.

8.34 Write a program to declare global variables, read and display data using member functions.

```
#include<iostream.h>
#include<conio.h>
int j, k, l, m; // global variable
class A
{
    private :
```

```

    int a;
    int j;
    public :
    void get ()
    {
        cout<<"\n Enter value for a,j,j and k : ";
        cin>>a>>j>>::j>>k;
    }
    void show ()
    {
        cout<<endl<<" a = " <<a <<" j = " <<j <<" ::j = " <<::j <<" k = " <<k ;
    }
};
int main ()
{
    clrscr();
    class B
    {
        int b;
        int l;
        public :
        void get ()
        {
            cout<<"\n Enter value for b,l,l and m : ";
            cin>>b>>l>>::l>>m;
        }
        void show ()
        {
            cout<<"\n b = " <<b <<" l = " <<l <<" ::l = " <<::l <<" m = " <<m;
        }
    };
    A x;
    B y;
    x.get();
    y.get();
    x.show();
    y.show();
    return 0;
}

```

OUTPUT

```

Enter value for a,j,j and k : 1 2 3 4
Enter value for b,l,l and m : 5 6 4 3
a = 1 j = 2 ::j = 3 k = 4
b = 5 l = 6 ::l = 4 m = 3

```


Explanation: The above program is so far same as previous one. In addition, in this program global variables `j`, `k`, `l`, and `m` are declared. The member functions `get()` and `show()` read and display values of member variables as well as global variables. Here, both the classes contain a single data member variable with the same name as global variables. Thus, to access the global variable where necessary, scope access operator is used. The output of the program is as shown in the above program.

8.26 empty, static, and const CLASSES

The classes without any data members or member functions are called as empty classes. These types of classes are not frequently used. The empty classes are useful in exception handling. For exception handling, refer the chapter 19 exception handling. The syntax of empty class is as follows:

EMPTY CLASSES

<pre>class nodata { }; class vacant { };</pre>
--

We can also precede the declaration of classes by the keywords `static`, `const`, `volatile`, etc. But there is no any effect in the `class` operations. Such declaration can be done as follows:

CLASSES AND OTHER KEYWORDS

<pre>static class boys { }; const class data { }; volatile class area { };</pre>
--

8.27 MEMBER FUNCTION AND NON-MEMBER FUNCTION

So far we used non-member function `main()` for declaring objects and calling member functions. Apart from `main()` function other non-member functions can also be used. The member function can also invoke a non-member function and vice versa. When a member function calls a non-member function, it is necessary to put its prototype inside the calling function or at the beginning of the program. It is a better practice to put prototype at the beginning of the program that is visual to the entire program. It is also possible to put definition of the non-member function before `class` declaration. This method allows member function to invoke outside non-member function without need of prototype declaration. But this approach creates problem when an outside non-member function attempts to invoke member function.

We know that member functions can be called using object of that class. If a non-member function is defined before `class` declaration, it is not possible to create an object in that function. Hence, the best choice is to put prototype of the non-member function at the beginning of the program that makes easy for both non-member function and member function to call each other. The following program explains practically whatever we learned about member function and non-member function in this section.

8.35 Write a program to call a member function using non-member function.

```
#include<iostream.h>
#include<conio.h>
void moon(void); // Function prototype declaration
class mem
{
    public :
    void earth() { cout<<"On earth"; }
};
int main()
{
    clrscr();
    mem k;
    moon();
    return 0;
}
void moon()
{
    mem j;
    j.earth();
    cout<<endl<<"On moon ";
}
```

OUTPUT

On earth

On moon

Explanation: In the above program, moon() is a non-member function and its prototype is declared at the beginning of the program. The function main() calls function moon(). In function moon(), object j of type class mem is declared and a member function earth() is invoked. Thus, non-member function calls the member function.

8.28 THE main() FUNCTION AS A MEMBER FUNCTION

We know that the function main() is the starting execution point of every C/C++ program. The main() can be used as a member function of the class. But the execution of program will not start from this member function. The compiler treats member function main() and the user-defined main() differently. No ambiguity is observed while calling function. The following program narrates this concept.

8.36 Write a program to make main() as a member function.

```
#include<conio.h>
#include<iostream.h>
```

```
class A
{
    public:
    void main()
    {
        cout<<endl<<"In member function main() ";
    }
};

int main()
{
    clrscr();
    A *a;
    a->main();
    return 0;
}
```

OUTPUT

In member function main()

Explanation: In the above program, class A is declared and has one member function main(). In the non-member function main(), the object a invokes the member function main() and a message is displayed as shown in the output.

8.29 OVERLOADING MEMBER FUNCTIONS

Member functions are also overloaded in the same fashion as other ordinary functions. We learned that overloading is nothing but a function is defined with multiple definitions with same function name in the same scope. The following program explains the overloaded member function.

8.37 Write a program to overload member function of a class.

```
#include<iostream.h>
#include<stdlib.h>
#include<math.h>
#include<conio.h>
class absv
{
    public :
    int num (int);
    double num (double);
};

int absv:: num (int x)
{
    int ans;
    ans=abs(x);
```

```

    return (ans);
}
double absv :: num (double d)
{
    double ans;
    ans=fabs(d);
    return (ans);
}
int main()
{
    clrscr();
    absv n;
    cout<<"\nAbsolute value of -25 is "<<n.num(-25);
    cout<<"\nAbsolute value of -25.1474 is "<<n.num(-25.1474);
    return 0;
}

```

OUTPUT

Absolute value of -25 is 25

Absolute value of -25.1474 is 25.1474

Explanation: In the above program, the class `absv` has a member function `num()`. The `num()` function is overloaded for integer and double. In function `main()` the object `n` invokes the member function `num()` with one value. The compiler invokes the overloaded function according to the value. The function returns the absolute value of the number.

8.30 OVERLOADING main() FUNCTIONS

In the last two subtitles, we learnt how to make `main()` as member function and how to overload member function. Like other member function, `main()` can be overloaded in the class body as a member function. The following program explains this concept:

8.38 Write a program to declare `main()` as a member function and overload it.

```

#include<conio.h>
#include<iostream.h>
class A
{
    public:
    void main(int i)
    {
        cout<<endl<<"In main (int) : "<<i;
    }
    void main (double f)
    {

```

```

    cout<<"\nIn main(double) : "<<f;
    }
    void main (char *s)
    {
        cout<<endl<<"In main (char ) : "<<s;
    }
};
int main()
{
    clrscr();
    A *a;
    a->main(5);
    a->main(5.2);
    a->main("C++");
    return 0;
}

```

OUTPUT

In main(int) :5

In main(double) :5.2

In main(char) : C++

Explanation: This program is same as the previous one. Here, the `main()` function is used as a member function and it is overloaded for integer, float, and character.

It is not possible to overload the non-member `main()` function, which is the source of the C/C++ program and hence the following program will not be executed and displays the error message "Cannot overload 'main'".

```

#include<iostream.h>
void main()
{ }
main (float x, int y)
{
    cout<<x<<y;
    return 0;
}

```



TIP

The `main()` is the only function that cannot be overloaded.

8.31 THE `main()`, MEMBER FUNCTION, AND INDIRECT RECURSION

When a function calls itself, then this process is known as recursion or direct recursion. When two functions call each other repetitively, such type of recursion is known as indirect recursion.

Consider the following program and explanation to understand the indirect recursion using OOP. The program without using OOP is also described for programmers who are learning C++.

8.39 Write a program to call function main() using indirect recursion.

```
//Indirect Recursion Using OOP//
#include<iostream.h>
#include<conio.h>

int main (int);
class rec
{
    int j;
public:
    int f;
    rec (int k, int i)
    {
        clrscr();
        cout<<"[ ";
        f=i;
        j=k;
    }
    ~rec()
    { cout<<"\b\b] Factorial of number : "<<f ; }
    void pass()
    {
        cout<<main (j--)<<" * ";
    }
};

rec a(5,1);
main (int x)
{
    if (x==0)
        return 0;
    a.pass();
    a.f=a.f*x;
    return x;
}
```

OUTPUT

[0 * 1 * 2 * 3 * 4 * 5] Factorial of number : 120

Explanation: In the above program, class rec is declared with constructor, destructor, member function pass(), and two integer variables. The integer variable j is private and f is public. The function main() is defined with one integer argument. Usually,

`main()` with arguments is used for the applications on the dos prompt. In this program, `main()` is called recursively by member function `pass()`. When a function call is made, the value of member data variable is decreased first and then passed. Thus, the `main()` passes value to itself. In function `pass()`, `main()` function is invoked and its return value is displayed. The public data member is directly used and by applying multiplication operation, factorial of a number is calculated.

Generally, objects are declared inside the function `main()`. But in this program function `main()` is used in recursion. Hence, if we put the object declaration statement inside the `main()`, in every call of `main()` object is created and the program will not run properly. To avoid this, the object is declared before `main()`.

Constructor is used to initialize data members as well as to clear the screen. Destructor is used to display the factorial value of the number. All the statements that we frequently put in `main()` are written outside of `main()`.

Before the `class` declaration, prototype of `main()` is given, because the member functions do not know about; and the prototype declaration provides information about `main()` to member function.



TIP

For C programmers the previous program in C style is explained as follows.

8.40 Write a program to call function `main()` using in-direct recursion in C style.

```
//Indirect Recursion in C style//
#include<iostream.h>
#include<conio.h>
#include<process.h>
int m=5;
int f=1;
int j;
main (int x)
{
    void pass (void);
    if (x==0)
    { clrscr();
      cout<<endl<<"Factorial of number ="<<f;
      return 0;
    }
    f=f*x;
    pass();
    return x;
}
void pass() { main (m--); }
```

OUTPUT

Factorial of number =120

Explanation: The logic of the program is same as the previous one. The user-defined function `pass()` has only one job to invoke function `main()`. The `if` conditions inside `main()` check the value of variable `x`. If value of `x` is zero, then `if` block is executed that displays factorial of the number and terminates the programs.

8.32 BIT FIELDS AND CLASSES

Bit field provides exact amount of bits required for storage of values. If a variable value is 1 or 0 we need a single bit to store it. In the same way, if the variable is expressed between 0 and 3, then the two bits are sufficient for storing these values. Similarly if a variable assumes values between 0 and 7, then three bits will be enough to hold the variable and so on. The number of bits required for a variable is specified by non-negative integer followed by a colon.

To hold the information, we use the variables. The variables occupy minimum one byte for `char` and two bytes for `integer`. Instead of using complete integer if bits are used, memory space can be saved. For example, to know the information about the vehicles, following information has to be stored in the memory:

- (1) PETROL VEHICLE
- (2) DIESEL VEHICLE
- (3) TWO_WHEELER VEHICLE
- (4) FOUR_WHEELER VEHICLE
- (5) OLD MODEL
- (6) NEW MODEL

In order to store the status of the above information, we may need two bits for the type of fuel as to whether the vehicle is of petrol or diesel type, three bits for its type as to whether the vehicle is two- or four-wheeler, and similarly, three bits for the model of the vehicle. Total bits required for storing the information would be 8 bits, that is one byte. It means that the total information can be packed into a single byte. Eventually bit fields are used for conserving the memory. The amount of memory saved by using bit fields will be substantial which is proved from the above example.

However, there are restrictions on bit fields when arrays are used. Arrays of bit fields are not permitted. Also the pointer cannot be used for addressing the bit field directly, although the use of the member access operator (`->`) is acceptable. The unnamed bit fields could be used for padding as well as for alignment purposes.

- (1) Bits fields should have integral type. A pointer and array type is now allowed.
- (2) Address of bit fields cannot be obtained using `&` operator.

The `class` for the above problem would be as follows:

```
class vehicle
{
    unsigned type: 3;
    unsigned fuel: 2;
    unsigned model: 3;
};
```

The colon (`:`) in the above declaration tells to the compiler that bit fields are used in the `class`

and the number after it indicates how many bits are required to allot for the field. A simple program is illustrated as follows:

8.41 Write a program to use bit fields with classes and display the contents of the bit fields.

```
#include<conio.h>
#include<iostream.h>
#define PETROL 1
#define DIESEL 2
#define TWO_WH 3
#define FOUR_WH 4
#define OLD 5
#define NEW 6
class vehicle
{
    private :
    unsigned type : 3;
    unsigned fuel : 2;
    unsigned model : 3;
    public:
    vehicle()
    {
        type=FOUR_WH;
        fuel=PETROL;
        model=NEW;
    }
    void show()
    {
        if (model==NEW)
            cout<<"\n New Model ";
        else
            cout<<"\n Old Model ";
        cout<<"\n Type of Vehicle : "<< type;
        cout<<"\n Fuel : "<<fuel;
    }
};
int main()
{
    clrscr();
    vehicle v;
    cout<<" Size of Object : "<<sizeof(v)<<endl;

    v.show();
}
```

```
    return 0;
}
```

OUTPUT

Size of Object : 1

New Model

Type of Vehicle : 4

Fuel : 1

Explanation: In the above program, using #define macros are declared. The information about the vehicle is indicated with integers from 1 to 6. The class vehicle is declared with bit fields. The number of bits required for each member is initialized. As per the program, type of vehicle requires 3 bits, fuel requires 2 bits, and model requires 3 bits. An object v is declared. The constructor initializes bits fields with data. The output of the program displays integer value stored in the bit fields, which can be verified with macro definitions initialized at the beginning of the program.

8.33 NESTED class

When a class is defined in another class, it is known as nesting of classes. In nested class the scope of inner class is restricted by outer class. Simple programming example using public specifier is demonstrated for understanding.

8.42 Write a program to display some message using nested class.

```
#include<iostream.h>
#include<conio.h>
class one
{
    public:
    class two
    {
        public:
        void display()
        {
            cout<< "Wonderful language C++\n";
        }
    };
};
int main()
{
    clrscr();
    one::two x;
    x.display();
}
```

```
    return 0;
}
```

OUTPUT

Wonderful language C++

Explanation: In the above example, one class is nested in another; that is class two is nested in class one. By using scope resolution operator, the inner class member function is accessed and “Wonderful language C++” is displayed in the above program.

8.34 MORE PROGRAMS

8.43 Write a program to accept string and display the string. Use null character for determining the end of string.

```
#include<iostream.h>
#include<conio.h>
class text
{
    char str[50];
public :

    void get ()
    {
        cout<<"Enter text : ";
        cin.getline(str,50);
    }

    show (int x)
    {
        if (str[x]=='\0')
            return 0;
        else
        {
            cout<<str[x];
            return 1;
        }
    }
};

int main()
{
    clrscr();
    int k=-1;
    text s;
    s.get();
```

```

    cout<<"Entered text : ";
    while (s.show(++k));
    return 0;
}

```

OUTPUT

Enter text : Object Oriented Programming

Entered text : Object Oriented Programming

Explanation: In the above program, the class `text` is defined with one data member of character type. The `get()` function is used to read text through the keyboard. The `show()` function is defined with integer argument. The `show()` function displays the string character by character. The integer variable `x` shows the element number in the character array `str[50]`. The `if` statement checks whether the current character is null or other. If the character is null then it returns 0, otherwise it displays the character and returns 1. In function `main()`, `s` is an object of class `text`. The object `s` calls `gets()` and reads text.

The `show()` function is called within the bracket of `while` loop. The integer variable `k` is incremented before passing it to the function `show()`. In the statement, `++k` increment first takes place and then incremented value is sent. The array element counting starts from zero. In order to display the string from first character, the initial value of `k` should be 0. Thus, initializing `k` to `-1` and applying increment operator as prefix can do this.

We know that the `while` loop is executed till the given test condition evaluates to 1. Thus, till the function `show()` returns 1 the loop is executed and when it returns 0 the loop is terminated. The function `show()` returns 0 only when it reaches the end of text. Meanwhile, the entire text is displayed on the screen.

8.44 Write a program to enter two strings and concatenate them. Display the resulting string.

```

#include<iostream.h>
#include<string.h>
#include<conio.h>
class text
{
    char str1[15];
    char str2[15];
    char str3[30];
    public :

    void get()
    {
        cout<<"\n Enter First String : ";
        cin.getline(str1,15);
        cout<<"\n Enter Second String : ";
    }
}

```



```
cin.getline (str2,15);
}

len()
{
return (strlen(str1));}

void show()
{
cout<<"\nFirst String : "<<str1;
cout<<"\nSecond Strng : "<<str2;
cout<<"\nThird String : "<<str3;
}
combine (int x,int y)
{
str3[x]=str1[x];
if (x>=y)
str3[x]=str2[x-y];
if (str2[x-y]=='\0')
return 0;
else
return 1;
}
};

int main()
{
clrscr();
int k=-1,y;
text s;
s.get();
y=s.len();
while (s.combine (++k,y));
s.show();
return 0;
}
```

OUTPUT

Enter First String : CPLUS
Enter Second String : PLUS
First String : CPLUS
Second String : PLUS
Third String : CPLUSPLUS

Explanation: In the above program, the `class text` has three character array data members, namely `str1[15]`, `str2[15]`, and `str3[30]`, and has also four member functions, namely `get()`, `len()`, `show()`, and `combine()`. The `get()` function is used to read strings through the keyboard; the `len()` function returns the length of the first string; the `show()` function displays the strings; and the `combine()` function combines the first and second strings and assigns to the third string. The integer variable `k` is initialized to `-1`. The integer variable `y` contains the length of the first string. The function `combine()` is called within the bracket of `while` loop and two integers variable `k` and `y` are passed.

In `combine()` function, `x` indicates character element position in character arrays `str3[]` and `str1[]`. The statement `str3[x]=str1[x]` assigns characters of `str1[]` array-to-array `str3[]`. This assignment continuous till `x >= y` (`y` is length of the first string). When `x` is greater than `y`, the statement `str3[x]=str2[x-y]` is executed; that is, assignment of second string is now carried out.

The `x-y` displays the element number of second string. The second `if` statement in the `combine()` function checks whether the null character has met or not. If yes it returns `0`, otherwise `1`. The values `0` or `1` returned by `combine()` are collected by the `while` loop. The `while` loop is executed till it gets `1` from function `combine()`. When the `combine()` function returns `0`, the `while` loop is terminated. The `show()` function after `while` loop displays all the strings.



TIP

The reader may be in confusion as to why so much effort is required to write simple programs in C++. We can easily solve these problems with very short codes in C. The objective is to master how to solve these problems using OO concepts and to make the reader think more on objects. In the previous examples, the reader might have noticed that even for a small task, it might be calculation or comparison, and for every task, we defined member functions. The `class` should provide every operation in the form of method or member function needed by objects and the object should not be dependent on `main()` or other non-member function for any requirement. This is the pure object-oriented programming.

8.45 Write a program to declare data member of a `class` as public. Initialize and display them without using function.

```
#include<iostream.h>
#include<conio.h>
class boy
{ public:
    int weight;
    float height;
};
int main()
{
    clrscr();
    boy raj, sonu;
```

```

raj.weight=35;
raj.height=4.5;

sonu.weight=30;
sonu.height=4.1;
cout<<"\n Weight of Raj = "<<raj.weight;
cout<<" Height of Raj = "<<raj.height;
cout<<"\n Weight of Sonu = "<<sonu.weight;
cout<<" Height of Sonu = "<<sonu.height;
return 0;
}

```

OUTPUT

Weight of Raj = 35 Height of Raj = 4.5
Weight of Sonu = 30 Height of Sonu = 4.1

Explanation: In the above program, the class boy contains two public members, namely weight and height. The members are public and can be accessed directly without using member function. In function raj and sonu are two objects of the class boy. The data elements of weight and height of both objects are initialized and displayed.

8.46 Write a program to declare constant member function arguments.

```

#include<iostream.h>
#include<conio.h>
class data
{
    private :
    int d;
    public :
    void set() { d=10; }
    void show() { cout<<endl<<"d="<<d; }
    void sub (data const &a, data const &b)
    { d=a.d-b.d; }
};
int main()
{
    clrscr();
    data a,b,c;
    a.set();
    b.set();
    c.sub(a,b);
    c.show();
}

```

```
    return 0;
}
```

OUTPUT

D=0

Explanation: In the above program, the arguments of function `sub()` are declared as constant. Hence, an attempt to modify these arguments will generate an error. Thus, by declaring the member variable arguments `const`, we can prevent them from modification.

8.47 Write a program to show the difference between private and public data members of a class.

```
#include<iostream.h>
#include<conio.h>
class player
{
    private :
        char name[20];
        int age;
    public :
        float height;
        float weight;
};
int main()
{
    clrscr();
    class player a;
    // a.name ="Sanjay" ; // not accessible
    a.height=5.5;
    a.weight=38;
    cout<<"Height : " <<a.height;
    cout<<"\nWeight : " <<a.weight;
    return 0;
}
```

OUTPUT

Height : 5.5

Weight : 38

Explanation: In the above program, a class `player` is defined with four member variables `char name[20]`, `int age`, `float height`, and `float weight`. The first two members are private and last two members are public. In the function `main()`, `a` is an object

of type class player. The public member variables of class player are height and weight initialized with 5.5 and 38, respectively, through object a. It is not possible to access the private member of the class directly. Hence, the variables name and age cannot be accessed. Any attempt to access them through the object will display an error message “‘player::name’ is not accessible” or “‘player::age’ is not accessible”.

8.48 Write a program to initialize private and public member variables of the class. Display the contents of member variables.

```
#include<iostream.h>
#include<conio.h>
class num
{
    int x;
    float y;

    public :
    char z;
    void readdata(int j , float k)
    {
        x=j;
        y=k;
    }

    void display()
    {
        cout<<"x="<<x <<" y="<<y;
    }
};

int main()
{
    clrscr();
    class num j;
    j.z='C';
    j.readdata(10,10.5);
    j.display();
    cout<<" z= "<<j.z;
    return 0;
}
```

OUTPUT

x=10 y=10.5 z= C

Explanation: In the above example, the class num contains private as well as public member variables. The two private member variables x and y are assigned values using member

functions. The public member variable `z` is assigned directly in function `main()`. The variable `j` is an object, `readdata()` is a member function. The variables `j` and `k` are parameters to be passed. Consider the following statements.

- a) `j.y=10.5 // invalid statement`
- b) `j.z='C' // valid statement`

The statement (a) is invalid because member variable `y` is private whereas statement (b) is a valid because member variable `z` is public and accessible by the object of the same class.

8.34.1 Member Function Inside the class

8.49 Write a program to declare class with member variables and functions. Read and display the data using the member functions.

```
#include<iostream.h>
#include<conio.h>
class player
{
    private :
        char name[20];
        int age;
        float height;
        float weight;
    public:
        void setdata()
        {
            cout<<"Enter Name Age Height Weight \n";
            cin>>> name >>age >> height >> weight;
        }

        void show()
        {
            cout<<"\nName : " <<name;
            cout<<"\nAge : " <<age;
            cout<<"\nHeight : " <<height;
            cout<<"\nWeight : " <<weight;
        }
};

int main()
{
    clrscr();
    class player a;
    a.setdata();
    a.show();
}
```

```
    return 0;
}
```

OUTPUT

```
Enter Name Age Height Weight
Sanjay 24 5.5 54
Name :Sanjay
Age :24
Height :5.5
Weight :54
```

Explanation: In the above program, the class `player` contains four private member variables and two public member functions `setdata()` and `show()`. The definition of both the functions is inside the class. In the previous example, we noticed that the object of any class could not access the private member variables of the class. To access the private member variables of the class, member function of that class are used. In this program, `setdata()` and `show()` are the member functions of class `player`. The `setdata()` function reads data through the keyboard and `show()` functions display the data on the screen. These member functions cannot be called directly as ordinary functions. Consider the statement `a.setdata()`. Here, `a` is an object of class `player` followed dot `.` operator and function name. An attempt to call these member functions directly without using object of that class will generate an error in the program such as “Function ‘ ‘ should ‘ ‘ have a prototype”.

In this program, function definition of the member function is inside the class. It is also possible to put prototype of the function inside the class and definition outside the class. While defining the function, it is necessary to specify class name followed by scope access operator `.`. This declaration tells the compiler the relation between the class and the member function. For understanding this concept, the following program is illustrated as follows.

8.34.2 Member Function Outside the class

8.50 Write a program to declare class with member variables and functions. Read and display the data using the member functions. Declare function definition outside the class.

```
#include<iostream.h>
#include<conio.h>
class player
{
    private :
    char name[20];
    int age;
    float height;
    float weight;
    public:
```

```
void setdata();
void show();
};
void player::setdata()
{
    cout<<"Enter Name Age Height Weight \n";
    cin>>> name >>age >> height >> weight;
}
void player::show()
{
    cout<<"\nName :" <<name;
    cout<<"\nAge :" <<age;
    cout<<"\nHeight :" <<height;
    cout<<"\nWeight :" <<weight;
}
int main()
{
    clrscr();
    class player a;
    a.setdata();
    a.show();
    return 0;
}
```

OUTPUT

```
Enter Name Age Height Weight
Ajay 24 5.2 45
Name :Ajay
Age :24
Height :5.2
Weight :45
```

Explanation: The above program is same as previous one. The only difference is that the function definition of the member functions is outside the class. The prototypes of these functions are inside the class, which are enough for these functions to join membership with class player.

Consider the following statements.

```
void player:: setdata()
Void player:: show()
```

Both these statements tell the compiler that the functions `setdata()` and `show()` are member functions of class player.

So far we declared the entire member functions as public. It is also possible to declare member functions as private. A private function can be called only from public member functions. Even an

object of same class cannot access the private function. An example is illustrated below based on this idea.

8.34.3 Private Member Functions

8.51 Write a program to declare private member function and call this member function using another public member function.

```
#include<iostream.h>
#include<conio.h>
#include<math.h>
class num
{
    private :
        int x;
        int sqr(int) ;
    public:
        input()
        {
            int n;
            cout<<"Enter a Non-zero Number :";
            cin>>n;
            return sqr(n) ;
        }
};
num :: sqr (int k)
{
    return k*k;
}
int main()
{
    clrscr();
    int sq;
    class num j;
    sq=j.input();
    cout<<"\nSquare of "<<sqr(sq) <<" is " <<sq;

    // j.sqr(5); not accessible

    return 0;
}
```

OUTPUT

Enter a Non-zero Number : 25
Square of 25 is 625

Explanation: In the above program, the function `sqr()` is declared as `private` and the function `input()` as `public`. The `input()` function reads an integer through the keyboard and calls the `private` function `sqr()`. The function `sqr()` calculates the square of the number and returns result to the `input()` function. The `input()` function again returns this value to the variable `sq` declared in `main()`. The last `cout` statement displays the entered number and its square. The statement given in remark is invalid statement, because an object cannot access `private` member. The `input()` `public` function acts as an inter-mediator in between `private` function and an object of the class. Without `public` functions, it is not possible for the object to access `private` functions.

8.52 Write a program to enter hours. Convert it into seconds and minutes. Display results.

```
#include<iostream.h>
#include<conio.h>
class hour
{
    int hours;
    int minutes;
    int seconds;
    public :
    void input(void);
    void show(void);
    void convert();
};
void hour::input()
{
    cout<<"\n Enter Hour: ";
    cin>>hours;
}
void hour::show()
{
    cout<<"Hour = " <<hours <<"Minute = " <<minutes <<"Second = " <<seconds;
}
void hour::convert()
{ minutes=hours*60;
  seconds=minutes*60;}
main()
{ clrscr();
  hour X,Z;
  cout<<"\n Object X";
  X.input();
  X.convert();
  cout<<"\nX: ";
```

```

    X.show();
    return 0;
}

```

OUTPUT

Object X

Enter Hour : 4

X : Hour = 4 Minute = 240 Second = 14400

Explanation: In the above program, the `class hour` is declared with three integer members and three member functions. The `input()` function reads number of hours through the keyboard. The function `convert()` calculates minutes and seconds and assigns results to minutes and seconds variables. The function `show()` displays the contents of all the member variables. In the function `main()`, `X` is an object of `class hour`. The object `X` calls all the member functions one by one. The values of member variables after execution are displayed at the output.

8.53 Write a program to count the number of vowels present in the entered string.

```

#include<stdio.h>
#include<iostream.h>
#include<conio.h>
#include<ctype.h>
#include<string.h>
struct vowels
{
    private :
    char str [20];
    public :
    void input ()
    {
        cout<<"Enter text in small case : ";
        gets(str);
    }
    length()
    {
        int l=strlen(str);
        return l;
    }
    vowel (int x)
    {
        if (str[x]=='a' || str[x]=='e' || str[x]=='i' || str[x]=='o' ||
            str[x]=='u')
        {
            cout<<" " <<str[x];
            return 1;
        }
    }
}

```

```

    }
    else
    return 0;
    }
};
main()
{
    clrscr();
    int i,l,c=0;
    vowels b;
    b.input();
    l=b.length();
    for (i=0;i<l;i++)
    {
        c=c+b.vowel(i);
    }
    cout<<"\n"<<c<<" Vowels are present in the string";
    return 0;
}

```

OUTPUT

Enter text in small case : Programming

o a i

3 Vowels are present in the string

Explanation: In the above program, instead of `class` keyword `struct` keyword is used. The `struct vowels` has only one member variable, which is a character array `str`. The function `input()` reads text through the keyboard and assigns it to the member variable `str[20]`. The function `length()` determines the length of the string. The function `vowel()` when called receives an integer from calling function. The `if` statement checks whether the (**x**) the character of the string is vowel or consonant. If it is one of the vowels, the function returns 1 otherwise 0.

In `main()`, `b` is an object of `struct vowels`. Using `for` loop, the object `b` calls the function `vowel()` repetitively. The return value (0 or 1) is added to a variable `c` during repetitive calls. The value of `c` finally displays the number of vowels present in the string.

8.54 Write a program to enter text. Find a given character in the string and replace it with another given character.

```

#include<stdio.h>
#include<iostream.h>
#include<conio.h>
#include<string.h>
class fandr

```



```
{
    char str [40];
    char f;
    char r;
    public :

    void accept ()
    {
        cout<<" Enter text : ";
        cin.getline(str,40);
        cout<<" Find what (char) : ";
        f=getche();
        cout<<"\n Replace with (char) : ";
        r=getche();
    }

    void display (int d)
    {
        cout.put (str[d]);
    }

    len()
    { int l=strlen(str);
      return(l);
    }

    void find ( int i)
    {
        if (str[i]==f)
            replace(i);
    }

    void replace ( int k)
    { str[k]=r; }
};

main()
{
    clrscr();
    int i,l;
    fandr b;
    b.accept();
    l=b.len();
    cout<<"\n Replaced text : ";
    for (i=0;i<l;i++)
    {
        b.find(i);
        b.display(i);
    }
}
```

```

    }
return 0;
}

```

OUTPUT

Enter text : Progra__er

Find what (char) : _

Replace with (char) : m

Replaced text : Programmer

Explanation: In the above program, the class `fandr` have three character data type member variables. The function `accept()` reads the text through the keyboard. Here, the function `cin.getline(str, 40)` is used to connect with the keyboard. The first argument is the name of array and second is size of array. Followed by this, the character to be traced and replaced is entered in variable `f` and `r`. The function `len()` is obviously used for calculating length of the string. The function `find()` receives an integer from calling function. The `if` statement checks whether the specified character by the integer is equal to a given character or not. If the character is found, the `replace()` function is called which replaces the traced character with value of '`r`' variable. The `for` loop in `main()` function repetitively calls the functions `find()` and `display()`. Remember, the function `replace()` is called by `find()` function whenever the `if` condition is true. The `cout.put()` is used to display the character on the screen. Thus, find and replace operations execute.

8.55 Write a program to find largest out of ten numbers.

```

#include<iostream.h>
#include<conio.h>
#include<process.h>
class num {
int number[10];
public :
input (int i)
{
    cout<<" Enter Number ("<i+1<<" : " ;
    cin>>>number[i];
    return(number[i]);
}

void large ( int s, int m)
{
    if (number[s]==m)
    {
        cout<<"\n Largest number : "<<number[s];
        exit(1);
    }
}

```

```
}  
};  
  
main()  
{  
clrscr();  
int i, sum=0, k;  
num b;  
  
for (i=0; i<10; i++)  
sum=sum+ b.input(i);  
  
for (k=sum; k>=0; k--)  
{  
    for (i=0; i<10; i++)  
        b.large(i, k);  
}  
return 0;  
}
```

OUTPUT

```
Enter Number (1) :125  
Enter Number (2) :654  
Enter Number (3) :246  
Enter Number (4) :945  
Enter Number (5) :258  
Enter Number (6) :159  
Enter Number (7) :845  
Enter Number (8) :940  
Enter Number (9) :944  
Enter Number (10) :485  
Largest number : 945
```

Explanation: In the above program, the class num has one integer array as its member variable with element size 10 (number[10]). The input() function reads an integer through the keyboard and returns it to main() function. The first for loop repetitively calls function input(). The return value of input() function is added to variable sum. The second and third for loops together are used to find the largest number in the array. The second and third for loop variables are used as arguments for the function large(). As per iteration, the successive element numbers and decremented value of sum assigned to variable k. The values of variables k and loop variable i are sent to function large(). In each third for loop iteration, the second for loop executes once and value of sum is decremented. The if condition in large() function checks both the arguments. If the arguments are the same, the largest number is displayed, and the exit(1) function terminates the program, otherwise the program execution continues.

8.56 Write a program to count the numbers between 1 and 100, which are not divisible by 2,3, and 5.

```
#include<stdio.h>
#include<iostream.h>
#include<conio.h>
class div
{
    static int num;
    public :
    void check(int n)
    {
        if (n%2!=0 && n%3!=0 && n%5!=0)
        {
            cout<<n <<"\t";
            num++;
        }
    }
    void show()
    { cout<<endl <<"\n Total numbers : "<<num; }
    void loop();
};
void div::loop()
{
    int x;
    cout<<endl<<"\n Numbers from 1 to 100 not divisible by 2,3 & 5\n\n";
    for ( x=0 ;x<=100;x++)
        check(x);
}
int div::num=0;
int main()
{
    clrscr();
    int x;
    div d;
    d.loop();
    d.show();
    return 0;
}
```

OUTPUT

Numbers from 1 to 100 not divisible by 2,3 & 5

1 7 11 13 17 19 23 29 31 37

```

41 43 47 49 53 59 61 67 71 73
77 79 83 89 91 97
Total numbers : 26

```

Explanation: In the above program, the class `div` is declared with one static data member variable, that is `num`. The class `div` also contains member functions `check()`, `show()`, and `loop()`. The function `check()` checks the received value with `if` statement whether it is divisible by 2, 3, and 5 or not. If it is not divisible, the number is displayed and the value of static data member `num` is incremented. The `loop()` function contains `for` loop and calls the function `check()` with one argument. The prototype of function is given inside the class and definition outside the function, because the member function with any loop defined inside the class cannot be expanded inline and will result in a warning message. The more applicable practice is to write the definition of this kind of function outside the class. The function `show()` displays the total number of numbers that satisfies the condition. The object `d` declared in function `main()` invokes the member functions `loop()` and `show()`.

8.57 Write a program to generate the n^{th} Fibonacci number by using recursion.

Fibonacci series: 1 1 2 3 5 8 13

```

#include<iostream.h>
#include<conio.h>
class Fibonacci
{
    public:
    fib(int x)
    {
        int f;
        if (x==0)
            return 0;
        if (x==1)
            return 1;
        else
            f=fib(x-1)+fib(x-2);
        return f;
    }
};
int main()
{
    clrscr();
    Fibonacci a;
    int y;
    cout<<"\nEnter the number:";

```

```
cin>>y;
cout<<"\nSum of Fibonacci numbers upto "<<y<<" is "<<a.fib(y) ;
getch();
return 0;
}
```

Output:

Enter the number:7

Sum of Fibonacci numbers upto 7 is 13

SUMMARY

- (1) A `class` in C++ is similar to `structure` in C. Using `class` or `structure`, a programmer can merge one or more dissimilar data types and a new custom data type can be created.
- (2) In C++, `classes` and `structures` contain member variables and member functions in their declarations with `private` and `public` access blocks that restrict the unauthorized use. The defined `classes` and `structures` further can be used as custom data type in the program to declare objects.
- (3) The `private` and `public` are new keywords in C++. The `private` keyword is used to protect specified data and functions from illegal use whereas the `public` keyword allows access permission.
- (4) The member function can be defined as `private` or `public` inside the `class` or outside the `class`.
- (5) To access `private` data members of a `class`, member functions are used.
- (6) The difference between member function and normal function is that the normal function can be invoked freely, whereas the member function can be invoked only using the object of the same `class`.
- (7) The `static` is a keyword used to preserve the value of a variable. When a variable is declared as `static`, it is initialized to zero. A `static` function or data element is only recognized inside the scope of the present compile.
- (8) When a function is defined as `static`, it can access only `static` member variables and functions of the same `class`. The `static` member functions are called using its `class` name without using its objects.
- (9) The member functions of a `class` can also be declared as `constant` using `const` keyword. The constant functions cannot modify any data in the `class`.
- (10) An object of a `class` can be passed to the function as arguments like variables of other data type. When an object is passed-by-value, then this method is called as `pass-by-value`, whereas when reference of an object is passed to the function then this method is called as `pass-by-reference`.
- (11) When `classes` are declared inside a function, then such `classes` are called as `local classes`. The `local classes` have access permission to `global` variables as well as `static` variables. The `local classes` should not have `static` data member and functions.

EXERCISES

(A) Answer the following questions

- (1) Explain `class` and `struct` with their differences.
- (2) Which operators are used to access members?
- (3) Explain the uses of `private` and `public` keywords. How are they different from each other?
- (4) Explain the features of a member function.
- (5) What are `static` member variables and functions?
- (6) How are `static` variables initialized? Explain with a statement.
- (7) What are `friend` functions and `friend` classes?
- (8) How are `static` functions and `friend` functions invoked?
- (9) What do you mean by `constant` function?
- (10) What are local classes?
- (11) What is recursion?
- (12) What are bit fields?
- (13) List the keywords terminated with a colon with their use.
- (14) Can member functions be `private`?
- (15) What is the concept of data hiding? What are its advantages in applications?
- (16) Is it possible to access `private` data members without using member function? If yes, explain the procedure with an example.
- (17) What are `static` objects?
- (18) What is the difference between object and variable?

(B) Answer the following by selecting the appropriate option

- (1) The members of a `class` are by default
 - (a) **private**
 - (b) `public`
 - (c) `protected`
 - (d) none of the above
- (2) `class` data members can be accessed by
 - (a) **public member functions**
 - (b) `private` member function
 - (c) both (a) and (b)
 - (d) none of the above
- (3) The objects of `class` can directly access
 - (a) **public members**
 - (b) `private` members
 - (c) `protected` members
 - (d) all of the above
- (4) The `private` data members can be accessed through functions declared under
 - (a) `private` section
 - (b) **public section**
 - (c) `protected` section
 - (d) all of the above
- (5) The `protected` data members can be accessed by `private` and `protected` member functions using
 - (a) non-member functions
 - (b) **public member functions**
 - (c) directly by object
 - (d) scope resolution operator
- (6) For data hiding, data are generally declared in
 - (a) **private section**
 - (b) `public` section
 - (c) both (a) and (b)
 - (d) none of the above
- (7) `class` declaration provides
 - (a) data hiding
 - (b) abstraction
 - (c) encapsulation
 - (d) **all of the above**
- (8) Member functions can be defined
 - (a) only inside the `class`
 - (b) only outside the `class`

- (c) **inside as well as outside class**
 - (d) none of the above
- (9) A `class` contains
- (a) only data
 - (b) only functions
 - (c) **data and functions**
 - (d) neither data nor functions
- (10) When a `class` is defined, memory allocation is done
- (a) at the same time
 - (b) **when objects are declared**
 - (c) when functions are called
 - (d) when values are passed
- (11) Private data are accessible to
- (a) objects directly
 - (b) **objects through public member functions**
 - (c) main function
 - (d) none of the above
- (12) Public data members and functions are accessible by
- (a) objects directly
 - (b) private members
 - (c) external function
 - (d) **both (a) and (c)**
- (13) Inline function is not possible when
- (a) a function contains a loop
 - (b) `goto` statement exists
 - (c) **both (a) and (b)**
 - (d) none of the above
- (14) Functions defined outside the `class` can be accessed using
- (a) **scope resolution operator**
 - (b) logical operator
 - (c) reference
 - (d) arithmetic operator
- (15) Inline function
- (a) **saves memory**
 - (b) takes extra time
 - (c) executes slowly
 - (d) wastes memory
- (16) An inline function
- (a) duplicates code
 - (b) **speeds up the execution**
 - (c) both (a) and (b)
 - (d) neither (a) nor (b)
- (17) When inline function is called
- (a) control is passed to function
 - (b) function is executed directly
- (c) **function code is replaced at that point**
- (d) none of the above
- (18) The public member function can access
- (a) private data
 - (b) public data
 - (c) protected data
 - (d) **all of the above**
- (19) Public data can be accessed by
- (a) private member function
 - (b) public member function
 - (c) protected member function
 - (d) **all of the above**
- (20) By default, all member functions defined inside the `class` are treated as
- (a) **inline function**
 - (b) external functions
 - (c) main function
 - (d) none of the above
- (21) An encapsulated object is often called as
- (a) **abstract data type**
 - (b) abstract `class`
 - (c) both (a) and (b)
 - (d) neither (a) nor (b)
- (22) When one member function is called inside another member function, it is called
- (a) cascading functions
 - (b) referencing functions
 - (c) **nesting member function**
 - (d) none of the above
- (23) static data member is stored in memory as
- (a) **only one copy**
 - (b) number of copies
 - (c) two copies
 - (d) no copy
- (24) When a variable is declared as `static`, it is
- (a) initialized to one
 - (b) **initialized to zero**
 - (c) not initialized
 - (d) none of the above
- (25) A static data member can be recognized
- (a) **inside scope of class**
 - (b) outside the `class`

- (c) in `main()` function
(d) in non-member function
- (26) When a variable is declared as `static`, objects share
(a) **common data**
(b) different data
(c) no data
(d) none of the above
- (27) A `static` variable is accessible only within
(a) **class**
(b) `main()` function
(c) non-member function
(d) none of the above
- (28) The memory for `static` data member is allocated
(a) **only once**
(b) twice
(c) thrice
(d) number of times
- (29) If `count` is a `static` data member and `item` is a class, then
`int item::count;`
what is the value of `count` initially?
(a) **zero**
(b) one
(c) no value
(d) cannot be determined.
- (30) Initialization of `static` data member must be done
(a) inside the `class`
(b) **right after class definition**
(c) in `main()` function
(d) in member function
- (31) Which `static` data member can be accessed by objects of the same `class`?
(a) `private`
(b) `protected`
(c) **public**
(d) none of the above
- (32) A `static` member function can access
(a) **only static member variable**
(b) non-`static` member variable
(c) `static` as well as non-`static` variables
(d) only non-`static` member variable
- (33) A `static` member function can be called by
(a) objects only
(b) `class` name and scope resolution operator
(c) **object as well as class name and scope resolution operator**
(d) none of the above
- (34) A `private static` member function can be accessed by
(a) `private static` member function
(b) `public static` member function
(c) objects of `class`
(d) **both (a) and (b)**
- (35) A non-member function that can access the `private` data of `class` is known as
(a) **friend function**
(b) `static` function
(c) member function
(d) library function
- (36) The size of object is equal to
(a) **total size of data variables**
(b) total size of member functions
(c) both (a) and (b)
(d) none of the above
- (37) The `class` without any data members or member functions is called as
(a) **empty class**
(b) non-empty `class`
(c) both a and b
(d) none of the above

(C) Attempt the following programs

- (1) Write a program to declare a `class` with three integer public data variables. Initialize and display them.
- (2) Write a program to declare private data member variables and public member function. Read and display the values of data member variables.

- (3) Write a program to declare private data member and a function. Also declare a public member function. Read and display the data using private function.
- (4) Write a program to declare three classes S1, S2, and S3. The classes have a private data member variable of character data type. Read strings for the classes S1 and S2. Concatenate the strings read and assign it to the data member variable of class S3.
- (5) Write a program to enter positive and negative numbers. Enter at least 10 numbers. Count the positive and negative numbers using classes and objects.
- (6) Write a program to declare class with private member variables. Declare member function as static. Read and display the values of member variables.
- (7) Write a program to declare a class temp_ture as given below. Declare an array of five objects. Read and display the data using arrays.


```
class temp_ture
{
    private:
    char date[12],
    ct1[15], ct2[15],
    ct3[15], ct4[15];
    int temp[4]
}
```
- (8) Write a program to declare a class with two integers. Read values using a member function. Pass the object to another member function. Display the difference between them.
- (9) Write a program to define three classes. Define a friend function. Read and display the data for three classes using common functions. Use friend functions.
- (10) Write a program to define a local class. Also define global variables with the same name as that of member variables of class. Read and display the data for global and member variables.
- (11) Write a program to generate Fibonacci series using recursion with member function.
- (12) Write a program to overload a member function and display the string, int, and float using overloading function.
- (13) Write a program to calculate sum of digits of an entered number using indirect recursion. Recursion should be between main() and member function.
- (14) Write a program to display a string in reverse using recursion.
- (15) Write a program to add numbers from 1 to 10.
- (16) Write a program to find area of a right angle triangle.
- (17) Write a program to compute area and perimeter of a square.
- (18) Write a program to compute average marks obtained by a student in five subjects.
- (19) Write a program to display even numbers from 1 to 10.
- (20) Write a program to display numbers divisible by 5 between 1 and 100.
- (21) Write a program to display numbers divisible by 11 and 3 between 1 and 100.
- (22) Write a program to find area of a cube.
- (23) Write a program to find negative and positive numbers of an array.

(D) Find the bugs in the following programs

(1)

```
class data
{
    private;
};
```

(2)

```
class data
{
    private:
    int x=20;
}
```

(3)

```
#include<iostream.h>
#include<conio.h>
class data
{ int x; };
void main()
{ data A;
  A.x=30; }
```

(4)

```
#include<iostream.h>
#include<conio.h>
class data
{ int x;
  public:
    void show(void);
};
void show() { cout<<"\n
In show() "; }
void main()
{ data A;
  A.show(); }
```

(5)

```
class
{
  public:
    void print()
    { cout<<a; }
};
void main()
{
}
```

Constructors and Destructors

9

CHAPTER

CHAPTER OUTLINE

- 9.1 Introduction
- 9.2 Constructors and Destructors
- 9.3 Characteristics of Constructors and Destructors
- 9.4 Applications with Constructors
- 9.5 Constructors with Arguments (Parameterized Constructor)
- 9.6 Overloading Constructors (Multiple Constructors)
- 9.7 Array of Objects Using Constructors
- 9.8 Constructors with Default Arguments
- 9.9 Copy Constructors
- 9.10 The `const` Objects
- 9.11 Destructors
- 9.12 Calling Constructors and Destructors
- 9.13 Qualifier and Nested Classes
- 9.14 Anonymous Objects
- 9.15 Private Constructors and Destructors
- 9.16 Dynamic Initialization Using Constructors
- 9.17 Dynamic Operators and Constructors
- 9.18 `main()` as a Constructor and Destructor
- 9.19 Recursive Constructors
- 9.20 Program Execution before `main()`
- 9.21 Constructor and Destructor with Static Members
- 9.22 Local Versus Global Object
- 9.23 More Programs

9.1 INTRODUCTION

When a variable is declared and if not initialized, it contains garbage value. The programmer needs to assign appropriate value to the variable. The compiler itself cannot carry out the process of initialization of a variable. The programmer needs to explicitly assign a value to the variable. Initialization prevents the variable from containing garbage value.

Consider an example

```
float height;    // variable declaration
height=5.5;     // assigning value to variable
```

In the above example, `height` is a variable of `float` type. It holds any garbage value before initialization. In the next statement, variable `height` is initialized with 5.5.

C++ handles abstract data type, which is the combination of one or more basic data types. An object holds copies of one or more individual data member variables. When an object is created, its data member contains garbage value.

We learnt in the previous chapter that declaring static member variables facilitates the programmer to initialize member variables with desired values. The drawback of static members is that only one copy of the static member is created for the entire class. All objects share the same copy, which does not provide security.

Another approach is to define an object as static. When an object is declared as static, all its member variables are initialized to zero. This is a useful approach. Declaring static object does not create common copies of member variables. Every static object has its own set of member variables. The drawback of static object is that the contents of static object remain throughout the program occupying more memory space. The following program illustrates this point.

9.1 Write a program to declare static objects and display their contents.

```
#include<iostream.h>
#include<conio.h>

class data
{
    int x;
    float y;
public:
    void show()
    {
        x++;
        y++;
        cout<<"\n x="<<x <<"\n y="<<y;
    }
};

void main()
{
```

```

    clrscr();
    static data A,B;
    cout<<"\n Object A";
    A.show();
    cout<<"\n Object B";
    B.show();
}

```

OUTPUT

Object A

x=1

y=1

Object B

x=1

y=1

Explanation: In the above program, objects A and B are declared as static. Their data member variables are initialized to zero. The member function `show()` increments value of member variables by one and displays them. Both the objects A and B invoke the function `show()`. The contents displayed are same for both the objects. Therefore, we can say that individual copies are created for each static object.

The main drawback of static object is that its value remains in the memory throughout the program. The following program illustrates that the static object remains in the memory even if it goes out of scope.

9.2 Write a program to demonstrate that static object persists its values.

```

#include<iostream.h>
#include<conio.h>

class data
{
    int x;
    float y;
public:
    void show()
    {
        x++;
        y++;
        cout<<"\n x="<<x <<"\n y="<<y;
    }
};

void main()
{

```

```
clrscr();
void display (void);
display();
display();
}
void display()
{
    static data K;
    K.show();
}
```

OUTPUT

```
x=1
y=1
x=2
y=2
```

Explanation: In the above program, function `display()` is a normal function. The member function `show()` performs the same task as described in the previous program. The static object `K` is declared in the function `display()`. The object `K` is local object of function `display()`. The function `main()` invokes function `display()` two times. In the first call, the contents displayed are one and one. In the second call, the contents displayed are two and two. It means in second call, the previous values are not cleared. Hence, we can say that static objects are intact or persist their values. The static object `K` remains throughout the program and is not destroyed even if execution of `display()` terminates.

9.2 CONSTRUCTORS AND DESTRUCTORS

In the previous chapter, we defined a separate member function for reading input values for data members. Using object, member function is invoked and data members are initialized. The programmer needs to call the function. C++ provides a pair of in-built special member functions called **constructor** and **destructor**. The constructor constructs the objects and destructor destroys the objects. In operation, they are opposite to each other. The compiler automatically executes these functions. The programmer does not need to make any effort for invoking these functions.

The C++ run-time arrangement takes care of execution of constructors and destructors. When an object is created, constructor is executed. The programmer can also pass values to the constructor to initialize member variables with different values. The destructor destroys the object. The destructor is executed at the end of the function when objects are of no use or go out of scope. It is optional to declare constructor and destructor. If the programmer does not define them, the compiler executes implicit constructor and destructor.

Constructors and destructors are special member functions. They decide how the objects of a class are created, initialized, copied, and destroyed. Their names are distinguished from all other member functions because their names are same as the class they belong to. The only difference is that destructor is preceded by a `~` (tilde) operator.

Constructors and destructors have many attributes as that of normal member functions. We can declare and define them within the class, or declare them within the class and define them outside; however, they have few unique characteristics.

9.2.1 Constructors

If a `class B` has one or more constructors, one of them is invoked each time when we define an object `b` of `class B`. The constructor creates object `b` and initializes it. Constructors are also called when local or temporary objects of a class are created.

Example

```
B () { }
```

9.2.2 Destructors

Destructors are opposite to the constructor. The process of destroying the class objects created by constructors is done in destructor. The destructors have the same name as their class, preceded by a tilde (~). A destructor is automatically executed when object goes out of scope. It is also invoked when delete operator is used to free the memory allocated with class pointer. Similar to the constructor, it is not possible to define overloaded destructors and pass arguments to them. The class can have only one destructor. Destructors are called when these objects go out of scope.

Example

```
~B () { }
```

9.3 CHARACTERISTICS OF CONSTRUCTORS AND DESTRUCTORS

9.3.1 Constructors

- (1) Constructors have the same name as that of the class they belong to.
- (2) Constructors are executed when an object is declared.
- (3) Constructors have neither `return` value nor `void`.
- (4) The main function of constructor is to initialize objects and allocate appropriate memory to objects.
- (5) Though constructors are executed implicitly, they can be invoked explicitly.
- (6) Constructors can have default values and can be overloaded.
- (7) The constructor without arguments is called as default constructor.

9.3.2 Destructors

- (1) Destructors have the same name as that of the class they belong to preceded by ~ (tilde).
- (2) Similar to constructors, the destructors do not have `return` type and not even `void`.
- (3) Constructors and destructors cannot be inherited, though a derived class can call the constructors and destructors of the base class.
- (4) Destructors can be virtual, but constructors cannot.

- (5) Only one destructor can be defined in the destructor. The destructor does not have any arguments.
- (6) Destructors neither have default values nor can be overloaded.
- (7) Programmer cannot access addresses of constructors and destructors.
- (8) TURBO C++ compiler can define constructors and destructors if they have not been explicitly defined. They are also called on many cases without explicit calls in programs. Any constructor or destructor created by the compiler will be public.
- (9) Constructors and destructors can make implicit calls to operators `new` and `delete` if memory allocation/de-allocation is needed for an object.
- (10) An object with a constructor or destructor cannot be used as a member of a union.

9.4 APPLICATIONS WITH CONSTRUCTORS

The initialization of member variables of class is carried out using constructors. The constructor also allocates required memory to the object. An example of a constructor is as follows:

```
class num
{
    private:
        int a, b,c;
    public:

        num (void); // declaration of constructor
        - - - - -
        - - - - -
};
num:: num (void) // definition of constructor
{
    a=0;b=0;c=0; // value assignment
}
main()
{class num x;
}
```

In the above example, class `num` has three member integer variables `a`, `b`, and `c`. The declaration of constructor can be done inside the class and definition outside the class. In definition, the member variables of a class `num` are initialized to zero.

In the function `main()`, `x` is an object of type class `num`. When an object is created, its member variables (private and public) are automatically initialized to the given value. The programmer need not write any statement to call the constructor. The compiler automatically calls the constructors. If programmer writes a statement for calling a constructor, a constructor is called again. In case there is no constructor in the program, the compiler calls a dummy constructor. The constructor without argument is known as default constructor.

9.3 Write a program to define a constructor and initialize the class data member variables with constants.

```
#include<iostream.h>
#include<conio.h>

class num
{
    private:
        int a,b,c;
    public:
        int x;
        num(void); // declaration of constructor
        void show()
        { cout<<"\n x="<<x <<"a="<<a<<"b="<<b<<"c="<<c; }
};

num:: num (void) // definition of constructor
{
    cout<<"\n Constructor called";
    x=5; a=0;b=1;c=2;
}

main()
{
    clrscr();
    num x;
    x.show();
    return 0;
}
```

OUTPUT

Constructor called
x = 5 a= 0 b= 1 c= 2

Explanation: In the above program, the class `num` is declared with four integers `a`, `b`, `c`, and `x`. The variables `a`, `b`, and `c` are private and `x` is a public variable. The class also has `show()` function and constructor prototype declaration. The function `show()` displays the contents of the member variables on the screen. The definition of a constructor is done outside the class. In the function `main()`, `x` is an object of class `num`. When an object is created, constructor is automatically invoked and member variables are initialized to given values as per the constructor definition. The values of variables `x`, `a`, `b`, and `c` are as per the output shown. The compiler calls the constructor for every object created. For each object the constructor is executed once; that is, the number of times the execution of constructor is equal to the number of objects created. The below-given program explains this point.

9.4 Write a program to show that for each object constructor is called separately.

```
#include<iostream.h>
#include<conio.h>

class num
{
    private:
        int a;
    public:
        num (void) // definition of constructor
        {
            cout<<"\n Constructor called.";
            a=1;
            cout<<"a="<<a;
        }
};

main()
{
    clrscr();
    num x,y,z;
    num a[2];
    return 0;
}
```

OUTPUT

Constructor called. a = 1

Constructor called. a = 1

Constructor called. a = 1

Constructor called. a = 1

Constructor called. a = 1

Explanation: In the above program, x, y, and z are objects of class num. A[2] is an array of objects. For each individual object constructor is called. The total number of objects declared is five; hence, the constructor is called five times.

9.5 Write a program to read values through the keyboard. Use constructor.

```
#include<iostream.h>
#include<conio.h>

class num
{
    private:
        int a,b,c;
```

```
public:
num(void); // declaration of constructor
void show()
{ cout<<"\n"<<"a=" <<a <<"b="<<b <<"c="<<c; }
};
num:: num (void) // definition of constructor
{
    cout<<"\n Constructor called";
    cout<<"\n Enter Values for a,b and c:";
    cin>> a>>b>>c;
}
main()
{
    clrscr();
    class num x;
    x.show();
    return 0;
}
```

OUTPUT

Constructor called
Enter Values for a,b and c: 1 5 4
a= 1 b= 5 c= 4

Explanation: The above program is same as the previous one. In this program, whenever an object is created, the constructor is called and it reads the integer values through the keyboard. Thus, the entered constants are assigned to member variables of the class. Here, the constructor is used like other functions.

9.5 CONSTRUCTORS WITH ARGUMENTS (PARAMETERIZED CONSTRUCTOR)

In the previous example, constructors initialize the member variables with given values. It is also possible to create constructor with arguments, and such constructors are called parameterized constructors. For such constructors, it is necessary to pass values to the constructor when an object is created. Consider the following example:

```
class num
{
    private:
    int a, b, c;
    public:
```

```

num (int m, int j, int k); // declaration of constructor
with arguments
- - - - -
- - - - -
};
num:: num (int m, int j, int k) // definition of construc-
tor with arguments
{
    a=m;
    b=j;
    c=k;
}
main()
{
    class num x=num (4,5,7); // Explicit call
    class num y (9,5,7); // Implicit call
}

```

In the above example, the declaration and definition of a constructor contains three integer arguments. In the definition, the three arguments m, j, and k are assigned to member variables a, b, and c.

In a situation like this when class has a constructor with arguments, care is to be taken while creating objects. So far, we created the object using the following statement:

```
num x
```

This statement will not work. We need to pass required arguments as per the definition of constructor. Hence, to create object, the statements must be as given below:

```

(a) class num x=num (4,5,7); // Explicit call
(b) class num y (9,5,7); // Implicit call

```

The statements (a) and (b) can be used to create objects, which not only create objects but also pass given values to the constructor. The method (a) is called explicit call and the method (b) implicit call.

9.6 Write a program to create a constructor with arguments and pass the arguments to the constructor.

```

#include<iostream.h>
#include<conio.h>

class num
{
    private:

```

```

    int a,b,c;
    public:
        num(int m, int j, int k); // declaration of constructor with
arguments
        void show()
        {
            cout<<"\na="<<a <<"b="<<b <<"c="<<c;
        }
};
num:: num (int m, int j, int k) // definition of constructor with
arguments
{
    a=m;
    b=j;
    c=k;
}
main()
{
    clrscr();
    num x=num(4,5,7); // Explicit call
    num y(1,2,8); // Implicit call
    x.show();
    y.show();
    return 0;
}

```

OUTPUT

```

a= 4 b= 5 c= 7
a= 1 b= 2 c= 8

```

Explanation: In the above program, x and y are objects of class num. When objects are created, three values are passed to the constructor. These values are assigned to the member variables. The function show () displays the contents of member variables.

9.6 OVERLOADING CONSTRUCTORS (MULTIPLE CONSTRUCTORS)

Similar to functions, it is also possible to overload constructors. In the previous examples, we declared single constructors without arguments and with all arguments. A class can contain more than one constructor. This is known as constructor overloading. All constructors are defined with the same name as the class they belong to. All the constructors contain different number of arguments. Depending upon the number of arguments, the compiler executes appropriate constructor. Table 9.1 describes object declaration and appropriate constructor to it. Consider the following example:

```

class num
{
    private:
        int a;
        float b;
        char c;
    public:
        a) num (int m, float j , char k);
        b) num (int m, float j);
        c) num();
        d) class num x (4,5.5, 'A');
        e) class num y (1,2.2);
        f) class num z;
}

```

■ **Table 9.1** Overloaded Constructors

Constructor Declaration	Object Declaration
(a) num (int m, float j, char k);	(d) num x (4,5.5, 'A');
(b) num (int m, float j);	(e) num y (1,2.2);
(c) num();	(f) num z;

In the above example, the statements **(a)**, **(b)**, and **(c)** are constructor declarations and **(d)**, **(e)**, and **(f)** are the object declarations. The compiler decides which constructor to be called depending on the number of arguments present with the object.

When object **x** is created, the constructor with three arguments is called because the declaration of an object is followed by three arguments. For object **y**, constructor with two arguments is called and lastly the object **z**, which is without any argument, is candidate for constructor without argument.

9.7 Write a program with multiple constructors for the single class.

```

#include<iostream.h>
#include<conio.h>

class num
{
    private:
        int a;
        float b;
        char c;
    public:
        num(int m, float j , char k);
        num (int m, float j);
        num();
}

```

```
void show()
{
    cout<<"\n\ta="<<a<<"b="<<b<<"c="<<c;
}
};
num:: num (int m, float j , char k)
{
    cout<<"\n Constructor with three arguments";
    a=m;
    b=j;
    c=k;
}
num:: num (int m, float j)
{
    cout<<"\n Constructor with two arguments";
    a=m;
    b=j;
    c=' ';
}
num:: num()
{
    cout<<"\n Constructor without arguments";
    a=b=c=NULL;
}
main()
{
    clrscr();
    class num x(4,5.5,'A');
    x.show();
    class num y(1,2.2);
    y.show();
    class num z;
    z.show();
    return 0;
}
```

OUTPUT

Constructor with three arguments

a= 4 b= 5.5 c= A

Constructor with two arguments

a= 1 b= 2.2 c=

Constructor without arguments

a= 0 b= 0 c=

Explanation: In the above program, three constructors are declared. The first constructor is with three arguments, second with two, and third without any argument. While creating objects, arguments are passed. Depending on number of arguments, compiler decides on the constructor to be called. In this program, x, y, and z are three objects created. The x object passes three arguments, the y object passes two arguments, and the z object passes no arguments. The function show() is used to display the contents of the class members.

9.8 Write a program to overload constructor and display date and time.



```
#include<iostream.h>
#include<conio.h>

class date_time
{
    int d,m,y,hrs,min,sec;
public:
    date_time(int i,int j,int k,int a,int b,int c);
    date_time();
    void print()
    {
        cout<<"\nDate="<<d<<"Month="<<m<<"Year="<<y<<endl;
        cout<<"\n Hour="<<hrs<<"Minutes=" <<min<<"seconds="<<sec;
    }
};

date_time::date_time(int i,int j,int k,int a,int b,int c)
{
    d=i;
    m=j;
    y=k;
    hrs=a;
    min=b;
    sec=c;
}

date_time::date_time()
{
    cout<<"\nEnter Date Month Year (dd/mm/yy) : ";
    cin>>d>>m>>y;
    cout<<"\nEnter Hours Minutes Seconds (hr:min:sec) : ";
    cin>>hrs>>min>>sec;
}

int main()
{
    clrscr();
    date_time n(1,1,2012,14,45,21);
    n.print();
}
```

```

        cout<<"\n-----";
        date_time m;
        cout<<"\n-----";
        m.print();
        return 0;
    }

```

OUTPUT

```

Date = 1 Month = 1 Year =2012
Hour = 14 Minutes = 45seconds =21
-----
Enter Date Month Year (dd/mm/yy):4 4 2012
Enter Hours Minutes Seconds (hr:min:sec):23 45 12
-----
Date = 4 Month = 4 Year =2012
Hour = 23 Minutes = 45seconds =12

```

Explanation: The program has two constructors. The first constructor has six parameters. Six arguments are passed through the object to the first constructor.

The second constructor takes the date and time details entered through keyboard.

9.7 ARRAY OF OBJECTS USING CONSTRUCTORS

An array is a collection of similar data types. We can also create an array of objects. The array elements are stored in contiguous memory locations. An array of objects is one of the most important data structures when data is itself an object. Program on it is illustrated using constructor.

9.9 Write a program to declare array of objects by using constructor.

```

#include<iostream.h>
#include<conio.h>
#include<stdio.h>

class capital
{
    public:
        char name[20],cpt[20];
        capital();
};

capital::capital()
{
    cout<<"\nState and its capital:";
    cin>>name>>cpt;
}

```

```

}
int main()
{
    clrscr();
    capital c[3];
    return 0;
}

```

OUTPUT

State and its capital: Maharashtra Mumbai

State and its capital: Tamilnadu Chennai

State and its capital: Andhra-Pradesh Hyderabad

Explanation: The above program is generalized and can be extended to large number of objects. We have created an array of objects c[3] in the above program. The object c[3] initializes three names of the states and their capitals through keyboard, and the object c[3] calls the constructor three times.

9.8 CONSTRUCTORS WITH DEFAULT ARGUMENTS

Similar to functions, it is also possible to declare constructors with default arguments. Consider the following example:

```
power(int 9,int 3);
```

In the above example, the default value for the first argument is nine and two for second.

```
power p1 (3);
```

In this statement, object p1 is created and 9 raise to 3 expression n is calculated. Here, one argument is absent hence default value 9 is taken, and its third power is calculated. Consider the following example on the above discussion:

9.10 Write a program to declare default arguments in a constructor. Obtain the power of the number.

```

#include<iostream.h>
#include<conio.h>
#include<math.h>
class power
{
    private:
    int num;

```



```

    int power;
    int ans;
    public:
    power(int n=9,int p=3); // declaration of constructor with
default arguments
    void show()
    {

        cout<<"\n"<<num <<"raise to"<<power <<"is" <<ans;
    }
};
power:: power (int n,int p )
{
    num=n;
    power=p;
    ans=pow(n,p) ;
}
main()
{
    clrscr();
    class power p1,p2(5);
    p1.show();
    p2.show();
    return 0;
}

```

OUTPUT

9 raise to 3 is 729

5 raise to 3 is 125

Explanation: In the above program, the class power is declared. It has three integer member variables and one member function show(). The show() function is used to display the values of member data variables. The constructor of class power is declared with two default arguments. In the function main(), p1 and p2 are two objects of class power. The p1 object is created without argument. Hence, the constructor uses default arguments in pow() function. The p2 object is created with one argument. In this call of constructor, the second argument is taken as default. Both the results are shown in output.

9.9 COPY CONSTRUCTORS

The constructor can accept arguments of any data type including user-defined data types, exclusive of the class to which it belongs. Consider the following examples shown in Table 9.2.

■ **Table 9.2** Copy Constructors

Statement (a)	Statement (b)
<pre>class num { private: ---- ---- ---- public: num(num); }</pre>	<pre>class num { private: ---- ---- ---- public: num(num&); }</pre>

In the example (a), an argument of the constructor is same as that of its class. Hence, this declaration is wrong. It is possible to pass reference of object to the constructor. Such declaration is known as copy constructor. The example (b) is valid and can be used to copy constructor.

When we pass an object by value into a function, a temporary copy of that object is created. All copy constructors require one argument, with reference to an object of that class. Using copy constructors, it is possible for the programmers to declare and initialize one object using refer-

9.11 Write a program to pass an object with reference to constructor. Declare and initialize other objects.

```
#include<iostream.h>
#include<conio.h>

class num
{
    int n;
public:
    num() { } // constructor without argument
    num (int k) { n=k;} // constructor with one argument
    num (num &j) // copy constructor
    {
        n=j.n;
    }
    void show (void) {cout<<n;}
};

main()
{
    clrscr();
    num J(50);
    num K(J);
    num L=J;
    num M;
    M=J;
```

```

    cout<<"\n Object J Value of n:";
    J.show();
    cout<<"\n Object K Value of n:";
    K.show();
    cout<<"\n Object L Value of n:";
    L.show();
    cout<<"\n Object M Value of n:";
    M.show();
    return 0;
}

```

OUTPUT

Object J Value of n: 50

Object K Value of n: 50

Object L Value of n: 50

Object M Value of n: 50

ence of another object. Thus, whenever a constructor is called, a copy of an object is created.

Explanation: In the above program, class `num` is declared with one integer member variable `n` and three constructors. In function `main()`, the object `J` is created and 50 is passed to constructor. It is passed by value; hence, constructor with one argument is invoked. When object `K` is created with one object, the copy constructor is invoked, object is passed, and data member is initialized. The object `L` is created with assignment with object `J`; this time also copy constructor is invoked. The compiler copies all the members of object `J` to destination object `L` in the assignment statement `num L = J`. When object is created without any value, such as `M`, default constructor is invoked. The copy constructor is not invoked even if object `J` is assigned to object `M`.

The data member variables that are dynamically allocated should be copied to the target object explicitly by using assignment statement or by copy constructor as per the following statement:

```
num L=J; // copy constructor is executed
```

Consider the statement

```
M=J;
```

Here, `M` and `J` are predefined objects. In this statement, copy constructor is not executed. The member variables of object `J` are copied to object `M` member by member. An assignment statement assigns value of one entity to another.

The statement `num L = J;` initializes object `L` with `J` during definition. The member variables of `J` are copied member by member into object `L`. This statement invokes constructor. This statement can be written as `num L (J)`, which we frequently use to pass values to the constructor.

9.10 THE `const` OBJECTS

In the previous chapter, we have studied the constant functions. The `const` declared functions do not allow the operations that alter the values. In the same fashion, we can also make the object constant by the keyword `const`. Only constructor can initialize data member variables of

constant object. The data member of constant objects can be read-only and any effort to alter values of variables will generate an error. The data members of constant object are also called read-only data members. The constant object can access only constant functions. If constant object tries to invoke a non-member function, an error message will be displayed.

9.12 Write a program to declare constant object. Also, declare constant member function and display the contents of member variables.

```
#include<iostream.h>
#include<conio.h>

class ABC
{
    int a;
public:
    ABC (int m)
    { a=m; }
    void show() const
    { cout<<"a="<<a; }
};

int main()
{
    clrscr();
    const ABC x(5);
    x.show();
    return 0;
}
```

OUTPUT

A=5

Explanation: In the above program, class ABC is declared with one member variable and one constant member function show(). The constructor ABC is defined to initialize the member variable. The show() function is used to display the contents of member variable. In main(), the object x is declared as constant with one integer value. When object is created, the constructor is executed and value is assigned to data member. The object x invokes the member function show().

9.11 DESTRUCTORS

The destructor is also a special member function like constructor. Destructors destroy the class objects created by constructors. The destructors have the same name as their class, preceded by a tilde (~).

For local and non-static objects, the destructor is executed when the object goes out of scope. In case the program is terminated by using return or exit() statements, the destructor is ex-

executed for every object existing at that time. It is not possible to define more than one destructor. The destructor is only one way to destroy the object. Hence, they cannot be overloaded.

A destructor neither requires any arguments nor returns any value. It is automatically called when object goes out of scope. Destructor releases memory space occupied by the objects.

The program given below explains the use of destructor.

9.13 Write a program to demonstrate execution of constructor and destructor.

```
#include<iostream.h>
#include<constream.h>

struct text
{
    text() // Constructor
    {
        cout<<"\n Constructor executed.";
    }
    ~text() // Destructor
    {
        cout<<"\n Destructor executed.";
    }
};

void main()
{
    clrscr();
    text t; // Object declaration
}
```

OUTPUT

Constructor executed.

Destructor executed.

Explanation: In the above program, the class `text` contains constructor and destructor. In function `main()`, object `t` is executed. When object `t` is declared, constructor is executed. When object goes out of scope, destructor is executed.

9.14 Write a program to create an object and release them using destructors.

```
#include<iostream.h>
#include<conio.h>

int c=0; // counter for counting objects created and destroyed.
class num
```



```

{
    public:
    num()
    {
        c++;
        cout<<"\n Object Created: Object("<<c <<")";
    }
    ~num()
    {
        cout<<"\n Object Released: Object("<<c <<")";
        c--;
    }
};

main()
{
    clrscr();
    cout<<"\n In main() \n";
    num a,b;
    cout<<"\n\n In Block A\n";
    {
        class num c;
    }
    cout<<"\n\n Again In main()\n";
    return 0;
}

```

OUTPUT

```

In main()
Object Created: Object(1)
Object Created: Object(2)
In Block A
Object Created: Object(3)
Object Released: Object(3)
Again In main()
Object Released: Object(2)
Object Released: Object(1)

```

Explanation: In the above program, the variable `c` is initialized with zero. The variable `c` is incremented in constructor and decremented in destructor. When objects are created, the compiler calls the constructor. Objects are destroyed or released when destructor is called. Thus, the value of a variable changes as the constructors and destructors are called. The value of a variable of `c` shows number of objects created and destroyed. In this program, the objects `a` and `b` are created in `main()`. The object `c` is created in another block, that is, in block A. The object `c` is created

and destroyed in the same block. The objects are local to the block in which they are defined. The objects a and b are released as the control passes to `main()` block. The object created last is released first.

9.12 CALLING CONSTRUCTORS AND DESTRUCTORS

The compiler automatically calls the constructor and destructor. We can also call the constructor and destructor in the same fashion as we call the normal user-defined function. The calling methods are different for constructors and destructors. In practice, it may not be useful; but for the sake of understanding, few examples are illustrated on this concept.

9.15 Write a program to invoke constructor and destructor.

```
#include<iostream.h>
#include<conio.h>

class byte
{
    int bit;
    int bytes;
public:
    byte()
    {
        cout<<"\n Constructor invoked";
        bit=64;
        bytes=bit/8;
    }
    ~byte()
    {
        cout<<"\n Destructor invoked";
        cout<<"\n Bit="<<bit;
        cout<<"\n Byte="<<bytes;
    }
};

int main()
{
    clrscr();
    byte x;
    byte(); // calling constructor
    // x.byte() ; // invalid statement
    // x.byte::byte() // valid statement
    // ~ byte(); // invalid statement
    // byte.~byte(); // invalid statement
    // x.~byte(); // Member identifier expected
    x.byte::~~byte();
```

```
    return 0;
}
```

OUTPUT

Constructor invoked

Constructor invoked

Destructor invoked

Bit = 64

Byte = 8

Destructor invoked

Bit = 64

Byte = 8

Destructor invoked

Bit = 64

Byte = 8

Explanation: In the above program, the class `byte` is declared with two integer members. The class has constructors and destructors. In the function `main()`, `x` is an object of class `byte`. When an object is created, the compiler automatically invokes the constructor. The program also invokes constructor. Thus, constructor is executed two times, that is, implicitly and explicitly. The constructor is called without the help of object of that class. If we try to call the constructor with the help of object `x` as per the statement `x.byte()`, the compiler displays an error message.

The destructor cannot be invoked like constructor. Calling destructor requires the help of object and class name. Only object of the class alone is not able to invoke the destructor, the object needs the help of the class. The statement `x.~byte()` is invalid, the compiler displays the error message `Member identifier expected`. The destructor can be invoked using the statement `x.byte::~~byte()`. Here, in addition, class name is also specified.

The constructor and the destructor can call each other. The statement `byte::byte()` is used for calling destructor `~byte()` within the body of constructor `byte()`. The destructor `~byte()` can call the constructor `byte()` using the statement `byte()`. The constructor and the destructor may contain condition statements; such destructors or constructors are called conditional destructor or conditional constructor, respectively. The conditional constructor can be created using `if-else` or `switch()` statements. The example given below illustrates this.

9.16 Write a program to define conditional constructor and destructor.

```
#include<iostream.h>
#include<conio.h>
int c=0;

class byte
{
    int bit;
    int bytes;
```

```
public:
~byte(); // Prototype declaration
byte()
{ cout<<"\n Constructor invoked";
bit=64;
bytes=bit/8;
byte::~~byte(); // call to destructor
}
};
byte::~~byte()
{
    if (c==0)
    {
        // byte() // call to constructor
        cout<<"\n Destructor invoked";
        cout<<"\n Bit="<<bit;
        cout<<"\n Byte="<<bytes;
    }
    c++;
}
int main()
{ clrscr();
  byte x,y;
  return 0;
}
```

OUTPUT

Constructor invoked

Destructor invoked

Bit = 64

Byte = 8

Constructor invoked

Explanation: In the above program, x and y are objects of class byte. The compiler automatically invokes the constructor. The constructor invokes the destructor. The variable c is declared and initialized with zero. It is a global variable declared before main() and can be used anywhere in the program. The destructor contains conditional statement if. The if block is executed only when c is zero. When destructor is called the first time, the if block is executed and c is turned to a non-zero value. Next time when destructor is called, the if condition evaluates false and the if block is not executed. Though destructor is called, no result is displayed on the screen. Remember that the constructors and destructors are executed for equal number of times. It is possible for the programmer to use the constructor and the destructor like other user-defined functions.

9.13 QUALIFIER AND NESTED CLASSES

The class declaration can also be done inside the other class. While declaring object of such class, it is necessary to precede the name of the outer class. The name of outer class is called qualifier name, and the class defined inside is called nested class.

The use of nested classes enhances the power of abstract data type and helps to construct more powerful data structure. The qualifier (host class) and the nested class follow the same access conditions. The nested classes are useful to hide a particular class and its objects within a qualifier class. The following program illustrates this.

9.17 Write a program to declare classes within classes and access the variables of both the classes.

```
#include<iostream.h>
#include<constream.h>

class A // Host class or qualifier class
{
    public:
    int x;
    A() { x=10; }
    class B // Nested class
    {
    public:
    int y;
    B() { y=20; }
    void show()
    {
        A a;
        cout<<"x="<<a.x<<endl;
        cout<<"y="<<y<<endl;
    }
    }; // End of nested (inner) class
}; // End of qualifier (outer) class

void main()
{
    clrscr();
    A::B b;
    b.show();
}
```

OUTPUT

x=10
y=20

Explanation: In the above program, class B is declared inside class A. All the members of both the classes are declared public so that they can be easily accessed. Both the classes have constructors to initialize the member variables. The class B also has one member function show() to display the contents on the screen.

In function main(), A::B b declares object of class B. The name of class A is preceded because the class B is inside the class A. If we declare an object using the statement B b, the program will be executed with a warning message "Use qualified name to access nested type 'A::B' ". Here, the class A acts as a qualified class.

The show() function is a member of class B. Due to this, variable y is its member variable and can be directly accessed. To access the variable of a qualified class A, object of class A a is declared and through this object variable x is accessed. The data variables are public. In case of private, we can access member variables using member functions. There is no limit for declaring classes within classes. The following program explains this.

9.18 Write a program to declare multiple qualifier classes and declare an object of every class.

```
#include<iostream.h>
#include<conio.h>

class A
{
    public:
    int x;
    A() { x=5;
        cout<<"x="<<x;
    }
}

class B
{
    public:
    int y;
    B()
    {
        y=10;
        cout<<"y="<<y;
    }
}

class C
{
    public:
    int z;
    C()
    {
        z=15;
    }
}
```

```

        cout<<"z="<<z;
    }
    };
};
};
void main()
{
    clrscr();
    A a; // outer class object
    A::B b; // middle class object
    A::B::C c; // inner class object
}

```

OUTPUT

x= 5 y =10 z =15

Explanation: In the above program, classes A, B, and C are declared. The class B is declared inside the class A. The class C is declared inside the class B. The classes A and B are qualified classes for class C. The class A is qualified class for class B. Each class has a constructor that initializes and displays the contents of the object.

9.14 ANONYMOUS OBJECTS

Objects are created with names. It is possible to declare objects without name. Such objects are known as anonymous objects. We learnt how to invoke constructors and destructors. When constructors and destructors are invoked, the data members of the class are initialized and destroyed, respectively. Thus, without object we can initialize and destroy the contents of the class. All these operations are carried out without object or we can also assume that the operations are carried out using an anonymous object, which exists but is hidden. The second thought is correct because if we apply pointer `this` (explained in Chapter 13 “Pointers and arrays”), we get the address of the object. It is not possible to invoke member function without using object but we can invoke the special member functions `constructors` and `destructors` which compulsorily exist in every class. Thus, without the use of constructor and destructor, the theory of anonymous object cannot be implemented in practice. Consider the following example:

9.19 Write a program to create an anonymous object. Initialize and display the contents of member variables.

```

#include<conio.h>
#include<iostream.h>

class noname
{
    private:

```

```
int x;
public:
noname (int j)
{
cout<<"\n In constructor";
x=j;
show();
}
noname()
{
cout<<"\n In constructor";
x=15;
show();
}
~noname() { cout<<"\n In destructor"; }
noname *const show()
{
cout<<endl<<"x:"<<x;
return this;
}
};
void main()
{
clrscr();
noname();
noname(12);
}
```

OUTPUT

In constructor

x: 15

In destructor

In constructor

x: 12

In destructor

Explanation: In the above program, class `noname` is declared with one integer data member. The class also has constructor, destructor, and member function. The member function `show()` is used to display the contents on the screen.

In function `main()`, no object is created. The constructor is called directly. Calling constructor directly implies creating anonymous objects. In the first call of the constructor, the member variable is initialized with 15. The constructor invokes the member function `show()` that displays the value of `x` on the screen.

It is not possible to create more than one anonymous object at a time. When constructor execution work is over destructor destroys the object. Again, the constructor with one argument is called and integer is passed to it. The member variable is initialized with 12 and it is displayed by `show()` function. Finally, the destructor is executed that marks the end of program.



TIP

Calling constructor directly means creating anonymous objects.

9.15 PRIVATE CONSTRUCTORS AND DESTRUCTORS

We learnt that when a function is declared as private, it could be invoked by the public function of the same class. So far, we declared constructors and destructors as public. The constructor and destructor can be declared as private. The constructor and destructor are automatically executed when an object is executed and when an object goes out of scope. Nevertheless, when the constructor and destructor are private, they cannot be executed implicitly, and hence, it is a must to execute them explicitly. The following program is illustrated concerning this.

9.20 Write a program to declare constructor and destructor as private and call them explicitly.

```
#include<conio.h>
#include<iostream.h>

class A
{
    private:
        int x;
        ~A() { cout<<"\n In destructor ~A()"; }
        A() {x=7; cout<<"\n In constructor A()"; }
    public:
        void show()
        {
            this->A::A(); // invokes constructor
            cout<<endl<<"x="<<x;
            this->A::~A(); // invoke destructor
        }
};

void main()
{
    clrscr();
    A *a;
    a->show();
}
```

OUTPUT**In constructor A()****x = 7****In destructor ~A()**

Explanation: In the above program, class `A` is declared. It has one data member, constructor, and destructor. The function `show()` is declared in public section. In function `main()`, `a` is a pointer to class `A`. When pointer object is created, no constructor is executed. When `show()` function is invoked, the constructor is invoked and its return value is assigned to pointer `*this`. (The pointer `*this` is explained in Chapter 13 “Pointers and arrays.”) Every member function holds `this` pointer that points to the calling object. Here, the pointer points to object `a`. When the pointer invokes the constructor, constructor is invoked. Consider the statement `this = this->A::A()`. The statement invokes the zero-argument constructors. The contents are displayed by the `cout` statement and again the `this` pointer invokes the destructor.

9.16 DYNAMIC INITIALIZATION USING CONSTRUCTORS

After declaration of the class data member variables, they can be initialized at the time of program execution using pointers. Such initialization of data is called dynamic initialization. The benefit of dynamic initialization is that it allows different initialization modes using overloaded constructors. Pointer variables are used as argument for constructors. The following example explains dynamic initialization using overloaded constructor.

9.21 Write a program to initialize member variables using pointers and constructors.

```
#include<iostream.h>
#include<conio.h>
#include<string.h>
class city
{
    char city[20];
    char state[20];
    char country[20];
public:
    city() { city[0]=state[0]=country[0]=NULL; }
    void display( char *line);
    city(char *cityn)
    {
        strcpy(city, cityn);
        state[0]=NULL;
    }
    city(char *cityn,char *staten)
```

```

{
    strcpy(city,cityn);
    strcpy(state,staten);
    country[0]=NULL;
}
city(char *cityn,char *staten, char *countryn)
{
    _fstrcpy(city,cityn);
    _fstrcpy(state,staten);
    _fstrcpy(country,countryn);
}
};
void city:: display (char *line)
{
    cout<<line<<endl;
    if (_fstrlen(city)) cout<<"City:"<<city<<endl;
    if (strlen(state)) cout<<"State:"<<state <<endl;
    if (strlen(country)) cout<<"Country:"<<country <<endl;
}
void main()
{
    clrscr();
    city c1("Mumbai"),
    c2("Nagpur", "Maharashtra"),
    c3("Nanded", "Maharashtra", "India"),
    c4('\0', '\0', '\0');
    c1.display("====*====");
    c2.display("====*====");
    c3.display("====*====");
    c4.display("====*====");
}

```

OUTPUT

```

====*====
City: Mumbai
====*====
City: Nagpur
State: Maharashtra
====*====
City: Nanded
State: Maharashtra
Country: India
====*====

```

Explanation: In the above program, the class `city` is declared with three-character arrays: `city`, `state`, and `country`. The class `city` also has four constructors: zero-argument constructor, one-argument constructor, two-argument constructor, and three-argument constructor. The `display()` member function is used to display the contents on the screen.

In function `main()`, `c1`, `c2`, `c3`, and `c4` are declared and strings are passed. According to the number of arguments, respective constructor is executed. The function `display()` is called by all the four objects and information displayed is as shown in the output. While calling function `display()`, the line format “=====” is passed that is displayed before displaying the information. The `display()` function also contains if statements that check the length of the string and display the string only when the variable contains string. In case the data variable contains NULL, the string will not be displayed. The object `c4` is initialized with NULL character. The use of `c4` object is only to display a line at the end of a program.

9.17 DYNAMIC OPERATORS AND CONSTRUCTORS

When constructor and destructor are executed they internally use `new` and `delete` operators to allocate and de-allocate memory. Dynamic construction means allocation of memory by constructor for objects, and dynamic destruction means releasing memory using the destructor. Consider the following program that explains the use of `new` and `delete` operators with constructor and destructor.

9.22 Write a program to use `new` and `delete` operators with constructor and destructor. Allocate memory for the given number of integers. Also release the memory.

```
#include<iostream.h>
#include<conio.h>

int number (void);
class num
{
    int *x;
    int s;
public:
    num()
    {
        s=number();
        x= new int [s];
    }
    ~num() { delete x; }
    void input();
    void sum();
};

void num:: input()
{
    for ( int h=0;h<s;h++)
```

```

    {
        cout<<"Enter number ["<<h+1<<"] : ";
        cin>>x[h];
    }
}
void num:: sum()
{
    int adi=0;
    for ( int h=0;h<s;h++)
        adi+=x[h];
    cout<<"sum of elements="<<adi;
}
number()
{
    clrscr();
    int n;
    cout<<"How many numbers:";
    cin>>n;
    return n;
}
void main()
{
    num n1;
    n1.input();
    n1.sum();
}

```

OUTPUT

```

How many numbers: 3
Enter number [1]: 1
Enter number [2]: 4
Enter number [3]: 5
sum of elements = 10

```

Explanation: In the above program, class num is declared with two variables and they are integer pointer *x and integer variable s. The class also contains constructor and destructor. The responsibility of constructor is to allocate memory using new operator for a given number of integers by the user. The non-member function number() is used to input number of elements through the keyboard and the entered numbers are stored in the variable s. The variable s is used with the new operator to allocate memory. The memory is allocated to the pointer x.

The member function input() reads elements and the for loop repeats the statement cin>>x[h] for s (number of total elements) times. The x[h] is used like array. The x is the starting address and h indicates the successive locations. The address of pointer x remains

unchanged. The value of `h` is added to value of `x`. In this way, all these numbers are stored in continuous memory locations. If you are still confused how successive memory locations are accessed, go through the pointer arithmetic operations.

The `sum()` function is used to perform addition of all the entered numbers and displays the same on the screen. The destructor is finally executed which releases the memory using `delete` operator.



TIP

In the above program, if we remove the operators `new` and `delete`, the program will work successfully. However, this is suitable for small programs, developed for demonstration purpose. In real application, it needs large amount of memory to be allocated or released. Hence, it is very essential to check the memory before doing the process. For example, when any program is loaded in the memory, the operating system checks for available system resources that are sufficient enough to load the requested application by the user. In such a case, the operating system checks memory resources and if memory is insufficient, the compiler displays the same message to the user.

9.18 main() AS A CONSTRUCTOR AND DESTRUCTOR

We learnt that the constructor and destructor have the same name as their class. To use `main()` as a constructor and destructor, we need to define class with the name `main`. So far, we have declared object of classes without using the keyword `class` or `struct` because C++ treats classes like built-in data types. The use of keyword `class` or `struct` is only compulsory when the class name is `main`. This is because execution of program starts with the function `main()`. The following program clears this point.

9.23 Write a program to declare class with name `main`.

```
#include<conio.h>
#include<iostream.h>
class main
{
    public:
        main() { cout<<"\n In constructor main()"; }
        ~main() { cout<<"\n In destructor main()"; }
};
void main()
{
    clrscr();
    class main a;
}
```

OUTPUT

In constructor `main()`

In destructor `main()`

Explanation: In the above program, the class is declared with the name `main`. Hence, it is compulsory to use keyword `class` or `struct` while declaring objects.

**TIP**

When the class name is `main`, it is compulsory to use keyword `class` or `struct` to declare objects.

9.19 RECURSIVE CONSTRUCTORS

Similar to normal and member functions, constructors also support recursion. The following program explains this.

9.24 Write a program to invoke constructor recursively and calculate the triangular number of the entered number.



```
#include<iostream.h>
#include<conio.h>
#include<process.h>

class tri_num
{
    int f;
public:
    tri_num() { f=0; }
    void sum( int j) { f=f+j; }
    tri_num(int m)
    {
        if (m==0)
        {
            cout<<"Triangular number:"<<f;
            exit(1);
        }
        sum(m);
        tri_num::tri_num(--m);
    }
};

void main()
{
    clrscr();
    tri_num a;
    a.tri_num::tri_num(5);
}
```

OUTPUT

Triangular number: 15

Explanation: In the above program, class `tri_num` is declared and with private integer `f` and member function `sum()`. The class also has zero-argument constructor and one-argument constructor.

In function `main()`, `a` is an object of `tri_num` class. When object `a` is declared, the zero-argument constructor is executed. The object `a` invokes the constructor explicitly and passes integer value to it. The passed value is received by the variable `m` of the constructor. The `if` statement checks the value of variable `m` and if it is zero, the triangular number is displayed and the program is terminated.

The `sum()` function is called by the constructor and the cumulative sum is calculated and stored in the member variable `f`. Followed by the call of function `sum()`, the constructor `tri_num()` is executed and the value of `m` is decreased by one. Thus, the recursion takes place and each time value of `m` decreases. When the value of `m` becomes zero, as mentioned above, the program terminates and we get the triangular number.



TIP

Recursion is not difficult, though somewhat confusing. If you are still confused about recursion concept, execute the program in a single step to know the flow of the program. First, try the recursion with normal function in C style and after perfect understanding try with member function and constructors, but do not drop it.

9.20 PROGRAM EXECUTION BEFORE `main()`

It is impossible to execute a program without `main()`. The declaration of objects is allowed before `main()`. We know that when an object is created, constructor and destructor are executed. The constructor can call other member functions. When an object is declared before `main()`, it is called as global object. For global object, constructor is executed before `main()` function and destructor is executed after the completion of execution `main()` function. The following program illustrates this.

9.25 Write a program to declare global object and observe execution of constructor.

```
#include<iostream.h>
#include<constream.h>

struct text
{
    text() // Constructor
    {
        cout<<"\n Constructor executed.";
    }
    ~text() // Destructor
    {
        cout<<"\n Destructor executed.";
    }
}
```



```
};
text t; // Global object
void main()
{ }
```

OUTPUT

Constructor executed.

Destructor executed.

Explanation: In the above program, object `t` is declared before `main()`. It is a global object. As soon as an object is declared, its constructor is invoked immediately. However, the execution of every program starts from function `main()`, but for the object declared before `main()`, constructor is executed before execution of `main()`. The destructor for such object is executed after the complete execution of function `main()`. The constructor can invoke other member functions. These member functions are also executed before `main()`. The following program explains this.

9.26 Write a program to declare object before `main()` and invoke member function.

```
#include<iostream.h>
#include<conio.h>
class A
{
    private:
    char *y;
    int x;
    public:
    A()
    {
        clrscr();
        cout<<"\n In constructor ";
        x=15;
        show(); // invokes member function
    }
    void show() { cout<<endl<<"In show() x="<<x; }
    ~A() { cout<<endl<<"In destructor"; }
};
A a;
void main() { }
```

OUTPUT

In constructor

In show() x=15

In destructor

Explanation: In the above program, class A is declared with member variable character pointer and integer x. The class also has constructor, destructor, and member function show(). Before main(), object a is declared and constructor is executed that initializes the member variables. The constructor invokes the member function show(). Finally, the destructor is executed.

9.21 CONSTRUCTOR AND DESTRUCTOR WITH STATIC MEMBERS

Every object has its own set of data members. When a member function is invoked, only copy of data member of calling object is available to the function. Sometimes, it is necessary for all the objects to share fields which are common for all the objects. If the member variable is declared as static, only one copy of such member is created for entire class. All objects access the same copy of static variable. The static member variable can be used to count the number of objects declared for a particular class. The following program helps you to count the number of objects declared for a class.

9.27 Write a program to declare a static member variable. Count the number of objects created and destroyed.

```
#include<iostream.h>
#include<constream.h>

class man
{
    static int no;
    char name;
    int age;
public:
    man()
    {
        no++;
        cout<<"\n Number of Objects exists: "<<no;
    }
    ~ man()
    {
        --no;
        cout<<"\n Number of objets exists:"<<no;
    }
};

int man:: no=0;
void main()
{
    clrscr();
    man A,B,C;
    cout<<"\n Press any key to destroy object";
```

```

    getch();
}

```

OUTPUT

```

Number of Objects exists: 1
Number of Objects exists: 2
Number of Objects exists: 3
Press any key to destroy object
Number of objects exists: 2
Number of objects exists: 1
Number of objects exists: 0

```

Explanation: In this program, the class `man` has one static data member `no`. The static data member is initialized to zero. Only one copy of static data member is created and all objects share the same copy of static data member.

In function `main()`, objects `A`, `B`, and `C` are declared. When objects are declared, constructor is executed and static data member `no` is increased with one. The constructor also displays the value of `no` on the screen. The value of static member shows us the number of objects present. When the user presses a key, destructor is executed, which destroys the object. The value of static variable is decreased in the destructor. The value of static member variable shows the number of existing objects.

9.22 LOCAL VERSUS GLOBAL OBJECT

The object declared outside all function bodies is known as global object. All functions can access the global object. The object declared inside a function body is known as local object. The scope of local object is limited to its current block. We learnt the behavior of constructor and destructor with local and global object.

When global and local variables are declared with the same name, the scope access operator is used to access the global variable in the current scope of local variable. We know that the local variable gets first precedence than the global variable. The same is applicable for objects. When a program contains global and local objects with the same name, the local object gets first preference in its own block. In such a case, scope access operator is used with global object. The following program describes this.

9.28 Write a program to show difference between local and global object.

```

#include<iostream.h>
#include<constream.h>

class text
{
    public:

```

```

    void show (char *c)
    {
        cout<<"\n"<<c;
    }
};
text t; // global object declaration
void main()
{
    text t; // local object declaration
    ::t.show("Global"); // call using global object
    t.show("Local"); // call using local object
}

```

OUTPUT

Global

Local

Explanation: In the above program, the object `t` is declared in local as well as global scope. In function `main()`, using scope access operator and `t`, the function `show()` is invoked. The first time function `show()` is invoked using global object. The second call to function `show()` is made by local object. The local object does not require scope access operator. The scope access operator is used only when the same name for object is used in global and local scopes. If scope access operator is not used before object, for both times function `show()` is invoked by local object.

9.23 MORE PROGRAMS

9.29 Write a program to pass arguments to constructor. Define conditional constructor.

```

#include<iostream.h>
#include<conio.h>
#include<math.h>

struct num
{
    num (int,char);
    ~num()
    { cout<<"\n Destructor invoked"; }
};

num::num (int s,char b)
{

```

```

switch(b)
{
case 'R':
cout<<"\n Square Root of"<<s <<"is" <<sqrt(s);
break;
case 'S':
cout<<"\n Square of"<<s <<"is" <<pow(s,2);
break;
case 'C':
cout<<"\n Cube of"<<s <<"is" <<pow(s,3);
break;
default:
cout<<"\n Invalid choice";
}
}
int main()
{
clrscr();
int u;
char o;
cout<<"Enter a number:";
cin>>u;
cout<<"Enter operation (S)quare, (C)ube, Square (R)oot";
cin>>o;
num x(u,o);
return 0;
}

```

OUTPUT

Enter a number: 64

Enter operation (S)quare, (C)ube, Square (R)oot R

Square Root of 64 is 8

Destructor invoked

Explanation: In the above program, struct num is declared. The structure is without any data member. The struct contains constructor and destructor. In function main(), x is an object. Before creating object x, an integer and a character are entered through the keyboard in variables u and o, respectively. The variables u and o are passed to the constructor. The switch case statement within the constructor selects appropriate operation depending upon the value of variable o.

9.30 Write a program to copy constructor. Change the values of an object by calling the constructor.

```
#include<iostream.h>
#include<conio.h>

class data
{
    int u;
    char c;
    float f;
public:
    ~data() { };
    data() {u=c=f=NULL; };
    data (int a,char b, float h) { u=a,c=b,f=h;}
    data (data &k)
    {
        u=k.u;
        c=k.c;
        f=k.f;
    }
    void show()
    { cout<<"\n u="<<u <<"c="<<(char)c <<"f="<<f; }
};

void main()
{
    clrscr();
    data k(5,68,8.2);
    k.show();
    data l(k);
    l.show();
    data z;
    z.show();
    z=data(1,70,5.8);
    z.show();
}
```

OUTPUT

u = 5 c = D f= 8.2

u = 5 c = D f= 8.2

u = 0 c = f= 0

u = 1 c = F f= 5.8

Explanation: In the above program, the class `data` is declared with three data member variables and one member function `show()`. The class `data` also contains destructor and overloaded constructors. For explanation of this program, please refer Program 5.7. In this program, the object `z` is created and no arguments are passed. In such a case, the constructor with no arguments is executed and the entire data members are initialized to `NULL`. The statement `z = data (1, 70, 5.8)` explicitly calls the constructor and the object `z` is initialized. The result of the program is as given above.

9.31 Write a program to call the constructor without using object.

```
#include<iostream.h>
#include<conio.h>

class A
{
    public:
    char c;
    int i;
    float f;
    A()
    {
        cout<<"\n Enter char, int and float value";
        cin>>c>>i>>f;
        out(c,i,f);
    }
    void out (char l,int k, float j )
    {
        cout<<"\n char value="<<l;
        cout<<"\n int value="<<k;
        cout<<"\n float value="<< j;
        cout<<"\n";
    }
};

int main()
{
    clrscr();
    A()=A()=A();
    return 0;
}
```

OUTPUT

```
Enter char, int and float value A 65 6.5
char value = A
```

```

int value = 65
float value = 6.5
Enter char, int and float value B 66 6.6
char value = B
int value = 66
float value = 6.6
Enter char, int and float value C 67 6.7
char value = C
int value = 67
float value = 6.7

```

Explanation: In the above program, class A has three member variable c, i, and f of char, int, and float type. The class A also one-member function out () and a constructor. The constructor invokes the out () function to output the contents on the screen. In function main (), no object is declared. The constructor is called directly.

9.32 Write a program to demonstrate the use of copy constructor.

```

#include<iostream.h>
#include<conio.h>

class data
{
    private:
        int x;
        float y;
    public:
        data() { }
        data (int xx, float yy) { x=xx; y=yy; }
        data operator=(data & d)
        {
            cout<<endl<<"Assignment operator executed";
            x=d.x;
            y=d.y;
            return data (x,y);
        }
        data (data &d)
        {
            cout<<endl<<"copy constructor executed";
            x=d.x;
            y=d.y;
        }
        void show() { cout<<"X=" <<x<<"Y=" <<y; }

```



```
};  
void main()  
{  
    clrscr();  
    data d1(12,5.8);  
    data d2,d4;  
    d4=d2=d1;  
    data d3=d1;  
    cout<<"\n Object d1:";  
    d1.show();  
    cout<<"\n Object d2:";  
    d2.show();  
    cout<<"\n Object d3:";  
    d3.show();  
    cout<<"\n Object d4:";  
    d4.show();  
}
```

OUTPUT

Assignment operator executed

Assignment operator executed

copy constructor executed

Object d1: X=12 Y=5.8

Object d2: X=12 Y=5.8

Object d3: X=12 Y=5.8

Object d4: X=12 Y=5.8

Explanation: In the above program, d1, d2, d3, and d4 are objects of the class data. The statement d4 = d2 = d1 invokes the overloaded operator = and assignment is carried out. The statement data d3 = d1 executes the copy constructor. The operator = () returns an object of class data type and initializes it using three-parameter constructor. The above approach creates an extra copy of the object that occupies extra memory space. To overcome this problem, we can use copy constructor method in which no new object is created.

9.33 Write a program to invoke constructors of nested classes.

```
#include<iostream.h>  
#include<conio.h>  
  
class A  
{  
    public:  
    int x;
```

```
    A() { x=5;
        cout<<endl<<"In constructor A x="<<x;
    }
class B
{
    public:
    int y;
    B()
    {
        y=10;
        cout<<endl<<"In constructor B y="<<y;
    }
    class C
    {
    public:
    int z;
    C()
    {
        z=15;
        cout<<endl<<"In constructor C z="<<z;
    }
    };
};
};
void main()
{
    clrscr();
    A();
    A::B();
    A::B::C();
}
```

OUTPUT

In constructor A x= 5

In constructor B y =10

In constructor C z =15

Explanation: In the above program, classes A, B, and C are declared. The class B is declared inside the class A. The class C is declared inside the class B. A and B are qualified classes for class C. The class A is qualified class for class B. Each class has a constructor that initializes and displays the contents of the object.

SUMMARY

- (1) C++ provides a pair of in-built functions called `constructor` and `destructor`. The compiler automatically executes these functions. When an object is created, constructor is executed. The programmer can also pass values to the constructor to initialize member variables with different values. The destructor destroys the object. It is executed at the end of program when objects are of no use.
- (2) Constructors and destructors decide how the objects of a class are created, initialized, copied, and destroyed. They are member functions. Their names are distinguished from all other member functions because they have the same name as the class they belong to.
- (3) It is also possible to create constructor with arguments as in normal functions.
- (4) Similar to functions, it is also possible to overload constructors and assign default arguments.
- (5) When we pass an object by value into a function, a temporary copy of that object is created. All copy constructors require one argument, with reference to an object of that class. Using copy constructors, it is possible for the programmers to declare and initialize one object using reference of another object. Thus, whenever a constructor is called, a copy of an object is created.
- (6) We can also make the object constant by the keyword `const`. Any effort to alter values of variables made by it will generate an error. The constant object can access only constant functions.
- (7) The compiler automatically calls the constructor and destructor. We can also call the constructor and destructor in the same fashion as we call the normal user-defined function. The calling methods are different for constructor and destructor.
- (8) Objects are created with names. It is possible to declare objects without name. Such objects are known as anonymous objects.
- (9) When the constructor and destructor are private, they cannot be executed implicitly, and hence, it is a must to execute them explicitly.
- (10) We can also call the constructor and destructor in the same fashion as we call the normal user defined function.
- (11) The class declaration can also be done inside the other class. While declaring object of such class, it is necessary to precede the name of the outer class. The name of outer class is called qualifier name, and the class defined inside is called as nested class.
- (12) The dynamic construction means allocation of memory by constructor for objects and dynamic destruction means releasing memory using the destructor.
- (13) To use `main()` as a constructor and destructor, we need to define class with the name `main`.
- (14) Similar to normal and member functions, constructors also support recursion.
- (15) For global object, constructor is executed before `main()` function and destructor is executed after the completion of execution `main()` function.

EXERCISES

(A) Answer the following questions

- (1) What are constructors and destructors?
- (2) Explain the characteristics of constructors and destructors.
- (3) Explain constructors with arguments. How are arguments passed to the constructor?

- (4) What do you mean by overloading of constructors? How it benefits the programmer?
- (5) Explain constructor with default arguments.
- (6) What is copy constructor?
- (7) What are constant objects? How are they declared?
- (8) How are constructors and destructors called explicitly?
- (9) What is the difference between calling methods for constructors and destructors?
- (10) Is it possible for a constructor and destructor to call each other?
- (11) What are conditional constructors and destructors?
- (12) What is anonymous object?
- (13) What are nested and qualifier classes?
- (14) What is the difference between local object and global object?
- (15) What is static object? How it is different from normal object?
- (16) How are private constructors and destructors executed?
- (17) How will you declare constant object?
- (18) What is default constructor?
- (19) What is parameterized constructor?
- (20) Explain the negative aspect of static objects.

(B) Answer the following by selecting the appropriate option

- (1) Constructor and destructor are automatically invoked by
 - (a) **compiler**
 - (b) operating system
 - (c) `main()` function
 - (d) object
- (2) Constructor is executed when
 - (a) **object is declared**
 - (b) object is destroyed
 - (c) both (a) and (b)
 - (d) none of the above
- (3) The destructor is executed when
 - (a) **object goes out of scope**
 - (b) object is not used
 - (c) object contains nothing
 - (d) none of the above
- (4) When memory allocation is essential, the constructor makes implicit call to
 - (a) **new operator**
 - (b) `malloc()`
 - (c) `memset()`
 - (d) random access memory
- (5) Destructors can be
 - (a) **overloaded**
 - (b) of any data type
 - (c) able to return result
 - (d) explicitly called
- (6) Constructors have the same name as
 - (a) **the class they belong to**
 - (b) the current program file name
 - (c) class name and preceded by ~
 - (d) both (a) and (c)

- (7) The following program displays

```
#include<iostream.h>
class A
{
    int x;
public:
    A() {x=10;}
    ~A() {}
};
void main()
{
    A a;
    cout<<(unsigned)
    a.A::A();
}
```

- (a) **address**
- (b) value
- (c) both (a) and (b)
- (d) none of the above
- (8) The following program returns address of

```
#include<iostream.h>
class A
{public:
    A(){} ~A(){} A *display()
    {return (&A());}
};
void main()
{ void *p;
  clrscr();
}
```

```

    A a;
    p=a.display();
    cout<<p;
}

```

- (a) **constructor**
- (b) `display()`
- (c) first element if any
- (d) none of the above

(C) Attempt the following programs

- (1) Write a program to declare a class with private data members. Accept data through constructor and display the data with destructor.
- (2) Write a program to pass an object to constructor and carry out copy constructor. Display contents of all the objects.
- (3) Write a program to declare a class with three data members. Declare overloaded constructors with no arguments, one argument, two arguments, and three arguments. Pass values in the object declaration statement. Create four objects and pass values in such a way that the entire four constructors are executed one by one. Write appropriate messages in constructor and destructor so that the execution of program can be understood.
- (4) Write a program to declare a class with two data members. Also, declare and define member function to display the content of the class data members. Create object A. Display the contents of object A. Again initialize the object A using explicit call to constructor. This time pass appropriate values to constructor. Display the contents of object A using member function.
- (5) Write a program to call constructor recursively. Calculate factorial of a given number.
- (6) Write a program to create an array of strings. Read and display the strings using constructor and destructor. Do not use member functions.
- (7) Write a program to create object without name.

(D) Find the bugs in the following programs

- (1)

```
class text
{ text() { cout<<"Start"; }
~text() { cout<<"\n End"; }
};

void main() { text t; }
```
- (2)

```
struct text
{ public:
    text (char *c)
    { cout<<"\n"<<c; }
};
```

- ```
void main() { text t; }
```
- (3) 

```
class text
{ char *c;

public:
 text (*c) { cout<<"\n"<<c;
 }

};

void main() { text t("I
WON"); }
```

# Operator Overloading and Type Conversion

## 10 CHAPTER

### CHAPTER OUTLINE

- 10.1 Introduction
- 10.2 The Keyword Operator
- 10.3 Overloading Unary Operators
- 10.4 Operator Return Type
- 10.5 Constraint on Increment and Decrement Operators
- 10.6 Overloading Binary Operators
- 10.7 Overloading with `friend` Function
- 10.8 Overloading Assignment Operator (`=`)
- 10.9 Type Conversion
- 10.10 Rules for Overloading Operators
- 10.11 One-Argument Constructor and Operator Function
- 10.12 Overloading Stream Operators
- 10.13 More Programs

## 10.1 INTRODUCTION

Operator overloading is one of the important and useful features of C++. We are familiar with function overloading in which multiple functions use the same name. The concept of operator overloading is somewhat similar to that of function overloading.

A symbol that is used to perform an operation is called an operator. It is used to perform an operation with constants and variables. A programmer cannot build an expression without an operator.

C++ frequently uses user-defined data types that are a combination of one or more basic data types. C++ has the facility to create/build user-defined data type. User-defined data types created from class or struct are nothing but a combination of one or more variables of basic data types.

The compiler knows how to perform various operations using operators for the built-in types; however, for the objects those are instance of the class, the operation routine must be defined by the programmer.

For example, in traditional programming languages the operators such as  $+$ ,  $-$ ,  $\leq$ ,  $\geq$ , etc. can be used only with basic data types such as `int` or `float`. The operator  $+$  (plus) can be used to perform addition of two variables, but the same is not applicable for objects. The compiler cannot perform addition of two objects. The compiler would throw an error if addition of two objects is carried out. The compiler must be made aware of the addition process of two objects. When an expression including operation with objects is encountered, a compiler searches for the definition of the operator, in which a code is written to perform an operation with two objects. Thus, to perform an operation with objects we need to redefine the definition of various operators. For example, for addition of objects  $A$  and  $B$ , we need to define operator  $+$  (plus). Redefining the operator plus does not change its natural meaning. It can be used for both variables of built-in data type and objects of user-defined data type.

Operator overloading is one of the most valuable concepts introduced by C++ language. It is a type of polymorphism. Polymorphism permits to write multiple definitions for functions and operators. C++ has a number of standard data types such as `int`, `float`, `char`, etc. The operators  $+$ ,  $-$ ,  $*$ , and  $=$  are used to carry operations with these data types. Operator overloading helps the programmer to use these operators with the objects of classes. The outcome of operator overloading is that the objects can be used in as natural manner as the variables of basic data type. Operator overloading provides the capability to redefine the language in which the working operator can be changed.

Consider an example,

```
a=c+d;
c=a-d;
```

where,  $a$ ,  $c$ , and  $d$  are variables of basic data types like `int` or `float`. The use of operator  $+$ ,  $-$ , and  $=$  is valid. However, if we try these operators with the object, the compiler displays an error message “Illegal structure operation.”

```
class number
{
 public:
 int x;
 int y;
};
```

For example,  $A$ ,  $B$ , and  $C$  are three objects of class `number`. Each object holds an individual copy of member variable  $x$  and  $y$ . We want to perform addition of  $A$  and  $B$  and store the result in  $C$ . For the sake of understanding, the member variables are declared in public section. The addition of  $A$  and  $B$  implies addition of member variables of  $A$  and member variables of  $B$ . The result of this operation will be stored in member variables of  $C$ . This feature can be implemented as follows

**10.1 Write a program to perform addition of two objects and store the result in third object. Display contents of all the three objects.**

```
#include<iostream.h>
#include<constream.h>

class number
{
 public:
 int x;
 int y;
 number() { } // ZERO ARGUMENT CONSTRUCTOR
 number (int j, int k) // TWO ARGUMENT CONSTRUCTOR
 {
 x=j;
 y=k;
 }
 void show()
 {
 cout<<"\n x="<<x<<"y="<<y;
 }
};

void main()
{
 clrscr();
 number A(2,3),B(4,5),C;
 A.show();
 B.show();
 C.x=A.x+B.x; // ADDITION OF TWO OBJECTS
 C.y=A.y+B.y; // USING MEMBER VARIABLES DIRECTLY
 C.show();
}
```

**OUTPUT**

**x=2 y=3**

**x=4 y=5**

**x=6 y=8**

**Explanation:** In the above program, A, B, and C are objects of class number. Using constructor, objects are initialized. Consider the following statements:

C.x=A.x+B.x;

C.y=A.y+B.y;



In the above statements, addition of members of objects *A* and *B* is performed and stored in *C*. Each member variable is accessed individually and stored in member variable of *C*. For example, member *x* of *A* and member *x* of *B* are added and stored in *x* of *C*. Similarly, addition of the other members is carried out.

In this program we cannot perform operation  $C=A+B$ . The operation with objects is complicated because it involves operation of one or more data member variables which are part of objects.

## OPERATOR OVERLOADING

The capability to relate the existing operator with a member function and use the resulting operator with objects of its class as its operands is called operator overloading.

### 10.2 THE KEYWORD OPERATOR

The keyword `operator` defines a new action or operation to the operator.

```
Syntax:
Return type Operator operator symbol (pa-
rameters)
{
 statement1;
 statement2;
}
```

The keyword `"operator"`, followed by an operator symbol, defines a new (overloaded) action of the given operator.

```
Example:
number operator + (number D)
{
 number T;
 T.x=x+D.x;
 T.y=y+D.y;
 return T;
}
```

Overloaded operators are redefined within a C++ class using the keyword `operator` followed by an operator symbol. When an operator is overloaded, the produced symbol is called the operator function name. The above declarations provide an extra meaning to the operator. Operator functions should be either member functions or friend functions. A friend function requires one argument for unary operators and two for binary operators, while a member function requires one argument for binary operators and no argument for unary operator. When the member function is called, the calling object is passed implicitly to the function and hence available for member function. While using friend functions, it is essential to pass the objects by value or reference. The prototype of operator functions in classes can be written as follows.

```

(a) void operator ++();
(b) void operator --();
(c) void operator -();
(d) num operator+(num);
(e) friend num operator * (int ,num);
(f) void operator = (num);

```

Operator overloading can be carried out in the following steps.

- (a) Define a class which is to be used with overloading operations.
- (b) The public declaration section of the class should contain the prototype of the function `operator()`.
- (c) Define the definition of the `operator()` function with proper operations for which it is declared.

### 10.2 Write a program to perform addition of two objects using `operator` keyword.

```

#include<iostream.h>
#include<constream.h>

class number
{
 public:
 int x;
 int y;
 number() { } // ZERO ARGUMENT CONSTRUCTOR
 number (int j, int k) // TWO ARGUMENT CONSTRUCTOR
 {
 x=j;
 y=k;
 }
 number operator + (number D)
 {
 number T;
 T.x=x+D.x;
 T.y=y+D.y;
 return T;
 } void show()
 {
 cout<<"\n x="<<x<<"y="<<y;
 }
};

void main()
{
 clrscr();

```

```

 number A(2,3), B(4,5), C;
 A.show();
 B.show();
 C=A+B;
 C.show();
}

```

## OUTPUT

```

x=2 y=3
x=4 y=5
x=6 y=8

```

**Explanation:** In the above program *A*, *B*, and *C* are objects of class *number*. Here, we performed the addition using statement *C=A+B*. Remember, in the last program we were not able to execute this statement. Instead of this, two separate statements were used to perform addition.

In this program, the statements that perform addition operation of each individual member of objects are written in function operator. The operator has return type and single argument. It also uses a local object (*T*) to hold addition as long as the operator function is active. Whenever the statement *C=A+B* is executed, the compiler searches for definition of operator *+*. The object *A* invokes the operator function and object *B* is passed as argument. The copy of object *B* is stored in the formal argument *D*. The member variables of *A* are directly available in operator function as the function is invoked by the same object. The addition of individual members are carried out and stored in member variable of object *T*. The return type of operator function is same as that of its class. The function returns object *T* and it is assigned to variable *C*.

## 10.3 OVERLOADING UNARY OPERATORS

Overloading devoid of explicit argument to an operator function is called unary operator overloading. The operators *++*, *--*, and *-* are unary operators. The unary operators *++* and *--* can be used as *prefix* or *suffix* with the functions. These operators have only single operand. The following examples illustrate the overloading of unary operators.

### 10.3 Write a program to increment member variables of object. Overload unary *++* operator.

```

#include<iostream.h>
#include<constream.h>

class num
{
 private:
 int a,b,c,d;
 public :
 num (int j, int k, int m ,int l)
 {

```

```

 a=j;
 b=k;
 c=m;
 d=1;
}
void show(void);
void operator ++();
};
void num :: show()
{
 cout<<"A="<<a<<"B=" <<b<<"C="<<c<<"D="<<d;
}
void num :: operator ++()
{
 ++a; ++b; ++c; ++d;
}
main()
{
 clrscr();
 num X(3,2,5,7);
 cout<<"\n Before Increment of X:";
 X.show();
 ++X;
 cout<<"\n After Increment of X:";
 X.show();
 return 0;
}

```

## OUTPUT

**Before Increment of X : A= 3 B= 2 C= 5 D= 7**

**After Increment of X : A= 4 B= 3 C= 6 D= 8**

**Explanation:** In the above example the class num contains four integer variables a, b, c, and d. The class also has two-member functions show() and operator ++() and one parameterized constructor. The constructor is used to initialize object. The show() displays the contents of the member variables. The operator ++() overloads the unary operator ++. When this operator is used with integer or float variables, its value is increased by one. In this function, ++ operator precedes each member variable of class. This operation increments the value of each variable by one.

In function main(), the statement ++X calls the function operator ++(), where X is an object of the class num. The function can also be called using statement X. operator ++(). In the output, values of member variables before and after increment operations are displayed.

**10.4 Write a program to overload – operator.**

```

#include<iostream.h>
#include<conio.h>

class num
{
 private:
 int a,b,c,d;
 public :
 num (int x, int y, int z, int w)
 {
 a=x;
 b=y;
 c=z;
 d=w;
 }
 void show(void);
 void operator - ();
}; void num :: show()
{
 cout<<"A="<<a<<"B=" <<b<<"C="<<c<<"D="<<d;
}

void num :: operator - ()
{
 a=-a;
 b=-b;
 c=-c;
 d=-d;
} main()
{
 clrscr();
 num X(2,2,8,4);
 cout<<"\nBefore Negation of X:";
 X.show();
 -X;
 cout<<"\nAfter Negation X:";
 X.show();
 return 0;
}

```

**OUTPUT**

**Before Negation of X : A= 2 B= 2 C = 8 D = 4**

**After Negation X : A= -2 B= -2 C = -8 D = -4**

**Explanation:** The above program is same as previous one. Here, the operator – is overloaded. The statement –X calls the function operator – (). The function operator– () makes all the member variables negative. The function show () displays the values of member variables on the screen.

## 10.4 OPERATOR RETURN TYPE

In the last few examples we declared the operator () of void types, that is, it will not return any value. However, it is possible to return value and assign to it other object of the same type. The return value of operator is always of class type, because the operator overloading is only for objects. An operator cannot be overloaded for basic data type. Hence, if the operator returns any value, it will be always of class type. Consider the following program.

### 10.5 Write a program to return values from operator () function.

```
#include<iostream.h>
#include<conio.h>

class plusplus
{
 private:
 int num;
 public :
 plusplus() { num=0; }
 int getnum() { return num;}
 plusplus operator ++ (int)
 {
 plusplus tmp;
 num=num+1;
 tmp.num=num;
 return tmp;
 }
};

void main()
{
 clrscr();
 plusplus p1, p2;
 cout<<"\n p1="<<p1.getnum();
 cout<<"\n p2="<<p2.getnum();
 p1=p2++;
 cout<<endl<<" p1="<<p1.getnum();
 cout<<endl<<" p2="<<p2.getnum();
 p1++;
 // p1++=2;
 cout<<endl<<" p1="<<p1.getnum();
 cout<<endl<<" p2="<<p2.getnum();
}
```

**OUTPUT**

```

p1 = 0
p2 = 0
p1 = 1
p2 = 1
p1 = 2
p2 = 1

```

**Explanation:** In the above program class `plusplus` is declared with one private integer `num`. The class constructor initializes the object with zero. The member function `getnum()` returns current value of variable `num`. The operator `++()` is overloaded and it can handle as postfix increment of the objects. In case of an increase in the prefix it will flag an error.

The `p1` and `p2` are objects of the class `plusplus`. The statement `p1=p2++` first increments the value of `p2` and then assigns it to the object `p1`. The values displayed will be one for the objects. The object `p1` is increased. This time, the values of object displayed will be two and one.

## 10.5 CONSTRAINT ON INCREMENT AND DECREMENT OPERATORS

When an operator (increment/decrement) is used as prefix with object, its value is incremented/decremented before operation, and, on the other hand, the postfix use of operator increments/decrements the value of variable after its use.

When `++` and `--` operators are overloaded, there exists no difference between the postfix and prefix overloaded operator functions. The system has no way of determining whether the operators are overloaded for postfix or prefix operation. Hence, the operator must be overloaded in such a way that it will work for both prefix and postfix operations. The `++` or `--` operator overloaded for prefix operation works for both prefix and postfix operations but with warning message, but vice versa is not possible. To make a distinction between prefix and postfix notations of operator, a new syntax is used to indicate postfix operator overloading function. The syntaxes are as follows.

```

Operator ++ (int) // postfix notation
Operator ++() // prefix notation

```

The argument followed by operator (`++` or `--`) should have type `'int'`. When a postfix operator `++` or operator `--` is declared, the last parameter must be declared with the type `int`. No other types such as `float`, `long`, etc. are allowed. We can use this operator with all types of variables including `float`, `long`, etc. Declaring `int` does not mean that it is only for integer type. The following program illustrates overloading of `++` operator in postfix and prefix styles.

### 10.6 Write a program to overload `++` and `--` operator for prefix and postfix use.

```

#include<iostream.h>
#include<constream.h>

class number
{
 float x;

```

```

public:
number (float k) { x=k; }
void operator ++ (int) // postfix notation
{ x++; }
void operator --() // prefix notation
{ --x; }
void show() { cout<<"\n x="<<x; }
};
void main()
{
 clrscr();
 number N(2.3);
 cout<<"\n Before Incrimenting: ";
 N.show();
 cout<<"\n After Incrimenting: ";
 N++; // postfix increment
 N.show();
 cout<<"\n After Decrementation:";
 --N; // prefix decrement
 N.show();
}

```

## OUTPUT

**Before incrementing:**

**x=2.3**

**After incrementing:**

**x=3.3**

**After decrementation:**

**x=2.3**

**Explanation:** In this program, operator ++ and – are overloaded. The ++ operator is overloaded for postfix use and – operator is overloaded for prefix use. You can see that the keyword (int) is followed by the operator ++, which is necessary for postfix notation of operator. The operator – is overloaded for prefix operation. Here, a value of float member variable is incremented and decremented.

## 10.6 OVERLOADING BINARY OPERATORS

Overloading with a single parameter is called binary operator overloading. Similar to unary operators, binary operators can also be overloaded. Binary operators require two operands, and they are overloaded by using member functions and friend functions.

### (1) Overloading binary operators using member functions

If overloaded as a member function, binary operators require one argument. The argument contains value of the object, which is to the right of the operator. If we want to perform the addition of two objects o1 and o2, the overloading function should be declared as follows.



```
operator(num o2);
```

Where, num is a class name and o2 is an object.  
To call function operator the statement is as follows:  
o3=o1+o2;

We learnt that a member function can be called by using class of that object. Hence, the called member function is always preceded by the object. Here, in the above statement, the object o1 invokes the function operator() and the object o2 is used as an argument for the function. The above statement can also be written as follows.

```
o3=o1.operator +(o2);
```

Here, the data members of o1 are passed directly and data members of o2 are passed as an argument. While overloading binary operators, the left-hand operand calls the operator function and the right-hand operator is used as argument.

(2) Overloading binary operators using friend functions

The friend functions can be used alternatively with member functions for overloading of binary operators. The friend function requires two operands to be passed as arguments.

```
o3=o1+o2;
o3=operator+(o1,o2);
```

Both the above statements have the same meaning. In the second statement, two objects are passed to the operator function.

The use of member function and friend function produces the same result. friend functions are useful when we require performing an operation with operand of two different types. Consider the statements

```
X=Y+3;
X=3+Y;
```

where X and Y are objects of the same type. The first statement is valid. However, the second statement will not work. The first operand must be an object of the same class. This problem can be overcome by using friend function. The friend function can be called without using an object. The friend function can be used with standard data type as left-hand operand and an object as right-hand operand.

The following programs are illustrated based on the above discussion.

### 10.7 Write a program to overload + binary operator.

```
#include<iostream.h>
#include<conio.h>

class num
{
 private:
```

```

 int a,b,c,d;
 public:
 void input(void);
 void show(void);
 num operator+(num);
};
void num :: input()
{
 cout<<"\n Enter Values for a,b,c and d:";
 cin>>a>> b>>c>>d;
}
void num :: show()
{
 cout<<"A="<<a <<"B=" <<b <<"C="<<c <<"D="<<d <<"\n";
}
num num :: operator +(num t)
{
 num tmp;
 tmp.a=a+t.a;
 tmp.b=b+t.b;
 tmp.c=c+t.c;
 tmp.d=d+t.d;
 return (tmp);
}
main()
{
 clrscr();
 num X,Y,Z;
 cout<<"\n Object X";
 X.input();
 cout<<"\n Object Y";
 Y.input();
 Z=X+Y;
 cout<<"\nX:";
 X.show();
 cout<<"Y:";
 Y.show();
 cout<<"Z:";
 Z.show();
 return 0;
}

```

## OUTPUT

### Object X

Enter Values for a,b,c and d : 1 4 2 1

**Object Y****Enter Values for a,b,c and d : 2 5 4 2****X : A= 1 B= 4 C = 2 D = 1****Y : A= 2 B= 5 C = 4 D = 2****Z : A= 3 B= 9 C = 6 D = 3**

**Explanation:** In the above program, binary operator + is overloaded. Using the overloading operator +, addition of member variables of two objects is performed and results are assigned to member variables of third object. In this program X, Y, and Z are objects of class num. The statement Z=X+Y invokes the operator function. In this statement, the object y is assigned to object t of operator function and member variables of X are accessed directly. The object tmp is used for holding the result of addition, and it is returned to object Z after function execution. The function show() displays the values of three objects.

**10.8 Write a program to perform multiplication using an integer and object. Use friend function.**

```
#include<iostream.h>
#include<conio.h>

class num
{
 private:
 int a,b,c,d;
 public :
 void input(void);
 void show(void);
 friend num operator * (int ,num); // friend function
 declaration
};

void num :: input()
{
 cout<<"\n Enter Values for a,b,c and d:";
 cin>>a>> b>>c>>d;
}

void num :: show()
{
 cout<<"A="<<a<<"B=" <<b<<"C="<<c<<"D="<<d <<"\n" ;
}

num operator * (int a, num t)
{
 num tmp;
 tmp.a=a*t.a;
 tmp.b=a*t.b;
 tmp.c=a*t.c;
```

```
 tmp.d=a*t.d;
 return (tmp);
 }
main()
{
 clrscr();
 num X,Z;
 cout<<"\n Object X";
 X.input();
 Z=3*X;
 cout<<"\nX:";
 X.show();
 cout<<"Z:";
 Z.show();
 return 0;
}
```

## OUTPUT

### Object X

Enter Values for a,b,c and d : 1 2 2 3

X : A= 1 B= 2 C= 2 D= 3

Z : A= 3 B= 6 C= 6 D= 9

**Explanation:** In the above program, the equation  $Z=3*X$  contains integer and class object. We know that the left-hand operand is always used to invoke the function and the right-hand operand is passed as an argument. In such type of equations, member functions are not useful because the left-hand operand is integer and cannot invoke the function. Hence, the function operator  $*$  () is declared as friend. The friend function calls the operator  $*$  () and carries the multiplication of each member variable by three. The results are displayed in the output.

## 10.7 OVERLOADING WITH friend FUNCTION

The friend functions are more useful in operator overloading. They offer better flexibility, which is not provided by the member function of the class. The difference between member function and friend function is that the member function takes argument explicitly. On the contrary, the friend function needs the parameters to be explicitly passed. The syntax of operator overloading with friend function is as follows:

```
friend return-type operator operator-symbol (variable1, variable2)
{
 statement1;
 statement2;
}
```

The keyword `friend` precedes function prototype declaration. It must be written inside the class. The function can be defined inside or outside the class. The arguments used in `friend` function are generally objects of the `friend` classes. A `friend` function is similar to normal function; the only difference being that `friend` function can access private member of the class through the objects. `friend` function has no permission to access private members of a class directly. However, it can access the private members through objects of the same class.

### 10.9 Write a program to overload unary operator using `friend` function.

```
#include<iostream.h>
#include<constream.h>

class complex
{
 float real,imag;
public:
 complex() // zero argument constructor
 {
 real=imag=0;
 }
 complex (float r, float i) // two argument constructor
 {
 real=r;
 imag=i;
 }
 friend complex operator - (complex c)
 {
 c.real=-c.real;
 c.imag=-c.imag;
 return c;
 }
 void display()
 {
 cout<<"\n Real:"<<real;
 cout<<"\n Imag:"<<imag;
 }
};

void main()
{
 clrscr();
 complex c1(1.5,2.5),c2;
 c1.display();
 c2=-c1;
 cout<<"\n\n After Negation \n";
 c2.display();
}
```



**OUTPUT****Real : 1.5****Imag : 2.5****After Negation****Real : -1.5****Imag : -2.5**

**Explanation:** In the above program, operator `-` is overloaded using friend function. The operator function is defined as friend. The statement `c2=-c1` invokes the operator function. This statement also returns the negated values of `c1` without affecting actual value of `c1` and assigns it to object `c2`.

The negation operation can also be used with an object to alter its own data member variables. In such a case, the object itself acts as a source and destination object. This can be accomplished by sending reference of object. The following program illustrates this.

**10.10 Write a program to pass reference of an object to operator function.**

```
#include<iostream.h>
#include<constream.h>

class complex
{
 float real,imag;
public:
 complex (float r, float i) // two argument constructor
 {
 real=r;
 imag=i;
 }
 friend void operator - (complex & c)
 {
 c.real=-c.real;
 c.imag=-c.imag;
 }
 void display()
 {
 cout<<"\n Real:"<<real;
 cout<<"\n Imag:"<<imag;
 }
};

void main()
{
 clrscr();
 complex c1(1.5,2.5);
 c1.display();
```

```

 -c1;
 cout<<"\n\n After Negation \n";
 c1.display();
}

```

## OUTPUT

```

Real : 1.5
Imag : 2.5
After Negation
Real : -1.5
Imag : -2.5

```

**Explanation:** In the above program, the object `c1` itself acts as a source and destination object. The reference of object is passed to operator function. The object `c` is a reference object of `c1`. The values of object `c` are replaced by itself by applying negation.

## 10.8 OVERLOADING ASSIGNMENT OPERATOR (=)

In this section, data members of one object are initialized with some values, and the same values are assigned to another object with assignment operator. Assignment operator can be overloaded by two ways:

- (1) Implicit overloading
- (2) Explicit overloading

Following program demonstrates overloading of assignment (=) operator implicitly:

### 10.11 Program to overload the assignment operator (=) implicitly.

```

#include<iostream.h>
#include<conio.h>

class num
{
 private:
 int x;
 public:
 num(int a);
 void show();
};

int main()
{
 clrscr();
 num a1(2), a2(8);
 cout<<"\n Before overloading assignment operator:";
 cout<<"\n A=";

```

```

 a1.show();
 cout<<"\n B=";
 a2.show();
 a2=a1; //Implicit overloading assignment operator
 cout<<"\n After overloading assignment operator
implicitly(b=a): ";
 cout<<"\n A=";
 a1.show();
 cout<<"\n B=";
 a2.show();
 return 0;
}
num::num(int a)
{
 x=a;
}
void num::show()
{
 cout<<x<<" ";
}

```

## OUTPUT

**Before overloading assignment operator:**

**A = 2**

**B = 8**

**After overloading assignment operator implicitly(b=a):**

**A = 2**

**B = 2**

Following program demonstrates overloading of assignment (=) operator explicitly:

### 10.12 Program to overload the assignment operator (=) explicitly.

```

#include<iostream.h>
#include<conio.h>

class num
{
 private:
 int x;
 public:
 num(int a);
 void operator = (num b);
 void show();
};

```



```
int main()
{
 clrscr();
 num a1(100), a2(200);
 cout<<"\n Before overloading assignment operator:";
 cout<<"\n A=";
 a1.show();
 cout<<"\n B=";
 a2.show();
 cout<<"\nAfter overloading assignment operator Explicitly:";
 a2.operator=(a1);
 cout<<"\n A=";
 a1.show();
 cout<<"\n B=";
 a2.show();
 return 0;
}

num::num(int a)
{
 x=a;
}

void num::show()
{
 cout<<x<<" ";
}

void num::operator = (num b)
{
 x=b.x;
}
```

## OUTPUT

**Before overloading assignment operator:**

**A = 100**

**B = 200**

**After overloading assignment operator Explicitly:**

**A = 100**

**B = 100**

## 10.9 TYPE CONVERSION

We learnt that when constants and variables of various data types are clubbed in a single expression, automatic type conversion takes place. This is so for basic data types. The compiler is unknown about the user-defined data type and about their conversion to other data types.

The programmer should write the routines that convert basic data type to user-defined data type or vice versa. There are three possibilities of data conversion as given below:

- (a) Conversion from basic data type to user-defined data type (class type)
- (b) Conversion from class type to basic data type
- (c) Conversion from one class type to another class type

### 10.9.1 Conversion from Basic to Class Type

The conversion from basic to class type is easily carried out. It is automatically done by the compiler with the help of in-built routines or by applying type casting. In this type, the left-hand operand of = sign is always class type and the right-hand operand is always basic type. The below-given program explains the conversion from basic to class type.

**10.13 Write a program to define constructor with no argument and with float argument. Explain how compiler invokes constructor depending on data type.**

```
#include<iostream.h>
#include<conio.h>

class data
{
 int x;
 float f;
public :
 data()
 {
 x=0;
 f=0;
 }
 data (float m)
 {
 x=2;
 f=m;
 }
 void show()
 {
 cout<<"\n x= "<<x<<" f="<<f;
 cout<<"\n x= "<<x<<"f="<<f;
 }
};

int main()
{
 clrscr();
 data z;
 z=1;
 z.show();
}
```



```

 z=2.5;
 z.show();
 return 0;
}

```

### OUTPUT

```

x= 2 f= 1
x= 2 f= 1
x= 2 f= 2.5
x= 2 f= 2.5

```

**Explanation:** In the above program, the class `data` has two member variables each of integer and float types respectively. It also has two constructors one with no argument and the other with float argument. The member function `show()` displays the contents of the data members. In function `main()`, `z` is an object of class `data`. When `z` is created, the constructor with no argument is called and data members are initialized to zero. When `z` is initialized to one, the constructor with float argument is invoked. The integer value is converted to float type and assigned member variable `f`. Again when `z` is assigned to 2.5, same process is repeated. Thus, the conversion from basic to class type is carried out.

### 10.9.2 Conversion from Class Type to Basic Data Type

In the previous example, we studied how compiler makes conversion from basic to class type. The compiler does not have any knowledge about the user-defined data type built using classes. In this type of conversion, the programmer needs to explicitly tell the compiler how to perform conversion from class to basic type. These instructions are written in a member function. Such type of conversion is also known as overloading of type cast operators. The compiler first searches for the operator keyword followed by data type and if it is not defined, it applies the conversion functions. In this type, the left-hand operand is always of basic data type and the right-hand operand is always of class type. During this conversion, the statement should satisfy the following conditions:

- (1) The conversion function should not have any argument.
- (2) Do not mention return type.
- (3) It should be a class member function.

### 10.14 Write a program to convert class type data to basic type data.

```

#include<iostream.h>
#include<conio.h>

class data
{
 int x;
 float f;

```

```
public:
data()
{
x=0;
f=0;
}
operator int()
{
return (x);
}
operator float()
{ return f; }
data (float m)
{
x=2;
f=m;
}
void show()
{
cout<<"\n x= "<<x<<"f="<<f;
cout<<"\n x= "<<x<<"f="<<f;
}
};
int main()
{
clrscr();
int j;
float f;
data a;
a=5.5;
j=a; // operator int() is executed
f=a; // operator float() is executed
cout<<"\n Value of j : "<<j;
cout<<"\n Value of f : "<<f;
return 0;
}
```

## OUTPUT

**Value of j : 2**

**Value of f : 5.5**

**Explanation:** In the above program, the class data has two member variables each of integer and float data type. It also contains constructors as per described in the last example.

In addition, it contains overloaded data types `int` and `float`. These functions are useful for conversion of data from class type to basic type. Consider the following statements:

```
(a) j=a;
(b) f=a;
```

In the first statement object `a` is assigned to integer variable `j`. We know that class type data is a combination of one or more basic data types. The class contains two member functions `operator int()` and `operator float()`. Both these function are able to convert data types from class to basic. In statement (a), variable `j` is of integer type, the function `operator int()` is invoked and integer value data member is returned. In statement (b), `f` is of `float` type, the member function `operator float()` is invoked.

### 10.9.3 Conversion from One Class Type to Another Class Type

We learnt how to convert basic data type to class type and vice versa. Now the third method is conversion from class type to another class type. When an object of one class is assigned to object of another class, it is necessary to give clear-cut instructions to the compiler about how to make conversion between these two user-defined data types. The method must be instructed to the compiler. There are two ways to convert object data type from one class to another. One is to define a conversion operator function in source class or a one-argument constructor in a destination class. Consider the following example:

```
X=A;
```

Here, `X` is an object of class `XYZ` and `A` is an object of class `ABC`. The class `ABC` data type is converted to class `XYZ`. The conversion happens from class `ABC` to `XYZ`. The `ABC` is a source class and `XYZ` is a destination class.

We know the operator function `operator data-type()`. Here, data type may be built-in data type or user-defined data type. In the above declaration, the data type indicates target type of object. Here, the conversion takes place from class `ABC` (source class) to class `XYZ` (destination class).

### 10.15 Write a program to convert integer to date and vice versa using conversion function in source class.

```
#include<iostream.h>
#include<stdlib.h>
#include<string.h>
#include<conio.h>

class date
{
 char d[10];
public:
 date() { d[0]=NULL; }
 date (char *e) { strcpy(d,e); }
 void show() { cout<<d; }
};
```

```
class dmy
{
 int mt,dy,yr;
public:
 dmy() { mt=dy=yr=0; }
 dmy (int m, int d, int y)
 {
 mt=m;
 dy=d;
 yr=y;
 }
 operator date()
 {
 char tmp[3],dt[9];
 itoa(dy,dt,10);
 strcat(dt,"-");
 itoa(mt,tmp,10);
 strcat(dt,tmp);
 strcat(dt,"-");
 itoa(yr,tmp,10);
 strcat(dt,tmp);
 return (date(dt));
 }
 void show()
 {
 cout<<dy<<" "<<mt<<" "<<yr;
 }
};

int main()
{
 clrscr();
 date D1;
 dmy D2(1,7,99);
 D1=D2;
 cout<<endl<<"D1=";
 D1.show();
 cout<<endl<<"D2=";
 D2.show();
 return 0;
}
```

## OUTPUT

**D1=7-1-99**

**D2=7 1 99**

**Explanation:** In the above program, `date` and `dmy` are two classes declared. In function `main()`, `D1` is an object of class `date` and `D2` is an object of class `dmy`. The object `D2` is initialized.

The statement `D1=D2` initializes `D1` with `D2`. Here, both the objects `D1` and `D2` are of different types hence the conversion function `date()` is called to perform the conversion from one object to another object.

### 10.16 Write a program to convert integer to date and vice versa using conversion function in destination class.

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
#include<string.h>

class dmy
{
 int d,m,y;
public:
 dmy() { d=m=y=0; }
 dmy (int da, int ma, int ya) { d=da; m=ma; y=ya; }
 int day() { return (d); }
 int month() { return (m); }
 int year() { return (y); }
 void show() { cout<<d<<" "<<m<<" "<<y; }
};

class date
{
private :
 char dts[9];
public :
 date() { dts[0]=0; }
 date (char *e) { strcpy (dts,e); }
 void show() { cout<<dts; }
 date (dmy k)
 {
 int d=k.day(); // first member function
 int m=k.month(); // second member function
 int y=k.year(); // third member function
 char tmp[3];
 itoa(d,dts,10);
 strcat(dts,"-");
 itoa(m,tmp,10);
 strcat(dts,tmp);
 strcat(dts,"-");
 }
};
```

```

 itoa(y,tmp,10);
 strcat(dts,tmp);
 }
};
int main()
{
 clrscr();
 date D1;
 dmy D2(1,3,77);
 D1=D2;
 cout<<endl<<"D1=";
 D1.show();
 cout<<endl<<"D2=";
 D2.show();
 return 0;
}

```

## OUTPUT

**D1=1-3-77**

**D2=1 3 77**

**Explanation:** In the above program, as soon as the statement `D1=D2` is executed, the one-argument constructor defined in the class `date` is invoked. The one-argument constructor carries the conversion. The constructor calls three-member function of `dmy` class to get the day, month, and year of the date. Here, in this program conversion is done by using the constructor in destination object.

### 10.17 Write a program to convert class data type to another class data type.

```

#include<iostream.h>
#include<conio.h>

class minutes
{
 int m;
public:
 minutes()
 { m=240; }
 get()
 { return (m); }
 void show()
 { cout<<"\n Minutes="<<m; }
};

class hours

```



```

{
 int h;
 public:
 void operator = (minutes x);
 void show()
 { cout<<"\n Hours="<<h; }
};
void hours:: operator = (minutes x)
{
 h=x.get()/60;
}
int main()
{
 clrscr();
 minutes minute;
 hours hour;
 hour=minute;
 minute.show();
 hour.show();
 return 0;
}

```

## OUTPUT

**Minutes = 240**

**Hours = 4**

**Explanation:** In the above program two classes are declared. The class minutes has one integer member variable m and two member functions get () and show (). It also contains constructor without argument. The class hours has one integer member variable and show () member function. The class hours contain overloaded operator function. In function main (), minute is an object of class minutes and hour is an object of class hours. The program converts minutes to hours. The equation hour=minute invokes the operator function. In function operator (), x is an object of class minutes. The object x invokes the member function get () that returns total number of minutes. Number of hours is obtained by dividing the total number of minutes by 60. The equation h=x.get () /60 performs this task and assigns result to h. Thus, the result of the program is as per given above.

## 10.10 RULES FOR OVERLOADING OPERATORS

- Overloading of an operator cannot change the basic idea of an operator. When an operator is overloaded, its properties like syntax, precedence, and associativity remain constant. For example A and B are objects. The following statement

A+=B;

assigns addition of objects  $A$  and  $B$  to  $A$ . The overloaded operator must carry the same task like original operator according to the language. The following statement must perform the same operation like last statement:

```
A=A+B;
```

- Overloading of an operator must never change its natural meaning. An overloaded operator  $+$  can be used for subtraction of two objects, but this type of code decreases the utility of the program. Remember that the aim of operator overloading is to comfort the programmer to carry various operations with objects. Consider the following program:

#### 10.18 Misuse of operator overloading. Perform subtraction using $+$ operator.

```
#include<iostream.h>
#include<constream.h>

class num
{
 int x;
public:
 num() { x=0; }
 num (int k) { x=k; }
 num operator + (num n)
 {
 num s;
 s.x=x-n.x;
 return s;
 }
 void show() { cout<<"\n x="<<x; }
};

void main()
{
 clrscr();
 num a(10), b(5), c;
 c=a+b;
 c.show();
}
```

#### OUTPUT

**X=5**

**Explanation:** In the above program, the operator  $+$  is overloaded. It performs subtraction. Such type of misuse must be avoided while overloading operators. The programmer thought that it would perform addition, but in reality it performs subtraction.

- The overloaded operator should contain one operand of user-defined data type. Overloading operators is only for classes. We cannot overload the operator for built-in data types.

- Overloaded operators have the same syntax as the original operator. They cannot be prevailing over the original operators.
- There is no higher limit to the number of overloading for any operator. An operator can be overloaded for a number of times if the arguments are different in each overloaded operator function.
- Operator overloading is applicable within the scope (extent) in which overloading occurs.
- Only existing operators can be overloaded. We cannot create a new operator.
- C++ has a wide range of operators. However, few operators cannot be overloaded to operate in a same manner like built-in operators. The operators given in Table 10.1 cannot be overloaded.

**Table 10.1** Non-overloadable Operators

| Operator | Description                |
|----------|----------------------------|
| .        | Member operator            |
| . *      | Pointer to member operator |
| ::       | Scope access operator      |
| ?:       | Conditional operator       |
| sizeof() | Size of operator           |
| # and ## | Preprocessor symbols       |

For example, operators such as `?:`, `::`, and `. *` are combinations of more than one symbol. The condition operator needs three arguments. It is only one operator that requires three arguments. Hence, the above operators cannot be overloaded.

- The operators given in Table 10.2 cannot be overloaded using `friend` function.

**Table 10.2** Non-overloadable Operators with `friend` Function

| Operator | Description                      |
|----------|----------------------------------|
| ()       | Function call delimiter/operator |
| =        | Assignment operator              |
| []       | Subscripting operator            |
| ->       | Class member access operator     |

- When unary operator is overloaded using member functions, it requires no explicit `friend` argument and returns no values. When it is overloaded using `friend` function, it requires one reference argument.
- When binary operator is overloaded using `friend` function, it requires two arguments. When it is overloaded using member function, it requires one argument.

## 10.11 ONE-ARGUMENT CONSTRUCTOR AND OPERATOR FUNCTION

We studied that a single-argument constructor or an operator function could be used for conversion of objects of different classes. A large range of classes as class libraries are available with the compiler; however, they are linked with the main program. Their source code is invisible to

us. Only object of these classes can be used. The user cannot change the in-built classes. The problem occurs when a programmer attempts conversion from object of class declared by him/her to type of in-built class. This problem can be avoided by defining conversion routine in the user-defined class. The conversion routine may be single-argument constructor or an operator function. It depends on whether the object is a source or destination. Table 10.3 describes conversion type and place of routine to be defined, followed by description.

**Table 10.3** Conversion Types

| S. No. | Conversion Type | Routine in Destination Class | Routine in Source Class |
|--------|-----------------|------------------------------|-------------------------|
| A      | Class to class  | Constructor                  | Conversion function     |
| B      | Class to basic  | Not applicable               | Conversion function     |
| C      | Basic to class  | Constructor                  | Not applicable          |

- (a) In case both the source and destination objects are of user-defined type, the conversion routine can be carried out using operator function in source class or using constructor in destination class.
- (b) If the user-defined object is a destination object, the conversion routine should be carried out using single-argument constructor in the destination object's class.
- (c) In case the user-defined object is a source object, the conversion routine should be carried out using an operator function in the source object's class.

## Avoid Ambiguity

Defining multiple conversion routines puts the compiler in an uncertain condition. The compiler fails to select appropriate conversion routines. For example, if one argument constructor is present in destination class and operator function in source class, the compiler cannot select appropriate routines. Hence, while defining conversion routines, follow the conditions given in Table 10.3.

## 10.12 OVERLOADING STREAM OPERATORS

The predefined objects `cin` and `cout` are used to perform various input/output operations in C++. The extraction operator (`>>`) is used with `cin` object to carry out input operations. The insertion operator (`<<`) is used with `cout` object to carry out output operations. In Chapter 2 we learnt how to create objects similar to `cin` and `cout`. It is also possible to overload both these extraction and insertion operators with `friend` function.

The syntax for overloading (`<<`) insertion operator is as follows:

```
friend ostream & operator << (ostream & put , vl)
{
 // code
 return put;
}
```

The keyword `friend` precedes the declaration. The `ostream` is an output stream class followed by reference and keyword `operator`. The `put` is an output stream object like `cout`. The `v1` is a user-defined class object. The following program explains overloading of insertion operator with `friend` function.

**10.19 Write a program to overload insertion operator (<<) with friend function.**

```
#include<iostream.h>
#include<constream.h>

class string
{
 char *s;
public:
 string (char *k)
 {
 s=k;
 }
 friend ostream & operator << (ostream &put, string & k)
 {
 put <<k.s;
 return put;
 }
};

int main()
{
 clrscr();
 string s("INDIA");
 cout<<s;
 return 0;
}
```

**OUTPUT**

**INDIA**

**Explanation:** In the above program the insertion operator << is overloaded with `friend` function. The overloaded operator allows us to display contents of object directly using `cout` statement. The statement `cout<<s;` displays contents of object `s` on the screen.

Similarly, extraction operator can be overloaded. The syntax for overloading extraction operator is as follows:

```
friend istream & operator >> (istream & get, v2)
{
 // code
 return get;
}
```

The following program explains overloading of extraction operator.

**10.20 Write a program to overload extraction operator using friend function.**

```
#include<iostream.h>
#include<constream.h>

class string
{
 char *s;
public:
 friend istream & operator >> (istream &get, string & k)
 {
 cout<<"\n Enter a string :";
 get >>k.s;
 return get;
 }
};

int main()
{
 clrscr();
 string s;
 cin>>s; // input string
 return 0;
}
```

**OUTPUT****BEST LUCK**

**Explanation:** This program is same as the previous one. Here, extraction operator is overloaded. The object `s` is directly used with `cin` statement. After execution of this statement, the overloaded operator is invoked.

**10.13 MORE PROGRAMS****10.21 Write a program to overload > operator.**

```
#include<iostream.h>
#include<conio.h>

class A
{
 int a;
public:
 int operator > (A);
 void inA ()
 {
```

```

 cout<<"\n Enter a number :";
 cin>>a;
}
void outA() { cout<<"\n Largest Number="<<a;}
};
int A ::operator > (A l)
{
 if (a>l.a)
 return 1;
 else
 return 0;
}
int main()
{
 clrscr();
 A J,K;
 J.inA();
 K.inA();
 if (J>K) J.outA();
 else K.outA();
 return 0;
}

```

## OUTPUT

**Enter a number : 9**  
**Enter a number : 4**  
**Largest Number = 9**

**Explanation:** In the above program, class A has one integer type data member and inA() and outA() are member functions. The function inA() is used to read an integer through the keyboard and outA() is used to display the contents of data variable on the screen. The operator > is overloaded. The comparison of explicit (K) and implicit (J) arguments is performed by the if statement. If J is greater, one is returned otherwise zero. If the return value is one, that is, J is greater in function main(), if block is executed otherwise else block is executed.

### 10.22 Write a program to overload != Operator.

```

#include<iostream.h>
#include<conio.h>

class A
{
 int a;
public:

```

```

 int operator != (A);
 void inA ()
 { cout<<"\n Enter a number:";
 cin>>a;
 }
 void m1() { cout<<"\n Both numbers are same "; }
 void m2() { cout<<"\n Both numbers are different "; }
};
int A ::operator != (A l)
{
 if (a!=l.a)
 return 0;
 else
 return 1;
}
int main()
{
 clrscr();
 A J,K;
 J.inA();
 K.inA();
 if (J!=K) J.m1();
 else K.m2();
 return 0;
}

```

## OUTPUT

**Enter a number : 4**

**Enter a number : 4**

**Both numbers are same**

**Explanation:** The above program is same as the previous one. The only difference is that (!=) not equal to operator is overloaded.

**10.23 Write a program to overload & operator. When this operator is used with the object, the program should prompt for inputting a number.**

```

#include<iostream.h>
#include<conio.h>

class A
{
 int a;
public:

```



```

 A operator & (A);
 void inA ()
 { cout<<"\n Enter a number:";
 cin>>a;
 }
 void outA()
 {
 cout<<"\n a="<<a;
 }
};
A A :: operator & (A l)
{
 cout<<"\n Enter Two Numbers:";
 cin>>a>>l.a;
 return (l);
}
int main()
{
 clrscr();
 A J,K;
 K=J&K;
 J.outA();
 K.outA();
 return 0;
}

```

## OUTPUT

**Enter Two Numbers : 8 9**

**a = 8**

**a = 9**

**Explanation:** In the above program the operator & is overloaded. J and K are objects of class A. The equation K=J&K calls the overloaded operator. The overloaded function reads integer and assigns to member variables of objects J and K. The return value is assigned to object K. The outA() function displays the contents of member variable of calling object.

### 10.24 Write a program to overload < and > operator.

```

#include<iostream.h>
#include<conio.h>

class A
{
 int a;
public:

```

```

 A operator > (A);
 void operator < (A);
};
A A ::operator > (A l)
{
 cout<<"\n Enter Two Numbers:";
 cin>>a>>l.a;
 return (l);
}
void A :: operator < (A l)
{
 cout<<"\nEntered Numbers are:"<<l.a<<" "<<a;
}
int main()
{
 clrscr();
 A J,K;
 K=J>J; // Reads numbers
 K<J; // Prints numbers
 return 0;
}

```

## OUTPUT

**Enter Two Numbers : 45 98**  
**Entered Numbers are : 45 98**

**Explanation:** In the above program the operator < and > are overloaded. The > operator is used for reading numbers and < is used for displaying numbers. The output of the program is shown above.

### 10.25 Write a program to add and multiply two complex objects. Use operator overloading.

```

#include<iostream.h>
#include<conio.h>

class complex
{
 public:
 double real, imag;
};

complex setdata(double rr, double ii);
void show (complex cc);
complex operator + (complex ca, complex cb);
complex operator * (complex ca, complex cb);

```

```
void main()
{
 clrscr();
 complex H,I,J,K;
 H=setdata(1.3,1.7);
 I=setdata(2.2,2.2);
 J=H+I;
 K=H*I;
 cout<<endl<<"H=";
 show (H);
 cout<<endl<<"I=";
 show (I);
 cout<<endl<<"J=";
 show (J);
 cout<<endl<<"K=";
 show(K);
}

complex setdata (double rr, double ii)
{
 complex tmp;
 tmp.real=rr;
 tmp.imag=ii;
 return tmp;
}

void show (complex s)
{
 cout<<"["<<s.real<<','<<s.imag<<"]";
}

complex operator + (complex ca, complex cb)
{
 complex tmp;
 tmp.real=ca.real+cb.real;
 tmp.imag=ca.imag+cb.imag;
 return tmp;
}

complex operator * (complex ca,complex cb)
{
 complex tmp;
 tmp.real=ca.real*cb.real;
 tmp.imag=ca.imag*cb.imag;
 return tmp;
}
```

**Explanation:** In the above program, the complex class has two data members of double data types. They are real and imaginary. The function `setdata()` is used to set values of class members. The function `show()` displays the contents of the object on the screen. The operators `+` and `*` are overloaded and defined to add and multiply two complex objects. In function `main()`, `H`, `I`, `J`, and `K` are four objects of the class `complex`. The objects `H` and `I` are initialized using the `setdata()` function. The statements `J=H+I` and `K=H*I` do the addition and multiplication of two objects by using the overloaded operator. The `show()` function displays the contents of object.

### 10.26 Write a program to declare matrix class and perform addition of matrix class objects.

```
#include<iostream.h>
#include<conio.h>
#include<iomanip.h>

class matrix
{
 public:
 int num[3][3];
};

matrix operator + (matrix a, matrix b);
void show (matrix r);
void main()
{
 clrscr();
 matrix x= { 3,2,5,
 1,2,4,
 4,7,8 };
 matrix y= { 4,7,4,
 2,1,4,
 3,2,1 };
 matrix z;
 z=x+y;
 show (x);
 show(y);
 show(z);
}

matrix operator + (matrix x, matrix y)
{
 matrix l;
 int i,j;
 for (i=0;i<3;i++)
 {
 for (j=0;j<3;j++)
 l.num[i][j]=x.num[i][j]+y.num[i][j];
 }
}
```

{coderipe}

```

 }
 return l;
}
void show (matrix g)
{
 int i,j;
 cout<<endl;
 for (i=0;i<3;i++)
 {
 cout<<endl;
 for (j=0;j<3;j++)
 cout<<setw(8)<<g.num[i][j];
 }
}

```

### OUTPUT

```

3 2 5
1 2 4
4 7 8
4 7 4
2 1 4
3 2 1
7 9 9
3 3 8
7 9 9

```

**Explanation:** In the above program, the class matrix has one array element num [3] [3]. The objects x, y, and z are of matrix type. The objects x and y are initialized. The addition (+) operator is overloaded and it performs addition of elements of two matrix objects and assigns the sum to the third object. The function show() display the contents of the objects. The statement z=x+y executes the overloaded operator.

### 10.27 Write a program to overload ++ operator in postfix style.

```

#include<iostream.h>
#include<conio.h>

class postfix
{
 private :
 int c;
 public :
 postfix() { c =0; }
 postfix (int g) { c =g; }
}

```

```

 postfix operator ++() { return postfix(++c); }
 postfix operator ++ (int) { return postfix(c++); }
 void show() { cout<<c; }
};
void main()
{
 clrscr();
 postfix A,B,C;
 B=A++;
 cout<<endl<<"B=";
 B.show();
 cout<<endl<<"A=";
 A.show();
 C=++A;
 cout<<endl<<"C=";
 C.show();
}

```

**OUTPUT****B=0****A=1****C=2**

**Explanation:** In the above program, the operator ++ is overloaded in suffix and postfix fashion. The difference in the two declaration statements of overloaded operator is the `int` in the parenthesis in second statement. It tells the compiler to make postfix version of operator (++). In function `main()`, objects A, B, and C are of postfix type. The objects A, B, and C are initialized to zero by the constructor.

The statement `B=A++` assigns value of A to B and then increments object A. The value displayed for B is zero and A is one. The statement `C=++A` increments object A first and then assigns the value to C. The value of A before increment is one. The value displayed for C is two. The function `show()` is used to display the contents of the object.

**10.28 Write a program to convert text to integer and integer to text.**

```

#include<iostream.h>
#include<stdlib.h>
#include<string.h>
#include<conio.h>

class text
{
 private :
 char txt[20];
 public :
 text() { txt[0]=NULL; }
}

```

```

 text (char *t) { strcpy (txt,t); }
 text (int d) { itoa(d,txt,10); }
 void show() { cout<<txt; }
 operator int();
};
text::operator int()
{
 int m,tt=0,g=1;
 m=strlen(txt)-1;
 while (m>=0)
 {
 tt=tt+(txt[m]-48)*g;
 m--;
 g*=10;
 }
 return (tt);
}
void main()
{
 clrscr();
 text t1=457;
 cout<<endl<<"t1=";
 t1.show();
 t1=111;
 cout<<endl<<"t1=";
 t1.show();
 text t2("325");
 int j=int(t2);
 cout<<endl<<"j ="<<j;
 text t3("254");
 j=t3;
 cout<<endl<<"j ="<<j;
}

```

## OUTPUT

```

t1=457
t1=111
j =325
j =254

```

**Explanation:** In the above program, constructor with one argument converts the integer to text (user-defined data type). The constructor is executed when an object of text class is created with one parameter. The `itoa()` function converts integer value to string. For conversion of text to

integer, overloaded case operator is used. This operator revives object of text class and converts it to the integer type value.

**10.29 Write a program to convert square to square root and vice versa.**

```
#include<iostream.h>
#include<conio.h>
#include<math.h>

class square
{
 int s;
public:
 get() { return s; }
 square (int j) { s=j; }
 void show() { cout<<"\n Square="<<s; }
};

class root
{
 float r;
public:
 root (int k) { r=k; }
 operator square ()
 {
 return r*r;
 }
 root (square k)
 {
 r=sqrt(k.get());
 }
 void show() { cout<<"\n Square Root="<<r; }
};

void main()
{
 clrscr();
 square s(81);
 root r(0);
 r=s; // root = square
 r.show();
 s.show();
 r=10; // invokes constructor
 s=r; // square = root
 r.show();
 s.show();
}
```



**OUTPUT**

**Square Root=9**  
**Square=81**  
**Square Root=10**  
**Square=100**

**Explanation:** In the above program two classes square and root are defined. The conversion routines are defined in class root. The statement `r=s;` uses one-argument constructor for conversion, and the statement `s=r;` invokes the operator function for conversion.

**SUMMARY**

- (1) Operator overloading is one of the most helpful concepts introduced by C++ language. Operator overloading provides the capability to redefine the language in which working of operator can be changed.
- (2) Overloaded operators are redefined within a C++ class using the keyword operator followed by an operator symbol. When an operator is overloaded, the produced symbol is called the operator function name.
- (3) Overloading an operator cannot change the basic idea of an operator. When an operator is overloaded, its properties like syntax, precedence, and associativity remain constant.
- (4) The keyword operator defines a new action or operation to the operator.
- (5) The operator ++, --, and - are unary operators. The unary operators ++ and -- can be used as prefix or suffix with the functions. These operators have only single operand.
- (6) Binary operators require two operands. Binary operators are overloaded by using member functions and friend functions.
- (7) The conversion routine may be single-argument constructor or an operator function. It depends on whether the object is a source or destination object.
- (8) Defining multiple conversion routines puts the compiler in an uncertain condition. The compiler fails to select appropriate conversion routines.
- (9) There are three possibilities of data conversion. They are as follows:  
 Conversion from basic data type to user-defined data type (class type) – the conversion from basic to class type is easily carried out. It is automatically done by the compiler with the help in-built routines or by applying type casting.  
 Conversion from class type to basic data type – the compiler does not have any knowledge about the user-defined data type built using classes. In this type of conversion, the programmer needs to explicitly tell the compiler how to perform conversion from class to basic type. These instructions are written in a member function. Such type of conversion is also known as overloading of type cast operators.  
 Conversion from one class type to another class type – when an object of one class is assigned to object of another class, it is necessary to give clear-cut instructions to the compiler about how to make conversion between these two user-defined data types. Using constructor or conversion this function can be performed.
- (10) The conversion routine may be single-argument constructor or an operator function. It depends on whether the object it's a source or destination object. Table 10.3 describes conversion type and place of routine to be defined.

## EXERCISES

### (A) Answer the following questions

- (1) What do you mean by operator overloading?
- (2) What is the use of keyword operator?
- (3) What are the rules for overloading operators?
- (4) What is the difference between operator overloading and function overloading?
- (5) What is the difference between overloading of binary operators and unary operators?
- (6) How are `friend` functions used to carry out overloading of operators? In which situation are they helpful?
- (7) Explain conversion of data from basic to class type. Explain the role of compiler.
- (8) Explain conversion from class to basic type.
- (9) Explain conversion from one class type to another class type.
- (10) What are source and destination objects?
- (11) List the operators that cannot be overloaded.
- (12) Describe the rules for operator overloading.
- (13) Write syntaxes for overloading extraction and insertion operator with `friend` function.

### (B) Answer the following by selecting the appropriate option

- (1) The keyword operator is used to overload an
  - (a) **operator**
  - (b) function
  - (c) class
  - (d) none of the above
- (2) Consider the equation  $Z=3*X$ . To overload the `*` operator one of the following function is used. Z and X are objects of the same class.
  - (a) **friend**
  - (b) virtual
  - (c) public
  - (d) none of the above
- (3) One of the following conversions is automatically carried by the compiler
  - (a) **conversion from basic data type to user-defined data type (class type)**
  - (b) conversion from class type to basic data type
  - (c) conversion from one class type to another class type
  - (d) none of the above
- (4) One of the following operators cannot be overloaded
  - (a) plus operator (+)
  - (b) **dot operator (.)**
  - (c) & ampersand operator
  - (d) `--` operator
- (5) In postfix overloading of operator (`++` or `--`), the last argument should have type
  - (a) float
  - (b) void
  - (c) **int**
  - (d) long
- (6) A, B, and C are objects of the same class. To execute the statement  $C=A+B$ , the operator must be overloaded.
  - (a) **+**
  - (b) =
  - (c) both (a) and (b)
  - (d) none of the above
- (7) The operator function returns value of
  - (a) basic type
  - (b) void type
  - (c) **class type**
  - (d) all types

### (C) Attempt the following programs

- (1) Write a program to overload `<` operator and display the smallest number out of two objects.
- (2) Write a program to overload `=` operator. Assign values of data members of one object to another object of the same type.

- (3) Write a program to overload == operator. Compare two objects using overloaded operator.
- (4) Write a program to carry conversion from class type to basic data type.
- (5) Write a program to carry conversion from one class data type to another class type.
- (6) Write a program to evaluate the equation  $A=B*3$ , where A and B are objects of the same class. Use friend function.
- (7) Write a program to pass reference of object to operator function and change the contents of object. Use single object as source and destination object.
- (8) Write a program to declare two classes rupees and dollar. Declare objects of both the classes. Convert rupees to dollar and vice versa. Perform conversion using user-defined conversion routines.

**(D) Find the bugs in the following programs**

(1)

```
class num
{ int x;
 public:
 num (int k) { x=k; }
 int operator + (num n)
 { num s(0);
 s.x=x-n.x;
 return x;
 }
};
void main()
{ num a(1), b(5),c(0);
 c=a+b; }
```

(2)

```
struct num
{ int x;
 num (int k) { x=k; }
 void operator ++ ()
 { ++x; }
};
void main()
{
 num b(2);
 b++;
 cout<<b.x;
}
```

# Inheritance

# 11

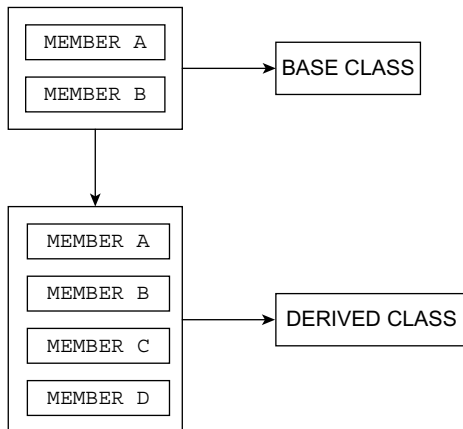
## CHAPTER

### CHAPTER OUTLINE

- 11.1 Introduction
- 11.2 Reusability
- 11.3 Access Specifiers and Simple Inheritance
- 11.4 Protected Data with Private Inheritance
- 11.5 Types of Inheritance
- 11.6 Single Inheritance
- 11.7 Multilevel Inheritance
- 11.8 Multiple Inheritance
- 11.9 Hierarchical Inheritance
- 11.10 Hybrid Inheritance
- 11.11 Multipath Inheritance
- 11.12 Virtual Base Classes
- 11.13 Constructors, Destructors, and Inheritance
- 11.14 Object as a Class Member
- 11.15 Abstract Classes
- 11.16 Qualifier Classes and Inheritance
- 11.17 Constructors in Derived Class
- 11.18 Pointers and Inheritance
- 11.19 Overloading Member Functions
- 11.20 Advantages of Inheritance
- 11.21 Disadvantages of Inheritance
- 11.22 More Programs

## 11.1 INTRODUCTION

Inheritance is one of the most useful and essential characteristics of object-oriented programming.



**Fig. 11.1** Inheritances

The existing classes are the main components of inheritance. The new classes are created from existing ones. The properties of the existing classes are simply extended to the new classes. The new classes created by using such a method are known as *derived classes*, and the existing classes are known as *base classes*, as shown in Figure 11.1. The relationship between the base and derived classes is known as kind of relationship. The programmer can define new member variables and functions in the derived class. The base class remains unchanged. The object of the derived class can access members of the base as well as derived classes. On the other hand, the object of the base class cannot access members of the derived classes. The base class does not know about their subclasses.

## 11.2 REUSABILITY

Reusability means the reuse of properties of the base class in the derived classes. Reusability permits the reuse of members of the previous class. We can add extra features in the existing class. This is possible by creating a new class from the existing class. The new class will have the features of both the old and the derived new class. In Figure 11.1, the base class is reused. Reusability is achieved using inheritance. Inheritance and reusability are not different from each other. The outcome of inheritance is reusability.

In this chapter, we will deal with implementation of the mechanism of inheritance and understand its features by illustrating several examples. One can find the inheritance property in several languages, but inheritance property in C++ is slightly different. C++ enables an inheritance of all the properties of the previous class, and there is a flexibility to add new members in the derived class, which are not available in the parent class.

The base class is also called *superclass*, *parent*, or *ancestor*, and the derived class is called *subclass*, *child*, or *descendent*. It is also possible to derive a class from a previously derived class. A class can be derived from more than one class.

### INHERITANCE

The procedure of creating a new class from one or more existing classes is termed *inheritance*.

## 11.3 ACCESS SPECIFIERS AND SIMPLE INHERITANCE

Before elaborating the inheritance mechanism, it is necessary to recall our memory for access rules studied in the Chapter 8 on *classes and objects*. We have studied the access specifiers *private* and *public* in detail and briefly studied the *protected* access specifiers.

The public members of a class can be accessed by an object directly outside the class. Directly means when objects access the data member without the member function of the class. The private members of the class can only be accessed by the public member function of the same class. The protected access specifier is the same as private. The only difference is that it allows its derived classes to access the protected members directly without the member functions. This difference is explained later with suitable examples.

A new class can be defined as per the syntax given below. The derived class is indicated by associating with the base class. A new class also has its own set of member variables and functions. The syntax given below creates the derived class.

```
class name of the derived class: access specifiers - name of the base
class
{

 _____ // member variables of new class (derived class)

}
```

The names of the derived and base classes are separated by a colon (:). The access specifiers may be private or public. The keyword private or public is specified followed by a colon. Generally, the access specifier is to be specified. In the absence of an access specifier, the default is private. The access specifiers decide whether the characteristics of the base class are derived privately or publicly. The derived class also has its own set of member variables and functions. The following are the possible syntaxes of declaration:

(1)

```
class B: public A
{
 // Members of class B
};
```

In the above syntax, class A is a base class, and class B is a derived class. Here, the class B is derived publicly.

(2)

```
class B: private A // private derivation
{
 // members of class B
};
```

(3)

```
class B: A // by default private derivation
{
 // members of class B
};
```

(4)

```
class B: protected A // same as private
{
 // members of class B
};
```

(5)

```
struct B: A // public derivation
{
 // members of class B
};
```

In the above syntaxes, the class B is derived privately from the base class A. If no access specifier is given, the default mode is private. The use of `protected` access specifier is the same as `private` access. If `struct` is used instead of `class`, the default derivation is public. It is important to note the following points:

- (a) When a public access specifier is used (example 1), the public members of the base class are public members of the derived class. Similarly, the protected members of the base class are protected members of the derived class.
- (b) When a private access specifier is used, the public and protected members of the base class are the private members of the derived class.

## Public Inheritance

The class can be derived either publicly or privately. No third type exists. When a class is derived publicly, all the public members of the base class can be accessed directly in the derived class. However, in private derivation, an object of the derived class has no permission to directly access even public members of the base class. In such a case, the public members of the base class can be accessed using public member functions of the derived class.

In case the base class has private member variables and a class derived publicly, the derived class can access the member variables of the base class using only member functions of the base class. The public derivation does not allow the derived class to access the private member variable of the class directly as is possible for public member variables. The following example illustrates public inheritance where base class members are declared public and private.

### 11.1 Write a program to derive a class publicly from base class. Declare the base class with its member under public section.

```
#include<iostream.h>
#include<constream.h>
// PUBLIC DERIVATION //
class A // BASE CLASS
{
 public:
```

```

 int x;
};
class B: public A // DERIVED CLASS
{
 public:
 int y;
};
int main()
{
 clrscr();
 B b; // DECLARATION OF OBJECT
 b.x=20;
 b.y=30;
 cout<<"\n member of A:"<<b.x;
 cout<<"\n Member of B:"<<b.y;
 return 0;
}

```

## OUTPUT

**Member of A : 20**

**Member of B : 30**

**Explanation:** In the above program, two classes are defined, and each contains one public member variable. The `class B` is derived publicly from the class `A`. Consider the following statement:

```
class B : public A
```

The above statement is used to derive the new class. The keyword `public` is used to derive the class publicly. The access specifier is followed by the name of the base class.

In function `main()`, `b` is an object of `class B`. The object `b` can access the members of `class A` as well as of `class B` through the following statements:

```

b.x=20; // Access to base class members
b.y=30; // Access to derived class members

```

Thus, the derived class holds the members of the base class, and its object has the permission to access the members of the base class.

## 11.2 Write a program to derive a class publicly from base class. Declare the base class member under private section.

```

#include<iostream.h>
#include<constream.h>
// PUBLIC DERIVATION //

class A // BASE CLASS

```



```
{
 private:
 int x;
 public:
 A() {x=20;}
 void showx()
{
 cout<<"\n x="<<x;
}
};
class B : public A // DERIVED CLASS
{
 public:
 int y;
 B() {y=30;}
 void show()
 {
 showx();
 cout<<"\n y="<<y;
 }
};
int main()
{
 clrscr();
 B b; // DECLARATION OF OBJECT
 b.show();
 return 0;
}
```

## OUTPUT

**x=20**  
**y=30**

**Explanation :** In the above program, the class A has one private member, default constructor, and member function `showx()`. The class B is derived from the class A publicly. The class B contains one public member variable `y`, default constructor, and member function `show()`.

In function `main()`, `b` is an object of the derived class B. Though the class B is derived publicly from the class A, the status of the members of the base class remains unchanged. The object `b` can access public members, but cannot access the private members directly. The private members of the base class can be accessed using the public member function of the base class.

The object `b` invokes the member function `show()` of the derived class. The function `show()` invokes the `showx()` function of the base class.

The object `b` can access the member function defined in both base and derived classes. The following statements are valid:

```
b.showx(); // Invokes member function of base class
b.show(); // Invokes member function of derived class
```

The constructor and member functions of the derived class cannot access the private members. Due to this, separate constructor and member functions are defined in both base and derived classes.

### Private Inheritance

The objects of the privately derived class cannot access the public members of the base class directly. Hence, the member functions are used to access the members.

#### 11.3 Write a program to derive a class privately. Declare the member of base class under public section.

```
// PRIVATE INHERITANCE //
#include<iostream.h>
#include<constream.h>

class A // BASE CLASS
{
 public:
 int x;
};

class B : private A // DERIVED CLASS
{
 public:
 int y;
 B()
 {x=20;
 y=40;
 }
 void show()
 {cout<<"\n x="<<x;
 cout<<"\n y="<<y;
 }
};

int main()
{
 clrscr();
 B b; // DECLARATION OF OBJECT
 b.show();
 return 0;
}
```

**OUTPUT****x=20****y=40**

**Explanation:** In the above program, the class B is derived privately from the class A. The member variable x is a public member of the base class. However, the object b of the derived class cannot directly access the variable x. The following statements are invalid:

```
b.x=30; // cannot access
```

The class B is derived privately. Hence, its access is restricted. The member function of the derived class can access the members of the base class. The function show() does the same.

**11.4 Write a program to derive a class privately.**

```
#include<iostream.h>
#include<constream.h>

class A // BASE CLASS
{
 int x;
 public:
 A()
 {x=20;}
 void showx()
 {
 cout<<"\n x="<<x;
 }
};

class B : private A // DERIVED CLASS
{
 public:
 int y;
 B()
 { y=40;
 }
 void show()
 {
 showx();
 cout<<"\n y="<<y;
 }
};

int main()
{
 clrscr();
```

```

 B b; // DECLARATION OF OBJECT
 b.show();
return 0;
}

```

## OUTPUT

```

x=20
y=40

```

**Explanation:** In the above program, the member variables of the base class and derivation methods are private. The derived class and its objects cannot access the private members of the base class directly. Hence, separate constructor and member functions are defined in both base and derived classes. The object `b` invokes the member function `show()` to display the members. The constructors are used to initialize the member variables.

From the last few programs, it is clear that handling the public data of a class is not a tough task for the programmer. However, while accessing the private data, difficulties are faced by the programmer, because they cannot be accessed without the member functions. Even the derived class has no permission to access the private data directly. Hence, the programmer needs to define separate constructor and member functions to access the private data of that particular base class. Due to this, the length of the program is increased.

## 11.4 PROTECTED DATA WITH PRIVATE INHERITANCE

The member function of the derived class cannot access the private member variables of the base class. The private members of the base class can be accessed using the public member function of the same class. This approach makes the program lengthy. To overcome the problems associated with private data, the creator of C++ introduced another access specifier `protected`. The `protected` is similar to `private`, but it allows the derived class to access the private members directly. Consider the following example:

### 11.5 Write a program to declare protected data in base class. Access data of base class declared under Protected section using member functions of derived class.

```

// PROTECTED DATA //
#include<iostream.h>
#include<constream.h>

class A // BASE CLASS
{
 protected: // protected declaration
 int x;
};
class B : private A // DERIVED CLASS
{

```

```

 int y;
public:
 B()
 {
 x=30;
 y=40;
 }
 void show()
 {
 cout<<"\n x="<<x;
 cout<<"\n y="<<y;
 }
};
int main()
{
 clrscr();
 B b; // DECLARATION OF OBJECT
 b.show();
 return 0;
}

```

## OUTPUT

```

x=30
y=40

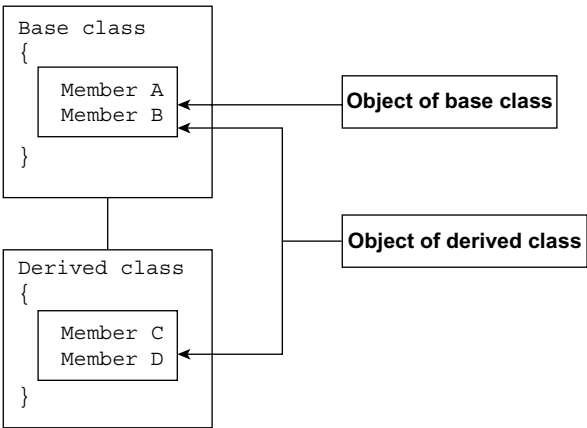
```

**Explanation:** In this program, the data member variable of class A is declared under the protected section. The class A contains neither default constructor nor any member function. The class b is derived from class A. The member function of the derived class B can access the protected member of the base class.

Thus, the protected mechanism reduces the program size. The derived class need not depend on the member function of the base class to access the data. The protected data safeguard the data from direct use and allow only the derived classes to access the data.

The protected access specifier should be used when it is known in advance that a particular class can be used as a base class. The data members of the classes that can be used as a base class should be declared as protected. In such classes, the programmer need not define functions. The member functions of the base class are rarely useful.

In the inheritance mechanism, the derived classes have more number of members as compared with the base class. The derived class contains the properties of a base class and a few of its own. The object of the derived class can access the members of both base and derived classes. However, the object of the base can access the members of only the base class and not any of the derived class, as shown in Figure 11.2. Hence, the programmer always uses objects of the derived classes and performs operations. The member function of the base class cannot access the data of the derived class. Obviously, the programmer uses the object of the derived classes and possibly avoids defining the member function in the base class.

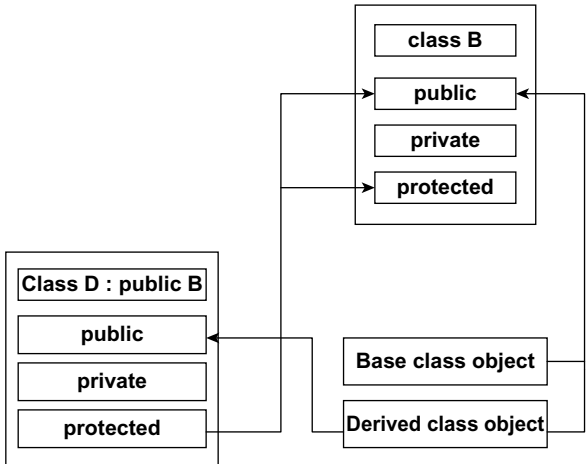


**Fig. 11.2** Difference between base and derived class objects

We have learned about the access specifiers in detail. Now before moving on to a new topic, let us revise a few points related to `private`, `public`, and `protected` keywords. Table 11.1 gives a description of access specifiers followed by an explanation. Figure 11.3 gives a pictorial representation of access control.

**Table 11.1** Access specifiers with their scope

| Sr.No. | Base class access mode | Derived class access mode |                   |                      |
|--------|------------------------|---------------------------|-------------------|----------------------|
|        |                        | Private derivation        | Public derivation | Protected derivation |
| A      | public                 | private                   | public            | protected            |
| B      | private                | Not inherited             | Not inherited     | Not inherited        |
| C      | protected              | private                   | protected         | protected            |



**Fig. 11.3** Access scope of class members

- (a) Accessible to the member function of the same class, derived class, and using objects. When a class is derived by private derivation, public members of the base class are private in the derived class.
- (b) Accessible to the member function inside its own class but not in the derived class. The derived class cannot access the private members of the base class directly. The private members of the base class can be accessed only using the public member function of the same class.
- (c) Accessible to the member function of the base and derived classes. When a class is derived privately, the protected members of the base class become private, and when they are derived publicly, the protected members remain protected in the derived class.

The syntax of public, private, and protected access specifiers is as follows:

Syntax:  
Public: <declarations>  
Private: <declarations>  
Protected: <declarations>

If the member variables of a class are protected, the access specifier should be private. A protected member can be considered a hybrid of public and private members. A protected member is public for its derived class and private for other class members. Member functions and friends can use the member classes derived from the declared class. Only objects of the derived class do this task. It is possible to override the default struct access with private or protected. However, it is not possible to override the default union access. Figure 11.5 shows a pictorial representation of the access control in classes.

- (1) All private members of the class are accessible to public members of the same class. They cannot be inherited.
- (2) The derived class can access the private members of the base class using the member function of the base class.
- (3) All the protected members of the class are available to its derived classes and can be accessed without the use of the member function of the base class. In other words, we can say that all protected members act as public for the derived class.
- (4) If any class is prepared for deriving classes, it is advisable to declare all members of the base class as protected, so that derived classes can access the members directly.
- (5) All the public members of the class are accessible to its derived class. There is no restriction for accessing elements.
- (6) The access specifier required while deriving class is either private or public. If not specified, private is default for classes and public is default for structures.
- (7) Constructors and destructors are declared in the public section of the class. If declared in the private section, the object declared will not be initialized and the compiler will flag an error.
- (8) The private, public, and protected sections (visibility sections) can be defined several times in any sequence. In Figure 11.4, you can observe that the public section is declared twice, the private section is defined at last, and the protected section is declared in between two public sections. The sections can be declared in any sequence for any number of times.

```

class <class name>
{
 public: // public section
 <declarations>
 protected: // protected section
 <declarations>
 public: // public section
 <declarations>
 private: // private section.
 <declarations>
};

```

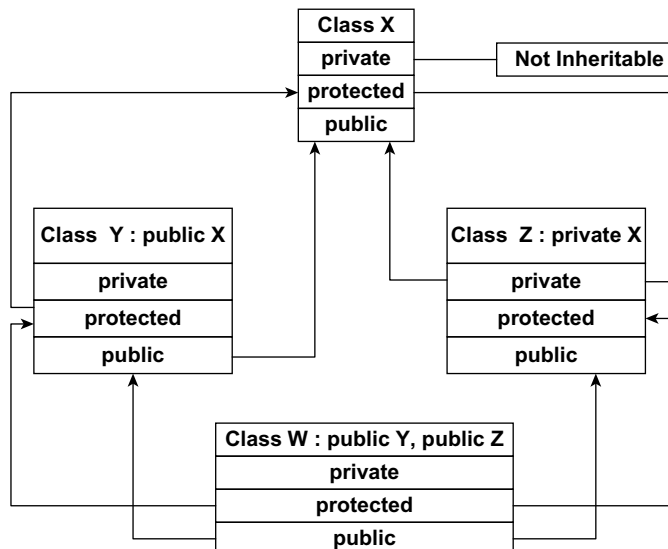
**Fig. 11.4** Visibility sections

### Member functions scope

The following type of functions can have access to the protected and private members of the class. Table 11.2 describes these functions. Figure 11.5 shows access scope.

**Table 11.2** Access controls of functions

| Sr. No. | Type of Functions     | Access modes |           |        |
|---------|-----------------------|--------------|-----------|--------|
|         |                       | Private      | Protected | Public |
| A       | Class member function | ✓            | ✓         | ✓      |
| B       | Derived class member  | ✗            | ✓         | ✓      |
| C       | Friend function       | ✓            | ✓         | ✓      |
| D       | Friend class member   | ✓            | ✓         | ✓      |

**Fig. 11.5** Access scope in classes



- (a) The class member function can access the private, protected, and public members.
- (b) The derived class member function cannot access the private members of the base class directly. However, the private members can be accessed using the member functions of the base class.
- (c) The friend function of a friend class can access the private and protected members.
- (d) The member function of a friend class can access the private and protected member of the class.

## 11.5 TYPES OF INHERITANCE

So far, we have learned examples of simple inheritance that use one base class and one derived class. The process of inheritance can be either simple or complex.

This depends on the following points:

- (1) **Number of base classes:** The program can use one or more base classes to derive a single class.
- (2) **Nested derivation:** The derived class can be used as a base class, and a new class can be derived from it. This can be possible at any extent.

Based on the above points, inheritance is classified as follows:

- (a) Single Inheritance
- (b) Multiple Inheritance
- (c) Hierarchical Inheritance
- (d) Multilevel Inheritance
- (e) Hybrid Inheritance
- (f) Multi-path Inheritance

The different types of inheritance are described in Figure 11.6. The base classes are at the top, and the derived classes are at the bottom. The arrow pointing from top to bottom indicates that the properties of the base class are inherited by the derived class and vice versa is not applicable.

### (a) Single Inheritance

This occurs when only one base class is used for the derivation of a derived class. Further, derived class is not used as a base class, such a type of inheritance that has one base and derived class is known as `single inheritance`. Figure 11.6 (a) indicates single inheritance.

### (b) Multiple Inheritance

When two or more base classes are used for the derivation of a class, it is called `multiple inheritance`. Figure 11.6 (b) indicates multiple inheritance.

### (c) Hierarchical Inheritance

When a single base class is used for the derivation of two or more classes, it is known as `hierarchical inheritance`. Figure 11.6 (c) indicates hierarchical inheritance.

### (d) Multilevel Inheritance

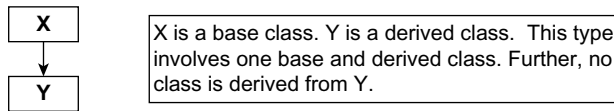
When a class is derived from another derived class, that is, the derived class acts as a base class, such a type of inheritance is known as `multilevel inheritance`. Figure 11.6 (d) indicates multilevel inheritance.

### (e) Hybrid Inheritance

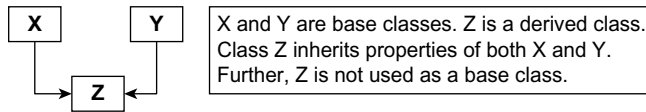
A combination of one or more types of inheritance is known as `hybrid inheritance`. Figure 11.6 (e) indicates hybrid inheritance.

## (f) Multipath Inheritance

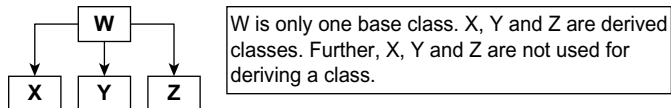
When a class is derived from two or more classes, those are derived from the same base class. Such a type of inheritance is known as multipath inheritance. Figure 11.6 (f) indicates multipath inheritance.



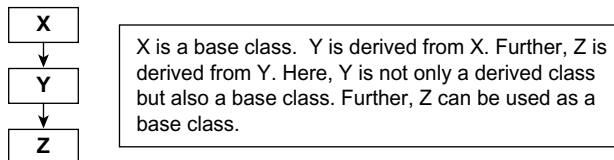
## (a) Single Inheritance



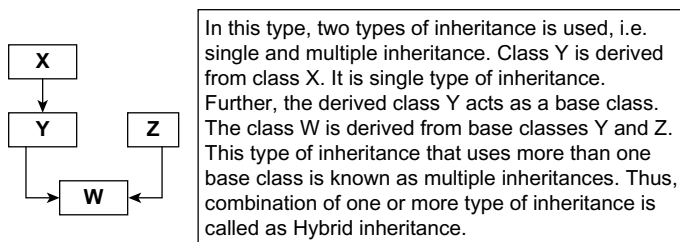
## (b) Multiple Inheritance



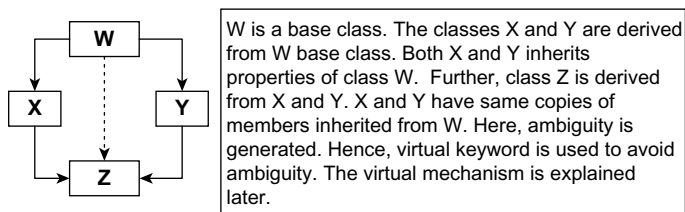
## (c) Hierarchical Inheritance



## (d) Multilevel Inheritance



## (e) Hybrid Inheritance



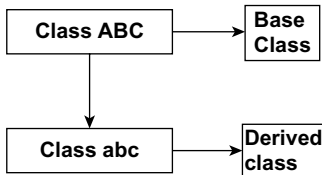
## (f) Multipath Inheritance

**Fig. 11.6** Types of inheritance

## 11.6 SINGLE INHERITANCE

When only one class is derived from a single base class, such a derivation of a class is known as *single inheritance*; further, the derived class is not used as a base class. This type of inheritance uses one base and one derived class.

The new class is termed *derived class*, and the old class is called *base class*, as shown in Figure 11.7. A derived class inherits data member variables and functions of the base class. However, constructors and destructors of the base class are not inherited in the derived class.



**Fig. 11.7** Single inheritance

The newly created class receives entire characteristics from its base class. In single inheritance, there are only one base class and one derived class. The single inheritance is not as complicated as compared with other types of inheritance. Consider the following example:

In the above diagram, class ABC is a base class, and class abc is a derived class. The arrow shows that class abc is derived from class ABC. The program given below illustrates the single inheritance.

### 11.6 Write a program to show single inheritance between two classes.

```

#include<iostream.h>
#include<constream.h>

class ABC
{
 protected:
 char name[15];
 int age;
};

class abc : public ABC // public derivation
{
 float height;
 float weight;
 public:
 void getdata()
 {
 cout<<"\n Enter Name and Age:";
 cin>>name>>age;
 cout<<"\n Enter Height and Weight:";
 cin>>height >>weight;
 }
 void show()
 {

```



```

 cout<<"\n Name:"<<name <<"\n Age:"<<age<<" Years";
 cout<<"\n Height:"<<height <<" Feets"<<"\n Weight:"<<weight
 <<"Kg.";
 }
};
int main()
{
 clrscr();
 abc x;
 x.getdata(); // Reads data through keyboard.
 x.show(); // Displays data on the screen.
 return 0;
}

```

## OUTPUT

**Enter Name and Age : Santosh 24**

**Enter Height and Weight : 4.5 50**

**Name : Santosh**

**Age : 24 Years**

**Height : 4.5 Feets**

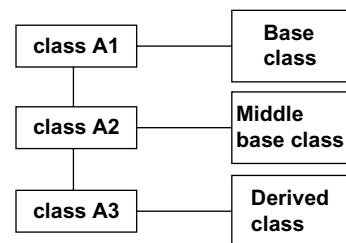
**Weight : 50 Kg.**

**Explanation:** In the above program, the two classes ABC and abc are declared. The class ABC has two protected data members, name and age. The class abc has two float data members, height and weight with a two-member function getdata() and show(). The class abc is derived from class ABC. The statement class abc: public ABC defines the derived class abc. In function main(), x is an object of the derived class abc. The object x invokes the member function getdata() and show(). This function reads and displays data, respectively.

## 11.7 MULTILEVEL INHERITANCE

The procedure of deriving a class from a derived class is called multilevel inheritance.

In the above figure, class A3 is derived from class A2. The class A2 is derived from class A1. The class A3 is a derived class. The class A2 is a derived class as well as a base class for class A3. The class A2 is called an intermediate base class. The class A1 is a base class of classes A2 and A3. The series of classes A1, A2, and A3 is called an inheritance pathway, as shown in Figure 11.8.



**Fig. 11.8** Multilevel inheritance

**11.7 Write a program to create multilevel inheritance. Create classes A1, A2, and A3.**

```
// Multilevel inheritance //
#include<iostream.h>
#include<constream.h>

class A1 // Base class
{
 protected:
 char name[15];
 int age;
};

class A2 : public A1 // Derivation first level
{
 protected:
 float height;
 float weight;
};

class A3 : public A2 // Derivation second level
{
 protected:
 char sex;
 public:
 void get() // Reads data
 {
 cout<<"Name:"; cin>>name;
 cout<<"Age:"; cin>>age;
 cout<<"Sex:"; cin>>sex;
 cout<<"Height:"; cin>>height;
 cout<<"Weight:"; cin>>weight;
 }
 void show() // Displays data
 {
 cout<<"\nName:" <<name;
 cout<<"\nAge:" <<age <<" Years";
 cout<<"\nSex:" <<sex;
 cout<<"\nHeight:" <<height <<" Feets";
 cout<<"\nWeight:" <<weight <<" Kg.";
 }
};

int main()
{
 clrscr();
 A3 x; // Object Declaration
 x.get(); // Reads data
 x.show(); // Displays data
```



```
 return 0;
}
```

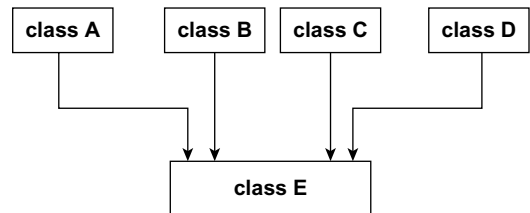
## OUTPUT

```
Name : Balaji
Age : 26
Sex : M
Height : 4
Weight : 49.5
Name : Balaji
Age : 26 Years
Sex : M
Height : 4 Feets
Weight : 49.5 Kg.
```

**Explanation:** In the above program, the classes A1, A2, and A3 are declared. The member variables of all these classes are protected. The class A2 is derived from class A1. The class A3 is derived from class A2. Thus, the class A2 acts as a derived class as well as a base class. The function get() reads data through the keyboard, and the function show() displays data on the screen. Both the functions are invoked using object x of class A3.

## 11.8 MULTIPLE INHERITANCE

Multiple inheritance is a latest addition to the C++ language. When a class is derived from more than one class, this type of inheritance is called multiple inheritance. A class can be derived by inheriting the properties of more than one class. Properties of various pre-defined classes are transferred to a single derived class. Figure 11.9 shows multiple inheritance.



**Fig. 11.9** Multiple inheritance

### 11.8 Write a program to derive a class from multiple base classes.

```
// Multiple Inheritance //
#include<iostream.h>
#include<constream.h>
class A {protected: int a;}; // class A declaration
class B {protected: int b;}; // class B declaration
class C {protected: int c;}; // class C declaration
class D {protected: int d;}; // class D declaration
// class E : public A, public B, public C, public D
class E : public A,B,C,D // Multiple derivation
```



```

{
 int e;
 public:
 void getdata()
 {
 cout<<"\n Enter values of a,b,c & d & e:";
 cin>>a>>b>>c>>d>>e;
 }
 void showdata()
 {
 cout<<"\n a="<<a <<" b="<<b <<" c = "<<c <<" d = "<<d <<" e="<<e;
 }
};
int main()
{
 clrscr();
 E x;
 x.getdata(); // Reads data
 x.showdata(); // Displays data
 return 0;
}

```

## OUTPUT

**Enter values of a,b,c & d & e : 1 2 4 8 16**  
**a=1 b = 2 c = 4 d= 8 z= 16**

**Explanation:** In the above program, classes A, B, C, D, and E are declared with a one-integer member variable each. The class E has a two-member function `getdata()` and `showdata()`, respectively. The `getdata()` is used to read integers through the keyboard, and `showdata()` is used to display the contents on the screen. The class E is derived from the classes A, B, C, and D. The classes A, B, C, and D act as a base class. The derivation is carried out with the statement `class E: public A, B, C, and D`. The class members of A are publicly derived, and the members of other classes are privately derived. To derive all class members publicly, they would be as `class Z: public A, public B, public C, and public D`. Remember, the meaning of both the statements is not same. Most programmers refer to the second format, as it reduces the code. The output of the program is as given above.

## 11.9 HIERARCHICAL INHERITANCE

We learned that in inheritance, one class could be inherited from one or more classes. In addition, new members are added to the derived class. The inheritance also supports the hierarchical arrangement of a program. Several programs require the hierarchical arrangement of classes, in which derived classes share the properties of the base class. A hierarchical unit exhibits top-

down style through splitting a compound class into several simple subclasses. Figure 1.16 in Chapter 1 is a perfect example of the hierarchy of classes. The program based on it is illustrated as follows:

**11.9 Write a program to show hierarchical inheritance.**

```
#include<constream.h>
#include<iostream.h>

class red
{
 public:
 red() {cout<<"Red";};
};

class yellow
{
 public:
 yellow() {cout<<"Yellow";}
};

class blue
{
 public:
 blue() {cout<<"Blue";}
};

class orange : public red, public yellow
{
 public:
 orange() {cout<<"=Orange";}
};

class green : public blue, public yellow
{
 public:
 green() {cout<<"=Green";}
};

class violet : public red, public blue
{
 public:
 violet() {cout<<"=Violet";}
};

class reddishbrown : public orange, public violet
{
 public:
 reddishbrown() {cout<<"=Reddishbrown";}
};

class yellowishbrown : public green, public orange
```



```

{
 public:
 yellowishbrown() {cout<<"=Yellowishbrown";}
};
class bluishbrown : public violet, public green
{
 public:
 bluishbrown() {cout<<"=Bluishbrown";}
};
int main()
{
 clrscr();
 reddishbrown r;
 endl(cout);
 bluishbrown b;
 endl(cout);
 yellowishbrown y;
 endl(cout);
 return 0;
}

```

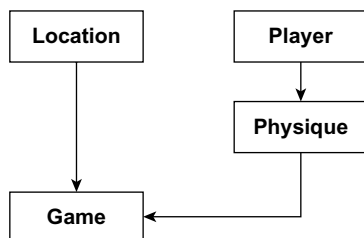
## OUTPUT

**Red Yellow = Orange Red Blue = Violet = Reddishbrown**  
**Red Blue = Violet Blue Yellow = Green = Bluishbrown**  
**Blue Yellow = Green Red Yellow = Orange = Yellowishbrown**

**Explanation:** For each color, a separate class with constructors is declared. The classes red, blue, and yellow are base classes. The class orange is derived from red and yellow. The class green is derived from blue and yellow. The class violet is derived from red and blue.

The class reddishbrown is derived from orange and violet; the class yellowishbrown is derived from green and orange; and, lastly, the class bluishbrown is derived from violet and green.

In function `main()`, objects of the classes reddishbrown, yellowishbrown, and bluishbrown are declared. When the objects are declared, constructors are executed from the base to derived classes. When executed, the constructor displays the class name (color name). The output shows the names of different colors and resulting colors after their combination.



**Fig. 11.10** Hybrid inheritance

## 11.10 HYBRID INHERITANCE

A combination of one or more types of inheritance is known as hybrid inheritance. Sometimes, it is essential to derive a class using more types of inheritance. Figure 11.10 shows hybrid inheritance. In the below given diagram, the class game is derived from two base classes, that is, location and physique. The class physique is also derived from the class player.

**11.10 Write a program to create a derived class from multiple base classes.**

```
// Hybrid Inheritance //
#include<iostream.h>
#include<constream.h>

class PLAYER
{
 protected:
 char name[15];
 char gender;
 int age;
};

class PHYSIQUE : public PLAYER
{
 protected:
 float height;
 float weight;
};

class LOCATION
{
 protected:
 char city[10];
 char pin[7];
};

class GAME : public PHYSIQUE, LOCATION
{
 protected:
 char game[15];
 public:
 void getdata()
 {
 cout<<"Enter Following Information\n";
 cout<<"Name:"; cin>>name;
 cout<<"Gender:"; cin>>gender;
 cout<<"Age:"; cin>>age;
 cout<<"Height:"; cin>>height;
 cout<<"Weight:"; cin>>weight;
 cout<<"City:"; cin>>city;
 cout<<"Pincode:"; cin>>pin;
 cout<<"Game:"; cin>>game;
 }
 void show()
 {
 cout<<"\n Entered Information";
 cout<<"\nName:"; cout<<name;
```

```
 cout<<"\nGender:"; cout<<gender;
 cout<<"\nAge:"; cout<<age;
 cout<<"\nHeight:"; cout<<height;
 cout<<"\nWeight:"; cout<<weight;
 cout<<"\nCity:"; cout<<city;
 cout<<"\nPincode:"; cout<<pin;
 cout<<"\nGame:"; cout<<game;
 }
};
int main()
{
 clrscr();
 GAME G;
 G.getdata();
 G.show();
 return 0;
}
```

## OUTPUT

### Enter Following Information

**Name : Mahesh**

**Gender : M**

**Age : 25**

**Height : 4.9**

**Weight : 55**

**City : Nanded**

**Pincode : 431603**

**Game : Cricket**

### Entered Information

**Name : Mahesh**

**Gender : M**

**Age : 25**

**Height : 4.9**

**Weight : 55**

**City : Nanded**

**Pincode : 431603**

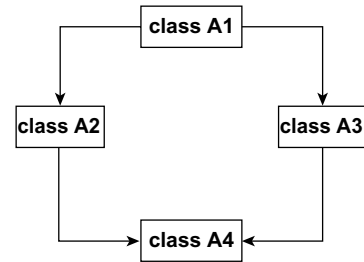
**Game : Cricket**

**Explanation:** In the above program, `player`, `physique`, `location`, and `game` classes are defined. All the data members of these four classes are protected. The class `physique` is derived from the base class `player`. The class `game` is derived from `physique` and `location`; that is, the derived class `game` has two base classes `physique` and `location`. The `location` is a separate class and is not derived from any other class. The `getdata()` and

`show()` are functions of class `game` and reads and display data, respectively. In `main()`, the object `G` of class `game` calls these functions one by one to read and write the data. The output of the program is shown at the end of the above program.

### 11.11 MULTIPATH INHERITANCE

When a class is derived from two or more classes, those are derived from the same base class. Such a type of inheritance is known as multipath inheritance. The multipath inheritance also consists of many types of inheritance, such as multiple, multilevel, and hierarchical, as shown in Figure 11.11.



**Fig. 11.11** Ambiguity in classes

```

Consider the following example:
class A1
{
 protected:
 int a1;
};
class A2 : public A1
{
 protected:
 int a2;
};
class A3: public A1
{
 protected: int a3;
};

class A4: public A2,A3
{
 int a4;
};

```

In the above example, classes `A2` and `A3` are derived from class `A1`; that is, their base class is similar to class `A1` (hierarchical inheritance). Both classes `A2` and `A3` can access the variable `a1` of class `A1`. The class `A4` is derived from classes `A2` and `A3` by multiple inheritance. If we try to access the variable `a1` of class `A1`, the compiler shows error messages as given below.

```

(a) Error PRG58.CPP 30: Member is ambiguous: 'A1::a1' and 'A1::a1'
(b) Error PRG58.CPP 37: Member is ambiguous: 'A1::a1' and 'A1::a1'

```

In the above example, we can observe all types of inheritance, that is, multiple, multilevel, and hierarchical. The derived class `A4` has two sets of data members of class `A1` through the middle base classes `A2` and `A3`. The class `A1` is inherited twice.

## 11.12 VIRTUAL BASE CLASSES

To overcome the ambiguity occurring due to multipath inheritance, the C++ provides the keyword `virtual`. The keyword `virtual` declares the specified classes virtual. The example given below illustrates the virtual classes:

```
class A1
{
protected:
int a1;
};
class A2 : virtual public A1 // virtual class declaration
{
protected:
int a2;
};
class A3: virtual public A1 // virtual class declaration
{
protected:
int a3;
};
class A4: public A2,A3
{
int a4;
};
```

When classes are declared as virtual, the compiler takes essential caution to avoid the duplication of member variables. Thus, we make a class virtual if it is a base class that has been used by more than one derived class as their base class.

Consider another example that will give you a clear idea of virtual classes as well as the necessary care to be taken while declaring virtual classes.

- (a) A base class cannot be specified more than once in a derived class.

```
class A
{-----
-----};
class A1: A, A {...}; // Illegal Declaration
```

Here, class A is specified twice. Hence, it is an illegal declaration.

- (b) A base class can be indirectly passed to the derived class more than once.

```
class L: public A {...}
class K: public A {...}
class J: public L, public K {...} // Legal declaration
```

In the above case (b) , each object of class J will contain two sub-objects of class A through classes L and K.

- (c) The case (b) causes problems. To avoid duplication, we can add the keyword `virtual` to a base class specifier.

```
For example,
class L: virtual public A {...}
class K: virtual public A {...}
class J: public K, public J {...}
```

The keyword `virtual` always appears before the class name. A is now a virtual base class, and class J has only one sub-object of class A.

### 11.11 Write a program to declare virtual base classes. Derive a class using two virtual classes.

```
// VIRTUAL BASE CLASSES //
#include<iostream.h>
#include<conio.h>
class A1
{
 protected:
 int a1;
};
class A2 : public virtual A1 // virtual declaration
{
 protected:
 int a2;
};
class A3 : public virtual A1 // virtual declaration
{
 protected:
 int a3;
};
class A4 : public A2,A3 // virtual declaration
{
 int a4;
 public:
 void get()
 {
 cout<<"Enter values for a1, a2,a3 and a4:";
 cin>>a1>>a2>>a3>>a4;
 }
 void put()
```

```

 {
 cout<<"a1="<<a1 <<"a2="<<a2 <<"a3="<<a3 <<"a4="<<a4;
 }
};
int main()
{
 clrscr();
 A4 a;
 a.get(); // Reads data
 a.put(); // Displays data
 return 0;
}

```

### OUTPUT

**Enter values for a1, a2,a3 and a4 : 5 8 7 3**  
**a1= 5 a2 = 8 a3 = 7 a4 = 3**

**Explanation:** In the above program, the classes A1, A2, A3, and A4 are declared, and each contains one protected member variable. The class A4 has two member functions get () and put (). The get () function reads integers through the keyboard. The put () function displays the contents of the member variables on the screen. The classes A2 and A3 are derived from class A1. While deriving these two classes, the class A1 is declared as virtual as per the following statements:

```

class A2: public virtual A1 // derivation of class A2
class A3: public virtual A1 // derivation of class A3
class A4: public A2,A3 // derivation of class A4

```

The class A4 is derived from two classes A2 and A3 as per the statement (c). In function main(), the object a of class A4 invokes the member function get () and put (). The output of the program is as shown above.

## 11.13 CONSTRUCTORS, DESTRUCTORS, AND INHERITANCE

The constructors are used to initialize member variables of the object, and the destructor is used to destroy the object. The compiler automatically invokes constructors and destructors. The derived class does not require a constructor, if the base class contains a zero-argument constructor. In case the base class has a parameterized constructor, then it is essential for the derived class to have a constructor. The derived class constructor passes arguments to the base class constructor. In inheritance, normally derived classes are used to declare objects. Hence, it is necessary to define constructors in the derived class. When an object of a derived class is declared, the constructors of the base and derived classes are executed.

### 11.13.1 Constructors and destructors in base and derived classes

In inheritance, destructors are executed in reverse order of constructor execution. The destructors are executed when an object goes out of scope. To know the execution of constructors and destructors, let us study the following program:

#### 11.12 Write a program to show sequence of execution of constructor and destructor in multiple inheritance.

```
#include<iostream.h>
#include<constream.h>

class A // base class
{
 public:
 A()
 {
 cout<<"\n Zero-argument constructor of base class A";}
 ~A()
 {
 cout<<"\n Destructor of the class A";}
};

class B // base class
{
 public:
 B()
 {
 cout<<"\n Zero-argument constructor of base class B";}
 ~B() {cout<<"\n Destructor of the class B" ;}
};

class C : public A, public B // Derivation of class
{
 public:
 C()
 {
 cout<<"\n Zero-argument constructor of derived class C";}
 ~C() {cout<<"\n Destructor of the class C";}
};

int main()
{
 clrscr();
 C objc; // Object declaration
 return 0;
}
```



**OUTPUT****Zero-argument constructor of base class A****Zero-argument constructor of base class B****Zero-argument constructor of derived class C****Destructor of the class C****Destructor of the class B****Destructor of the class A**

**Explanation:** In the above program, classes A and B are two classes. The class C is derived from classes A and B. The constructors of the base classes are executed first followed by the derived class. The destructor of the derived class is executed first followed by the base class.

**11.13 Write a program to use constructor and destructor in all classes (base and derived class). Read and display the data.**

```
#include<iostream.h>
#include<conio.h>

class A1
{
 protected:
 char name[15];
 int age;
 A1()
 {
 cout<<"Name:"; cin>>name;
 cout<<"Age:"; cin>>age;
 }
 ~A1()
 {
 cout<<"\nName:" <<name;
 cout<<"\nAge:" <<age;
 }
};

class A2 : public A1
{
 protected:
 float height;
 float weight;
 A2()
 {
 cout<<"Height:"; cin>>height;
 cout<<"Weight:"; cin>>weight;
 }
}
```

**{coderipe}**

```
 ~ A2 ()
 {
 cout<<"\nHeight:"<<height;
 cout<<"\nWeight:"<<weight;
 }
};
class A3 : public A2
{
 char sex;
public:
 A3 ()
 {
 cout<<"Sex:"; cin>>sex;
 }
 ~ A3 ()
 {
 cout<<"\nSex:" <<sex;
 }
};
int main()
{
 clrscr();
 A3 x;
 return 0;
}
```

## OUTPUT

```
Name : Ajay
Age : 20
Height : 4.5
Weight : 40
Sex : M
Sex : M
Height : 4.5
Weight : 40
Name : Ajay
Age : 20
```

**Explanation:** In the above program, classes A1, A2, and A3 have their own constructors and destructors. Here, the class A1 is the base class of A2, and A2 is the base class of A3 or, in other words, A3 is a derived class of A2 and A2 is a derived class of A1.

**Execution of constructors:** The execution of constructors takes place from the base class to the derived class. Thus, the sequence of the execution of constructors is A1 (), A2 (), and A3 ().

**Execution of destructors:** The execution of destructors is in opposite order as compared with constructors, that is, from the derived class to the base class. Thus, the sequence of the execution of destructors is `A3()`, `A2()`, and `A1()`. We learnt in Chapter 9 that the object created last destroyed first. The same rule is applicable in the execution of constructors and destructors in inheritance.

**11.14 Write a program to create derived class from base classes. Use constructor and destructors.**

```
#include<iostream.h>
#include<conio.h>

class in_t
{
 protected:
 int i;
 public:
 in_t()
 {
 cout<<"\n Constructor in_t()";
 i=1;
 }
 ~in_t()
 {
 cout<<"\n Destructor in_t()";
 }
};

class floa_t
{
 protected:
 float f;
 public:
 floa_t()
 {
 cout<<"\n Constructor floa_t()";
 f=1.5;
 }
 ~floa_t()
 {
 cout<<"\n Destructor floa_t()";
 }
};

class cha_r : public in_t, floa_t
{
 protected:
```

```

 char c;
public:
 void show()
 {
 cout<<"\n\ti="<<i;
 cout<<"\n\tf="<<f;
 cout<<"\n\tc="<<c;
 }
 cha_r()
 {
 cout<<"\n Constructor cha_r()";
 c='A';
 }
 ~cha_r()
 {
 cout<<"\n Destructor cha_r()";
 }
};
main()
{
 clrscr();
 cha_r a;
 a.show();
 return 0;
}

```

**OUTPUT****Constructor in\_t()****Constructor floa\_t()****Constructor cha\_r()****i= 1****f= 1.5****c= A****Destructor cha\_r()****Destructor floa\_t()****Destructor in\_t()**

**Explanation:** In the above program, three classes `in_t`, `floa_t`, and `cha_r` are defined, and each contains one protected variable of type `int`, `float`, and `char`, respectively. Each class contains a constructor and a destructor. In function `main()`, `a` is an object of class derived class `chr_r`. The member function `show()` is invoked by the object `a` to display the contents of member variables of all the classes. The constructors are executed from the base to the derived class, and destructors are loaded from the derived to the base class. The following program explains the execution of constructors and destructors in classes in greater detail:

### 11.13.2 Base and derived classes without constructors

In the absence of constructors in base and derived classes, the implicit constructors and destructors are executed when objects are declared. Consider the following program:

**11.15 Write a program to derive a class. The base and derive classes should be without constructors.**

```
#include<iostream.h>
#include<conio.h>

class I
{
 public:
 int x;
};

class II : public I
{
 int y;
 public:
 void set (int j, int k)
 {
 x=j;
 y=k;
 }
 void show()
 {
 cout<<"x="<<x <<"y="<<y;
 }
};

int main()
{
 clrscr();
 II i;
 i.set(4,5);
 i.show();
 return 0;
}
```

#### OUTPUT

**X = 4 y = 5**

**Explanation:** In the above program, class I is a base class, and class II is derived from I. Both classes are without constructors. The class II has used member functions set () and show () to assign and display data.

### 11.13.3 Base class with constructors and derived class without constructors

**11.16** Write a program to declare a base class with constructor and a derived class without constructor.

```
#include<iostream.h>
#include<constream.h>

class I
{
 public:
 int x;
 I()
 {
 cout<<"In base class constructor";
 x=10;
 }
};

class II : public I
{
 int y;
 public:
};

int main()
{
 clrscr();
 II i;
 return 0;
}
```

#### OUTPUT

##### In base class constructor

**Explanation:** The base class `I` has a constructor, and the derived class `II` does not have a constructor. The `i` is an object of the derived class `II`. When `i` is declared, the constructor of the base class is executed. The derived class is without a constructor; hence, only the base class constructor is executed.

### 11.13.4 Base class without constructors and derived class with constructors

**11.17** Write a program to declare base class without constructor and derived class without constructor.

```
#include<iostream.h>
#include<constream.h>
```

```

class I
{
};
class II : public I
{
 public:
 II()
 {
 cout<<"\nIn derived class constructor\n";
 }
};
int main()
{
 clrscr();
 II i; // Object declaration
 return 0;
}

```

## OUTPUT

### In derived class constructor

**Explanation:** In the above program the base class `I` has no constructor. The derived class `II` has a constructor. The constructor is executed when the object of class `II` is declared.

### 11.13.5 Base and derived classes with constructors

**11.18 Write a program to declare both base and derived classes with constructor.**

```

#include<iostream.h>
#include<conio.h>

class I
{
 public:
 I()
 {
 cout<<"\n In base class constructor";
 };
 class II : public I
 {
 public:
 II() {cout<<"\n In derived class constructor\n";}
 };
};

```

```
int main()
{
 clrscr();
 II i;
 return 0;
}
```

## OUTPUT

**In base class constructor**

**In derived class constructor**

**Explanation:** In the above program, both the base and derived classes contain constructors. When the object of the derived class type is declared and constructors of both the classes are executed, the constructor of the base class is executed first followed by the constructor of the derived class.

### 11.13.6 Base class with various constructors and derived class with one constructor

**11.19 Write a program to declare multiple constructors in base class and single constructor in derived class.**

```
#include<iostream.h>
#include<conio.h>

class I
{
 public:
 int x;
 I() {cout<<"\nZero argument base class constructor";}
 I (int k)
 {
 cout<<"\nOne argument base class constructor";
 }
};

class II : public I
{
 int y;
 public:
 II (int j)
 {
 cout<<"\nOne argument derived class constructor";
 y=j;
 }
};
```



```
int main()
{
 clrscr();
 II i(2);
 return 0;
}
```

## OUTPUT

**Zero argument base class constructor**

**One argument derived class constructor**

**Explanation:** In the above program, the class `I` has a zero- and a one-argument constructor. The class `II` has only a one-argument constructor. The object `i` is declared with an integer. The zero-argument constructor of the base class and the one-argument constructor of the derived class are executed.

### 11.13.7 Base and derived classes without default constructors

**11.20 Write a program to declare base and derived class without default constructor.**

```
#include<iostream.h>
#include<conio.h>

class I
{
 public:
 int x;
 I (int k)
 {
 x=k;
 cout<<"\nOne argument base class constructor";
 };
};

class II : public I
{
 int y;
 public:
 II (int j):I(j)
 {
 cout<<"\nOne argument derived class constructor";
 y=j;
 }
};

int main()
{
```

```

 clrscr();
 II i(2);
 return 0;
}

```

## OUTPUT

**One argument base class constructor**

**One argument derived class constructor**

**Explanation:** In the above program, no default constructor is declared. In the class II one argument, a constructor is declared, and the base class constructor is explicitly invoked. When object i is declared, a one-argument constructor of both the classes is executed. In the absence of an explicit call of the base class constructor, the compiler will display the error message “Can-not find default constructor to initialize base class ‘I.’”

### 11.13.8 Constructors and multiple inheritance

**11.21 Write a program to derive a class using multiple base classes. Observe the execution of constructor when the object of derived class is declared.**

```

#include<iostream.h>
#include<conio.h>
class I
{
 public:
 I()
 {
 cout<<"\n Zero argument constructor of base class I";
 }
};
class II
{
 public:
 II() {cout<<"\n Zero argument constructor of base class II";}
};
class III : public II,I
{
 public:
 III()
 {
 cout<<"\n Zero argument constructor of base class III";}
};
int main()
{

```

```

 clrscr();
 III i;
 return 0;
}

```

## OUTPUT

**Zero argument constructor of base class II**

**Zero argument constructor of base class I**

**Zero argument constructor of base class III**

**Explanation:** The classes I and II are the base classes of the derived class III. In function main(), i is an object of the derived class III. The execution of constructors depends on the sequence given while deriving a class as per the following statement:

```
class III : public II,I
```

Here, the class II is the first base class, and class I is the second base class; the execution sequence of the constructors will be as shown in the output.

### 11.13.9 Constructors in multiple inheritance with explicit calls

**11.22 Write a program to derive a class using multiple base classes. Invoke the constructors of base classes explicitly.**

```

#include<iostream.h>
#include<conio.h>

class I
{
 public:
 I() {cout<<"\n Zero argument constructor of base class I";}
};

class II
{
 public:
 II() {cout<<"\n Zero argument constructor of base class II";}
};

class III : public II,I
{
 public:
 III():II(), I()
 {
 cout<<"\n Zero argument constructor of base class III";
 }
};

```

```
int main()
{
 clrscr();
 III i;
 return 0;
}
```

## OUTPUT

**Zero argument constructor of base class II**

**Zero argument constructor of base class I**

**Zero argument constructor of base class III**

**Explanation:** In this program in the class III, explicitly the constructor of both the base classes II and I are invoked. The execution sequence can be observed in the output as shown above. The execution sequence of the constructors depends on the sequence of the base class and not on the explicit calls.

### 11.13.10 Multiple inheritance and virtual class

**11.23 Write a program to derive a class using multiple base classes. Invoke the constructors of base classes explicitly. Declare any one base class as virtual.**

```
#include<iostream.h>
#include<conio.h>

class I
{
public:
 I()
 {
 cout<<"\n Zero argument constructor of base class I";
 }
};

class II
{
public:
 II()
 { cout<<"\n Zero argument constructor of base class II"; }
};

class III : public II, virtual I
{
public:
 III() : II(), I()
 {
```

```

 cout<<"\n Zero argument constructor of base class III";
 }
};
int main()
{
 clrscr();
 III i;
 return 0;
}

```

## OUTPUT

**Zero argument constructor of base class I**  
**Zero argument constructor of base class II**  
**Zero argument constructor of base class III**

**Explanation:** In this program, the base class I is declared a virtual class while deriving the class. The constructor of the virtual class is executed first. The execution of the constructor here is not according to the sequence of the base class.

### 11.13.11 Execution of constructors in multilevel inheritance

**11.24 Write a program to derive a classes using multilevel inheritance and observe the execution sequence of constructors.**

```

#include<iostream.h>
#include<conio.h>

class I
{
 public:
 I()
 {
 cout<<"\n Zero argument constructor of base class I";
 }
};

class II : public I
{
 public:
 II()
 {
 cout<<"\n Zero argument constructor of base class II";
 }
};

class III : public II
{

```

```

 public:
 III() {cout<<"\n Zero argument constructor of base class III";}
};
int main()
{
 clrscr();
 III ii;
 return 0;
}

```

## OUTPUT

**Zero argument constructor of base class I**

**Zero argument constructor of base class II**

**Zero argument constructor of base class III**

**Explanation:** In this program, the class II is derived from the class I. The class III is derived from the class II. The class II is a base as well as a derived class. In function main(), ii is an object of the class III. The constructors are executed from base to derived classes as shown in the output. Table 11.3 shows the order of execution of the constructors.

**Table 11.3** Execution sequences of constructors

| Statements                                          | Sequence of execution                                                                                           | Remarks                |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------|
| Class II : public I                                 | I() – Base class constructor<br>II() – Derived class constructor                                                | Single inheritance     |
| Class III: public I, II                             | I() – Base class constructor<br>II() – Base class constructor<br>III() – Derived class constructor              | Multiple inheritance   |
| Class III: public I, virtual II                     | II() – Virtual class constructor<br>I() – Base class constructor<br>III() – Derived class constructor           | Multiple inheritance   |
| class II : public I {};<br>class III: public II {}; | I() – First base class constructor<br>II() – Second base class constructor<br>III() – Derived class constructor | Multilevel inheritance |

## 11.14 OBJECT AS A CLASS MEMBER

Properties of one class can be used in another class using inheritance or using the object of a class as a member in another class. Declaring the object as a class data member in another class is also known as *delegation*. When a class has an object of another class as its member, such a class is known as a *container class*.

In inheritance, the derived class can use members of the base class. Here, the derived class is a kind of base class. The programmer can also add new members to the derived class.

In delegation, the class consists of objects from other classes. The composed class uses the properties of other classes through their objects. This kind of a relationship is known as *has-a-relationship* or *containership*.

```

class I
{
 ***** // members

}
class II
{
 I j; // object of class I as member

}

```

### 11.25 Write a program use object of one class in another class as a member.

```

#include<iostream.h>
#include<conio.h>

class I
{
 public:
 int x;
 I()
 {
 cout<<"\n Constructor of class I";
 x=20;
 }
};

class II
{
 public:
 int k;
 I y;
 II()
 {
 k=30;
 cout<<"\n Constructor of class II";
 }
 void show()
 {
 cout<<"\n x="<<y.x <<" k="<<k;
 }
};

int main()
{
 clrscr();

```

```

 ii.show();
 return 0;
}

```

## OUTPUT

**Constructor of class I**

**Constructor of class II**

**x= 20 k = 30**

**Explanation:** In the above program, the class II contains integer k and object y of class I. In function main(), object ii is an object of class II. The constructor of class I is executed first, because when the compiler reaches the class II, it finds an object of class I. We know that the object declaration always executes the constructor of that class. Thus, the object of class I is declared, and the constructor of class I is executed. The constructor of class II is executed, and the variable of class II is initialized. The member function show() displays the contents of x and k. The contents of class I member is obtained from object y. The dot (.) operator is used to access the elements of class I.

### 11.26 Write a program to access member variable of base class using object, scope access operator and direct.

```

#include<iostream.h>
#include<conio.h>

class A1
{
 public:
 char name[15];
 int age;
};

class A2: public A1
{
 private:
 A1 a;
 float height;
 float weight;
 public:
 A2()
 {
 clrscr();
 cout<<"Access Using Scope Access operator\n";
 cout<<"Name:"; cin>>A1::name;
 cout<<"Age:"; cin>>A1::age;
 cout<<"Access Using object of the class\n";

```



```

 cout<<"Name:"; cin>>a.name;
 cout<<"Age:"; cin>>a.age;
 cout<<"Access Using direct member variables\n";
 cout<<"Name:"; cin>>name;
 cout<<"Age:"; cin>>age;
 cout<<"Height:"; cin>>height;
 cout<<"Weight:"; cin>>weight;
}
~A2()
{
 cout<<"\nDisplay using Scope Access operator\n";
 cout<<"\nName:" <<A1::name;
 cout<<"\nAge:" <<A1::age;
 cout<<"\nDisplay Using object of the class\n";
 cout<<"\nName:" <<a.name;
 cout<<"\nAge:" <<a.age;
 cout<<"\nAccess Using direct member variables\n";
 cout<<"\nName:" <<name;
 cout<<"\nAge:" <<age;
 cout<<"\nHeight:" <<height;
 cout<<"\nWeight:" <<weight;
}
};
int main()
{
 A2 x;
 return 0;
}

```

## OUTPUT

```

Name : Ajay
Age : 21
Access Using object of the class
Name : Amit
Age : 20
Access Using direct member variables
Name : Arun
Age : 19
Height : 5.5
Weight: 31
Display using Scope Access operator
Name : Arun
Age : 19
Display Using object of the class

```

**Name : Amit**  
**Age : 20**  
**Access Using direct member variables**  
**Name : Arun**  
**Age : 19**  
**Height: 5.5**  
**Weight: 31**

**Explanation:** In the above program, A1 and A2 are two classes. The class A2 is derived from the class A1. The class A1 has two public member variables. The class A2 has three private members; one of the members is an object of class A1, that is, object a. The object a holds its separate set of member variables of class A1. There are three ways of accessing the member variables of base class A1. Table 11.4 describes the status of access of the member variables of class A1 and the derived class A2 when we use access specifiers such as private, public, and protected.

**Table 11.4** Access Specifiers

| Access (using)           | Public   | Private      | Protected    |
|--------------------------|----------|--------------|--------------|
| Scope Access :: operator | Possible | Not Possible | Possible     |
| Object                   | Possible | Not Possible | Not Possible |
| Direct                   | Possible | Not Possible | Possible     |

- (1) The first way is to use scope access operators as per the following statements:

```
cin>>A1::name;
cin>>A1::age;
```

In the above statements, A1 is a class name, and name and age are member variables of class A1. When an access specifier is public or protected, the above statements are valid.

- (2) The second method is to use an object of the base class, as per the following statements:

```
cin>>a.name;
cin>>a.age;
```

In the above statement, the member variables of class A1 are accessed using an object of the same class, who is the member of the derived class. This is possible only when the member variables of class A1 are public and is not possible if the member variables of class A1 are protected or private.

- (3) The third method directly uses the member variables. Direct access is possible when the access specifier is public or protected, as per the following statements:

```
cin>>name;
cin>>age;
```

**11.27 Write a program to derive a class from two base classes. Use objects of both the classes as member variables for derived class. Initialize and display the contents of classes using constructor and destructor.**

```
#include<iostream.h>
#include<conio.h>

class A
{
 public :
 int a1;
};

class B
{
 public:
 int b1;
};

class AB
{
 public:
 A a;
 B b;
 public :
 AB()
 {
 a.a1=65;
 b.b1=66;
 cout<<"a1="<<a.a1 <<"b1="<<b.b1;
 }
 ~AB(){};
};

int main()
{
 clrscr();
 AB ab;
 return 0;
}
```

### OUTPUT

**a1=65 b1=66**

**Explanation:** In the above program, classes A and B are declared with one integer each. The class AB is declared, which contains objects of classes A and B as member variables. The constructor of class AB initializes the member variables of classes A and B. The constructor also displays contents on the screen. Finally, the destructor destroys the object.

## 11.15 ABSTRACT CLASSES

When a class is not used for creating objects, it is called an `abstract class`. The abstract class can act as a base class only. It is a layout abstraction in a program, and it allows a base on which several levels of inheritance can be created. The base classes act as a foundation of the hierarchical class. An abstract class is developed only to act as a base class and to inherit, and no objects of this class are declared. An abstract class gives a skeleton or a structure; using this, other classes are shaped. The abstract class is central and generally present at the starting of the hierarchy. The `hierarchy of classes` means a chain or group of classes that are involved with one another. In the last program, `class A` is an abstract class, because no instance (object) of class A is declared.

## 11.16 QUALIFIER CLASSES AND INHERITANCE

The following program explains the behavior of qualifier classes and the classes declared within them with inheritance.

### 11.28 Write a program to create derived class from the qualifier class.

```
#include<iostream.h>
#include<conio.h>

class A
{
 public:
 int x;
 A() {}
 class B
 {
 public:
 int y;
 B() {}
 };
}; class C : public A,A::B
{
 public:
 int z;
 void show()
 {
 cout<<endl<<"x="<<x <<"y="<<y<<"z="<<z;
 }
 C (int j,int k, int l)
 {
 x=j;
 y=k;
 z=l;
 }
};
```

```
int main()
{
 clrscr();
 C c(4,7,1);
 c.show();
 return 0;
}
```

## OUTPUT

**x = 4 y =7 z = 1**

**Explanation:** In the above program, the class B is defined inside the class A. The class A is a qualifier class of class B. The class C is inherited from the classes A and B. In the statement `class C: public A, A::B`, class C is inherited from A and B. To access the class B, it is preceded by the qualifier class A and the scope access operator. If we mention only class A, the class B will not be considered for inheritance. Similarly, if we mention only class B, the qualifier class A will not be considered for inheritance.

## 11.17 CONSTRUCTORS IN DERIVED CLASS

When a class is declared, a constructor is also declared inside the class in order to initialize data members. It is not possible to use a single constructor for more classes. Every class has its own constructor and destructor with a similar name as the class. When a class is derived from another class, it is possible to define a constructor in the derived class, and the data members of both base and derive classes can be initialized. It is not essential to declare a constructor in a base class. Thus, the constructor of the derived class works for its base class; such constructors are called `constructors` in the derived class or `common constructors`.

### 11.29 Write a program to initialize member variables of both base and derived class using a constructor of derived class.

```
#include<iostream.h>
#include<conio.h>

class A
{
 protected:
 int x;
 int y;
};

class B : private A
{
 public:
 int z;
 B() {x=1,y=2,z=3;
```

```

 cout<<"x="<<x <<"y="<<y <<"z="<<z; }
};
int main()
{
 clrscr();
 B b;
 return 0;
}

```

**OUTPUT**

**x= 1 y =2 z=3**

**Explanation:** In the above program, the classes A B are declared. The class B is derived from the class A. The constructor of the class B initializes member variables of both classes. Hence, it acts as a common constructor of both base and derived classes.

**11.18 POINTERS AND INHERITANCE**

The private and public member variables of a class are stored in successive memory locations. A pointer to a public member variable gives us access to private member variables.

The same is true for the derived class. The member variables of the base and derived classes are also stored in successive memory locations. The following program explains the mechanics of accessing private data members of the base class using the address of public member variables of the derived class using pointers. Here, no member functions are used.

**11.30 Write a program to access private member variables of base class using pointers**

```

#include<iostream.h>
#include<conio.h>

class A
{
 private:
 int x;
 int y;
 public:
 A() {
 x=1;
 y=2;
 }
};

class B : private A
{
 public:

```

```

 int z;
 B() {z=3;}
};
int main()
{
 clrscr();
 B b; // object declaration
 int *p; // pointer declaration
 p=&b.z; // address of public member variable is stored in
 pointer
 cout<<endl<<"Address of z:"<<(unsigned)p <<"<<"Value of
 z:"<<*p;
 p--; // points to previous location
 cout<<endl<<"Address of y:"<<(unsigned)p <<"<<"Value of
 y:"<<*p;

 p--;
 cout<<endl<<"Address of x:"<<(unsigned)p <<"<<"Value of
 x:"<<*p;
 return 0;
}

```

## OUTPUT

**Address of z : 65524 Value of z :3**  
**Address of y : 65522 Value of y :2**  
**Address of x : 65520 Value of x :1**

**Explanation:** In the above program, class A contains two private member variables x and y. The constructor initializes the member variables. The class B is derived from the class A. The class B has one public member variable. In function main(), b is an object of class B. The pointer p is an integer pointer. The address of the member variable z of the derived class B is assigned to the pointer p. By applying a decrease operation, we get the previous memory locations where the member variables of the base class are stored. The values of all member variables are displayed along with their addresses.

## 11.19 OVERLOADING MEMBER FUNCTION

The derived class can have a similar function name as the base class member function. An object of the derived class invokes the member function of the derived class even if a similar function is preset in the base class.

### 11.31 Write a program to overload member function in base and derived class.

```

#include<iostream.h>
#include<constream.h>

```

```
class B
{
 public:
 void show()
 {
 cout<<"\n In base class function";
 }
};
class D: public B
{
 public:
 void show()
 {
 cout<<"\n In derived class function";
 }
};
int main()
{
 clrscr();
 B b; // b is object of base class
 D d; // d is object of derived class
 b.show(); // Invokes Base class function
 d.show(); // Invokes Derived class function
 d.B::show(); // Invokes Base class function
 return 0;
}
```

## OUTPUT

**In base class function**

**In derived class function**

**In base class function**

**Explanation:** In this program, the class D is derived from the class B. Both the classes have a similar function show() as a member function. In function main(), the objects of both classes B and D are declared. The object b invokes the member function show(). The object b is the object of the base class; hence, it invokes the member function of show() of the base class. In addition, the object of the base class cannot invoke the member function of the derived class, because the base class does not have information on the classes derived under it.

The object d of the derived class invokes the function show(). When functions with a similar name and argument list are present in both the base and derived classes, the object of the derived class gives first priority to the function of its own class. Thus, the statement d.show() invokes the function of the derived class. To invoke the function of the base class with the object of the derived class, the class name and scope access operator precede



the matching function name. From the above program, it is also clear that we can declare the objects of both base and derived classes. The objects of both base and derived classes are independent of one another.

## 11.20 ADVANTAGES OF INHERITANCE

- (1) The most frequent use of inheritance is for deriving classes using existing classes, which provides reusability. The existing classes remain unchanged. By reusability, the development time of software is reduced.
- (2) The derived classes extend the properties of base classes to generate more dominant objects.
- (3) The same base classes can be used by a number of derived classes in class hierarchy.
- (4) When a class is derived from more than one class, all the derived classes have similar properties to those of base classes.

## 11.21 DISADVANTAGES OF INHERITANCE

- (1) Though object-oriented programming is frequently propagandized as an answer for complicated projects, inappropriate use of inheritance makes programs more complicated.
- (2) Invoking member functions using objects creates more compiler overheads.
- (3) In class hierarchy, various data elements remain unused, and the memory allocated to them is not utilized.

## 11.22 MORE PROGRAMS

**11.32 Write a program to explain execution of constructor and destructor in single inheritance.**

```
#include<iostream.h>
#include<conio.h>
// Constructor, Destructor and Single Inheritance //

class Father
{
 protected:
 char name[20];
 public:
 Father()
 {cout<<"\n Base Class Constructor.";
 cout<<"\n Enter Father Name:";
 cin>>name;
 }
 ~Father()
 {
```

```
 cout<<"\n Base Class Destructor.\n";
 }
};
class Child: private Father
{
 char cname[15];
public:
 Child()
 {
 cout<<"\n Derived Class Constructor.";
 cout<<"\n Enter Child Name:";
 cin>>cname;
 }
 ~ Child()
 {
 cout<<"\n Derived Class Destructor\n";
 cout<<" "<<cname <<" "<<name;
 }
};
int main()
{
 clrscr();
 Child C;
 return 0;
}
```

## OUTPUT

**Base Class Constructor.**

**Enter Father Name : Manoj**

**Derived Class Constructor.**

**Enter Child Name : Sanjay**

**Derived Class Destructor**

**Sanjay Manoj**

**Base Class Destructor.**

**Explanation:** In the above program, two classes, class father and class child, are declared with one character data member each. Both the classes contain a constructor and a destructor. The class child is derived from the class father. Hence, the class father is the base class, and the class child is the derived class.

In function main(), C is an object of the derived class child. The constructor of the base class is executed first followed by the constructor of the derived class. However, the destructor of the derived class is executed first. After this, the destructor of the base class is executed.

Here, the destructor of the derived class prints the contents of both the data members. The base class has no knowledge of the derived class. Hence, it cannot access the data members of the derived class.

---

**11.33 Write a program to explain execution of constructor and destructor in multilevel inheritance.**

```
#include<iostream.h>
#include<conio.h>
// Constructor, Destructor and Multilevel Inheritance //
class Grandfather
{
 protected:
 char gname[20];
 public:
 Grandfather()
 {
 cout<<"\n Constructor of class Grandfather\n";
 cout<<"\n Enter Grandfather Name:";
 cin>>gname;
 }
 ~Grandfather()
 {
 cout<<"\n Destructor of class Grandfather\n";
 }
};
class Father : public Grandfather
{
 protected:
 char name[20];
 public:
 Father()
 {cout<<"\n Constructor of class Father\n";
 cout<<"\n Enter Father Name:";
 cin>>name;
 }
 ~Father()
 {
 cout<<"\n Destructor of class Father\n";
 }
};
class Child: private Father
{
 char cname[15];
 public:
```

```
 Child()
 {
 cout<<"\n Constructor of class Child\n";
 cout<<"\n Enter Child Name:";
 cin>>cname;
 }
 ~ Child()
 {
 cout<<"\n Destructor of class Child\n";
 cout<<"\n Grand Father:"<<gname;
 cout<<"\n Father:"<<name;
 cout<<"\n Child:"<<cname;
 }
};
int main()
{
 clrscr();
 Child C;
 return 0;
}
```

## OUTPUT

**Constructor of class Grandfather**

**Enter Grandfather Name : x**

**constructor of class Father**

**Enter Father Name : y**

**Constructor of class Child**

**Enter Child Name : z**

**Destructor of class Child**

**Grand Father : x**

**Father : y**

**Child : z**

**Destructor of class Father**

**Destructor of class Grandfather**

**Explanation:** In the above program, three classes are declared. They are grandfather, father, and child. Each one of them has a character data-type member. Each class contains a pair of a constructor and a destructor. The class father is derived from the class grandfather. Similarly, the class child is derived from the class father. The class grandfather is a base class of the class father. The class father is a base class of the class child. The class child is derived from the class father. The class father is an intermediate class that acts as a base class as well as a derived class. Constructors are executed from the base class to the derived class, and destructors are executed from the derived class to the base class.

**11.34 Write a program to explain multilevel inheritance with member functions.**

```
#include<iostream.h>
#include<conio.h>
// Multilevel Inheritance and member functions//
class Grandfather
{
 private:
 char gname[20];
 public:
 void getg()
 {
 cout<<"\n Enter Grandfather Name:";
 cin>>gname;
 }
 void showg()
 {cout<<"\n Grandfather Name:"<<gname;}
};
class Father : public Grandfather
{
 private:
 char name[20];
 public:
 void getf()
 {
 cout<<"\n Enter Father Name:";
 cin>>name;
 }
 void showf()
 {cout<<"\n Father Name:" <<name;}
};
class Child : public Father
{
 private:
 char cname[15];
 public:
 void getc()
 {
 getg();
 getf();
 cout<<"\n Enter Child Name:";
 cin>>cname;
 }
}
```

```
void showc()
{
 showg();
 showf();
 cout<<"\n Child:"<<cname;
}
};
int main()
{clrscr();
 Child C;
 C.getc();
 C.showc();
 return 0;
}
```

## OUTPUT

```
Enter Grandfather Name : XXX
Enter Father Name : YYY
Enter Child Name : ZZZ
Grandfather Name : XXX
Father Name : YYY
Child : ZZZ
```

**Explanation:** The above program is similar to the previous one. Instead of constructor and destructor members, functions are used.

### 11.35 Write a program to explain how constructor and destructor are executed in multiple inheritance.

```
#include<iostream.h>
#include<conio.h>

class A
{
 public:
 A() {cout<<"\n Constructor of class A";}
 ~A() {cout<<"\n Destructor of class A";}
};

class B
{
 public:
 B() {cout<<"\n Constructor of class B";}
 ~B() {cout<<"\n Destructor of class B";}
};
```

```

class C
{
 public:
 C() {cout<<"\n Constructor of class C";}
 ~C() {cout<<"\n Destructor of class C";}
};
class D : public A,B,C
{
 public:
 D() {cout<<"\n Constructor of class D";}
 ~D() {cout<<"\n\n Destructor of class D";}
};
int main()
{
 clrscr();
 D x;
 return 0;
}

```

## OUTPUT

**Constructor of class A**  
**Constructor of class B**  
**Constructor of class C**  
**Constructor of class D**  
**Destructor of class D**  
**Destructor of class C**  
**Destructor of class B**  
**Destructor of class A**

**Explanation:** In the above program the class D is a derived class from base classes A, B, and C. Each class contains a constructor and a destructor. This type of inheritance is called multiple inheritance. The classes A, B, and C are base classes. The execution of construction depends on the order of the base class given while deriving the class. Consider the statement `class D: public A, B, C`. Here, the execution of constructors starts from base class A to base class C followed by derived class D. If we change the order similar to the statement `class D: public C, B, A`, the execution of construction starts from class C to class A followed by derived class D.

### 11.36 Write a program to explain execution of constructor and destructor in hybrid inheritance.

```

#include<iostream.h>
#include<conio.h>
// Constructors and Hybrid Inheritance //

```

```
class A
{
 public:
 A() {cout<<"\n Constructor of class A";}
 ~A() {cout<<"\n Destructor of class A";}
};
class B : public A
{
 public:
 B() {cout<<"\n Constructor of class B";}
 ~B() {cout<<"\n Destructor of class B";}
};
class C
{
 public:
 C() {cout<<"\n Constructor of class C";}
 ~C() {cout<<"\n Destructor of class C";}
};
class D : public B,C
{
 public:
 D() {cout<<"\n Constructor of class D";}
 ~D() {cout<<"\n\n Destructor of class D";}
};
int main()
{
 clrscr();
 D x;
 return 0;
}
```

## OUTPUT

**Constructor of class C**  
**Constructor of class A**  
**Constructor of class B**  
**Constructor of class D**  
**Destructor of class D**  
**Destructor of class B**  
**Destructor of class A**  
**Destructor of class C**

**Explanation:** The above program is similar to the previous one. The class B is derived from class A. The class D is derived from classes B and C. This type of inheritance is called hybrid inheritance. The execution of constructors is similar to multiple inheritance.



**11.37 Write a program to demonstrate single inheritance.**

```

#include<iostream.h>
#include<conio.h>

class A
{
 protected:
 int c;
 public:
 A() {c=0;}
 A (int j) {c=j;}
 void show() {cout<<endl<<"c="<<c;}
 void operator ++() {c++;}
};

class B : public A
{
 public:
 void operator --() {c--;}
};

int main()
{
 clrscr();
 B a;
 ++a;
 a.show();
 --a;
 a.show();
 return 0;
}

```

**OUTPUT****c = 1****c = 0**

**Explanation:** In the above program, the class B is inherited from class A, and hence, the object of the derived class B can access the member functions of the base class. When the object is created, the constructor of the base class is executed. The derived class has no constructor. The member of the derived class can access the members of the base class if the access specifier is public or protected. A protected member can be accessed by the member function of a similar class or any class derived from it.

**11.38 Write a program to access function derived from private inheritance.**

```

#include<iostream.h>
#include<conio.h>

class B

```

```

{
 public:
 void one() {cout<<endl<<"one";}
 void two() {cout<<endl <<"two";}
};
class D : private B
{
 public:
 B::one;
};
int main()
{
 clrscr();
 D d;
 d.one();
 //d.two(); // not accessible
 return 0;
}

```

**Explanation:** In the above program, the class B has two member functions, one() and two(). The class D is derived using private inheritance. The public section of the derived class contains declaration B::one that allows the object of the derived class to access the function. The function one() is accessible, and the function two() is not accessible.

### 11.39 Write a program to derive classes using multilevel inheritance. Create a common constructor in the lowermost derived class.

```

#include<iostream.h>
#include<conio.h>

class A
{
 public:
 int x;
 A() {}
};

class B : public A
{
 public:
 int y;
 B() {}
};

class C : public B
{
 public:
 int z;
}

```

```

void show()
{
 cout<<endl<<"x="<<x <<"y="<<y<<"z="<<z;
}
C (int j,int k, int l)
{
 x=j;
 y=k;
 z=l;
}
};
int main()
{
 clrscr();
 C c(4,7,1);
 c.show();
 return 0;
}

```

### OUTPUT

**x = 4 y =7 z = 1**

**Explanation:** In the above program, classes A, B, and C are declared. The class B is inherited from the class A, and the class C is inherited from the class B. The classes A and B have a zero-argument constructor. The class C has three argument constructors and an initialized element of base classes A and B. Thus, the constructor defined in class C acts as a common constructor between these three classes.

In function main(), the variable **c** is an object of class C, and three integers are passed to it. The function show() is used to display the contents of the variables.

#### 11.40 Write a program to explicitly call constructor of base classes in multiple inheritances.

```

#include<constream.h>
#include<iostream.h>

struct X
{
 X (int a) // constructor
 {
 cout<<a;
 }
};

struct Y // constructor

```

```
{
 Y (int b) {cout<<b;}
};
struct Z : public X, public Y
{
 Z (int p, int q,int r): X(p), Y(q) // constructor
 {
 cout<<r;
 }
};
int main()
{
 clrscr();
 Z z(1,2,3); // Object of derived class
 return 0;
}
```

## OUTPUT

123

**Explanation:** The constructors of the base class are executed first followed by the derived class constructor. When a class is derived using multiple inheritance, the execution of base class constructors depends on the sequence of the base class given in the derivation statement. The statement is as follows:

```
struct Z : public X, public Y
```

Here, both X and Y are base classes, and the constructor of class X is executed first followed by the constructor of class Y.

Both base and derived classes contain a one-argument constructor. When an object is declared, three values are passed to it. In the derived class constructor, constructors of both the classes are explicitly invoked with one argument. Thus, arguments of base as well as derived classes are initialized using constructors.

## SUMMARY

- (1) Inheritance is one of the most useful and essential characteristics of an object-oriented programming language. Inheritance allows the programmer to utilize the previously defined classes with newer ones. The new class is assembled using the properties of the existing classes.
- (2) The procedure of developing a new class from an old class is termed inheritance.
- (3) The new class is termed a derived class, and the old class is called a base class.
- (4) Single inheritance: When a new class is derived from only one base

class, such a type of inheritance is called a single inheritance.

- (5) **Multilevel inheritance:** The procedure of deriving a class from a derived class is called multilevel inheritance.
- (6) **Multiple inheritance or hierarchical inheritance:** When a class is derived from more than one class, this type of inheritance is called multiple inheritance or hierarchical inheritance.
- (7) **Hybrid inheritance:** When a class is derived from another one or more base classes, this process is termed hybrid inheritance.
- (8) When classes are declared virtual, the compiler takes essential caution to avoid duplication of member variables. Thus, we make a class virtual if it is a base class that has been used by more than one derived class as their base class.
- (9) **Execution of constructors in inheritance:** The execution of constructors takes place from base class to derived class.
- (10) **Execution of destructors in inheritance:** The execution of destructors is in opposite order as compared with constructors, that is, from derived class to base class.
- (11) **Public:** If member variables of a class are public, any function can access them.

In C++, the members of a struct and union are by default public.

- (12) **Private:** If member variables of a class are private, member functions and friends can only access them, declared in a similar class. The members of a class are by default private.
- (13) **Protected:** If member variables of a class are protected, its scope is similar to that for private functions. In addition, member functions and friends can use the member classes derived from the declared class, but only in objects of the derived type.
- (14) When a class is not used for creating objects, it is called an abstract class.
- (15) The constructor of the derived class works for its base class; such constructors are called common constructors.
- (16) The derived class can have a similar function name as the base class member function. An object of the derived class invokes a member function of the derived class even if a similar function is preset in the base class.
- (17) Properties of one class can be used in another class using inheritance or using the object of a class as a member in another class. Declaring the object as a class data member in another class is also known as *delegation*. When a class has an object of another class as its member, such a class is known as a *container class*.

## EXERCISES

### (A) Answer the following questions

- (1) What do you mean by inheritance?
- (2) What do you mean by base class and derived class?
- (3) Describe the various types of inheritance with examples.
- (4) What is the difference between single and multilevel inheritance?
- (5) What is the difference between multilevel and hybrid inheritance?
- (6) How are constructors and destructors executed in multilevel inheritance?
- (7) What is the use of the keyword *virtual*?
- (8) What do you mean by *virtual classes*?
- (9) What are abstract classes?
- (10) Describe the use of public, private, and protected access specifiers.

- (11) What are nested classes?
- (12) Explain the mechanism for accessing private members of the base class using pointers.
- (13) What is the difference between private and protected access specifiers?
- (14) Do you think that accessing private data using pointers is a limitation of encapsulation?
- (15) How do structures and classes provide inheritance differently?
- (16) What is the difference between private and protected inheritance?
- (17) What are the advantages and disadvantages of inheritance?
- (18) What do you mean by object delegation?
- (19) What is a common constructor?
- (20) Explain hierarchical inheritance.

**(B) Answer the following by selecting the appropriate option**

- (1) What will be the output of the following program?

```
class A
{public:
int a;
A() {a=10;}
};
class B : public A
{public:
int b;
B() {b=20;}
~ B() {cout<<"\n a="<<a <<"
b="<<b; }
};
void main() {B();}
```

- (a) **a = 10 b = 20**
- (b) a = 20 b = 10
- (c) a = 30 b = 30
- (d) none of the above
- (2) Identify the access specifier
- (a) **public**
- (b) virtual
- (c) void
- (d) \_class
- (3) An object a cannot access the variable

```
class A
{public:
int a;
private:
int b;
public:
A() {a=10,b=20;}
};
void main() {A a;}
```

- (a) **b**
- (b) a

- (c) both (a) and (b)
- (d) both a and b are accessible
- (4) Private data members of a class can be accessed by
- (a) **public member function of a similar class**
- (b) directly by the object
- (c) private member function of a similar class
- (d) none of the above
- (5) In multilevel inheritance, the middle class acts as
- (a) **base class as well as derived class**
- (b) only base class
- (c) only derived class
- (d) none of the above
- (6) In single inheritance, constructors are executed from
- (a) **base class to derived class**
- (b) derived class to base class
- (c) both (a) and (b)
- (d) none of the above
- (7) In the following program, the object of which class can access all member variables?

```
struct A {int x;};
struct B: A {int y;};
struct C: B {int z;};
struct D: C {int k;};
```

- (a) **Object of class D**
- (b) Object of class B
- (c) Object of class C
- (d) Object of class A
- (8) The protected keyword allows
- (a) **derived class to access base class members directly**

- (b) prevents direct access of public members  
 (c) allows objects to access private members  
 (d) all of the above
- (9) The class is declared virtual when  
 (a) **two or more classes involved in inheritance have a common base class**  
 (b) more than one class is derived  
 (c) we want to prevent a base class from inheritance  
 (d) one of the above
- (10) The ambiguity of members normally occurs in  
 (a) single inheritance  
 (b) multilevel inheritance  
 (c) multiple inheritance  
 (d) **none of the above**
- (11) In the following program, class A is an
- ```
class A {int x;};
class B: A {int y;};
void main() {B b;}
```
- (a) **abstract class**
 (b) virtual class
 (c) derived class
 (d) none of the above
- (12) Class A is a base class of class B. The relationship between them
 (a) **is a kind of a relationship**
 (b) has a relationship
 (c) is a relationship
 (d) none of the above

(C) Attempt the following programs

- (1) Write a program to define three classes A, B, and C. Each class contains private data members. Derive class C from classes A and B by applying multiple inheritance. Read and display the data using a constructor and a destructor.
- (2) Write a program to declare classes X, Y, and Z. Each class contains one character array as a data member. Apply multiple inheritance. Concatenate a string of classes X and Y and store it the class Z. Show all the three strings. Use a constructor and a destructor.
- (3) Write a program to calculate the salary of a medical representative based on the sales. Bonus and incentive to be offered to him will be based on total sales. If the sale exceeds Rs.150000/- follow the particulars of Column (1); otherwise, follow those given in Column (2). Apply a conditional constructor and destructor.

Column 1	Column 2
Basic = Rs. 3000. Hra = 20% of basic. Da = 110% of basic. Conveyance = Rs.500. Incentive = 10% of sales. Bonus = Rs. 1500.	Basic = Rs. 3000. Hra = 20% of basic. Da = 110% of basic. Conveyance = Rs.500. Incentive = 5% of sales. Bonus = Rs. 1000.

- (4) Write a program to calculate the energy bill. Read the starting and ending meter reading. The charges are as follows:
- (5) The price of a trophy depends on the type of material from which it is created. The following table gives the list of prizes. Write a program to input the serial number of the material. In addition, display the material and its prize.

No. of Units Consumed	Rates in (Rs.)
200 – 500	4.50
100 – 200	3.50
Less than 100	2.50

Serial no.	Material	Prize of trophy
01	Gold	Rs. 20000/-
02	Silver	Rs. 9500/-

03	Steel	Rs. 5000/-
04	Bronze	Rs. 500/-
05	Pewter	Rs. 750/-

- (6) A newspaper agent pays two variant rates for the delivery of newspapers. A delivery boy can earn Rs.2/- on a morning delivery but only Rs. 1.50/- on an evening delivery. Boys are salaried either for a morning delivery or for an evening paper round but not both morning and evening paper rounds. Write a program to read (input) the code for rounds (0 for morning and 1 for evening round) and the number of total paper rounds in one week done by a boy; compute his earnings and display it.

(D) Find bugs in the following programs

(1)

```
struct A {int x;};
struct B {int y;};
class C: public A, B {};
void main()
{
    C c;
    c.x=20;
    c.y=30;
}
```

(2)

```
struct A {int x;};
struct B: A {int y;};
struct C: A {int z;};
struct D: B,C {int k;};
void main()
{
    D d;
    d.x=20;
}
```

- (7) The postage for ordinary post is Rs. 2/- for the first 15 grams and Rs. 1 for each additional 10 grams. Write a program to calculate the charge of the postage for a post weighing N grams. Read the weights of N packets and display the total amount of postage.
- (8) Declare a class of vehicle. Derived classes are two-wheelers, three-wheelers, and four-wheelers. Display the properties of each type of vehicle using the member function of classes.
- (9) Create classes' country, state, city, and village. Arrange these classes in a hierarchical manner.

(3)

```
struct A
{int x;
struct B {int y;};};
struct C : A::B,A {};
void main()
{C c;
c.x=40;
c.y=20;
}
```

(4)

```
struct A {int x;};
class B : A
{A a;};
void main()
{
    B b; b.x=20;
    b.a.x=30;
}
```


This page is intentionally left blank.

Arrays

12

CHAPTER

CHAPTER OUTLINE

- 12.1 Introduction
- 12.2 One-dimensional Array Declaration and Initialization
- 12.3 Characteristics of Arrays
- 12.4 Accessing Array Elements Through Pointers
- 12.5 Arrays of Pointers
- 12.6 Passing Array Elements to a Function
- 12.7 Passing Complete Array Elements to a Function
- 12.8 Initialization of Arrays Using Functions
- 12.9 Two-dimensional Arrays
- 12.10 Pointers and Two-dimensional Arrays
- 12.11 Three- or Multi-dimensional Arrays
- 12.12 Arrays of Classes

12.1 INTRODUCTION

An array is a very popular, linear, homogenous, and useful data structure that is used to store similar types of data elements in contiguous memory locations under one variable name. An array can be declared to be of any standard or custom data type. The array of characters (strings) type works somewhat differently from the array of integers, floating numbers, and so on.

12.2 ONE-DIMENSIONAL ARRAY DECLARATION AND INITIALIZATION

The declaration of a one-dimensional array is as follows:

Declaration

<code>int a[5] ;</code>

It tells the compiler that 'a' is an integer type of array, and it should store five integers. In this example, 5 is the subscript enclosed within square brackets. The compiler reserves two bytes of memory for each integer array element; that is, 10 bytes are reserved for storing five integers in the memory.

In the same way, arrays of different data types are declared as follows:

One-dimensional Array Declaration
<pre>char ch[10]; float real[10]; long num[5];</pre>

The array initialization is done as follows:

Array Initialization
<pre>int a[5] = {1,2,3,4,5};</pre>

Here, five elements are stored in an array 'a'. The array elements are stored sequentially in separate locations. Then, the question arises of how to call each element individually from this bunch of integer elements. The reading of array elements begins from zero.

Array elements are accessed with the name of the array, and the number within the square brackets specifies the element number. In other words, array elements are called with array names followed by element numbers. Table 12.1 explains the accessing elements.

Table 12.1 Calling Array Elements

a[0]	refers to 1st element i.e. 1
a[1]	refers to 2nd element i.e. 2
a[2]	refers to 3rd element i.e. 3
a[3]	refers to 4th element i.e. 4
a[4]	refers to 5th element i.e. 5

12.3 CHARACTERISTICS OF ARRAYS

- (1) The Declaration `int a[5]` is nothing but a creation of five variables of integer types. Instead of declaring five variables for five values, the programmer can define them in an array.
- (2) All the elements of an array share the same name, and they are distinguished from one another with the help of an element number.
- (3) The element number in an array plays a major role for calling each element.
- (4) Any particular element of an array can be modified separately without disturbing the other elements.

For example, `int a[5] = {1,2,3,4,8};`

If the programmer needs to replace 8 with 10, he/she is not required to change all the other elements except 8. To carry out this task, the statement `a[4] = 10` can be used. Here, the other four elements are left unchanged.

- (5) Any element of an array `a[]` can be assigned/equated to another ordinary variable or an array variable of its type.

For example
`b = a[2];`
`a[2] = a[3];`

- (a) In the statement `b = a[2]` or vice versa, the value of `a[2]` is assigned to `'b'`, where `'b'` is an integer.
- (b) In the statement `a[2] = a[3]` or vice versa, the value of `a[3]` is assigned to `a[2]`, where both the elements are of the same data type.
- (6) The array elements are stored in contiguous memory locations. The amount of storage required for holding the elements of the array depends on its type and size. The total size in bytes for a single-dimensional array is computed as shown below.

$\text{Total bytes} = \text{size of (data type)} \times \text{size of array}$

The rules for array declaration are similar in both C and C++. Simple programs on arrays are demonstrated with C and C++.

12.1 Program executed in C

```
#include<stdio.h>
#include<conio.h>

void main()
{
    char text[3] = "xyz";
    char txt[3] = "abc";
    clrscr();
    printf ("\n %s",text);
}
```

OUTPUT

xyz_abcW__

12.2 Program executed in C++

```
#include<conio.h>
#include<iostream.h>

void main()
{
    char text[3] = "xyz";
    char txt[3] = "abc";
    clrscr();
    cout<<text;
}
```

OUTPUT

XyzÅ

Explanation: Both the above programs are similar except for the syntax. The first and second programs are executed in C (Ver 2.0) and C++ (ver 3.0), respectively.. The character arrays are declared to be exactly equal to the length of the string. The null character is not taken into account. That is why in C the `printf()` statement displays the contents of the next character array followed by the character array. In C++, the `cout` also displays the garbage. It is better to take the null character into account while declaring and initializing the string array. Thus, the above character array can be accurately declared as follows:

(a) `char text[4] = "xyz";`
 (b) `char txt[4] = "abc";`

Now, you will get the correct output.

Given below are a few programs on one-dimensional arrays

12.3 Program to read 5 integers through keyboard and display them.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int num[5],i;
    clrscr();
    for(i=0;i<5;i++)
    {
        cout<<"\n Enter any no:";
        cin>>num[i];
    }
    cout<<" Numbers are:";
    for(i=0;i<5;i++)
    {
        cout<<num[i];
    }
    return 0;
}
```

OUTPUT

```
Enter any no:12
Enter any no:34
Enter any no:45
Enter any no:67
Enter any no:89
Numbers are:12 34 45 67 89
```

12.4 Program to enter 5 numbers through keyboard and display them in ascending order.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int num[5],i,j,temp;
    clrscr();
    cout<<"\n Enter 5 numbers:";
    for(i=0;i<5;i++)
    {
        cin>>num[i];
```



```

    }
    for(i=0;i<5;i++)
    {
        for(j=i+1;j<5;j++)
        {
            if(num[i]>num[j])
            {
                int temp=num[i];
                num[i]=num[j];
                num[j]=temp;
            }
        }
    }

    cout<<"\n Numbers in ascending order are as follows:";
    for(i=0;i<5;i++)
    {
        cout<<" "<<num[i];
    }
    return 0;
}

```

OUTPUT

Enter 5 numbers:23 56 78 99 2

Numbers in ascending order are as follows: 2 23 56 78 99

Explanation: For arranging numbers in the ascending order, the numbers are sorted with two for loops, and the same sorted numbers are displayed.

12.4 ACCESSING ARRAY ELEMENTS THROUGH POINTERS

We can quickly and easily access array elements using pointers, as these elements are stored in contiguous memory locations. A pointer variable contains an address, and it is easy to manipulate data with the help of addresses. When a pointer is incremented, the address gets incremented, and we can access the contents of memory locations. This particular method requires less memory; hence, execution is fast.

12.5 Program to display elements of an array using pointer. Display addresses of elements.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    int *p,num[5]={1,2,3,4,5},j;

```

```

    clrscr();
    p=&num[0];
    cout<<"\nNumber Address"<<endl;
    for (j=0;j<5;j++)
    cout<<" "<<*(p+j) <<" "<<unsigned(p+j)<<endl;
    return 0;
}

```

OUTPUT

Number	Address
1	65452
2	65454
3	65456
4	65458
5	65460

12.5 ARRAYS OF POINTERS

So far, we have studied arrays of different standard data types, such as arrays of `int`, `float`, and `char`. In the same way, the 'C++' language also supports arrays of pointers. It is nothing but a collection of addresses. Here, we store the addresses of variables for which we have to declare arrays as pointers.

12.6 Write a program to store addresses of different elements of an array using array of pointers.

```

#include<conio.h>
#include<iostream.h>

int main()
{
    int *arrp[3];
    int arr[3]={5,10,15},k;
    for(k=0;k<3;k++)
    arrp[k]=arr+k;
    clrscr();
    cout<<"\n\t Address Element"<<endl;
    for (k=0;k<3;k++)
    {
        cout<<"\t" <<unsigned(arrp[k]);
        cout<<"\t"<<*(arrp[k])<<endl;
    }
    return 0;
}

```

```
}

```

OUTPUT

Address Element

```
65418  5
65420  10
65422  15
```

Explanation: In the above program, `*arrp[3]` is declared as an array of pointers. Using first for loop, the addresses of various elements of array `'arr[]'` are assigned to `'*arrp[]'`. The second for loop picks up addresses from `'*arrp[]'` and displays the values present at those locations. Here, each element of `'*arrp[]'` points to the respective element of array `'arr[]'`.

Table 12.2 Arrays of pointers in memory

Element no.	Array of values	Element no.	Array of addresses
arr[0]	5	arrp[0]	65418
arr[1]	10	arrp[1]	65420
arr[2]	15	arrp[2]	65422

12.6 PASSING ARRAY ELEMENTS TO A FUNCTION

We can pass elements to a function by using call-by-value or call-by-reference methods. In the call-by-value method, elements (values) of an array are passed to the function; whereas in the call-by-reference method, addresses of elements are passed to the function.

The following program demonstrates the call-by-value method:

12.7 Program to pass elements of an array to a function by using call by value.

```
#include<iostream.h>
#include<conio.h>
void display(int);
void main()
{
    int num[5]={1,2,3,4,5},i;
    clrscr();
    cout<<"\nElements in the reverse order are as follows:";
    for(i=4;i>=0;i--)
    {
        display(num[i]);
    }
}
```



```

}
void display(int x)
{
    cout<<" "<<x;
}

```

OUTPUT

Elements in the reverse order are as follows: 5 4 3 2 1

Explanation: In the above program, individual elements of array num[5] are passed to the function display(int), and the same are displayed in the called function.

The following program demonstrates the call-by-reference method:

12.8 Program to pass elements of an array to a function by using call by reference.

```

#include<iostream.h>
#include<conio.h>
void display(int *);
void main()
{
    int num[5]={1,12,3,4,5},i;
    clrscr();
    cout<<"\nElements of the array are as follows:";
    for(i=0;i<5;i++)
    {
        display(&num[i]);
    }
}
void display(int *x)
{
    cout<<" "<<*x;
}

```

OUTPUT

Elements of the array are as follows: 1 2 3 4 5

Explanation: In the above program, addresses of individual array elements are passed to the function display(int *). The x contains the address of an array element, and *x is the value stored at that address.

12.7 PASSING COMPLETE ARRAY ELEMENTS TO A FUNCTION

It is also possible to pass the entire elements of an array to a function instead of passing the individual elements of an array. The following program explains the concept:

12.9 Program to pass entire elements of an array to a function.

```
#include<iostream.h>
#include<conio.h>
void show(int *,int);

void main()
{
    int num[5]={1,2,3,4,5},i;
    clrscr();
    cout<<"\nElements of the array are as follows:";
    show(&num[0],5);
}

void show(int *x,int y)
{
    int i;
    for(i=0;i<=y-1;i++)
    {
        cout<<" "<<*x;
        x++;
    }
}
```

OUTPUT

Elements of the array are as follows: 1 2 3 4 5

Explanation: In the above program, the address of the zeroth element, that is, `&num[0]`, is passed to the function `show()`. The `for` loop is used to access the array elements using pointers. The `show()` function is invoked with two arguments. The first argument is the address of the zeroth element, and the second one is the number that represents the total number of elements in the array.

12.8 INITIALIZATION OF ARRAYS USING FUNCTIONS

The programmers always initialize the arrays using statements such as `int d[] = {1, 2, 3, 4, 5};` instead of this, the function can also be directly called to initialize the array. The following program illustrates this point.

12.10 Write a program to initialize an array using functions.

```
#include<stdio.h>
#include<conio.h>

main()
{
    int k,c(),d[]={c(),c(),c(),c(),c()};
    clrscr();
```

```

printf ("\n Array d[] elements are :");
for (k=0;k<5;k++)
printf ("%2d",d[k]);
return (NULL);
}
c()
{
static int m,n;
m++;
printf ("\nEnter Number d[%d] : ",m);
scanf ("%d",&n);
return(n);
}

```

OUTPUT

```

Enter Number d[1] : 4
Enter Number d[2] : 5
Enter Number d[3] : 6
Enter Number d[4] : 7
Enter Number d[5] : 8
Array d[] elements are : 4 5 6 7 8

```

Explanation: A function can be called in the declaration of an array. In the above program, `d[]` is an integer array, and `c()` is a user-defined function. When called, the function `c()` reads values through the keyboard. The function `c()` is called from an array; that is, the values returned by the function are assigned to the array. The above program will not work in C.

12.9 TWO-DIMENSIONAL ARRAYS

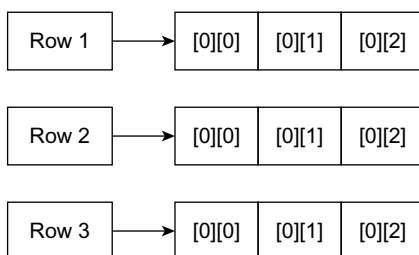


Fig. 12.1 Two-dimensional array

Two-dimensional arrays can be considered a rectangular display of elements with rows and columns, and this is also known as a *matrix*. Consider the following example `int x[3][3]`. The two-dimensional array can be declared as shown in Fig. 12.1.

Table 12.1 Arrangement of two-dimensional array elements

	Column 0	Column 1	Column 2
Row 0	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>
Row 1	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
Row 2	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>

The arrangement of array elements shown in Table 12.1 is only for the sake of understanding. Actually, the elements are stored in continuous memory locations. The two-dimensional array is a collection of two one-dimensional arrays. The meaning of the first argument is in `x[3][3]`

and means the number of rows; that is, the number of one-dimensional arrays, and the second argument indicates the number of elements. The `x[0][0]` means the first element of the first row and column. In one row, the row number remains the same but the column number changes. The number of rows and columns is called the range of the array. A two-dimensional array clearly shows the difference between logical assumptions and the physical representation of data. The computer memory is linear and any type of array may one, two- or multi-dimensional array it is stored in continuous memory location Fig. 12.2.

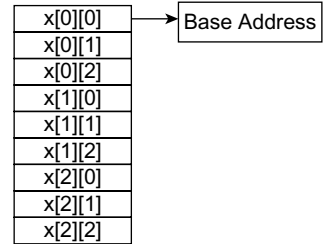


Fig. 12.2 Storage of two-dimensional array

12.11 Program to demonstrate two dimensional array.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int i,j;
    int a[3][3]={1,2,3,4,5,6,7,8,9};
    clrscr();
    cout<<"\n Array elements and address ";
    cout<<"\n \t Col-0          Col-1          Col-2";
    cout<<"\n \t =====          =====          =====";
    cout<<"\nRow0";
    for (i=0;i<3;i++)
    {
        for (j=0;j<3;j++)
            cout<<"\t          "<< a[i][j];
        if(i==2)
            break;
        cout<<"\nRow"<<i+1;
    }
    return 0;
}
```

OUTPUT

Array elements and address

	Col-0	Col-1	Col-2
	=====	=====	=====
Row0	1	2	3
Row1	4	5	6
Row2	7	8	9

Explanation : From the above program's output, we can conclude that the one-dimensional array can be accessed using a single loop. However, for the two-dimensional array, two loops are required for rows and columns. The inner loop helps access the row-wise elements, and the outer loop changes the column number.

12.12 Program to read marks and percentage of students using two dimensional array.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int student[5][2],i,j;
    clrscr();
    for(i=0;i<5;i++)
    {
        cout<<"\n Enter the Roll no and percentage of the student:";
        cin>>student[i][0]>>student[i][1];
    }
    cout<<"\n Roll_no \t percentage ";
    for(i=0;i<5;i++)
    {
        cout<<"\n";
        cout<<student[i][0]<<"\t"<<(student[i][1]);
    }
    return 0;
}
```

{coderipe}

OUTPUT

```
Enter the Roll no and percentage of the student:1 89
Enter the Roll no and percentage of the student:2 78
Enter the Roll no and percentage of the student:3 76
Enter the Roll no and percentage of the student:4 56
Enter the Roll no and percentage of the student:5 90
Roll_no    percentage
1           89
2           78
3           76
4           56
5           90
```

Explanation: In the above program, student [5][2] is declared as an array. The array student [5][2] contains 5 rows and 2 columns. Roll number and percentage scored by the students are read through the keyboard and displayed.

12.10 POINTERS AND TWO-DIMENSIONAL ARRAYS

A matrix can represent the two-dimensional elements of an array. In order to display the elements of the two-dimensional array using pointers, it is essential to have ‘&’ operator as a prefix with the array name followed by element numbers.

12.13 Write a program to display array elements and their addresses using pointers.

```
#include<iostream.h>
#include<stdio.h>
#include<conio.h>

void main()
{
    clrscr();
    int i,j=1,*p;
    int a[3][3]={ {1,2,3}, {4,5,6}, {7,8,9} };
    cout<<"\n\tElements of an array with their addresses";
    p=&a[0][0];
    cout<<"\n";
    for (i=0;i<9;i++,j++)
    {
        cout<<" "<<*p<<" "<<unsigned(p);
        if (j==3)
        {
            cout<<"\n";
            j=0;
        }
        p++;
    }
}
```

OUTPUT

Elements of an array with their addresses

```
1 65506 2 65508 3 65510
4 65512 5 65514 6 65516
7 65518 8 65520 9 65522
```

Explanation : In the above program, the two-dimensional array is declared and initialized. The base address of the array is assigned to integer pointer ‘p’. While assigning the base address of the two-dimensional array, the ‘&’ operator is pre-fixed with the array name followed by the element numbers; otherwise, the compiler shows an error. The statement `p = &a[0][0]` is used in this context. The pointer ‘p’ is printed and incremented in the `for` loop till it prints the entire array of elements. The `if` statement splits a line when three elements in each row are printed.

12.11 THREE- OR MULTI-DIMENSIONAL ARRAYS

The 'C++' program helps in creating an array of multi dimensions. The compiler determines the restrictions on it. The syntax of the declaration of multi-dimensional arrays is as follows:

```
Data Type_Arrayname [S1] [S2] [S3] ... [Si];
```

where S_i is the size of the i^{th} dimensions.

Three-dimensional arrays can be initialized as follows.

```
int mat[3][3][3] = { 1,2,3, 4,5,6, 7,8,9, 1,4,7, 2,5,8,
                    3,6,9, 1,4,4, 2,4,7, 8,8,5};
```

A three-dimensional array can be considered an array of arrays of arrays. The outer array contains three elements. The inner array is two dimensional with regard to size [3][3].

12.14 Write a program to explain the working of three-dimensional array.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int array_3d[3][3][3];
    int a,b,c;
    clrscr();
    for (a=0;a<3;a++)
    for (b=0;b<3;b++)
    for (c=0;c<3;c++)
    array_3d[a][b][c]=a+b+c;
    for (a=0;a<3;a++)
    {
        cout<<"\n";
        for (b=0;b<3;b++)
        {
            for (c=0;c<3;c++)
            cout<<" "<<array_3d[a][b][c];
            cout<<"\n";
        }
    }
    return 0;
}
```

OUTPUT

```
0 1 2
1 2 3
2 3 4
```

```

1 2 3
2 3 4
3 4 5
2 3 4
3 4 5
4 5 6

```

Explanation : The three-dimensional array `array_3d` is initialized. The first three `for` loops are used for adding the values of `a`, `b` & `c`. Here, initially, `a` & `b` are zero, and '`c`' varies from 0 to 2. Hence, the addition of `a`, `b` & `c` will be 0 1 2. This will be printed in the first row. The second output row is one in which `a` = 0, `b` = 1, and `c` varies from 0 to 2. Thus, the output of the second row will be 1 2 3. In this way, the values of `a`, `b` & `c` are changed, and a total of 27 iterations are carried out.

12.12 ARRAYS OF CLASSES

As we know, an array is a collection of similar data types. In the same way, we can also define an array of classes. In such a type of an array, every element is of class type. An array of class objects can be declared as follows.

```

class stud
{
public:
char name[12];      // class declaration
int rollno;
char grade[2];
};
class stud st[3]; // declaration of array of class objects

```

In the above example, `st[3]` is an array of three elements containing three objects of class `stud`. Each element of `st[3]` has its own set class of member variables; that is, `char name[12]`, `int rollno`, and `char grade[2]`. A program is explained as given below.

12.15 Write a program to display names, roll numbers, and grades of 3 students who have appeared for the examination. Declare the class of name, roll numbers, and grade. Create an array of class objects. Read and display the contents of the array.

```

#include<stdio.h>
#include<conio.h>
#include<iostream.h>

void main()
{
    int k=0;
    class stud
    {

```




```

        public:
            char name[12];
            int rollno;
            char grade[2];
    };
    class stud st[3];
    while (k<3)
    {   clrscr();
        gotoxy(2,4);
        cout<<"Name:";
        gotoxy(17,4);
        cin>>st[k].name;
        gotoxy(2,5);
        cout<<"Roll No.:";
        gotoxy(17,5);
        cin>>st[k].rollno;
        gotoxy(2,6);
        cout<<"Grade:";
        gotoxy(17,6);
        cin>>st[k].grade;
        st[k].grade[1]='\0';
        puts ("press any key..");
        getch();
        k++;
    }

    k=0;
    clrscr();
    cout<<"\nName\tRollno Grade\n";
    while (k<3)
    {
        cout<<st[k].name<<"\t"<<st[k].rollno<<" \t"<<st[k].
            grade<<"\n";
        k++;
    }
}

```

OUTPUT

Name	Rollno	Grade
Balaji	50	A
Manoj	51	B
Sanjay	55	C

Explanation: In the above program, class `stud` is declared. Its members are `char name[12]`, `int rollno`, and `char grade[2]`, and all are public. The array `st[3]` of class `stud` is declared. The first while loop and `cin()` statements within it are used for repetitive data reading. The second while loop and `printf()` statements within it display the contents of the array. In the above program, all the member variables are public. If the members are private, the above program will not work. Next, we need to declare member functions to access the data. The program given below explains the reading of private data using member functions.

12.16 Write a program to display names, roll numbers, and grades of 3 students who have appeared for the examination. Declare the class of name, roll numbers, and grade private. Create an array of class objects. Read and display the contents of the array using member functions.

```
#include<stdio.h>
#include<conio.h>
#include<iostream.h>

void put()
{
    cout<<name<<"\t"<<rollno<<"\t"<<grade<<"\n";
}
};

class stud
{
    private :
        char name[12];
        int rollno;
        char grade[2];
    public :
void get()
{   clrscr();
    gotoxy(2,4);
    cout<<"Name      : ";
    gotoxy(17,4);
    cin>>name;
    gotoxy(2,5);
    cout<<"Roll No. : ";
    gotoxy(17,5);
    cin>>rollno;
    gotoxy(2,6);
    cout<<"Grade      : ";
    gotoxy(17,6);
    cin>>grade;
    grade[1]='\0';
```

```

    puts ("press any key..");
    getch();
}
int main()
{
    int m=0;
    class stud st[3];
    while (m<3)
    {
        st[m].get();
        m++;
    }
    cout<<"\nName\tRollno Grade\n";
    m=0;
    while (m<3)
    {
        st[m].put();
        m++;
    }
    return 0;
}

```

OUTPUT

Name	Rollno	Grade
Balaji	50	A
Manoj	51	B
Sanjay	55	C

Explanation: The above program is similar to the previous one. Here, the class members are private. It is not possible to access the private members directly. Using member functions `get ()` and `put ()`, data are read and displayed.

SUMMARY

- (1) An array is a collection of similar data types that are stored in different memory locations.
- (2) The array elements are stored in continuous memory locations. The amount of storage required for holding the elements of an array depends on its type and size.
- (3) The declaration and initialization of one-, two-, and multi-dimensional arrays are studied in this chapter with programming examples.
- (4) How can array elements be accessed through pointers? The answer to this question is supported by theory and programs.
- (5) Passing individual array elements to a function and passing entire elements to a function are studied in this chapter.

- (6) Pointers and two-dimensional arrays are dealt with in this chapter.
- (7) Arrays and classes are also explained.

EXERCISES

(A) Answer the following questions

- (1) What are arrays? How are the elements of an array stored?
- (2) Explain two-dimensional arrays.
- (3) Explain multi-dimensional arrays.
- (4) The array name contains the base address of the array. Can we change the base address of the array?
- (5) What is the relationship between array name and element number? How are elements referred to by using the base address?
- (6) Can we store values and addresses in the same array? Explain.
- (7) Mention the differences between character array and integer array.
- (8) Explain the accessing of array elements by passing value to a function with a programming example.
- (9) Explain the meaning of call by reference with a programming example.
- (10) What do you mean by initialization of array with function?

(B) Answer the following by selecting the appropriate option

- (1) What will happen if you assign values in a few locations of an array?
 - (a) Rest of the elements will be set to 0
 - (b) Compiler error message will be displayed
 - (c) Possible system crash
- (2) When an array is passed to function, what gets passed in reality?
 - (a) Address of the array
 - (b) Element numbers
 - (c) Values of the array
- (3) Which is the correct statement to declare an array?
 - (a) `int x[];`
 - (b) `int x[5];`
 - (c) `int x{5};`
- (4) The array name itself is a pointer to
 - (a) 0th element
 - (b) 1st element
 - (c) last element
- (5) The array name itself is a
 - (a) constant pointer object
 - (b) pointer
 - (c) address
 - (d) none of the above
- (6) `int x[5], *p; and p = x;` then, following which one operation is wrong?
 - (a) `x++;`
 - (b) `p++;`
 - (c) `p = x;`
- (7) `int x[3];` the base address of x is 65460, and the elements are stored at locations
 - (a) 65460, 65462, 65464
 - (b) 65460, 65461, 65462
 - (c) 65460, 65464, 65468
- (8) `int j[4]` the `sizeof(j)` and `sizeof(int)` will display the value
 - (a) 8, 2
 - (b) 2, 8
 - (c) 2, 2

(C) Attempt the following programs

- (1) Write a program to display all the elements of a character array using pointers.
- (2) Write a program to read the elements of two-dimensional arrays and display them with and without pointers.
- (3) Write a program to read five elements from an array. Perform addition and multiplication of these elements.
- (4) Write a program to read a number containing three digits. Perform the square

- roots of each digit. For example, the number is 149. Output should be the square root of each digit, that is, 1 2 3.
- (5) Write a program to read the numbers of any lengths. Count the odd numbers and display them in ascending order.
 - (6) Write a program to find the total marks of three subjects Physics, Chemistry, and Math of three students. Display their ids and marks with the declaration and initialization of arrays.
 - (7) Write a program to initialize two arrays having equal elements. Compute addition and subtraction of the respective elements of both these arrays and display them.
 - (8) Write a program to initialize and read 26 six alphabets in upper case and their ASCII values in reverse (Z to A).
 - (9) Write a program to enter a string in lower as well as in upper case. Convert lower case to upper case and vice versa and display them in reverse order.
 - (10) Read the marks of five subjects obtained by five students in an examination. Display the top two student codes and their marks.
 - (11) Initialize two arrays and merge them. Display the elements in descending order.
 - (12) Write a program to perform the transpose of the matrix.
 - (13) Write a program to check whether the given string is a palindrome.
 - (14) Write a program to sort the numbers using the bubble-sort method.
 - (15) Write a program to sort the numbers using the selection-sort method.
 - (16) Write a program to search for the target element in an array and its position from the last element.
 - (17) Write a program to merge two arrays.
 - (18) Write a program to compute the sum of even and odd numbers.
 - (19) Write a program on the multiplication table of the entered number.

Pointers

13

CHAPTER

CHAPTER OUTLINE

- 13.1 Introduction
- 13.2 Features of Pointers
- 13.3 Pointer Declaration
- 13.4 Arithmetic Operations with Pointers
- 13.5 Pointer to Pointer
- 13.6 `void` Pointers
- 13.7 `wild` Pointers
- 13.8 Pointer to Class
- 13.9 Pointer to Object
- 13.10 The `this` Pointer
- 13.11 Pointer to Derived Classes and Base Class
- 13.12 Pointer to Members
- 13.13 Accessing Private Members with Pointers
- 13.14 Direct Access to Private Members
- 13.15 Addresses of Objects and `void` Pointers
- 13.16 More Programs

13.1 INTRODUCTION

Many learners feel that `pointers` is a puzzling topic. However, pointers can make programs quicker, straightforward, and memory efficient. `C/C++` gives more importance to pointers. Hence, it is important to know the operation and applications of pointers. Pointers are used as tools in `C/C++` and if you fail to understand them, you will lose the power of `C/C++`.

Similar to `C`, in `C++`, variables are used to hold data values during program execution. When declared, every variable occupies certain memory locations. It is possible to access and display the address of the memory location of a variable using ‘`&`’ operator. Memory is arranged in series

of bytes. These series of bytes are numbered from zero onward. The number specified to a cell is known as `memory address`. A pointer variable stores the memory address of any type of variable. The pointer variable and normal variable should be of the same type. The pointer is denoted by (*) asterisk symbol.

A byte is nothing but a combination of eight bits, as shown in Figure 13.1. The binary numbers 0 and 1 are known as bits. Each byte in the memory is specified with a unique (matchless) memory address. The memory address is an unsigned integer starting from zero to uppermost addressing capacity of the microprocessor. The number of memory locations pointed by a pointer depends on the type of the pointer. The programmer should not worry about the addressing procedure of the variables. The compiler takes care of this. The pointers are either 16 bits or 32 bits long.

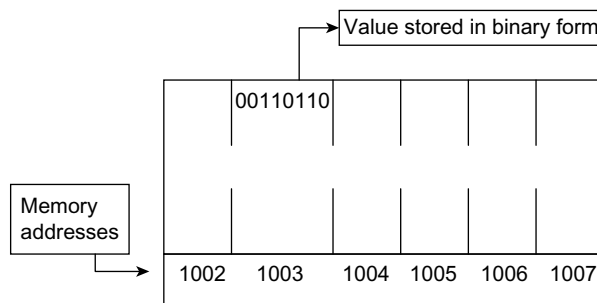


Fig. 13.1 Memory representation

The allocation of memory during program run time is called `dynamic memory allocation`. Such a type of memory allocation is essential for data structure, and it is efficiently handled by pointers. Arrays are another reason for using pointers. Arrays are used to store more values. Actually, the name of the array is a pointer. Command-line arguments are one more reason for using pointers. These arguments are passed to programs and are stored in an array of pointers `argv []`.

POINTERS

A pointer is a memory variable that stores a memory address. Pointers can have any name that is legal for other variables, and it is declared in the same fashion as other variables, but it is always denoted by (*) operator.

13.2 FEATURES OF POINTERS

- (1) Pointers save memory space.
- (2) Execution time with pointers is faster, because data are manipulated with the address, that is, direct access to memory location.
- (3) Memory is accessed efficiently with the pointers. The pointer assigns as well as releases the memory space. Memory is dynamically allocated.
- (4) Pointers are used with data structures. They are useful for representing two-dimensional and multi-dimensional arrays.

- (5) We can access the elements of any type of array, irrespective of its subscript range.
- (6) Pointers are used for file handling.
- (7) Pointers are used to allocate memory in a dynamic manner.
- (8) In C++, a pointer declared to a base class could access the object of a derived class. However, a pointer to a derived class cannot access the object of a base class. The compiler will generate an error message “cannot convert ‘A* to B*,’” where A is the base class and B is the derived class.

13.3 POINTER DECLARATION

Pointer variables can be declared as follows:

Example

```
int *x;  
float *f;  
char *y;
```

- (1) In the first statement, ‘x’ is an integer pointer, and it informs the compiler that it holds the address of any integer variable. In the same way, ‘f’ is a float pointer that stores the address of any float variable, and ‘y’ is a character pointer which stores the address of any character variable.
- (2) The indirection operator (*) is also called the dereference operator. When a pointer is dereferenced, the value at that address stored by the pointer is retrieved.
- (3) Normal variables provide direct access to their own values; whereas a pointer indirectly accesses the value of a variable to which it points.
- (4) The indirection operator (*) is used in two distinct ways with pointers: declaration and dereference.
- (5) When a pointer is declared, the star indicates that it is a pointer, not a normal variable.
- (6) When the pointer is dereferenced, the indirection operator indicates that the value at that memory location stored in the pointer is to be accessed rather than the address itself.
- (7) It should also be noted that * is the same operator which can be used as the multiplication operator. The compiler knows which operator to call, based on the context.
- (8) The ‘&’ is the address operator, and it represents the address of the variable. The address of any variable is a whole number. The operator ‘&’ immediately preceding the variable returns the address of the variable. In the below given example, ‘&’ immediately precedes the variable ‘num,’ which provides the address of the variable.

13.1 Write a program to display the address of the variable.

```
#include<iostream.h>  
#include<conio.h>  
main()  
{  
    int n;  
    clrscr();
```



```

    cout<<"Enter a Number = ";
    cin>>n;
    cout<<"Value of n = "<<n;
    cout<<"Address of n= " <<(unsigned)&n;
    getch();
}

```

OUTPUT

Enter a Number = 10

Value of n = 10

Address of n=4068

Explanation: The memory location of a variable is system dependent. Hence, the address of a variable cannot be predicted immediately. In the above example, the address of the variable 'n' that is observed is **4068**. In Figure 13.2, three blocks are shown to be related to the above program. The first block contains the variable name. The second block represents the value of the variable. The third block is the address of the variable 'n,' where 10 is stored. Here, 4068 is the memory address. The address of the variable depends on various things; for instance, memory model, addressing scheme, and present system settings.

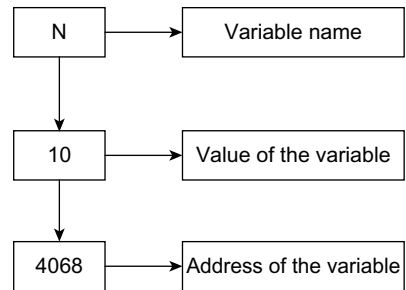


Fig. 13.2 Variable and its memory address

13.2 Write a program to declare a pointer. Display the value and address of the variable-using pointer.

```

#include<stdio.h>
#include<iostream.h>
#include<conio.h>

void main()
{
    int *p;
    int x=10;
    p=&x;
    clrscr();
    printf ("\nx=%d&x=%u\t(Using printf())",x,p);
    cout<<"\n x="<<x <<" &x="<< &x <<"(Using cout())";
    printf ("\nx=%d&x=%u\t(Using pointer)",*p,p);
}

```

```

        cout<<"\n *p="<<*p <<"\t&p=" <<p <<"\t(Contents of
        pointer)";
    }

```

OUTPUT

```

x=10    &x = 65524                (Using printf())
x=10    &x = 0x8f94fff4           (Using cout())
x=10    &x = 65524                (Using pointer)
*p=10   &p = 0x8f94fff4           (Contents of pointer)

```

Explanation: In the above program, `*p` is an integer pointer and `x` is an integer variable. The address of variable `x` is assigned to pointer `p`. Ampersand (`&`) operator preceded with a variable displays the memory location of that variable. The `printf()` and `cout()` statements display addresses in different formats. The `printf()` statement displays the address as an unsigned integer, whereas the `cout()` statement displays the address in a hexadecimal format. We can access the contents of variable `x` using pointer `p`. The output of the program is as shown above.

13.3 Write a program to display memory address of a variable. Type cast the memory address from hexadecimal to unsigned integer.

```

#include<iostream.h>
#include<conio.h>
void main()
{
    int a;
    float b;
    char c;
    clrscr();
    cout<<"\n Enter integer number = ";
    cin>>a;
    cout<<"\n Enter float number = ";
    cin>>b;
    cout<<"\n Enter character = ";
    cin>>c;
    cout<<"\nThe entered int is = "<<a;
    cout<<"\nThe entered float is = "<<b;
    cout<<"\nThe entered char is = "<<c;
    cout<<"\n\nThe entered number is stored at location = "
        <<(unsigned)&a;
    cout<<"\n The entered float is stored at location="
        "<<(unsigned)&b;

```

{coderipe}

```

    cout<<"\n The entered char is stored at location = "<<(unsigned) &c;
    getch();
}

```

OUTPUT

```

Enter integer number    = 34
Enter float number      = 343.34
Enter character          = g
The entered int         is = 34
The entered float       is = 343.339996
The entered char        is = g
The entered number      is stored at location = 4096
The entered float       is stored at location = 4092
The entered char        is stored at location = 4091

```

Explanation: In the above program, variables a, b, and c of int, float, and char type are declared. The entered values are displayed along with their addresses. The addresses of the variables are displayed by a preceding ampersand (&) operator with a variable name. The type-casting syntax (unsigned) is done to type cast the hexadecimal address into an unsigned integer.

Here, you can observe the addresses in descending order. This is because all automatic variables are stored in a stack. The stack always starts from top to lower memory addresses.

13.4 ARITHMETIC OPERATIONS WITH POINTERS

We can perform different arithmetic operations by using pointers. Increment, decrement, prefix, and postfix operations can be performed with pointers. The effects of these operations are shown in Table 13.1.

Table 13.1 Pointers and Arithmetic Operations

Data Type	Initial Address	Operation		Address After Operations		Required Bytes
int i = 2	4046	++	--	4048	4044	2
char c = 'x'	4053	++	--	4054	4052	1
float f = 2.2	4058	++	--	4062	4054	4
long l = 2	4060	++	--	4064	4056	4

From the above table, while referring to the first entry, we can observe that on increment of the pointer variable for integers, the address is incremented by two; that is, 4046 is the original address and on increment, its value will be 4048, because integers require two bytes. Similarly, when the pointer variable for integer is decreased, its address 4048 becomes 4046.

Similarly, for characters, floating point numbers and long integers require 1, 4, and 4 bytes, respectively. After the effect of increment and decrement, the memory locations are shown in Table 13.1.

The following program explains the increase and decrease of pointers:

13.4 Program on pointer incrementation and decrementation.

```
#include<iostream.h>
#include<conio.h>
int main()
{
    clrscr();
    int x=10;
    int *p;
    p=&x;
    cout<<"\n Address of p:"<<unsigned(p) ;
    p=p+4;
    cout<<"\n Address of p:"<<unsigned(p) ;
    p=p-2;
    cout<<"\n Address of p:"<<unsigned(p) ;
    return 0;
}
```

OUTPUT

```
Address of p:65524
Address of p:65532
Address of p:65528
```

Explanation: In the above program, p holds the address of x. The initial address of x is 65524. When p is incremented by 4, it means that the address is increased by 8, because each integer needs two bytes. Here, the address obtained is 65532. Similarly, when the address of x is decreased to 2, the address finally obtained is 65528.

13.5 Program on changing the values of variables using pointer.

```
#include<iostream.h>
#include<conio.h>
int main()
{
    clrscr();
    int x=10;
    int *p;
    p=&x;
    cout<<"\n Value of x:"<<*p;
    *p=*p+10;
```

```

cout<<"\n Value of x:"<<*p;
*p=*p-2;
cout<<"\n Value of x:"<<*p;
return 0;
}

```

OUTPUT

```

Value of x:10
Value of x:20
Value of x:18

```

Explanation: In the above program using pointers, the value of the variable x is first increased and then decreased.

13.5 POINTER TO POINTER

Pointer to pointer is a pointer that stores the address of another pointer. There can be a chain of pointers depending on applications/requirements. In Figure 13.3, x is a simple variable, p is a pointer to the variable x, and q is a pointer to x. The values of variables 'x, *p, and **q' are shown in the boxes, and their addresses are shown outside the boxes.

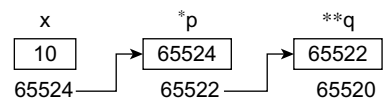


Fig. 13.3 Pointer to pointer

13.6 Program to demonstrate the concept of pointer to pointer.

```

#include<iostream.h>
#include<conio.h>
void main()
{
    clrscr();
    int x=10;
    int *p;
    int **q;
    p=&x;
    q=&p;
    cout<<"\n Value of x ="<<x;
    cout<<"\n Value of x = "<<*p;
    cout<<"\n Value of x = "<<**q;
    cout<<"\n Addresss of x "<<unsigned(*q);
    cout<<"\n Addresss of x "<<unsigned(p);
    cout<<"\n Addresss of x "<<unsigned(&x);
    cout<<"\n\n Addresss of p "<<unsigned(&p);
}

```

```

        cout<<"\n  Adderss of p "<<unsigned(q) ;
        cout<<"\n  Adderss of q "<<unsigned(&q) ;
    }

```

OUTPUT

```

Value of x =10
Value of x = 10
Value of x = 10
Adderss of x 65524
Adderss of x 65524
Adderss of x 65524
Adderss of p 65522
Adderss of p 65522
Adderss of q 65520

```

The program on pointer to pointer is as follows:

Explanation: In the above program, the value of x is initialized as 10. The same value is displayed with the first three cout statements. p holds the address of x, q holds the address of p, and the same are displayed using pointers.

13.6 void POINTERS

Pointers can also be declared as void type. void pointers cannot be dereferenced without explicit-type conversion. This is because, being void, the compiler cannot determine the size of the object that the pointer points to. Though void pointer declaration is possible, void variable declaration is not allowed. Thus, the declaration void p will display an error message "Size of 'p' is unknown or zero" after compilation.

It is not possible to declare void variables such as pointers. A pointer points to an existing entity. A void pointer can point to any type of variable with proper type casting. The size of the void pointer displayed will be two. When a pointer is declared as void, two bytes are allocated to it. Later, using type casting, the number of bytes can be allocated or de-allocated. void variables cannot be declared, because memory is not allocated to them, and there is no place to store the address. Therefore, void variables cannot actually serve the purpose they are made for.

13.7 Write a program to declare a void pointer. Assign address of int, float, and char variables to the void pointer using typecasting method. Display the contents of various variables.

```

// void pointers //
#include<stdio.h>
#include<iostream.h>
#include<conio.h>
int p;

```

```

float d;
char c;
void *pt = &p;           // pt points to p
void main (void)
{
    clrscr();
    *(int *) pt = 12;
    cout<<"\n p="<<p;
    pt = &d;              // pt points to d
    *(float *)pt = 5.4;
    cout<<"\n r="<<d;
    pt=&c;                 // pt points to c
    *(char* )pt='S';
    cout<<"\n c="<<c;
}

```

OUTPUT

```

p=12
r=5.4
c=S

```

Explanation: In the above example, variables `p`, `d`, and `c` are variables of type `int`, `float`, and `char`, respectively. Pointer `pt` is a pointer of type `void`. These entire variables are declared before `main()`. The pointer is initialized with the address of integer variable `p`, that is, the pointer `p` points to variable `x`.

The statement `*(int *) pt = 12` assigns the integer value 12 to pointer `pt`, that is, to a variable `p`. The contents of the variable `p` are displayed using the succeeding statement. The declaration `*(int *)` tells the compiler that the value assigned is of integer type. Thus, the assignment of `float` and `char` type is carried out. The statements `*(int *) pt = 12`, `*(float *) pt = 5.4`, and `*(char*) pt = 'S'` help the compiler exactly determine the size of data type.

13.7 wild POINTERS

Pointers are used to store memory addresses. An improper use of pointers creates many errors in the program. Hence, pointers should be handled cautiously. When a pointer points to an unallocated memory location or to a data value whose memory is de-allocated, such a pointer is called a **wild pointer**. The wild pointer generates garbage memory location and pendent reference.

When a pointer pointing to a memory location vanishes, the memory is transformed into garbage memory. It indicates that a memory location exists but a pointer is destroyed. This happens when the memory is not de-allocated explicitly.

The pointer becomes a wild pointer due to the following reasons:

- (1) Pointer declared but not initialized
- (2) Pointer alteration
- (3) Accessing destroyed data

- (1) When a pointer is declared and not initialized, it holds an illicit address. It is very hard to manipulate such a pointer. Consider the following example:

13.8 Write a program to use wild pointer.



```
#include<iostream.h>
#include<conio.h>
int main()
{
    clrscr();
    int *x;
    for (int k=0;k<10;k++)
        cout<<x[k]<<" ";
    return 0;
}
```

OUTPUT

28005 27760 29793 29541 29728 8303 25954 28704 25205 26988

Explanation: In the above program, pointer x is declared and not initialized. Using for loop, the location-containing pointer is increased, and successive addresses are displayed.

- (2) The careless assignment of a new memory location in a pointer is called **pointer alternation**. This happens when another wild pointer accesses the location of a legal pointer and converts the legal pointer into a wild one. Consider the following program:
- (3) Sometimes, the pointers attempt to access the data that no longer have life. The following program illustrates this:

13.9 Write a program to display the output when a pointer is accessing a temporary data of the memory.

```
#include<iostream.h>
#include<conio.h>
char*instring();
char*inchar();
void main()
{
    clrscr();
    char *ps,*pc;
    ps=instring();
    pc=inchar();
    cout<<" String      : "<<*ps<<endl;
    cout<<" Character  : "<<*pc<<endl;
}

char *instring()
```



```

    {
        char str[] = "cpp";
        return str;
    }
char *inchar()
{
    char g;
    g = 'D';
    return &g;
}

```

OUTPUT

```

String      : c
Character   : .

```

Explanation: In the above program, `ps` and `pc` are character pointers. The function `in-string()` and `inchar()` returns references (base address of the string or character), and they are stored in the pointers `ps` and `pc`, respectively. When the control exits from these functions and returns to `main()`, local variables inside the user-defined functions are destroyed. Thus, the pointers `ps` and `pc` point to the data that are destroyed. The contents displayed of the pointers will be as shown in the output.

13.8 POINTER TO CLASS

We know that a pointer is a variable that holds the address of another data variable. The variable may be of any data type, that is, `int`, `float`, or `double`. In the same way, we can also define pointer to class. Here, the starting address of the member variables can be accessed. Thus, such pointers are called class pointers.

```

Example:

class book
{char name [25];
  char author [25];
  int pages;
};
(a) class book *ptr;
    or
    struct book *ptr;

```

In the above example, `*ptr` is a pointer to a class `book`. Both statements (a) and (b) are valid. The syntax for using pointers with members is as given below.

```

1) ptr->name 2) ptr->author 3) ptr->pages.

```

By executing these three statements, the starting address of each member can be estimated.

13.10 Write a program declares a class. Declare pointer to class. Initialize and display the contents of the class member.

```
#include<iostream.h>
#include<conio.h>
void main()
{
    class man
    {
        public :
        char name[10];
        int  age;
    };
    man m={"RAVINDRA", 15};
    man *ptr;
    ptr=&(man)m;

    // *ptr=(man)m;
    // *ptr=man(m);
    // *ptr=m;
    // ptr=&m;
    clrscr();
    cout<<"\n" <<m.name <<" " <<m.age;
    cout<<"\n"<<ptr->name<<" " <<ptr->age;
}
```

OUTPUT

```
RAVINDRA 15
RAVINDRA 15
```

Explanation: In the above program, the pointer `ptr` points to the object `m`. The statement `ptr = &(man) m` assigns the address of the first member element of the class to the pointer `ptr`. Using the dot operator and arrow operator, contents can be displayed. The display of the class contents is possible, because the class variables are declared as public. The below given statements can also be used to assign the address of the object to the pointer.

- `ptr=&(man)m;`
- `*ptr=(man)m;`
- `*ptr=man(m);`
- `*ptr=m;`
- `ptr=&m;`

13.9 POINTER TO OBJECT

Similar to variables, objects also have an address. A pointer can point to a specified object. The following program illustrates this:

13.11 Write a program to declare an object and pointer to the class. Invoke the member functions using pointer.

```
#include<iostream.h>
#include<conio.h>
class Bill
{
    int qty;
    float price;
    float amount;
public :
    void getdata (int a, float b, float c)
    {
        qty=a;
        price=b;
        amount=c;
    }
    void show()
    {
        cout<<"Quantity    : " <<qty <<"\n";
        cout<<"Price      : " <<price <<"\n";
        cout<<"Amount    : " <<amount <<"\n";
    }
};
int main()
{
    clrscr();
    Bill s;
    Bill *ptr =&s;
    ptr->getdata(45,10.25,45*10.25);
    (*ptr).show();
    return 0;
}
```

OUTPUT

```
Quantity    : 45
Price       : 10.25
Amount      : 461.25
```

Explanation: In the above program, the class `Bill` contains two float and one int members. The class `Bill` also contains the member function `getdata()` and `show()` to read and display the data. In function `main()`, `s` is an object of class `Bill`, and `ptr` is a pointer of the same class. The address of object `s` is assigned to pointer `ptr`. Using pointer `ptr` with arrow operator (`->`) and dot operator (`.`), members and functions are invoked. The statements used for invoking functions are as given below.

```
ptr->getdata (45,10.25,45*10.25);
(*ptr).show();
```

Here, both the pointer declarations are valid. In the second statement, `ptr` is enclosed in brackets, because the dot operator (`.`) has higher precedence as compared with the indirection operator (`*`). The output of the program is as shown above.

13.12 Write a program to create dynamically an array of objects of class type. Use new operator.

```
#include<iostream.h>
#include<conio.h>
class Bill
{
    int qty;
    float price;
    float amount;
public :
    void getdata (int a, float b, float c)
    {
        qty=a;
        price=b;
        amount=c;
    }
    void show()
    {
        cout<<"Quantity   : " <<qty <<"\n";
        cout<<"Price       : " <<price <<"\n";
        cout<<"Amount      : " <<amount <<"\n";
    }
};
int main()
{
    clrscr();
    Bill *s= new Bill[2];
    Bill *d =s;
```

```

    int x,i;
    float y;
    for (i=0;i<2;i++)
    {
        cout<<"\nEnter Quantity and Price : ";
        cin>>x >>y;
        s->getdata(x,y,x*y);
        s++;
    }
    for (i=0;i<2;i++)
    {
        cout<<endl;
        d->show();
        d++;
    }
    return 0;
}

```

OUTPUT

```

Enter Quantity and Price : 5 5.3
Enter Quantity and Price : 8 9.5
Quantity   : 5
Price      : 5.3
Amount     : 26.5
Quantity   : 8
Price      : 9.5
Amount     : 76

```

Explanation: In the above program, the class `Bill` is similar to that in the previous example. In `main()`, using new memory allocation operator, the memory required for two objects is allocated to pointer `s`, that is, 10 bytes. The first `for` loop accepts the data through the keyboard. Immediately after this, the data are sent to the member function `getdata()`. The pointer `s` is incremented. After incrementation, it points to the next memory location of its type. Thus, two records are read through the keyboard. The second `for` loop is used to display the contents on the screen. Here, the function `show()` is invoked. The logic used is similar to that used in the first loop. The functions are invoked using pointers, and this has been explained in the previous example.

13.10 THE `this` POINTER

Use of `this` pointer is now outdated. Objects are used to invoke the non-static member function of the class. For example, if `p` is an object of class `P` and `get()` is a member function of `P`, the statement `p.get()` is used to call the function. The statement `p.get()` operates on `p`. In the same way, if `ptr` is a pointer to a `P` object, the function called `ptr->get()` operates on `*ptr`.

However, the question is, how does the member function `get()` to understand which `p` it is functioning on? C++ compiler provides `get()` with a pointer to `p` called `this`. The pointer `this` is transferred as an unseen parameter in all calls to non-static member functions. The keyword `this` is a local variable that is always present in the body of any non-static member function.

The keyword `this` does not need to be declared. The pointer is rarely referred to explicitly in a function definition. However, it is used implicitly within the function for member references. For example, if `p.get(k)` is called, where `k` is a member of `P`, the keyword `this` is set to `&p` and `k` is set to `this->k`, which is equivalent to `p.k`.

13.13 Write a program to use this pointer and return pointer reference.

```
#include<iostream.h>
#include<conio.h>
class number
{
    int num;
public :
    void input()
    {
        cout<<"\n Enter a Number : ";
        cin>>num;
    }
    void show()
    { cout<<"\n The Minimum Number : "<<num; }
    number min( number t)
    { if (t.num<num)
        return t;
        else
        return *this; }
};
void main()
{
    clrscr();
    number n,n1,n2;
    n1.input();
    n2.input();
    n=n1.min(n2);
    n.show();
}
```

OUTPUT

```
Enter a Number : 152
Enter a Number : 458
The Minimum Number : 152
```

Explanation: In the above program, the class `number` contains one integer number variable `num`. The class also contains member functions `input()`, `show()` and `min()`. The `input()` function reads an integer through the keyboard. The `show()` function displays the contents on the screen. The `min()` function finds a minimum number out of two. The variables `n`, `n1`, and `n2` are objects of class `number`.

In function `main()`, the objects `n1` and `n2` call the function `input()` and read integers. Both the objects `n1` and `n2` are passed to function `min()` with the statement `n=n1.min(n2)`. The object `n` receives the returned value by the function `min()`. The object `n1` calls the function `min()`. The object `n2` is passed as an argument in the function `min()`.

In function `num()`, the formal object `t` receives the contents of argument `n2`. In the same function, the object `n1` is also accessible. As studied earlier, the pointer `this` is present in the body of every non-static member function, and it points to the object `n1`. Using the pointer `this`, we can access the individual member variable of object `n1`. The `if` statement compares the two objects, and the `return` statement returns the smaller object.

13.14 Write a program to enter name and age of two persons. Find the elder person. Use this pointer.



```
#include<iostream.h>
#include<conio.h>

class name
{
    char str[15];
    int age;
public:
    void input()
    {
        cout<<"\n Enter Name and age : ";
        cin>>str;
        cin>>age;
    }
    void show()
    {
        cout<<"\n\nElder person ";
        cout<<"\n Name: " <<str;
        cout<<"\n Age: " <<age;
    }
    name display( name x)
    {
        cout<<"\n\n Contents of object n1 (this pointer)";
        cout<<"\n Name:" <<this->str;
        cout<<"\n Age:" <<this->age;
        cout<<"\n\n Contents of object n2 (x)";
        cout<<"\n Name: " <<x.str;
```

```

        cout<<"\n Age: " <<x.age;
        if (this->age>x.age)
            return *this;
        else
            return x;
    }
};

void main()
{
    clrscr();
    name n,n1,n2;
    n1.input();
    n2.input();
    n=n1.display(n2);
    n.show();
}

```

OUTPUT

```

Enter Name and age : Mahesh 25
Enter Name and age : Suresh 30
Contents of object n1(this pointer)
Name : Mahesh
Age : 25
Contents of object n2 (x)
Name : Suresh
Age : 30
Elder person
Name : Suresh
Age : 30

```

Explanation: The above program is similar to the previous one. Here, contents of explicitly and implicitly passed objects are displayed. The `this` pointer points to the hidden argument (implicit argument). The `if` statement compares the member variable `age` of both the objects and returns the greater one. The object `n` collects the return value of function `display()` and calls the function `show()`. The function `show()` displays the contents of object `n`. Working of `this` pointer is illustrated in Fig. 13.4.

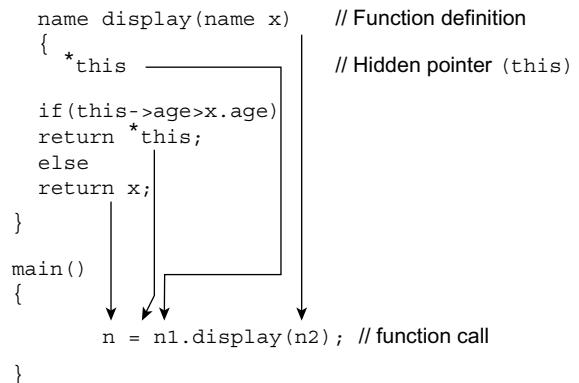


Fig. 13.4 Working of `this` pointer

13.11 POINTER TO DERIVED CLASSES AND BASE CLASS

It is possible to declare a pointer that points to the base class as well as the derived class. One pointer can point to different classes. For example, X is a base class and Y is a derived class. The pointer pointing to X can also point to Y.

13.15 Write a program to declare a pointer to the base class and access the member variable of base and derived class.

```
// Pointer to base object //
#include<iostream.h>
#include<conio.h>
    class A
    {
        public :
        int b;
        void display()
        {
            cout<<"b = " <<b <<"\n";
        }
    };
class B : public A
{
    public :
    int d;
    void display()
    {
        cout<<"b= " <<b <<"\n" <<" d="<<d <<"\n";
    }
};
main()
{
    clrscr();
    A *cp;
    A base;
    cp=&base;
    cp->b=100;
    // cp->d=200; Not Accessible
    cout<<"\n cp points to the base object \n";
    cp->display();
    B b;
    cout<<"\n cp points to the derived class \n";
    cp=&b;
    cp->b=150;
```

```
// cp->d=300; Not accessible
    cp->display();
    return 0;
}
```

OUTPUT

```
cp points to the base object
b = 100
cp points to the derived class
b = 150
```

Explanation: In the above program, A and B are two classes with a one-integer member variable and member function. The class B is derived from class A. The pointer cp points to the class A. The variable base is an object of the class A. The address of the object base is assigned to pointer cp. The pointer cp can access the member variable b, a member of the base class, but cannot access the variable d, a member of the derived class. Thus, the following statement is invalid:

```
a) cp->d = 200;
```

The variable b is an object of class B. The address of object b is assigned to the pointer cp. However, b is an object of the derived class; access to the member variable d is not possible. Thus, the following statement is invalid:

```
b) cp->d = 300;
```

Here, cp is a pointer of the base class, and it cannot access the members of the derived class.

13.16 Write a program to declare a pointer to the derived class and access the member variable of base and derived class.

```
// Pointer to derived object //
#include<iostream.h>
#include<conio.h>

class A
{
    public :
    int b;
    void display()
    {
        cout<<"b = " <<b <<"\n";
    }
};
```

```

class B : public A
{
    public:
    int d;
    void display()
    {
        cout<<"\tb= " <<b <<"\n" <<"\td= " <<d <<"\n";
    }
};
main()
{
    clrscr();
    B *cp;
    B b;
    cp=&b;
    cp->b=100;
    cp->d=350;
    cout<<"\n cp points to the derived object \n";
    cp->display();
    return 0;
}

```

OUTPUT

cp points to the derived object

b= 100

d= 350

Explanation: The above program is similar to the previous one. The only difference is that the pointer `cp` points to the object of the derived class. The pointer `cp` is a pointer of the `class B`. The variable `b` is an object of the `class B`. The address of `b` is assigned to the pointer `cp`. The pointer `cp` can access the member variables of both base and derived classes. The output of the program is shown above.

13.17 Write a program to declare pointer to class. Invoke constructor to initialize data member. Call member function using pointer.

```

#include<iostream.h>
#include<conio.h>

class A
{
    private :
    int a;
    public:

```

```

        A()
        {
            a=20;
        }
void show()
{
    cout<<"\n a = "<<this->a ;
}
};
int main()
{
    clrscr();
    A *p,*q;
    q=p->A::A();
    p->show();
    q->show();
    return 0;
}

```

OUTPUT

a = 20

a = 20

Explanation: In the above program, *p and *q are pointers of class A. The constructor is not executed for pointers. The constructor of class A is explicitly called pointer p. Data members of both pointer objects p and q are initialized to 20. The statements p->show() and q->show() are used to invoke the member function using pointers. The output of the program is shown above.

13.18 Write a program to declare object and pointer to class. Assign value of object to pointer to object. Display their values. Also carry out conversion from basic type to class type.

```

#include<iostream.h>
#include<conio.h>
#include<alloc.h>

class A
{
    private :
        int a;
    public:
        A()

```

```

        {
            a=30;
        }
        void show() { cout<<"\n a = "<<a ;}
        A (int x)
        {
            this->a=x;
        }
    };
    int main()
    {
        clrscr();
        A k,*b,a;
        *b=50;
        k=*b;
        b->show();
        a.show();
        k.show();
        return 0;
    }

```

OUTPUT

```

a = 50
a = 30
a = 50

```

Explanation: In the above program, a and k are objects of class A. The *b is a pointer object. The objects a and k are initialized to 30 by the constructor. The statement *b = 50 assigns 50 to data member a. This is carried out by the conversion constructor A (int). The value of *b is assigned to object k. The member function show() is called by all the three objects, and the output of the program is as given above.

13.12 POINTER TO MEMBERS

It is possible to obtain the address of a member variable and store it to a pointer. The following programs explain how to obtain address and to access member variables with pointers.

13.19 Write a program to initialize and display the contents of the structure using dot (.) and arrow (->) operator.

```

#include<iostream.h>
#include<conio.h>

int main()

```

```

{
    clrscr();
    struct c
    {
        char *name;
    };
    c b, *bp;
    bp->name=" CPP";
    b.name="C &";
    cout<<b.name;
    cout<<bp->name;
    return 0;
}

```

OUTPUT

C & CPP

Explanation: In the above program, structure c is declared. The variable b is the object, and bp is a pointer object of structure c. The elements are initialized and displayed using dot (.) and arrow (->) operators. These are the traditional operators that are commonly used in C. C++ allows two new operators .* and ->* to carry out the same task. These c++ operators are recognized as pointers of member operators.

13.20 Write a program to initialize and display the contents of the structure using .* and arrow ->* operators.

```

#include<iostream.h>
#include<conio.h>

struct data
{
    int x;
    float y;
};

int main()
{
    clrscr();
    int data ::*xp=&data::x;
    float data::*yp=&data::y;
    data d={11,1.14};
    cout<<endl<<d.*xp<<endl<<d.*yp;
    data *pp;
    pp=&d;
}

```

```

    cout<<endl<<pp->*xp<<endl<<pp->*yp;
    d.*xp=22;
    pp->*yp=8.25;
    cout<<endl<<d.*xp<<endl<<d.*yp;
    cout<<endl<<pp->*xp<<endl<<pp->*yp;
    return 0;
}

```

OUTPUT

```

11
1.14
11
1.14
22
8.25
22
8.25

```

Explanation: In the above program, struct data is defined, and it has two data members of integer and float type. Consider the following statements:

```

int data ::*xp = &data::x;
float data::*yp = &data::y;

```

The *xp and *yp are pointers. The class name followed by scope access operator points to the fact that they are pointers to member variables of structure int x and float y. The rest of the statement initializes the pointers with addresses of the x and y, respectively.

The rest of the program uses operators .* and ->* to initialize and access the member elements in the same fashion as the operator dot (.) and arrow (->).

13.21 Write a program to declare and initialize an array. Access and display the elements using .* and ->* operators.

```

#include<iostream.h>
#include<conio.h>

struct data
{
    int x;
    float y;
};

int main()
{
    clrscr();
    int data ::*xp=&data::x;

```

```

float data::*yp=&data::y;
data darr[]={ {12,2.5},
{58,2.4},
{15,5.7} };
for (int j=0;j<=2;j++)
cout<<endl<<darr[j].*xp<<" "<<darr[j].*yp;
return 0;
}

```

OUTPUT

```

12  2.5
58  2.4
15  5.7

```

Explanation: The above program is similar to the previous one. An array `darr[]` is initialized. The for loop and the operator `.*` access the elements, and the elements are displayed on the console.

13.22 Write a program to declare variables and pointers as members of class and access them using pointer to members.

```

#include<iostream.h>
#include<conio.h>

class data
{
public:
int x;
float y;
int *z;
float *k;
int **l;
float **m;
};

void main()
{
clrscr();
int data::*xp=&data::x;
float data::*yp=&data::y;
int *data::*zp=&data::z;
float *data::*kp=&data::k;
int **data::*lp=&data::l;
float **data::*mp=&data::m;
data ob={51,4.58,&ob.x,&ob.y,&ob.z,&ob.k};

```



```

data *pp;
pp=&ob;
cout<<endl<<ob.*xp<<endl<<ob.*yp;
cout<<endl<<*(ob.*zp)<<endl<<*(ob.*kp);
cout<<endl<<**(ob.*lp)<<endl<<**(ob.*mp);
cout<<endl<<pp->*xp<<endl<<pp->*yp;
cout<<endl<<*(pp->*zp)<<endl<<*(pp->*kp);
cout<<endl<<**(pp->*lp)<<endl<<**(pp->*mp);
*(ob.*zp)=11;
**(pp->*mp)=8.24;
cout<<endl<<ob.*xp<<endl<<ob.*yp;
}

```

OUTPUT

```

51
4.58
51
4.58
51
4.58
51
4.58
51
4.58
51
4.58
11
8.24

```

Explanation: In the above program, the class and data are declared, and all members are public. The ob is an object of the class data, and it is initialized. The variable pp is a pointer, and it is initialized with the address of the object ob. The remaining program statements use the pointers of member operators and display the contents of the class on the screen.



TIP

Pointers of members are not associated with any particular object.

13.23 Write a program that invokes member function using pointer to functions.

```

#include<iostream.h>
#include<conio.h>

```

```
class data
{
    public:
    void joy1()
    {
        cout<<endl<<(unsigned)this<<" in joy1";
    }
    void joy2()
    {
        cout<<endl<<(unsigned)this<<" in joy2";
    }
    void joy3()
    {
        cout<<endl<<(unsigned)this<<" in joy3" <<endl;
    }
};
void main()
{
    clrscr();
    data od[4];
    void (data ::*m[3])()={&data::joy1,&data::joy2,&data::joy3};
    for (int x=0;x<=3;x++)
    {
        for (int y=0;y<=2;y++)    (od[x].*m[y])();
    }
}
```

OUTPUT

```
65522 in joy1
65522 in joy2
65522 in joy3
65523 in joy1
65523 in joy2
65523 in joy3
65524 in joy1
65524 in joy2
65524 in joy3
65525 in joy1
65525 in joy2
65525 in joy3
```

Explanation: In the above program, class data have three-member variables joy1, joy2, and joy3. The array m[3] () holds the addresses of member functions. The nested for loop and statement within it invokes member functions.

13.24 Write a program to declare pointer to member variable and display the contents of the variable.

```
#include<iostream.h>
#include<conio.h>

class A
{
    public :
    int x;
    int y;
    int z;
};

void main()
{
    clrscr();
    A a;
    int *p;
    a.x=5;
    a.y=10;
    a.z=15;
    p=&a.x;
    cout<<endl<<"x= " <<*p;
    p++;
    cout<<endl<<"y= " <<*p;
    p++;
    cout<<endl<<"z= " <<*p;
}
```

OUTPUT

x= 5
y= 10
z= 15

Explanation: In the above program, the class A is declared with three public member variables x, y, and z. In function main(), a is an object of class A, and p is an integer pointer. The three-member variables are initialized with 5, 10, and 15 using the assignment operation.

The address of variable x is assigned to a pointer. The postfix increase operation gives successive memory locations, and values stored at those locations are accessed and displayed.

Thus, member variables are stored in successive memory locations. Using the same concept, we can also access private members.

13.13 ACCESSING PRIVATE MEMBERS WITH POINTERS

In the previous topic, we learned how to access public members of a class using pointers. The public and private member variables are stored in successive memory locations. The following program explains how private members can also be accessed using pointers.

13.25 Write a program to access private members like public members of the class using pointers.

```
#include<iostream.h>
#include<conio.h>

class A
{
    private:
        int j;
    public :
        int x;
        int y;
        int z;
}

A()
{
    j=20;
};

void main()
{
    clrscr();
    A a;
    int *p;
    a.x=11;
    a.y=10;
    a.z=15;
    p=&a.x;
    p--;
    cout<<endl<<"j  ="<<*p;
    p++;
    cout<<endl<<"x=  "<<*p;
    p++;
    cout<<endl<<"y=  "<<*p;
    p++;
    cout<<endl<<"z=  "<<*p;
}
```

OUTPUT

```

j = 20
x = 11
y = 10
z = 15

```

Explanation: This program is similar to the previous one. In addition, the `class A` has one private member variable. The private member variables can be accessed using functions, and no direct access is available to them. However, using pointers, we can access them as in the case of public member variables.

13.14 DIRECT ACCESS TO PRIVATE MEMBERS

So far, we have initialized private members of the class using the constructor or member function of the same class. In the previous sub-topic, we also learned how private members of the class can be accessed using pointers. In the same way, we can initialize private members and display them. One can do this by obtaining the address of any public member variable and using the address of the object. The address of the object contains the address of the first element. Programs dealing with this are given below.

13.26 Write a program to initialize the private members and display them without using member functions.

```

#include<iostream.h>
#include<conio.h>

class A
{
    private :
        int x;
        int y;
};

void main()
{
    clrscr();
    A a;
    int *p=(int*)&a;
    *p=3;
    p++;
    *p=9;
    p--;
    cout<<endl <<"x= " <<*p;
    p++;
    cout<<endl<<"y= " <<*p;
}

```

OUTPUT**x= 3****y= 9**

Explanation: In the above program, a is an object of class A. The address of the object is assigned to integer pointer p by applying type casting. The pointer p points to private member x. Integer value is assigned to *p, that is, x. Address of object a is increased and by accessing the memory location value 9 is assigned to y. The p- - statement sets the memory location of x. Using the cout statement contains of x is displayed.

13.27 Write a program to initialize and display private members using pointers.

```
#include<iostream.h>
#include<conio.h>

class A
{
    private :
        int x;
        int y;
};
class B : public A
{
    public :
        int z;
void show(int *k)
{
    cout<<"x= "<<*k << " y ="<<*(k+1) <<" z="<<*(k+2);
}
};
void main()
{
    clrscr();           // clears the screen
    B b;               // object declaration
    int *p;             // pointer declaration
    p=&b.z;             // address of z is assigned to p
    *p=3;               // initialization of z
    p--;               // points to previous location
    *p=4;               // initialization of y
    p--;               // points to previous location
    *p=5;               // initialization of x
    b.show(p);         // passing address of x to function show()
}
```

OUTPUT**x= 5 y =4 z=3**

Explanation: In the above program, `class A` has two private integer member variables `x` and `y`. The `class B` has a one-integer variable `z`, and it is derived from `class A`. In function `main()`, `b` is an object of `class B`, and `p` is a pointer. The pointer `p` is initialized with the address of public member variable `z`. The memory location of `p` is initialized with 3 (initialization of `z`). By applying a decrease operation on pointers, previous locations are accessed, and initializations of memory locations are done (initialization of `y` and `x`). The address of `x` is passed to function `show()`. By applying an increase operation with the pointer address, the successive elements are displayed.

13.15 ADDRESSES OF OBJECTS AND `void` POINTERS

The size of an object is equal to the number of total member variables declared inside the class. The size of the member function is not considered in the case of objects. The object itself contains the address of first member variables. By obtaining the address, we can access the member variables directly, irrespective of whether they are private or public. The address of the object cannot be assigned to the pointer of the same type. This is because an increase operation on pointers will set the address of the object according to its size (size of object = size of total member variables). The address of the object should be increased according to the basic data type. To overcome this situation, a `void` pointer is useful. The type of `void` pointer can be changed at run time using type-casting syntax. Thus, all the member variables can be accessed directly. Consider the following program:

13.28 Write a program to declare `void` pointers and access member variables using `void` pointers.

```
#include<iostream.h>
#include<conio.h>
#include<stdio.h>

class A
{
    protected:
        int x;
        int y;
};

class B : public A
{
    public :
        int z;
        B()
        {
            x=10; y=20; z=30;
        }
};

void main()
```

```

{
    clrscr();           // clears the screen
    B b;               // object declaration
    int j;             // declaration of integer j
    void *p=&b;         // declaration & initialization of void
                        // pointer
    for (j=0;j<3;j++)   // for loop executed for three times
        printf ("\n member variable [%d] = %d",j+1,*((int*)p+j));
    // cout<<j+1<<" " <<*((int*)p+j)<<endl;
}

```

OUTPUT

```

member variable [1] = 10
member variable [2] = 20
member variable [3] = 30

```

Explanation: In the above program, the `b` is an object of the derived class `B`. The address of the object `b` is assigned to the `void` pointer `p`. Using the `for` loop and the type-casting operation, integer values are accessed. The address of the `void` pointer is increased by two bytes. The `cout` statement now and then will not display the result accurately, and, hence, the `printf()` statement is used.

The type-casting operation is done according to the data type of member variables. Here, all the class-member variables are of integer type. In any case, the class contains variables of integer, float, and char type. Thus, it is necessary to type cast according to the sequence of the member variables.

13.16 MORE PROGRAMS

13.29 Write a program to declare the array of class objects. Use constructors and destructors. Read names and display with their length.

```

#include<iostream.h>
#include<conio.h>
#include<string.h>
#include<iomanip.h>

class word
{
    private :
        char name[20];
        int len;
    public :
        word()

```



```

{
    cout<<"\nEnter Name : ";
    cin.getline(name,20);
    len=strlen(name);
}
~word()
{
    cout<<name <<"("<<len<<")\n";
}
};
int main()
{
    clrscr();
    word w[5];
    cout<<"\nEnter names with their length \n\n";
    return 0;
}

```

OUTPUT

```

Enter Name : Sourav
Enter Name : Sachin
Enter Name : Rahul
Enter Name : Ajay
Enter Name : Ajit
Entered names with their length
Ajit(4)
Ajay(4)
Rahul(5)
Sachin(6)
Sourav(6)

```

Explanation: In the above program, the class `word` has one data member of character-type array. The class also contains a constructor and a destructor. In function `main()`, an array of five objects is defined. The constructor executes five times, and names are entered through the keyboard. The constructor also calculates the length of the entered string. The destructor displays the string and its length.

13.30 Write a program to create array of objects. Initialize and display them using non-member function.

```

#include<iostream.h>
#include<conio.h>
class A

```

```
{
    public:
        char c;
        int i;
        float f;
        A()
        {
            cout<<"\n Enter char, int and float value ";
            cin>>c>>i>>f;
        }
};
int main()
{
    void out(A);
    clrscr();
    A a[]={A(),A()};
    out(a[0]);
    out (a[1]);
    return 0;
}
void out (A c)
{
    cout<<"\n char value = "<<c.c;
    cout<<"\n int value = "<<c.i;
    cout<<"\n float value = "<<c.f;
    cout<<"\n";
}
```

OUTPUT

```
Enter char, int and float value T 2 2.5
Enter char, int and float value J 4 3.7
char value = T
int value = 2
float value = 2.5
char value = J
int value = 4
float value = 3.7
```

Explanation: In the above program, class A holds three elements c, i, and f of char, int, and float data type. The class A also has a constructor that is used for the initialization of objects. The initialization of objects is nothing but assigning value to the associated member elements of the class. In main() function array of object is declared. The constructor of the class is invoked and values to the member variables are assigned. The class A is defined before function main(). Another non-member function out() is defined for displaying the contents of the object.

Array elements are passed to the function `out()`, and contents are displayed. The `out()` function has one argument of type `class A`. The argument receives the copy of the contents of array objects.

13.31 Write a program to pass this pointer as argument to the member function and display the contents.

```
#include<iostream.h>
#include<conio.h>

class A
{
    private:
        char c;
        int i;
        float f;
    public:
        A()
        {
            cout<<"\n Enter char, int and float value ";
            cin>>c>>i>>f;
            out(this);
        }
        void out (A *a )
        {
            cout<<"\n char value = "<<a->c;
            cout<<"\n int value = "<<a->i;
            cout<<"\n float value = "<<a->f;
            cout<<"\n";
        }
};

int main()
{
    clrscr();
    A a;
    return 0;
}
```

OUTPUT

```
Enter char, int and float value F 2 2.8
char value = F
int value = 2
float value = 2.8
```

Explanation: In the above program, the pointer `this` is passed to the member function `out()`. In function `out()` using `->` operator, the contents of elements are displayed.

We know that the `this` pointer is always present in every member function and it points to the calling object.

13.32 Write a program to assign value using `this` pointer.

```
#include<iostream.h>
#include<conio.h>

class data
{
    private :
        int j;
    public:
        void setv (int v)
        {
            j=v;
            cout<<"\n j = "<<j;
            this->j=++v;
            cout<<"\n j = "<<j;
        }
};

int main()
{
    clrscr();
    data d;
    d.setv(7);
    return 0;
}
```

OUTPUT

```
j = 7
j = 8
```

Explanation: In the above program, the class `data` have only one element, that is, `int j`. The member function `setv()` is used to assign the value to the data element. The value passed to the `setv()` function is assigned to the member variable directly and using `this` pointer. Before assigning the value using `this` pointer, the variable `v` is incremented. The values of `j` in two assignments are given in the output.

13.33 Write a program to enter integer numbers, count them, and make fragments of equal lengths. The fragment length to be entered by the user.

Tip: In case the total number of elements is odd, the last segment contains the remaining elements.

```
#include<stdio.h>
#include<conio.h>
```

```
#include<iostream.h>

main()
{
    int *p,*k,s=1;
    int j,c=0,f,i=0;
    clrscr();
    k=p;
    printf ("\n Enter numbers (0) to exit :");
    while (1)
    {
        cin>>*p;
        if (*p==0) break;
        c++;
        p++;
    }
    cout<<endl<<c <<" elements are entered ";
    cout<<endl<<" Enter length of fragment : ";
    cin>>f;
    cout<<endl<<"Fragment " <<s<<" :";
    for (j=0;j<c;j++)
    {
        i++;
        cout<<" " <<*(k+j);
        if (i==f)
        {
            cout<<" ";
            s++;
            cout<<"\nFragment " <<s<<" :";
            i=0;
        }
    }
    return 0;
}
```

OUTPUT

```
Enter numbers (0) to exit :4 5 2 1 3 4 8 7 9 0
9 elements are entered
Enter length of fragment : 2
Fragment 1 : 4 5
Fragment 2 : 2 1
Fragment 3 : 3 4
Fragment 4 : 8 7
Fragment 5 : 9
```

Explanation: In the above program, pointers p, k and integer variables s, j, c, f, and i are declared. The integer variable s is initialized to one, and i is initialized to zero. The address of pointer p is assigned to k. Using while loop and pointer p, numbers are read through the keyboard. The pointer p is incremented in every iteration, and it points to successive memory locations. The numbers entered are stored in these memory locations. The if statement checks the entered number and if it is zero, control comes out of the loop. The variable c counts the number of elements entered.

The number of elements to be stored in each segment is to be entered through the keyboard. Variable f holds the number of total elements. The for loop executes from zero to c-1. The pointer k and loop variable j are used to access successive memory locations and the elements stored in them from the starting address. The statement `cout<<" "<<*(k+j)` displays elements on the screen.

13.34 Write a program to assign different types of data to void pointers. Display the data using void pointers on the screen using typecast syntaxes according to the data type.

```
#include<iostream.h>
#include<conio.h>
#define SIZE 25

void show ( void *vp, char status)
{
    switch(status)
    {
        case 'l':
            cout<<"\n The long integer entered : "<<*((long*)
vp);
            delete vp;
            break;
        case 'i':
            cout<<"\n The integer entered : "<<*((int*)vp);
            delete vp;
            break;
        case 's' :
            cout<<"The entered string : "<<(char*) vp;
            delete vp;
            break;
        case 'f':
            cout<<"\n The float entered : "<<*((float*)vp);
            delete vp;

        break;
    }
}

void main()
```

```

{
    clrscr();
    int *i;
    char *s;
    float *f;
    long *l;
    char choice;
    cout<<"\n Enter data type (s)tring, (i)nt, (f)loat (l)ong:";
    cin.get(choice);
    switch(choice)
    {
        case 'l' :
            l=new long;
            cout<<"\n Enter an long integer : ";
            cin>>*l;
            show(l,choice);
            break;
        case 's' :
            s=new char[SIZE];
            cout<<"\n Enter a string : ";
            cin>>s;
            show(s,choice);
            break;
        case 'i' :
            i=new int;
            cout<<"\n Enter an integer : ";
            cin>>*i;
            show(i,choice);
            break;
        case 'f' :
            f= new float;
            cout<<"\n Enter a float : ";
            cin>>*f;
            show (f,choice);
            break;
        default :
            cout<<"\n Invalid choice " ;
    }
}

```

OUTPUT

Enter data type (s)tring, (i)nt, (f)loat (l)ong :l
Enter an long integer : 52114
The long integer entered : 52114

Explanation: In the above program, the pointers `i`, `s`, `f`, and `l` of `int`, `char`, `float`, and `long` type are declared, respectively. The character variable `choice` is declared. The program prompts the user to enter his/her choice. The `switch()` case executes the appropriate case according to the user choice. By using new operators, memory is allocated to the respective pointers and again, the program prompts the user to enter data. The data and choice entered by the user are passed to function `show()`. The function `show()` contains void pointers. In function `show()`, the type of void pointer is changed to `char`, `long`, `int`, and `float`. The type depends on the value of variable `choice`. Thus, the entered data are displayed.

SUMMARY

- (1) Similar to C, in C++, variables are used to hold data values during program execution. When declared, every variable occupies certain memory locations.
- (2) Pointers save the memory space. The execution time with pointers is faster, because data are manipulated with the address.
- (3) The indirection operator (*) is also called the dereference operator. When a pointer is dereferenced, the value at that address stored by the pointer is retrieved.
- (4) The '&' is the address operator, and it represents the address of the variable. The address of any variable is a whole number.
- (5) Pointers can also be declared as void type. void pointers cannot be dereferenced without explicit-type conversion. This is because, being void, the compiler cannot determine the size of the object that the pointer points to.
- (6) The pointer `this` is transferred as an unseen parameter in all calls to non-static member functions. The keyword `this` is a local variable that always presents in the body of any non-static member function.
- (7) This chapter covers topics on pointer to derived classes and base class, pointer to members, accessing private members with pointers, direct access to private members, addresses of objects, and void pointers.
- (8) More programs are given in this chapter to understand the concepts on pointers.

EXERCISES

(A) Answer the following questions

- (1) What are pointers?
- (2) What is a void pointer?
- (3) What is a wild pointer?
- (4) What is a `this` pointer?
- (5) What are the features and uses of pointers?
- (6) In which situations does the pointer become harmful?
- (7) Explain any two characteristics of pointers.
- (8) How can private members be accessed using pointers?
- (9) Why is a declaration of void variables not permitted?

(B) Answer the following by selecting the appropriate option

- (1) The pointer holds
 - (a) **Address of the variable**
 - (b) Value of the variable
 - (c) Both (a) and (b)
 - (d) None of the above
- (2) An integer-type pointer can hold only the address of
 - (a) **Integer variable**
 - (b) Float variable
 - (c) Any variable
 - (d) None of the above
- (3) The address of the variable is displayed by the symbol
 - (a) **&(Ampersand)**
 - (b) * (asterisk)
 - (c) ! (not operator)
 - (d) New operator
- (4) The size of () object is equal to
 - (a) **Total size of data member variables**
 - (b) Total size of member functions
 - (c) Size of large elements
 - (d) None of the above
- (5) Private member variables can be accessed directly using
 - (a) **Pointers**
 - (b) Arrays
 - (c) `this` pointer
 - (d) None of the above
- (6) The object itself is a
 - (a) **Pointer**
 - (b) Variable
 - (c) Class member
 - (d) None of the above
- (7) The size of `void` pointers is
 - (a) 2 bytes
 - (b) **0 bytes**
 - (c) 4 bytes
 - (d) None of the above
- (8) The `this` is present in every
 - (a) Member function
 - (b) **Non-member function**
 - (c) Every object
 - (d) All of the above
- (9) In the following program, the statement `a = &(b = &x);`

```
void main()
{
    int x = 10;
    int **a, *b;
    a = &(b = &x);
    cout<<"x = "<<**a;
    cout<<"\nx = "<<*b;
}
```

 - (a) **Address of b is assigned to a**
 - (b) Address of x is assigned to a
 - (c) Pointer a points to b
 - (d) Both (a) and (c)

(C) Attempt the following programs

- (1) Write a program to declare an integer pointer. Store 10 numbers in the pointer. Use a new operator to allocate memory for 10 integers. Read and display the numbers.
- (2) Try the above program without allocating memory. Use an increment operator to get successive locations in the memory. While displaying numbers, retrieve previous memory locations using a decrement operator (Tip: The program without a new operator shows warnings; ignore it for the time being, but in practical application, a new operator is essential.).
- (3) Write a program to find the determinant of a 2×2 matrix using pointers.
B(1,1) B(1,2)
B(2,1) B(2,2)
 The determinant of the matrix is **B(1,1)*B(2,2)-B(1,2)*B(2,1)**.
- (4) Write a program to find the number of zero elements in a 3×3 matrix. Read and display the elements. Also, write this program using pointers.
- (5) Write a program to declare an integer array. Display the contents of the array using pointers.

- (6) Write a program using pointers to exchange the values of two float variables.
- (7) Write a program to calculate the square and cube of the entered number using pointers.
- (8) Write a program to display 10 floating numbers of an array using pointers.
- (9) Write a program to accept a string using character pointers and display it.
- (10) Write a program using functions and pointers to display an array of 10 integers in reverse order.
- (11) Write a program using functions and pointers to perform the addition of two one-dimensional arrays.
- (12) Write a program to perform the multiplication of two matrices (3×3) using pointers.

(D) Find bugs in the following programs

(1)

```
class set
{   int x; };

void main()
{
    set a;
    cout<<endl<<(unsigned)&a.x;
}
```

(2)

```
void main()
{
    int x = 20;
    float *p;
    p = &x;
    cout<<x;
}
```

(3)

```
void main()
{
    void y(int);
    y = 30;
    cout<<y;
}
```

(4)

```
void main()
{
    void *y;
    (int) y = 30;
    cout<<(unsigned)y;
}
```

(5)

```
void main()
{   char *x;
    int u = 10;
    x = &u;
}
```

(6)

```
void main()
{
    int x = 10;
    int *a,*b;
    a=&(b = &x);
    cout<<"x = "<<*a;
    cout<<"\nx = "<<*b;
}
```

This page is intentionally left blank.

C++ and Memory Models

14 CHAPTER

CHAPTER OUTLINE

- 14.1 Introduction
- 14.2 Memory Models
- 14.3 Dynamic Memory Allocation
- 14.4 The `new` and `delete` Operators
- 14.5 Heap Consumption
- 14.6 Overloading `new` and `delete` Operators
- 14.7 Overloading `new` and `delete` in Classes
- 14.8 Execution Sequence of Constructor and Destructor
- 14.9 Specifying Address of an Object
- 14.10 Dynamic Objects
- 14.11 Calling Conventions

14.1 INTRODUCTION

Memory is one of the critical resources of a computer system. It is limited and can also be exhausted. One should be conscious about the memory modules while an application is being developed. In this chapter, we will learn about stack, the heap, dynamic memory allocation, dynamic objects, and calling conventions.

14.2 MEMORY MODELS

The memory model sets the supportable size of code and data areas as shown in Figure 14.1. We need to specify an appropriate memory model before compiling and linking the source code. Selection of the memory model is done from the option menu of the C++ editor. The compiler option is selected, and one of the memory models is chosen. Using memory models, we can set the size limits of the data and code. C/C++ programs always use different segments for code and

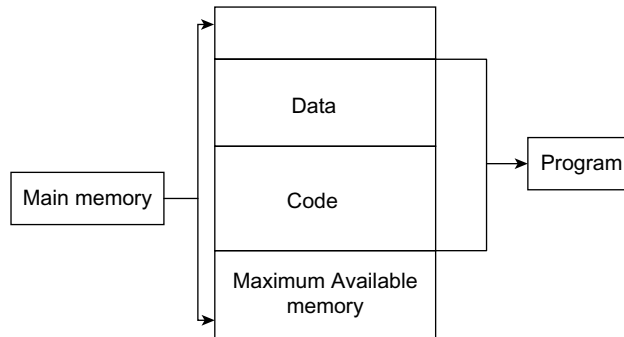


Fig. 14.1 Memory organization

data. The memory model you opt for decides the default method of memory addressing, and the default memory model is small. Table 14.1 describes the properties of all memory models.

Tiny: Use the tiny model when memory is at an absolute premium. All four segment registers (CS, DS, ES, and SS) are initialized with the same address, and all addressing is accomplished using 16 bits. Total memory capacity in this case is 64 Kbytes. In short, memory capacity is abbreviated as KB. This means that the code, data, and stack should fit within the same 64-Kbytes segment. Programs are executed quickly if this model is selected. Near pointers are always used. Tiny model programs can be converted to .COM format.

Small: All codes should fit into a single 64-KB segment, and all data should fit into a second 64-KB segment. All pointers are 16 bits in length. Execution speed is similar to that in the tiny model. This model is used for average-sized programs. Near pointers are always used.

Medium: All data should fit into a single 64-KB segment. However, the code is allowed to use multiple segments. All pointers to data are 16 bits, but all jumps and calls require 32-bit addresses. With this, the access to data is fast. However, slower program execution is observed with this model. This model is suitable for big programs that do not store a large amount of data in memory. Far pointers are used for codes but not for data.

Compact: All codes should fit into a 64-KB segment, but the data can use multiple segments. However, no data item can surpass 64 KB. All pointers to data are 32 bits, but jumps and calls can use 16-bit addresses. Slow access to data and quick code execution will be observed in this setting.

Table 14.1 Memory models

Sr.No.	Memory Model	Segment			Type of Pointer	
		Code	Data	Stack	Code	Data
1	Tiny	64 K			near	near
2	Small	64 K	64 K		near	near
3	Medium	1 MB	64 K		far	near
4	Compact	64 K	1 MB		near	far
5	Large	1 MB	1 MB		far	far
6	Huge	1 MB	64K each	64K each	far	far

The compact model is preferred if your program is small, but you require to point a large amount of data. The compact model is the opposite of the medium model. Far pointers are used for data but not for code. The code is limited to 64 K, whereas the data have a 1-MB range. All functions are near by default, and all data pointers are far by default.

Large: Both code and data are allowed to use multiple segments. All pointers are 32 bits in length. However, no single data item can exceed 64 KB. Code execution is slower.

A large model is used for very big programs only. Far pointers are used for both code and data, both of which specify a 1-MB range. All functions and data pointers are far by default.

Huge: Both code and data are allowed to use multiple segments. Every pointer is 32 bits in length and it is different from large memory model only by pointer math and not by segments. Because of this huge model, code execution is slowest.

Use the Huge model for very big programs only. Far pointers are used for both code and data. Turbo C++ usually bounds the size of all data to 64K. The Huge memory model sets aside that limit, permitting data to hold more than 64K. This model permits multiple data segments (each 64K in size), up to 1 MB for code, and 64K for stack. All functions and data pointers are supposed to be far.

Segment and offset address

Every address has two parts: segment and offset. We can separate these address parts using the following two macros defined in dos.h header file.

FP_SEG() – This macro is used to obtain the segment address of the given pointer variable.

FP_OFF() – This macro is used to obtain the offset address of the given pointer variable.

The following programs illustrate the working of both these macros.

14.1 Write a program to obtain segment and offset address.

```
#include<dos.h>
#include<iostream.h>
#include<constream.h>

int main()
{
    clrscr();
    int ch;
    cout<<"\n Complete Address of ch:"<<&ch;
    cout<<"\n Segment address:"<<hex<<FP_SEG(&ch) ;
    cout<<"\n Offset address:"<<hex<<FP_OFF(&ch) ;
    return 0;
}
```

OUTPUT

Complete Address of ch : 0x8f34fff2

Segment address : 8f34

Offset address : fff2

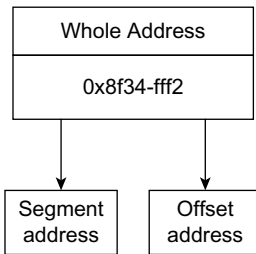


Fig. 14.2 Segment and offset addresses

Explanation: In the above program, variable `ch` is declared. The first `cout` statement displays the whole address of the variable `ch`, that is, `0x8f34fff2`. The macro `FP_SEG()` returns the segment address. The statement used is `FP_SEG(&ch)`. Here, the address of `ch` is passed to the macro. In the same way, the offset address is obtained using the statement `FP_OFF(&ch)`. You can observe the complete address as displayed above. The separated addresses are also similar. Figure 14.2 makes this point clearer.

`far`, `huge`, and `near` are keywords. They are summarized in Table 14.2.

Table 14.2 Keywords summary

Keyword	Data	Code	Pointer Arithmetic
<code>near</code>	16-bit address	16-bit address	16-bit
<code>far</code>	32-bit address	32-bit address	16-bit
<code>huge</code>	32-bit address	Keyword not applicable	32-bit

far pointer: A far pointer is a 32-bit pointer that contains both segment and offset address parts.

huge pointer: A huge pointer is 32 bits long, and it contains both segment and offset address parts.

near pointer: A near pointer is 16 bits long. It uses the contents of the CS or DS register for the segment part. The offset part of the address is stored in 16 bits near the pointer.

14.2 Write a program to declare far, near and huge pointer. Display their sizes.

```

#include<iostream.h>
#include<constream.h>

int main()
{
    clrscr();
    char far *f; // far pointer declaration
    char near *n; // near pointer declaration
    char huge *h; // huge pointer declaration
    cout<<"\n Size of far pointer:"<<sizeof(f);
    cout<<"\n Size of near pointer:"<<sizeof(n);
    cout<<"\n Size of huge pointer:"<<sizeof(h);
    return 0;
}
  
```

OUTPUT

Size of far pointer: 4

Size of near pointer: 2

Size of huge pointer: 4

Explanation: In the above program, the pointers `f`, `n`, and `h` are declared. In the pointer declaration, the pointers are preceded by the keywords `far`, `near`, and `huge`. The `sizeof()` operators display the size of the pointers in bytes.

14.3 Write a program to use far pointer.

```
#include<iostream.h>
#include<constream.h>

int main()
{
    clrscr();
    char far *s; // pointer declaration
    s=(char far*)0xB8000000L; // starting address
    *s='W'; // displays character on the screen
    return 0;
}
```

OUTPUT

W

Explanation: Character pointer `*s`, which is a far pointer, is declared. The address `0xB8000000L` is assigned to it. It is a starting address outside the data segment. The address is converted into `char far*` type by applying type casting. The statement `*s='W'` displays the character 'W' on the screen.

14.3 DYNAMIC MEMORY ALLOCATION

For the execution of the program, it is essential to bring the program into the main memory. When a program does not fit into the main memory, parts of it are brought into the main memory one by one, and program gets executed. Of course, the parts of the program that are not currently in the main memory are resident at the secondary memory, such as a floppy or a hard disk or any other magnetic disk. When a new segment of a program is to be moved into a full main memory, it should replace another segment that is already resident in the main memory. These are replacement policies, and their description is beyond the scope of this book. Hence, the programmer should not be concerned with the method of transfer of the program from secondary to primary memory or replacement policies.

The technique by which a program can obtain storage space in the main memory at run time is called *dynamic allocation*. In this method, the space for the program is allocated from the free space during the execution of the program. Memory requested by the program is allocated by the system from the memory heap. The free region of the memory is called the *heap*. The heap changes depending on the memory model used. Conceptually, the C++ programs are stored in the free space from the heap. Overall, 8086 memory models are described in this chapter. The amount of memory requirement is decided by how the program is designed. For example, if a program is developed with many recursive functions, then this is implemented with a stack.

The next section describes the C++ operators `new` and `delete` that can be used for achieving dynamic memory allocation and deallocation, respectively.

14.4 THE `new` AND `delete` OPERATORS

So far, we have used the `new` and `delete` operators in short programs, but for applying them in huge applications, we need to understand them completely. A small mistake in syntax may cause a critical error and possibly, a corrupt memory heap. Before beginning a study on the memory heap in detail, let us repeat a few points regarding `new` and `delete` operators.

- (1) The `new` operator not only creates an object but also allocates memory.
- (2) The `new` operator allocates memory from the heap that is also called a `free store`.
- (3) The object created and the memory allocated by using the `new` operator should be deleted and memory should be released by the `delete` operator; otherwise, such a mismatch operation may corrupt the heap or may crash the system. According to the ANSI standard, it is a valid outcome for this invalid operation, and the compiler should have routines to handle such errors.
- (4) The `delete` operator not only destroys objects but also releases allocated memory.
- (5) The `new` operator creates an object, and it remains in the memory until it is released using the `delete` operator. Sometimes, the object deleted using the `delete` operator remains in the memory.
- (6) If we send a null pointer to the `delete` operator, it is secure. Using `delete` to zero has no result.
- (7) The statement `delete x` does not destroy the pointer `x`. It destroys the object associated with it.
- (8) Do not apply C functions such as `malloc()`, `realloc()`, or `free()` with `new` and `delete` operators. These functions are unfit for object-oriented techniques (Table 14.3).
- (9) Do not destroy the pointer repetitively or more than once. First time, the object is destroyed and memory is released. If second time the same object is deleted, the object is sent to the destructor, and, no doubt, it will corrupt the heap.
- (10) If the object created is not deleted, it occupies the memory unnecessarily. It is a good habit to destroy the object and release the system resources.

Table 14.3 Difference between `new` and `malloc()`

<code>new</code> Operator	<code>malloc()</code>
Creates objects	Allocates memory
Returns pointer of relevant type	Returns <code>void</code> pointer
It is possible to overload a <code>new</code> operator	<code>malloc()</code> cannot be overloaded

14.4 Write a program to allocate memory to store 3 integers. Use `new` and `delete` operators for allocating and de-allocating memory. Initialize and display the values.

```
#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    int i, *p;
```

```

    p=&i;
    p=new int[3];
    *p=2; // first element
    *(p+1)=3; // second element
    *(p+2)=4; // third element
    cout<<"Value Address";
    for (int x=0;x<3;x++)
    cout<<endl<<* (p+x) <<"\t" <<(unsigned) (p+x) ;
    delete []p;
    return 0;
}

```

OUTPUT

Value Address

2	3350
3	3352
4	3354

Explanation: In the above program, integer variable `i` and pointer `*p` are declared. The pointer `p` is initialized with the address of variable `i`, and by using the `new` operator memory for three integers is allocated. The pointer `*p` can hold three integers in successive memory locations. The pointer variable is initialized with numerical values. The `for` loop is used to display the contents of the pointer `*p`. The `delete` operator releases the memory.

14.5 Write a program to allocate memory to store 3 floats. Use new and delete operators for allocating and de-allocating memory. Initialize and display the values.

```

#include<iostream.h>
#include<conio.h>

int main()
{
    clrscr();
    float i, *p;
    p=&i;
    p=new float[3];
    for (int x=0;x<3;x++)
    *(p+x)=x+3.11;
    cout<<"Value Address";
    for ( x=0;x<3;x++)
    cout<<endl<<* (p+x) <<"\t" <<(unsigned) (p+x) ;
    delete []p;
    return 0;
}

```

OUTPUT

Value	Address
3.11	4698
4.11	4702
5.11	4706

Explanation: In this program, the memory for three float numbers is allocated to the float pointer p. The first for loop initializes the pointer p, and the second for loop displays the values on the screen. The delete operator releases the memory.

14.6 Write a program to allocate memory for two objects. Initialize and display the contents and de-allocate the memory.

```
#include<iostream.h>
#include<conio.h>

struct boy
{
    char *name;
    int age;
};

int main()
{
    clrscr();
    boy *p;
    p=new boy[2];
    p->name="Rahul";
    p->age=20;
    (p+1)->name="Raj";
    (p+1)->age=21;
    for (int x=0;x<2;x++)
    {
        cout<<"\nName:"<<(p+x)->name<<endl<<"Age:"<<(p+x)->age;
    }
    delete []p;
    return 0;
}
```

OUTPUT

Name : Rahul
Age : 20
Name : Raj
Age : 21

Explanation: In the above program, structure `boy` is declared, and memory for two objects is allocated to the pointer `p`. The pointer `p` is initialized, and the first `for` loop displays the contents of the pointer on the screen. Finally, the `delete` operator deallocates the memory.

14.7 Write a program to use new and delete operators to create and destroy objects.

```
#include<iostream.h>
#include<string.h>
#include<conio.h>

class student
{
    private :
        char name[25];
        int age;
    public :
        student()
        {
            strcpy(name, "");
            age=0;
        }
        student (char *s,int g)
        {
            strcpy (name,s);
            age=g;
        }
        void assign (char *s, int g)
        {
            strcpy(name,s);
            age=g;
        }
        void display()
        {
            cout<<endl<<name<<"\t" <<age;
        }
        ~student()
        { cout<<endl<<"In destructor"; }
};

void main()
{
    clrscr();
    student *s;
    s=new student ;
    s->assign ("santosh",24);
```



```

    student *t;
    t=new student ("Amit",15);
    s->display();
    t->display();
    delete s;
    delete t;
}

```

OUTPUT

```

santosh 24
Amit 15
In destructor
In destructor

```

Explanation: In the above program, the class `student` has two data members of character and integer type. The class also has zero-argument, two-argument constructors, and member function `display()`. The constructor initializes the object, and the `display()` function displays data on the screen.

In function `main()`, `s` and `t` are pointers to class `student` and using the `new` operator, memory is allocated and constructors are executed. The objects are initialized, and contents are displayed by calling `display()` function. The `delete` operator destroys the objects, and `destructor()` is executed after this.

14.5 HEAP CONSUMPTION

The heap is used to allocate memory during program execution, that is, run time. In assembly language, there is no such thing as a heap. All memory is for the programmer, and he/she can use it directly. In various ways, C/C++ is a better programming environment than assembly language. However, a cost has to be paid to use either C or C++. The cost is separation from the machine. We cannot use memory anywhere; we need to ask for it. The memory from which we receive information is called the heap.

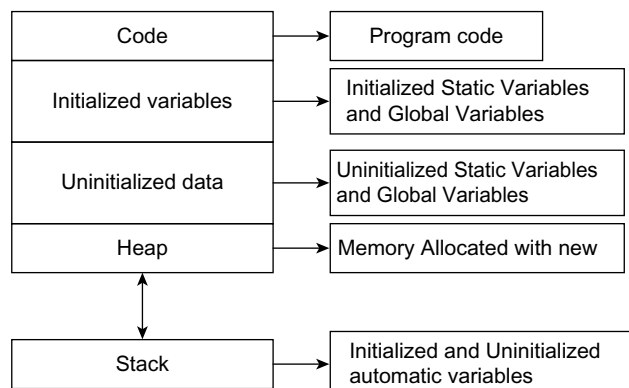


Fig. 14.3 Heap review

The C/C++ compiler may place automatic variables on the stack. It may store static variables earlier than loading the program. The heap is the piece of memory that we allocate. Figure 14.3 shows a view of the heap.

Local variables are stored in the stack, and the code is in code space. The local variables are destroyed when the function returns. Global variables are stored in the data area, and they are accessible by all functions. The heap can be considered a huge section of memory in which large numbers of memory locations are placed sequentially.

As stated earlier, all local variables are stored in the stack. As soon as the function execution is completed, local variables are destroyed, and the stack becomes empty. The heap is not cleared until the program execution is completed. It is the user's task to free the memory. The memory allocated from the heap remains available until the user explicitly deallocates it.

While solving problems related to memory allocation, we always believe that large memory is available, and the heap is at no time short of memory. It is bad programming to depend on such guesses, which may cause an error in application. In C, the function `malloc()` is used to allocate the memory, and if this function fails to allocate the memory, it returns NULL pointer. By checking the return value of the function `malloc()`, failure or success of the memory allocation is tested, and appropriate sub-routines are executed.

C++ allows us to apply similar logic with `new` operators. C++ allows us two function pointers known as `_new_handler` and `set_new_handler`. The `_new_handler` holds a pointer to a function. It requires no arguments and returns void.

If a `new` operator fails to allocate the memory requested, it will invoke the function `*_new_handler` and again, it will attempt the memory allocation. By default, the function `*_new_handler` directly closes the program. It is also possible to substitute this handler with a function that releases memory. This can be directly accomplished by executing the function `set_new_handler` that returns a pointer to the handler.

14.8 Write a program to use `set_new_handler` function.

```
#include<iostream.h>
#include<new.h>
#include<stdlib.h>
#include<conio.h>

void m_warn()
{
    cerr << "\n Cannot allocate!";
    exit(1);
}

int main()
{
    clrscr();
    set_new_handler(m_warn);
    char *k=new char[50];
    cout<<"\n First allocation:k=" << hex << long(k);
    k=new char[64000U];
    cout<<"\n Second allocation:k=" << hex << long(k);
```

```

    set_new_handler(0); // Reset to default.
    return 0;
}

```

OUTPUT

First allocation: k = 8fa40d48
Cannot allocate!

Explanation: In the above program, the `set_new_handler` is initialized with the function `m_warn()`. The `m_wanr()` displays a warning message. The variable `p` is a character pointer and using the `new` operator memory for 50 characters, it is allocated to the pointer `p`. The `cout` statement displays the starting address of the memory allocated.

Consider the statement `p = new char [64000U]`. In this statement, the `new` operator attempts to allocate memory to the pointer `p`. In case the `new` operator fails to allocate the memory, it calls the `setnew_handler`, and `m_warn()` function is executed.

14.6 OVERLOADING `new` AND `delete` OPERATORS

In C++, at any time when memory allocation and deallocation is concerned, the `new` and `delete` operators are used. These operators are invoked from the compiler's library functions. They are a part of C++ language and are very effective. Similar to other operators, the `new` and `delete` operators are overloaded. The following programs illustrate this:

14.9 Write a program to overload `new` and `delete` operator.

```

#include<iostream.h>
#include<stdlib.h>
#include<new.h>
#include<conio.h>

void main()
{
    clrscr();
    void warning();
    void *operator new (size_t, int);
    void operator delete (void*);
    char *t=new ('#') char [10];
    cout<<endl<<"First allocation:p="<<(unsigned) long(t)<<endl;
    for (int k=0;k<10;k++)
        cout<<t[k];
    delete t;
    t=new ('*') char [64000u];
    delete t;
}

void warning()

```

```

{
    cout<<"\n insufficient memory";
    exit(1);
}
void *operator new (size_t os, int setv)
{
    void *t;
    t=malloc(os);
    if (t==NULL) warning();
    memset(t, setv, os);
    return (t);
}
void operator delete(void *ss) {free(ss);}

```

OUTPUT

First allocation : p=3376

#####

Insufficient memory

Explanation: In the above program, the new and delete operators are overloaded. The size_t is used to determine the size of the object. The new operator calls the warning() function when the malloc() function returns NULL.

Consider the statement `t = new ('*') char [64000u]`. When memory allocation is requested by this statement, the new operator fails to allocate the memory and calls the function warning(). When called, the delete operator releases the memory using the free() function. Internally, in this program, malloc() and free() functions are used to allocate and deallocate the memory.

14.10 Write a program to overload new and delete operator.

```

#include<iostream.h>
#include<stdlib.h>
#include<new.h>
#include<conio.h>

void main()
{
    clrscr();
    void warning();
    void *operator new (size_t);
    void operator delete (void*);
    char *t;
    t=new char[5];
    cout<<endl<<"First allocation:p="<<(unsigned) long(t)<<endl;

```



```

    for (int k=0;k<10;k++)
    {
        *(t+k)='-';
        cout<<*(t+k);
    }
    delete t;
    t=new char [64000U];
    delete t;
}

void warning()
{
    cout<<"\nInsufficient memory";
    exit(1);
}

void *operator new (size_t os)
{
    void *t;
    t=malloc(os);
    if (t==NULL)
        warning();
    return (t);
}

void operator delete(void *ss) {free(ss);}

```

OUTPUT

First allocation: p=3374

Insufficient memory

Explanation: The above program is similar to the previous one. In overloading of the new operator, only one argument is used. In the previous examples, the overloaded new and delete operators use the malloc() and free() function to allocate and deallocate memory. The same can be done with new and delete operators. The following programs illustrate the overloading of new and delete operators:

14.11 Write a program to overload new and delete operator. Read five numbers and find largest out of them.

```

#include<iostream.h>
#include<conio.h>
const int NUM=5;

class number
{
    private :

```

```
    int *series;
    public :
void *operator new (size_t)
{
    number *num;
    num=::new number;
    num->series=new int [NUM];
    return num;
}
void operator delete ( void* n)
{
    number *num;
    num=(number *) n;
    delete (int *) num->series;
    ::delete n;
}
void input();
int large (void);
};
void number ::input()
{
    for (int i=0;i<NUM;i++)
    {
        cout<<"Number["<<i+1<<"]="";
        cin>>series[i];
    }
}
int number ::large()
{
    int l=0;
    for (int i=0;i<NUM;i++)
    {
        if (l<series[i])
            l=series[i];
    }
    return l;
}
void main()
{
    clrscr();
    number *l_num=new number;
    cout<<"Enter numbers"<<endl;
    l_num->input();
```

```

    cout<<"Largest Number:"<<l_num->large();
    delete l_num;
}

```

OUTPUT

```

Enter numbers
Number[ 1] = 4
Number[ 2] = 7
Number[ 3] = 8
Number[ 4] = 2
Number[ 5] = 3
Largest Number : 8

```

Explanation: In the above program, the `new` and `delete` operators are overloaded. The `new` operator creates objects as well as allocates data members. The statement `number *l_num = new number` executes the overloaded `new` operator. The statement `num = ::new number` and `::delete n` creates and destroys objects respectively. The `number::new` and `number::delete` are recursively executed. The statement `num = ::new number` creates an object of the `number` class. If the scope access operator is omitted, the overloaded operator is executed recursively and results in a stack overflow.

If we prefix the scope access operator to the `new` operator, it will invoke the standard `new` operator built-in language instead of a defined one. The class `number` has a data member variable `*series`. The statement `num->series = new int [NUM]` creates an array of `NUM` elements and allocates memory. It invokes the `::new` operator.

Similarly, the overloaded `delete` operator releases the memory allocated by the `new` operator. It executes the overloaded operator function `void operator delete (void* n)`.

14.7 OVERLOADING `new` AND `delete` IN CLASSES

The `new` and `delete` operators can be overloaded in a similar manner to other operators. It is interesting to observe their working when they are overloaded for multiple classes. The following program explains the working of overloaded `new` and `delete` operators in multiple classes:

14.12 Write a program to overload `new` and `delete` operators

```

#include<iostream.h>
#include<stdlib.h>
#include<string.h>
#include<conio.h>
#include<new.h>
#define T_BYTES 2

void warning()
{
    cout<<endl<<"Cannot allocate memory";
}

```

```
        exit(1);
    }
class boy
{
    private :
    char name [20];
    int age;
    float height;
    public :
    void *operator new (size_t);
    void operator delete (void *t);
    void assign(char *nm, int a, float h);
    void display();
    ~boy();
};
struct limit
{
    boy o;
    int position ;
};
int signal=0;
struct limit *l=0;
void *boy ::operator new (size_t s)
{
    int j;
    if (signal==0)
    {
        l=(limit*) malloc(s * T_BYTES);
        if (l==NULL)    warning();
        for (j=0;j<T_BYTES;j++)
            l[j].position=0;
        signal=1;
        l[0].position=1;
        return &l[0].o;
    }
else
    {
        for (j=0;j<T_BYTES;j++)
        {
            if (l[j].position==0)
            {
                l[j].position=1;
                return &l[j].o;
            }
        }
    }
}
```

```

        }
        warning();
    }
    return NULL;
}
void boy ::operator delete ( void *t)
{
    if (t==NULL) return ;
    for (int j=0;j<T_BYTES;j++)
    {
        if (t==&l[j].o)
        {
            l[j].position=0;
            strcpy(l[j].o.name,"");
            l[j].o.age=0;
            l[j].o.height=0.0;
        }
    }
}
void boy::assign (char *nm, int a, float h)
{
    strcpy (name,nm);
    age=a;
    height=h;
}
void boy::display()
{
    cout<<endl<<name<<"\t"<<age<<"\t"<<height;
}
boy::~~boy()
{
    free(l);
    cout<<endl<<"In Destructor (Memory released)";
}
void main()
{
    clrscr();
    void warning();
    set_new_handler (warning);
    boy *b1,*b2,*b3;
    b1=new boy;
    b1->assign("Sanjay",20,4.5);
    b2=new boy;
    b2->assign("Vijay",22,5.1);
}

```

```

    b1->display();
    b2->display();
    delete b1;
    delete b2;
    b1->display();
    b2->display();
    b1=new boy;
    b2=new boy;
    b3=new boy;
    b3->display();
}

```

OUTPUT

Sanjay 20 4.5

Vijay 22 5.1

In Destructor (Memory released)

In Destructor (Memory released)

0 0

0 0

Cannot allocate memory

Explanation: In the above program, `new` and `delete` operators are overloaded class by class. The macro `T_BYTES` is initialized with the number of records for which the memory is to be occupied. The member function `assign()` is used to initialize the data members of the objects. The `display()` function is used to display the contents on the screen. The `warning()` function displays a warning message when called. It is associated with `set_new_handler()` and executed when memory allocation fails. The `class boy` is declared with three member variables.

In function `main()`, `*b1`, `*b2`, and `*b3` are pointers to structure and by using the `new` operator, memory is allocated to them. Consider the statement `l = (limit*) malloc (s * T_BYTES);` here, the memory is allocated only for two records, and, hence, when we try to allocate memory for the third object, the function `warning()` is executed and it prompts the message **“Cannot allocate memory.”** The `delete` operator is used to release the memory allocated. The statements `delete b1` and `delete b2` release the memory allocated by these objects. After releasing the memory, if each function calls the `display()` function, the contents displayed of these objects is zero. This is obviously the effect of the `delete` operator.

14.8 EXECUTION SEQUENCE OF CONSTRUCTOR AND DESTRUCTOR

Overloaded `new` and `delete` operators functioning within the class are always static. We know that the static function can be invoked without specifying the object. Hence, `this` pointer is absent in the body of static functions. The compiler invokes the overloaded `new` operator and allocates memory before executing the constructor. The compiler also invokes the overloaded `delete` operator function and deallocates memory after execution of the destructor. The following program makes the concept clearer:

14.13 Write a program to display the sequence of execution of constructor and destructor in classes when operators new and delete are overloaded.

```
#include<iostream.h>
#include<stdlib.h>
#include<string.h>
#include<conio.h>

class boy
{
    char name[10];
public :
    void *operator new (size_t );
    void operator delete (void *q);
    boy();
    ~boy();
};

char limit[sizeof(boy)];
boy::boy()
{
    cout<<endl<<"In Constructor";
}

boy::~~boy()
{ cout<<endl<<"In Destructor"; }

void *boy ::operator new (size_t s )
{
    cout<<endl<<"In boy ::new operator";
    return limit;
}

void boy::operator delete (void *q)
{ cout<<endl<<"In boy ::delete operator"; }

void main()
{
    clrscr();
    boy *e1;
    e1=new boy;
    delete e1;
}
```

OUTPUT

In boy ::new operator
In Constructor
In Destructor
In boy ::delete operator

Explanation: In this program, the `new` and `delete` operators are overloaded. The class also has a constructor and a destructor. The overloaded `new` operator function is executed first followed by the class constructor. We know that the constructor is always used to initialize members. The constructor also allocates memory by calling the `new` operator implicitly. Here, the `new` operator is overloaded. As soon as control of the program reaches the constructor, before executing any statement in the constructor it invokes the overloaded `new` operator.

Similarly, when an object goes out of scope, the destructor is executed. The destructor invokes the overloaded `delete` operator to release the memory allocated by the `new` operator.

14.9 SPECIFYING ADDRESS OF AN OBJECT

The compiler assigns an address to the object created. The programmer has no power over the address of the object. However, it is possible to create an object at the memory location given by the program explicitly. The address specified should be big enough to hold the object. The object should be created in such a way that it must be destroyed by invoking the destructor explicitly. The following program makes the concept clearer:

14.14 Write a program to create object at given memory address.

```
#include<iostream.h>
#include<constream.h>

class data
{
    int j;
public:
    data ( int k) {j=k;}
    ~data() { }
    void *operator new ( size_t, void *u)
    {
        return (data *)u;
    }
    void show() {cout<<"j="<<j;}
};

void main()
{
    clrscr();
    void *add;
    add=(void*)0x420;
    data *t=new (add) data(10);
    cout<<"\n Address of object:"<<t;
    cout<<endl;
    t->show();
    t->data::~data();
}
```


OUTPUT**Address of object : 0x8f800420****j=10**

Explanation: In the above program, the new operator is overloaded. The void pointer *add is declared and initialized with address 0x420. The pointer object *p is declared, and an address is assigned to it. The new operator is also invoked to allocate memory. In the same statement, the constructor is also invoked, and a value is passed to it. The member function show() displays the value of a member variable. Finally, the statement t->data::~~data(); invokes the destructor to destroy the object. We can confirm the address of the object by displaying its address. The address of the object would be the same as the specified one.

14.10 DYNAMIC OBJECTS

As discussed earlier, C++ supports dynamic memory allocation/deallocation. C++ allocates memory and initializes the member variables. An object can be created at run time; such an object is called a **dynamic object**. The construction and destruction of the dynamic object is explicitly done by the programmer. The new and delete operators are used to allocate and deallocate memory to such objects. A dynamic object can be created using the new operator as follows:

```
ptr = new classname;
```

The new operator returns the address of the object created, and it is stored in the pointer ptr. The variable ptr is a pointer object of the same class. The member variable of the object can be accessed using the pointer and -> (arrow) operator. A dynamic object can be destroyed using the delete operator as follows:

```
delete ptr;
```

The delete operator destroys the object pointed by the pointer ptr. It also invokes the destructor of a class. The following program explains the creation and destruction of dynamic objects:

14.15 Write a program to create dynamic object.

```
#include<iostream.h>
#include<constream.h>

class data
{
    int x,y;
public:
    data()
    {
        cout<<"\n Constructor";
        x=10;
        y=50;
    }
    ~data() {cout<<"\n Destructor";}
```

```

void display()
{
    cout<<"\n x="<<x;
    cout<<"\n y="<<y;
}
};
void main()
{
    clrscr();
    data *d; // declaration of object pointer
    d=new data; // dynamic object
    d->display();
    delete d; // deleting dynamic object
}

```

OUTPUT

Constructor

x=10

y=50

Destructor

Explanation: The d is a pointer object. The statement `d = new data;` creates an anonymous object and assign its address to the pointer d. Such an object is called a **dynamic object**. The constructor is executed when the dynamic object is created. The statement `d->display();` invokes the member function, and contents of the members are displayed. The statement `delete d;` destroys the object by releasing the memory and invoking the destructor.

14.11 CALLING CONVENTION

Calling convention means how parameters are pushed on the stack when a function is invoked. Table 14.4 describes the calling convention method of a few languages.

Table 14.4 Calling Conventions

Language	Order	Parameter Passed
Basic	In order (left to right)	By address
Fortran	In order (left to right)	By address
C	In reverse order (right to left)	By value and address
C++	In reverse order (right to left)	By value, address, and reference
Pascal	In order (left to right)	As values

In C/C++, parameters are passed from right to left; whereas in other languages such as basic, FORTRAN calling convention is in order from left to right. The following program explains the calling convention of C++:

14.16 Write a program to demonstrate calling convention of C++.

```

#include<iostream.h>
#include<constream.h>

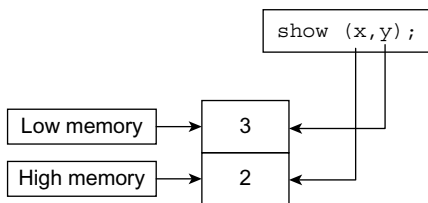
void main()
{
    clrscr();
    void show (int,int);
    int x=2,y=3;
    show (x,y);
}

void show (int x, int y)
{
    cout<<x;
    cout<<endl;
    cout<<y;
}

```

OUTPUT

2
3

**Fig. 14.4** Stack with arguments

Explanation: In this program, integer variables x and y are initialized to two and three, respectively. The `show()` function `show()` is invoked, and x and y are passed as per the statement `show (x, y)`. The parameters are passed from right to left. The variable y is passed first followed by x . The values of variables with their position in the stack are displayed in Figure 14.4.

The value of the rightmost variable is pushed first followed by other variables in sequence.

SUMMARY

- (1) The memory model sets the supportable size of code and data areas.
- (2) The object created and the memory allocated by using the `new` operator should be deleted, and the memory should be released by the `delete` operator; otherwise, such a mismatch operation may corrupt

- the heap or may result in a system crash. According to the ANSI standard, it is a valid outcome for this invalid operation, and the compiler should have routines to handle such errors.
- (3) The heap is used to allocate memory during program execution, that is, run time.

The C/C++ compiler may place automatic variables on the stack. It may store static variables before loading the program. The heap is the piece of memory that we allocate.

- (4) In C++, at any time when memory allocation and deallocation is concerned, the `new` and `delete` operators are used. These operators are a part of C++ language and are very effective. Similar to other operators, the `new` and `delete` operators are also overloaded.
- (5) When a constructor and a destructor are executed, `new` and `delete` operators

are invoked. When these operators are overloaded, the compiler invokes these overloaded `new` and `delete` operators.

- (6) An object can be created at run time; such an object is called a `dynamic object`. The construction and destruction of a dynamic object is explicitly done by the programmer.
- (7) `Calling convention` means how parameters are pushed on the stack when a function is invoked. In C/C++, parameters are passed from right to left; whereas in basic, FORTRAN parameters are passed from left to right.

EXERCISES

(A) Answer the following questions

- (1) What are the different types of memory models?
- (2) Explain the properties of `new` and `delete` operators.
- (3) What do you mean by a heap? Explain it in detail.
- (4) Explain overloading of `new` and `delete` operators.
- (5) What do you mean by a dynamic object? How are such objects created?
- (6) What do you mean by calling conventions?
- (7) Explain the process of calling convention in C/C++.
- (8) Explain the use of `set_new_handler`.
- (9) Explain the differences between the operator `new` and `malloc()` function.
- (10) What are segment and offset addresses?
- (11) Explain the use of macros `FP_SEG()` and `FP_OFF()`.

(B) Answer the following by selecting the appropriate option

- (1) The dynamic objects are created
 - (a) **at run time**
 - (b) compile time
 - (c) both (a) and (b)
 - (d) none of the above
- (2) The `new` operator is used to
 - (a) **allocate memory**
 - (b) deallocate memory
 - (c) delete objects
 - (d) none of the above
- (3) The `delete` operator is used to
 - (a) allocate memory
 - (b) **deallocate memory**
 - (c) create objects
 - (d) none of the above
- (4) In C/c++, when a function is called, parameters are passed
 - (a) **from right to left**
 - (b) from left to right
 - (c) from top to bottom
 - (d) none of the above
- (5) In c++, it is possible to pass values to functions by
 - (a) call by value
 - (b) call by address
 - (c) call by reference
 - (d) **all of the above**

(C) Attempt the following programs

- (1) Write a program to create a dynamic object.
- (2) Write a program to allocate memory for 10 integers to a pointer. Assign and display 10 integers. Destroy the object as well.
- (3) Write a program to invoke a function with parameters. How are parameters passed? Are they passed from right to left or from left to right? Explain.
- (4) Write a program to create an object at a specified memory location.

Binding, Polymorphisms, and Virtual Functions

15 CHAPTER

CHAPTER OUTLINE

- 15.1 Introduction
- 15.2 Binding in C++
- 15.3 Pointer to Base and Derived Class Objects
- 15.4 Virtual Functions
- 15.5 Rules for Virtual Functions
- 15.6 Array of Pointers
- 15.7 Pure Virtual Functions
- 15.8 Abstract Classes
- 15.9 Working of Virtual Functions
- 15.10 Virtual Functions in Derived Classes
- 15.11 Object Slicing
- 15.12 Constructors and Virtual Functions
- 15.13 Virtual Destructors
- 15.14 Destructors and Virtual Functions

15.1 INTRODUCTION

One of the key features offered by C++ is polymorphisms. It is essential to know the concepts of polymorphisms and associated topics. In this chapter, we are going to learn and implement static (early) binding, dynamic (late) binding, polymorphisms, and virtual functions. An attempt is made to illustrate every point of this new topic in an easy way, and complicated subtopics are explained in a simple way.

The word *poly* means many, and *morphism* means several forms. Both the words are derived from Greek language. Thus, by combining these two words, a new whole word called

polymorphism is created, which means various forms. We have learnt about overloading of functions and operators. It is also one type of polymorphism. The information pertaining to various overloaded member functions and arguments is known to the compiler while compiling. Thus, the compiler selects an appropriate function and collects the arguments at compile time. This is called *early binding* or *static binding*. This is also known as *compile time polymorphism*.

In C++, the function can be bound at either compile time or run time. Deciding a function call at compile time is called *compile time* or *early* or *static binding*. Deciding a function call at run time is called *run time* or *late* or *dynamic binding*. Dynamic binding permits to suspend the decision of choosing a suitable member function until run time. Two types of polymorphism are shown in Figure 15.1.

POLYMORPHISMS

A polymorphism is a technique in which various forms of a single function can be defined and shared by various objects to perform an operation.

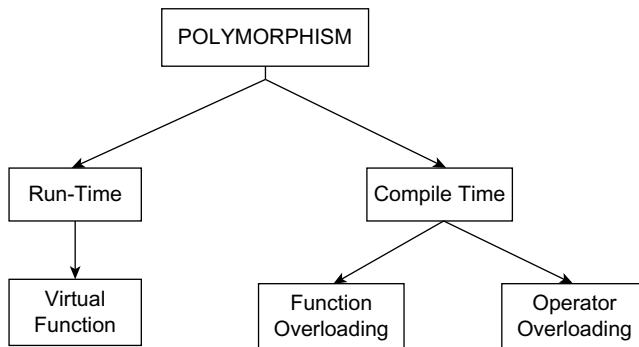


Fig. 15.1 Polymorphisms in C++

15.2 BINDING IN C++

Though C++ is an object-oriented programming language, it is very much inspired by procedural language. A program in C++ is executed sequentially line by line. Each line of the source program after translation is in the form of machine language. Each line in the machine language is assigned a unique address. Similarly, when a function call is encountered by the compiler during execution, the function call is translated into machine language, and a sequential address is provided. Thus, *binding* refers to the process that is to be used for converting functions and variables into machine language addresses. The C++ supports two types of binding: static or early binding and dynamic or late binding.

15.2.1 Static (Early) Binding

In this section, we are going to focus on static or early binding. Even though similar function names are used at many places, their references and their positions are indicated explicitly by the compiler. Their ambiguities are fixed at compile time. Consider the following example.

```

class first // base class
{
    int d;
    public:
        void display() {-----} // base class member
                                function
};
class second : public first // derived class
{
    int k;
    public:
        void display() {-----} // member function of
derived class
}

```

In the above program, base and derived classes have a similar member function name `display()` but after conversion into machine language, addresses for two `display()` functions are different and unique. Thus, the compiler instructs the CPU to jump to the different addresses when similar `display()` function calls are executed at various places in the program. This is not an overloaded function, because its prototype is the same and it is defined in different classes.

The following program is based on early binding:

15.1 Write a program to invoke function using scope access operator.

```

#include<iostream.h>
#include<conio.h>

class first
{
    int b;
    public:
        first() {b=10;}
        void display() {cout<<"\n b="<<b;}
};

class second: public first
{
    int d;
    public:
        second() {d=20;}
        void display() {cout<<"\n d="<<d;}
};

int main()
{
    clrscr();
}

```



```

    second s;
    s.first::display();    // Invokes base class function
    s.display();           // Invokes derived class function
    return 0;
}

```

OUTPUT

```

b = 10
d = 20

```

Explanation: In the above program, the class `first` is a base class and class `second` is a derived class. Both the classes contain one integer data member and member function. The `display()` function is used to display the content of the data member. Both the classes contain a similar function name. In function `main()`, `s` is an object of the derived class `second`. Hence, in order to invoke the `display()` function of the base class, the scope access operator is used. When base and derived classes have similar function names, in such a situation, it is very essential to provide information to the compiler at run time about the member functions. The mechanism that provides run-time selection of a function is called a polymorphism.

A `virtual` keyword plays an important role in late binding. Before introducing virtual functions, let us have a look at the given below program. The following program shows what happens when functions are not declared `virtual`. This is an example of early binding.

15.2 Write a program to invoke member function of base and derived class using pointer of base class.

```

#include<iostream.h>
#include<conio.h>

class first
{
    int b;
public:
    first() {b=10;}
    void display() {cout<<"\n b=" <<b;}
};

class second: public first
{
    int d;
public:
    second() {d=20;}
    void display() {cout<<"\n d=" <<d;}
};

int main()
{
    clrscr();
    first f,*p;

```

```
    second s;  
    p=&f;  
    p->display();  
    p=&s;  
    p->display();  
    return 0;  
}
```

OUTPUT

```
b = 10  
b = 10
```

Explanation: In the above program, the class `first` is a base class and class `second` is a derived class. The variable `f` is an object of the base class, and `s` is an object of the derived class. The pointer `*p` is an object pointer of the base class. The address of the base class object is assigned to pointer `p`, and `display()` function is called. Again, the address of the derived class is assigned to pointer `p`, and `display()` function is called. Both the times, the `display()` function of base class is executed. This is so, because though in the second call `p` contains the address of the object of the derived class, yet the `display()` function of the base class replaces the existence of the `display()` function of the derived class. In order to execute various forms of the same function defined in the base, derived class, run-time binding is necessary, and it can be achieved using the `virtual` keyword.

15.2.2 Dynamic (Late) Binding

In the case of a few programs, it is impossible to know which function is to be called until run time. This is called dynamic binding. Dynamic binding of member functions in C++ can be done using the `virtual` keyword. The member function followed by the `virtual` keyword is called a virtual function. Consider the following example:

```
class first // base class  
{  
    int d;  
    public:  
    virtual void display() {-----} // base class member  
function  
};  
class second: public first // derived class  
{  
    int k;  
    public:  
    virtual void display() {-----} // member function  
of derived class  
}
```

In the example of the above program, base and derived classes have a similar member function `display()` preceded by the keyword `virtual`. The various forms of virtual functions in base and derived classes are dynamically bound. The references are detected from the base class. All virtual functions in derived classes are considered virtual, supposing they match the base class function exactly in the number and types of parameters sent. If there is no match between these functions, they will be assumed as overloaded functions. The virtual function should be defined in the public section. If the function is declared virtual, the system will use dynamic (late) binding, which is carried out at run time. Otherwise, early or compile-time binding is used.

Dynamic binding can be implemented with function pointers. In this method, the pointer points to a function instead of a variable. A simple programming example is given below based on late binding.

15.3 Write a program to perform few arithmetic operations on floating numbers using functions. Use function pointer (Late binding).



```
#include<iostream.h>
#include<conio.h>

float add(float m, float n)
{
    return m+n;
}

float sub(float m, float n)
{
    return m-n;
}

float mul(float m, float n)
{
    return m*n;
}

int main()
{
    clrscr();
    float x,y;
    cout<<"Enter two numbers:";
    cin>>x>>y;
    int task;
    do
    {
        cout<<"Enter task (1=add, 2=sub, 3=mul):";
        cin>>task;
    }
    while (task< 1 || task > 3);
```

```
float (*pt)(float, float);  
// Function pointer named pt is created  
switch (task)  
{  
case 1:  
pt=add;  
break;  
case 2:  
pt=sub;  
break;  
case 3:  
pt=mul;  
break;  
}  
cout<<"Result of operation is:" << pt(x,y) << endl;  
return 0;  
}
```

OUTPUT

Enter two numbers: 2.5 2.5

Enter task (1=add, 2=sub, 3=mul): 3

Result of operation is : 6.25

Explanation: In the above program, instead of calling functions directly, we have called them through function pointer `pt`. The compiler is unable to use static or early binding in this case. In this program, the late binding concept is implemented. The compiler has to read the addresses held in the pointers toward different functions. Until run time, decisions are not taken as to which function needs to be executed; hence, it is late binding.

15.3 POINTER TO BASE AND DERIVED CLASS OBJECTS

In inheritance, the properties of existing classes are extended to the new classes. The new classes that can be created from the existing base class are called as derived classes. The inheritance provides the hierarchical organization of classes. It also provides the hierarchical relationship between two objects and indicates the shared properties between them. All derived classes inherit properties from the common base class. Pointers can be declared to the point base or derived class. Pointers to objects of the base class are type compatible with pointers to objects of the derived class. A base class pointer can point to objects of both the base and derived class. In other words, a pointer to the object of the base class can point to the object of the derived class; whereas a pointer to the object of the derived class cannot point to the object of the base class, as shown in Figure 15.2.

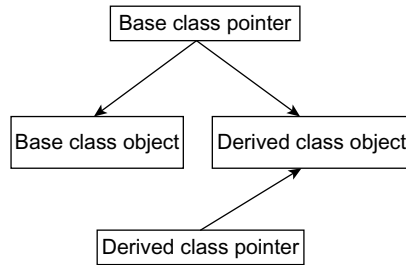


Fig. 15.2 Type compatibility of base and derived class pointers

15.4 Write a program to access members of base and derived class using pointer objects of both classes.

```

#include<iostream.h>
#include<constream.h>

class W
{
    protected:
        int w;
    public:
        W (int k) { w=k; }
        void show()
        {
            cout<<"\n In base class W";
            cout<<"\n W="<<w; };
};

class X: public W
{
    protected:
        int x;
    public:
        X (int j, int k): W( j)
        {
            x=k;
        }
        void show()
        {
            cout<<"\n In class X";
            cout<<"\n w="<<w;
            cout<<"\n x="<<x;
        }
};
  
```

```

};
class Y: public X
{
    public:
    int y;
};
void main()
{
    clrscr();
    W *b;
    b = new W(20);    // pointer to class W
    b->show();
    delete b;
    b = new X(5,2);   // pointer to class X
    b->show();
    ((X*)b)->show();
    delete b;
    X x(3,4);
    X *d=&x;
    d->show();
}

```

OUTPUT

In base class W

W=20

In base class W

W=5

In class X

w=5

x=2

In class X

w=3

x=4

Explanation: In the above program, the class W is a base class. The X is derived from W, and class Y is derived from class X. Here, the type of inheritance is multilevel inheritance. The variable *b is a pointer object of the class W. The statement `b = new W (20) ;` creates a nameless object and returns its address to the pointer b. The `show ()` function is invoked by the pointer object b. Here, the `show ()` function of base class W is invoked. Using **delete** operator, the pointer b is deleted.

The statement `b = new X (5, 2) ;` creates a nameless object of class X and assigns its address to the base class pointer b. Here, it should be noted that the object b is a pointer object of

the base class, and it is initialized with the address of the derived class object. Again, the pointer `b` invokes the function `show()` of the base class and not the function of class `X` (derived class.). To invoke the function of the derived class `X`, the following statement is used:

```
((X*)b) ->show();
```

In the above statement, typecasting (upcasting) is used. The upcasting forces the object of class `W` to behave as if it were the object of class `X`. This time, the function `show()` of class `X` (derived class) is invoked. The process of obtaining the address of a derived class object, and treating it as the address of a base class object is known as *upcasting*.

The statement `X x(3, 4);` creates object `x` of class `X`. The statement `X *d = &x;`

declares the pointer object `d` of the derived class `X` and assigns the address of `x` to it. The pointer object `d` invokes the derived class function `show()`. Figure 15.3 shows a pictorial representation of the above explanation.

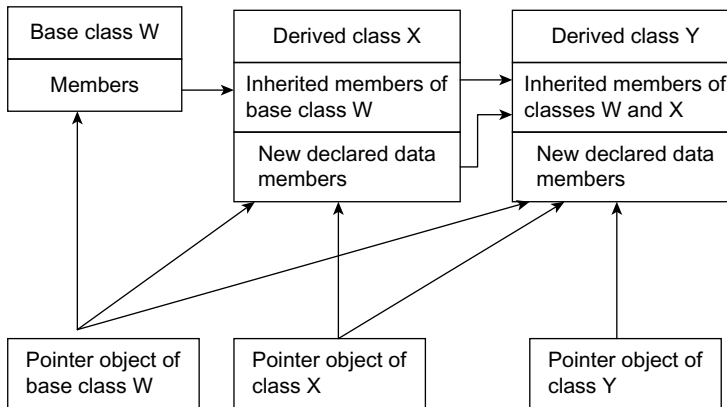


Fig. 15.3 Type compatibility of base and derived class pointers

Here, the pointer of class `W` points to its own class as well as its derived class. The pointer of derived class `X` can point to its own class but cannot point to its base class.

15.4 VIRTUAL FUNCTIONS

Virtual functions of the base class should be redefined in the derived classes. The programmer can define a virtual function in a base class, and can then use the same function name in any derived class, even if the number and type of arguments are matching. The matching function overrides the base class function of a similar name. Virtual functions can only be member functions. We can also declare the functions as given below:

```
int Base::get(int) and int Derived::get(int) even when they are not virtual.
```

The base class version is available to derived class objects via scope override. If they are virtual, only the function associated with the actual type of the object is available. With virtual functions, we cannot alter just the function type. It is illegal, therefore, to redefine a virtual function so that

it varies only in the return type. If two functions with a similar name have different arguments, C++ compiler considers them different, and the virtual function mechanism is dropped.

15.5 RULES FOR VIRTUAL FUNCTIONS

- (1) The virtual function should not be static and must be a member of a class.
- (2) The virtual function may be declared as a friend for another class. An object pointer can access the virtual functions.
- (3) A constructor cannot be declared as virtual, but a destructor can be declared as virtual.
- (4) The virtual function should be defined in the public section of the class. It is also possible to define the virtual function outside the class. In such a case, the declaration is done inside the class, and the definition is outside the class. The `virtual` keyword is used in the declaration and not in the function declarator.
- (5) It is also possible to return a value from virtual functions similar to other functions.
- (6) The prototype of the virtual function in the base class and derived class should be exactly the same. In case of a mismatch, the compiler neglects the virtual function mechanism and treats these functions as overloaded functions.
- (7) Arithmetic operations cannot be used with base class pointers.
- (8) If a base class contains a virtual function and if the same function is not redefined in the derived classes, in such a case, the base class function is invoked.
- (9) The operator keyword used for operator overloading also supports the `virtual` mechanism.

15.5 Write a program to declare virtual function and execute the same function defined in the base and derived class.

```
#include<iostream.h>
#include<conio.h>

class first
{
    int b;
    public:
    first() {b=10;}
    virtual void display() {cout<<"\n b=" <<b;}
};

class second : public first
{
    int d;
    public:
    second() {d=20;}
    void display() {cout<<"\n d="<<d;}
};

int main()
{
    clrscr();
```



```

    first f,*p;
    second s;
    p=&f;
    p->display();
    p=&s;
    p->display();
    return 0;
}

```

OUTPUT

```

b = 10
d = 20

```

Explanation: The above program is similar to the previous one. The only difference is that a virtual keyword precedes the display() function of the base class as per the statement virtual void display() {cout<<"\n b = " <<b;}. The virtual keyword does the run-time binding. In the first call, the display() function of the base class is executed and in the second call, that is, after assigning the address of the derived class to pointer p, displays() function of the derived class is executed.

15.6 Write a program to use pointer for both base and derived class and call the member function. Use virtual keyword.

VIRTUAL FUNCTIONS

```

#include<iostream.h>
#include<conio.h>
class super
{
    public:
        virtual void display() {cout<<"\n In function display() class
super";}
        virtual void show() {cout<<"\n In function show() class su-
per";}
}
;
class sub : public super
{
    public:
        void display() {cout<<"\nIn function display() class sub";}
        void show() {cout<<"\n In function show() class sub";}
};
int main()
{

```

```

    clrscr();
    super S;
    sub A;
    super *point;
    cout<<"\n Pointer point points to class super\n";
    point=&S;
    point->display();
    point->show();
    cout<<"\n\n Now Pointer point points to derived class sub\n";
    point=&A;
    point->display();
    point->show();
    return 0;
}

```

OUTPUT

Pointer point points to class super

In function display() class super

In function show() class super

Now Pointer point points to derived class sub

In function display() class sub

In function show() class sub

Explanation: In the above program, the base class `super` and the derived class `sub` have member functions a similar name. They are `display()` and `show()`. In function `main()`, the variable `S` is an object of class `super`, and the variable `A` is an object of derived class `sub`. The pointer variable `point` is a pointer to the base class. The address of object `S` is assigned to the pointer `S`. The pointer calls both the member functions. Similarly, the variable `A` is an object of the derived class `sub`. The address of `A` is assigned to the pointer `point` and again, the pointer calls the member functions.

The member functions of the base class are preceded by the keyword `virtual`. If the `virtual` keyword is removed, in both the function calls, the member functions of the base class are executed. The member functions for the derived class are not executed, though the pointer has the address of the derived class. If the `virtual` keyword is not removed, firstly, the member function of the base class is executed, and then member function of derived class.

15.6 ARRAY OF POINTERS

Polymorphism refers to late or dynamic binding; that is, the selection of an entity is decided at run time. In class hierarchy, methods with similar names can be defined, which perform different tasks, and then, the selection of the appropriate method is done using dynamic binding. Dynamic binding is associated with object pointers. Thus, addresses of different objects can be stored in an array to invoke functions dynamically. The following program explains this concept:

15.7 Write a program to create array of pointers. Invoke functions using array objects.

```
#include<iostream.h>
#include<constream.h>

class A
{
    public:
    virtual void show()
    {
        cout<<"A\n";
    }
};

class B : public A
{
    public:
    void show() {cout<<"B\n";}
};

class C : public A
{
    public:
    void show()
    {cout<<"C\n";
    }
};

class D : public A
{
    public:
    void show()
    {
        cout<<"D\n";
    }
};

class E : public A
{
    public:
    void show()
    {
        cout<<"E";
    }
};

void main()
{
    clrscr();
    A a;
```

```

B b;
C c;
D d;
E e;
A *pa[]={&a,&b,&c,&d,&e};
for ( int j=0;j<5;j++)
    pa[j]->show();
}

```

OUTPUT

A
B
C
D
E

Explanation: In the above program, class A is a base class. The classes B, C, D, and E are classes derived from class A. All these classes have a similar function `show()`. In function `main()`, `a`, `b`, `c`, `d`, and `e` are objects of classes A, B, C, D, and E, respectively. The function `show()` of the base class is declared virtual. An array of pointer `*pa` is declared, and it is initialized with addresses of the base and derived class objects; that is, `a`, `b`, `c`, `d`, and `e`. Using `for` loop and array, each object invokes function `show()`. The output is as shown above. If the base class function `show()` is non-virtual, then the very time function `show()` of base class is executed. Figure 15.4 illustrates this concept more clearly.

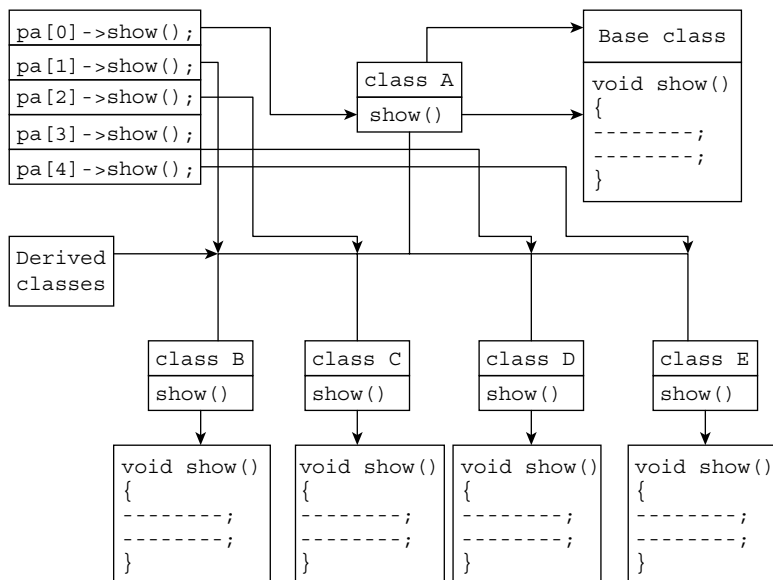


Fig. 15.4 Early and late binding of functions

15.7 PURE VIRTUAL FUNCTIONS

In practical applications, the member function of the base class is rarely used for doing any operation; such functions are called do-nothing functions, dummy functions, or pure virtual functions. The do-nothing functions or pure functions are always virtual functions. Usually, pure virtual functions are defined with a null body. This is so, because derived classes should be able to override them. Any normal function cannot be declared as a pure function. After the declaration of a pure function in a class, the class becomes an abstract class. It cannot be used to declare any object. Any attempt to declare an object will result in the error “cannot create instance of abstract class.” The pure function can be declared as follows:

Declaration of pure virtual function

```
virtual void display() =0; // pure function
```

In the above declaration of the function, the `display()` is a pure virtual function. The assignment operator is not used to assign zero to this function. It is used just to instruct the compiler that the function is a pure virtual function and that it will not have a definition.

A pure virtual function declared in the base class cannot be used for any operation. The class containing the pure virtual function cannot be used to declare objects. Such classes are known as abstract classes or pure abstract classes. Anyone who attempts to declare an object from the abstract class would be reported an error message by the compiler. In addition, the compiler will display the name of the virtual function present in the base class. The classes derived from the pure abstract classes are required to re-declare the pure virtual function. All other derived classes without pure virtual functions are called concrete classes. The concrete classes can be used to create objects. A pure virtual function is similar to an unfilled container that the derived class is made to fill.

15.8 Write a program to declare pure virtual functions.

```
#include<iostream.h>
#include<conio.h>

class first
{
    protected:
        int b;
    public:
        first() {b=10;}
        virtual void display() =0; // pure function
};

class second: public first
{
    int d;
    public:
        second() {d=20;}
};
```

```

    void display() {cout<<"\n b="<<b <<" d="<<d;}
};
int main()
{
    clrscr();
    first *p;
    // p->display // abnormal program termination
    second s;
    p=&s;
    p->display();
    return 0;
}

```

OUTPUT

b= 10 d = 20

Explanation: In the above program, the `display()` function of the base class is declared a pure function. The pointer object `*p` holds the address of the object of the derived class and invokes the function `display()` of the derived class. Here, the function `display()` of the base class does nothing. If we try to invoke the pure function using the statement `p->display()` as per the given remarks in the above program (`//p->display//abnormalprogram termination`), the program is terminated with the error “abnormal program termination.”

15.8 ABSTRACT CLASSES

Abstract classes are similar to a skeleton on which new classes are designed to assemble a well-designed class hierarchy. The set of well-tested abstract classes can be used, and the programmer only extends them. Abstract classes containing virtual functions can be used to help in program debugging. When various programmers work on the same project, it is necessary to create a common abstract base class for them. The programmers are restricted to create a new base class.

The development of such software can be demonstrated by creating a header file. The file `abstract.h` is an example of a file containing an abstract base class that is used for debugging purpose. Contents of the file `abstract.h` are as follows:

Contents of abstract.h header file

```

#include<iostream.h>
struct debug
{
    virtual void show()
    {
        cout<<"\n No function show() defined for this class";
    }
};

```

15.9 Write program to use abstract class for program debugging.

```

#include "abstract.h"
#include <constream.h>

class A : public debug
{
    int a;
public:

    A(int j=0) {a=j;
}
void show() {cout<<"\nIn class A a="<a;}
};

class B : public debug
{
    int b;
public:
    B (int k ) {b=k;}
};

void main()
{
    clrscr();
    A a(1);
    B b(5);
    a.show();
    b.show();
}

```

OUTPUT**In class A a=1****No function show() defined for this class**

Explanation: Observe the contents of file abstract.h. The struct debug contains the virtual function show(). This function is declared in the file abstract.h and the same is inserted using #include directive in the above program. The classes A and B are derived from class debug, which is defined in the header file abstract.h. In function, a and b are the objects of classes A and B, respectively. The statement a.show(); invokes the function show(), and the value of a is displayed. The object b also invokes the function show(). However, the class B does not have the function show(). Hence, the function show() of an abstract base class is executed, which displays a warning message.

Traditional languages do not provide such facilities. An abstract class develops into a dominant and powerful interface when the software system undergoes various changes. It is essential

to confirm that the debugging interface is accurately constructed. If changes are made in the actual project, it is compulsory to add appropriate methods to the abstract base class. In case the programmer needs to define a function `warn()` to the class `debug`, then the header file can be updated. The contents of the file would be as follows:

Contents of abstract.h header file
<pre>#include<iostream.h> struct debug { virtual void show() { cout<<"\n No function show() defined for this class"; } }; virtual void warn() { cout<<"\n No function warn() defined for this class"; } };</pre>

While defining such an abstract class, the following points should be kept in mind:

- (1) Do not declare an object of abstract class type.
- (2) An abstract class can be used as a base class.
- (3) The derived class should not have pure virtual functions. Objects of the derived class can be declared.

15.9 WORKING OF VIRTUAL FUNCTIONS

Before learning about the mechanism of virtual functions, let us revise a few points related to virtual functions:

- (1) Binding means a link between a function call and the real function that is executed when the function is called.
- (2) When a compiler knows which function to call before execution, it is known as early binding.
- (3) Dynamic binding means the actual function invoked at run time is dependent on the address stored in the pointer. In this binding, a link between function call and actual function is made during program execution.
- (4) The keyword `virtual` prevents the compiler from performing early binding. Binding is postponed until program execution.

The following programs illustrate the step-by-step working of virtual functions:

15.10 Write a program to define virtual and non-virtual functions and determine the size of the objects.

```
#include<iostream.h>
#include<conio.h>

class A
{
    private:
        int j;
    public:
        virtual void show() {cout<<endl<<"In A class";}
};

class B
{
    private:
        int j;
    public:
        void show() {cout<<endl<<"in B class";}
};

class C
{
    public:
        void show() {cout<<endl<<"In C class";}
};

void main()
{
    clrscr();
    A x;
    B y;
    C z;
    cout<<endl<<"Size of x="<<sizeof (x);
    cout<<endl<<"Size of y="<<sizeof (y);
    cout<<endl<<"Size of z="<<sizeof (z);
}
```

OUTPUT

Size of x = 4

Size of y = 2

Size of z = 1

Explanation: In the above program, the class A has only one data element of integer type, but the size of the object displayed is 4. The size of the object of class B that contains a one-integer data element is two, and finally, the size of the object of class C is displayed as one, even if the class C has no data element.

The function `show()` of class A is prefixed by the `virtual` keyword. Without the `virtual` keyword, the size of objects would be 2, 2 and 1. The size of object x with a virtual function in class A is the addition of data member `int` (2 bytes) and void pointers (2 bytes). When a function is declared as `virtual`, the compiler inserts a void pointer. In class C, even if it has no object, the size of the object displayed is 1, and this is due to the compiler, who assumes the size of object z not to be zero, as every object should have an individual address. Hence, minimum size one is considered. The minimum nonzero positive integer is one.

To perform late binding, the compiler establishes VTABLE (virtual table) for every class and its derived classes having a virtual function. The VTABLE contains addresses of the virtual functions. The compiler puts the address of the virtual functions in the VTABLE. If no function is redefined in the derived class that is defined as `virtual` in the base class, the compiler takes the address of the base class function.

When an object of the base or derived class is created, a void pointer is inserted in the VTABLE, called `VPTR` (vpointer). The `VPTR` points to the VTABLE. When a virtual function is invoked, using the base class pointer, the compiler speedily puts a code to obtain the `VPTR` and searches for the address of the function in the VTABLE. In this way, an appropriate function is invoked, and dynamic binding takes place.

The `VPTR` should be initialized with the beginning address of the apposite VTABLE. When the `VPTR` is initialized with the apposite VTABLE, the type of the object can be determined by itself. However, it is useless if is applied at the point when a virtual function is invoked.

Assume a base class pointer object points to the object of a derived class. If a function is invoked using the base class pointer, the compiler uses a different code to accomplish the function call. The compiler begins from the base class pointer. The base class pointer holds the address of the derived class object. With the aid of this address, the `VPTR` of the derived class is obtained. Via `VPTR`, the VTABLE of the derived class is obtained. In the VTABLE, the address of the function being invoked is acquired and the function is called. All the above processes are handled by the compiler, and the user need not worry about them. The following program makes the concept clearer:

15.11 Write a program to define virtual member functions in derived classes.

```
#include<iostream.h>
#include<conio.h>

class shop
{
    public:
    virtual void area() {cout<<endl<<"In area of shop";}
    virtual void rent() {cout<<endl<<"In rent of shop";}
```

```
    void period() {cout<<endl<<"In period of shop";}
};
class shopA : public shop
{
    public:
    void area() {cout<<endl<<"In area of shopA";}
    void rent() {cout<<endl<<"In rent of shopA";}
};
class shopB : public shop
{
    public:
    void area() {cout<<endl<<"In area of shopB";}
    void rent() {cout<<endl<<"In rent of shopB";}
    void period() {cout<<endl<<"In period of shopB";}
};
class shopC: public shop
{
    void area() {cout<<endl<<"In area of shopC";}
};
void main()
{
    clrscr();
    shop *p;
    shop s;
    p=&s;
    p->area();
    p->rent();
    p->period();
    shop *sa,*sb,*sc;
    shopA a;
    shopB b;
    shopC c;
    sa=&a;
    sb=&b;
    sc=&c;
    sa->area();
    sa->rent();
    sb->area();
    sb->rent();
    sc->area();
    sc->rent();
    sa->period();
```

```

    e.area();
    sb->period();
    shop d;
    d.area();
    shopA e;
    shopC f;
    f.rent();
}

```

OUTPUT

In area of shop
 In rent of shop
 In period of shop
 In area of shopA
 In rent of shopA
 In area of shopB
 In rent of shopB
 In area of shopC
 In rent of shop
 In period of shop
 In period of shop
 In area of shop
 In area of shopA
 In rent of shop

Explanation: In the above program, four classes are declared, as shown in Figure 15.5.

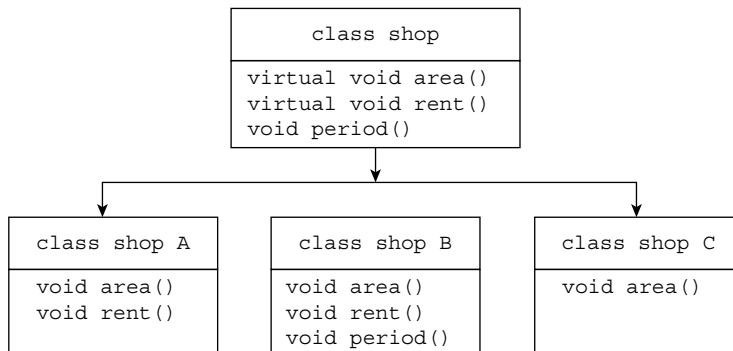


Fig. 15.5 Base and derived classes

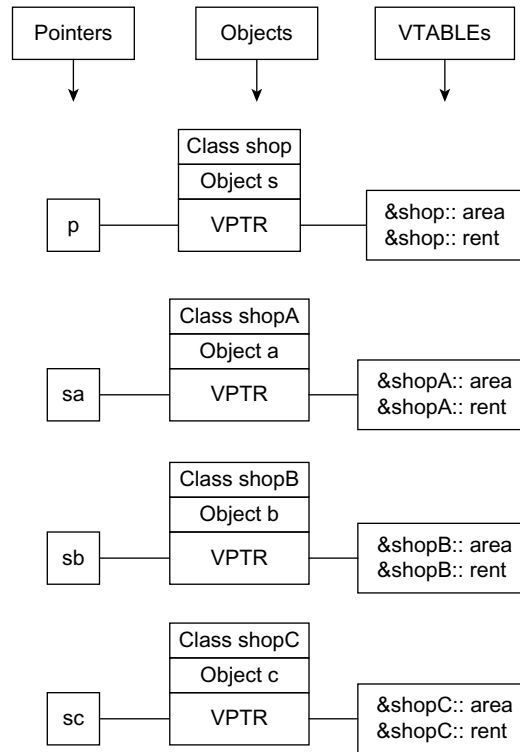


Fig. 15.6 VPTR AND VTABLES

As discussed earlier, a VTABLE is formed for each class having a virtual function and for the derived class of the same class. The VTABLE is formed for the following classes: shop, shopA, shopB, and shopC. All of the above four VTABLES hold the address of virtual functions. In addition, the compiler would place a VPTR that points to the particular VTABLE, as shown in Figure 15.6.

The class shopC is without functions rent (). Therefore, the VTABLE holds the address of the base class rent () function. Consider the following statements:

```
shop *p; // Base class object pointer
shop s; // object of base class
p = &s; // Address of s is assigned to p
```

The pointer p contains the address of the object s. Now, consider the following statements:

```
p->area(); // Invokes function area() of base class
p->rent(); // Invokes function rent() of base class
p->period(); // Invokes function period() of base class
```

From the above functions, first two function area () and rent () are virtual, and period () is a non-virtual function. Though the address of the base class object or derived class object is stored in the base class pointer, the function of the base class is invoked, because the period () is a non-virtual function. The member function area () is declared as virtual in the base class. All the three derived classes shopA, shopB, and shopC contain the function area (). The execu-

tion of these functions depends on the address stored in the base class pointer. For example, `p` contains the address of object `a`; the functions of class `shopA` are invoked. Similarly, for storing addresses of objects `b` and `c`, functions of class `B` and class `C` can be invoked.

```
p = &s; // Address of s is assigned to p
```

In the above statement, `p` contains the address of the object `s` (base class). Therefore, when the function `area()` is invoked by the pointer `p`, `VPTR` is created from the object `s`. With the help of `VPTR`, `VTABLE` of the class `shop` is obtained, and the address of the function `area()` for class `shop` is accessed. With the aid of an address, `shop::area()` is finally invoked. Consider the following statements:

```
shop *sa; // object pointer declaration
shopA a; // object of derived class
sa = &a; // assigns address of derived class object to base class pointer
sa->area(); // invokes function area()
sa->rent(); // invokes function rent()
```

In the statement `sa = &a;` the address of the derived class object is stored in the base class pointer. When the function `area()` is invoked, the `VPTR` of object `a` is used to obtain the `VTABLE` of class `shopA`. From this `VTABLE`, the address of `shopA::area()` and `shopA::rent` are obtained and finally invoked. Likewise, for objects `b` and `c`, binding is achieved.

Consider the following statements:

```
sa->period();
sb->period();
```

Function `period()` is a non-virtual function. The `VTABLE` is not used to call the function `shop::period()`. In addition, the function `period()` is not redefined in the derived classes. Now, concentrate on the following statements:

```
shop d; // Base class object
d.area();
shopA e; // Derived class object
e.area();
shopC f; // Derived class object
f.rent();
```

In the above statements, dynamic binding is not performed, and, hence, `VPTR` and `VTABLE` are not created. The functions are called as usual.

C++ places addresses of the virtual member function in the virtual table. When these member functions are called, the accurate address is obtained from the virtual table. This entire procedure takes time. Therefore, the virtual function makes program execution a bit slow. Conversely, it provides better flexibility.

15.10 VIRTUAL FUNCTIONS IN DERIVED CLASSES

We know that when functions are declared virtual in the base class, it is mandatory to redefine virtual functions in the derived class. The compiler creates `VTABLES` for the derived class and stores addresses of functions in it. In case the virtual function is not redefined in the derived class, the `VTABLE` of the derived class contains the address of the base class virtual function. Thus, the

VTABLE contains addresses of all functions. Thus, it is not possible for a function to exist but for its address to not be present in the VTABLE. Consider the following program, which shows the possibility of a function existing but an address not being located in the VTABLE:

15.12 Write a program to redefine a virtual base class function in the derived class. Also, add a new member in the derived class. Observe the VTABLE.

```
#include<iostream.h>
#include<conio.h>

class A
{
    public:
    virtual void joy()
    {
        cout<<endl<<"In joy of class A";
    }
};

class B: public A
{
    public:
    void joy() {cout<<endl<<"In joy of class B";}
    void virtual joy2()
    {
        cout<<endl<<"In joy2 of class B";
    }
};

void main()
{
    clrscr();
    A *a1,*a2;
    A a3;
    B b;
    a1=&a3;
    a2=&b;
    a1->joy();
    a2->joy();
    // a2->joy2(); // joy2 is not member of class A
}
```

OUTPUT

In joy of class A

In joy of class B

Explanation: In the above program, the base class A contains one virtual function `joy()`. In the derived class B, the function `joy()` is redefined and defines a new virtual function `joy2()`. Figure 15.7 shows VTABLES created for the base and derived class.

```
a2->joy2();
```

The above statement will generate an error message. In the above statement, the pointer `a2` is treated as only a pointer to the base class object. The function `joy2()` is not a member of base class A. Hence, it is not allowed to invoke the function `joy2()` using the pointer object `a2`. However, an address of the derived class is assigned to the base class pointer, and the compiler has no way to determine that we are working with a derived class object. The compiler avoids calling virtual functions present only in derived classes. In a hierarchical class organization of various levels, if it is essential to invoke a function at any level by using a base class pointer, then the function should be declared virtual in the base class.

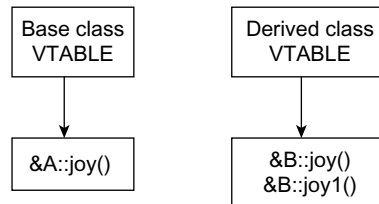


Fig. 15.7 VTABLES for base and derived class

15.11 OBJECT SLICING

Virtual functions permit us to manipulate both base and derived objects using similar member functions with no modifications. Virtual functions can be invoked using a pointer or reference. If we do so, object slicing takes place. The following program takes you to the real thing:

15.13 Write a program to declare reference to object and invoke functions.

```
#include<constream.h>
#include<iostream.h>

class B
{
    int j;
public:
    B (int jj) {j=jj;}
    virtual void joy()
    {
        cout<<"\n In class B";
        cout<<endl<<" j="<<j;
    }
};

class D : public B
{
    int k;
public:
    D (int jj, int kk) : B (jj)
```



```

    {k=kk;}
    void joy()
    {
        B::joy();
        cout<<"\n In class D";
        cout<<endl<<" k="<<k;
    }
};
void main()
{
    clrscr();
    B b(3);
    D d (4,5);
    B &r=d;
    cout<<"\n Using Object";
    d.joy();
    cout<<"\n Using Reference";
    r.joy();
}

```

OUTPUT

Using Object

In class B

j= 4

In class D

k= 5

Using Reference

In class B

j= 4

In class D

k= 5

Explanation: In the above program, a reference object r is created to an object d using the statement `B &r = d;`. The member function `joy()` is invoked using r and d objects. The output is similar.

15.14 Write a program to demonstrate object slicing.

```

#include<iostream.h>
#include<constream.h>

class A
{
    public:
    int a;

```

```

    A() {a=10;}
};
class B: public A
{
    public:
    int b;
    B() {a=40; b=30;}
};
void main()
{
    clrscr();
    A x;
    B y;
    cout<<" a"<<x.a <<" ";
    x=y;
    cout<<" Now a"<<x.a;
}

```

OUTPUT**a=10 now a=40**

Explanation: In the above program, class A has only one data member a and derived class B has one member b. In function `main()`, objects x and y of classes A and B are declared. The object x has only one member, that is, a and the object y has two members a and b. The statement `x = y`, that is, the derived class object, is assigned to the base class object. In such an assignment, only base, class part of the derived object is assigned to the base class object. Thus, if an object of a derived class is assigned to a base class object, the compiler allows it. However, it copies only the base class members of the object, and this process is known as **object slicing**.

15.12 CONSTRUCTORS AND VIRTUAL FUNCTIONS

It is possible to invoke a virtual function using a constructor. A constructor makes the virtual mechanism illegal. When a virtual function is invoked through a constructor, the base class virtual function will not be called; instead, the member function of a similar class is invoked.

15.15 Write a program to call virtual function through constructor.

```

#include<iostream.h>
#include<constream.h>

class B
{
    int k;
    public:
    B (int l) {k=l;}
}

```

```

    virtual void show() {cout<<endl<<" k="<<k;}
};
class D: public B
{
    int h;
public:
    D (int m, int n) : B (m)
    {
        h=n;
        B *b;
        b=this;
        b->show();
    }
    void show()
    {
        cout<<endl<<" h="<<h;
    }
};
void main()
{
    clrscr();
    B b(4);
    D d(5,2);
}

```

OUTPUT

h=2

Explanation: In the above program, the base class B contains a virtual function. In the derived class D, a similar function is redefined. Both the base and derived classes contain a constructor. In the derived class constructor, the base class pointer *b is declared. We know that this pointer contains the address of the object calling the member function. Here, the `this` pointer holds the address of the object d. The pointer object b invokes the function `show()`. The derived class `show()` function is invoked.

Here, the object d is not fully constructed; then, how does it invoke the member function of a similar class? This is possible, because a virtual function call reaches ahead into inheritance.

15.13 VIRTUAL DESTRUCTORS

We have learned how to declare virtual functions. Likewise, destructors can be declared as virtual. The constructor cannot be virtual, as it requires information about the accurate type of the object in order to construct it properly. The virtual destructors are implemented in a similar manner to virtual functions. In constructors and destructors, pecking order (hierarchy) of base and

derived classes is constructed. Destructors of derived and base classes are called when a derived class object addressed by the base class pointer is deleted.

For example, a derived class object is constructed using a new operator. The base class pointer object holds the address of the derived object. When the base class pointer is destructed using the delete operator, the destructor of the base and derived class is executed. The following program explains this:

15.16 Write a program to define virtual destructors.

```
#include<iostream.h>
#include<conio.h>

class B
{
    public:
    B() {cout<<endl<<"In constructor of class B";}
    virtual ~B() {cout<<endl<<"In destructor of class B";}
};

class D: public B
{
    public:
    D() {cout<<endl<<"In constructor of class D";}
    ~ D() {cout<<endl<<"In destructor of class D";}
};

void main()
{
    clrscr();
    B *p;
    p= new D;
    delete p;
}
```

OUTPUT

In constructor of class B
In constructor of class D
In destructor of class D
In destructor of class B

Explanation: In the above program, the destructor of the base class B is declared as virtual. A dynamic object is created, and the address of the nameless object that is created is assigned to pointer p. The new operator allocates the memory required for data members. When the object goes out of scope, it should be deleted, and the same should be performed by the statement delete p;. When the derived class object is pointed by the base class pointer object, in order to invoke the base class destructor, virtual destructors are useful.

15.14 DESTRUCTORS AND VIRTUAL FUNCTIONS

When a virtual function is invoked through a non-virtual member function, late binding is performed. When a virtual function is called through the destructor, the redefined function of a similar class is invoked. Consider the following program:

15.17 Write a program to call virtual function using destructors.

```
#include<iostream.h>
#include<conio.h>
class B
{
    public:
    ~ B() {cout<<endl<<" in virtual destructor";}
    virtual void joy() {cout<<endl<<"In joy of class B";}
};
class D : public B
{
    public :
    ~ D()
    {
        B *p;
        p=this;
        p->joy();
    }
    void joy() {cout<<endl<<" in joy() of class D";}
};
void main()
{
    clrscr();
    D X;
}
```

OUTPUT

in joy() of class D
in virtual destructor

Explanation: In the above program, the destructor of the derived class function joy() is invoked. The member function joy() of the derived class is invoked followed by the virtual destructor.

SUMMARY

- (1) The word *poly* means many, and *morphism* means several forms. Both the words are derived from Greek language. Thus, by combining these two words, a new whole word called *polymorphism* is created, which means various forms.
- (2) The information pertaining to various overloaded member functions is to be given to the compiler while compiling. This is called *early binding* or *static binding*. The deciding function call at run time is called *run time* or *late* or *dynamic binding*. Dynamic binding permits to suspend the decision of choosing a suitable member function until run time.
- (3) Pointers to the object of a base class are type compatible with pointers to the object of a derived class. The reverse is not possible.
- (4) Virtual functions of the base class should be redefined in the derived classes. The programmer can define a virtual function in a base class, and can then use a similar function name in any derived class.
- (5) Addresses of different objects can be stored in an array to invoke the function dynamically.
- (6) In practical applications, the member function of the base class is rarely used for doing any operation; such functions are called *do-nothing functions*, *dummy functions*, or *pure virtual functions*.
- (7) All other derived classes without pure virtual functions are called *concrete classes*.
- (8) Abstract classes are similar to a skeleton on which new classes are designed to assemble a well-designed class hierarchy. They are not used for object declaration.
- (9) Virtual functions can be invoked using a pointer or a reference.
- (10) If an object of a derived class is assigned to a base class object, the compiler allows it. However, it copies only the base class members of the object, and this process is known as *object slicing*.
- (11) It is possible to invoke a virtual function using a constructor. The constructor makes the virtual mechanism illegal.
- (12) We have learned how to declare virtual functions. Likewise, destructors can be declared as virtual. The constructor cannot be virtual. The virtual destructors are implemented in a similar manner to virtual functions. Destructors of derived and base classes are called when a derived class object addressed by the base class pointer is deleted.

EXERCISES

(A) Answer the following questions

- (1) What is polymorphism?
- (2) Explain compile-time and run-time binding.
- (3) Explain the use of a virtual keyword.
- (4) What are pure functions? How are they declared?
- (5) Is it possible to declare an object of the class that contains a pure function?
- (6) What is the difference between a virtual function and virtual classes?
- (7) What are virtual destructors?
- (8) How does a C++ compiler accomplish dynamic binding?
- (9) Where do we use virtual functions? Give their applications.

- (10) What is early binding and late binding?
- (11) Explain object slicing.
- (12) Explain virtual destructors.
- (13) What are abstract classes? How can they be used for a debugging program?
- (14) What are VPTR and VTABLE? Explain in detail.
- (15) Describe the rules for declaring virtual functions.
- (16) What is the difference between a base class pointer and a derived class pointer?

(B) Answer the following by selecting the appropriate option

- (1) Consider the statement `virtual void display()=0`. The `display()` function is
 - (a) **pure virtual function**
 - (b) pure member function
 - (c) normal function
 - (d) all of the above
- (2) The do-nothing function is nothing but
 - (a) **pure virtual function**
 - (b) pure member function
 - (c) both (a) and (b)
 - (d) none of the above
- (3) Static binding is done
 - (a) **at the time of compilation of the program**
 - (b) at run time
 - (c) both (a) and (b)
 - (d) none of the above
- (4) Dynamic binding is done using the keyword
 - (a) **virtual**
 - (b) inline
 - (c) static
 - (d) void
- (5) The `virtual` keyword solves the
 - (a) **ambiguity in the base and derived class**
 - (b) ambiguity in derived classes
 - (c) ambiguity in base classes
 - (d) none of the above
- (6) B is a base class object, and D is a derived class object. The statement `B = D`
 - (a) **copies all elements of object D to object B**
 - (b) copies only the base portion of object D to B
 - (c) copies only the derived portion of object D to B
 - (d) none of the above
- (7) When a base class is not used for objects, the declaration is called
 - (a) abstract class
 - (b) container class
 - (c) concrete class
 - (d) derived class
- (8) The derived class without a pure virtual function is called
 - (a) concrete class
 - (b) abstract class
 - (c) container class
 - (d) derived class
- (9) A pointer to base class object can hold addresses of
 - (a) only derived class objects
 - (b) only base class objects
 - (c) addresses of a base class object and its derived class object
 - (d) none of the above

(C) Attempt the following programs

- (1) Write a program to declare a function `show()` in base and derived classes. Display a message through the function to know the name of the class whose member function is executed. Use the late binding concept using the `virtual` keyword.
- (2) Write a program to define classes A, B, and C. The class C is derived from classes A and B. Define `count()` member function in all the classes as virtual. Count the number of objects created.
- (3) Write a program to declare a matrix class, which has a data-member integer array as 3×3 . Derive class matrix A from class matrix B and matrix B from matrix A. All these classes should have a function `show()` to display the contents. Read and display the elements of all three matrices.

- (4) Write a program to declare the reference object. Invoke the member function of the class using the reference object.
- (5) Write a program to demonstrate object slicing.
- (6) Write a program to demonstrate the use of an abstract class.
- (7) Write a program to redefine a virtual base class function in the derived class. In addition, add a new member in the derived class. Observe the VTABLE.
- (8) Write a program to define virtual, non-virtual functions and determine the size of the objects.
- (9) Write a program to invoke the member function of the base and derived class using the pointer of the base class.
- (10) Write a program to access the members of the base and derived class using the pointer objects of both classes.

(D) Find the bugs in the following programs

(1)

```
class B {virtual void
display()=0;};
void main() {B d;}
```

(2)

```
class B
{
public:
virtual void display()
{
cout<<"\n no function display
defined in this class.";
}
};
struct D : B {};
void main()
{
D d;
d.display();
}
```

(3)

```
class B
{
    int a,b,c;
    public:
    B() {a=10, b=20,c=40;}
};
class D : public B {};
void main()
{
    B b;
    D d;
    d=b;
}
```

(4)

```
class B {};
class D : public B
{
    int i,j,k;
    public:
    D() {i=5; j=10; k=15;};
void main()
{
    B b;
    D d;
    b=d;
    cout<<b.i;
}
```

(5)

```
class B
{public:
    B() {cout<<endl<<"In con-
structor of class B";}
    virtual ~B() =0;
};
class D : public B
{
    public:
    D() {cout<<endl <<"In con-
structor of class D";}
    ~D() {cout<<endl<<"In de-
structor of class D";}
};
void main()
{
    B *p;
    p=new D;
    delete p;
}
```


This page is intentionally left blank.

Applications with Files

16 CHAPTER

CHAPTER OUTLINE

- 16.1 Introduction
- 16.2 File Stream Classes
- 16.3 Steps of File Operation
- 16.4 Checking for Errors
- 16.5 Finding End of a File
- 16.6 File Opening Modes
- 16.7 File Pointers and Manipulators
- 16.8 Manipulators with Arguments
- 16.9 Sequential Access Files
- 16.10 Binary and ASCII Files
- 16.11 Random Access Operations
- 16.12 Error Handling Functions
- 16.13 Command-Line Arguments
- 16.14 Strstreams
- 16.15 Sending Output to Devices
- 16.16 More Programs

16.1 INTRODUCTION

With the advancement of information technology, sea information is available on the Internet. A huge amount of data is processed in the computer networking. The information can be uploaded or downloaded from a desktop computer. The information transfer in computer networking in day-to-day life is in the form of files. The files can be written, read, or updated depending on the applications. The data are saved in the file on the disk. The file is an accumulation of data stored on the disk. The stored data can be retrieved or updated. The console I/O function reads

data through input devices and displays them on the screen. The data read through these functions are temporarily stored in variables or arrays. The data stored vanishes when the program ends. In order to store the data permanently, we need to apply the disk I/O function. The disk I/O functions are associated with disk files. These functions perform read and write operations with the file. Similar to other languages, C++ also provides functions that allow the programmer to perform read and write operations with the file. The disk I/O functions in C++ are very different as compared with C, though the entire disk I/O functions of C can be used in C++. However, these functions are not appropriate for the object-oriented conditions.

Secondary storage devices such as floppy disks and hard disks are used to store data in the form of files. The main memories of a computer such as random access memory or read-only memory are not used for the storage of files. This is because the main memory of a computer is limited and cannot hold a large amount of data. Another reason is that the main memory is volatile; that is, when the computer is switched off, the contents of RAM vanish. The operating system is a set of system programs. The operating system provides programs for file management and controlling storage devices such as hard disks and floppy disks, as shown in Figure 16.1.

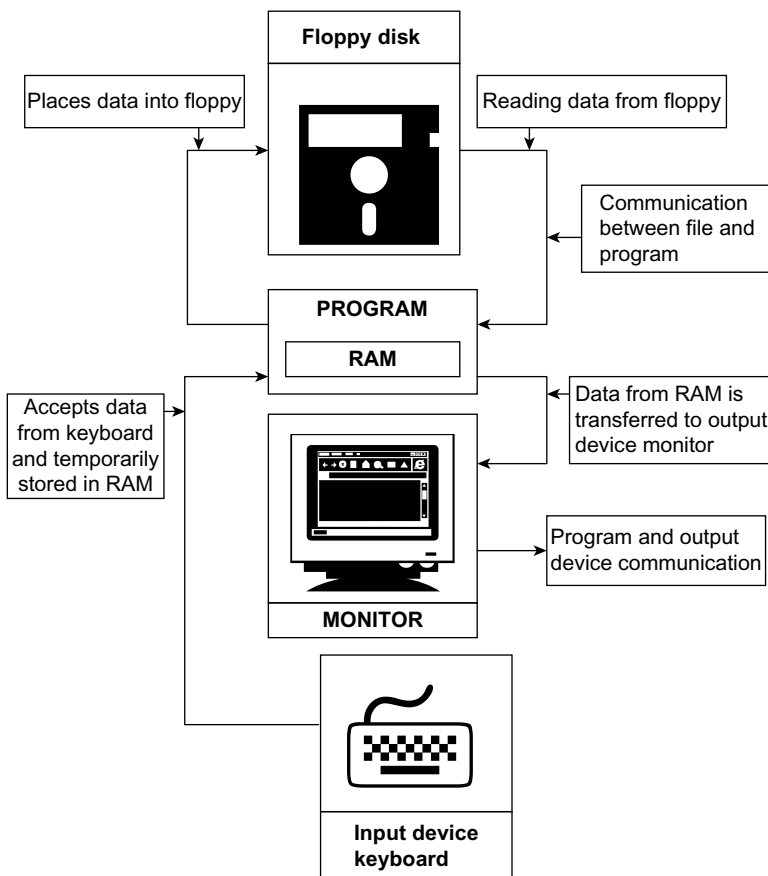


Fig. 16.1 Communication between program, file, and output device

As shown in Figure 16.1, the data read from the keyboard are stored in variables. Variables are created in RAM (type of primary memory). On applying the disk, I/O operations selected or all variables created in RAM can be stored to secondary storage devices such as hard disks or floppy disks. In Figure 16.1, the floppy disk is indicated. It is also possible to read data from secondary storage devices. When data are read from such devices, they are placed in the RAM and then, console I/O operations are used to transfer them to the screen. RAM is used to hold data temporarily. The variables are used to store data during program execution, and the variables are created in RAM. Hence, the secondary storage device RAM plays an important role.

The file is an accumulation of the data stored on the disk created by the user. The programmer assigns a file name. The file names are unique and are used to identify the file. No two files can have a similar name in the same directory. There are various types of files such as text files, program files, data files, and executable file. Data files contain a combination of numbers, alphabets, symbols, and so on called data.

Data communication can be performed between programs and output devices or between files and programs. File streams are used to carry the communication among the above-mentioned devices. The stream is nothing but a flow of data in bytes in sequence. If data were received from input devices in sequence, then it is called a `source stream`, and if the data were passed to output devices, then it is called a `destination stream`. Figure 16.2 shows the input and output streams. The input stream brings data to the program, and the output stream collects data from the program. In this way, the input stream extracts data from the file and transfers it to the program; whereas the output stream stores the data in the file provided by the program.

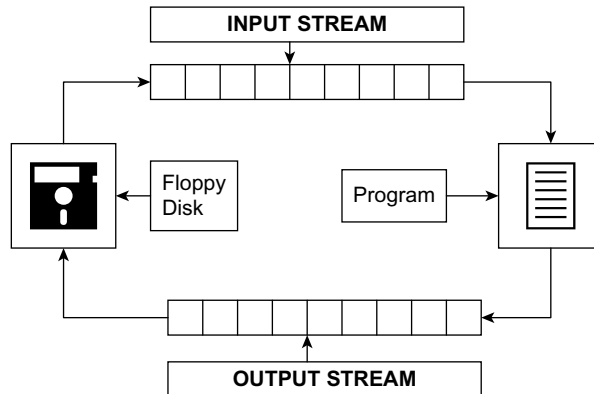


Fig. 16.2 Input and output streams

16.2 FILE STREAM CLASSES

A stream is nothing but a flow of data. In the object-oriented programming, the streams are controlled using the classes. The operations with the files mainly consist of two types. They are read and write. C++ provides various classes, as shown in Figure 16.3, to perform these operations. The `ios` class is the base class. All other classes are derived from the `ios` class. These classes contain several member functions that perform input and output operations. The `streambuf` class has low-level routines for controlling data buffer.

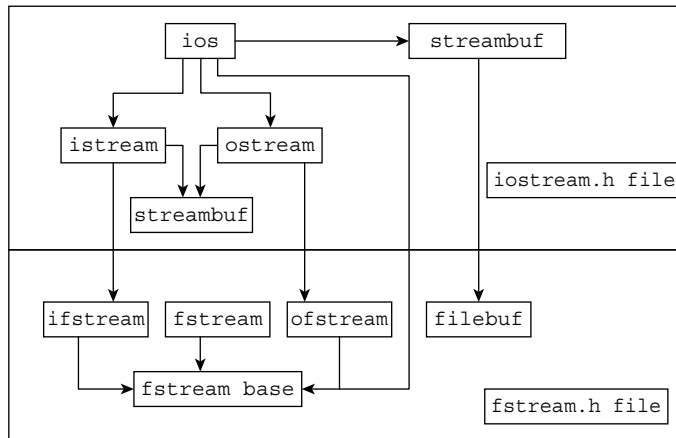


Fig. 16.3 iostream class summary

The `istream` and `ostream` classes control input and output functions, respectively. The `ios` is the base class of these two classes. The member functions of these classes handle formatted and unformatted operations (for more details about formatted and unformatted functions, please refer Chapter 3) input and output in C++ (section 3.6). The functions `get()`, `getline()`, and `read()` and overloaded extraction operators (`>>`) are defined in the `istream` class. The functions `put()`, `write()`, and overloaded insertion operators (`<<`) are defined in the `ostream` class.

The `iostream` class is also a derived class. It is derived from `istream` and `ostream` classes. There are three other useful derived classes. They are `istream_withassign`, `ostream_withassign`, and `iostream_withassign`. They are derived from `istream`, `ostream`, and `iostream`, respectively.

The classes `ifstream` and `ofstream` are derived from `istream` and `ostream`, respectively. These classes handle input and output with the disk files. The header file `fstream.h` contains a declaration of `ifstream`, `ofstream`, and `fstream` classes, including `isotream.h` file. This file should be included in the program while doing disk I/O operations.

The filebuf: `filebuf` accomplishes input and output operations with files. The `streambuf` class does not organize streams for input or output operations. The derived classes of `streambuf` perform these operations. It also arranges a space for keeping input data and for sending output. The I/O functions of the classes `istream` and `ostream` invoke the `filebuf` functions to perform the insertion or extraction on the streams. It holds constant `openprot` used in function `open()` and `close()` as a member.

The fstreambase: The `fstreambase` acts as a base class for `fstream`, `ifstream`, and `ofstream`. The functions such as `open()` and `close()` are defined in `fstreambase`.

The ifstream: This class is derived from `fstreambase` and `istream` by multiple inheritance. It can access the member functions such as `get()`, `getline()`, `seekg()`, `tellg()`, and `read()`. It allows input operations and provides the `open()` function with the default input mode.

The ofstream: This class is derived from `fstreambase` and `ostream` classes. It can access the member functions such as `put()`, `seekp()`, `write()`, and `tellp()`. It allows output operations and provides the member function `open()` with the default output mode.

The **fstream**: It allows simultaneous input and output operations on a `filebuf`. The member function of the base classes `istream` and `ostream` starts the input and output. For example, `fstream` invokes the member function `istream::getline()` to read characters from the file. It provides the `open()` function with the default input mode.

16.3 STEPS OF FILE OPERATIONS

Before performing file operations, it is necessary to create a file. The operation of a file involves the following basic activities:

- File name
- Opening the file
- Reading or writing the file (file processing)
- Detecting errors
- Closing the file

The file name can be a sequence of characters, called a `string`. Strings are always declared with a character array. Using the file name, a file is recognized. The length of the file name depends on the operating system; for example, WINDOWS-98 supports long file names, whereas MS-DOS supports only eight characters. A file name also contains an extension of three characters. The file name should not be a device name such as `LPT1` and `CON`. The list of device names is given in Table 16.6. You might have observed the `.cpp` extension to the C++ program file name separated by the dot (`.`). The extension is optional. The following file names are valid in MS-DOS and WINDOWS-98 operating systems:

<code>data.dbf</code>	// extension is <code>.dbf</code>
<code>tc.exe</code>	// extension is <code>.exe</code>
<code>prg.cpp</code>	// extension is <code>.cpp</code>
<code>prg.exe</code>	// extension is <code>.exe</code>
<code>prg.obj</code>	// extension is <code>.obj</code>
<code>Marks</code>	// without extension

The first step in the disk file I/O operation is the creation of a file stream object and connecting it with the file name. The classes `ifstream`, `ofstream`, and `fstream` can be used for creating a file stream defined in the header file `fstream.h`. The selection of the class is according to the operation that is to be carried out with the file. The operation may be read or write. Two methods are used for the opening of a file. They are as follows:

- | |
|---|
| (a) Constructor of the class |
| (b) Member function <code>open()</code> |

(a) Constructor of the class

When objects are created, a constructor is automatically executed, and objects are initialized. In the same way, the file stream object is created using a suitable class, and it is initialized with the file name. The constructor itself uses the file name as the first argument and opens the file. The class `ofstream` creates output stream objects, and the class `ifstream` creates input stream objects.

Consider the following examples:

```
(a) ofstream out ("text");
(b) ifstream in ("list");
```

In the statement (a), `out` is an object of the class `ofstream`; file name `text` is opened, and data can be written to this file. The file name `text` is connected with the object `out`. Similarly, in the statement (b), `in` is an object of the class `ifstream`. The file `list` is opened for input and connected with the object `in`. It is also valid to use a similar name for input and output operations.

It is possible to use these file objects in program statements such as stream objects. Consider the following statements:

```
cout<<"One Two Three";
```

The above statement displays the given string on the screen.

```
out<<"One Two Three";
```

The above statement writes the specified string into the file pointed by the object `out` as shown in Figure 16.4. The insertion operator `<<` has been overloaded appropriately in the `ostream` class to write data to the appropriate stream.

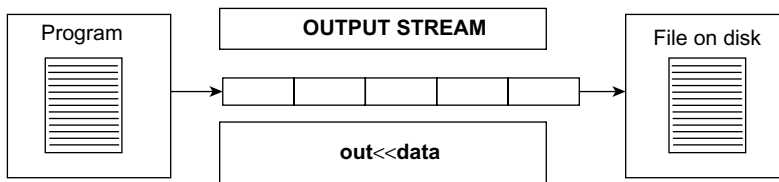


Fig. 16.4 Interaction between `fstream` object and disk file

Consider the following statements:

```
out<<"One Two Three";           // Write string to the
                                // file
out<<age;                        // Write contents of vari-
                                // able
out<<5000;                       // Write 5000 to file
out<<'A';                        // Write character 'A' to
                                // file
out<<123.14;                     // Write 123.14 to file
out<<123.34<<endl<<5000<<'A';    // Multiple values in one
                                // statement
```

In the last statement, we have separated each value using an “`endl`” manipulator. It is essential, because when we store values such as 15 and 123.34, they are stored in the file as strings; that is, 123.34 would be stored as ‘1’, ‘2’, ‘3’, ‘.’, ‘3’, ‘4’.

The value 123.34 requires four bytes, but when stored in a file, it occupies six bytes. When we need to store more numeric values, more bytes are occupied and the file size is increased. Every data item should be separated by a delimiter. This is essential, because during the read operation, the extraction operator cannot determine where one number is ending and another is beginning. A similar problem can be observed while reading strings. The above limitation can be overcome using the binary file operation that is discussed later in the same chapter.

Similarly, in the following statements,

```
in>>string; // Reads string from the file where
              string is a character array
in>>num;     // Reads number from the file where
              num is an integer variable
```

the `in` object reads data from the file associated with it, as shown in Figure 16.5. For reading data from a file, we have to create an object of the `ifstream` class. The new line character (“\n”) inserted at the end of every line helps the overloaded operator separate the various items stored in the file. When numbers are read back from the file, they are converted into their binary format.

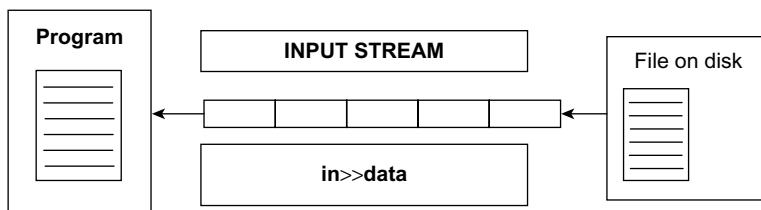


Fig. 16.5 Interaction between `ifstream` object and disk file

16.1 Write a program to open an output file using `fstream` class.

```
#include<fstream.h> //File I/O
#include<conio.h>

int main()
{
    clrscr();
    ofstream out("text");
    out << "One Two Three Four\n";
    out << "1 2 3 4";
    out << "\n** The End **";
    return 0;
}
```

Explanation: In the above program, the statement `ofstream out ("text")` text is opened and connected with the object `out`.

Use of object out
<pre>out <<"One Two Three Four \n"; out <<"1 2 3 4"; out <<"\n** The End **";</pre>

The above statements write data (enclosed in quotation marks) to the file pointed by the object out, that is, text. The operator << is an insertion operator, and it writes the data in the file named text. To open the file text, the user can use Turbo C++ editor and click **DOS Shell** of the file menu. On DOS, the prompt user should type the file name and press the enter key. In addition, a further editor can be opened from the DOS prompt, and the user can see the contents of the text file. The contents of the file can be seen by the user, and it appears as follows:

Contents of the file text
<pre>One Two Three Four 1 2 3 4 ** The End **</pre>

While opening a file for the writing operation, the user should give the new file name. If the given file already exists, its contents are erased and it is treated as a new file.

16.2 Write a program to read data from file using object of ifstream class.

```
#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    char *n;
    ifstream in("text"); // Opens a file in read mode
    in >>n; // Reads string from file
    cout<<n <<" "; // Displays string to the console
    in >>n ;
    cout<<n <<" ";
    in >>n ;
    cout<<n <<" ";
    return 0;
}
```

OUTPUT

One Two Three

Explanation: In the above program, the statement `ifstream in ("text");` is opened and connected with the object in. The statement `in >>n` reads data from the file

and assigns it to the variable followed by the (>>) extraction operator. Here, the variable *n is a character pointer. If a numeric value is read using character type, the data cannot be used for arithmetic operations. If character-type data are read using a numeric-type variable, ASCII values of the character read will be displayed. Hence, the variable type should match with the data type. The statement `cout<<n <<" ";` displays the data stored in variable n.

In the above programs, the file associated with the object are automatically closed when the stream object goes out of scope. In order to explicitly close the file, the following statement is used:

Closing of files
<code>out.close();</code> <code>in.close();</code>

Here, out is an object, and `close()` is a member function that closes the file connected with the object out. Similarly, the file associated with the object in is closed by the member function `close()`.

16.3 Write a program to write and read text in a file. Use ofstream and ifstream classes.

```
#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    char name[15];
    int age;
    ofstream out("text");
    cout<<"\n Name:";
    cin>>name;
    cout<<"Age:";
    cin>>age;
    out<<name<<"\t";
    out<<age <<"\n";
    out.close(); // File is closed
    ifstream in ("text");
    in>>name;
    in>>age;
    cout<<"\nName\t:"<<name<<"\n";
    cout<<"Age:"<<age;
    in.close();
    return 0;
}
```

OUTPUT**Name : Sameer****Age : 24****Name : Sameer****Age : 24**

Explanation: The above program is a combination of the previous two programs. The `out` is an object of the `ofstream` class, and it is linked with the output file `text`. The data entered by the user is written in the file `text`. The `close()` function closes the file associated with the object `out`. Again, the same file is opened by the `in` object of class `ifstream` for reading purpose. The data recently written are read and displayed on the screen. Thus, in a program, both write and read operations are performed. The use of function `close()` is essential for changing the mode of the file. In case a file is not closed and an attempt is again made to open it in another mode, no compile time error is generated. However, the result will not be satisfactory.

(b) The open() function

In the last few programs, we have studied how files could be opened for reading and writing using constructors. Now, the second approach can be used with the `open()` function. The `open()` function is used to open a file, and it uses the stream object. The `open()` function has two arguments. First is the file name, and second is the mode. The mode specifies the purpose of opening a file; that is, read, write, append, and so on. The details are discussed in heading 16.5. In the following examples, the default mode is considered. The default values for `ifstream` is `(ios::in)` reading only and for `fstream` is `(ios::out)` writing only.

(A) Opening file for write operation

```
ofstream out;           // Creates stream object out
out.open ("marks.dbf"); // Opens file and links with the
                        // object out
out.close()             // Closes the file pointed by the ob-
                        // ject out
out.open ("result.dbf"); // Opens another file
```

(B) Opening file for read operation

```
ifstream in;           // Creates stream object in
in.open (" marks.dbf"); // Opens file and link with the
                        // object in
in.close() ;           // Closes the file pointed by
                        // object in
in.open ("result.dbf"); // Opens another file
```

16.4 Write a program to open multiple files for writing and reading purposes. Use open() function.

```
#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    // Writing data //
    out.open ("months"); // Opens file
    out<<"March\n"; // Writes string to the f ile
    out<<"April\n";
    out <<"June\n";
    out.close(); // Closes the file
    out.open ("days"); // Opens another file
    out <<"31\n";
    out <<"30\n";
    out <<"30\n";
    out.close(); // closes the file
    // reading data //
    #define T 20
    char text[T];
    ifstream in;
    in.open ("months"); // opens file for reading
    cout<<"\nMonth Names \n";
    while (in)
    {
        in.getline(text,T);
        cout<<text<<"\n";
    }
    in.close();
    in.open ("days");
    cout<<"\nDays \n";
    while (in)
    {
        in.getline(text,T);
        cout<<text<<"\n";
    }
    in.close();
    return 0;
}
```

OUTPUT

Month Names

March

April

June

Days

31

30

30

Explanation: In the above program, `out` is an object of the `ofstream` class. The object `out` is used with the `open()` function to open files for the write operation. Two files are opened: `months` and `days`. Month names are written in the file `months`, and number of days is written in the file `days`. The data are written in the files one after another. The same object is used for both the files; hence, it is necessary to close a previously linked file with the object before opening a new file. In this program, the `month` file is opened first and after closing it, the file `days` is opened. The files are closed using the function `close()`. The same procedure is implemented for opening the same files for the reading purpose. When the end of the file is reached, the `while` loop terminates.

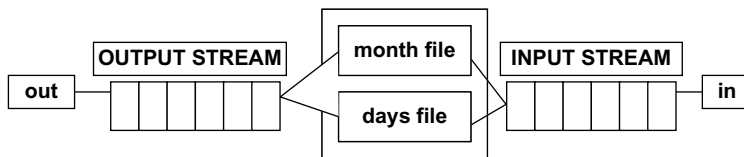


Fig. 16.6 Stream objects with multiple files

As shown in Figure 16.6, the `out` object opens the `month` and `days` file one after another for writing. The object `in` opens these files for reading. Before opening the next file, the previously associated file with the object is closed. A single object cannot open multiple files simultaneously.

16.4 CHECKING FOR ERRORS

Until now, we have carried out the file operations without thinking whether they are performed successfully or not. If any fault occurs during a file operation, the fault cannot be detected. Various errors can be made by the user while performing a file operation. Such errors should be reported in the program to avoid further program failure. When a user attempts to read a file that does not exist or opens a read-only file for writing purpose, the operation fails in such situations. Such errors should be reported, and proper actions have to be taken before further operations are performed.

The `!` (logical negation operator) overloaded operator is useful for detecting errors. It is a unary operator and, in short, it is called a `not` operator. The `(!)` `not` operator can be used

with objects of stream classes. This operator returns a non-zero value if a stream error occurs during an operation. Consider the following program:

16.5 Write a program to check whether the file is successfully opened or not.

```
#include<fstream.h>
#include<constream.h>

void main()
{
    clrscr();
    ifstream in ("text");
    if (!in) cerr <<"\n File is not opened"<<in;
    else cerr <<"\n file is opened"<<in;
}
```

OUTPUT

File is opened 0x8f15ffd2

Explanation: In the above program, an existing file text is opened for reading. The file is opened successfully. Hence, no error is generated. The if () statement checks the contents in objects using the following statement: if (!in) . The value of the object is **0x8f15ffd2**. In case the operation fails, the value of the object in would be 0x8f160000 .

16.6 Write a program to detect error in file operation using ! Operator

```
#include<fstream.h>
#include<constream.h>
#include<iomanip.h>

main()
{
    clrscr();
    char c, f_name[10];
    cout<<"\n Enter file name:";
    cin>>f_name;

    ifstream in(f_name);
    if (!in)
    {
        cerr<<" Error in opening file"<<f_name<<endl;
        return 1;
    }
    in>>resetiosflags(ios::skipws);
    while (in)
```

```
{  
    in>>c;  
    cout<<c;  
}  
return 0;  
}
```

OUTPUT

```
Enter file name : TEXT  
One Two Three Four  
1 2 3 4  
** The End **
```

Explanation: In the above program-using constructor of the class `ifstream`, a file is opened. The file that is to be opened is entered by the user during program execution. If the file does not exist, the `ifstream` object (`in`) contains 0. The `if ()` statement checks the value of an object `in`, and if the operation fails, a message will be displayed and the program is terminated. In case the file exists, using the `while ()` loop, the contents of the file are read one character at a time and displayed on the screen. You can observe the use of the NOT operator with object `in` in `if ()` and the `while ()` statement. If the statement `in>>resetiosflags (ios::skipws);` is removed, the contents of the file would be displayed without space in one line. The operator `>>` ignores the white space character. Consider the following statement:

```
With NOT operator  
if (!in)  
{  
    statement1;  
    else  
    statement2  
}
```

The NOT operator is used in association with the object. It is also possible to perform the operation without the use of the NOT operator. The following statement works opposite as compared with the above statement:

```
Without NOT operator  
if (in)  
{  
    statement1;  
    else  
    statement2  
}
```

16.5 FINDING END OF A FILE

While reading data from a file, it is necessary to find where the file ends, that is, the end of the file. The programmer cannot predict the end of the file. If in a program, while reading the file, the program does not detect the end of the file, the program drops in an infinite loop. To avoid this, it is necessary to provide correct instructions to the program that detects the end of the file. Thus, when the end of the file is detected, the process of reading data can be easily terminated. The `eof()` member function() is used for this purpose. The `eof()` stands for the end of the file. It is an instruction given to the program by the operating system that the end of the file is reached. It checks the `ios::eofbit` in the `ios::state`. The `eof()` function returns the non-zero value, when the end of the file is detected; otherwise, it is zero.

16.7 Write a program to read and display contents of file. Use `eof()` function.

```
#include<fstream.h>
#include<constream.h>
#include<iomanip.h>

main()
{
    clrscr();
    char c, f_name[10];
    cout<<"\n Enter file name:";
    cin>>f_name;
    ifstream in(f_name);
    if (!in)
    {
        cerr<<" Error in opening file"<<f_name<<endl;
        return 1;
    }
    while (in.eof()==0)
    {
        in.get(c);
        cout<<c;
    }
    return 0;
}
```

OUTPUT

Enter file name : text

Programming with ANSI and TurboC

Explanation: The above program is similar to the previous one. Here, the member function `eof()` is used in the `if()` statement. The program displays the contents of the file `text`. While specifying a file name for reading purpose, be sure that it exists.

The `while()` or `if()` statement checks whether the value is zero or non-zero (address of object). Here, `zero` means false, and `non-zero` means true. The details of the above functions are discussed later in the same chapter.

16.6 FILE OPENING MODES

In previous examples, we have learned how to open files using constructors and the `open()` function using the objects of `ifstream` and `ofstream` classes. The opening of the file also involves several modes depending on the operation to be carried out with the file. The `open()` function has the following two arguments:

Syntax of <code>open()</code> function
<code>object.open ("file_ name", mode);</code>

Here, the object is a stream object, followed by the `open()` function. The bracket of the `open` function contains two parameters. The first parameter is the name of the file, and the second is the mode in which the file is to be opened. In the absence of a mode parameter, a default parameter is considered. The file mode parameters are as shown in Table 16.1.

Table 16.1 File modes

Mode parameter	Operation
<code>ios::app</code>	Adds data at the end of file
<code>ios::ate</code>	After opening character pointer goes to the end of file
<code>ios:: binary</code>	Binary file
<code>ios::in</code>	Opens file for reading operation
<code>ios::nocreate</code>	Opens unsuccessfully if the file does not exist
<code>ios::noreplace</code>	Opens files if they are already present
<code>ios::out</code>	Open files for writing operation
<code>ios::trunc</code>	Erases the file contents if the file is present

- (1) The mode `ios::out` and `ios::trunc` are the same. When `ios::out` is used, if the specified file is present, its contents will be deleted (truncated). The file is treated as a new file.
- (2) When the file is opened using `ios::app` and `ios::ate` modes, the character pointer is set to the end of the file. The `ios:: app` lets the user add data at the end of the file, whereas the `ios::ate` allows the user to add or update data anywhere in the file. If the given file does not exist, a new file is created. The mode `ios::app` is applicable to the output file only.
- (3) The `ifstream` creates an input stream and an `ofstream` output stream. Hence, it is not compulsory to give mode parameters.
- (4) While creating an object of the `fstream` class, the programmer should provide the mode parameter. The `fstream` class does not have the default mode.

- (5) The file can be opened with one or more mode parameters. When more than one parameter is necessary, a bit-wise OR operator separates them. The following statement opens a file for appending. It does not create a new file if the specified file is not present.

File opening with multiple attributes
<code>out.open ("file1", ios::app ios::nocreate)</code>

16.9 Write a program to open a file for writing and store float numbers in it.

```
#include<fstream.h>
#include<iomanip.h>

void main()
{
    float a=784.52, b=99.45, c =12.125;
    ofstream out ("float.txt", ios::trunc);
    out<<setw(10)<<a<<endl;
    out<<setw(10)<<b<<endl;
    out<<setw(10)<<c<<endl;
}
```

Explanation: In the above program, the file "float.txt" is opened. If the file already exists, its contents are truncated. The three float numbers are written in the file.

16.10 Write a program to open a file in binary mode. Write and read the data.

```
#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    char data[32];
    out.open ("text", ios::out | ios::binary);
    cout<<"\n Enter text"<<endl;
    cin.getline(data, 32);
    out <<data;
    out.close();
    ifstream in;
    in.open("text", ios::in | ios::binary);
    cout<<endl<<"Contents of the file \n";
    char ch;
    {
        ch= in.get();
        cout<<ch;
    }
}
```

```

    }
    return 0;
}

```

OUTPUT

Programming In ANSI and TURBO-C
Contents of the file
Programming In ANSI and TURBO-C

Explanation: The above program is similar to the previous one. The only difference is that here files are opened in binary mode.

16.7 FILE POINTERS AND MANIPULATORS

All file objects hold two file pointers that are associated with the file. These two file pointers provide two integer values. These integer values indicate the exact position of the file pointers in the number of bytes in the file. The read or write operations are carried out at the location pointed by these file pointers. One of them is called `get pointer` (input pointer), and the second one is called `put pointer` (output pointer). During reading and writing operations with files, these file pointers are shifted from one location to another in the file. The (input) `get pointer` helps in reading the file from the given location, and the output pointer helps in writing data in the file at the specified location. When read and write operations are carried out, the respective pointer is moved.

While a file is opened for the reading or writing operation, the respective file pointer input or output is by default set at the beginning of the file. This makes it possible to perform the reading or writing operation from the beginning of the file. The programmer need not explicitly set the file pointers at the beginning of files. To explicitly set the file pointer at the specified position, the file stream classes provides the following functions:

Read mode: When a file is opened in read mode, the `get pointer` is set at the beginning of the file, as shown in Figure 16.7. Hence, it is possible to read the file from the first character of the file.

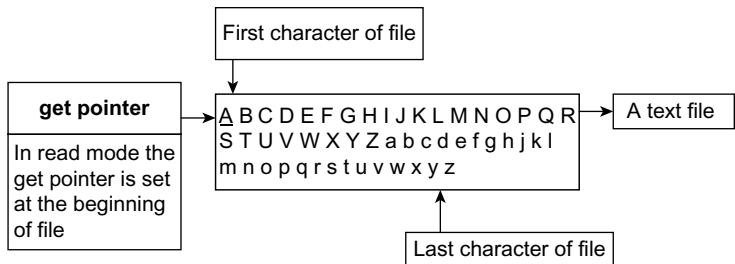


Fig. 16.7 Status of get pointer in read mode

Write Mode: When a file is opened in write mode, the `put pointer` is set at the beginning of the file, as shown in Figure 16.8. Thus, it allows the write operation from the beginning of the file. In case the specified file already exists, its contents will be deleted.

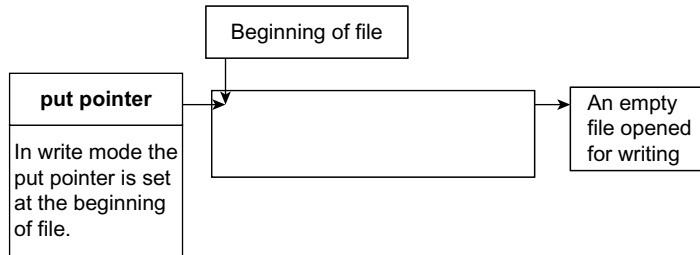


Fig. 16.8 Status of put pointer in write mode

Append Mode: This mode allows the addition of data at the end of the file. When the file is opened in append mode, the output pointer is set at the end of the file, as shown in Figure 16.9. Hence, it is possible to write data at the end of the file. In case the specified file already exists, a new file is created, and the output is set at the beginning of the file. When a pre-existing file is successfully opened in append mode, its contents remain safe and new data are appended at the end of the file.

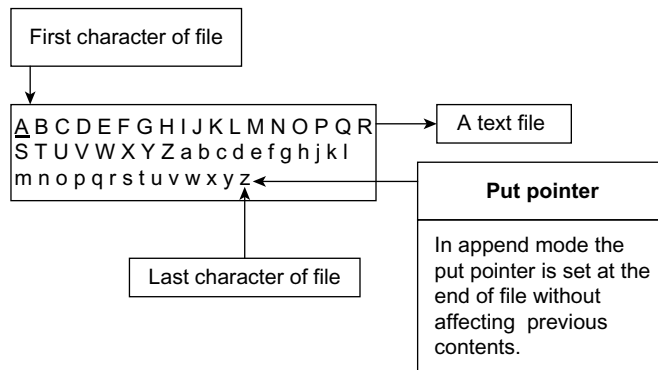


Fig. 16.9 Status of put pointer in append mode

C++ has four functions for the setting of points during file operation. The position of the cursor in the file can be changed using these functions. These functions are described in Table 16.2.

Table 16.2 File pointer handling functions

Function	Uses	Remark
seekg()	Shifts input (get) pointer to a given location.	Member of ifstream class
seekp()	Shifts output (put) pointer to a given location.	Member of ofstream class
tellg()	Provides the present position of the input pointer.	Member of ifstream class
tellp()	Provides the present position of the output pointer.	Member of ofstream class

As given in Table 16.2, the `seekg()` and `tellg()` are member functions of the `ifstream` class. All the above four functions are present in the class `fstream`. The class `fstream` is de-

rived from `ifstream` and `ofstream` classes. Hence, this class supports both input and output modes, as shown in Figure 16.10. The `seekp()` and `tellp()` work with the put pointer, and `tellg()` and `seekg()` work with the get pointer.

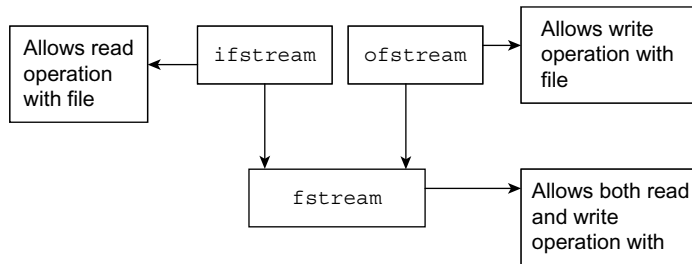


Fig. 16.10 Derivation of `fstream` class

Now consider the following examples:

16.11 Write a program to append a file.

```

#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    char data[25];
    out.open ("text", ios::out);
    cout<<"\n Enter text"<<endl;
    cin.getline(data, 25);
    out <<data;
    out.close();
    out.open ("text", ios::app );
    cout<<"\n Again Enter text"<<endl;
    cin.getline (data, 25);
    out<<data;
    out.close();
    ifstream in;
    in.open("text", ios::in);
    cout<<endl<<"Contents of the file \n";
    while (in.eof()==0)
    {
        in>>data;
        cout<<data;
    }
}
  
```

```
    return 0;
}
```

OUTPUT

Enter text

C-PLUS-

Again Enter text

PLUS

Contents of the file

C-PLUS-PLUS

Explanation: In the above program the file `text` is opened for writing, that is, output. The text read through the keyboard is written in the file. The `close()` function closes the file. Once more, the same file is opened in the append mode, and data entered through the keyboard are appended at the end of the file, that is, after the previous text. The append mode allows the programmer to write data at the end of the file. The `close()` function closes the file. The same file is opened using the object of the `ifstream` class for reading purpose. The `while` loop is executed until the end of the file is detected. The statements within the `while` loop read text from the file and display it on the screen.

16.12 Write a program to read contents of the file. Display the position of the get pointer.

```
#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    char data[32];
    out.open ("text", ios::out);
    cout<<"\n Enter text"<<endl;
    cin.getline(data, 32);
    out <<data;
    out.close();
    ifstream in;
    in.open("text", ios::in);
    cout<<endl<<"Contents of the file \n";
    int r;
    while (in.eof()==0)
    {
        in>>data;
        cout<<data;
        r=in.tellg();
    }
}
```

```

    cout<<" (<<r <<)" ;
}
return 0;
}

```

OUTPUT

Enter text

Programming In ANSI and TURBO-C

Contents of the file

Programming (11)In (14)ANSI (19)and (23)TURBO-C (31)

Explanation: The above program is similar to the previous one. In addition here, the function `tellg()` is used. This function returns the current file pointer position in the number of bytes from the beginning of the file. The number shown in brackets in the output specifies the position of the file pointer from the beginning of the file. The same program is illustrated below using the binary mode.

16.8 MANIPULATORS WITH ARGUMENTS

The `seekp()` and `seekg()` functions can be used with two arguments. Their formats with two arguments are as follows:

```

seekg(offset, pre_position);
seekp(offset, pre_position);

```

The first argument `offset` specifies the number of bytes the file pointer is to be shifted from the argument `pre_position` of the pointer. The `offset` should be a positive or negative number. The positive number moves the pointer in the forward direction, whereas the negative number moves the pointer in the backward direction. Fig 16.11 provides the status of pre-position arguments. The `pre_position` argument may have one of the following values:

- `ios::beg` Beginning of the file
- `ios::cur` Current position of the file pointer
- `ios::end` End of the file

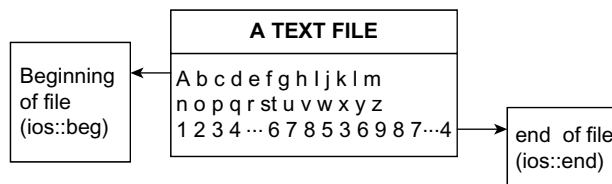


Fig. 16.11 Status of pre-position arguments

In the above figure, the status of `ios::beg` and `ios::end` is shown. The status of `ios::cur` cannot be shown to be similar to `ios::beg` or `ios::end`. The `ios::cur` means the present position of the file pointer. The `ios::beg` and `ios::end` may be referred to as `ios::cur`. Suppose the file pointer is in the middle of the file and you want to read the file from the beginning, you can set the file pointer at the beginning using `ios::beg`. However, if you want to read the file from the current position, you can use the option `ios::cur`.

The `seekg()` function shifts the associated file's input (get) file pointer. The `seekp()` function shifts the associated file's output (put) file pointer. Table 16.3 describes a few pointer offsets along with their working.

Table 16.3 File pointer with its arguments

Seek option	Working
<code>in.seekg(0,ios::beg)</code>	Go to the beginning of file
<code>in.seekg(0,ios::cur)</code>	Rest at the current position
<code>in.seekg(0,ios::end)</code>	Go to the end of file
<code>in.seekg(n,ios::beg)</code>	Shift file pointer to n+1 byte in the file
<code>in.seekg(n,ios::cur)</code>	Go front by n byte from the current position
<code>in.seekg(-n,ios::cur)</code>	Go back by n bytes from the present position.
<code>in.seekg(-n,ios::end);</code>	Go back by n bytes from the end of file

In Table 16.3, `in` is an object of the `ifstream` class.

16.13 Write a program to write text in the file. Read the text from the file from end of file. Display the contents of file in reverse order.

```
#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    char data[25];
    out.open ("text",ios::out);
    cout<<"\n Enter text"<<endl;
    cin.getline(data,25);
    out <<data;
    out.close();
    ifstream in;
    in.open("text", ios::in);
    cout<<endl<<"Reverse Contents of the file \n";
    in.seekg(0,ios::end);
    int m=in.tellg();
```

```

    char ch;
    for (int i=1;i<=m;i++)
    {
        in.seekg(-i,ios::end);
        in>>ch;
        cout<<ch;
    }
    return 0;
}

```

OUTPUT

```

Enter text
Visual_C_+_+
Reverse Contents of the file
+_+_C_lausiV

```

Explanation: In the above program, file `text` is opened in the output mode, and the string entered is written to the file. Again, the same file is opened for reading purpose. The statement `in.seekg (0, ios::end);` moves the get pointer at the end of the file. The `tellg()` function returns the current position of the file pointer in the file. Hence, the file pointer is set to the end of the file. The `tellg()` returns the number of last bytes, that is, the size of the file in bytes, and it is stored in the integer variable `m`. The `for` loop executes from 1 to `m`. The statement `in.seekg (-i, ios::end)` reads the `i`th byte from the end of the file. The statement `in>>ch` reads the character from the file indicated by the file pointer. The `cout` statement displays the read character on the screen. Thus, the contents of the file are displayed in reverse order.

16.14 Write a program to enter a text and again enter a text and replace the first word of the first text with the second text. Display the contents of the file.

```

#include<fstream.h>
#include<conio.h>

int main()
{
    clrscr();
    ofstream out;
    char data[25];
    out.open ("text",ios::out);
    cout<<"\n Enter text"<<endl;
    cin.getline(data,25);
    out <<data;
    out.seekp(0,ios::beg);
    cout<<"\nEnter text to replace the first word of first text:";
}

```

```

    cin.getline(data, 25);
    out<<data;
    out.close();
    ifstream in;
    in.open("text", ios::in);
    cout<<endl<<"Contents of the file \n";
    while (in.eof()!=1)
    { in>>data;
      cout<<data;
    }
    return 0;
}

```

OUTPUT

Enter text

Visual C++

Enter text to replace the first word of first text : Turbo-

Contents of the file

Turbo-C++

Explanation: In the above program, the text is entered and written in the file text. This process is explained in the previous examples. Here again, the statement `out.seekp(0, ios::beg);` sets the file pointer (put pointer) at the beginning of the file. Again, text is entered and written at the current file pointer position. The previous text is overwritten.

16.9 SEQUENTIAL ACCESS FILES

C++ allows the file manipulation command to access the file sequentially or randomly. The data of the sequential file should be accessed sequentially, that is, one character at a time. In order to access the *n*th number of bytes, all previous characters are read and ignored. There are a number of functions to perform read and write operations with the files. Some functions read/write single characters, and some functions read/write blocks of binary data. The `put()` and `get()` functions are used to read or write a single character, whereas `write()` and `read()` are used to read or write blocks of binary data.

`put()` and `get()` functions

The function `get()` is a member function of the class `fstream`. This function reads a single character from the file pointed by the get pointer, that is, the character at the current get pointer position is caught by the `get()` function.

The function `put()` function writes a character to the specified file by the stream object. It is also a member of the `fstream` class. The `put()` function places a character in the file indicated by the put pointer.

16.15 Write a program to write and read string to the file using put () and get () functions.

```
#include<fstream.h>
#include<conio.h>
#include<string.h>

int main()
{
    clrscr();
    char text[50];
    cout<<"\n Enter a Text:";
    cin.getline(text,50);
    int l=0;
    fstream io;
    io.open("data", ios::in | ios::out);
    while (l[text]!='\0')
        io.put(text[l++]);
    io.seekg(0);
    char c;
    cout<<"\n Entered Text:";
    while (io)
    {
        io.get(c);
        cout<<c;
    }
    return 0;
}
```

OUTPUT

Enter a Text : PROGRAMMING WITH C++

Entered Text : PROGRAMMING WITH C++

Explanation: In the above program, the file data is opened simultaneously in the read and write mode. The getline () function reads the string through the keyboard and stores it in the array text [50]. The statement io.put (text[l++]) in the first while loop reads one character from the array and writes it to the file indicated by the stream object io. The first while loop terminates when the null character is found in the text.

The statement io.seekg (0) sets the file pointer at the beginning of the file. In the second while loop, the statement io.get (c) reads one character at a time from the file, and the cout () statement displays the same character on the screen. The while loop terminates when the end of the file is detected.

16.10 BINARY AND ASCII FILES

The insertion and extraction operators known as `stream operators` handle formatted data. The programmer needs to format data in order to represent them in a suitable manner. The description of formatted and unformatted data is given in Chapter 2. ASCII codes are used by the I/O devices to share or pass data to the computer system, but the central processing unit (CPU) manipulates the data using binary numbers, that is, 0 and 1. For this reason, it is essential to convert the data while accepting data from input devices and displaying the data on output devices. Consider the following statements:

```
cout<<k;    // Displays value of k on screen
cin>>k;     // Reads value for k from keyboard
```

Here, `k` is an integer variable. The operator `<<` converts the value of the integer variable `k` into a stream of ASCII characters. In the same manner, the `>>` operator converts the ASCII characters entered by the user into binary form. The data are entered through the keyboard, which is a standard input device. For example, you entered 21. The stream operator `>>` gets ASCII codes of the individual digits of the entered number 21, that is, 50 and 49. The ASCII codes of 2 and 1 are 50 and 49, respectively. The stream operator `>>` converts the ASCII value into its equivalent binary format and assigns it to the variable `k`. The stream operator `<<` converts the value of `k` (21) that is stored in the binary format into its equivalent ASCII codes, that is, 50 and 49. Figure 16.12 shows a representation of integer numbers in ASCII and binary formats.

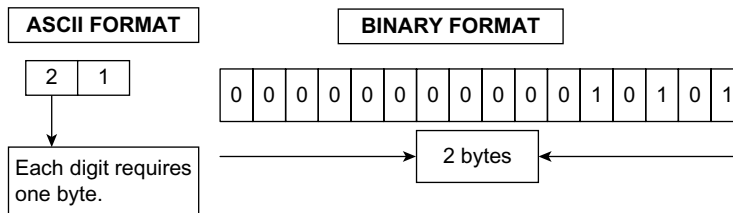


Fig. 16.12 Representation in binary and ASCII formats

16.16 Write a program to demonstrate that the data is read from the file using ASCII format.

```
#include<fstream.h>
#include<constream.h>

int main()
{
    clrscr();
    char c;
    ifstream in("data");
    if (!in)
    {
        cerr<<" Error in opening file.";
    }
}
```

```
    return 1;
}
while (in.eof()==0)
{
    cout<<(char)in.get();
}
return 0;
}
```

OUTPUT

PROGRAMMING WITH ANSI AND TURBO-C

Explanation: In the above program, the data file is opened in read mode. The file already exists. Using `get()` member function of the `ifstream` class, the contents of the file are read and displayed. Consider the following statement:

```
cout<<(char)in.get();
```

The `get()` function reads data from the file in the ASCII format. Hence, it is necessary to convert the ASCII number into an equivalent character. The typecasting format `(char)` converts the ASCII number into an equivalent character. In case the conversion is not done, the output would be as follows:

```
8082797182657777737871328773847232657883733265786832848582667967-1
```

The above displayed are ASCII numbers, and `-1` at the end indicates the end of the file.

After typecasting, the original string will be as shown in the output.

The write() and read() functions

The data entered by the user are represented in the ASCII format. However, the computer can understand only the machine format, that is, 0 and 1. When data are stored in the text, format numbers are stored as characters and occupy more memory space. The functions `put()` and `get()` read/write a character. The data are stored in the file in character format. If a large amount of numeric data are stored in the file, they will occupy more space. Hence, using `put()` and `get()` creates disadvantages.

This limitation can be overcome using `write()` and `read()` functions. The `write()` and `read()` functions use the binary format of data while in operation. In the binary format, the data representation is same in both the file and the system. Figure 16.12 shows the difference between the ASCII and binary format. The bytes required to store an integer in text form depend on its size, whereas in the binary format the size is fixed. The binary form is accurate and allows quick read and write operations, because no conversion takes place during operations. The formats of the `write()` and `read()` function are as given below.

```
in.read((char *) & P, sizeof(P));
out.write((char *) & P, sizeof(P));
```

These functions have two parameters. The first parameter is the address of the variable P. The second is the size of the variable P in bytes. The address of the variable is converted into char type. Consider the following program:

16.17 Write a program to perform read and write operations using write() and read() functions.

```
#include<fstream.h>
#include<conio.h>
#include<string.h>

int main()
{
    clrscr();
    int num[]={100,105,110,120,155,250,255};
    ofstream out;
    out.open("01.bin");
    out.write((char *) & num, sizeof(num));
    out.close();
    for (int i=0;i<7;i++) num[i]=0;
    ifstream in;
    in.open("01.bin");
    in.read((char *) & num, sizeof(num));
    for (i=0;i<7;i++) cout<<num[i]<<"\t";
    return 0;
}
```

OUTPUT

```
100 105 110 120 155 250 255
```

Explanation: In the above program, the integer array is initialized with 7 integer numbers. The file "01.bin" is opened. The statement `out.write((char *) & num, sizeof (num))` writes the integer array in the file. The `&num` argument provides the base address of the array, and the second argument provides the total size of the array. The `close()` function closes the file. Again, the same file is opened for reading purpose. Before reading the contents of the file, the array is initialized to a zero that is not necessary. The statement `in.read ((char *) & num, sizeof (num));` reads data from the file and assigns them to the integer array. The second `for` loop displays the contents of the integer array. The size of the file "01. bin" will be 14 bytes, that is, two bytes per integer. If the above data are stored without using the `write()` command, the size of the file will be 21 bytes.

Reading and writing class objects

The `read()` and `write()` functions perform read and write operations in a binary format that is exactly the same as an internal representation of data in the computer. Due to the capabilities of these functions, large data can be stored in a small amount of memory. Both these functions

are also used to write and read class objects to and from files. During read and write operations, only data members are written to the file, and the member functions are ignored. Consider the following program:

16.18 Write a program to perform read and write operations with objects using write() and read() functions.

```
#include<fstream.h>
#include<conio.h>

class boys
{
    char name [20];
    int age;
    float height;
public:
    void get()
    {
        cout<< "Name:"; cin>>name;
        cout<< "Age:"; cin>>age;
        cout<< "Height:"; cin>>height;
    }
    void show()
    {
        cout<<"\n"<<name<<"\t"<<age <<"\t"<<height;
    }
};

int main()
{
    clrscr();
    boys b[3];
    fstream out;
    out.open ("boys.doc", ios::in | ios::out);
    cout<<"\n Enter following information:\n";
    for (int i=0;i<3;i++)
    {
        b[i].get();
        out.write ((char*) & b[i],sizeof(b[i]));
    }
    out.seekg(0);
    cout<<"\n Entered information\n";
    cout<<"Name Age Height";
    for (i=0;i<3;i++)
    {
        out.read((char *) & b[i], sizeof(b[i]));
        b[i].show();
    }
}
```



```

    }
    out.close();
    return 0;
}

```

OUTPUT

Enter following information:

Name : Kamal

Age : 24

Height : 5.4

Name : Manoj

Age : 24

Height : 5.5

Name : Rohit

Age : 21

Height : 4.5

Entered information

Name Age Height

Kamal 24 5.4

Manoj 24 5.5

Rohit 21 4.5

Explanation: In the above program, the class `boys` contains data members' `name`, `age`, and `height` of `char`, `int`, and `float` type. The class also contains the member functions `get()` and `show()` to read and display the data. In function `main()`, an array of three objects is declared, that is, `b [3]`. The file "`boys.doc`" is opened in the output and input mode to write and read data. The first `for` loop is used to call the member function `get()`, and data read via the `get()` function is written to the file by the `write()` function. The same method is repeated while reading the data from the file. While reading data from the file, the `read()` function is used, and the member function `show()` displays the data on the screen.

16.11 RANDOM ACCESS OPERATION

Data files always contain a large amount of information, and the information always changes. The changed information should be updated; otherwise, the data files are not useful. Thus, to update data in the file, we need to update the data files with latest information. To update a particular record of the data file, the data may be stored anywhere in the file; it is necessary to obtain the location (in terms of byte number) at which the data object is stored.

The `sizeof()` operator determines the size of the object. Consider the following statements:

```
(a) int size = sizeof(o);
```

Here, `o` is an object, and `size` is an integer variable. The `sizeof()` operator returns the size of the object `o` in bytes, and it is stored in the variable `size`. Here, one object is equal to one record.

The position of the n th record or object can be obtained using the following statement:

$$(b) \text{ int } p = (n-1 * \text{size});$$

Here, p is the exact byte number of the object that is to be updated; n is the number of the object; and size is the size in bytes of an individual object (record).

Suppose we want to update the fifth record. The size of the individual object is 26.

$$(c) \text{ p} = (5-1*26) \text{ i.e. } p = 104$$

Thus, the fifth object is stored in a series of bytes from 105 to 130. Using `seekg()` and `seekp()` functions, we can set the file pointer at that position.

16.19 Write a program to create a text file. Add and modify records in the text file. The record should contain name, age, and height of a boy.

```
#include<stdio.h>
#include<process.h>
#include<fstream.h>
#include<conio.h>

class boys
{
    char name [20];
    int age;
    float height;
public:
    void input()
    {
        cout<< "Name:"; cin>>name;
        cout<< "Age:"; cin>>age;
        cout<< "Height:"; cin>>height;
    }
    void show (int r)
    {
        cout<<"\n"<<r<<"\t"<<name<<"\t"<<age <<"\t"<<height; }
    };
    boys b[3];
    fstream out;
    void main()
    {
        clrscr();
        void menu (void);
        out.open ("boys.doc", ios::in | ios::out | ios::noreplace);
        menu();
    }
    void menu(void)
```

```
{
void get(void);
void put(void);
void update(void);
int x;
clrscr();
cout<<"\n Use UP arrow key for selection";
char ch=' ';
gotoxy(1,3);
printf ("ADD()");
gotoxy(1,4);
printf ("ALTER()");
gotoxy(1,5);
printf ("EXIT()");
x=3;
gotoxy(7,x);
printf ("*");
while (ch!=13)
{
ch=getch();
if (ch==72)
{
if (x>4)
{
gotoxy(7,x);
printf (" ");
x=2;
}
gotoxy(7,x);
printf (" ");
gotoxy(7,++x);
printf ("*");
}
}
switch(x)
{
case 3 : get(); put(); getche(); break;
case 4 : put(); update(); put(); getche(); break;
default : exit(1);
}
menu();
}
```

```
void get()
{
    cout<<"\n\n\n\n Enter following information:\n";
    for (int i=0;i<3;i++)
    {
        b[i].input();
        out.write ((char*) & b[i],sizeof(b[i]));
    }
}
void put()
{
    out.seekg(0,ios::beg);
    cout<<"\n\n\n Entered information \n";
    cout<<"Sr.no Name Age Height";
    for (int i=0;i<3;i++)
    {
        out.read((char *) & b[i],
        sizeof(b[i]));
        b[i].show(i+1);
    }
}
void update()
{
    int r, s=sizeof(b[0]);
    out.seekg(0,ios::beg);
    cout<<"\n"<<"Enter record no. to update:";
    cin>>r;
    r=(r-1)*s;
    out.seekg(r,ios::beg);
    b[0].input();
    out.write ((char*) & b[0],sizeof(b[0]));
    put();
}
```

OUTPUT

Use UP arrow key for selection

ADD (*)

ALTER()

EXIT()

Enter following information :

Name : Sachin

Age : 28

```

Height : 5.4
Name : Rahul
Age : 28
Height : 5.5
Name : SauravAge : 29
Height : 5.4
Entered information
Sr.no Name Age Height
1 Sachin 28 5.4
2 Rahul 28 5.5
3 Saurav 29 5.4

```

Explanation: In the above program, the class `boys` contains the data member's name, age, and height. The class `boys` also contains the member functions `input()` and `show()`. The `input()` function is used to read data, and the `show()` function displays data on the screen.

After class definition and before the `main()` function array of objects, `b[3]` and `fstream` object `out` are declared. They are declared before `main()` for global access. The file `boys.doc` is opened in the input and output modes to perform both read and write operations.

The `menu()` function displays the menu on the screen. The menu items can be selected using the up arrow key. Hit enter to start the operation. There are another three user-defined functions. They are `get()`, `put()`, and `update()`. The `get()` function calls the member function `input()` to read data through the keyboard. The `get()` function writes the data using the `write()` function. The `put()` function calls the member function `show()`. The `put()` function calls the member function `show()`. The `put()` function reads the data from the file using the `read()` function.

The `update()` function is used to modify the previous record. The `seekg()` function sets the file pointer at the beginning of the file. The `sizeof()` operator determines the size of the object and stores it in the variable `s`.

16.12 ERROR HANDLING FUNCTIONS

Until now, we have performed the file operation without any knowledge of the failure or success of the function `open()` that opens the file. There are many reasons; they may result in errors during read/write operations of the program.

- (1) An attempt to read a file that does not exist
- (2) The file name specified for opening a new file may already exist
- (3) An attempt to read the contents of a file when the file pointer is at the end of the file
- (4) Insufficient disk space
- (5) Invalid file name specified by the programmer
- (6) An effort to write data to the file that is opened in the read-only mode
- (7) A file opened may be already opened by another program
- (8) An attempt to open the read-only file for writing operation
- (9) Device error

The `stream` state member from the class `ios` receives values from the status bit of the active file. The class `ios` also contains many different member functions. These functions read the status bit of the file where an error occurred during program execution are stored. These functions are depicted in Table 16.5, and various status bits are described in Table 16.4.

All streams such as `ofstream`, `ifstream`, and `fstream` contain the state connected with them. Faults and illegal conditions are managed (controlled) by setting and checking the state properly. Figure 16.13 describes it more clearly.

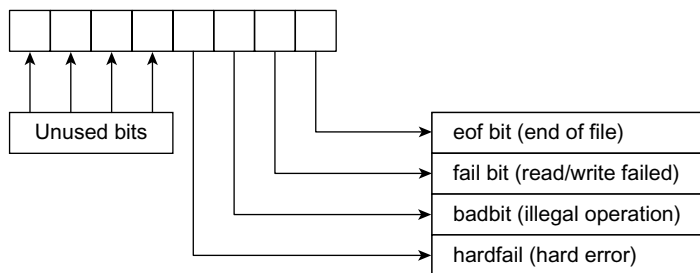


Fig. 16.13 Status bits

Table 16.4 Status Bits

eofbit	End of file encountered.	0x01
failbit	Operation unsuccessful	0x02
badbit	Illegal operation due to wrong size of buffer	0x04
hardfail	Critical error	0x08

Table 16.5 Error trapping functions

Functions	Working and return value
fail()	Returns non-zero value if an operation is unsuccessful. This is carried out by reading the bits <code>ios::failbit</code> , <code>ios::badbit</code> , and <code>ios::hardfail</code> of <code>ios::state</code> .
eof()	Returns non-zero value when the end of the file is detected; otherwise, it returns zero. The <code>ios::eofbit</code> is checked.
bad()	Returns non-zero value when an error is found in the operation. The <code>ios::badbit</code> is checked.
good()	Returns non-zero value if no error occurred during the file operation, that is, no status bits were set. This also indicates that the above functions are false. When this function returns true, we can proceed with the file operation.
rdstate()	Returns the stream state. It returns the value of various bits of the <code>ios::state</code> .

The following examples illustrate the techniques of error checking:

(a) An attempt to open a non-existent file for reading

```
ifstream in("data.txt");
if (!in){    cout<< "File not found";    }
```

In the above format, an attempt is made to open a file for reading. If the file already exists, it will be opened; otherwise, the operation fails. Thus, by checking the value of the object `in`, we can confirm the failure or success of the operation and according to this, further processing can be decided.

(b) An attempt to open a read-only file for writing

```
ofstream out("data.txt");
if (!out)
    cout<<"Unable to open file";
else
    cout<<"File opened";
```

Suppose the `data.txt` file is protected (marked read only) or used by another application in a multitasking operating environment. If the same file is opened in the write mode as shown above, the operation fails. By checking the value of the object `out` with the `if()` statement, we can catch the error and transfer the program control to a suitable sub-routine.

(c) Checking end of file

```
ifstream in("data.txt");
while (!in.eof())
{
    // read data from file
    // display on screen
}
```

We may seek to open an existing file and read its contents. After opening a file in the read mode, it is necessary to read the characters from the file using an appropriate function (`read()` or `get()`). While reading a file, the `get` pointer is advanced to the successive characters, and the same process can be repeated using loops. The compiler cannot determine the end of the file. The `eof()` function determines the end of the file. Thus, by checking the value of the `eof()` function, we can determine the end of the file. In addition, by checking the value of the object, the end of the file is determined. Such conditions should be placed in the `while` loop parentheses. While reading the file, use only `while` or `for` loop.

(d) Illegal file name

```
ifstream in("*+**");
while (!in.eof())
{
    // read data from file
    // display on screen
}
```

While performing a file operation, it is the user's responsibility to specify the correct file name. If an illegal file name is specified by the user, the file operation fails. In the above format, `"*+**"` is given as a file name that is invalid.

(e) Operation with unopened file

```
ifstream in("DATA");
while (!in.eof())
{
    // read data from file
    // display on screen
}
```

Suppose the “DATA” file does not exist and an attempt is made to open it for reading. Any operation applied with this file will be of no use. Hence, while performing a file operation, first we have to check whether the file is successfully opened or not. After the confirmation, we can proceed to the next step.

Programs referred to in the above discussion are explained below.

16.20 Write a program to detect whether the file is opened successfully or not.

```
#include<fstream.h>
#include<constream.h>
#include<string.h>
ifstream in; // Global object

void main()
{
    clrscr();
    void show (void);
    in.open("dat") ;
    char c;
    if (in!=0) show();
    else
        cout<<"\n File not found";
}

void show()
{
    char c;
    cout<<"\n Contents of file:";
    while (in)
    {
        in.get(c);
        cout<<c;
    }
}
```

OUTPUT

File not found

Explanation: In the above program, the object `in` of the `ifstream` class is declared globally. It can be accessed by any normal function. In the `main()` function, `in` is the object used for opening the file. If the `open()` function fails to open the file, it returns a zero; otherwise, it returns a non-zero value. The `if` statement checks the value of the object `in`, and if it is a non-zero `show()` function, it is invoked; otherwise, “File not found” message is displayed. The `show()` function reads the file and displays the contents on the screen. In the above program, the `open()` function tries to open “dat” file, which does not exist. Hence, the output is “File not found.” If the specified file exists, the contents of the file will be displayed.

16.21 Write a program to display status of various errors trapping functions.

```
#include<fstream.h>
#include<conio.h>

void main()
{
    clrscr();
    ifstream in;
    in.open ("text.txt", ios::nocreate);
    if (!in)
        cout<<"\n File not found";
    else
        cout<<"\nFile="<<in;
        cout<<"\nError state="<<in.rdstate();
        cout<<"\ngood()="<<in.good();
        cout<<"\neof()="<<in.eof();
        cout<<"\nfail()="<<in.fail();
        cout<<"\nbad()="<<in.bad();
        in.close();
}
```

OUTPUT

```
File not found
Error state = 4
good() = 0
eof() = 0
fail() = 4
bad() = 4
```

Explanation: In the above program, an attempt is made to open a non-existent file. The `if` statement checks the value of the object `in`. The specified file does not exist; hence, it displays the message “File not found.” The program also displays the values of various bits using the functions `good()`, `eof()`, `bad()`, `fail()`, and `rdstate()` error trapping functions. For more information about these functions, please refer Table 16.4.

16.13 COMMAND-LINE ARGUMENTS

An executable program that performs a specific task for the operating system is called a `command`. The commands are issued from the command prompt of the operating system. Some arguments are associated with the commands; hence these arguments are called `command-line arguments`. These associated arguments are passed to programs.

Similar to C, in C++, every program starts with a `main()` function, and this function marks the beginning of the program. We have not provided any arguments so far in the `main()` function. Here, we can make arguments in the main function as in other functions. The `main()` function can receive two arguments, and they are (1) `argc` (argument counter) and (2) `argv` (argument vector). The first argument contains the number of arguments, and the second argument is an array of char pointers. The `*argv` points to the command-line arguments. The size of the array is equal to the value counted by the `argc`. The information contained in the command line is passed on to the program through these arguments when the `main()` is called up by the system.

- (1) **Argument `argc`:** The argument `argc` counts the total number of arguments passed from command prompt. It returns a value that is equal to the total number of arguments passed through the `main()`.
- (2) **Argument `argv`:** It is a pointer to an array of character strings that contains names of arguments. Each word is an argument.

Syntax - `main ( int argc, char * argv[] );`
Example - `ren file1 file2.`

Here, `file1` and `file2` are arguments, and `copy` is a command. The first argument is always an executable program followed by associated arguments. If you do not specify the argument, the first program name itself is an argument but the program will not run properly and will flag an error. The contents of `argv []` would be as follows:

```
argv [0] → ren
argv [1] → file1
argv [2] → file2
```

16.22 Write a program to simulate rename command using command line arguments.

```
#include<stdio.h>
#include<fstream.h>
#include<conio.h>
#include<process.h>

main(int argc, char *argv[])
{
    fstream out;
    ifstream in;
```

```

if (argc<3 )
{
cout<<"Insufficient Arguments";
exit(1);
}
in.open(argv[1],ios::in | ios::nocreate);
if (in.fail())
{
cout<<"\nFile Not Found";
exit(0);
}
in.close();
out.open(argv[2],ios::in | ios::nocreate);
if (out.fail())
{ rename(argv[1],argv[2]); }
else
cout<<"\nDuplicate file name or file is in use.";
return 0;
}

```

Explanation: In the above program, the `main()` receives two file names. The existence of the file can be checked by opening it in the read mode. If the file does not exist, the program is terminated. On the other hand, if the second file exists, the renaming operation cannot be performed. The renaming operation is carried out only when the first file exists and the second file does not exist. Make exe file of this program and use it on the command prompt.

16.14 STRSTREAMS

ostrstream

The `strstream` class is derived from the `istrstream` and `ostrstream` classes. The `strstream` class works with the memory. Using the object of the `ostrstream` class, different types of data values can be stored in an array.

16.23 Write a program to demonstrate use of ostrstreams object.

```

#include<strstream.h>
#include<iomanip.h>
#include<conio.h>

main()
{
    clrscr();
    char h='C';

```

```

int j=451;
float PI=3.14152;
char txt[]="applications";
char buff[70];
ostream o (buff,70);
o<<endl <<setw(9)<<"h="<<h<<endl <<setw(9)
    <<"j="<<oct<<j<<endl
<<setw(10)<<"PI="<<setiosflags(ios::fixed)<<PI<<endl
    <<setw(11)
<<"txt="<<txt <<ends;
cout<<o.rdbuf();
return 0;
}

```

OUTPUT

```

h=C
j=703
PI=3.14152
txt= applications

```

Explanation: The `stringstream` deals with the memory. If we want to pick characters from an `stringstream` or we want to add characters into the `stringstream`, this can be done by creating `istringstream` and `stringstream` objects. When the object `o` is created, the constructor of the `stringstream` is executed. Once an object of the `stringstream` is created, we can assign any formatted text to the array associated with it. The statement `cout<<o.rdbuf()` displays the formatted information on the screen.

istringstream

It is one of the base classes of the `stringstream` class. Using the object of the `istringstream` class, data can be extracted from an array. Suppose a character array contains numbers and characters. You can extract a number from an array and assign it to an integer variable. Similarly, other values can be extracted from an array. The following program illustrates this:

16.24 Write a program to demonstrate the use of `istringstream`.

```

#include<sstream.h>
#include<conio.h>

main()
{
    clrscr();
    char *book;
    int pages;

```

```

float price;
char *text="550 175.75 C++";
istream o(text);
o>>pages>>price>>book;
cout<<endl <<pages <<endl <<price<<endl<book;
cout<<o.rdbuf();
return 0;
}

```

OUTPUT

```

550
175.75
C++

```

Explanation: The `istream` is the opposite of the `ostream`. It picks different types of data from an array. In the above program, a character pointer `text` contains the data of integer, float, and character type. Using the object of the `istream` class, we can separate the contents and store them in appropriate variables. The `*book` pointer variable displays a string. The remaining contents are displayed by the function `rdbuf()`.

16.15 SENDING OUTPUT TO DEVICES

It is also possible to send information of files directly to devices such as a printer or monitor. Table 16.6 describes various devices along with their names and descriptions. The following program illustrates the use of such devices in the program:

Table 16.6 Standard Devices

Device Name	Description
CON	Console (monitor screen)
COM1 or AUX	Serial port – I
COM2	Serial port – II
LPT1 OR PRN	Parallel printer – I
LPT2	Parallel printer – II
LPT3	Parallel printer – III
NUL	Dummy device

16.25 Write a program to read a file and sent data to the printer.

```

#include<fstream.h>
#include<iostream.h>
#include<conio.h>

```

```

#include<process.h>
#define eject out.put('\x0C');

void main()
{
    clrscr();
    char h;
    char name[20];
    cout<<"Enter file name:";
    cin>> name;
    ifstream in (name);
    if (!in)
    {
        cerr <<endl<<"File opening error";
        _cexit();
    }
    ofstream out ("LPT1");
    if(!out)
    {
        cerr <<endl<<"device opening error";
        _cexit();
    }
    while (in.get(h) !=0)
        out.put(h);
    eject;
}

```

Explanation: In the above program, the file name is entered through the keyboard, and it is opened for reading purpose. The `ifstream` object `in` opens the file. The `ofstream` object `out` activates the printer. The `if` statements check both the objects for detecting operation status, that is, whether the operations have failed or are successful. The `while` loop reads data from the file and using the `put()` statement, it passes it to the devices associated with the object `out`. In this program, the data read are passed to the printer. The macro `eject` defined at the beginning of the program advances the page of the printer. In case the printer is not attached, the message displayed will be as given below.

Error Message
System Message
Error accessing LPT1 device
» Retry « Cancel

The user can select `retry` if he or she had attached the printer; otherwise, by selecting `cancel`, the operation can be cancelled.

16.16 MORE PROGRAMS

16.26 Write a program to copy contents of one file to another file.

```
#include<fstream.h>
#include<conio.h>
#include<process.h>

main()
{
    clrscr();
    char s[12],t[12],c;
    fstream out;
    ifstream in;
    cout<<"\n Enter a Source file name:";
    cin>>s;
    cout<<"\n Enter a target file name:";
    cin >>t;
    in.open(s,ios::in | ios::nocreate);
    if (in.fail())
    {
        cout<<"\nFile "<<s <<" Not Found";
        exit(1);
    }
    out.open(t,ios::out | ios::nocreate);
    if (out.fail())
    {
        out.open(t,ios::out);
        while (in.eof()==0)
        {
            in.get(c);
            out.put(c);
        }
    }
    else
        cout<<"\n Target file already exist.";
    in.close();
    out.close();
    return 0;
}
```

OUTPUT

Enter a Source file name : DATA
Enter a target file name : TEXT

Explanation: In the above program, the user enters source and target file names. The existence of the files is checked. In case the source file is absent and the target file is already present, the copying of data will not take place. Appropriate messages are displayed when the file names are not properly entered.

The target file is opened in read and nocreate mode. The nocreate flag prevents the opening of a new file if the file is absent. If it is unsuccessful, then the `open()` statement within the `if` statement opens the file for writing. The `while` loop executes the function till the file pointer reaches the end of the source file. The `get()` statement reads data from the source file, and the `put()` statement writes the read data to the target file. Thus, the copying of data is carried out. After termination of the `while` loop, both the files are closed using `close()` functions.

16.27 Write a program to copy content of one file in another file in reverse order. Display the contents of the screen.

```
#include<fstream.h>
#include<conio.h>
#include<process.h>

main()
{
    clrscr();
    char s[12],t[12],c;
    fstream out;
    ifstream in;
    cout<<"\n Enter a Source file name:";
    cin>>s;
    cout<<"\n Enter a target file name:";
    cin >>t;
    in.open(s,ios::in ); //| ios::nocreate);
    if (in.fail())
    {
        cout<<"\nFile"<<s <<" Not Found";
        exit(1);
    }
    else
    in.seekg(0,ios::end);
    out.open(t,ios::out | ios::nocreate);
    int b;
    if (out.fail())
    {
        out.open(t,ios::out);
        in.seekg(0,ios::end);
        b=in.tellg();
        for (int i=1;i<=b;i++)
```



```
    ifstream in ("data.txt"); // opens file for reading
    in>>name >>age>>ht; // reads data from file and assigns to
variables
    cout<<endl<<"Name:"<<name; // display data on the screen
    cout<<endl<<"Age:"<<age;
    cout<<endl<<"Height:"<<ht;
}
```

OUTPUT

Name : SANJAY

Age : 25

Height : 4.5

Explanation: In the above program, the file "data.txt" is opened in the write mode. The values of the variable's name, age, and ht are written to the file. The file is closed using the close() function.

Again, the same file is opened in the read mode. The data read are assigned to respective variables and displayed on the screen using the cout () statement.

16.29 Write a program to write data to the file in string format also read and display the data in the same fashion.

```
#include<fstream.h>
#include<conio.h>

void main()
{
    clrscr();
    char text[100];
    ofstream out ("data.txt");
    out<<" Programming with ANSI and Turbo C";
    out<<"\n Teaches you C with practical programs";
    out.close();
    ifstream in ("data.txt");
    while (!in.eof())
    {
        in.getline(text,100);
        cout<<endl<<text;
    }
}
```

OUTPUT

Programming with ANSI and Turbo C
Teaches you C with practical programs

Explanation: This program is similar to the previous one. Here, a string is written to the file. Using the `getline()` function, the string is read from the file and displayed. As soon as the end of the file is detected, the `while` loop terminates.

16.30 Write a program to copy contents of one file to another file. Use `rdbuf()` function.

```
#include<fstream.h>
#include<conio.h>

void main()
{
    clrscr();
    char sfile[20], dfile[20];
    cout<<"\nEnter source file:";
    cin>>sfile;
    cout<<"\nEnter destination file:";
    cin>>dfile;
    ifstream in(sfile);
    ofstream out(dfile);
    out<<in.rdbuf();
    in.close();
    out.close();
}
```

OUTPUT

Enter source file : data.txt
Enter destination file : c.txt

Explanation: In the above program, two character arrays `sfile [20]` and `dfile [20]` are declared to hold source and destination file names, respectively. The `rdbuf()` copies entire the source file to the target file.

16.31 Write a program to display strings and their addresses.

```
#include<sstream.h>
#include<conio.h>

main()
{
    clrscr();
    char s[]="Sunday";
    char *r="Monday";
    cout<<"Strings"<<endl;
    cout<<s <<endl;
    cout<<r <<endl;
    cout<<endl<<"Addresses"<<endl;
```

```

    cout<<&s<<endl; // c style
    cout<<(void*) r<<endl;
    cout<<(unsigned) &s<<endl;
    return 0;
}

```

OUTPUT

Strings

Sunday

Monday

Addresses

0x8f9cffee

0x8f9c00b1

65518

Explanation: In the above program, character array `s[]` and character pointer `r` are initialized with strings. The strings and their memory addresses are displayed using `cout()` statements. The address can be displayed using the `&` operator in traditional C style. We can also display the address as per the statement `cout<<(void*) r`. The address is converted from `char*` into `void*`. The statement `cout<<(unsigned) &s` converts the address into an unsigned integer and displays it.

16.32 Write a program to show errors occurring during file opening operations.

```

#include<fstream.h>
#include<conio.h>
#include<process.h>

void main()
{
    clrscr();
    void errors (ofstream &);
    ofstream f;
    f.open ("data.txt",ios::noreplace);
    if (!f) errors(f);
    else f <<"Hope is a walking dream";
    f.close();
}

void errors (ofstream &f)
{
    cout<<endl<<"File opening errors";
    cout<<endl <<"Error state="<<f.rdstate();
    cout<<endl<<"fail()="<<f.fail();
    cout<<endl<<"eof()="<<f.eof();
    cout<<endl<<"bad()="<<f.bad();
}

```

```

    cout<<endl<<"good()="<<f.good();
    _cexit();
}

```

OUTPUT

File opening errors

Error state = 4

fail()=4

eof() = 0

bad()=4

good()=0

Explanation: In the above program, the file `data.txt` is in the write mode. If the file opening operation fails, the function `error()` is executed. The `if` statement checks the value of the object `f`, and if it is zero, then the function `errors()` are invoked, which display the errors. The error states displayed in the output are illustrated in Table 16.7:

Table 16.7 Return values of functions

<code>rdstate()</code> (error state = 4)	The <code>rdstate()</code> function gives the value 4, and it points that the file operation was unsuccessful.
<code>fail()=4</code> & <code>bad()=4</code>	These functions display a non-zero value, and this is due to an error that is generated during the operation.
<code>eof() = 0</code>	This function returns zero, because the file pointer is not at the end of the file.
<code>good() = 0</code>	This function returns zero, because no bit sets.

16.33 Write a program to enter numbers using command line arguments. Calculate the product of all the numbers.

```

#include<sstream.h>
#include<conio.h>

void main (int argc, char *argv[])
{
    int k=1;
    long n,s=1;
    if (argc<2)
    {
        cout<<" Enter numbers ";
        return ;
    }
    while (--argc)
    {
        istringstream value (argv[k]);
        value>>n;
        s*=n;
    }
}

```

```

    k++;
}
cout<<endl<<" Multiplication of entered numbers:"<<s<<endl;
}

```

OUTPUT

C:\tc3>cmd 4 4 4

Multiplication of entered numbers : 64

C:\tc3>cmd 45545 2

Multiplication of entered numbers : 91090

Explanation: The above program can be executed on command prompt by creating its exe file. The number of arguments entered by the user is checked by the first `if` statement; if arguments are less than two, the message displayed will be "Enter numbers."

If the user enters numbers followed by the file name, the product of all the numbers is calculated and displayed. The object `value` is the object of the class `istream`, and it is connected to the buffer. Consider the statement `value>>n`; here, the object `value` takes data from the buffer and assigns it to the variable `n`.

SUMMARY

- (1) Nowadays, a huge amount of data is processed in the computer networking. The information is uploaded or downloaded from the desktop computer. The information transfer in computer networking in day-to-day life is done in the form of files. The data are saved in the file on the disk. The file is an accumulation of data stored on the disk.
- (2) A stream is nothing but the flow of data. In the object-oriented programming, the streams are controlled using the classes.
- (3) The `istream` and `ostream` classes control the input and output functions, respectively.
- (4) The `iostream` class is also a derived class. It is derived from the `istream` and `ostream` classes. There are another three useful derived classes. They are `istream_withassign`, `ostream_withassign`, and `iostream_withassign`, and they are derived from `istream`, `ostream`, and `iostream`, respectively.
- (5) `filebuf` accomplishes input and output operations with the files. The `fstream`-base acts as a base class for `fstream`, `ifstream`, and `ofstream`. The `ifstream` class is derived from the `fstreambase` and `istream` classes by multiple inheritance. It can access the member functions such as `get()`, `getline()`, `seekg()`, `tellg()`, and `read()`. The `ofstream` class is derived from the `fstreambase` and `ostream` classes. It can access the member functions such as `put()`, `seekp()`, `write()`, and `tellp()`. The `fstream` class allows for simultaneous input and output on a `filebuf`. The member function of the `istream` and `ostream` in the base classes starts the input and output, respectively.
- (6) The following two methods are used for the opening of a file: (A) Constructor of the class; (b) Member function `open()` of the class.

- (7) The class `ofstream` creates output stream objects, and the class `ifstream` creates input stream objects.
- (8) The `close()` member function closes the file.
- (9) The `open()` function uses the same stream object. The `open()` function has two arguments. The first is the file name, and the second is the mode.
- (10) When the end of the file is detected, the process of reading data can be easily terminated. The `eof()` function() is used for this purpose. The `eof()` stands for the end of the file. It is an instruction given to the program by the operating system that the end of the file is reached. The `eof()` function returns one when the end of the file is detected.
- (11) The mode `ios::out` and `ios::trunc` are near about same. The `ios::app` lets the user add data at the end of the file, whereas `ios::ate` allows the user to add or update data anywhere in the file.
- (12) The `seekg()` function shifts the associated file's input (get) file pointer. The `seekp()` function shifts the associated file's output (put) file pointer.
- (13) `seekg()` -- Shifts input (get) pointer to a given location
- (14) `seekp()` -- Shifts output (put) pointer to a given location
- (15) `tellg()` -- Provides the present position of the input pointer
- (16) `tellp()` -- Provides the present position of the output pointer
- (17) The `put()` and `get()` functions are used to read or write a single character, whereas the `write()` and `read()` functions are used to read or write blocks of binary data.
- (18) The `fail()`, `eof()`, `bad()`, and `good()` are error trapping functions.
- (19) An executable program that performs a specific task for the operating system is called a command. The commands are issued from the command prompt of the operating system. Some arguments are to be associated with the commands; hence, these arguments are called command-line arguments.
- (20) Syntax - `main ( int argc, char * argv[]);`

EXERCISES

(A) Answer the following questions

- (1) What is a stream?
- (2) What is a file?
- (3) Describe the different classes derived from `ios` that control the disk I/O operations.
- (4) Describe the two methods of opening a file.
- (5) Explain the detection of the end of a file with the function `eof()`.
- (6) Describe the syntax of the `open()` function with its arguments.
- (7) What are the different types of file opening modes? List their names along with their meanings.
- (8) Describe file manipulators with their syntaxes.
- (9) Describe the various error trapping functions.
- (10) What are the possible reasons for the failure of the `open()` function?
- (11) What are command-line arguments?
- (12) Explain the use of the NOT operator and `eof()` function.
- (13) Explain sequential and random file operations.
- (14) How would you write data in a file in binary format?
- (15) What are the limitations of using the `put()` and `get()` functions?
- (16) Explain the uses of the `ostream` and `istream` classes.
- (17) Explain the differences between binary and ASCII files.

(B) Answer the following by selecting the appropriate option

- (1) The `eof()` stands for
 (a) **end of the file**
 (b) error opening the file
 (c) error of the file
 (d) none of the above
- (2) Command-line arguments are used with function
 (a) **main()**
 (b) member function
 (c) with all functions
 (d) none of the above
- (3) The statement `in.seekg(0,ios::end)` sets the file pointer
 (a) **at the end of the file**
 (b) at the beginning of the file
 (c) in the middle of the file
 (d) none of the above
- (4) The `close()` function
 (a) **closes the file**
 (b) closes all opened files
 (c) closes only read mode file
 (d) none of the above
- (5) The `write()` function writes
 (a) **single character**
 (b) an object
 (c) a string
 (d) none of the above
- (6) What will be the value of the variable `c` after execution of the following program?

```
#include<fstream.h>
void main()
{
    char text[5];
    ofstream out ("data.
    txt");
    out<<"Programming with
    ANSI and Turboc C";
    out.close();
    ifstream in ("data.txt");
    int c=0;
    while (!in.eof())
    { in.getline(text,5);
      c++; }
}
```

 (a) **c = 9**
 (b) c = 2
 (c) c = 5
 (d) c = 1
- (7) What will be the values of the variables `j` and `k` after execution of the following program?

```
#include<fstream.h>
void main()
{
    ofstream j;
    ifstream k;
    cout<<"j="<<sizeof(j)
    <<endl;
    cout<<"k="<<sizeof(k);
}
```

 (a) **j = 78 k = 80**
 (b) j = 80 k = 80
 (c) j = 80 k = 78
 (d) none of the above
- (8) The object of the `fstream` class provides
 (a) **both read and write operations**
 (b) only read operation
 (c) only write operation
 (d) none of the above
- (9) To add data at the end of the file, the file should be opened in
 (a) **append mode**
 (b) read mode
 (c) write mode
 (d) both (a) and (c)
- (10) When a file is opened in read or write mode, the file pointer is set
 (a) **at the beginning of the file**
 (b) at the end of the file
 (c) in the middle of the file
 (d) none of the above
- (11) While performing a file operation, the following file should be included:
 (a) **fstream.h**
 (b) iostream.h
 (c) cstream.h
 (d) all of the above
- (12) The constructor of the following class requires a file name and mode for opening a file:
 (a) **ifstream**
 (b) ofstream
 (c) fstream
 (d) all of the above

(C) Attempt the following programs

- (1) Write a program to open a file in the output and input mode. Accept data and write to the file. Display the contents of the file.
- (2) Write a program to write and read data in a file using object I/O functions `write()` and `read()`. Declare a class with the data member's name [20], int billno, int amount_debited, and int received _amount and int balance. Add 10 records and display the list of people with their balances. The user should have a facility to modify the existing records.
- (3) Write a program to read and write a data file. While writing data, pay attention to the following instructions:
 - (a) Add one space between two successive words.
 - (b) Capitalize the first character of sentences.
 - (c) If numerical data are present, enclose them within brackets.
- (4) Write a program to copy the contents of one file to another file. Use command-line arguments.
- (5) Write a program to read a file. Display the total number of characters, words, lines, and blank spaces in the file.
- (6) Write a program to exchange the contents of two files. Use command-line arguments.
- (7) Write a program to compare two files in terms of the number of bytes.
- (8) Write a program to count the characters and numerals present in a file.
- (9) Write a program to print the contents of a file on a printer.
- (10) Write a program to check whether the given file exists or not. Display a suitable message. The user should enter the file name.
- (11) Write a program to write the contents of one file in reverse order into another file.
- (12) Write a program to find the length of a file.

(D) Find the bugs in the following programs

```
(1)
ifstream in;
void main()
{
    ifstream in;
    void show (void);
    in.open("data") ;
    if (in!=0) show();
    else cout<<"\n File not
                                fount";
}
void show()
{ char c;
  cout<<"\n Contents    of
                        file:";
  while (in) {in.get(c);
    cout<<c;}
```

```
}
Tip: Assume the file "data"
exists.
```

```
(2)
void main()
{
    char name[15];
    ofstream out("text");
    cin >>name;
    out<<name<<"\t";
    ifstream in("text");
    in>>name;
    cout<<"\nName\
t:"<<name<<"\n";
    in.close();
}
```

(3)

```
void main()
{
    ifstream in("data");
    while (in) { cout<<(char)
        in.get(); }
    return 0;
}
```

Tip: Assume the file "data" exists.

(4)

```
void main()
{
    fstream in("data");

    while (in.eof()==0)
    {
        cout<<(char)in.get();
    }
}
```

This page is intentionally left blank.

Generic Programming with Templates

17

CHAPTER

CHAPTER OUTLINE

- 17.1 Introduction
- 17.2 Need for Templates
- 17.3 Definition of a Class Template
- 17.4 Normal Function Templates
- 17.5 Working of Function Templates
- 17.6 Class Templates with More Parameters
- 17.7 Function Templates with More Arguments
- 17.8 Overloading of Template Functions
- 17.9 Member Function Templates
- 17.10 Recursion with Template Functions
- 17.11 Class Templates with Overloaded Operators
- 17.12 Class Templates Revisited
- 17.13 Class Templates and Inheritance
- 17.14 Bubble Sort Using Function Templates
- 17.15 Guidelines for Templates
- 17.16 Differences between Templates and Macros
- 17.17 Linked List with Templates
- 17.18 More Programs

17.1 INTRODUCTION

The template is one of the most useful characteristics of the C++. A few old compilers do not allow the template mechanism. Templates are a part of ANSI C++ standard. All major compilers support templates in their new versions. Instantiation is the activity of creating a particular type using a template. The specific classes are known as instance of the template. The template provides generic programming by defining generic classes. In templates, generic data types are used as arguments, and they can handle a variety of data types. A function that works for all C++ data types is called a generic function. Templates help the programmer declare groups of functions or classes. When used with functions, they are called function templates. For example, we can create a template for the function, `square()`. It helps us calculate the square of a number of any data type, including `int`, `float`, `long`, and `double`. The templates can be associated with classes; such templates are called class templates. A template is almost similar to a macro, and it is type secured. The difference between a template and a macro is illustrated in sub-topic 17.16.

17.2 NEED FOR TEMPLATES

A template is a technique that allows a single function or class to work with different data types. Using a template, we can create a single function that can process any type of data; that is, the formal arguments of a template function are of template (generic) type. They can accept data of any type, such as `int`, `float`, and `long`. Thus, a single function can be used to accept values of a different data type. Figures 17.1 and 17.2 clear the difference between the non-template function and template function. Usually, we overload a function when we need to handle different data types. This approach increases the program size. The disadvantages are that not only the program length is increased but also more local variables are created in the memory. A template safely overcomes all the limitations occurring during the overloading function and allows better flexibility to the program. The portability provided by a template mechanism can be extended to classes. The following sections describe the use of templates in different concepts of C++ programming. Let us understand the process of the declaration of a template.

17.3 DEFINITION OF CLASS TEMPLATES

In order to declare a class of template type, the following syntax is used:

Template Declaration
<pre>template < class T> class name_of_class { // class data member and func- tion }</pre>

The first statement `template < class T>` tells the compiler that the following class declaration can use the template data type. The `T` is a variable of template type that can be used in the class to define a variable of template type. Both `template` and `class` are keywords. The `<>` (angle bracket) is used to declare the variables of template type that can be used inside the class to define the variables

of template type. One or more variables can be declared separated by a comma. Templates cannot be declared inside classes or functions. They should be global and should not be local.

T k;

where, k is the variable of template type. Most of the authors use T for defining a template; instead of T, we can use any alphabet.

17.1 Write a program to show values of different data types using overloaded constructor.

```
#include<iostream.h>
#include<conio.h>

class data
{
    public:
        data(char c)
        {cout<<"\n"<< " c="<<c <<" Size in bytes:"<<size of(c);}
        data(int c)
        {cout<<"\n"<< " c="<<c <<" Size in bytes:"<<size of(c);}
        data(double c)
        {cout<<"\n"<< " c="<<c <<" Size in bytes:"<<size of(c);}
};

int main()
{
    clrscr();
    data h('A'); // passes character type data
    data i(100); // passes integer type data
    data j(68.2); // passes double type data
    return 0;
}
```

OUTPUT

c = A Size in bytes :1
c = 100 Size in bytes :2
c = 68.2 Size in bytes :8

Explanation: In the above program, the class data contains three overloaded one-argument constructors. The constructor is overloaded for char, int, and double type. In function main(), three objects h, i, and j are created, and the values passed are of different types. The values passed are char, int, and double type. The compiler invokes different constructors for various data types. Here, in order to manipulate different data types, we require to overload the constructor; that is, defining a separate function for each non-compatible data type. This approach has the following disadvantages:

- (1) Re-defining the functions separately for each data type increases the source code and requires more time.

- (2) The program size is increased. Hence, more disk space is occupied.
- (3) If the function contains a bug, it should be corrected in every function.
- (4) Figure 17.1 shows you the working of a program.

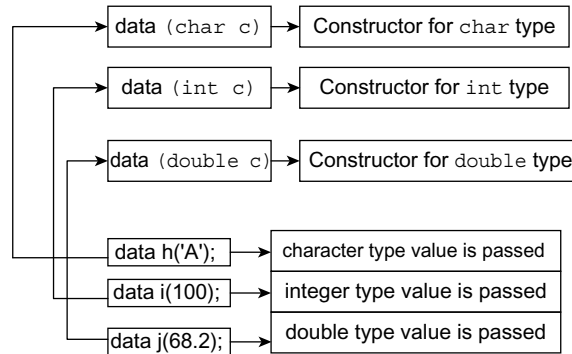


Fig. 17.1 Working of non-template function

From the above program, it is clear that for each data type we need to define a separate constructor function. According to data, type of argument passed respective constructor is invoked. C++ provides templates to overcome such a problem and helps the programmer develop a generic program. The same program is illustrated with the template as follows:

17.2 Write a program to show values of different data types using constructor and template.

```

#include<iostream.h>
#include<conio.h>

template<class T>
class data
{
public:
    data (T c)
    {
        cout<<"\n"<< " c="<<c <<" Size in bytes:"<<size of(c);
    }
};

int main()
{
    clrscr();
    data <char> h('A');
    data <int> i(100);
    data <float> j(3.12);
    return 0;
}
  
```

```
}

```

OUTPUT

```
c = A Size in bytes :1
c = 100 Size in bytes :2
c = 3.12 Size in bytes :4
```

Explanation: In the above program, the constructor contains a variable of template T. The template class variable can hold values of any data type. While declaring an object, the data type name is given before the object. The variable of template type can accept the values of any data type. Thus, the constructor displays the actual values passed. The template variable `c` can hold the values of any data type. The value and space in bytes required by these variables are displayed at the output. The size of data type changes according to the data types used in the program. Figure 17.2 shows the working of the program.

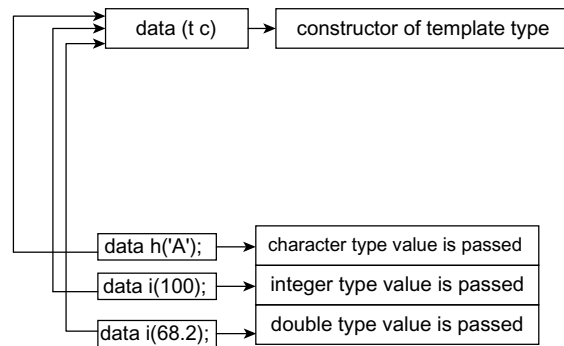


Fig. 17.2 Working of template function

In the above program, different values are passed using constructors, but for all data types, the same template function is used.

17.4 NORMAL FUNCTION TEMPLATES

In the previous section, we observed how to make a template class. In the same manner, a normal function (not a member function) can also use template arguments. The difference between normal and member functions is that normal functions are defined outside the class. They are not members of any class and, hence, can be invoked directly without using the object of a dot operator. The member functions are class members, and they can be invoked using the object of the class they belong to. The declaration of template member functions is described later in this chapter. In C++, normal functions are commonly used as in C. However, the user who wants to use C++ as better C can utilize this concept. The declaration of a normal template function can be done in the following manner:

Normal Template Function Declaration

<pre>template < class T> retuntype function_name (arguments) { // code }</pre>

The following program shows the practical working of template function:

17.3 Write a program to define normal template function.

```
#include<iostream.h>
#include<conio.h>

template<class T>
void show ( T x)
{cout<<"\n x="<<x ;}

void main()
{
    clrscr();
    char c='A' ;
    int i=65;
    double d=65.254;
    show(c);
    show(i);
    show(d);
}
```

OUTPUT

```
x=A
x=65
x=65.254
```

Explanation: Before the body of the function `show()`, the template argument `T` is declared. The function `show()` has one argument `x` of template type. As explained earlier, the template type variable can accept all types of data. Thus, the normal function `show` can be used to display values of different data types. In `main` function, the `show()` functions are invoked with `char`, `int`, and `double` type of values being passed. The same is displayed in the output.

You are now familiar with utilities of templates. One more point to remember is that when we declare a class template, we can define the class data member of the template type as well as the member function of the class can also use the template member. For making a member function of template type, no separate declaration is needed. The following program explains the above point:

17.4 Write a program to define data members of template type.

```
#include<iostream.h>
#include<conio.h>

template<class T>
class data
{
    T x;
public:
    data (T u) {x=u;}
    void show (T y)
    {
        cout<<" x="<<x;
        cout<<" y=" <<y<<"\n";
    }
};

int main()
{
    clrscr();
    data <char> c('B');
    data <int> i(100);
    data <double> d(48.25);
    c.show('A');
    i.show(65);
    d.show(68.25);
    return 0;
}
```

OUTPUT

```
x=B y=A
x=100 y=65
x=48.25 y=68.25
```

Explanation: In this program, before the declaration of class data, template <class T> is declared. This declaration allows the entire class, including member function and data member, to use the template-type argument. We have declared data member x of template type. In addition, the one-argument constructor and member function show() also have one formal argument of template type.

17.5 Write a program to create square() function using template.

```
#include<iostream.h>
#include<conio.h>
```

```
template <class S>
class sqr
{
    public:
        sqr (S c)
        {
            cout<<"\n"<< " c="<<c*c;
        }
};
int main()
{
    clrscr();
    sqr <int> i(25);
    sqr <float> f(15.2);
    return 0;
}
```

OUTPUT

```
c = 625
c = 231.039993
```

Explanation: In the above program, the class `sqr` is declared. It contains a constructor with one argument of template type. In `main()` function, the object `i` indicates `int` type, and `f` indicates `float` type. The objects `i` and `f` invoke the constructor `sqr()` with values 25 and 15.2, respectively. The constructor displays the squares of these numbers.

17.5 WORKING OF FUNCTION TEMPLATES

In the last few examples, we have learned how to write a function template that works with all data types. After compilation, the compiler cannot guess with which type of data the template function will work. When the template function is called at that moment, from the type of argument passed to the template function, the compiler identifies the data type. Then, every argument of template type is replaced with the identified data type; this process is called *instantiating*. Thus, according to different data types, respective versions of the template function are created. Though the template function is compatible for all data types, it will not save any memory. When template functions are used, four versions of functions can be used. The data types `int`, `char`, `float`, and `double` are created. The programmer need not write separate functions for each data type.

17.6 CLASS TEMPLATES WITH MORE PARAMETERS

Similar to functions, classes can be declared to handle different data types. Such classes are known as *class templates*. These classes are of generic type, and member functions of these classes can operate on different data types. The class template may contain one or more

parameters of generic data type. The arguments are separated by commas with a template declaration. The declaration is as follows:

Templates with multiple parameters <pre>template <class T1, class T2> class name_of_class { // class declarations and definitions }</pre>
--

17.6 Write a program to define a constructor with multiple template variables.

```
#include<iostream.h>
#include<conio.h>
template <class T1, class T2>
class data
{public:
    data (T1 a, T2 b)
    {cout<<"\n a="<<a <<" b="<<b;}
};
int main()
{clrscr();
data <int,float> h(2,2.5);
data <int,char> i(15,'C');
data <float,int> j(3.12,50);
return 0;}
```

OUTPUT

```
a = 2 b = 2.5
a = 15 b = C
a = 3.12 b = 50
```

Explanation: In the above program, the data constructor has two arguments of generic type. While creating objects, the type of arguments is mentioned in angle brackets. When arguments are more than one, they are separated by commas. Consider the statement `data <int, float> h ( 2 , 2 . 5 ) ;` it tells the compiler that the first argument is of integer type and the second argument is of float type. When objects are created, constructors are called, and values are received by the template arguments. The output of the program is as given above.

17.7 FUNCTION TEMPLATES WITH MORE ARGUMENTS

In the previous examples, we defined the constructors with arguments. In the same manner, we can also define member functions with generic arguments. The format is as follows:

Function Template Declaration

```
template<class T>
return_data_type function_name (parameters of template type)
{
    Statement1;
    Statement2;
    Statement3;
}
```

17.7 Write a program to display the elements of integer and float array using template variables with member function.

```
#include<iostream.h>
#include<conio.h>
template <class T1, class T2>
class data
{
    public:
    void show (T1 a, T2 b)
    {cout<<"\na="<<a <<" b="<<b;}
};
int main()
{
    clrscr();
    int i[]={3,5,2,6,8,9,3};
    float f[]={3.1,5.8,2.5,6.8,1.2,9.2,4.7};
    data <int,float> h;
    for (int m=0;m<7;m++)
        h.show(i[m],f[m]);
    return 0;
}
```

OUTPUT

```
a = 3 b = 3.1
a = 5 b = 5.8
a = 2 b = 2.5
a = 6 b = 6.8
a = 8 b = 1.2
a = 9 b = 9.2
a = 3 b = 4.7
```

Explanation: In the above program, array elements of the integer and float array are passed to the function show(). The function show() has two arguments of template type, that is, a and

b. The `show()` function receives integer and float values and displays them. The output of the program is shown above.

17.8 Write a program to exchange values of two variables. Use template variables as function arguments.



```
#include<iostream.h>
#include<conio.h>

template <class E>
void exchange(E &a, E &b)
{
    E t=a;
    a=b;
    b=t;
}

int main()
{
    clrscr();
    int x=5,y=8;
    cout<<"\n Before exchange "<<"x="<<x <<" y="<<y;
    exchange (x,y);
    cout<<"\n After exchange "<<"x="<<x <<" y="<<y;
    return 0;
}
```

OUTPUT

Before exchange x= 5 y= 8

After exchange x= 8 y= 5

Explanation: In the above program, `exchange()` is a user-defined function. It has two arguments of template type. In `main()`, the function `exchange()` is invoked, and two integers are passed. The function `exchange()` receives these values by reference. The values of two variables are exchanged with the help of temporary variable `t`. In the output, the values of variables `x` and `y` before and after execution of the function `exchange()` are displayed.

17.9 Write a program to define function template with multiple arguments.

```
#include<iostream.h>
#include<conio.h>

template <class A, class B, class C>
void show(A c,B i,C f)
{cout<<"\n c="<<c <<" i="<<i <<" f="<<f;}

int main()
```

```
{
    clrscr();
    show('A', 8, 50.25);
    show(7, 'B', 70.89);
    return 0;
}
```

OUTPUT

C = A i = 8 f = 50.25

C = 7 i = B f = 70.89

Explanation: In the above program, a template with A, B, and C classes are declared. The function show() has three arguments of type A, B, and C respectively. In main(), the function show() is invoked, and three values of different data types are passed. The function show() displays the values on the screen.

17.8 OVERLOADING OF TEMPLATE FUNCTIONS

A template function also supports the overloading mechanism. It can be overloaded by a normal function or a template function. While invoking these functions, an error occurs if no accurate match is met. No implicit conversion is carried out in the parameters of template functions. The compiler observes the following rules for choosing an appropriate function when the program contains overloaded functions:

- 1) Searches for an accurate match of functions; if found, it is invoked
- 2) Searches for a template function through which a function that can be invoked with an accurate match can be generated; if found, it is invoked
- 3) Attempts a normal overloading declaration for the function
- 4) In case no match is found, an error will be reported

17.10 Write a program to overload a template function.

```
#include<iostream.h>
#include<conio.h>

template <class A>
void show(A c)
{
    cout<<"\n Template variable c="<<c;
}

void show (int f)
{
    cout<<"\n Integer variable f="<<f;
}
```

```
int main()
{
    clrscr();
    show('C');
    show(50);
    show(50.25);
    return 0;
}
```

OUTPUT

Template variable c = C

Integer variable f = 50

Template variable c = 50.25

Explanation: In the above program, the function `show()` is overloaded. One version contains template arguments, and the other version contains integer variables. In `main()`, the `show()` function is invoked thrice with `char`, `int`, and `float` values that are passed. The first call executes the template version of the function `show()`, the second call executes the integer version of the function `show()`, and the third call again invokes the template version of the function `show()`. Thus, in the manner described above, template functions are overloaded.

17.9 MEMBER FUNCTION TEMPLATES

In the previous example, the template functions defined were inline, that is, they were defined inside the class. It is also possible to define them outside the class. While defining them outside, the function template should define the function, and the template classes are parameterized by the type argument.

17.11 Write a program to define definition of member function template outside the class and invoke the function.

```
#include<iostream.h>
#include<conio.h>
template<class T>
class data
{
    public:
        data (T c);
};
template<class T>
data<T>::data (T c)
{
    cout<<"\n"<< " c="<<c;
```



```

}
int main()
{
    clrscr();
    data <char> h('A');
    data <int> i(100);
    data <float> j(3.12);
    return 0;
}

```

OUTPUT

```

c = A
c = 100
c = 3.12

```

Explanation: In the above program, the constructor is defined outside the class. In such a case, the member function should be preceded by the template name as per the following statements:

Statements of program 17.11

<pre> template<class T> data<T>::data (T c) </pre>
--

The first line defines the template, and the second line indicates the template class type T.

17.10 RECURSION WITH TEMPLATE FUNCTIONS

Similar to normal function and member function, the template function also supports the recursive execution of itself. The following program illustrates this:

17.12 Write a program to invoke template function recursively.

```

#include<iostream.h>
#include<conio.h>
#include<process.h>
#include<stdlib.h>
#include<assert.h>
template<class O>
void display (O d)
{
    cout<<(float) (rand()%(d*d))<<"\n";
    // if (d==1) exit(1);
    assert(d!=1);
    display(--d);
}

```

```
}  
void main()  
{  
    int x=10;  
    clrscr(),  
    display(x);  
}
```

OUTPUT

46

49

38

12

28

17

11

7

0

0

Assertion failed: d!=1, file REC_TEMP.CPP, line 14

Abnormal program termination

Explanation: The template function generates a random number and displays it each time the function `display()` is invoked. The function `rand()` defined in `stdlib.h` is used. The function calls itself recursively until the value of `d` becomes 1. The `assert()` statement checks the value of `d` and terminates the program when the condition is satisfied. The `assert()` is defined in `assert.h`. We can also use the `if` statement followed by the `exit()` statement as given in the comment statement.

17.11 CLASS TEMPLATES WITH OVERLOADED OPERATORS

The class template permits the overloaded operators and member functions to be declared. The syntax for creating the overloaded operator function is similar to the class template members and functions. The following program explains the overloaded operators with the template class:

17.13 Write a program to overload + operator for performing addition of two template based class objects.

```
#include<iostream.h>  
#include<conio.h>  
template<class T>  
class num
```

```

{
    private:
        T number;
    public:
        num() {number=0;}
        void input()
        {
            cout<<"\n Enter a number: ";
            cin>>number;
        }
        num operator +(num);
        void show() {cout<<number;}
};

template<class T>
num <T> num <T> :: operator + (num <T> c)
{
    num <T> tmp;
    tmp.number=number+c.number;
    return (tmp);
}

void main()
{
    clrscr();
    num <int> n1,n2,n3;
    n1.input();
    n2.input();
    n3=n1+n2;
    cout<<"\n\t n3=";
    n3.show();
}

```

OUTPUT

Enter a number : 8

Enter a number : 4

n3 = 12

Explanation: In the above program, class num is declared with a template variable number. The input () function is used to read the number through the keyboard. The operator + () function performs the addition of elements of two template objects.

In function main(), n1, n2, and n3 are three objects of the num class. The statement n3 = n1+n2 invokes the overloaded operator, and an addition of two objects is performed. The show () function displays the contents of the object.

17.12 CLASS TEMPLATES REVISITED

As explained earlier, the template mechanism can be used safely in every concept of C++ programming such as functions and classes. We have also learned how to define class templates. Class templates are frequently used for storage of data and can be very powerfully implemented with data structures such as stacks and linked lists. So far, our conventional style is to create a stack or linked list that manipulates or stores only a single type of data; for example, `int`. Suppose we want to store a `float` data type; we need to create a new class to handle the `float` data type and this can be repeated for various data types. However, the template mechanism saves our work of re-defining codes by allowing a single template-based class or function to work with different data types.

Reusability is one the facilities provided by inheritance, in which a class can be reused. Templates also provide reusability. Inheritance allows the reuse of a previously defined class, whereas using the template repetitive definition of functions can be avoided and a single function can be used for multiple data types. Hence, the template-based function code is reused instead of defining multiple functions.

17.14 Write a program to create template-based stack. Store integer and float in it.

```
#include<constream.h>
#include<iostream.h>
#include<stdlib.h>

template <class S>
class stack
{
    S num[5];
    int head;
public:
    stack() {head=-1;}
    void push ( S d)
    {
        if (head==4) cout<<endl<<"Stack is Full";
        else
        {
            head++;
            num[head]=d;
        }
    }
    S pop()
    {
        if (head== -1) {cout<<"\n Stack is empty";
            return NULL;
        }
        else
```

```

    {
        S d=num[head];
        head--;
        return d;
    }
};
void main()
{
    clrscr();
    stack <int>s1;
    for (int j=10;j<=60;j+=10)
        s1.push(j);
    for ( j=0;j<=5;j++)
        cout<<endl<<s1.pop();
    stack <float>s2;
    for ( j=1;j<=6;j++)
        s2.push(.1+j );
    for ( j=0;j<=5;j++)
        cout<<endl<<s2.pop();
}

```

OUTPUT

Stack is Full

50

40

30

20

10

Stack is empty

0

Stack is Full

5.1

4.1

3.1

2.1

1.1

Stack is empty

0

Explanation: As shown in the output, the stack class created can store any type of data. In this program, it stores integers and float type of data. The stack can store five elements. The member function `push()` is used to add elements in the stack, and the `pop()` function is

used to remove elements from the stack. Both functions display messages when the stack is either full or empty. When we perform the `push()` operation beyond the limit, that is, pushing elements more than the stack capacity, the message displayed will be “Stack is full.” When the `pop()` operation is performed beyond the limit, that is, in an attempt to erase the element when there is no element stored in the stack, the message will be “Stack is empty.” In the function `main()`, `s1` and `s2` are template-type objects. Using these objects, integer and float values are added and displayed, respectively. The order in which the elements are pushed into the stack is exactly opposite (reverse) to the order in which they are popped from the stack.

17.13 CLASS TEMPLATES AND INHERITANCE

The template class can also act as a base class. When inheritance is applied with the template class, it helps compose a hierarchical data structure known as a `container class`.

- 1) Derive a template class and add a new member to it. The base class should be of template type.
- 2) Derive a class from a non-template class. Add a new template-type member to the derived class.
- 3) It is also possible to derive classes from the template base class and to omit the template features of the derived classes.

As said earlier, the template characteristics of the base class can be omitted by declaring the type while deriving the class. All the template-based variables are substituted with basic data types. The following declaration illustrates the derivation of a class using a template featured base class.

```
template <class TL, . . .>
class XYZ
{
    // Template-type data members and member functions
}
template <class TL, . . .>
class ABC: public XYZ <TL,...>
{
    // Template-type data members and member functions
}
```

17.15 Write a program to derive a class using template base class.

```
#include<iostream.h>
#include<constream.h>

template<class T>
class one
{
    protected:
```

```

    T x,y;
    void display()
    {
        cout<<x;
    }
};
template <class S>
class two : public one <S>
{
    S z;
public:
    two (S a, S b, S c)
    {
        x=a;
        y=b;
        z=c;
    }
    void show()
    {
        cout<<"\n x="<<x <<" y="<< y<<" z="<<z;
    }
};
void main()
{
    clrscr();
    two <int> i(2,3,4) ;
    i.show();
    two <float> f(1.1,2.2,3.3);
    f.show();
}

```

OUTPUT

x=2 y=3 z=4
x=1.1 y=2.2 z=3.3

Explanation: In the above program, class one is a base class, and class two is a template-derived class. Consider the statement

```

template <class S>
class two: public one <S>

```

Using the above statement, derivation is carried out. The base class name is followed by the template class name S. The function `show()` is a member of the derived class `two()`. In function `main()`, `i` and `f` are objects of class two. The object `i` is used to pass integer elements, and `f` is used to pass float numbers.

17.14 BUBBLE SORT USING FUNCTION TEMPLATES

In application software, large amounts of data are manipulated and sorting techniques are frequently used in such operations. The sorting mechanism helps view the information in different ways. The template function can also be used in the sorting operation. The following program illustrates the use of the template function with the bubble sorting method:

17.16 Write a program to sort integer and float array elements using bubble sort method.

```
#include<iostream.h>
#include<conio.h>

template <class S>
void b_sort ( S u[], int k)
{
    for (int x=0;x<k-1;x++)
        for (int y=k-1;x<y;y--)
            if (u[y]<u[y-1]) // checks successive numbers
            {
                S p;
                p=u[y]; // assigns to temprary variable
                u[y]=u[y-1]; // exchange of elements
                u[y-1]=p; // collects from temporary variable
            }
}

void main()
{
    clrscr();
    int i[5]={4,3,2,1,-4};
    float f[5]={5.3,5.1,5.5,4.5,3.2};
    b_sort(i,5); // call to function
    b_sort(f,5); // call to function
    int x=0,y=0;
    cout<<"\nInteger array elements in ascending order:";
    while (x<5) cout<<i[x++] <<" "; // display contents of
                                     // integer array
    cout<<"\nFloat array elements in ascending order:";
    while ( y<5) cout<<f[y++] <<" "; // displays contents of
                                     // float array
}
```

{coderipe}

OUTPUT

Integer array elements in ascending order: -4 1 2 3 4

Float array elements in ascending order : 3.2 4.5 5.1 5.3 5.5

Explanation: In the above program, `b_sort()` is a function with template and integer arguments. The two `for` loops are used to obtain two successive numbers of the array. The `if` statement checks the two successive numbers, and if the second number is greater than the first, they are exchanged using assignment statements. In the function `main()`, integer and float arrays are declared and initialized. The base addresses (of both arrays) of arrays and the number of elements are passed to the function `b_sort()`. The function `b_sort()` arranges the contents of both the arrays in ascending order. The two `while` loops display the contents of the arrays.

17.15 GUIDELINES FOR TEMPLATES

- (1) Templates are applicable when we want to create a type secure class that can handle different data types with similar member functions.
- (2) The template classes can also be involved in inheritance. For example,

```
template<class T>
class data : public base<T>
```

Both `data` and `base` are template classes. The class `data` are derived from the template class `base`.

- (3) The template variables also allow us to assign default values. For example,

```
template <class T, int x = 20>
class data
{
    T num[x];
}
```

- (4) The name of the template class is written differently in various situations. When it is a class declaration, it is declared in the following manner:

```
class data {};
```

For member functions, the declaration is as follows:

```
void data <T>:: show (T d) {}
where show() is a member function.
```

Finally, while declaring objects, the class name and specific data are specified before the object name.

```
data <int> i1 // Object of class data supports
              integer values
data <float> f1 // Object of class data supports
              float values
```

- (5) All template arguments declared in the template argument list should be used for the definition of formal arguments. If one of the template arguments is not used, the template will be specious. Consider the following example:

```
template < class T>
T show (void)
{
    return x;
}
```

In the above example, the template argument T is not used as a parameter, and the compiler will report an error.

```
template<class T>
void show (int y)
{
    T tmp;
}
```

In the above example, the template-type argument is not used as an argument. The system will probably crash.

```
template <class T, class S>
void max ( T & k)
{
    S p;
}
```

The template variable S is not used. Therefore, compile time error is generated.

17.16 DIFFERENCES BETWEEN TEMPLATES AND MACROS

- (1) Macros are not type safe; that is, a macro defined for integer operations cannot accept float data. They are expanded with no type checking.
- (2) It is difficult to find error in macros.
- (3) In case a variable is post-incremented or decremented, the operation is carried out twice.

Consider the following macro:

```
# define max(a) a + ++a
void main()
{
    int a=10, c;
    c=max(a);
    cout<<c;
}
```

The macro defined in the above macro definition is expanded twice. Hence, it is a serious limitation of macros. The limitation of this program can be removed using a template as shown below.

17.17 Write a program to increment the value of a variable using template.

```
#include<iostream.h>
#include<constream.h>

template<class T>
T max(T k)
{
    ++k;
    return k;
}

void main()
{
    clrscr();
    int a=10, c;
    c=max(a);
    cout<<c;
}
```

OUTPUT

11

Explanation: In the above program, the template function `max()` is defined. An integer value is passed to this function. The function `max()` increments the value and returns it.

17.17 LINKED LISTS WITH TEMPLATES

The linked lists are one of the popular data structures. The following program explains the working of a linked list with templates.

17.18 Write a program to create linked list with template class

```
#include<iostream.h>
#include<constream.h>

template <class L>
class list
{
    L t;
    list *next;
public:
    list ( L t);
    void add_num (list *node)
```

```

    {
        node->next=this;
        next=NULL;
    }
    list *obtainnext() {return next;}
    L getnum() {return t;}
};
template <class L> list <L>::list (L y)
{
    t=y;
    next=0;
}
void main()
{
    clrscr();
    list <int> obj(0);
    list <int> *opt, *last;
    last=&obj;
    int j=0;
    while (j<15)
    {
        opt=new list <int> (j+1);
        opt->add_num (last);
        last=opt;
        j++;
    }
    opt=&obj;
    j=0;
    while (j<15)
    {
        cout<<opt->getnum() << " ";
        opt=opt->obtainnext();
        j++;
    }
}

```

OUTPUT

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14

Explanation: In the above program, the class template is declared same as in the previous programs. The class list has two arguments. One is of template L type (t), and the other is an object of the class list (*next). The pointer next is a pointer to the next element of the linked list. The add_num() function is used to add successive numbers in the linked list. The function obtain-

`next()` function returns the address of the next element. The function declarator of the `obtain-next()` is preceded by the symbol `*`, because it returns the address of the element. In the function `add_num()`, the next pointer is initialized with zero, that is, the next element is initialized to zero. If we do not initialize the next with zero, it will become a wild pointer and may point to any location of the system. Using a loop, successive numbers are added and displayed.

17.18 MORE PROGRAMS

17.19 Write a program to overload[] bracket. Use template variable for displaying float numbers.

```
#include<iostream.h>
#include<conio.h>
template<class T>
class display
{
    public:
    void operator[] ( T j)
    {
        cout<<" "<<j;
    }
};
int main()
{
    clrscr();
    display <float> start;
    for (float k=0.5;k<10;k++)
        start[k];
    return 0;
}
```

OUTPUT

0.5 1.5 2.5 3.5 4.5 5.5 6.5 7.5 8.5 9.5

Explanation: The first statement defines the template class `T`. In the class `display`, the `operator[]` is overloaded. The variable `j` is an object of the class `T`. In `main()`, the `for` loop is executed ten times, and the statement `start[]` is executed 10 times. The output of the program is as given above.

17.20 Write a program to declare pointer to template class. Display the data of different data types.

```
#include<iostream.h>
#include<conio.h>
```

```
template<class T>
class data
{
    public:
        data (T *c) {cout<<"\n"<< " c="<<*c;}
};
int main()
{
    clrscr();
    char x='A';
    int y=100;
    float z=68.2;
    data <char> h(&x) ;
    data <int> i(&y);
    data <float> j(&z);
    return 0;
}
```

OUTPUT

```
c = A
c = 100
c = 68.199997
```

Explanation: In the above program, c is a template pointer. In the function main(), x, y, and z are variables of char, int, and float data types that are declared and initialized. The address passed is received by the pointer c, and the values are displayed.

17.21 Write a program to access array elements of different types of arrays.

```
#include<iostream.h>
#include<conio.h>

template<class T>
class data
{
    public:
        data (T *c);
};

template<class T>
data <T> ::data (T *c)
{
    for (int k=0;k<3;k++)
    {
        cout.precision(2);
```

```

        cout<<*c <<" ";
        c++;
    }
    cout<<"\n";
}
int main()
{
    clrscr();
    char x[3]="abc";
    int y[3]={100,200,300};
    float z[3]={68.22,55.25,45.85};
    data <char> h(x) ;
    data <int> i(y) ;
    data <float> j(z) ;
    return 0;
}

```

OUTPUT

a,b,c,
100,200,300,
68.22,55.25,45.85,

Explanation: In the above program, variable `c` is a template pointer. In the function `main()`, `x`, `y`, and `z` are the arrays of `char`, `int`, and `float` data type. The base address of these arrays is passed to the template. Thus, by an increment operation on the template pointer `c`, successive elements of the arrays are displayed.

17.22 Write a program to define template `max()` and find the maximum value out of two variables passed to template.

```

#include<iostream.h>
#include<conio.h>

template<class T>
T max ( T x, T y) {return (x>y) ? x: y;}

void main()
{
    clrscr();
    int q=78, r=87;
    cout<<endl<<"max (int)="<<max(q,r) ;
    float i=5.2, o=7.4;
    cout<<endl<<"max (float)="<<max(i,o) ;
}

```

```
char mh='B', ms='b';
cout<<endl<<"max (char) = "<<max (mh, ms);
}
```

OUTPUT

```
max (int) =87
max (float) =7.4
max (char) =b
```

Explanation: In the above program, max () is a template function that can be used to determine the maximum value of different data types. In this program, sets of values of int, float, and char are passed to the function and template, and the function template returns the maximum value.

17.23 Write a program to exchange values of two variable using templates.

```
#include<iostream.h>
#include<conio.h>

template<class T>
T exchange (T &x, T &y)
{
    T z;
    z=x;
    x=y;
    y=z;
    return 0;
}

void main()
{
    clrscr();
    int j=10, k=20;
    exchange (j,k);
    cout<<endl<<"j="<<j<<"\t"<<"k="<<k;
    char c='Y', h='B';
    exchange (h,h);
    cout<<endl<<"c="<<c<<"\t"<<"h="<<h;
}
```

OUTPUT

```
j=20 k=10
c=Y h=B
```


Explanation: In the above program, the template function `exchange()` is defined. Different types of values are passed to the `exchange()` function. The function `exchange()` exchanges the values between two variables. The `cout()` statements display the contents of variables.

17.24 Write a program to pass multiple values to the function template.

```
#include<iostream.h>
#include<conio.h>
template <class A, class B, class C>
void mult(A x, B y, C z)
{cout<<" j="<x <<" f="<y <<" str="<z;}
void main()
{
    clrscr();
    int j=45;
    float f=7.8;
    char *str="string";
    mult(j,f,str);
}
```

OUTPUT

j= 45 f=7.8 str = string

Explanation: In the above program, the template function `mult()` has three template arguments. In the function `main()`, variables of `int`, `float`, and `char` are declared and initialized. These variables are passed to the template function `mult()`. The function `mult()` displays contents of the variables on the screen.

17.25 Write a program to overload a template function and display the float, integer, and string for a given number of times.

```
#include<iostream.h>
#include<conio.h>
template <class O>
void display (O d) {cout<<d<<"\n";}
template <class O>
void display (O d, int t)
{
    int k=0;
    while (k<t)
        cout<<d<<"\n",
        k++;
}
void main()
```

```
{  
    clrscr(),  
    display(2.14),  
    display(101,3),  
    display ("Virus !",4);  
}
```

OUTPUT**2.14****101****101****101****Virus !****Virus !****Virus !****Virus !**

Explanation: In the above program, the template function `display()` is overloaded with one and two parameters. When the template function with one argument is called, it displays the given number or character for a single time. When called with two arguments, the first argument is displayed for a number of times that is equal to the second argument. In the output, float value 2.14 and the integer number 101 are displayed thrice. The string “**Virus!**” is displayed four times.

SUMMARY

- (1) A template is one of the most useful characteristics of C++. It is newly added in C++. Instantiation is the activity of creating a particular type using a template. The specific classes are known as instances of the template. The template provides generic programming by defining generic classes. In templates, generic data types are used as arguments, and they can handle a variety of data types.
- (2) The declaration and definition of every template class starts with the keyword `template` followed by the parameter list.
- (3) The class template may contain one or more parameters of generic data type.
- (4) The arguments are separated by commas with a template declaration.
- (5) Function templates can be defined with one or more parameters.
- (6) A template function also supports the overloading mechanism. It can be overloaded by a normal function or a template function.
- (7) It is also possible to define member functions outside the class. While defining them outside, the template function should define the function, and the template classes are parameterized by the type argument.

EXERCISES

(A) Answer the following questions

- (1) What are templates?
- (2) How can a normal function be declared as a template function?
- (3) How can a member function be declared as a template function?
- (4) How can templates be used for generic programming?
- (5) Explain the need of templates.
- (6) What do you mean by overloading of template function?
- (7) Explain class templates.
- (8) What are the differences between templates and macros?
- (9) Explain inheritance with class templates.
- (10) Explain the working of template functions.
- (11) Describe the guidelines for defining templates.

(B) Answer the following by selecting the appropriate option

- (1) Templates are suitable for
 - (a) **any data type**
 - (b) basic data type
 - (c) derived data type
 - (d) all of the above
- (2) Templates can be declared using the keyword
 - (a) **template**
 - (b) try
 - (c) class
 - (d) none of the above
- (3) The template class is also called
 - (a) **generic class**
 - (b) container class
 - (c) virtual class
 - (d) base class
- (4) Function templates can accept
 - (a) only one parameter
 - (b) only two parameters
 - (c) **any number of parameters**
 - (d) none of the above
- (5) In the statement `temp <void> x` where `temp` is the class template,
 - (a) **object `x` can be used to pass integer values**
 - (b) object `x` can be used to pass float values
 - (c) object `x` cannot be used to pass values but can invoke member functions
 - (d) none of the above

- (6) Observe the following program:

```
void show (int x)
{cout<<x;} // normal
                function

template<class T>
void show (T t) {cout<<"\
t"<<t;} // templatefunc-
                tion

void main()

{
    show(5);
}
```

Which function will be invoked first?

- (a) **normal function**
 - (b) template function
 - (c) ambiguity is generated
 - (d) none of the above
- (7) Select the correct template definition:
 - (a) **template<class T>**
 - (b) class<template T>
 - (c) template<T>
 - (d) template class<T>
- (8) Function templates are usually defined
 - (a) **in function `main()`**
 - (b) globally
 - (c) anywhere
 - (d) in a class

- (9) Select the true statement from those given below related with function templates:
- (a) **The variable names used as formal arguments in the template function should be unique.**
 - (b) Every formal variable in the template definition should appear at least once in the function signature.
 - (c) All definitions of template functions should start with the keyword `template`.
 - (d) The code in the template varies each time the template function is instantiated.
- (10) One of the following statements declares a function template that does not return any value and contains a pointer of type `T`:
- (a) **`void sort (T *a);`**
 - (b) `void sort (T *a);`
 - (c) `T sort (T *a);`
 - (d) `void sort (T a);`

(C) Attempt the following programs

- (1) Write a program to define a template and display the absolute value for data type `int`, `double`, and `long`.
- (2) Write a program to define a template and display the following lines through it
 - (a) String "ABCDEFGHIJKL"
 - (b) Digits 1,2,3,4,5,6,7,8,9
 - (c) Float 1.2,2.2,3.2,5.4,8.5,2.6,2.4
- (3) Write a program to create a template to find the maximum value stored in an array.
- (4) Write a program to find the reverse of a given number using the template-based function.
- (5) Write a program to convert a given decimal number into binary form using a template function. The user should be able to convert integers and decimal numbers into hexadecimal.
- (6) Write a program to show values of different data types using constructors and templates.
- (7) Write a program to define normal template functions.
- (8) Write a program to pass an object to the template function. Display the contents of the members of objects.
- (9) Write a program to display the reverse string using the template function.
- (10) Write a template function to find the minimum and maximum values contained in an array.
- (11) Write a template function to compute the addition of all the elements of an array.

(D) Find the bugs in the following programs

(1)

```
template<class T>
void show (T t) {cout<<t;}
class data
{
    public:
    int x,y,z; data() {x=1;
y=2; z=3;}
};
void main()
{
    data X;
    show(X);
}
```

(2)

```
template<class T>
class temp
{
    T t;
    public:
    temp() {t=0;}
    void show()
    {cout<<};};
void main()
{temp <void> x;
x.show();}
```

(3)

```

template<class T>
class one
{protected: T x,y;};
class two : public one <T>
{S z;
  public:
  two (S a, S b, S c)
  {x=a; y=b; z=c;}
  void show()
  {cout<<"\n  x="<<x  <<"
y="<< y<<" z="<<z;}
};

```

```

void main()
{
    two <float>
f(1.1,2.2,3.3);
    f.show();
}

```

(4)

```

template<class T>
T main (T x)
{cout<<x;}

```

Working with Strings

18

CHAPTER

CHAPTER OUTLINE

- 18.1 Introduction
- 18.2 Moving from C String to C++ String
- 18.3 Declaring and Initializing String Objects
- 18.4 Relational Operators
- 18.5 Handling String Objects
- 18.6 String Attributes
- 18.7 Accessing Elements of Strings
- 18.8 Comparing and Exchanging
- 18.9 Miscellaneous Functions
- 18.10 More Programs

18.1 INTRODUCTION

A string is nothing but a sequence of characters. Strings can contain small and capital letters, numbers, and symbols. A string is to be treated as an array of characters with terminating `\0`. Each element of a string occupies a byte in the memory. Every string is terminated by a null character. The last character of such a string is a null (`'\0'`) character, and the compiler identifies the null character during the execution of the program. The programmer need not take care of the null character. In a few cases, the null character is to be specified explicitly. The null character is represented by `'\0'`, which is the last character of a string. The null (`\0`) character is a byte with all bits at logic zero. Hence, ASCII and Hex values are zero. The string is stored in the memory in the following manner:

```
char country[6] = "INDIA"; // Declaration and initialization of an array country with 6 characters
```

Here, text `"INDIA"` is assigned to an array `country` [6]. The text is enclosed within double quotation marks.

I	N	D	I	A	'\0'
73	78	68	73	65	00

As shown above, each character occupies a single byte in the memory. At the end of the string, a null character is inserted by the compiler. The first row shows the elements of the string, and the second row shows their corresponding ASCII values. The compiler takes care of storing the ASCII numbers of the characters in the memory. In addition, the ASCII value of the null character is also stored in the memory.

The programmer can use the string in a program for storing and controlling the text. The text comprises words, sentences, names, and so on. The various operations with strings such as copying, comparing, concatenation, or replacing require a lot of effort in 'C' programming. These strings are called a C-style string. In 'C,' the string functions are declared in the `string.h` header file. Thus, using this header file, string manipulation is done. C++ also supports c style functions. Table 18.1 describes these functions.

Table 18.1 String library functions

Functions	Description
<code>strlen()</code>	Determines the length of a string
<code>strcpy()</code>	Copies a string from the source to the destination
<code>strncpy()</code>	Copies the characters of a string to another string up to the specified length
<code>strcmp()</code>	Compares the characters of two strings (Function discriminates between small & capital letters.)
<code>stricmp()</code>	Compares two strings (Function does not discriminate between small & capital letters.)
<code>strncmp()</code>	Compares the characters of two strings up to the specified length
<code>strnicmp()</code>	Compares the characters of two strings up to the specified length. Ignores case.
<code>strlwr()</code>	Converts uppercase characters of a string into lowercase
<code>strupr()</code>	Converts lowercase characters of a string into uppercase
<code>strdup()</code>	Duplicates a string
<code>strchr()</code>	Determines the first occurrence of a given character in a string
<code>strrchr()</code>	Determines the last occurrence of a given character in a string
<code>strstr()</code>	Determines the first occurrence of a given string in another string
<code>strcat()</code>	Appends the source string to the destination string
<code>strncat()</code>	Appends the source string to the destination string up to a specified length
<code>strrev()</code>	Reverses all the characters of a string
<code>strset()</code>	Sets all the characters of a string with a given argument or symbol
<code>strnset()</code>	Sets a specified number of characters of a string with a given argument or symbol
<code>strspn()</code>	Finds up to what length two strings are identical
<code>strpbrk()</code>	Searches for the first occurrence of a character in a given string and then, displays the string starting from that character

The following program explains the use of the above functions:

18.1 Write a program to declare string (character array). Read string through the keyboard and count the length of the string using string library function.

```
#include<iostream.h>
#include<constream.h>
#include<string.h>

int main()
{
    char name[15];
    clrscr();
    cout<<"\n Enter Your name:";
    cin>>name;
    cout<<"\n Length of name is:"<<strlen(name);
    return 0;
}
```

OUTPUT

Enter Your name : Suraj
Length of name is : 5

Explanation: In the above program, the string is declared using the statement `char name [15]` ; using the library function `strlen()` , the total number of characters entered is displayed.

18.2 Write a program to display reverse of entered string.

```
#include<iostream.h>
#include<constream.h>
#include<string.h>

int main()
{
    char name[15];
    clrscr();
    cout<<"\nEnter Your name:";
    cin>>name;
    cout<<"\n Reverse string is:"<<strrev(name);
    return 0;
}
```

OUTPUT

Enter Your name : Akash
Reverse string is : hsakA



Explanation: In the above program, a character array `name[]` is declared. The string is entered through the keyboard. The `strrev()` function displays a reverse string.

18.3 Write a program to initialize a string using different formats.

```
#include<iostream.h>
#include<constream.h>
#include<string.h>

int main()
{
    clrscr();
    char text[]="Welcome"; // Using double quote
    char text1[]={ 'W', 'e', 'l', 'c', 'o', 'm', 'e', '\0' }; // using
    single quote
    cout<<"\n First string:"<<text;
    cout<<"\n Second string:"<<text1;
    return 0;
}
```

OUTPUT

First string : Welcome

Second string : Welcome

Explanation: In this program, two methods of initialization of arrays are used. Here, both declaration and initialization are done in the same statement.

```
char text []="Welcome";
```

In this statement, the array `text[]` is declared, and it is initialized with the string "Welcome." It is an easy way to initialize the character array. It is not necessary to include the null character. Here also, the size of the array is not mentioned. When declaration and initialization are done in the same statement, the compiler determines the size of the array. Hence, it is optional to mention the size of the array in the subscript[] brackets.

```
char text1[]={ 'W', 'e', 'l', 'c', 'o', 'm', 'e', '\0' }; // using single
// quotes
```

A character array (string) can also be initialized as shown in the above statement. However, in this method, each character should be included in single quotation marks and should be separated by commas. The string should be terminated by the null character. Hence, the second method is difficult as compared with the first. Here, the programmer has to specify the null character at the end of the string.

18.2 MOVING FROM C STRING TO C++ STRING

In the last few examples, we have observed that a string is nothing but a sequence of characters, and it is declared as a character array. However, the manipulation of a string in the form of a

character array requires more effort. C uses a library function defined in `string.h` to carry out string manipulation.

To make the string manipulation easy, the ANSI committee added a new class called a `string`. It allows us to define objects of the `string` type, and they can be used as a built-in data type. The `string` class is considered another container class and not a part of STL (standard template library). The programmer should include the `string` header file.

Instead of declaring a character array, an object of the `string` class is defined. Using the member function of the `string` class, string operations such as coping, comparison, and concatenation are carried out more easily as compared with C library functions.

The `string` class is very vast. It also contains several constructors, member functions, and operators. These constructors, member functions, and operators help us perform various operations with strings.

18.3 DECLARING AND INITIALIZING STRING OBJECTS

In C, we declared a string as follows:

```
char text[10];
```

where, as in C++, a string is declared as an object. The string object declaration and initialization can be done once by using a constructor of the `string` class. The constructors of the `string` class are described in Table 18.2.

Table 18.2 String constructors

Constructors	Meaning
<code>string();</code>	Produces an empty string
<code>string (const char *text);</code>	Produces a string object from a null-ended string
<code>string (const string & text);</code>	Produces a string object from another string object

We can declare and initialize string objects as follows:

Declaration of String Objects	
<code>string text;</code>	<code>// Using constructors without</code> <code>// arguments</code>
<code>string text("C++");</code>	<code>// Using constructors with one</code> <code>// argument</code>
<code>text1=text2</code>	<code>// Assignment of two string objects</code>
<code>text ="c++" + text1</code>	<code>// Concatenation of string objects</code>
<code>cin>> text</code>	<code>// Reading string without spaces</code> <code>// through the keyboard</code>
<code>getline (cin, text)</code>	<code>// Reading string with blank spaces</code>

Two string objects can be concatenated using overloaded `+` operator. The overloaded `+=` operator appends one string to the end of another string. The operators `<<` and `>>` are overloaded operators and can be used for input and output operations.

```

text1+=text is equivalent to text1=text1+text
text1+="xyz" is equivalent to text1=text1+"xyz"
cin >> text          // Reads string without spaces
cout<<text           // Displays the contents on the screen
getline (cin,text) // Reads string with blank spaces

```

Here are some illustrations based on the above concepts:

18.4 Write a program to declare string objects. Perform assignment and concatenation operations with the string objects.

```

#include<iostream>
#include<string>

using namespace std;

int main()
{
    string text; // Vacant string object
    string text1("C++"); // Constructor with one argument
    string text2("OOP");
    cout<<"text1:"<<text1 <<"\n";
    cout<<"text2:"<<text2 <<"\n";
    text=text1; // assignment operation
    cout<<"text:"<<text<<"\n";
    text=text1+" " + text2;
    cout<<"Now text:"<<text;
    return 0;
}

```

OUTPUT

```

text1 : C++
text2 : OOP
text : C++
Now text : C++ OOP

```

Explanation: In the above program, three-string objects `text`, `text1`, and `text2` are defined. The objects are not initialized. The objects `text1` and `text2` are initialized with the strings "C++" and "OOP" respectively. The object `text` is initialized with the contents of object `text1` using '=' operator as per the statement `text = text1`. Again, the joining of objects `text1` and `text2` is done, and the resulting string is assigned to the object `text` as per the statement `text = text1+" " + text2`.

The Table 18.3 describes various member functions of string class.

Table 18.3 String manipulating functions

FUNCTION	USE
<code>append()</code>	Adds one string at the end of another string
<code>assign()</code>	Assigns a specified part of a string
<code>at()</code>	Accesses the characters at a given location
<code>begin()</code>	Returns a reference to the beginning of a string
<code>capacity()</code>	Calculates the total elements that can be stored
<code>compare()</code>	Compares two strings
<code>empty()</code>	Returns false if the string is not empty; otherwise, it is true
<code>end()</code>	Returns a reference to the termination of a string
<code>erase()</code>	Erases the specified character
<code>find()</code>	Finds the given sub-string in the source string
<code>insert()</code>	Inserts a character at a given location
<code>length()</code>	Calculates the total number of elements of a string
<code>max_size()</code>	Calculates the maximum possible size of a string in a given system
<code>replace()</code>	Substitutes the specified characters with a given string
<code>resize()</code>	Modifies the size of the string as specified
<code>size()</code>	Provides the number of characters in the string
<code>swap()</code>	Exchanges the given string with another string

The operators used with arithmetic or comparison operations can be used with string objects. Table 18.4 describes these operators.

Table 18.4 String manipulating operators

Operator	Working
<code>=</code>	Assignment
<code>+</code>	Joining two or more strings
<code>+=</code>	Concatenation and assignment
<code>==</code>	Equality
<code>!=</code>	Not equal to
<code><</code>	Less than
<code><=</code>	Less than or equal to
<code>></code>	Greater than
<code>>=</code>	Greater than or equal to
<code>[]</code>	Subscription (used with array)
<code><<</code>	Insertion operator
<code>>></code>	Extraction operator

18.4 RELATIONAL OPERATORS

Table 18.4 describes various relational operators. These operators can be used with string objects for assignment, comparison, and so on. The following program illustrates the use of relational operators with string objects:

18.5 Write a program to compare two strings using string objects and relational operators.



```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1("OOP");
    string s2("OOP");
    if (s1==s2)
        cout<<"\n Both the strings are identical";
    else
        cout<<"\n Both the strings are different";
    return 0;
}
```

OUTPUT

Both the strings are identical

Explanation: In the above program, two string objects `s1` and `s2` are declared. Both the string objects are initialized with the string "OOP." The `if` statement checks whether the two strings are identical or different. An appropriate message will be displayed on comparison. Thus, in this program, the two string objects contain the same string, and, hence, it displays the message "Both the strings are identical".

18.6 Write a program to compare two strings.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1("aaa");
    string s2("bbb");
    if (s1>s2)
        cout<<"\n s1 is greater than s2";
}
```

```
else
if (s1==s2)
cout<<"\n Both the strings are identical";
else
cout<<"s2 is greater than s1";
return 0;
}
```

OUTPUT

s2 is greater than s1

Explanation: In the above program, two string objects s1 and s2 are declared and initialized with the strings "aaa" and "bbb", respectively. The nested if...else statement checks the two strings and displays the appropriate messages.

18.7 Write a program to compare two strings using standard function compare().

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1("aaa");
    string s2("bbb");
    int d=s1.compare(s2);
    if (d==0)
    cout<<"\n Both the strings are identical";
    else if (d>0)
    cout<<"s1 is greater than s2";
    else
    cout<<"s2 is greater than s1";
    return 0;
}
```

OUTPUT

s2 is greater than s1

Explanation: In the above program, two string objects s1 and s2 are declared and initialized with the strings "aaa" and "bbb", respectively. Both the string objects are compared using the compare() function. The return value of the function compare() is stored in the integer variable d. The function compare() returns zero if the two strings are similar; otherwise, a positive value is returned. Using the if...else conditions, appropriate messages are displayed.

18.5 HANDLING STRING OBJECTS

The member functions `insert()`, `replace()`, `erase()`, and `append()` are used to modify the string contents. The following program illustrates the use of these functions:

`insert()`

This member function is used to insert a specified string into another string at a given location. It is used in the following form:

```
s1.insert(3, s2);
```

where `s1` and `s2` are string objects. The first argument is the location number in the calling string where the second string is to be inserted.

18.8 Write a program to insert one string into another string using `insert()` function.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1("abchijk");
    string s2("defg");
    cout<<"\n s1="<<s1;
    cout<<"\n s2="<<s2;
    cout<<"\n after insertion";
    s1.insert(3, s2);
    cout<<"\n s1="<<s1;
    cout<<"\n s2="<<s2;
    return 0;
}
```

OUTPUT

```
s1= abchijk
s2= defg
after insertion
s1= abcdefghijk
s2= defg
```

Explanation: In the above program, two string objects `s1` and `s2` are declared and initialized with the strings "abchijk" and "defg". The `insert()` function inserts the string "defg" in the string "abchijk" at location 3. Now, the resulting string is "abcdefghijk". In the statement `s1.insert(3, s2)`, the object `s1` invokes the member function `insert()` and passes the arguments 3 and `s2` explicitly.

`erase()`

The `erase()` member function is used to erase/ remove specified characters from a specified location. It is used in the following form:

```
s1.erase (3,7);
```

where `s1` and `s2` are string objects. The first argument is the starting element number, and the second argument is the last element number, that is, the character elements from 3 to 7 are removed.

18.9 Write a program to remove specified characters from the string.

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1("abc12345defg");
    cout<<"\n s1="<<s1;
    cout<<"\n after erase()";
    s1.erase(3,5);
    cout<<"\n s1="<<s1;
    return 0;
}
```

OUTPUT

```
s1= abc12345defg
after erase()
s1= abcdefg
```

Explanation: In the above program, `s1` is a string object that is declared and initialized with the string "abc12345defg". The object `s1` invokes the member function `insert()` with two integers 3 and 5. The 3 indicates the starting element number, and 5 indicates the number of characters to be erased. The `insert()` function erases the next five characters from the 3rd character.

replace()

This member function replaces the given characters in a string. It requires three arguments as per the following format:

```
s1.replace (2,5,s2);
```

where `s1` and `s2` are string objects.

18.10 Write a program to replace the string with the given string.

```
#include<iostream>
#include<string>
```



```
using namespace std;

int main()
{
    string s1("abcdefg");
    cout<<"\n s1="<<s1;
    cout<<"\n after replace()";
    s1.replace(1,3,"BCD");
    cout<<"\n s1="<<s1;
    cout<<"\n";
    return 0;
}
```

OUTPUT

```
s1= abcdefg
after replace()
s1= aBCDefg
```

Explanation: In the above program, `s1` is a string object initialized with "abcdefg". The object `s1` invokes the member function `replace()` with three arguments. The first argument indicates the starting character element, the second argument indicates the location of the last character, and the third argument is a string that is to be replaced.

append()

The above function is used to add a string at the end of another string. It is used in the following format:

```
s1.append (s2);
```

where `s1` and `s2` are two objects. The contents of `s2` are appended in the string `s1`.

18.11 Write a program to append one string at the end of another string. Use `append()` function.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1("abcdefg");
    string s2("hijklmn");
    cout<<"\n s1="<<s1;
    cout<<"\n after append()";
```



```
s1.append(s2);  
cout<<"\n s1="<<s1;  
cout<<"\n";  
return 0;  
}
```

OUTPUT

```
s1= abcdefg  
after append()  
s1= abcdefghijklmn
```

Explanation: In the above program, two-string objects `s1` and `s2` are declared and initialized with the strings "abcdefg" and "hijklmn". The `s1` object invokes the member function `append()`, and `s2` is passed as an argument. The string `s2` is added at the end of the string `s1`.

18.6 STRING ATTRIBUTES

The various attributes of a string such as size, length, and capacity can be obtained using member functions. The size of the string indicates the total number of elements currently stored in the string. The capacity means the total number of elements that can be stored in a string. The maximum size means the largest valid size of the string supported by the system.

size()

The member function `size()` determines the size of the string object, that is, the number of bytes occupied by the string object. It is used in the following format:

```
s1.size()
```

where `s1` is a string object, and `size()` is a member function.

18.12 Write a program to display the size of the string object before and after initialization.

```
#include<iostream>  
#include<string>  
  
using namespace std;  
int main()  
{  
    string s1;  
    cout<<"\nsize:"<<s1.size();  
    s1="hello";  
    cout<<"\nNow size:"<<s1.size();  
    return 0;  
}
```

OUTPUT**size : 0****Now size : 5**

Explanation: In the above program, `s1` is a string object. In the first `cout` statement, the `s1` invokes the member function `size()`, which returns the size of the string object `s1`. The `size()` object returns the size zero, because the object `s1` is empty. The object `s1` is initialized with the string "hello". In the second `cout` statement again, the object `s1` invokes the member function `size()`. This time, the `size()` member function returns the size 5. Thus, the `size()` function determines the size of the string object.

`length()`

The member function `length()` determines the length of the string object, that is, the number of characters present in the string. It is used in the following format:

```
s1.length()
```

where `s1` is a string object, and `length()` is a member function.

The member function `size()` and `length()` exhibit a similar result. Each character occupies one byte in the memory. The number of bytes and the total number of elements present in the string are always same; hence, both these functions exhibit a similar result.

18.13 Write a program to calculate the length of the string. Use member function `length()`.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1;
    cout<<"\n Length:"<<s1.length();
    s1="hello";
    cout<<"\n Now Length:"<<s1.length();
    return 0;
}
```

OUTPUT**Length : 0****Now Length : 5**

Explanation: The above program is similar to the previous one. Here, instead of the `size()` function, the `length()` member function is used. The `length()` member function displays the length of the string object before and after initialization.

capacity()

The member function `capacity()` determines the capacity of the string object, that is, the number of characters it can hold. It is used in the following format:

```
s1.capacity()
```

where `s1` is a string object, and `capacity()` is a member function.

18.14 Write a program to display the capacity of the string object. Use member function `capacity()`.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1;
    cout<<"\n Capacity:"<<s1.capacity();
    s1="hello";
    cout<<"\n Capacity:"<<s1.capacity();
    s1="abcdefghijklmnopqrstuvwxyabcdefghijklmnop";
    cout<<"\n Capacity:"<<s1.capacity();
    return 0;
}
```

OUTPUT

Capacity : 0

Capacity : 31

Capacity : 63

Explanation: In the above program, `s1` is declared as a string object. The member function `capacity()` returns the capacity of the string object to hold the character elements. The maximum size of the string in this system is 4294967293 that is obtained by the member function `max_size()`. In this program, the `capacity()` function returns 31. We get different values in various situations. If the string object is empty, the function returns 0. In case the string object contains a string less than 32 characters, it returns 31. In this program, the string is initialized with 32 characters. The `capacity()` function returns the value 63, that is, 32 + 31.

max_size()

The member function `max_size()` determines the maximum size of the string object, that is, the number of characters it can hold. It is used in the following format:

```
s1.max_size()
```

where `s1` is a string object, and `max_size()` is a member function.

18.15 Write a program to display the maximum size of the string object.

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1;
    cout<<"\n Maximum size:"<<s1.max_size();
    return 0;
}
```

OUTPUT**Maximum size : 4294967293**

Explanation: In the above program, the s1 is declared as a string object. The max_size() function returns the maximum size of the string object, that is, 4294967293.

empty()

The empty() function determines whether the string is empty or filled. If the string is empty, it returns 1; otherwise, it returns 0. It is used in the following format:

```
s1.empty()
```

where s1 is a string object, and empty() is a member function.

18.16 Write a program to determine whether the string object is initialized or not. Use empty() member function.

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1;
    cout<<"\n Empty:"<<(s1.empty()? "True":"False");
    s1="abc";
    cout<<"\n Empty:"<<(s1.empty() ? "True":"False");
    return 0;
}
```

OUTPUT**Empty : True****Empty : False**

Explanation: In the above program, `s1` is declared as a string object, and it is not initialized. The member function `empty()` is called with a conditional operator. It displays the message `true (1)`, that is, the string is empty.

The string object `s1` is initialized with the string “abc” and again, the `empty()` function is invoked. This time, it returns `false (0)`, that is, the string is not empty.

18.7 ACCESSING ELEMENTS OF STRINGS

It is possible to access a particular word or a single character of a string with the help of a member function of the string class. The supporting functions are illustrated with suitable examples.

at ()

This function is used to access an individual character. It requires one argument that indicates the element number. It is used in the following format:

```
s1.at (5);
```

where `s1` is a string object, and 5 indicates the 5th element that is to be accessed.

18.17 Write a program to display the string elements one by one. Use the member function `at()`.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1("PROGRAMMING");
    for (int j=0;j<s1.length();j++)
        cout<<s1.at(j);
    return 0;
}
```

OUTPUT

PROGRAMMING

Explanation: In the above program, `s1` is a string object that is initialized with the string “PROGRAMMING.” The `for` loop is used to represent a successive character location. The `at()` function with a one-integer argument displays the characters. The variable `j` indicates the element number.

The statement `cout<<s1.at (j);` displays the characters. We can also use the overloaded operator `[]` to display the string without use of the `at()` function. Thus, the statement would be `cout<<s1. [j].`

find ()

The `find()` member function finds the given sub-string in the main string. It is used in the following format:

```
s1.find (s2);
```

where `s1` and `s2` are string objects. The `find()` function searches for the sub-string `s2` in the main string `s1`.

18.18 Write a program to find sub-string from the source string.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1 ("Bangalore is the capital of Karnataka");
    int x=s1.find("capital");
    cout<<"Capital is found at:"<<x;
    return 0;
}
```

OUTPUT

Capital is found at : 17

Explanation: In the above program, `s1` is a string object that is initialized with the string. The `find()` member function finds the given string in the source string. In this program, the `find()` function searches for the word "capital" in the string `s1`. The `find()` function returns the element number of the previous element from where the sub-string starts.

substr()

The member function `substr()` is used to find the sub-string in the main string. It requires two-integer arguments. The first argument indicates the starting element of the string, and the second argument indicates the last argument of the string. It is used in the following format:

```
s1.substr (s,e);
```

where `s1` is a string object. The variables `s` and `e` are integer variables that indicate the starting and ending element numbers, respectively, of the sub-string.

18.19 Write a program to retrieve the substring from the main string.

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
```

```
string s1 ("C plus plus");  
cout<<s1.substr(2,4);  
cout<<"\n";  
return 0;  
}
```

OUTPUT

Plus

Explanation: In the above program, `s1` is declared as a string object. It is initialized with the string "C plus plus." The member function `substr()` requires two-integer arguments, which indicate the starting and ending elements of the sub-string. The sub-string is displayed on the screen.

`find_first_of()`

This member function is used to find the first occurrence of the given character(s). It is used in the following format:

```
s1.find_first_of ('p');
```

where `s1` is a string object, and `p` is a character whose first occurrence is to be found.

18.20 Write a program to find the first occurrence of the given character. Use member function `find_first_of()`.

```
#include<iostream>  
#include<string>  
  
using namespace std;  
int main()  
{  
    string s1 ("C plus plus");  
    cout<<s1.find_first_of('p');  
    cout<<"\n";  
    return 0;  
}
```

OUTPUT

2

Explanation: In the above program, `s1` is declared as a string object that is initialized with the string "C plus plus." The member function `find_first_of()` searches for the character 'p' and when it finds it, it returns the element number of the previous character.

`find_last_of()`

This member function is used to find the last occurrence of the given character(s). It is used in the following format:

```
s1.find_last_of ('p');
```

where s1 is a string object, and p is a character whose last occurrence is to be found.

18.21 Write a program to find the last occurrence of the given character. Use member function find_last_of().

```
#include<iostream>
#include<string>

using namespace std;
int main()
{
    string s1 ("C plus plus");
    cout<<s1.find_last_of('p');
    cout<<"\n";
    return 0;
}
```

OUTPUT

7

Explanation: In the above program, s1 is declared as a string object that is initialized with the string "C plus plus." The member function find_last_of() searches for the character p and when its last occurrence is found, it returns the element number of the previous character.

18.8 COMPARING AND EXCHANGING

The string class contains functions that enable the programmer to compare and exchange the strings. The related functions are illustrated below with suitable examples:

compare()

The member function compare() is used to compare two strings or sub-strings. It returns 0 when the two strings are similar; otherwise, it returns a non-zero value. It is used in the following formats:

```
s1.compare (s2);
```

Here, s1 and s2 are two string objects.

```
s1.compare (0,4,s2,0,4);
```

Here, s1 and s2 are string objects. The integer number indicates the sub-string of both the strings that should be compared.

18.22 Write a program to compare two strings. Use `compare()` function.

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1 ("Take");
    string s2 ("Taken");
    int d=s1.compare(s2);
    cout<<"\n d="<<d;
    d=s1.compare (0,4,s2,0,4);
    cout<<"\n d="<<d;
    return 0;
}
```

OUTPUT

d= -1

d= 0

Explanation: In the above program, `s1` and `s2` are two string objects that are initialized with the strings "Take" and "Taken", respectively. The first `compare()` statement checks the two string objects. It returns `-1`; that is, the strings are not identical. In the second `compare()` statement, the element numbers of both the strings that should be compared are sent. Instead of comparing two strings wholly part of one string and whole string of other is compared and tested. In the above program, 0 to 4 elements of both the strings are compared. The function `compare()` returns 0; that is, the two sub-strings are identical.

swap()

The `swap()` member function is used to exchange the contents of two string objects. It is used in the following format:

```
s1.swap (s2);
```

where `s1` and `s2` are two string objects. The contents of `s1` are assigned to `s2` and vice versa.

18.23 Write a program to exchange the contents of two string objects. Use the member function `swap()`.

```
#include<iostream>
#include<string>
using namespace std;
```

```

int main()
{
    string s1 ("Take");
    string s2 ("Took");
    cout<<"\n s1="<<s1;
    cout<<"\n s2="<<s2;
    cout<<"\n After swap() ";
    s1.swap(s2);
    cout<<"\n s1="<<s1;
    cout<<"\n s2="<<s2;
    return 0;
}

```

OUTPUT

```

s1= Take
s2= Took
After swap()
s1= Took
s2= Take

```

Explanation: In the above program, string objects `s1` and `s2` are declared and initialized with "Take" and "Took", respectively. The `swap()` function exchanges the contents of `s1` and `s2`; that is, the contents of `s1` are assigned to `s2` and vice versa. The output of the program is as given above.

18.9 MISCELLANEOUS FUNCTIONS

`assign()`

This function is used to assign a string wholly/ partly to another string object. It is used in the following format:

```
s2.assign (s1)
```

where `s1` and `s2` are two string objects. The contents of the string `s1` are assigned to `s2`.

```
S2.assign (s1, 0, 5);
```

In the above format, elements from 0 to 5 are assigned to the object `s2`.

18.24 Write a program to assign a substring from main string to another string object.

```

#include<iostream>
#include<string>
using namespace std;

```

```
int main()
{
    string s1("c plus plus");
    string s2;
    int x=0;
    s2.assign(s1,0,6);
    cout<<s2;
    return 0;
}
```

OUTPUT

C plus

Explanation: In the above program, s1 and s2 are two string objects. The object s1 is initialized with the string "c plus plus". The object s2 invokes the assign() function with the integer argument 0, 6, which indicates the sub-string. The sub-string is assigned to the object s2, that is, "C plus".

begin()

This member function returns the reference of the first character of the string. It is used in the following format:

```
x = s1.begin();
```

where x is a character pointer, and s1 is a string object.

18.25 Write a program to find starting character of a given string.

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1("C plus plus");
    char *x;
    x=s1.begin();
    cout<<*x;
    return 0;
}
```

OUTPUT

C

Explanation: In the above program, s1 is a string object that is initialized with string "C plus plus". The x is a character pointer. The object s1 invokes the function begin() and returns the starting address of the string s1 to the character pointer x. The contents of x printed are 'C'; that is, the first character of the string.

18.10 MORE PROGRAMS

18.26 Write a program to count total number of vowels present in the given string.

```
#include<iostream>
#include<string>

using namespace std;

int main()
{
    string s1;
    string s2("aeiou");
    int i,j,c=0;
    cout<<"\n Enter a string:";
    getline(cin,s1);
    int l=s1.length();
    for (i=0;i<l;i++)
    {
        for (j=0;j<5;j++)
            if ( s1.at(i)==s2.at(j)) c++;
    }
    cout<<"\n Total vowels present are:"<<c;
    return 0;
}
```

{coderipe}

OUTPUT

Enter a string : c plus plus

Total vowels present are : 2

Explanation: In the above program, the string objects s1 and s2 are declared. The s2 is initialized with the string "aeiou". The string entered through the keyboard is assigned to the string object s1. Using the nested for loop and the **if** condition, vowels are checked in the main string. If a vowel is found, the counter variable c is incremented. At last, the value of c gives us the total number of vowels present in the string. The function at() gives the character at a particular location.

18.27 Write a program to find the length of the string excluding spaces.

```
#include<iostream>
#include<string>
```

{coderipe}

```
using namespace std;

int main()
{
    string s1;
    cout<<"\n Enter a string:";
    getline(cin,s1);
    int v=s1.length(),s=0;
    for (int m=0;m<v;m++)
    {
        if (s1.at(m)==' ')
            s++;
    }
    cout<<"\n Length of the string excluding spaces:"<<v-s;
    return 0;
}
```

OUTPUT

Enter a string : C PLUS PLUS

Length of the string excluding spaces : 9

Explanation: In the above program, `s1` is a string object. The string entered through the keyboard is assigned to the object `s1`. The length of the string `s1` is calculated using the `length()` function and stored in the variable `v`. The `for` loop executes from 0 to `v`. The `if` condition checks every character of the string to see whether it is a blank space or not. If it is a blank space, the counter variable `s` is incremented. When the `for` loop terminates, the variable `s` contains the total number of blank spaces present in the string. The string length is calculated by subtracting the spaces from the original length, that is, `l-s`.

18.28 Write a program that will read a line of text. Replace all the blank spaces with an underscore (_).

```
#include<iostream>
#include<string>

using namespace std;

int main()
{
    string s1;
    cout<<"\n Enter a string:";
    getline(cin,s1);
    int l=s1.length();
    for (int m=0;m<l;m++)
```

```

{
    if (s1.at(m) == ' ')
        s1.at(m) = '_';
}
cout<<"\n s1:"<<s1;
return 0;
}

```

OUTPUT

Enter a string : C PLUS PLUS

s1 : C_PLUS_PLUS

Explanation: The above program is similar to the previous one. Here, when a blank space is found, it is replaced with the sign "_" (underscore). The statement `s1.at(m) = '_'` performs this task.

18.29 Write a program to display the reverse string of the entered string.

```

#include<iostream>
#include<string>
using namespace std;
int main()
{
    string s1;
    cout<<"\n Enter a string:";
    getline(cin,s1);
    int v=s1.length();
    for (int m=v;m>=0;m--)
        cout<<s1[m];
    return 0;
}

```

OUTPUT

Enter a string : WORLD

DLROW

Explanation: In the above program, the length of the string object is stored in the variable `v`. The `for` loop executes from `v` to 0, that is, in descending order. The statement `cout<<s1[m]` displays the characters on the screen.

SUMMARY

- (1) A string is nothing but a sequence of characters. They are declared as a character array. Each element of the string occupies a byte in the memory. Every string is terminated by a null character.
- (2) To make the string manipulation easy, the ANSI committee added a new class called `string`. It allows us to define objects of a string type, and they can be used as a built-in data type. The `string` class is considered another container class and not a part of STL (standard template library).
- (3) Two string objects can be concatenated using overloaded `+` operator. The overloaded `+=` operator appends one string to the end of another string. The operators `<<` and `>>` are overloaded operators, and they can be used for input and output operations.
- (4) Table 18.4 describes various relational operators. These operators can be used with string objects for assignment, comparison, and so on.
- (5) The member functions `insert()`, `replace()`, `erase()`, and `append()` are used to modify the string contents.
- (6) The various attributes of the string such as size, length, and capacity can be obtained using member functions. The size of the string indicates the total number of elements currently stored in the string. The capacity means the total number of elements that can be stored in the string. The maximum size means the largest valid size of the string supported by the system.
- (7) It is possible to access a particular word or a single character of the string with the help of member functions of the string class, such as `at()`, `find()`, `substr()`, `find_first_of()`, and `find_last_of()`.
- (8) The string class contains functions that enable the programmer to compare and exchange the strings. These functions are `swap()` and `compare()`.

EXERCISES

(A) Answer the following questions

- (1) What are strings? How are they declared?
- (2) What is a null character? Why is it important?
- (3) Why is it necessary to count null characters while declaring a string?
- (4) What is the difference between "a" and "a?"
- (5) What is a string object?
- (6) What is the difference between a string object and a character pointer?
- (7) List any five string functions along with their uses. Explain their syntaxes.
- (8) What is the difference between the functions `strcmp()` and `stricmp()`?
- (9) What is the use of `strrev()` and `strlen()` functions?

(B) Answer the following by selecting the appropriate option

- (1) A and B are two string objects. A = "abc" and B = "xyz". A = A+B will produce
 - (a) "abcxyz"
 - (b) "abc"
 - (c) "xyzabc"
 - (d) none of the above
- (2) The statement `string a("abc")`
 - (a) initializes the string object

- (b) replaces the string
 - (c) appends the string
 - (d) none of the above
- (3) The string are initialized using
- (a) **a default constructor**
 - (b) an explicit constructor
 - (c) an assignment operator
 - (d) both (a) and (c)
- (4) We can access string characters using
- (a) **at () function**
 - (b) subscript operator []
 - (c) () operator
 - (d) none of the above

(C) Attempt the following programs

- (1) Write a program to read three strings “C,” “PLUS,” and “PLUS.” Concatenate them in a single string.
- (2) Write a program to create an array of string objects. Read at least 10 names. Display the names alphabetically.
- (3) Write a program to read a string. Count the number of vowels and spaces in the string.
- (4) Write a program to read a string. Add the same string in the reverse order to the end of the same string.
- (5) Write a program to read a string. Remove all duplicate letters from the string.
- (6) Write a program to read a string. Change the first letter of every capital word.
- (7) Write a program to display the reverse string of the entered string.
- (8) Write a program to exchange the contents of two string objects. Use the member function `swap()`.
- (9) Write a program to display the string elements one by one. Use the member function `at()`.
- (10) Write a program to determine whether the string object is initialized or not. Use the `empty()` member function.
- (11) Write a program to declare string objects. Perform assignment and concatenation with the string objects.
- (12) Write a program to arrange the following names in alphabetic order. The sorting should be done on the first three characters of the first name (Ashok, Alok, Akash, Amit, Amol, Anil, Ashish, and Anand).

Exception Handling

19

CHAPTER

CHAPTER OUTLINE

- 19.1 Introduction
- 19.2 Principles of Exception Handling
- 19.3 The Keywords `try`, `throw`, and `catch`
- 19.4 Guidelines for Exception Handling
- 19.5 Multiple `catch` Statements
- 19.6 Catching Multiple Exceptions
- 19.7 Re-throwing Exceptions
- 19.8 Specifying Exceptions
- 19.9 Exceptions in Constructors and Destructors
- 19.10 Controlling Uncaught Exceptions
- 19.11 Exceptions and Operator Overloading
- 19.12 Exceptions and Inheritance
- 19.13 Class Templates with Exception Handling
- 19.14 Guidelines for Exception Handling
- 19.15 More Programs

19.1 INTRODUCTION

While writing large programs, a programmer usually makes many mistakes. Due to this, bugs occur even in the released softwares. Developing an error-free program is the only objective and intention of the programmer. Programmers should take care to prevent errors. Errors can be trapped using exception-handling features.

A few languages support exception-handling features. Without this feature, the programmers should find out errors on their own. The errors may be logical errors or syntactic mistakes (syntax mistakes). The logical error remains in the program due to a poor understanding of the program.

The syntax mistakes made are due to a lack of understanding of the programming language. C++ provides an exception-handling procedure to reduce the errors that the programmer makes.

The programmer always faces unusual errors while writing programs. An exception is an abnormal termination of the program, which is executed in a program at run time or it may be called at run time when the error occurs. The exception contains warning messages such as invalid argument, insufficient memory, and division by zero. ANSI C++ is built in language functions for trapping errors and controlling exceptions. All C++ compilers support this newly added facility.

19.2 PRINCIPLES OF EXCEPTION HANDLING

Similar to errors, exceptions are also of two types. They are as follows:

- (a) Synchronous exceptions
- (b) Asynchronous exceptions

C++ has a well-organized, object-oriented method to control run-time errors that occur in the program. The goal of exception handling is to create a routine that detects and sends an exceptional condition in order to execute suitable actions. The routine needs to carry out the following responsibilities:

- (1) Detect the problem
- (2) Warn that an error has been detected
- (3) Accept the error message
- (4) Perform accurate actions without troubling the user

An exception is an object. It is sent from the part of the program where an error occurs to the part of the program that is going to control the error. An exception provides an explicit pathway that contains errors to the code which controls these errors.

19.3 THE KEYWORDS `try`, `throw`, AND `catch`

The exception-handling technique passes the control of a program from a location of exception in a program to an exception-handler routine linked with the `try` block. An exception-handler routine can only be called by the `throw` statement.

- (a) `try`: The `try` keyword is followed by a series of statements enclosed in curly braces.

Syntax of <code>try</code> statement
<pre>try { statement 1; statement 2; }</pre>

- (b) `throw`: The function of the `throw` statement is to send the exception found. The declaration of the `throw` statement is as follows:

Syntax of throw statement

<pre>throw (excep); throw excep; throw // re-throwing of an exception</pre>

The argument `excep` is allowed to be of any type, and it may be a constant. The `catch` block associated with the `try` block catches the exception thrown. The control is transferred from the `try` block to the `catch` block. The `throw` statement can be placed in a function or in a nested loop, but it should be in the `try` block. After throwing the exception, control passes to the `catch` statement.

- (c) **catch:** Similar to the `try` block, the `catch` block also contains a series of statements enclosed in curly braces. It also contains an argument of an exception type in parentheses.

Syntax of catch statement

<pre>try { Statement 1; Statement 2; } catch (argument) { statement 3; // Action to be taken }</pre>
--

When an exception is found, the `catch` block is executed. The `catch` statement contains an argument of exception type, and it is optional. When an argument is declared, the argument can be used in the `catch` block. After the execution of the `catch` block, the statements inside the blocks are executed. In case no exception is caught, the `catch` block is ignored, and if a mismatch is found, the program is terminated.

19.4 GUIDELINES FOR EXCEPTION HANDLING

The C++ exception-handling mechanism provides three keywords; they are `try`, `throw`, and `catch`. The keyword `try` is used at the starting of the exception. The `throw` block is present inside the `try` block. Immediately after the `try` block, the `catch` block is present. Figure 19.1 shows the `try`, `catch`, and `throw` statements.

As soon as an exception is found, the `throw` statement inside the `try` block throws an exception (a message for the `catch` block that an error has occurred in the `try` block statements). Only errors occurring inside the `try` block are used to throw exceptions. The `catch` block receives the exception that is sent by the `throw` block. The general form of the statement is as per Figure 19.2.

When the `try` block passes an exception using the `throw` statement, the control of the program passes to the `catch` block. The data type used by `throw` and `catch` statements should

be same; otherwise, the program is aborted using the `abort()` function, which is executed implicitly by the compiler. When no error is found and no exception is thrown, in such a situation, the `catch` block is disregarded, and the statement after the `catch` block is executed.

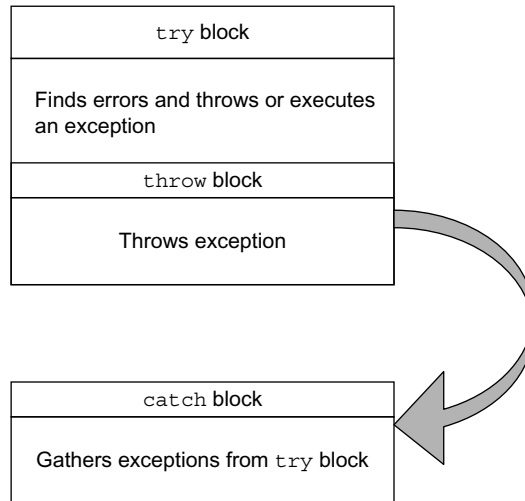


Fig. 19.1 Exception mechanism

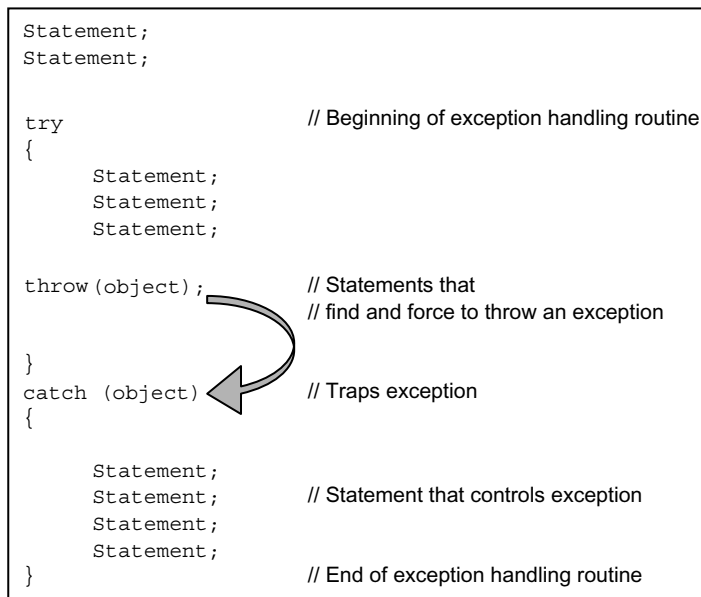


Fig. 19.2 try, throw, and catch blocks

19.1 Write a program to throw exception when $j=1$ otherwise perform the subtraction of x and y .

```
#include<iostream.h>
#include<conio.h>

int main()
{
    int x,y;
    cout<<"\n Enter values of x and y\n";
    cin>>x>>y;
    int j;
    j=x>y ? 0 :1;

    try

    {
        if (j==0)
        { cout<<"Subtraction (x-y)="<<x-y<<"\n"; }
        else { throw(j); }
    }
    catch (int k)
    {
        cout<<"Exception caught : j ="<<j <<"\n";
    }
    return 0;
}
```

OUTPUT**Enter values of x and y****7 8****Exception caught : j = 1****Enter values of x and y****4 8****Exception caught : j = 1**

Explanation: In the above program, the values of x and y are entered. The conditional operator tests the two numbers; if x is greater than y , zero is assigned to j ; otherwise, it is one. In the `try` block, the `if` condition checks the value of j ; the subtraction is carried out when j is zero; otherwise, the `else` block throws the exception. The `catch` statement catches the exception thrown. The output of the program is shown above.

19.2 Write a program to define function that generates exception.

```
#include<iostream.h>

void sqr()
{
    int s;
    cout<<"\n Enter a number:";
    cin>>s;
    if (s>0)
    {
        cout<<"Square="<<s*s;
    }
    else
    {
        throw (s);
    }
}

int main()
{
    try
    {
        sqr();
        sqr();
    }
    catch (int j)
    {
        cout<<"\n Caught the exception \n";
    }
    return 0;
}
```

OUTPUT

```
Enter a number : 5
Square = 25
Enter a number : 0
Caught the exception
```

Explanation: In the above program, the function *sqr()* is defined. The function *sqr()* reads an integer through the keyboard and displays its square. Before calculating the square, the *if* statement checks the number. If it is greater than zero, then the *if* block calculates the square and displays it; otherwise, the *else* block throws the exception. In the function *main()*, in the *try* block, the function is called twice. In case an exception is thrown from the function *sqr()*, the *catch* statement catches it, and the *catch* block is executed. This happens only when the user

enters the number 0 or less than zero. If the user enters 0 in the first call, then the exception is thrown, and the `catch` block is executed. The compiler ignores the next statement of the `try` block. Once the control skips from the `try` block, it never comes back to execute the remaining statements. Thus, in this program if in the first call the user enters 0, then the second call of the function `sqr()` is not taken into account.

19.5 MULTIPLE `catch` STATEMENTS

We can also define multiple catch blocks; in the `try` block, such programs also contain multiple throw statements based on certain conditions. The format of multiple catch statements is as follows:

```
try
{
    // try section
}
catch (object1)
{
    // catch section1
}
catch (object2)
{
    // catch section2
}
. . . . .
catch (type n object)
{
    // catch section-n
}
```

As soon as an exception is thrown, the compiler searches for an appropriate matching catch block. The matching catch block is executed, and control passes to the successive statement after the last catch block. In case no match is found, the program is terminated. In a multiple catch statement, if objects of many catch statements are similar to the type of an exception, in such a situation, the first catch block that matches is executed.

19.3 Write a program to throw multiple exceptions and define multiple catch statement.

```
#include<iostream.h>

void num (int k)
{
    try
    {
        if (k==0) throw k;
        else
```




```
    if (k>0) throw 'P';
    else
    if (k<0) throw .0;
    cout<<"*** try block ***\n";
}
catch(char g)
{
    cout<<"Caught a positive value \n";
}
catch (int j)
{
    cout<<"caught an null value \n";
}
catch (double f)
{
    cout<<"Caught a Negative value \n";
}
cout<<"*** try catch ***\n \n";
}
int main()
{
    cout<<"Demo of Multiple catches\n";
    num(0);
    num(5);
    num(-1);
    return 0;
}
```

OUTPUT

Demo of Multiple catches

caught an null value

*** try catch ***

Caught a positive value

*** try catch ***

Caught a Negative value

*** try catch ***

Explanation: In the above program, the function `num()` contains the `try` block with multiple `catch` blocks. In the function `main()`, the user-defined function `num()` is invoked with one argument. The `if` statement within the `try` block checks the number to see whether it is positive, negative, or zero. According to this, an exception is thrown, and the respective `catch` block is executed. Here, in the `throw` statement, objects of different data types such as `int`, `char`, and `double` are used to avoid ambiguity.

19.6 CATCHING MULTIPLE EXCEPTIONS

It is also possible to define a single or default `catch` block from one or more exceptions of different types. In such a situation, a single `catch` block is used to catch the exceptions thrown by the multiple `throw` statements.

```
catch
{
    // Statements for handling
    // all exceptions
}
```

19.4 Write a program to catch multiple exceptions.

```
#include<iostream.h>

void num (int k)
{
    try
    {
        if (k==0) throw k;
        else
        if (k>0) throw 'P';
        else
        if (k<0) throw .0;
        cout<<"*** try block ***\n";
    }
    catch
    {
        cout<<"\n Caught an exception\n";
    }
}

int main()
{
    num(0);
    num(5);
    num(-1);
    return 0;
}
```

OUTPUT

Caught an exception
Caught an exception
Caught an exception

Explanation: The above program is similar to the previous one. Here, only one difference is observed and that is, instead of multiple `catch` blocks, a single `catch` block is defined. For all the exceptions thrown, the same `catch` block is executed. It is a generic type of `catch` block. The statement `catch` catches all the exceptions thrown.

19.7 RE-THROWING EXCEPTION

It is also possible to again pass the exception received to another exception handler; that is, an exception is thrown from the `catch` block; this is known as the *re-throwing exception*. The following statement accomplishes this:

`throw;`

The above `throw` statement is used without any argument. This statement throws the exception caught to the next `try catch` statement. The following program illustrates this:

19.5 Write a program to re-throw an exception.

```
#include<iostream.h>

void sub( int j, int k)
{
    cout<<"inside function sub()\n";
    try
    {
        if (j==0)
            throw j;
        else
            cout<<"Subtraction="<<j-k<<"\n";
    }
    catch (int)
    {
        cout<<"Caught Null value \n";
        throw;
    }
    cout<<"** End of sub() ***\n\n";
}

int main()
{
    cout<<"\n inside function main()\n";
    try
    {
        sub (8,5);
        sub (0,8);
    }
    catch (int)
    {
```

```

    cout<<"caught null inside main() \n";
  }
  cout<<"end of function main() \n";
  return 0;
}

```

OUTPUT

```

inside function main()
inside function sub()
Subtraction = 3
** End of sub() ***
inside function sub()
Caught Null value
caught null inside main()
end of function main()

```

Explanation: In the above program, two `try` blocks are defined. One is defined in the function `sub()`, and the other is defined in the function `main()`. The `sub()` function has two integer arguments. When the `sub()` function is invoked, two integer values are sent to this function. The `if` statement in the `try` block of the `sub()` function checks whether the value of the first variable, that is, `j` is zero or non-zero. If it is non-zero, subtraction is carried out; otherwise, the `throw` statement throws an exception. The `catch` block inside the function `sub()` collects this exception and again throws the exception using the `throw` statement. Here, the `throw` statement is used without any argument. The `catch` block of the main function catches the re-thrown exception.

19.8 SPECIFYING EXCEPTIONS

The specified exceptions are used when we want to bind the function to `throw` only specified exceptions. Specific exception is thrown using condition statement and list of data items in `throw`. The format for such exceptions is as given below:

Specifying Exceptions
<pre> Data-type function_name (parameter list) throw (data type list) { Statement 1; Statement 2; Function definition statement 3; } </pre>

The data type list indicates the type of exception that is permitted to be thrown. If we want to deny a function from throwing an exception, declaring the data type list `void` as per the following statement can do this.

```
throw(); // void or vacant list
```

19.6 Write a program to restrict a function to throw only specified type of exceptions.

```
#include<iostream>

using namespace std;
void check (int k) throw (int,double)
{
    if (k==1) throw 'k';
    else
    if (k==2) throw k;
    else
    if (k== -2) throw 1.0;
    cout<<"\n End of function check()";
}
void main()
{
    try {
        cout<<"k==1\n";
        check(1);
        cout<<"k==2\n";
        check(2);
        cout<<"k== -2\n";
        check(-2);
        cout<<"k==3\n";
        check(3);
    }
    catch ( char g)
    {
        cout<<"Caught a character exception \n";
    }
    catch (int j)
    {
        cout<<"Caught a character exception \n";
    }
    catch (double s)
    {
        cout<<"Caught a double exception \n";
    }
    cout<<"\n End of main()";
}
```

**OUTPUT****k==1****Caught a character exception****End of main() Press any k**

Explanation: In the above program, the `check()` function is defined. It is followed by an exception specifying statement. The `if` statements check the value of the formal argument. According to the value, an exception is thrown. In function `main()`, the function `check()` is invoked with different values. The output of the program is as given above.

19.9 EXCEPTIONS IN CONSTRUCTORS AND DESTRUCTORS

A copy constructor is called in exception handling when an exception is thrown from the `try` block using the `throw` statement. The copy constructor mechanism is applied to initialize the temporary object. In addition, destructors are also executed to destroy the object. If an exception is thrown from the constructor, destructors are called only for those objects that are completely constructed.

19.7 Write a program to use exception handling with constructor and destructor.

```
#include<iostream.h>
#include<process.h>

class number
{
    float x;
public :
    number (float);
    number() {};
    ~number()
    {
        cout<<"\n In destructor";
    }
    void operator ++ (int) // postfix notation
    { x++; }
    void operator --() // prefix notation
    {
        --x; }
    void show()
    {
        cout<<"\n x="<<x;
    }
};

number :: number ( float k)
{
    if (k==0)
        throw number();
    else
        x=k;
```

```
}  
void main()  
{  
    try  
    {  
        number N(2.4);  
        cout<<"\n Before Increasing:";  
        N.show();  
        cout<<"\n After Increasing:";  
        N++; // postfix increment  
        N.show();  
        cout<<"\n After Decrementation:";  
        --N; // prefix decrement  
        N.show();  
        number N1(0);  
    }  
    catch (number)  
    {  
        cout<<"\n invalid number";  
        exit(1);  
    }  
}
```

OUTPUT

Before Increasing:

x=2.4

After Increasing:

x=3.4

After Decrementation:

x=2.4

In destructor

In destructor

invalid number

Explanation: In the above program, the operators ++ and – are overloaded. The class number() has two constructors and one destructor. The one-argument constructor is used to initialize the data member x with the value given by the user. The overloaded operators are used to increase and decrease the value of the object. When the user specifies the zero value, an exception is thrown from the constructor. The exception is caught by the catch statement. In function() main(), two objects N and N1 are declared. The exception is thrown when the object N1 is created. When the exception is thrown, the control goes catch block. The catch block terminates the program. The destructors are also invoked.

19.10 CONTROLLING UNCAUGHT EXCEPTIONS

C++ has the following functions, and these can be used to control uncaught exceptions.

terminate(): In case the exception handler is not defined when an exception is thrown, in such a case, the `terminate()` function is invoked. Implicitly, the `abort()` function is invoked.

19.8 Write a program caught uncaught exceptions.

```
#include<iostream.h>
class one {};
class two {};
void main()
{
    try
    {
        cout<<"An Uncaught exception\n";
        throw two();
    }
    catch (one)
    {
        cout<<"Exception for one";
    }
}
```

Explanation: In the above program, an exception is thrown for class `two` using the statement `throw two()`. The `catch` block is defined to handle the exception for class `one`, and the `catch` block contains an argument of class `one` type. When an exception is thrown, it does not find a match; hence, the program is terminated. For termination, the `abort()` function is called implicitly by the compiler.

`set_terminate()`

The `set_terminate()` function is used to transfer the control to the specified error-handling function. The `set_terminate()` function requires only one argument, which is nothing but a function name where control is transferred. It returns nothing. When no function is specified, the program is terminated by an implicit call to `abort()` function.

19.9 Write a program to demonstrate the use of `set_terminate()` function.

```
#include<iostream.h>
#include<except.h>

class one {};
class two {};
void skip()
```



```

{
    cout<<"\n function skip() is invoked";
}
void main()
{
    set_terminate(skip);
    try
    {
        cout<<"\n Throwing exception:";
        throw (two);
    }
    catch (one)
    {
        cout<<"\n An exception of one is caught";
    }
}
// At this point function skip is invoked

```

OUTPUT

**Throwing exception
function skip() is invoked**

Explanation: This program is similar to the previous one. The only difference is that in the previous program the compiler does not find a matching `catch` statement for the exception thrown by the `throw` statement. The program is terminated. In this program, at the beginning, a user-defined function `skip()` is defined. The `skip()` function is associated with the `set_terminate()` function. When a compiler does not find a matching exception instead of terminating a program, it invokes the function associated with the `set_terminate()` function. Here, the exception is thrown for an object of class `two`. The `catch` statement holds an object of class `one`. Hence, the thrown exception object does not match the `catch` object. Due to this mismatch, the function `skip()` is invoked to handle such an uncertain situation.

19.11 EXCEPTIONS AND OPERATOR OVERLOADING

An exception handling can be used with operator-overloaded functions. The following program illustrates the same:

19.10 Write a program to throw exception from overloaded operator function.

```

#include<iostream.h>
#include<process.h>

class number

```

```
{
    int x;
    public :
    number() {};
    void operator --();
    void show() { cout<<"\n x="<<x; }
    number ( int k) { x=k; }
};
void number :: operator --() // prefix notation
{
    if (x==0) throw number();
    else --x;
}
void main()
{
    try
    {
        number N(4);
        cout<<"\n Before Decrementation:";
        N.show();

        while (1)
        {
            cout<<"\n After Decrementation";
            --N;
            N.show();
        }

    }
    catch (number)
    {
        cout<<"\n Reached to zero";
        exit(1);
    }
}
```

OUTPUT**Before Decrementation****x=4****After Decrementation****x=3****After Decrementation****x=2****After Decrementation**

x=1

After Decrementation

x=0

After Decrementation

Reached to zero

Explanation: In this program, the operator – is overloaded. When used with class objects, this operator decreases the values of class members. Using the while () loop, the value of the object N is continuously decreased. The object N is initialized with four. The operator – () function checks the value of x (member of object N); if the value of x reaches zero, an exception is thrown, which is caught by the catch statement.

19.12 EXCEPTIONS AND INHERITANCE

In the last few examples, we have learned how the exception mechanism works with the operator function and with constructors and destructors. The following program explains how exception handling can be done with inheritance:

19.11 Write a program to throw an exception in derived class.

```
#include<iostream.h>
class ABC
{
    protected:
        char name[15];
        int age;
};
class abc : public ABC // public derivation
{
    float height;
    float weight;
    public:
    void getdata()
    {
        cout<<"\n Enter Name and Age:";
        cin>>name>>age;
        if (age<=0)
            throw abc();
        cout<<"\n Enter Height and Weight:";
        cin>>height >>weight;
    }
    void show()
```

```

    {
        cout<<"\n Name:"<<name<<"\n Age:"<<age<<" Years";
        cout<<"\n          Height:"<<height<<"Feets"<<"\n          Weight:"
<<weight<<"Kg.";
    }
};
void main()
{
    try
    {
        abc x;
        x.getdata(); // Reads data through keyboard.
        x.show(); // Displays data on the screen.
    }
    catch (abc) { cout<<"\n Wrong age"; }
}

```

OUTPUT

Enter Name and Age: Amit 0

Wrong age

Explanation: In the above program, the two classes ABC and abc are declared. The class ABC has two protected data members name and age. The class abc has two float data members' height and weight with two member functions getdata() and show(). The class abc is derived from class ABC. The statement `class abc: public ABC` defines the derived class abc. In the function main(), x is an object of the derived class abc. The object x invokes the member function getdata() and show(). This function reads and displays data, respectively. In the function getdata(), the if statement checks the age entered. If the age entered is zero or less than zero, an exception is thrown. The catch block is executed, and the message "Wrong age" is displayed.

19.13 CLASS TEMPLATES WITH EXCEPTION HANDLING

The following program illustrates how excepting handling can be implemented with class templates:

19.12 Write a program to show exception handling with class template.

```

#include<iostream.h>
#include<math.h>
class sq {};
template <class T>
class square
{

```

```
T s;
public:
square ( T x)
{
    sizeof(x)==1 ? throw sq() : s=x*x;
}
void show()
{
    cout<<"\n Square:"<<s;
}
};
void main()
{
    try
    {

        square <int> i(2);
        i.show();
        square <char> c('C');
        c.show();

    }
    catch (sq)
    {
        cout<<"\n Square of character cannot be calculated";
    }
}
```

OUTPUT

Square: 4

Square of character cannot be calculated

Explanation: In the above program, the class `square` uses the template type of argument. Using conditional operators, the size of the passed argument is checked. If it is equal to one, an exception is thrown; otherwise, the square is calculated and stored in the class data member `s`. The `show()` function displays the square on the screen. When character-type data are entered, an exception is thrown, because the size of the character is one and the square cannot be calculated.

19.14 GUIDELINES FOR EXCEPTION HANDLING

- (1) It is not essential for the `throw` statement to appear in the `try` block in order to throw an exception. The `throw` statement can be placed in any function, if the function is to be invoked through the `try` block.

<pre> try { statement; show (variable) statement; } catch (object) { statement; } </pre>	<pre> Show() { statement; throw (object); } </pre>
--	--

Fig. 19.3 throw statement out of try block

As shown in Figure 19.3, the `throw` statement is present in the `show()` function. The `show()` function is invoked inside the `try` block. Thus, exception handling can also be defined in the manner described above.

- (2) When an exception that is not specified is thrown, it is known as an unexpected exception.
- (3) In case an exception is thrown before the complete execution of a constructor, the destructor for that object will not be executed.
- (4) As soon as an exception is thrown, the compiler searches nearby handlers (`catch` blocks). After finding a match, it will be executed.
- (5) Overuse of exception handling increases the program size. So, apply it whenever most necessary. Incorrect use of exception handling is not consistent and generates bugs in the program. It is hard to debug such bugs.

19.15 MORE PROGRAMS

19.13 Write a program to enter five numbers in an integer array. Display the largest number. If the array contains null element, throw the exception.

```

#include<iostream.h>

int main()
{
    int max[5], k=0, m;
    cout<<"\n Enter five numbers:";
    for (k=0; k<5; k++)
        cin>>max[k];
    try
    {
        for (k=0; k<5; k++)
        {
            if (max[k]==0)

```

```

        throw max[k];
        if (max[k]>m)
            m=max[k];
    }
    cout<<"Largest number is : "<<m;
}
catch (int k)
{
    cout<<"\n caught a null value";
}
}

```

OUTPUT

Enter five numbers : 4 5 8 7 0
caught a null value

Explanation: In the above program, an integer array *max [5]* is declared. The *cin* statement within the first *for* loop reads integers through the keyboard. The second *for* loop and the second *if* statement within it checks the successive numbers of arrays and stores the largest number in the variable *m*. Before this, the first *if* statement checks whether the number is zero. If the number is zero, an exception is thrown.

19.14 Write a program to enter a string and display it and throw an exception when null character is detected.

```

#include<iostream.h>

int main()
{ char text[20];
  int x=0;
  cout<<"\n Enter text:";
  cin.getline(text,20);
  try
  {
    while (1)
    {

      if (text[x]=='\0')
        throw text[x];
      cout<<text[x];
      x++;
    }
  }
  catch (char c)

```

```
{  
    cout<<"\n End of string";  
}  
return 0;  
}
```

OUTPUT

Enter text : Haste is Waste

Haste is Waste

End of string

Explanation: In the above program, a character array *text* [20] is declared. The `cin` statement accepts the string through the keyboard. The `while` loop and the statements within it are used to display the string. The `if` statement checks every character of the string to see whether it is a null character or other character. If it is a null character, an exception is thrown; otherwise, the character is displayed on the screen. Thus, at last when a null character is found, an exception is thrown. The message “*End of string*” is displayed by the `catch` block.

SUMMARY

- (1) A few languages support the exception-handling feature. Without this feature, the programmer needs to detect bugs on their own.
- (2) The errors may be logical errors or syntactic mistakes (syntax mistakes). The logical error remains in the program due to an unsatisfactory understanding of the program. The syntax mistakes are due to a lack of understanding of the programming language itself.
- (3) ANSI C++ is built in language functions for trapping errors and controlling exceptions. All C++ compilers support this newly added facility.
- (4) C++ provides a type-secure, integrated procedure for coping with the predictable but peculiar conditions that arise in run time.
- (5) The goal of exception handling is to create a routine that detects and sends an exceptional condition in order to execute suitable actions.
- (6) An exception is an object. It is sent from the part of the program where an error occurs to the part of the program that is going to control the error.
- (7) C++ exception method provides three keywords; they are `try`, `throw`, and `catch`. The keyword `try` is used at the starting of exception. The entire exception statements are enclosed in curly braces. It is known as `try` block.
- (8) The `catch` block receives the exception sent by the `throw` block in the `try` block.
- (9) We can also define multiple `catch` blocks; in the `try` block, such programs also contains multiple `throw` statements based on certain conditions.
- (10) It is also possible to define a single or default `catch` block from one or more exceptions of different types. In such a situation, a single `catch` block is used for catching exceptions thrown by the multiple `throw` statements.

- (11) It is also possible to again pass the exception received to another exception handler; that is, an exception is thrown from the `catch` block; this is known as a re-throwing exception.
- (12) The specified exceptions are used when we want to bind the function to throw only the specified exception. Specific exception is thrown using condition statement and list of data items in throw.

EXERCISES

(A) Answer the following questions

- (1) What do you mean by exception handling?
- (2) How many different types of errors are encountered in the program?
- (3) Describe the role of keywords `try`, `catch`, and `throw` in exception handling.
- (4) What will happen if the programming language does not support the exception-handling feature?
- (5) What do you mean by re-throwing of an exception?
- (6) Explain specifying exception with necessary steps.
- (7) How are multiple `catch` blocks defined?
- (8) What will happen if an exception is thrown for which no matching `catch` block is defined?
- (9) Explain guidelines for exception handling.
- (10) Explain the mechanism of exception handling.
- (11) Explain the use of function `set_terminate()`. How can we associate a function with this statement?

(B) Answer the following by selecting the appropriate option

- (1) The statement catches the exception
 - (a) **catch**
 - (b) `try`
 - (c) `throw`
 - (d) `template`
- (2) In the multiple `catch` statement, the number of the `throw` statement is
 - (a) **same as the `catch` statement**
 - (b) twice the `catch` statement
 - (c) only one `throw` statement
 - (d) none of the above
- (3) Exception is generated in
 - (a) **try block**
 - (b) `catch` block
 - (c) `throw` block
 - (d) none of the above
- (4) In exception handling, one of the following functions is implicitly invoked:
 - (a) **abort()**
 - (b) `exit()`
 - (c) `assert()`
 - (d) none of the above
- (5) What will happen if the `throw()` function has an empty exception and you place it after a function argument list?
 - (a) No exception will be thrown
 - (b) Default exception will be invoked
 - (c) The nearest one exception is thrown
 - (d) Unexpected actions by the program
- (6) If the `catch` statement is defined to catch an exception for the base class type object, it can also catch all _____-derived classes of the base class.
 - (a) **errors**
 - (b) parameters
 - (c) exceptions for objects
 - (d) none of the above

(c) Attempt the following programs

- (1) Write a program to accept 10 integers in an array. Check all numbers in the array. When any negative number is found, throw an exception.
- (2) Write a program to accept a string. When any space is encountered, throw an exception.
- (3) Write a program to explain the concept of multiple `catch` statements.
- (4) Write a program to use the `catch` statement.
- (5) Write a program to call a nested function. The function has an exception. Include essential exception-handling statements.

This page is intentionally left blank.

Overview of Standard Template Library

20 CHAPTER

CHAPTER OUTLINE

- 20.1 Introduction to STL
- 20.2 STL Programming Model
- 20.3 Containers
- 20.4 Sequence Containers
- 20.5 Associative Containers
- 20.6 Algorithms
- 20.7 Iterators
- 20.8 Vectors
- 20.9 Lists
- 20.10 Mapss
- 20.11 Function Objects

20.1 INTRODUCTION TO STL

We have studied about the template that enables us to do generic programming. Generic functions or classes support all data types. The standard template library is an advance application of templates. It contains several in-built functions and operators that help the programmer develop complex programs. The programmer only needs to include an appropriate header file to use the function or operator from the file, such as library functions. The STL library helps write a bug-free program. In case any bug presents, it is easy to debug it. For example, if a programmer wants to create a link list, he/she may require to write a program that may be of 40 to 50 lines. However, using STL functions (list algorithm), it is a task of a few minutes.

The Standard Template Library (STL) is a new feature of C++ language. All famous compiler vendors provide the STL as a feature of their compilers. STL is vast and heterogeneous

collection of reusable container classes. It consists of vectors, lists, queues, and stacks. The STL is portable with various operating systems.

The *Meng Lee* and *Alexander Stepanov* of Hewlett-Packard introduced the STL and accepted it in 1994 as an addition to the customary C++ library. The STL provides well-coded and compiled data structures and functions that are helpful in generic programming; and it is reusable. The STL topics are very vast; hence, their complete description is beyond the scope of this book. Only major features are explained. STL contents are defined in the namespace `std`. It is essential to write the statement `using namespace std` at the beginning of the program.

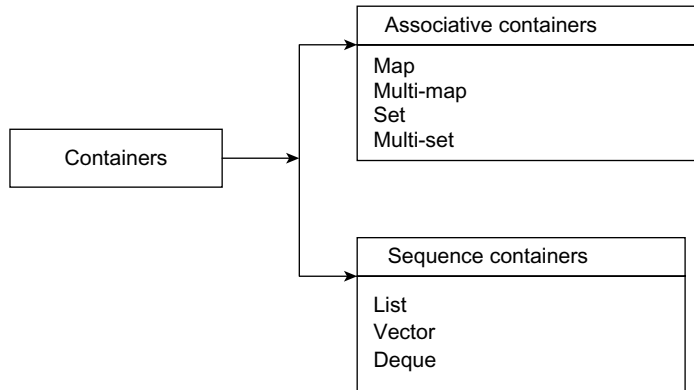
20.2 STL PROGRAMING MODEL

The STL is divided into three parts. They are containers, algorithms, and iterators. All these three parts can be used for different programming problems, and are closely associated with one another.

- (a) Containers: A container is an object that contains data or other objects. The standard C++ library has a number of container classes that allows the programmer to perform common tasks. These containers support generic programming and can be used for handling the data of different data types. All STL container classes are declared in namespace `std`; There are two types of containers, as shown in Figure 20.1: sequence container and associative container. Sequence containers are created to allow sequential and random access to members. Associative containers allow access to their elements through a key.
- (b) Algorithms: An algorithm is a technique that is useful to handle the data stored in containers. The STL comprises approximately 60 standard algorithms that give support to perform frequent and primary operations such as copying, finding, sorting, merging, and initializing. The standard algorithms are defined in the header file `<algorithm>`.
- (c) Iterators: An iterator is an object. It behaves exactly similar to a pointer. It indicates or points to the data element in a container. It is utilized to move between the data items of containers. Iterators can be incremented or decremented similar to pointers. They link algorithms with containers and handle the data stored in the containers.

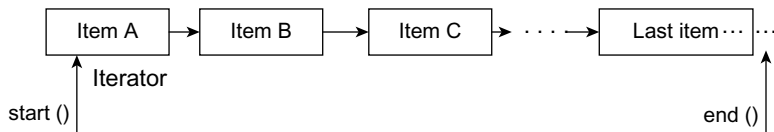
20.3 CONTAINERS

The standard C++ library has a sequence of container classes. The container classes are robust and support C++ programmers to control common programming operations. There are three types of STL container classes. They are sequence, associative, and derived containers. Sequence containers are developed to allow sequential and random access to all the members. Associative containers are expanded to get their elements by values. Derived containers such as stacks, queues, and priority queues can be created from various sequence containers. Containers are shown in Figure 20.1.

**Fig. 20.1** Classification of containers

20.4 SEQUENCE CONTAINERS

The STL sequence containers allow controlled sequential access to elements. Sequence containers hold data elements in a linear series, as observed in Figure 20.2. Every element is associated by its location within the series. All these elements enlarge themselves to enable the insertion of elements. These elements also allow various operations, such as copying and finding.

**Fig. 20.2** Data items in sequence container.

The STL has three types of sequence containers:

- (a) vectors
- (b) lists
- (c) deques

Iterators are used to get the elements in all these containers. All these containers have a different speed.

- (a) Vectors

We know that an array is used to store similar data elements. Elements of the array are stored in successive memory locations and are accessed in order from element number

0 onward. The vector class exactly acts similar to an array. This class is more secure and efficient than arrays.

It is used to insert or delete an element. All the elements are accessed randomly; that is, they support random access to individual elements.

A vector container class is accelerated to enable quick access to its elements in sequence. It is defined in the header file `<vector>`. A vector is able to enlarge itself. For example, if a vector declared for 5 elements is assigned 6 elements, then the vector automatically develops its size so that it can hold the 6th element. The vector class is declared in the following manner:

Declaration of Vector class
<code>template <class TE>, class B =allocator <TE> class vector</code>

Here, the class *TE* is the type of element in the vector; the class *B* is an allocator class. *M_allocators* are responsible for the allocation and release of memory. By default, the `new()` and `delete()` operators are also used for the allocation and release of memory. The default constructor of the class *TE* is invoked to produce a new element. This allows the entry of another parameter in order to define a default constructor for the user defined classes. The vectors that hold integers and floats are declared as follows:

Vector declaration for int and float data
<code>vector<int> vi // for integer elements</code>
<code>vector <float> vf // for float elements</code>

To declare a vector of 15 items, a constructor can be declared as follows:

```
vector<item> sale(15);
```

The compiler allocates memory for 15 elements. Here, the default constructor `item::item()` is used.

(b) Lists

The list container enables the programmer to perform the usual deletion and insertion of items. A list is also a sequence that can be accessed bi-directionally. It is defined in header file `<list>`. It acts as a double-linked list. Every node has a connection to both back and front nodes in the list. The iterator is used to transverse the link. The list class supports all the member functions of the vector class. The elements in the linked list are accessed using pointers. The list container has a technique known as an *iterator*, which is used to access the elements of the list container. An iterator is similar to a pointer. The iterator can be referenced similar to pointers to access elements.

(c) Deques

A deque is similar to a multiple-ended vector. Similar to the vector class, it has the ability to perform sequential read and write operations. The deque class allows improved front-end and back-end operations. Deques are perfectly appropriate for the operation that contains insertion and deletion at one or both ends and where sequential access is essential.

20.5 ASSOCIATIVE CONTAINERS

The associative container allows fast access to the object in the container. These containers are useful for huge dynamic tables in which we can search for an element randomly and sequentially. These containers use tree-like structures of elements instead of linked lists. These structures allow quick random update and retrieval operations. Associative containers are non-sequential and allow direct access to elements; these containers are divided into four categories:

- (1) sets
- (2) multisets
- (3) maps
- (4) multimaps

All the containers listed above hold data elements in a structure called a *tree*. The tree provides quick finding, deletions, and insertions. These containers perform very slowly in random access operations and are inappropriate for sorting operations.

Container sets and multisets store data elements. They also allow various operations to manage them. The variable name is used as a key name. Suppose a set contains objects of the `player` class. It can be sorted in ascending order using the names as keys. The multiset may have multiple sets of elements; that is, it permits duplicate data elements. The set does not allow duplicate elements.

Containers map and multimaps store both key names and values. The values are associated with key names. The values can be handled using key names. The values are also called *mapped values*. Multimaps allow various keys. Map containers permit single keys.

20.6 ALGORITHMS

Algorithms are independent template functions. They are not members or friend functions. They enable the programmer to manipulate the data stored in various containers. Algorithms carry out operations on containers by dereferencing iterators. Each container has its functions for performing common operations. An algorithm is nothing but a function template with arguments of iterator type. Algorithms receive iterators as arguments. The iterator informs the algorithm regarding the object of the container on which the operation is to be carried out. In addition, the STL contains approximately 60 standard algorithms. These functions enable quick operations in complicated and large operations. Including the `<algorithm>` header file, we can use these functions. The programmer reuses all these container classes.

Table 20.1 gives non-mutating sequence operations (algorithms).

Table 20.1 Non-mutating sequence operation

Operators	Use
<code>search_n()</code>	Searches a sequence of a given number of same elements.
<code>find_if()</code>	Searches first equivalent of a predicate in a sequence.
<code>search()</code>	Searches sub-sequence in other sequence.
<code>find_first_of()</code>	Searches a value from one sequence in another.
<code>find()</code>	Searches first presence of a value in a sequence.
<code>find_end()</code>	Searches last occurrence of a value in a sequence.

(Table 20.1 Contd.)

(Table 20.1 Contd.)

<code>adjacent_find()</code>	Searches contiguous pair of objects that are identical.
<code>mismatch()</code>	Searches element for which two sequences vary.
<code>count()</code>	Calculate presence of a value in a sequence.
<code>count_if()</code>	Calculate elements that similar a predicate
<code>equal()</code>	True if two series' are the identical.
<code>for_each()</code>	Performs a operation with each element.

Non-mutating sequence algorithms enable operations that do not alter the elements in a sequence. For example, the operators `for_each()`, `find()`, `search()`, and `count()`. The following program explains how to use the `for_each()` algorithm:

20.1 Write a program to demonstrate use of operator `for_each()`.

```
#include<iostream>
#include<vector>
#include<algorithm>

using namespace std;
template <class S>
class show
{
public:
void operator() ( const S& s)
{
    cout<<s;
}
};

int main()
{
    show<int> showvalue;
    vector<int> vi(4);
    for ( int j=0;j<4;++j)
        vi[j]=j;
    cout<<" for_each() \n";
    for_each(vi.begin(),vi.end(),showvalue);
    cout<<"\n";
    return 0;
}
```

OUTPUT

0123

Explanation: The operator () is overloaded in the class show. The showvalue is an object of the class show. A vector vi is created, which can hold four elements. Using the first for loop, 0 to 3 numbers are inserted in the vector. While reading numbers from the vector instead of using the for loop, the operator for_each() is used. The first argument of in for_each() operator (vi.begin()) gives the reference of the first element. The second argument gives the reference of the last element of the vector, and the third argument (showvalue) is a function object. Thus, using the for_each() operator, members of the vector can be accessed.

20.2 Write a program to demonstrate use of fill () algorithm.

```
#include<iostream>
#include<vector>
#include<algorithm>

using namespace std;
template <class S>

class show
{
    public:
    void operator()
    ( const S& s)
    {
        cout<<s;
    }
};

int main()
{
    show<int> showvalue;
    vector<int> vi(8);
    fill(vi.begin(),vi.begin() + 2,5);
    fill(vi.begin()+2,vi.begin() + 4,6);
    fill(vi.begin()+4,vi.end(),7);
    cout<<" for_each() \n";
    for_each(vi.begin(),vi.end(),showvalue);
    cout<<"\n";
    return 0;
}
```

OUTPUT

55667777

Explanation: This program is similar to the previous one. In addition, the algorithm fill () is used. The fill () is a mutating algorithm, because it changes the elements of the vector. The following statements are used to fill the element in the vector:

```
fill(vi.begin(), vi.begin() + 2, 5);
```

The above statement fills the first 2 elements with the value 5. The function `begin()` gives a beginning reference, and `begin() + 2` gives a reference for the second.

```
fill(vi.begin()+2, vi.begin() + 4, 6);
```

The above statement fills the value 6 at the third and fourth locations of the vector. Here also, the `begin()` function is used.

```
fill(vi.begin()+4, vi.end(), 7);
```

The above statement fills the value 7 from location number 5 to the end.

Table 20.2 describes sorting operations (algorithms).

Table 20.2 Sorting operations

Operators	Use
<code>binary_search()</code>	Performs a binary search on an indexed sequence
<code>equal()</code>	Searches whether two sequences are equal
<code>equal_range()</code>	Searches a sub-range of elements with a specified value
<code>includes()</code>	Searches if a sequence is a sub-sequence of another sequence
<code>inplace_merge()</code>	Combines two successive sorted sequences
<code>lexicographical_compare()</code>	Compares in ascending order one sequence with other
<code>lower_bound()</code>	Searches the first occurrence of a given value
<code>make_heap()</code>	Makes a heap from a sequence
<code>max()</code>	Returns greatest of two values
<code>max_element()</code>	Returns the highest elements inside a sequence
<code>merge()</code>	Combines two sorted sequences
<code>min()</code>	Returns smallest of two values
<code>min_element()</code>	Returns the smallest number of a sequence
<code>mismatch()</code>	Searches the mismatch among the elements of two sequence
<code>nth_elements()</code>	Places given elements in its appropriate places
<code>partial_sort()</code>	Sorts a portion of a sequence
<code>partial_sort_copy()</code>	Sorts a portion of a sequence and copies
<code>pop_heap()</code>	Erases the uppermost elements
<code>push_heap()</code>	Appends or adds an element to heap
<code>set_difference()</code>	Builds a sequence that is the variance of two ordered sets
<code>set_intersection()</code>	Makes a sequence that holds the intersection of ordered sets
<code>set_symmetric_difference()</code>	Yields a set that is the symmetric variance between two ordered sets
<code>set_union()</code>	Yields sorted union of two ordered sets
<code>sort()</code>	Sorts the sequence container

(Table 20.2 Contd.)

(Table 20.2 Contd.)

<code>sort_heap()</code>	Sorts a heap
<code>stable_partition()</code>	Puts elements matching an estimate first matching relative order
<code>stable_sort()</code>	Sorts arranging order of similar elements
<code>upper_bound()</code>	Searches the last occurrence of a given value

Table 20.3 describes mutating sequence operations (algorithms).

Table 20.3 Mutating sequence operations

Operators	Use
<code>swap_ranges()</code>	Exchange two sequences
<code>generate()</code>	Substitutes all elements with the result of an operation
<code>copy_backward()</code>	Duplicate (copies) a sequence from the ends
<code>fill()</code>	Fill up a series with a given value
<code>reverse()</code>	Opposites (reverse) the order of elements
<code>fill_n()</code>	Fill up first n elements with a given value
<code>copy()</code>	Duplicates (copies) a sequence
<code>unique()</code>	Erases similar contiguous elements
<code>generate_n()</code>	Substitutes first n elements with the result of an operation
<code>iter_swap()</code>	Exchanges elements pointed to by iterator
<code>random_shuffle()</code>	Inserts elements in random order
<code>remove()</code>	Erases elements of a given value
<code>replace()</code>	Substitutes elements with a specified value
<code>replace_if()</code>	Substitute elements matching a predicate
<code>remove_copy_if()</code>	Duplicates (copies) a sequence subsequently removing elements matching a predicate
<code>unique_copy()</code>	Copies after removing similar contiguous elements
<code>rotate()</code>	Rotates (turns) elements
<code>remove_if()</code>	Erases elements matching a predicate
<code>remove_copy()</code>	Duplicates (copies) a sequence subsequently removing a given value
<code>replace_copy()</code>	Duplicates (copies) a sequence substituting elements with a specified value
<code>transform()</code>	Applies an action to all elements
<code>replace_copy_if()</code>	Duplicates (copies) a sequence substituting elements matching a predicate
<code>rotate_copy()</code>	Issues a sequence into rotate sequence
<code>swap()</code>	Exchange two elements
<code>reverse_copy()</code>	Copies a sequence into opposite direction

Numeric algorithms are provided in Table 20.4

Table 20.4 Numeric operations

Operators	Use
<code>Inner_product()</code>	Collects the details of operation on a pair of sequence
<code>Adjacent_difference()</code>	Creates a sequence from another sequence
<code>Accumulate()</code>	Collects the return values of operation on a sequence
<code>partial_sum()</code>	Creates a sequence by applying an operation on a pair of sequence

20.7 ITERATORS

Iterators acts as pointers. They are used for accessing contents of container classes. They allow movement from one element to another element, and this is known as iterating. Each container type supports one kind of iterator according to the container's necessity. The types are input, output, forward, bi-directional, and random access. The hierarchy of iterators is shown in Figure 20.3 and described in Tables 20.5 and 20.6.

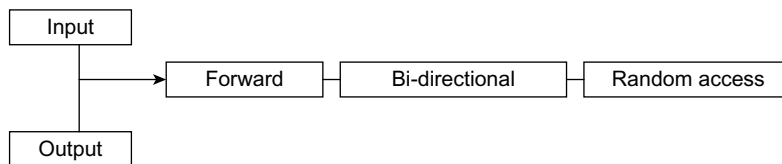


Fig. 20.3 Iterator hierarchy

Table 20.5 Operation of container classes

Iterator	Access mode	Way of action	I/O ability	Comments
Forward	Linear	Can move forward only	Can Write & read	Storable
Bi-directional	Linear	Can move Front & back	Can write & read	Storable
Random	Random	Can move Front & back	Can write & read	Storable
Input	Linear	Can move Frontward only	Can Read only	Non-storable
Output	Linear	Can move Frontward only	Can Write only	Non-storable

Table 20.6 Iterator status

Iterator status	Meaning
singular	The value of iterator does not dereference in any container.
Past-the-end	The iterator addresses to the object past the last object in the container.
dereferenceable	The iterator addresses the legal object in the container.

Iterators are used by the algorithms to carry out operations. Iterators are smart pointers. They are allowed to contain values that indicate one of the statuses of iterators as described in Table 20.10. Each iterator type supports all the attributes listed in Table 20.9 and shown in Figure 20.3. Iterators can be incremented, decremented, and their limits are up to the capacity of the containers. Containers have member functions that return iterators.

20.3 Write a program to transverse list using iterator.

```
#include<iostream>
#include<list>

using namespace std;
typedef list<int> num_list;

int main()
{
    num_list num;
    for (int j=0; j<=5; ++j)
        num.push_back(j);
    for (num_list::const_iterator ls=num.begin(); ls!=num.
end(); ++ls)
        cout<<*ls<<" ";
    return 0;
}
```

OUTPUT

0 1 2 3 4 5

Explanation: In the above program, *num* is an object of type *num_list*. The member function *push_back()* adds a number in the list. In the *for* loop, the statement *num_list::const_iterator ls = num.begin();* declares an *iterator* *ls* and initializes with the beginning reference of the list (reference of the first element). The second statement *ls! = num.end()* continues the loop until the *ls* points to the last element of the list. The statement *++ls* dereference the iterator so that successive numbers are accessed.

20.8 VECTORS

A vector is a more useful container. Similar to an array, it also stores elements in neighboring memory locations. Each element can be directly accessed using the operator[]. A vector is able to enlarge its size if an extra element is assigned to it. A class vector has various constructors that can be used to create vector objects.

The vector class supports various functions and they are listed in Table 20.7.

Table 20.7 Functions of vector class

Function	Use
swap()	Swap the elements in the given two vector
size()	Provides the number of elements
resize()	Changes the size of the vector
push_back()	Appends an element to the end
pop_back()	Erases the last elements
insert()	Inserts items (elements) in the vector
erase()	Erases given elements
end()	Provides reference to the end of the vector
empty()	Determine whether the vector is empty
clear()	Erases entire elements from the vector
capacity()	Provides the present capacity (limit) of the vector
begin()	Provides a reference to the starting element
back()	Provides a reference to the last element
at()	Provides a reference to an element

20.4 Write a program to add and display elements in the vector object.

```
#include<iostream>
#include<vector>

using namespace std;

int main()
{
    vector<int> e;
    e.push_back(5);
    e.push_back(8);
    e.push_back(9);
    cout<<"\n The elements are : "<<"\n";
    cout<<e[0]<<" ";
    cout<<e[1]<<" ";
    cout<<e[2]<<" ";
    return 0;
}
```

{coderipe}

OUTPUT

The elements are :
5 8 9

Explanation: In the above program, *e* is an object of the vector class that can hold data of integer type. The member function *push_back()* is used to add the element in the vector object. Three integer elements 5, 8, and 9 are added to the vector. These elements are stored in contiguous memory locations. These elements can be accessed in the same manner as we access the array elements. The vector object name followed by the element number in the [] operator displays the specified number. Here, the overloaded operator [] is used.

20.5 Write a program to insert an element in the vector object. Use member function and iterator.

```
#include<iostream>
#include<vector>

using namespace std;

int main()
{
    vector<int> e;
    for (int x=0;x<3;x++)
        e.push_back(x+1);
    cout<<"\n The elements are:"<<"\n";
    for (x=0;x<3;x++)
        cout<<e[x]<<" ";
    vector <int> :: iterator it =e.begin();
    it=it+1;
    e.insert(it,7);
    cout<<"\n The elements after insertion:"<<"\n";
    for (x=0;x<4;x++)
        cout<<e[x]<<" ";
    return 0;
}
```

OUTPUT

The elements are :

1 2 3

The elements after insertion :

1 7 2 3 Press any key to continue

Explanation: In the above program, the first for loop and the function *push_back()* add elements to the object *e*. The elements indicated by the loop variable *x* are stored in the object *e*. The second for loop and *cout ()* statement display the contents of the object *e*.

The *it* is an iterator. The statement *vector <int> :: iterator it = e.begin()* declares the iterator. The iterator acts similar to a pointer. The *begin()* function is used to assign the reference (address) of the first element. The statement *it = it+1* assigns the address of the second element where the

element is to be inserted. The member function is invoked with two arguments as per the *g* statement *e.insert(it, 7)*.

The first argument is an iterator that indicates the location, and the second argument is an integer value. The number 7 is inserted at the location indicated by the iterator *it*. The last `for` loop displays the contents of the vector object *e* after insertion.

20.6 Write a program to display the elements of vector object in ascending and descending orders.

```
#include<iostream>
#include<vector>

using namespace std;

int main()
{
    vector<int> e;
    cout<<"Elements in ascending order:";
    for (int x=0;x<8;x++)
    {
        e.push_back(x+1);
        cout<<x+1 <<" ";
    }
    int s=e.size()-1;
    cout<<"\n Elements in descending order:";
    for (int j=s;j>=0;j--)
        cout<<e[j]<<" ";
    cout<<"\n";
    return 0;
}
```

OUTPUT

Elements in ascending order : 1 2 3 4 5 6 7 8

Elements in descending order : 8 7 6 5 4 3 2 1

Explanation: In the above program, the first `for` loop and the member function *push_back()* are used to add numbers in the vector element *e*. The `cout ()` statement in the same loop displays the elements. All the elements are stored and displayed in ascending order. The member function *size()* returns the total number of the elements. The total number of the elements is stored in the integer variable *s*. The second `for` loop is executed in reverse order, and the value of the loop variable *j* is used with the object *e* to display the number. Thus, all the numbers are displayed in the reverse order.

20.7 Write a program to remove elements from vector object.

```

#include<iostream>
#include<vector>
#include<math.h>

using namespace std;

int main()
{
    vector<float> e;
    cout.precision(2);
    cout<<"\n Original elements:";
    for (int k=5;k<12;k++)
    {
        e.push_back(sqrt(k));
        cout<<sqrt(k)<<" ";
    }
    vector <float> ::iterator ir=e.begin();
    e.erase (ir+3,ir+5);
    int s=e.size();
    cout<<"\n The elements after erase():";
    for (k=0;k<s;k++)
        cout<<e[k]<<" ";
    cout<<"\n";
    return 0;
}

```

OUTPUT

Original elements : 2.2 2.4 2.6 2.8 3 3.2 3.3

The elements after erase() : 2.2 2.4 2.6 3.2 3.3

Explanation: In the above program, *e* is an object of the vector that is capable of storing float values. The square roots of 5 to 12 numbers are stored in the object *e*. The *erase()* function erases the 4th and 5th elements. The variable *ir* is an iterator that gives the reference of the element. The *erase()* can be used with one or more argument. The last *for* loop displays the elements after deletion.

20.8 Write a program to declare a vector object. Add elements and display them. Use member functions.

```

#include<iostream>
#include<vector>

```

```
using namespace std;

{
    for (int j=0;j<e.size();j++)
    {   cout<<e[j];
        cout<<"\t";
    }
}

int main()
{
    vector<int> e;
    cout<<"Initial size of e= " <<e.size() <<"\n";
    int k;
    cout<<"\n Enter five integers values:";
    for (int j=0;j<3;j++)
    {
        cin>>k;
        e.push_back(k);
    }

    cout<<" Size of e after adding 3 values:";
    cout<<e.size();
    cout<<"\n current contents:"<<"\n";
    show(e);
    return 0;
}
```

OUTPUT

Initial size of e= 0

Enter five integers values : 7 3 4

Size of e after adding 3 values : 3

Current contents :

7 3 4

Explanation: In the above program, *e* is an object of the vector class that is capable of storing integer values. The *size()* member function gives the number of elements stored in the object *e*. The *for* loop and the *cin* statement within it reads three integers through the keyboard. The member function *push_back()* adds the element to the object *e*. It adds the element at the end. After adding elements again, the *size()* function displays the number of elements. It displays 3, that is the number of total elements added in the object *e*. The *show()* function is used to display the contents of the object *e*. The *for* loop and the statement *cout <<e[j]* are used to display the elements. The overloaded operator *[]* is used to access the specified element by the variable *j*.

20.9 LISTS

The list is a frequently used feature. It allows a bi-directional linear list. Element can be inserted or deleted at both the ends. The elements in the list can be accessed sequentially. Iterators are used for accessing individual elements of the list.

The list class supports various functions and they are listed in Table 20.8.

Table 20.8 Functions of list class

Function	Task
clear()	Erases entire elements
back()	Provides reference to the end elements
empty()	Determines if the list is vacant or not
begin()	Provides reference to the first element
erase()	Erases given element
end()	Provides reference of the end element of the list
merge()	Combines two sorted lists
pop_fornt()	Erases the first elements
pop_back()	Erases the last elements
remove()	Erases elements as specified
reverse()	Reverse the list elements
insert()	Adds given element
push_back()	Appends an element to the end
push_front()	Appends an element to the front
size()	Provides the sizes of the list
unique()	Erases the identical elements in the list
resize()	Changes the size of the list
swap()	Swaps the element of a list with those in the calling list
sort()	Sorts the list elements

20.9 Write a program to create list of four numbers and display them.

```
#include<iostream>
#include<list>

using namespace std;

void show( list <int> &num)
{ list<int> :: iterator n;
  for (n=num.begin();n!=num.end(); ++n)
    cout<<*n<<" ";
```

```

}
int main()
{
    list<int> list;
    list.push_back(5);
    list.push_back(10);
    list.push_back(15);
    list.push_back(20);
    cout<<"Numbers are";
    show(list);
    return 0;
}

```

OUTPUT

Numbers are 5 10 15 20

Explanation: In the above program, *list* is the object of *list* container class that is able to hold integer values. The *push_back()* functions adds elements to the list. Four integer numbers are added in the list. The *show()* function is used to display the numbers on the screen. The *num* reference variable receives the reference of the first element. The iterator *n* is used to display the numbers. It is used exactly as a pointer.

20.10 Write a program to add element to both ends of the list and display the numbers.

```

#include<iostream>
#include<list>

using namespace std;

void show( list<int> &num)
{
    list<int> :: iterator n;
    for (n=num.begin(); n!=num.end(); ++n)
        cout<<*n<<" ";
}

int main()
{
    list<int> list;
    list.push_back(5);
    list.push_back(10);
    list.push_back(15);
    list.push_back(20);
    cout<<"\nNumbers are";
}

```

```

    show(list);
    list.push_front(1);
    list.push_back(25);
    cout<<"\nAfter adding Numbers are";
    show(list);
    return 0;
}

```

OUTPUT

Numbers are 50 10 15 20

After adding Numbers are 1 5 10 15 20 25

Explanation: The above program is similar to the previous one. In addition, here two elements are added to the front and rear ends of the list. The function `push_front()` adds elements to the front end of the list, and the function `push_back()` appends elements to the rear end of the list. The output shows all the elements of the list.

20.11 Write a program to delete elements from the list.

```

#include<iostream>
#include<list>

using namespace std;

void show( list <int> &num)
{
    list<int> :: iterator n;
    for (n=num.begin();n!=num.end(); ++n)
        cout<<*n<<" ";
}

int main()
{
    list <int> list;
    list.push_back(5);
    list.push_back(10);
    list.push_back(15);
    list.push_back(20);
    cout<<"\nNumbers are ";
    show(list);
    list.pop_front();
    list.pop_back();
    cout<<"\nAfter deleting Numbers are";
}

```

```

    show(list);
    return 0;
}

```

OUTPUT

Numbers are 5 10 15 20

After deleting Numbers are 10 15

Explanation: In the above program, the member function `pop_front()` is used to delete the front element of the list, and the function `pop_back()` is used to delete the back element of the list. The output shows the list of numbers.

20.12 Write a program to sort the list elements and display them.



```

#include<iostream>
#include<list>

using namespace std;

void show( list <int> &num)
{
    list<int> :: iterator n;
    for (n=num.begin();n!=num.end(); ++n)
        cout<<*n<<" ";
}

int main()
{
    list <int> list;
    list.push_back(23);
    list.push_back(19);
    list.push_back(5);
    list.push_back(15);
    list.push_back(25);
    list.push_back(20);
    cout<<"\n Unsorted list:";
    show(list);
    cout<<"\n Sorted list:";
    list.sort();
    show(list);
    return 0;
}

```

OUTPUT**Unsorted list :23 19 5 15 25 20****Sorted list : 5 15 19 20 23 25**

Explanation: In the above program, the `push_back()` function is used to add elements to the `list` object. The function `sort()` is used to sort the list element in ascending order. The program shows the list of elements in sorted and unsorted elements using the `show()` function.

20.13 Write a program to display the elements in reverse order.

```
#include<iostream>
#include<list>

using namespace std;
void show( list <int> &num)
{
    list<int> :: iterator n;
    for (n=num.begin();n!=num.end(); ++n)
        cout<<*n<<" ";
}
int main()
{
    list <int> list;
    list.push_back(23);
    list.push_back(19);
    list.push_back(5);
    list.push_back(15);
    list.push_back(25);
    list.push_back(20);
    cout<<"\n Original list:";
    show(list);
    cout<<"\n Reverse list:";
    list.reverse();
    show(list);
    return 0;
}
```

OUTPUT**Original list :23 19 5 15 25 20****Reverse list : 20 25 15 5 19 23**

Explanation: The above program is similar to a previous one. Here, the *reverse()* function arranges the list elements in reverse order. The output shows the list of elements in original and reversed order.

20.14 Write a program to copy elements of one list object to another object. Display the contents of both the objects.

```
#include<iostream>
#include<list>

using namespace std;

void show( list <int> &num)
{
    list<int> :: iterator n;
    for (n=num.begin();n!=num.end(); ++n)
        cout<<*n<<" ";
}

int main()
{
    list <int> listX,listY;
    listX.push_back(23);
    listX.push_back(19);
    listX.push_back(5);
    listX.push_back(15);
    listX.push_back(25);
    listX.push_back(20);
    listY=listX;

    cout<<"\n Elements of listX:";
    show(listX);
    cout<<"\n Elements of listY:";
    show(listY);

    return 0;
}
```

OUTPUT

Elements of listX :23 19 5 15 25 20
Elements of listY :23 19 5 15 25 20

Explanation: In the above program, listX and listY are two objects of the list container. The listX is initialized with six integer elements using the push_back() functions. The statement listY = listX copies elements of the listX object to the listY object.

20.15 Write a program to merge two lists and display the merged list.

```
#include<iostream>
#include<list>

using namespace std;
void show( list <int> &num)
{
    list<int> :: iterator n;
    for (n=num.begin();n!=num.end(); ++n)
        cout<<*n<<" ";
}

int main()
{
    list <int> listX,listY;
    listX.push_back(23);
    listX.push_back(19);
    listX.push_back(5);

    listY.push_back(15);
    listY.push_back(25);
    listY.push_back(20);
    cout<<>>\n Elements of listX:>>;
    show(listX);
    cout<<>>\n Elements of listY:>>;
    show(listY);

    listX.merge(listY);
    cout<<>>\n Merged list: <<;
    show(listX);
    return 0;
}
```

OUTPUT

Elements of listX :23 19 5

Elements of listY :15 25 20

Merged list : 15 23 19 5 25 20

Explanation: In the above program, the objects `listX` and `listY` are initialized with three objects each with the function `push_back()`. The `merge()` function merges the elements of two list objects into one. The elements are merged in the calling object. Thus, in this program, `listX` calls the function, and `listY` is sent as an argument. The `listX` contains its own elements and the elements of `listY`. In the output, the contents of objects `listX` and `listY` are displayed.

20.10 MAPS

A map is a series of pairs of key names and values associated with it as per Figure 20.4. Access of data values depends on the key, and it is very quick. We should specify the key to get the corresponding value.

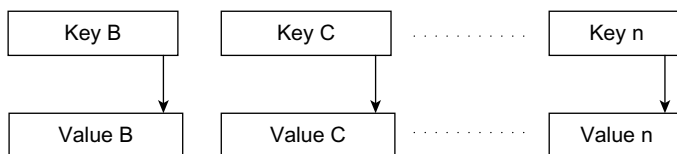


Fig. 20.4 Key and values in maps

20.16 Write a program to manipulate list of item names and their code numbers using maps.

```

#include<iostream>
#include<map>
#include<string>
using namespace std;

typedef map<string,int> item_map;

int main()
{
    int sz;
    string item_name;
    int codeno;
    item_map item;
    cout<<"Enter item name and code no number for 2 items: \n";
    for (int i=0;i<2;i++)
    {
        cin>>item_name;
        cin>>codeno;
    }
}

```

```

        item[item_name]=codeno;
    }
    item["PC"]=2510;

    item.insert(pair<string,int> ("printer",2211));
    sz=item.size();
    cout<<"\n Size of map:"<<sz <<"\n\n";
    cout<<"List of item name and code numbers \n";
    item_map::iterator t;
    for ( t=item.begin();t!=item.end();t++)
    cout<<(*t).first <<" " <<(*t).second<<"\n";cout<<"\n";
    cout<<"Enter item name:";
    cin>>item_name;
    codeno=item[item_name];
    cout<<"Code Number:"<<codeno <<"\n";
    return 0;
}

```

OUTPUT

Enter item name and code no number for 2 items:

TV 51

CD 52

size of map :4

List of item name and code numbers

CD 52

PC 2510

TV 51

printer 2211

Enter item name : PC

Code Number : 2510

Explanation: In the above program, using typedef statement `item_map`, a map variable is created. The first `for` loop and `cin` statement read two records through the keyboard. Using the member function `insert()` and assignment, two records are inserted. The `size()` member function returns the number of records. The variable `t` is an iterator to the map. The second `for` loop and the iterator produce the list of item names and code numbers. At the end, the program displays a code number that is associated with the entered item name. The `item_name` is a key. The list is automatically arranged in sorted order.

A map is generally also known as an *associative array*. The key is given with the help of the subscript operator [] in the following manner:

```
item["PC"]=2510;
```

The above statement creates a record for "PC" and links it with code number 2510. The *codeno* is an object. It is also possible to insert and remove pairs at any point in the map. The `insert()` `delete()` function performs these tasks. Frequently used functions are described in Table 20.10.

20.17 Write a program to clear map using `clear()` function.

```
#include<iostream>
#include<map>
#include<string>

using namespace std;

typedef map<string,int> item_map;

int main()
{
    int sz;
    string item_name;
    int codeno;
    item_map item;

    item["PC"]=2510;

    item.insert(pair<string,int> ("printer",2211));
    sz=item.size();
    cout<<"\n Size of map:"<<sz <<"\n\n";

    item.clear();
    sz=item.size();
    cout<<"\n after clear() Size of map:"<<sz <<"\n\n";
    return 0;
}
```

OUTPUT

Size of map :2

after clear() Size of map :0

Explanation: In the above program, two records are added in the map `item_map`. Using the `size()` function, the size (total number of records) is displayed; that is, the `clear()` function clears the contents of the map. After the `clear()` function, the size of the map becomes zero.

Table 20.9 describes important functions of map class.

Table 20.9 Important functions of the map class

Function	Task
<code>clear()</code>	Removes all elements from the map
<code>begin()</code>	Provides references of the first element
<code>erase()</code>	Removes the given elements
<code>insert()</code>	Inserts the elements as given
<code>empty()</code>	Determines whether the map is vacant or not
<code>end()</code>	Provides references to the end of the map
<code>size()</code>	Provides the size of the map
<code>find()</code>	Provides the location of the given elements
<code>swap()</code>	Swaps the elements of the given map with those of the calling map

20.11 FUNCTION OBJECTS

A function object is a function. It is embedded in a class and hence behaves similar to an object. The class has same characteristics of a template and can be put to use with different data types. The class holds only one member function and the overloaded () operator and never accommodates data. Function objects are frequently used as parameters for algorithms and containers.

```
sort ( num,num+3, greater <float>( ) );
```

where `num[]` is an array. We can use an object `greater<float>( )` to sort elements of the array in descending order. The standard template library also contains several pre-defined objects. These objects perform arithmetical and logical operations. The header file `<functional>` should be included while performing these operations. There are function objects that match foremost C++ operators. The variables `a` and `b` are objects of the class. `K` is a parameter sent to the function object. All the function objects described in Tables 20.10, 20.11, and 20.12 are defined in the *functional* header file.

Table 20.10 Relational function objects

Function object	Narration
<code>not_equal_to<K></code>	<code>a != b</code>
<code>equal_to<K></code>	<code>a == b</code>
<code>greater<K></code>	<code>a > b</code>
<code>greater_equal<K></code>	<code>a >= b</code>
<code>less<K></code>	<code>a < b</code>
<code>less_equal<K></code>	<code>a <= b</code>

Table 20.11 Logical function objects

Function object	Narration
<code>logical_and<K></code>	<code>a && b</code>
<code>logical_not<K></code>	<code>! a</code>
<code>logical_or<K></code>	<code>a b</code>

Table 20.12 Arithmetic function objects

Function object	Narration
<code>plus<K></code>	<code>a + b</code>
<code>divides<K></code>	<code>a / b</code>
<code>modulus<K></code>	<code>a % b</code>
<code>multiplies<K></code>	<code>a * b</code>
<code>minus<K></code>	<code>a - b</code>
<code>negate<K></code>	<code>- a</code>

20.18 Write a program to sort an array using function object.

```
#include<iostream>
#include<algorithm>
#include<functional>

using namespace std;
int main()
{
    float a[]={1.5,6.5,2.5};
    float b[]={6.4,4.8,1.4};
    sort (a,a+3,greater<int>());
    sort (b,b+3);
    for (int j=0;j<3;j++) cout<<a[j]<<" ";
    cout<<"\n";
    for (int k=0;k<3;k++)      cout<<b[k]<<" ";
    cout<<"\n";
    float s[6];
    merge (a,a+3,b,b+3,s);
    for (j=0;j<6;j++)  cout<<s[j] <<" ";
    cout<<"\n";
    return 0;
}
```

```
}
```

OUTPUT

```
6.5 2.5 1.5
1.4 4.8 6.4
1.4 4.8 6.4 6.5 2.5 1.5
```

Explanation: In the above program, two float arrays `a [ ]` and `b [ ]` are declared and initialized. Elements of two arrays are sorted by `sort ( )` function. The array `a [ ]` is sorted with the help of the function object `greater<float>`, and the array `b [ ]` is sorted without the function. At the end, the `merge ( )` function combines elements of both the arrays, and the resulting array is stored in the array `s [ ]`. The contents of the merged array are displayed.

20.19 Write a program to create function object.

```
#include<iostream>
using namespace std;
template <class S>
class show
{
public:
void operator() ( const S& s)
{
    cout<<s;
}
};
int main()
{
    show<int> showvalue;
    for (int j=0;j<8;++j)          showvalue(j);
}
```

OUTPUT

```
01234567
```

Explanation: In the above program, the `operator()` is overloaded in the class `show`. The `showvalue` is an object of the class `show`. The overloaded `operator()` can be used with an object of the `class show` type similar to the statement `showvalue(j)`; to display the value; so, it is called a *function object*.

SUMMARY

- (1) The STL is a new expansion in C++ language. The all-famous compiler vendors give the STL as a feature of their compilers.
- (2) The Meng Lee and Alexander Stepanov of Hewlett Packard introduced the STL.
- (3) The STL is portative between different operating systems. The STL contents are defined in the namespace `std`.
- (4) The important subdivisions are as follows: (A) Containers, (B) Algorithms, (C) Iterators.
- (5) A container is an object that holds data or other objects. An algorithm is a process that is useful to handle the data stored in containers. The STL consists of by-guess 60 standard algorithms that give support to perform frequent and primary operations such as copying, finding, sorting, merging, and initializing.
- (6) An iterator is an object that is treated as a pointer. It indicates or points to data elements in a container. It is utilized to move between the data of containers.
- (7) The standard C++ library has a sequence of container classes. The container classes are robust and support C++ programmers in controlling common programming operations.
- (8) Sequence containers hold data elements in a linear series.
- (9) We know that an array is used to store similar data elements. The data elements are stored in continuous memory locations. The container class `vector` is similar to an array.
- (10) A vector container class is accelerated to allow quick access to its elements in sequence.
- (11) The list container allows the programmer the usual insertion and deletion of items. It is defined in header file `<list>`. It acts as a double-linked list.
- (12) A deque is similar to a multiple-ended vector. It has the ability similar to a vector class to perform sequential read and write operations. The deque class allows improved front-end and back-end operations.
- (13) The `stack`, `queue`, and `priority queues` are called *container adaptors*. All these container adaptors can be produced from unlike sequence containers.
- (14) Algorithms are independent template functions. They are not member or friend functions. They enable the programmer to manipulate the data stored in various containers. Each container has its functions for performing common operations.
- (15) Iterator acts as pointers. They are used for accessing contents of container classes.
- (16) The vector is a more useful container. Similar to an array, it stores elements in neighboring memory locations. Each can be directly accessed using the operator `[]`. A vector is able to enlarge its size if an extra element is assigned to it.
- (17) The list is a frequently used feature. It allows a bi-directional linear list. Elements can be inserted or deleted at both the ends.
- (18) A map is a series of pairs of key names and values associated with it.

EXERCISES

(A) Answer the following questions

- (1) What is STL? Explain in brief.
- (2) Describe the different sub-divisions of STL with their definitions.
- (3) What are containers? Describe the types of containers with their components.
- (4) What do you mean by algorithms?
- (5) What are iterators?
- (6) What is the difference between an iterator and a pointer?
- (7) Describe any four functions of the vector class with their operations.
- (8) Describe any four functions of the list class with their operations.
- (9) What is a map? How does it work?
- (10) What do you mean by a function object?
- (11) Describe any four functions of the map class with their operations.
- (12) What are function objects?

(B) Answer the following by selecting the appropriate option

- (1) A container is an object that
 - (a) **stores data**
 - (b) temporarily holds data
 - (c) both (a) and (b)
 - (d) none of the above
- (2) An algorithm is a process that
 - (a) **processes the data**
 - (b) stores the data
 - (c) sorts the data
 - (d) none of the above
- (3) An iterator is similar to
 - (a) **a pointer**
 - (b) an array
 - (c) a class
 - (d) none of the above
- (4) The function object is similar to
 - (a) **an object**
 - (b) a variable
 - (c) both (a) and (b)
 - (d) none of the above
- (5) Algorithms are a part of
 - (a) **a standard template library**
 - (b) a class template
 - (c) an iostream
 - (d) none of the above

(C) Attempt the following programs

- (1) Write a program to demonstrate the use of operators `for_each()`.
- (2) Write a program to create function objects.
- (3) Write a program to manipulate a list of item names and their code numbers using maps.
- (4) Write a program to copy the elements of one list object to another object. Display the contents of both the objects.
- (5) Write a program to merge two lists and display the merged list.
- (6) Write a program to demonstrate the use of the `fill()` algorithm.
- (7) Write a program to transverse lists using iterators.
- (8) Write a program to add and display elements in the vector object.
- (9) Write a program to add elements to both ends of the list and display the numbers.
- (10) Write a program to insert an element in the vector object. Use member functions and iterators.

This page is intentionally left blank.

Additional Information about ANSI and TURBO-C++

21 CHAPTER

CHAPTER OUTLINE

- 21.1 Introduction
- 21.2 Innovative Data Types
- 21.3 New Type-Casting Operators
- 21.4 The Keyword `explicit`
- 21.5 The Keyword `mutable`
- 21.6 Namespace Scope
- 21.7 Nested Namespaces
- 21.8 Anonymous Namespaces
- 21.9 The Keyword `using`
- 21.10 Namespace Alias
- 21.11 The Standard Namespace `std`
- 21.12 ANSI and Turbo-C++ Keywords
- 21.13 ANSI and Turbo-C++ Header Files
- 21.14 C++ Operator Keywords

21.1 INTRODUCTION

This chapter deals with various concepts that have not been discussed so far, including new improvements suggested by the ANSI committee in C++ language. In addition, this chapter also contains information regarding additional keywords provided by Turbo C++. A list of commonly useful header files is also given.

The aim behind the improvements made by ANSI is to enable the programmer to develop real-world application programs in a simple and easy manner. The new characteristics added enabled a programmer improved achievements in complicated conditions. It is also possible to build programs without using the new characteristics. However, with additional features, the source code will be reduced for complex problems, and programs can be made shorter and more efficient.

21.2 INNOVATIVE DATA TYPES

The ANSI/ISO committee introduced new data types to enable a better scope of standard data types in C++. These new data types are `bool` and `wchart_t`.

The `bool` data type

The `bool` is a keyword. The `bool` data type is used to hold Boolean values, that is, true and false. The `true` and `false` are also keywords. The C++ treats true and false as values. The default numeric value of false is zero, and that of true is one. All conditional expressions return values of this type. For example, the expression `5>4` is true, and it returns one. The variables of these data types are declared in the following manner:

Declaration of bool variables	
<code>bool t1;</code>	<code>// t1 is a variable of bool data type</code>
<code>t1 = true</code>	<code>// t1 contains true (1) as value</code>
<code>bool f1 = false</code>	<code>// declaration and initialization are</code>
	<code>//done with same type</code>

These `bool` data-type variables can be used in mathematical expressions as per the following statement:

```
int m = true+2*5+t1;
```

When `bool` data types are used in expressions such as the one above, they are automatically evaluated to integers. It is also possible to change by nature the data-type pointers, floating and integer values to the `bool` data type.

Initialization of bool variables	
<code>bool a= 1;</code>	<code>// Variable a contains true value</code>
<code>bool b=0</code>	<code>// Variable b contains false value</code>
<code>bool c=1.2</code>	<code>// Variable c contains true value</code>

21.1 Write a program to declare `bool` variables and display the values.

```
#include<iostream>
using namespace std;

int main()
{
    bool t=true;
    bool f=false;
    cout<<"\n t="<<t;
    cout<<"\n f="<<f;
    bool g=1;
```

```
    if (g)
        cout<<"\n true";
    else
        cout<<"\n false";
    return 0;
}
```

OUTPUT

```
t = 1
f = 0
true
```

Explanation: In the above program, `t` and `f` are variables of the `bool` data type. The variable `t` contains `true`, and `f` contains `false`. Values of these variables are displayed using the `cout` statement. The variable `g` is another variable of the `bool` data type and contains value 1, that is, `true`. The `if` statement checks the value of the variable `g` and displays the appropriate message.

21.2 Write a program to declare variables of `bool` data type and use in mathematical expressions.

```
#include<iostream>
using namespace std;

int main()
{
    bool t=true;
    bool h,f=false;

    cout<<"\n t="<<t;
    cout<<"\n f="<<f;
    int x= 4*t+5*f;
    h=x;

    cout<<"\n x="<<x;
    cout<<"\n h="<<h;
    return 0;
}
```

OUTPUT

```
t = 1
f = 0
x= 4
h= 1
```

Explanation: In the above program, `t` and `h` are variables of the `bool` data type and contain the values `true` and `false`. The variable `x` is an integer variable. The variables `t` and `f` are used in mathematical expressions, and the result obtained is assigned to the variable `x`. The value of the integer variable `x` is assigned to the Boolean variable `h`. The output of the program is as given above.

In case the variable of the `bool` type is assigned a value more than 1, for example `bool x = 2;` here, the value of `x` will be considered `true`, that is, 1. The variable of the `bool` type also supports the increment `++` (postfix or pre-fix) operation. However, it cannot support the decrement `--` operation. When a `bool`-type variable with the value 0 (`false`) is incremented, its value becomes one. However, when a variable contains Boolean value one and if any attempt is made to increase it, its value remains the same.

21.3 Write a program to perform increment operation with variable of `bool` type.

```
#include<iostream>
using namespace std;

int main()
{
    bool x=0;
    ++x;
    cout<<x;
    return 0;
}
```

OUTPUT

1

Explanation: In the above program, `x` is a variable of the type `bool`. It is initialized with the value 0 (`false`). When the variable `x` is incremented, it contains one (`true`). The increment operator can be applied either prefix or postfix.

The `wchar_t` data type

The `wchar_t` is a character data type that holds 16-bit-long/-wide characters. The 16-bit characters are manipulated to symbolize the character sets of languages that contain more than 255 characters. ANSI C++ also defined a character literal called as `wide_character` literal. It occupies two bytes in memory. The `wide_character` literal starts with the letter `L`, as follows:

`L 'ab'`

It is known as `wide_character` literal.

The declaration of `wchar_t` in the header file `stdlib.h` is as follows:

```
typedef char wchar_t;
```

The following program demonstrates the use of the `wchar_t` data type.

21.4 Write a program to use wchar_t data type.

```
#include<iostream.h>
#include<stdlib.h>
#include<constream.h>

void main()
{
    clrscr();
    wchar_t c='A';
    cout<<c;
}
```

OUTPUT

A

Explanation: In the above program, variable `c` is of type `wchar_t` and it is initialized with “A.” The `cout` statement displays the same on the screen.

21.3 NEW TYPE-CASTING OPERATORS

We have studied that a type-casting operation is used to change a value from one type into another. This can be applied at a place where the automatic conversion of data types is not done.

Type casting
<code>double b = (double)j // c type-casting style</code>
<code>double a = double (d) // c++ type-casting style</code>

The above statements are correct, and nothing wrong happens when they are applied in practical applications. New type-casting operators introduced by the ANSI/ISO committee are as follows:

- (1) `static_cast`
- (2) `dynamic_cast`
- (3) `reinterpret_cast`
- (4) `constant_cast`

(1) The static_cast operator

The `static_cast` operator is used for the conversion of standard data types. Using this operator, base class pointers can be converted into derived class pointers. Their syntax is as follows:

```
static_cast <data-type> object
```

The `data_type` indicates the target data type of the cast. The specified object is converted into a new data type. The new syntax of type casting is easy. Hence, the above keyword is frequently used by the programmer instead of the old format.

21.5 Write a program perform type casting using static_cast operator.

```
#include<iostream.h>

int main()
{
    int k=65;
    cout<<"\n size of k="<<sizeof(k);
    cout<<"\n value of k="<<k;
    double d=static_cast<double>(k);
    cout<<"\n size of d="<<sizeof(d);
    cout<<"\n value of d="<<d;
    char c=static_cast<char>(k);
    cout<<"\n c="<<c;
    return 0;
}
```

OUTPUT

```
size of k=4
value of k=65
size of d=8
value of d=65
c = A
```

Explanation: In the above program, the integer k is initialized with 65. The variable d is of a double data type. The value of k is assigned to d using the `static_cast` operator. The value of k is assigned to char data-type variable c. The output displays the contents of the variable with the numbers of bytes occupied by them.

(2) The const_cast operator

The `const_cast` operator explicitly modifies the const or volatile of a variable. It is used in the following format:

```
const_cast <data-type> (object)
```

In this type of casting, our goal is to change the const or volatile nature of the variable. Hence, target and source data types are identical. This type of casting is frequently applied for removing the const attribute of a variable.

21.6 Write a program to convert constant to non-constant.

```
#include<iostream.h>

void main()
{
    const int x=0;
```

```

    int *p=(int *)&x;
    p=const_cast<int*>(&x);
}
class data
{private:
    int d;
public:
    void joy() const
    {
        (const_cast<data*>(this))->d=100;
    }
};

```

Explanation: In the above program, the address of the constant variable is assigned to the pointer of the non-constant variable, and this is done using the operator `const_cast`.

(3) The `reinterpret_cast` operator

The `reinterpret_cast` operator is useful when the programmer wants to transform one type into a dissimilar type. In practical applications, it is used to modify a pointer kind of an object to an integer kind or vice versa. It is used in the following format:

```
reinterpret_cast <data-type> (object )
```

21.7 Write a program to convert pointers to integers using `reinterpret_cast` operator.

```

#include<iostream.h>

void main()
{
    int b=487;
    int *rp=reinterpret_cast<int*>(b);
    cout<<endl<<"rp="<<rp;
    rp++;
    cout<<endl<<"rp="<<rp;
    b=reinterpret_cast<int>(rp);
    cout<<endl<<"b="<<b;
    b++;
    cout<<endl<<"b="<<b;
}

```

OUTPUT

```

rp=0x000001E7
rp=0x000001EB
b=491
b=492

```

Explanation: In the above program, `b` is an integer type of variable and initialized with 487. The variable `rp` is a pointer and initialized with the address of the variable. The `rp` is increased and the value is increased by four. The output shows the values of the variable.

21.8 Write a program to convert void pointer to char pointer using `reinterpret_cast` operator.

```
#include<iostream.h>
#include<string.h>

void *getadd()
{
    static char text[50];
    return text;
}

void main()
{
    char *p=reinterpret_cast<char*>(getadd());
    //char *p=getadd();
    strcpy (p," Well Come");
    cout<<p;
}
```

OUTPUT

Well Come

Explanation: In the above program, the function `getadd()` returns the base address of the array `text[50]` as a void address. The return type is void *. In the function `main()`, the obtained address is converted into character type using the operator `reinterpret_cast` and assigned to the character pointer `p`. The `strcpy()` function copies a string to the pointer `p`. Finally, the `cout` statement displays the text stored in the pointer `p`.

(4) The `dynamic_cast` operator

The `dynamic_cast` operator is used to change the type of an object during program execution. It is frequently used to type cast on polymorphic objects, that is, when the base class has a virtual function. This operator casts the base class pointer onto the derived class pointer. The operation with `dynamic_cast` is also known as *type-secure downcast*. It is used in the following format:

```
dynamic_cast <data-type> (object)
```

The object should be a base class object. Its type is tested and altered. This always does valid conversion. It verifies that the casting is allowable at execution time. It returns null, in case type casting is defective.

(5) RTTI using `typeid` operator

The runtime type information (RTTI) is a new improvement made by the ANSI/ISO committee. The `typeid()` operator is used to obtain the exact type of object or

variable during program execution. This operator returns a reference to an object maintained by the system. This object identifies the type of argument. Its syntax is as follows:

```
char *object class = typeid (object). name();
```

21.9 Write a program using typeid() to identify the type of object.

```
#include<iostream.h>
#include<typeinfo.h>

class one
{
    public:
    virtual void say()
    {
    }
};

class two: public one
{
};

class three: public one
{
};

void main()
{
    one *o;
    cout<<endl<<typeid(o).name();
    two *t;
    cout<<endl<<typeid(t).name();
}
```

OUTPUT

```
class one *
class two *
```

Explanation: In the above program, class one has one virtual function, say(). Class two is derived from class one. In the function main(), *o and *t are pointer objects of class one and two, respectively. The typeid() operator identifies the type of objects and displays the class name from which they are created.

21.10 Write a program to identify the type of variable using typeid().

```
#include<iostream.h>
#include<typeinfo.h>

void main()
{
    int i;
```

```

    cout<<endl<<typeid(i).name();
    float f;
    cout<<endl<<typeid(f).name();
}

```

OUTPUT

```

int
float

```

Explanation: In the above program, two variables of `int` (`i`) and `float` (`f`) are declared. Using the `typeid()` function, their types are displayed. The output is `int`, `float`, that is, the type of variables (`i`) and (`f`).

21.4 THE KEYWORD `explicit`

The keyword `explicit` is advantageous in order to declare the constructors of a class as being `explicit`. When a constructor is invoked with a single argument, implicit conversion is carried out. In this operation, automatic conversion is performed. The type of argument taken is modified to an object type. To avoid such automatic-type conversion, the keyword `explicit` is used followed by the constructor name. The above program explains the use of the keyword `explicit`.

21.11 Write a program to declare `explicit` constructor and avoid automatic conversion.

```

#include<iostream.h>

class XYZ
{
    int k;
public:
    explicit XYZ (int j)
    {k=j;}
    void show()
    {
        cout<<"\n k="<<k;
    }
};

int main()
{
    XYZ XY(545);
    // XYZ XY=450; invalid assignment
    XY.show();
    return 0;
}

```

OUTPUT**k=545**

Explanation: In the above program, the `class XYZ` is declared with a one-integer argument. The class also has a one-argument constructor followed by the keyword `explicit` and one member function `show()`. In the function `main()`, the statement `XYZ XY (545)` creates an object `XY` and initializes the member variable `k` with the number 545. The statement `XYZ XY = 450` is invalid, because the constructor is `explicit` and does not permit automatic conversion. This statement is valid only when the class constructor is not declared as being `explicit`.

21.5 THE KEYWORD `mutable`

We learned that a member function or object can be declared as constant using the keyword **`const`**. The constant data elements are not modified, and the constant member function can alter the value of any variable. The `mutable` is used when we need to declare a constant object but partly, that is, a specific data variable can be modified. Thus, the data variable that is to be modified is declared as `mutable`. The declaration is as follows:

```
mutable int k;
```

Here, the variable `k` can be modified though its class or object is declared as constant.

21.12 Write a program to use `mutable` keyword and modify the value of variable.

```
#include<iostream>
using namespace std;

class XYZ
{
    private:
        mutable int k;
    public:

        explicit XYZ (int d) {k=d;}
        void value() const {k+=10;}
        void show() const {cout<<"\n k="<<k;}
};

int main()
{
    const XYZ x(75);
    x.show();
    x.value();
    x.show();
    return 0;
}
```

OUTPUT**k= 75****k= 85**

Explanation: In the above program, the `class XYZ` is a one-integer mutable member. The class also has `value()` and `show()` constant member functions. In the function `main()`, the `x` is a constant object of `class XYZ`. When `x` is declared, the mutable member `k` is initialized with 75. When the `value()` function is invoked, it is incremented with 10. The `show()` function displays the values of the variables.

21.6 NAMESPACE SCOPE

C++ allows defining variables with a different scope such as local, global, blocks, classes, functions, etc. This can be done using the keyword `namespace` introduced by ANSI C++. The C++ standard library is the better example of `namespace`. All classes, templates, and functions are defined inside the `namespace std`. In the previous programs, we used the statement `using namespace std`. This tells the compiler that the members of this namespace are to be used in the current program.

Declaring a Namespace

The namespace can be defined in the programs. The declaration of a namespace is similar to a class declaration except that the namespace is not terminated by a semi-colon. It is declared in the following format:

Declaration of Namespace
<pre> namespace namespace_identifier { // Definitions of variables function and classes, etc. } Example namespace num { int n; void show (int k) {cout<<k;} } </pre>

In the above example, the variable `n` and the function `show()` are within the namespace scope `num`. We cannot access the variable `n` directly. To access the variable and initialize it with a value, the statement would be as follows:

```
num :: n=50;
```

Here, `n` is initialized to 50. The scope access operator is used to access the variable. This method of accessing elements becomes embarrassing. We can also access elements directly using the following declarations:

Accessing namespace
<pre>using namespace namespace_identifier // directive statement using namespace_identifier :: member // declaration method</pre>

The first statement allows the access of elements without using a scope access operator. The second statement allows access to only given elements.

Accessing namespace
<pre>using name space num n=10; // valid statement show(15); // valid statement using name space :: n; m=20; // valid statement show(14) // invalid statement</pre>

21.7 NESTED NAMESPACES

It is also possible to declare a nested namespace. When one namespace is nested in another namespace, it is known as a nested namespace.

Nested namespace declaration
<pre>namespace NUM { statement1; namespace NUM1 { int k = 10; } statement2; }</pre>

The variable `k` can be accessed using the following statements:

Variable accessing statements
<pre>cout<<NUM::NUM1::K OR using namespace NUM; cout<<NUM1::k;</pre>

21.8 ANONYMOUS NAMESPACES

A namespace without a name is known as an anonymous namespace. The members of anonymous namespaces can be accessed globally in all scopes. The frequent use of such a namespace is to protect global members from same name classes among files. Each file possesses a separate anonymous namespace. The following programs explain the use of namespaces:

21.13 Write a program to create namespace, declare, and access elements.

```
#include<iostream>
using namespace std;
namespace num
{
    int n;
    void show() {cout<<"\n n="<<n;}
}
int main()
{
    num::n=100;
    num::show();
    return 0;
}
```

OUTPUT

n=100

Explanation: In the above program, the namespace num contains one integer member n and one member function show(). In the function main(), the variable n is initialized to 100. The members are accessed using a scope access operator and the namespace name. The function show() displays the value of the variable.

21.9 THE KEYWORD using

Using directive

The keyword using can be used for a declaration as well as a directive. The using directive provides access to all variables declared within the namespace. When using the directive method, we can directly access the variable without specifying the namespace name. The following example illustrates this:

21.14 Write a program to create namespace, declare, and access elements. Use using directive method.

```
#include<iostream>
using namespace std;
namespace num{
    int n;
```

```
    void show() {cout<<"\n n="<<n;}
}
int main()
{
    using namespace num;
    n=100;
    show();
    return 0;
}
```

OUTPUT

n=100

Explanation: In the above program, the namespace name is created with a one-integer variable `n` function `show()`. The statement `using namespace num` allows direct access to members of the namespace. Hence, in the function `main()`, `n = 100` initializes `n` with 100 and `show()` displays the value of `n` on the screen.

Using a declaration:

In this method, the keyword `using` is optional. It is also possible to access a few members of the namespace directly outside the namespace. The following program explains the above points:

21.15 Write a program to use declaration method of namespace and access variables.

```
#include<iostream>
using namespace std;

namespace num
{
    int n;
    void show()
    {cout<<"\n n="<<n;}
}

int main()
{
    //using namespace num;
    num::n=100;

    num::show();
    return 0;
}
```

OUTPUT

n=100;

Explanation: The above program is similar to the previous one. The only difference is that here the namespace `num` is not included. In order to access the elements of namespace `num`, we need to precede the variable name with a scope access operator and a namespace name. The statement `num::n = 100;` accesses the variable `n` and initializes it with 100. In the statement `num::show();` the function `show()` is invoked using the same syntax.

21.16 Write a program to use declaration method of namespace and access variables and functions.

```
#include<iostream>
using namespace std;

namespace num
{
    int n;
    void show()
    {cout<<"\n n="<<n;}
}

int main()
{
    //using namespace num;
    using num::n;
    n=100;
    num::show();
    return 0;
}
```

Explanation: In the above program, the statement `using num::n;` brings the variable `n` in the current scope; that is, after this statement, it can be accessed directly by its name, for example, `n = 100`. Here, 100 is initialized to `n`. However, other variables (other than `n`) can be accessed by preceding a namespace name before their name such as `num::show()`.

```
using namespace num;
```

The above statement allows to access all members of the namespace `num` directly, and this style of accessing the elements of `num` namespace is called a `using directive`.

```
using num::n;
```

The above statement allows to access only member variable `n` of the namespace. The other member cannot be accessed directly. The member other than `n` can be accessed by specifying the namespace name before the member name as discussed earlier. This style of accessing elements is known as a `using declaration`.

21.17 Write a program to declare nested namespace and anonymous namespace.

```
#include<iostream>
using namespace std;
```

```
namespace num
{
    int j=200;
    namespace num1 {int k=400;}
}
namespace {int j=500;}
void main()
{   cout<<"j="<<num::j <<"\n";
    cout<<"k="<<num::num1::k <<"\n";
    cout<<"j="<<j <<"\n";
}
```

OUTPUT

```
j = 200
k = 400
j = 500
```

Explanation: In the above program, num and num1 are two namespaces. The num1 is declared inside the namespace num. The last namespace defined is the unnamed namespace. In function main(), the members of the namespaces are accessed using the scope access operator. The variable j is used in two different scopes.

21.18 Write a program to declare functions in namespace. Access the function in main().

```
#include<iostream>
using namespace std;

namespace fun
{
    int add (int a, int b) {return (a+b);}
    int mul (int a, int b); // prototype declaration
}

int fun :: mul (int a, int b) {return (a*b);}
int main()
{
    using namespace fun;
    cout<<"\n Addition:" <<add(20,5);
    cout<<"\n Multiplication:" <<mul(20,5);
    return 0;
}
```

OUTPUT

```
Addition : 25
Multiplication : 100
```

Explanation: In the above program, two functions `add()` and `mul()` are declared in namespace `fun`. In function `main()`, the statement `using namespace fun` allows us to access the elements of `fun` namespace directly. Two integer values are passed to the functions `add()` and `mul()`. They return the addition and multiplication of numbers, respectively.

21.19 Write a program to declare class in the namespace. Access the member functions.

```
#include<iostream>
using namespace std;
namespace clas_s
{
    class num
    {
    private:
        int t;
    public:
        num (int m)
        {t=m;}
        void show()
        {cout<<"\n t="<<t; }
    };
}
void main()
{
    // indirect access using scope access operator
    clas_s ::num n1(500);
    n1.show();
    // direct access using directive
    using namespace clas_s;
    num n2(800);
    n2.show();
}
```

OUTPUT

t = 500

t = 800

Explanation: In the above program, class `num` is defined in the namespace `clas_s`. The class has one integer variable `t` and the member function `show()`. The member function `show()` displays the contents of the variable. In the function `main()`, `n1` and `n2` are objects of class `num`. The data members are initialized using a constructor. The members of class `num` are accessed with and without the scope access operator. Both the methods of accessing elements are explained in previous programs.

21.10 NAMESPACE ALIAS

Alias means another name. A namespace alias is designed to specify another name to an existing namespace. It is useful if the previous name is long. We can specify a short name as alias and call the namespace members. The following program explains how an alias is specified:

21.20 Write a program to specify alias to existing namespace.

```
#include<iostream>
using namespace std;

namespace number
{
    int n;
    void show()
    {cout<<"\n n="<<n;}
}

int main()
{
    namespace num=number;
    num::n=200;
    number::show();
    return 0;
}
```

OUTPUT

n=200

Explanation: In the above program, the namespace `number` is defined. In the function `main()`, its alias name is created as follows:

```
Namespace num=number;
```

The `num` is another name given to the namespace `number`. The member of the namespace `number` can be accessed using both the names, that is, `num` and `number`. The variable `n` is accessed using the name `num`, and the function `show()` is accessed using the name `number`.

21.11 THE STANDARD NAMESPACE `std`

We frequently use the statement `using namespace std`. This statement inserts a complete library in the current program. This is similar to including a header file, and the contents of the header file are available in the current program. We can access all classes, functions, and templates declared inside this namespace.

```
using namespace std;
```

As studied earlier, the above method of specifying namespace is called a `using directive`. The above declaration makes it possible to access all members of the namespace directly.

All header files also use the namespace feature. If we include header files and namespace together in the same program, the variable declared in the header file will be global. Instead of the above declaration, you should observe the following declaration in order to prevent serious bugs. Consider the following program:

21.21 Write a program to access members of namespace std applying using declaration method.

```
#include<iostream>

int main()
{
    int age;
    std::cout<<"\nEnter your age:";
    std::cin>>age;
    std::cout<<"\n Your age is:"<<age;
}
```

OUTPUT

Enter your age : 24

Your age is : 24

Explanation: In the above program, the `l` member of namespace `std` cannot be accessed directly. Consider the following statement:

```
std::cout<<"\nEnter your age:";
```

Here, the namespace name `std` is preceded by the object `cout`. In the same way, the `cin` object is accessed.

Guidelines for Defining Namespace

- (1) The namespace is not terminated by semi-colon-like classes.
- (2) Declarations done outside the namespace are considered members of the global namespace.
- (3) Members of namespace are defined within the scope of namespace.
- (4) The namespace cannot be defined inside any function; it should be global.

21.12 ANSI AND TURBO-C++ KEYWORDS

ANSI C++ has introduced various new keywords according to new characteristics of the language. The following table describes the keywords of both ANSI and Turbo C++:

Table 21.1 Keywords supported by ANSI and Turbo C++

asm	else	namespace	template
auto	enum	new	this

(Table 21.1 Contd.)

(Table 21.1 Contd.)

bool	explicit	operator	throw
break	export	private	true
case	extern	protect	try
catch	false	public	typedef
char	float	register	typeid
class	for	reinterpret_cast	typename
const	friend	return	union
const_cast	goto	short	unsigned
continue	if	signed	using
default	inline	sizeof	virtual
delete	int	static	void
do	long	static_cast	volatile
double	main	struct	wchar_t
dynamic_cast	mutable	switch	while
Additional keywords for Turbo C++			
cdecl	_ss	_cs	_export
_fastcall	huge	interrupt	_loads
near	pascal	_saveregs	_seg

(1) The Keyword `pascal`:

The keyword `pascal` is used to declare a variable or function using a Pascal-style naming convention:

Syntax:

```
pascal <data definition>;
pascal <function definition>;
```

In addition, `pascal` declares Pascal-style parameter-passing conventions when applied to a function header (first parameter pushed first; the called function cleans up the stack).

Examples:

```
int pascal x;
void pascal show(int x, int y);
```

21.22 Write a program to use `pascal` keyword and change the calling convention of C++ to `pascal` style.

```
#include<iostream.h>
#include<constream.h>

void main()
{
```



```

    clrscr();
    void pascal show (int,int);
    int x=2;
    show (++x,x);
}
void pascal show (int x, int y)
{
    cout<<"\n x="<<x <<" y="<<y;
}

```

OUTPUT

x=3 y=3

Explanation: We know that in C/C++ the calling convention is from right to left; that is, the right most variable is first pushed in the stack. In order to replace this calling convention style with Pascal style, the keyword `pascal` can be used. This keyword is given in the prototype declaration and function declarator. When we pass an argument to this function, the calling convention will be from left to right. The calling conventions are described in Chapter 11.

(2) The Keyword `cdecl`

The keyword `cdecl` is used to declare a variable or a function in the C-style naming convention and pushes arguments in the stack in C-style parameter-passing conventions.

```

cdecl <data definition> ;
cdecl <function definition> ;

```

Example:

```

int cdecl l x;
void cdecl show(int x, int y);

```

21.23 Write a program to demonstrate the use of `cdecl` keyword.

```

#include<iostream.h>
#include<constream.h>

void main()
{
    clrscr();
    void cdecl show (int,int);
    int x=7;
    show (++x,x);
}

void cdecl show ( int x, int y)

```

```
{
    cout<<"\n x="<<x <<" y="<<y;
}
```

OUTPUT

```
x=8 y=7
```

Explanation: The above program is similar to the previous one. Here, the keyword `cdecl` is used to apply the calling convention of C.

21.13 ANSI AND TURBO-C++ HEADER FILES

ANSI / ISO (“Include”) Files

The ANSI/ISO committee has introduced a new style for header files: the header file now without the extension `.h`.

Example

```
#include<iostream>
#include<vector>
```

In previous examples, we frequently used the header files given above. However, the conventional method `<iostream.h>` is also valid. A few header files are renamed; for example, `<limit.h>` with `<climit>`, `math.h` with `<cmath>`, `<stdio.h>` with `<cstdio>`, and so on. Only a few names are listed in Table 21.2.

Table 21.2 ANSI/ISO header file names

C header files	C++ header files(new style)
<cstring>	<typeinfo>
<ctime>	<string>
<cstdlib>	<new>
<cassert>	<defines>
<cctype>	<sstream>
<cerrno>	<ostream>
<cfloat>	<iostream>
<climits>	<ios>
<cmath>	<iomanip>

Turbo C++ Header (“Include”) Files

Table 21.3 contains a few frequently used names of header files of Turbo C++. The list is very exhaustive. You can view the complete list of header files through **Help** menu (contents option) present in Turbo C++ editor.

■ **Table 21.3** Turbo C++ header files

iostream.h	complex.h	strstream.h
ostream.h	fstream.h	bcd.h
process.h	generic.h	string.h
stdlib.h	iomanip.h	values.h
math.h	new.h	float.h

21.14 C++ OPERATOR KEYWORDS

The C++ committee suggested keywords for operators, such as `&&`, `||`. The keywords can be used in expressions instead of operators. Table 21.4 describes the operator keywords.

■ **Table 21.4** C++ operator keywords

Operator	Keyword
<code>xor_eq</code>	<code>^=</code>
<code>xor</code>	<code>^</code>
<code>or_eq</code>	<code> =</code>
<code>or</code>	<code> </code>
<code>not_eq</code>	<code>!=</code>
<code>not</code>	<code>!</code>
<code>compl</code>	<code>~</code>
<code>bitor</code>	<code> </code>
<code>bitand</code>	<code>&</code>
<code>and_eq</code>	<code>&=</code>
<code>and</code>	<code>&&</code>

SUMMARY

- (1) The ANSI C++ added many new characteristics to the original C++. The new characteristics added enable improved achievements in complicated conditions.
- (2) The ANSI c++ has introduced new data types to enable a better scope of standard data types in C++. These new data are `bool` and `wchart_t`.
- (3) ANSI c++ defined new cast operators are `static_cast`, `dynamic_cast`, `reinterpret_cast`, and `constant_cast`.
- (4) The `static_cast` operator is used for the conversion of standard data types.
- (5) The `const_cast` operator explicitly modifies the `const` or `volatile` of the variable.
- (6) The `reinterpret_cast` operator is useful when the programmer wants to alter one type into a basically dissimilar type.
- (7) The `dynamic_cast` operator is used to change the type of an object during program execution. It is frequently used to type cast on polymorphic objects.

- (8) The typeid operator is useful to know the types of strange objects. During program execution, we can obtain their class name.
- (9) The keyword `explicit` is advantageous to declare the constructors of a class as being `explicit`.
- (10) The keyword `mutable` is used when we need to declare a constant object but partly; that is, a specific data variable can be modified.
- (11) C++ allows defining variables with a different scope such as local, global, blocks, classes, functions, etc. ANSI C++ has added a new keyword `namespace` to declare a scope that has global variables.
- (12) The namespace can be defined in the programs. The declaration of a namespace is same as the class declaration except that the namespace is not terminated by a semi-colon.
- (13) It is also possible to declare a nested namespace. When one namespace is nested in another namespace, it is known as a nested namespace.
- (14) A namespace without a name is known as an unnamed namespace. The members of unnamed namespaces can be accessed globally in all scopes.

EXERCISES

(A) Answer the following questions

- (1) Describe the new data types introduced by the ANSI C++ standard committee.
- (2) How is the `bool` data type useful?
- (3) Describe the `const_cast` operator with its use.
- (4) What do you mean by `dynamic_cast` and `static_cast`?
- (5) What is a namespace? How does it work?
- (6) What is a typeid operator?
- (7) When are the keywords `explicit` and `mutable` useful?
- (8) What are anonymous namespaces?
- (9) Explain alias of namespace.
- (10) List the additional keywords provided by Turbo C++.

(B) Answer the following by selecting the appropriate option

- (1) An unnamed namespace is one
 - (a) **that does not have a name**
 - (b) where the compiler gives the name automatically
 - (c) that is pre-defined
 - (d) that is none of the above
- (2) The keyword `explicit` is used to declare
 - (a) **constructors explicit**
 - (b) functions `explicit`
 - (c) destructors `explicit`
 - (d) none of the above
- (3) The keyword `mutable` is used to modify the
 - (a) **constant object**
 - (b) ordinary object
 - (c) volatile object
 - (d) none of the above
- (4) The typeid operator is used to know
 - (a) **type of the object and variable**
 - (b) type of the function
 - (c) return value of the function
 - (d) none of the above
- (5) An alias to an existing namespace means
 - (a) **another name to the existing namespace**
 - (b) name of the nested namespace
 - (c) a name given by the compiler to an unnamed namespace
 - (d) none of the above

- (6) One of the following keywords is not a part of ANSI C++:
 - (a) `far`
 - (b) `public`
 - (c) `static`
 - (d) **none of the above**
- (7) Consider the statement `bool x = 1;` the value of variable `x` after the increment operation will be
 - (a) **1**
 - (b) 2
 - (c) 0
 - (d) -1

(C) Attempt the following programs

- (1) Write a program to explain the use of the keywords `mutable` and `explicit`.
- (2) Write a program to declare a namespace with variables and functions. Use members of the namespace in the function.
- (3) Write a program to define a constant member function and a constant object. Modify the contents of member variables. Use the keyword `mutable`.
- (4) Write a program to demonstrate the use of the `static_cast` operator.
- (5) Write a program to declare `bool` variables and display their values.
- (6) Write a program to perform an increment operation with variables of `bool` type and display their values.
- (7) Write a program using `typeid()` to identify the type of object and data type.
- (8) Write a program to access the members of namespace `std` applying the declaration method.

C++ Graphics

22

CHAPTER

CHAPTER OUTLINE

- 22.1 Introduction
- 22.2 Computer Display Modes
- 22.3 Video Display and Display Adapters
- 22.4 Initialisation of Graphics
- 22.5 Few Additional Graphics Functions
- 22.6 Programs Using Library Functions
- 22.7 Working with Text
- 22.8 Filling Patterns with Different Colors and Styles
- 22.9 Mouse Programming
- 22.10 Drawing Noncommon Figures

22.1 INTRODUCTION

Graphics is one of the most interesting and useful topics in computers and C++ language. The goal of this chapter is to explore graphics information using C++ graphics.

While processing pictorial/graphical information on a computer screen using the C++ language, one needs to know the basics of graphics functions, different display adapters, the initialization procedure of graphics, and so on. In other words, a pictorial presentation can be done on computers with graphics by understanding the concepts described above. We can draw all figures of various shapes with computer graphics: lines, circles, ellipses, rectangles, triangles, and so on. Besides, nowadays, we are dealing with animation, multimedia, and so on, which work predominantly using graphics. The intention behind this chapter is to interact with graphics using the C++ language in an easy way. The basic concepts of graphics are explained with examples. In this chapter, standard library functions are described along with illustrations. We will briefly study about computer graphics.

22.2 COMPUTER DISPLAY MODES

Computer display is categorized in text and graphics modes.

So far, in the previous chapters after running the program in C++, we get the result on the screen in the form of characters. The mode in which the computer display represents the contents of the screen in terms of characters is called *text mode*.

In most of the computer applications, information is conveyed in the form of pictures/images. In numerous applications such as CAD/CAM, image processing, and multimedia, we would like to have pictorial or image information on the screen. In such a case, one needs to set the computer screen in graphics mode. In this mode, the screen consists of pixels. Fine/small dots on the screen are drawn in the form of pixels. Pixels are arranged in rows and columns, and they can be illuminated.

22.3 VIDEO DISPLAY AND DISPLAY ADAPTERS

A video display unit is one of the most useful output components of the computer system. This system comprises the following components:

- (1) Video screen where text and picture can be displayed
- (2) Video display adapter card, which is plugged into one of the expansion slots, provided on the motherboard.

The display adapter facilitates the programmer to show the various figures and text on to the screen with different resolutions. Resolution is decided by number of pixels (small dots) available on the screen. The display resolution is a combination of number of rows from top to bottom and number of pixels from left to right on each scan line. Higher the resolution more clear is the picture. With higher number of pixels the picture becomes clearer. Commonly available display adapters are Video display adapters, Color graphics adapters, and super video graphs adapters. All these adapters operate both in text and graphics modes. The clarity and quality of picture with VGA, SVGA etc are better than CGA and monochrome adapters. CGA adapters were used earlier and now it is absolute. Now a day, programmers prefer to use high resolution adapters such as EGA, VGA or SVGA.

The display adapter acts as an inter-mediator between processor and monitor. The display adapter has two parts a) VDU memory b) Circuitry that passes contents of VDU memory to the screen. The microprocessor writes the information into VDU memory. The display adapter transfers the information written by processor to screen. Every address of the VDU memory corresponds to the specific location of monitor screen.

22.4 INITILISATION OF GRAPHICS

The header file *graphics.h* is to be included at the beginning of the C++ program. This header file comprises the definitions and explanations of all the graphics functions. This file controls many functions and constants that are supported by the display adapters. The programmer can see the details of the graphics mode by writing `graphics_mode` in turbo c++ editor. After writing `graphics_mode`, bring the cursor at the start of the `graphics_mode` and press `ctrl+f1` and the enter key to see the details. On the screen, we get the table describing various graphic drivers, their graphic modes, resolutions, palettes, and so on.

The functions described in this chapter are defined in the *graphics.h* file, which should be included in every graphics program.

To switch from text mode to graphics mode, the function *initgraph()* should be invoked. It selects the best resolution and puts the corresponding number in *gm*.

The *initgraph()*:

This function initializes the graphics mode. Its prototype is defined in “*graphics.h*.” It is declared as per the following format:

```
void far initgraph (int far *gd, int far *gm,   char far *path-
todriver);
```

where *gd* is the graphics driver, and *gm* is the graph mode.

Before starting any drawing action, we need to initialize the graphics mode. The *initgraph()* function finds out the best graphics mode and sets the corresponding number in the variable *gm*. Using the value of *gm*, we can find out the details of the monitor, such as its type, resolution, number of video pages, and supporting colors.

The variable *gd* is used for graphics drivers. Graphics drivers are a sub-set of device drivers that are used in only the graphics mode. The graphics drivers files have an extension *.bgi*. Depending on the type of adapter, one of the supporting device driver files is selected. In the graphics mode, the cursor disappears, and the top left corner (0,0) of the console is considered the beginning.

The third parameter in *initgraph* is path driver (“c:\\tc\\bgi”). It specifies the graphics driver, that is, *initgraph()*; looks for graphics driver (*.BGI).

22.1 Write a program to switch over from text mode to graphics mode.

```
#include<stdio.h>
#include<conio.h>
#include<iostream.h>
#include<graphics.h>

int main()
{
    int gd=DETECT,gm;
    char x;
    cout<<"\n By default display is text mode:";
    cout<<"\n Press character 'g' to switch over to graphics
        mode:";
    cin>>x;
    if(x=='g')
    {
        initgraph(&gd,&gm,"c:\\tc\\bgi");
        outtextxy(40,60,"This is graphics mode");
    }
    else
```



```

    {
        cout<<"\n Sorry You are in text mode:";
    }
    getch();
    return 0;
}

```

OUTPUT

By default display is text mode:

Press character 'g' to switch over to graphics mode: g

This is graphics mode

Explanation: On compiling and running the program, the screen will be in the text mode. The screen will show the first two lines as shown in the o/p of the program. After pressing the character 'g,' the screen will move from the text to the graphics mode. The message displayed on the screen is "*This is graphics mode*" at (40,60) x,y co-ordinates, respectively. The output is in white with a black background. This effect can be observed after running the program.

22.5 FEW ADDITIONAL GRAPHICS FUNCTIONS

C++ supports enormous graphic functions. It is beyond the scope of this chapter to explain all the functions in a comprehensive manner. Nevertheless, an attempt is made to explain a few graphics functions. They are as follows:

(1) line();

This function draws a line between two specified points and is declared as below.

```
void far line (int x1, int y1, int x2, int y2);
```

This function draws a line from (x1, y1) to (x2, y2) using the current color, line style, and thickness.

(2) circle();

This function draws a circle at (x, y) of the given radius and is declared as below.

```
void far circle (int x, int y, int radius);
```

(3) arc();

This function draws an arc and is declared as below.

```
void far arc (int x, int y, int sa, int ea, int r);
```

where (x, y) is the center point. The variables *sa* and *ea* are the starting and ending angles, respectively, in degrees, and the variable *r* is the radius.

(4) ellipse();

This function draws an elliptical arc and is declared as below.

```
void far ellipse (int x, int y, int sa, int ea, int xr, int yr);
```

where (x, y) is the center point. The variables *sa* and *ea* are the starting and ending angles in degrees, respectively. The variables *xr* and *yr* are horizontal and vertical radii, respectively.

(5) **bar();**

This function draws a bar using the given co-ordinates. The *bar()* function draws a filled-in, rectangular, two-dimensional bar. This function can be declared as below.

```
void far bar (int left, int top, int right, int bottom);
```

(6) **rectangle();**

The function *rectangle()* draws a rectangle. Its syntax is as follows:

```
void far rectangle(int left, int top, int right, int bottom);
```

(7) **moveto();**

The function *moveto()* moves the current position to view the port position (x, y).

Its syntax is as follows:

```
void far moveto(int x,int y);
```

(8) **closegraph();**

This function shuts down the graphics system. The declaration is as below. The prototype is defined in “*graphics.h*.”

```
void far closegraph (void);
```

(9) **restorecrtmode();**

This function restores the screen mode to its pre-initgraph setting. Its prototype is defined in “*graphics.h*.”

```
void far restorecrtmode (void);
```

(10) **graphresult();**

This function returns an error code for the last failed graphics operation and is defined as below.

```
int far graphresult(void);
```

It returns the error code for the last graphics execution that reported an error and resets the error level to *grOK*, where *grOK* is the enumerated error code.

(11) **getmaxx(); and getmaxy();**

The *getmaxx()* returns the maximum *x* screen coordinate, and *getmaxy()* returns the maximum *y* screen coordinate. Its prototype is defined in “*graphics.h*.” These functions can be declared as below.

```
int far getmaxx (void);  
int far getmaxy (void);
```

(12) setcolor();

This function sets the current drawing color and is declared as below.

```
void far setcolor (int color);
```

(13) outtextxy();

This function displays a string at the specified location (graphics mode) and is declared below.

```
void far outtextxy (int x, int y, char far *textstring);
```

(14) settextstyle();

This function sets the current text attributes and is declared below.

```
void far settextstyle(int font, int direction, int fontsize);
```

This function has three integer arguments. The first argument indicates font; the second, direction; and the third, size of the font.

(15) settextjustify();

This function sets the text justification for the graphics mode and is declared as below.

```
void far settextjustify(int horiz, int vert);
```

It affects the text output with *outtext()* function, and so on. The text is justified horizontally and vertically.

(16) getcolor();

This function returns the current drawing color and is declared as follows:

```
void far getcolor(void);
```

22.6 PROGRAMS USING LIBRARY FUNCTIONS**(1) arc();**

This function is useful to draw a circular arc. It requires five arguments. The syntax would be

```
arc ( int x, int y, int a, int b , int c );
```

Here, the first two arguments, that is, *x* and *y*, serve as the center point. The arguments *a* and *b* are the starting and ending angles, respectively, and finally, *c* is the radius. The following programming example can be referred for drawing an arc.

22.2 Write a program to draw an arc using arc() function.

```
#include<stdio.h>
#include<conio.h>
#include<iostream.h>
#include<graphics.h>

int main()
{
    int gd =DETECT,gm;
```

```

        initgraph(&gd,&gm,"c:\\tc\\bgi");
        arc(200,175,70,180,50);
        getch();
        closegraph();
        restorecrtmode();
        return 0;
    }

```

Explanation: Consider the statement *initgraph(&gd,&gm,"c:\\tc\\bgi")*; which initializes the graphics mode. The required adapter files are present in the specified directory. The *arc()* function draws an arc by simply using parameter values. The function *closegraph()* shuts the graphics mode. The function *restorecrtmode()*; restores the screen mode to the original video mode. The DETECT is the enum data type of BGI graphics drivers.

22.3 Write a program to draw an arc using arc() function. Create a class containing private and public members.

```

#include<conio.h>
#include<graphics.h>
#include<dos.h>
#include<stdlib.h>
#include<iostream.h>

class Graphics
{
    private:
        int gd,gm;
    public:
        void getdata(int x,int y,int s,int e,int r);
        void grmode(void);
};

void Graphics::grmode()
{
    gd=DETECT;
    gm=VGA;
    initgraph(&gd,&gm,"c:\\tc\\bgi");
    setpalette(2,3);
    setcolor(5);
    getche();
}

void Graphics::getdata(int x,int y,int s,int e,int r)
{
    arc(x,y,s,e,r);
}

int main()
{
    Graphics P;

```

```
    clrscr();
    P.grmode();
    P.getdata(100,100,10,175,40);
    getch();
    return 0;
}
```

Explanation: The concept of class is applied in this program. Private and public members are declared, and the same members are accessed with an object P. The output is an arc with given coordinates. The programmer can view the output on the screen. Similarly, the programmer can run the following programs with an object-oriented approach.

2. circle():

This function draws a circle at (x, y) of the given radius. This function is declared as below.

```
void far circle (int x, int y, int radius);
```

In the above function, x and y are the co-ordinates; that is, they represent the center of the circle.

22.4 Write a program to draw a circle.

```
#include "graphics.h"
#include "conio.h"

int main()
{
    int gd=DETECT, gm, x=130, y=160, radius=100;
    initgraph(&gd, &gm, "c:/tc/bgi");
    circle(x, y, radius);
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}
```

Explanation: The function *circle()* is used to draw the circle().

3. line():

This function draws a line from (a,b) to (c,d).

22.5 Write a program to draw a line.

```
#include <stdio.h>
#include <conio.h>
#include <iostream.h>
#include <graphics.h>
```

```
int main()
{
    int gd=DETECT,gm;
    clrscr();
    initgraph(&gd,&gm,"c:/tc/bgi");
    line(20,20,20,200);
    getche();
    return 0;
}
```

Explanation: The function *line()* is used to draw a line. This function draws a vertical line as per the coordinates. The user may change the starting and ending coordinates and observe the position of the line on the screen.

4. arc(), circle() and line():

22.6 Write a program to draw circle, line, and arc using graphics function.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>

int main()
{
    int gd=DETECT,gm,x,y;
    initgraph(&gd,&gm,"c:\\tc\\bgi");
    x=getmaxx();
    y=getmaxy();
    setcolor(WHITE);
    outtextxy(1,20,"Circle");
    circle(40,100,40);
    outtextxy(200,10,"Arc");
    arc(x/3,y/8,180,70,30);
    line (210,150,110,150);
    outtextxy(150,140,»Line»);
    ellipse(215,150,0,70,150,150);
    outtextxy(290,10,»Ellipse»);
    getche();
    closegraph();
    restorecrtmode();
    return 0;
}
```

Explanation: The above program draws circles, arcs, ellipses, and lines depending on the given values. The user can change the arguments in the function and view the results.

5. line() and outtextxy():

22.7 Write a program to display rectangle by using *line()* function and display some text within it using *outtextxy()* function.

```
#include<stdio.h>
#include<conio.h>
#include<iostream.h>
#include<graphics.h>

int main()
{
    int gd=DETECT,gm;
    clrscr();
    initgraph(&gd,&gm,"c:/tc/bgi");
    line(20,20,20,200);
    outtextxy(22,35,"Rectangle made up of lines");
    line(230,20,230,200);
    line(230,20,20,20);
    line(230,200,20,200);
    getch();
    return 0;
}
```

Explanation: In the above program, we have used the *line()* function to draw a rectangle.

In order to display a rectangle using *line()*, we have to use the *line()* function four times.

6. rectangle():

22.8 Write a program to display rectangle by using *rectangle()* function .

```
#include<stdio.h>
#include<conio.h>
#include<iostream.h>
#include<graphics.h>

int main()
{
    int gd=DETECT,gm;
    clrscr();
    initgraph(&gd,&gm,"c:/tc/bgi");
    setcolor(GREEN);
    rectangle (50,30,150,200);
    getch();
    closegraph();
}
```

```

    restorecrtmode();
    return 0;
}

```

Explanation: This program draws the rectangle using the given arguments. The `setcolor()` sets the color of the rectangle.

7. Use of `setviewport()`, `outtextxy()` and `setcolor()`:

22.9 Use `setviewport()`, `outtextxy()`, `setcolor()`, `closegraph()` and display some message in 15 different colors.

```

#include<stdio.h>
#include<conio.h>
#include<iostream.h>
#include<graphics.h>

int main()
{
    int gd =DETECT,gm,j;
    initgraph(&gd,&gm,"c:\\tc\\bgi");
    char st[50];
    clrscr();
    setviewport(00,00,400,400,1);
    outtextxy(10,10,"View port");
    for(j=1;j<=15;j++)
    {
        setcolor(j);
        sprintf(st,"Existing Color number is :%d press any key",j);
        outtextxy(5,20+j*20,st);
        getch();
    }
    closegraph();
    return 0;
}

```

OUTPUT:

View port

```

Existing Color number is :1 press any key
Existing Color number is :2 press any key
Existing Color number is :3 press any key
Existing Color number is :4 press any key

```



```
Existing Color number is :5 press any key
Existing Color number is :6 press any key
Existing Color number is :7 press any key
Existing Color number is :8 press any key
Existing Color number is :9 press any key
Existing Color number is :10 press any key
Existing Color number is :11 press any key
Existing Color number is :12 press any key
Existing Color number is :13 press any key
Existing Color number is :14 press any key
Existing Color number is :15 press any key
```

Explanation: In the above program, `setviewport()`, `setcolor()`, and `outtextxy()` are the various functions included. The function `setviewport()` sets the current viewport for graphics output. The syntax of `viewport()` is as follows:

```
viewport(int left, int top , int right, int bottom, int clip)
```

The parameters at the left and top are used to specify the x and y co-ordinates of the top left corner of the screen.

The parameters at the right and bottom are used to specify the x and y co-ordinates of the bottom right corner of the screen.

The fifth parameter of the `viewport()`, that is, the `clip`, is set to 1. The `clip = 1` means that no printing takes place outside the viewport.

The function `outtextxy()` is used to print the text at given co-ordinates, and the function `setcolor()` is used to set different colors. The programmer can view the output message in different colors by pressing any key.

8. Use of `setfillstyle()`, `setcolor()` and `fillellipse()`

22.10 The following program illustrates a moving moon.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<dos.h>
int main()
{
    int gd =DETECT,gm,x;
    initgraph(&gd,&gm,"c:\\tc\\bgi");
    clrscr();
    setcolor(15);
    for (x=30;x<200;x+=10)
    {
        setfillstyle(SOLID_FILL,WHITE);
        fillellipse    (x,50,30,30);
```

```

        sleep(1);
        clrscr();
    }
    closegraph();
    restorecrtmode();
    return 0;
}

```

Explanation: In this program using the *fillellipse()* function, a moon-like shape is drawn. Using the loop, the drawn figure keeps on moving on the screen. The *sleep()* function halts the execution for a second. The *setfillstyle()* function sets the current fill pattern and fill color.

Besides the functions as cited earlier in this chapter, some additional functions as mentioned below can be used.

- (1) *getdrivename()* : This function displays the name of graphics driver name. For example EGAVGA, EGA VGA etc.
- (2) *getmaxcolor()* : returns the maximum color available with the adapter.
- (3) *getmaxmode()* : Returns number of display modes available.
- (4) *getmodename(int)* : This function returns name of the graphics mode.
- (5) *lineto()* : This function draws a line. It requires two integer arguments. It draws a line from current position to (x,y) where x and y are the coordinates to be specified by the user.
- (6) *moveto()* : This function moves the current position to (x,y) where x and y are the coordinates to be specified by the user.

Following program illustrates some of the above graphics functions..

9. Use of *moveto()*, *line()* and *lineto()*:

22.11 Write a program to shift the line coordinates to other positions with *movto()* and *lineto()* functions.

```

#include<stdio.h>
#include<conio.h>
#include<graphics.h>

int main()
{
    int gd =DETECT,gm;
    initgraph(&gd,&gm,"c:\\tc\\bgi");
    line(20,30,200,30);
    moveto(250,50);
    lineto(200,50);
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}

```

Explanation: In this program `line()` function is used for drawing a line. Also the effect of `moveto()` and `lineto()` functions can be seen by executing this program. You will observe that the line coordinates get changed with these functions.

22.7 WORKING WITH TEXTS

There are several functions for display texts in different fonts, sizes, and directions. Consider the following functions:

- (1) `outtextxy(int, int, string)`: This function is used to display the text in graphics mode.
- (2) `settextstyle()`: This function has three integer arguments. The first argument indicates font; the second, direction; and the third, size.

Use of `settextstyle()` and `outtextxy()`:

22.12 Write a program to display text in different fonts and sizes.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>

int main()
{
    int gd =DETECT,gm;
    initgraph(&gd,&gm,"c:\\tc\\bgi");
    settextstyle(3,4,30);
    outtextxy(10,110,"AMIT");
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}
```

Explanation: After execution of the program, the string “AMIT” is displayed. This is performed by giving arguments in the `settextstyle()` function.

Another new function that can be used is `settextjustify()`, which takes care of alignment of the text. This function is explained in the following program:

22.13 Write a program to display text in different fonts and sizes.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>

int main()
{
    int gd=DETECT,gm;
```

```

    initgraph(&gd,&gm, "c:\\tc\\bgi");
    settextstyle(1,0,10);
    settextjustify(0,2);
    outtextxy (10,2,"Hello");
    settextjustify(2,3);
    settextstyle(3,1,5);
    outtextxy (14,150,"Hello");
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}

```

Explanation: In the above program, the `settextjustify()` function is used. The syntax of `settextjustify()` is as follows:

```
void far settext(int horiz, int vert);
```

The function `settextjustify()` is used to set the current text justification, that is, horizontal and vertical.

The table of `settextjustify()` has two arguments. They are as follows:

Name	Value	Direction
HORIZ_DIR	0	LEFT to RIGHT
VER_DIR	1	BOTTOM to TOP

After execution of the program, the string “Hello” is displayed horizontally and vertically with a different font and font size. This is performed by giving different values in the `settextstyle()` function. Another new function used is `settextjustify()`, which takes care of the alignment of text.

Stylish Lines

22.14 Write a program to display text in different fonts and sizes.

```

#include<stdio.h>
#include<conio.h>
#include<graphics.h>

int main()
{
    int gd =DETECT,gm;
    initgraph(&gd,&gm, "c:\\tc\\bgi");
    clrscr();
    line(20,9,200,9);
    setlinestyle(1,1,1);
    line(40,9,400,9);
    setlinestyle(2,1,1);
    line(60,9,60,70);
}

```

```

    getch();
    closegraph();
    restorecrtmode();
    return 0;
}

```

Explanation: In this program using *setlinestyle()* function lines are drawn in different style. It requires three integer arguments. The first represents style of line second represents pattern and the third represents thickness.

22.8 FILLING PATTERNS WITH DIFFERENT COLORS AND STYLES

Now we will use a graphics function to fill the different patterns with different colors using different styles. In the following program a *setfillstyle()* function is used for drawing a bar. This function needs to pass two parameters the first being the pattern and second being the color.

22.15 Write a program to use the *setfillstyle()* and fill up a bar and 3d bar with different colors.

```

#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>

int main()
{
    int gd =DETECT,gm;
    initgraph(&gd,&gm, "c:\\tc\\bgi");
    setcolor(GREEN);
    setfillstyle(SOLID_FILL,RED);
    bar(20,30,200,40);
    moveto(250,50);
    bar(200,50,250,60);
    setfillstyle(CLOSE_DOT_FILL,GREEN);
    bar3d(150,100,200,200,20,1);
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}

```

Explanation: This program uses the *setfillstyle()* function for filling various colors with different styles. A bar and a 3-d bar are drawn with two different colors. The programmer can run the program and see its effects.

22.16 Write a program to draw the triband flag and a circle on the middle band. Use `setfillstyle()`, `setbkcolor()`, `setcolor()` functions. Show the flag pattern with colors on the screen.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>

int main()
{
    int gd =DETECT,gm;
    initgraph(&gd,&gm, "c:\\tc\\bgi");
    setfillstyle(1,6);
    bar(20,30,100,40);
    rectangle( 20,30,100,40);
    setfillstyle(1,15);
    bar(20,40,100,50);
    rectangle( 20,40,100,50);
    setfillstyle(1,2);
    bar(20,50,100,60);
    rectangle( 20,50,100,60);
    line(20,30,20,150);
    setbkcolor(1);
    setcolor(2);
    setfillstyle(1,3);
    circle(60,45,5);
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}
```

Explanation: This program uses the `setfillstyle()` function for filling various bars with different colors. The background color used is blue with a function `setbkcolor()`. Bars are drawn with different colors. A circle is drawn on the middle bar. The programmer can run the program and see the output.

22.17 Write a program to draw the rectangles duly filled with different patterns using `setfillstyle()` and show its effect on the VDU.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>
```



```

    initgraph(&gd,&gm, "c:\\tc\\bgi"); setcolor(RED);
    settextstyle(0,0,2);
    outtextxy(100,1,"User-defined Styles");
    settextstyle(0,0,0);
    setcolor(WHITE);
    for (ptr=0;ptr<12;ptr++)
    {
        setfillstyle(ptr,ptr);
        bar(a,b,a+70,b+70);
        rectangle(a,b,a+70,b+70);
        k=1+ptr;
        itoa(k,txt,10);
        outtextxy(a,b+100,txt);
        setcolor(ptr+1);
        outtextxy(a,b+110,design[ptr]);
        a=a+150;
        if (a>490)
        {
            b=b+150;
            a=40;
        }
    }
    getch();
    closegraph();
    restorecrtmode();
}

```

Explanation: In the above program, the *settextstyle()* function sets text attributes such as font size, direction of horizontal or vertical text, and size of the character. The message “user-defined styles” is displayed in a larger size with red color. Again, the text attributes are set to 0 so that they will not further affect the outputting text. The character pointer is initialized with pre-defined patterns (designs). The *setfillstyle()* function has two arguments: patterns and color to be filled. The *bar()* function draws the bar using an attribute of *setfillstyle()*. The *rectangle()* function draws a border for the various bars created by the *bar()* function. The *itoa()* function converts an integer into a string. The numbers 1 to 12 displayed below each bar are due to this function.

22.19 Write a program to draw the rectangle floodfill() function and show its effect on the VDU.

```

#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>

```



```

int main()
{
    int gd =DETECT,gm,x,y;
    initgraph(&gd,&gm, "c:\\tc\\bgi"); setcolor(WHITE);
    outtextxy(60,20,"RECTANGLE");
    setcolor(GREEN);
    rectangle(20,40,200,60);
    setfillstyle(1,3);
    floodfill(21,41,GREEN);
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}

```

Explanation: This program uses setfillstyle() and floodfill() functions for filling the rectangles with any pattern. It is not necessary to use bar() at first and then enclose it by rectangle as used in the previous program. Straightway, one can use floodfill() function for filling a pattern with different colors. The programmer can run the program and see output.

22.9 MOUSE PROGRAMMING

A mouse is an input device for the computer. It is used to move the cursor over the screen of a computer, and it points to the icon or with clicking the right button, some action can be done. When the mouse is moved, the cursor also moves over the screen. With arrow keys, the cursor also can be moved over the screen. However, moving the cursor with a mouse is much faster than using arrow keys.

A computer senses the connectivity of a mouse by loading a mouse driver program. Once the mouse driver program is loaded, various mouse functions can be accessed /provided. The AX register is to be loaded with different values as service numbers and issuing software interrupts such as 51h or 33h. In the following programs, the `int86()` C++ function calls the interrupts. Its prototype is in `dos.h`. Its arguments are interrupt number, `&in`, and `&out`, which are provided within parentheses. The `in` and `out` registers should be a type of `REGS`. `REGS` is a built-in `UNION` declaration in C++.

22.20 Write a program to display a rectangle which is a limited area of a mouse.

```

#include<conio.h>
#include<dos.h>
#include<graphics.h>
union REGS in,out;
int main()
{
    int x,y,cl,a,b;

```

```

    int gd=DETECT,gm;
    int mouse(void);
    void limitmsptr(int,int,int,int);
    clrscr();
    initgraph(&gd,&gm, "c:\\tc\\bgi");
    rectangle(100,100,550,400);
    mouse();
    limitmsptr(100,100,550,400);
    getch();
    return 0;
}

int mouse()
{
    in.x.ax=1;
    int86(51,&in,&out);
    return 1;
}

void limitmsptr(int a1,int b1,int a2,int b2)
{
    in.x.ax=5;
    in.x.cx=a1;
    in.x.dx=a2;
    int86(51,&in,&out);
    in.x.ax=7;
    in.x.cx=b1;
    in.x.dx=b2;
    int86(51,&in,&out);
}

```

Explanation: In the above program, the mouse is used in the graphics mode. We have to switch over the functioning of the mouse from the text to the graphics mode. The `initgraph()` function is responsible for this job.

The function `restrictmouseptr()`; defines the area of the mouse, that is, the mouse pointer area. In this function, `int86()` is invoked by initializing CPU registers with appropriate values.

The `mouse()` function checks whether the mouse driver is loaded or not. The `rectangle()` function is invoked for drawing the rectangle with the required coordinates. After running the program, a rectangle appears on the screen, which is the restricted area of the mouse. The mouse is activated in the rectangle area only.

In the `limitmsptr()` function, the `ax` register is initialized to 5, which limits the mouse pointer to the horizontal obstacle. The `cx` and `dx` are initialized with minimum and maximum x horizontal coordinates.

Similarly, `ax` is initialized to 7 for limiting the vertical obstacle. The `cx` and `dx` are further initialized to 100 and 400, respectively, for vertical y coordinates.

22.21 Write a program to display status of mouse button as to whether it is pressed or not and restrict the mouse between given co-ordinates on the screen. Use `int86()` function to call different ROM-BIOS services.

```
#include<conio.h>
#include<process.h>
#include<dos.h>
#include<graphics.h>
#include<iostream.h>

union REGS i,o;

void main()
{
    int stmouse(void);
    void sharow(void);
    void msarea(int, int, int,int);
    void pntat( int *, int *, int *);
    int gd=DETECT, gm,button,x,y;
    initgraph(&gd,&gm, "c:\\\\turboc3\\\\tc\\\\bgi");
    rectangle(0,56,100,400);
    setviewport (1,57,99,399,1);
    gotoxy(28,1);
    cout<<"\\n Mouse Button pressed";
    if (stmouse()==0)
    {
        closegraph();
        restorecrtmode();
        cout<<"\\n Mouse driver not found";
        exit(1);
    }
    msarea(0,100,50,190);
    sharow();
    gotoxy(52,3);
    cout<<"\\n Press any key to exit";
    while (!kbhit())
    {
        pntat(&button,&x,&y);
        gotoxy(5,3);
        (button & 1)==1 ? cout<<"LEFT":cout<<"*****";
        gotoxy(20,3);
        (button & 2)==2 ? cout<<"RIGHT":cout<<"*****";
        gotoxy(65,2);
        cout<<"Cursor Position";
        gotoxy(65,4);
    }
}
```

```

    cout<<"X="<< x<<"\t"<<"Y="<< y;
}
}
stmouse()
{
    i.x.ax=0;
    int86(0x33,&i,&o);
    return(o.x.ax);
}
void sharow()
{
    i.x.ax=1;
    int86(0x33,&i,&o);
}
void msarea(int x1,int y1,int x2,int y2)
{
    i.x.ax=7;
    i.x.cx=x1;
    i.x.dx=x2;
    int86(0x33,&i,&o);
    i.x.ax=8;
    i.x.cx=y1;
    i.x.dx=y2;
    int86(0x33,&i,&o);
}
void pntat( int *button, int *x,int *y)
{
    i.x.ax=3;
    int86(0x33,&i,&o);
    *button=o.x.bx;
    *x=o.x.cx;
    *y=o.x.dx;
}

```

Explanation: In the above program, the function *initgraph()* switches the mode from text to graphics. The DETECT is a macro that requests the *initgraph()* to set the graphics driver. The function *setviewport()* defines the viewport area that bounds the drawing operation inside the area.

The *stmouse()* function checks whether the mouse driver is loaded or not. If the mouse is not initialized, then the *closegraph()* function off-loads the graphics driver. The *restoremode()* function restores the mode.

The function *msarea()* defines the area in which the mouse pointer can be a pointer. In this function, *int86()* is invoked by initializing the CPU register with appropriate values.

The functions *msarea()* and *sharow()* call ROM-BIOS services under *33h* with appropriate values. The *while()* loop checks the button pressed and displays messages accordingly. The function *pnat()* displays the current cursor position. When the user presses a key, the program is terminated.

22.10 DRAWING NONCOMMON FIGURES

The following program illustrates the use of *drawpoly()* function. With this patterns of non-common shapes can be drawn.

22.22 Draw non-common shape patterns and fill it with some color.

```
#include<stdio.h>
#include<conio.h>
#include<graphics.h>
#include<stdlib.h>

int main()
{
    int arr[8];
    int gd =DETECT,gm,x,y;
    initgraph(&gd,&gm,"c:\\turboc3\\tc\\bgi");
    setcolor(GREEN);
    outtextxy(10,30,"NONCOMMON SHAPE");
    setcolor(GREEN);
    arr[0]=20;
    arr[1]=50;
    arr[2]=60;
    arr[3]=80;
    arr[4]=100;
    arr[5]=200;
    arr[6]=arr[0];
    arr[7]=arr[1];
    fillpoly(3,arr);
    drawpoly(3,arr);
    getch();
    closegraph();
    restorecrtmode();
    return 0;
}
```

Explanation: This program uses *drawpoly()* and *fillpoly()* functions. The programmer can run the program and see output. You will get non-regular shape figure with filled color on the screen.

SUMMARY

- (1) In this chapter, the reader is exposed to the basics of graphics programming in C++.
- (2) Initialization of graphics along with library graphics functions concepts are explained.
- (3) A number of programming examples are provided.
- (4) The reader is advised to go through other functions provided in the HELP of C++ graphics.
- (5) The last few programs help in learning mouse programming.

EXERCISES

(A) Answer the following by selecting the appropriate option

- (1) For drawing a bar the number of arguments to be passed are
 - (a) 3
 - (b) 4**
 - (c) 5
 - (d) 6
- (2) The initialization of graphics under C++ is done with the function
 - (a) loadgraph()
 - (b) initgraph()**
 - (c) modegraph()
- (3) Name the function to close the graphics mode in C++
 - (a) closeall()
 - (b) closegraph()**
 - (c) cls()
- (4) A video display unit is ——— device of the computer system.
 - (a) input
 - (b) output**
 - (c) input/output
- (5) Name the function that returns the maximum x screen coordinate
 - (a) getmaxx()**
 - (b) getmayy()
 - (c) getmaxy()
- (6) To switch over to the graphics mode a function ——— should be invoked.
 - (a) initgraph()**
 - (b) restorecrtmode()
 - (c) graphresult()
- (7) The prototype for closegraph() is defined in
 - (a) graphics.h**
 - (b) stdio.h
 - (c) conio.h
- (8) The function returns the maximum x screen coordinate
 - (a) getmaxx()**
 - (b) getmayy()
 - (c) getmaxy()
- (9) The function returns maximum y screen coordinate
 - (a) getmaxy()**
 - (b) getmayy()
 - (c) getmayx()

(B) State whether the following statements are true or false

- (1) The function getmaxx() returns the maximum x screen coordinate.
- (2) One should specify the value of the diameter for drawing a circle using the function circle().
- (3) A video screen is used only for displaying text.
- (4) The function setcolor() sets the current drawing color.
- (5) The outtextxy() function displays a string at the specified location.
- (6) The function closegraph() shuts the graphics mode.

- (7) The function `settextjustify()` sets the text justification for graphics mode.
- (8) The header file *graphics.h* comprises the definitions and explanations of all the graphics functions.
- (9) The `initgraph()` function finds out the best graphics mode.
- (10) The function `circle()` is a library function.

Answers

(1) T (2) F (3) F (4) T (5) T (6) T (7) T (8) T (9) T (10) F

(C) Attempt the following programs

- (1) Write a program to draw two overlapping circles.
- (2) Write a program to fill overlapping circles with different colors.
- (3) Write a program to draw 10 rectangles having a similar size one after the other in one vertical line.
- (4) Draw alternately bars and rectangles one after the other.
- (5) Draw different-shaped rectangles and fill them with different colors.
- (6) Draw a right-angle triangle.
- (7) Write a program to draw a circle and show the mouse cursor within the circle.
- (8) Use the concepts of object oriented programming and draw a circle, a bar and a rectangle.
- (9) Write a program to draw arcs with a varying radius from 20 to 40 units with an angle between 40 and 90°.
- (10) Write a program to draw filled circles within a rectangle.

Appendices

APPENDIX A

American Standard Code for Information Interchange

ASCII Value	Symbol	Description
000	(NUL)	Null char
001	☉ (SOH)	Start of Heading
002	● (STX)	Start of Text
003	♥ (ETX)	End of Text
004	♦ (EOT)	End of Transmission
005	♣ (ENQ)	Enquiry
006	♠ (ACK)	Acknowledgment
007	• (BEL)	Bell
008	▣ (BS)	Back Space
009	○ (HT)	Horizontal Tab
010	▣ (LF)	Line Feed
011	♂ (VT)	Vertical Tab
012	♀ (FF)	Form Feed
013	♪ (CR)	Carriage Return
014	♪ (SO)	Shift Out / X-On
015	☀ (SI)	Shift In / X-Off
016	▶ (DLE)	Data Line Escape
017	◀ (DC1)	Device Control 1 (often XON)
018	↕ (DC2)	Device Control 2
019	!! (DC3)	Device Control 3 (often XOFF)
020	¶ (DC4)	Device Control 4
021	§ (NAK)	Negative Acknowledgement
022	— (SYN)	Synchronous Idle
023	↕ (ETB)	End of Transmit Block
024	↑ (CAN)	Cancel
025	↓ (EM)	End of Medium
026	→ (SUB)	Substitute

ASCII Value	Symbol	Description
027	← (ESC)	Escape
028	¬ (FS)	File Separator
029	↔ (GS)	Group Separator
030	▲ (RS)	Record Separator
031	▼ (US)	Unit Separator
032	(BL)	Space (Blank)
033	!	Exclamation mark
034	“	Double quotes (or speech marks)
035	#	Number
036	\$	Dollar
037	%	Percentage
038	&	Ampersand
039	'	Single quote
040	(	Open parenthesis (or open bracket)
041	)	Close parenthesis (or close bracket)
042	*	Asterisk
043	+	Plus
044	,	Comma
045	-	Hyphen
046	.	Period, dot or full stop
047	/	Slash or divide
048	0	Zero
049	1	One
050	2	Two
051	3	Three
052	4	Four

ASCII Value	Symbol	Description
053	5	Five
054	6	Six
055	7	Seven
056	8	Eight
057	9	Nine
058	:	Colon
059	;	Semicolon
060	<	Less than (or open angled bracket)
061	=	Equals
062	>	Greater than (or close angled bracket)
063	?	Question mark
064	@	At symbol
065	A	Uppercase A
066	B	Uppercase B
067	C	Uppercase C
068	D	Uppercase D
069	E	Uppercase E
070	F	Uppercase F
071	G	Uppercase G
072	H	Uppercase H
073	I	Uppercase I
074	J	Uppercase J
075	K	Uppercase K
076	L	Uppercase L
077	M	Uppercase M
078	N	Uppercase N
079	O	Uppercase O
080	P	Uppercase P
081	Q	Uppercase Q
082	R	Uppercase R
083	S	Uppercase S
084	T	Uppercase T
085	U	Uppercase U
086	V	Uppercase V
087	W	Uppercase W
088	X	Uppercase X
089	Y	Uppercase Y
090	Z	Uppercase Z
091	[	Opening bracket
092	\	Backslash
093	]	Closing bracket

ASCII Value	Symbol	Description
094	^	Caret – circumflex
095	_	Underscore
096	`	Grave accent
097	a	Lowercase a
098	b	Lowercase b
099	c	Lowercase c
100	d	Lowercase d
101	e	Lowercase e
102	f	Lowercase f
103	g	Lowercase g
104	h	Lowercase h
105	i	Lowercase i
106	j	Lowercase j
107	k	Lowercase k
108	l	Lowercase l
109	m	Lowercase m
110	n	Lowercase n
111	o	Lowercase o
112	p	Lowercase p
113	q	Lowercase q
114	r	Lowercase r
115	s	Lowercase s
116	t	Lowercase t
117	u	Lowercase u
118	v	Lowercase v
119	w	Lowercase w
120	x	Lowercase x
121	y	Lowercase y
122	z	Lowercase z
123	{	Opening brace
124		Vertical bar
125	}	Closing brace
126	~	Equivalency sign – tilde
127	(DEL)	Delete
128	Ç	Latin capital letter C with cedilla
129	ü	Latin small letter u with diaeresis
130	é	Latin small letter e with acute
131	â	Latin small letter a with circumflex
132	ä	Latin small letter a with dieresis
133	à	Latin small letter a with grave
134	â	Latin small letter a with ring above

ASCII Value	Symbol	Description	ASCII Value	Symbol	Description
135	ç	Latin small letter c with cedilla	167	°	Masculine ordinal indicator
136	ê	Latin small letter e with circumflex	168	¿	Inverted question mark
137	ë	Latin small letter e with dieresis	169	¬	Reversed not
138	è	Latin small letter e with grave	170	¬	Not
139	ï	Latin small letter i with dieresis	171	½	Vulgar fraction one half
140	î	Latin small letter i with circumflex	172	¼	Vulgar fraction one quarter
141	ì	Latin small letter i with grave	173	¡	Inverted exclamation
142	Ä	Latin capital letter A with dieresis	174	«	Left-pointing double angle quotation
143	Å	Latin capital letter A with ring above	175	»	Right-pointing double angle quotation
144	É	Latin capital letter E with acute	176	░	Light shade
145	æ	Latin small letter ae	177	▒	Medium shade
146	Æ	Latin capital letter AE	178	▓	Dark shade
147	ô	Latin small letter o with circumflex	179	▮	Box drawings light vertical
148	ö	Latin small letter o with dieresis	180	├	Box drawings light vertical and left
149	ò	Latin small letter o with grave	181	┤	Box drawings vertical single and left double
150	û	Latin small letter u with circumflex	182	├┤	Box drawings vertical double and left single
151	ù	Latin small letter u with grave	183	┐	Box drawings down double and left single
152	ÿ	Latin small letter y with dieresis	184	└	Box drawings down single and left double
153	Ö	Latin capital letter O with dieresis	185	└┐	Box drawings double vertical and left
154	Ü	Latin capital letter U with dieresis	186	┐┐	Box drawings double vertical
155	¢	Cent	187	└┐	Box drawings double down and left
156	£	Lira	188	┐└	Box drawings double up and left
157	¥	Yen	189	└┐	Box drawings up double and left single
158	Pts	Peseta	190	└┐	Box drawings up single and left double
159	ƒ	Latin small letter f with hook	191	┘	Box drawings light down and left
160	á	Latin small letter a with acute	192	┘	Box drawings light up and right
161	í	Latin small letter i with acute	193	┘	Box drawings light horizontal and up
162	ó	Latin small letter o with acute			
163	ú	Latin small letter u with acute			
164	ñ	Latin small letter n with tilde			
165	Ñ	Latin capital letter N with tilde			
166	ª	Feminine ordinal indicator			

ASCII Value	Symbol	Description
194	┐	Box drawings light horizontal and down
195	┌	Box drawings light vertical and right
196	─	Box drawings light horizontal
197	├	Box drawings light vertical and horizontal
198	┤	Box drawings vertical single and right double
199	┘	Box drawings vertical double and right single
200	└	Box drawings double up and right
201	┴	Box drawings double down and right
202	┬	Box drawings double up and horizontal
203	┴	Box drawings double down and horizontal
204	┘	Box drawings double vertical and right
205	═	Box drawings double horizontal
206	┘	Box drawings double vertical and horizontal
207	┬	Box drawings up single and horizontal double
208	┬	Box drawings up double and horizontal single
209	┴	Box drawings down single and horizontal double
210	┴	Box drawings down double and horizontal single
211	└	Box drawings up double and right single
212	└	Box drawings up single and right double
213	┘	Box drawings down single and right double
214	┘	Box drawings down double and right single

ASCII Value	Symbol	Description
215	⦿	Box drawings vertical double and horizontal single
216	⦿	Box drawings vertical single and horizontal double
217	┘	Box drawings light up and left
218	┐	Box drawings light down and right
219	■	Full block
220	▀	Lower half block
221	▄	Left half block
222	▄	Right half block
223	▀	Upper half block
224	α	Greek small letter Alpha
225	β	Greek small letter Beta
226	Γ	Greek capital letter Gamma
227	π	Greek small letter Pi
228	Σ	Greek capital letter Sigma
229	σ	Greek small letter Sigma
230	μ	Greek small letter Mu
231	τ	Greek small letter Tau
232	Φ	Greek capital letter Phi
233	θ	Greek small letter Theta
234	Ω	Greek capital letter Omega (ohm)
235	δ	Greek small letter Delta
236	∞	Infinity
237	ø	Latin small letter o with stroke
238	Є	Cyrillic capital letter Ukrainian IE
239	∩	Intersection
240	≡	Identical to
241	±	Plus-minus
242	≥	Greater than or equal to
243	≤	Less than or equal to
244	∫	Top half integral
245	∫	Bottom half integral

ASCII Value	Symbol	Description
246	÷	Division
247	≈	Almost equal to
248	◦	White bullet
249	•	Bullet
250	·	Decimal point

ASCII Value	Symbol	Description
251	√	Square root
252	η	Greek small letter Eta
253	²	Superscript two
254	■	Black square
255		

APPENDIX B

Data Types in C++

Data Type	Size in Bytes	Range
char	1	–128 to 127
unsigned char	1	0 to 255
signed char	1	–128 to 127
int	2	–32768 to 32767
unsigned int	2	0 to 65535
signed int	2	–32768 to 32767
short int	2	–32768 to 32767
unsigned short int	2	0 to 65535
signed short int	2	–32768 to 32767
long int	4	–2147483648 to 2147483647
signed long int	4	–2147483648 to 2147483647
unsigned long int	4	0 to 4294967295
float	4	3.4E –38 to 3.4E + 38
double	8	1.7E –308 to 1.7E + 308
long double	10	3.4E –4932 to 1.1E + 4932
enum	2	–32768 to 32767
bool	1	true / false

APPENDIX C

Precedence of Operators in C++

# Group	Operator	Operation
1. Top	() [] -> :: .	Function call Array subscript C++ indirect component selector C++ scope access/resolution C++ direct component selector

(Continued)

# Group	Operator	Operation
2. Unary	!	Logical negation (NOT)
	~	Bitwise (1's) complement
	+	Unary plus
	-	Unary minus
	++	Pre-increment or post-increment
	--	Pre-decrement or post-decrement
	&	Address
	*	Indirection
	sizeof	Returns size of operand in bytes
	new delete	Dynamically allocates C++ storage Dynamically deallocates C++ storage
3. Multiplicative	*	Multiply
	/	Divide
4. Member Access	%	Remainder (modulus)
	.*	C++ deference
5. Additive	->*	C++ deference
	+	Binary plus
6. Shift	-	Binary minus
	<<	Shift left
7. Relational	>>	Shift right
	<	Less than
	<=	Less than or equal to
	>	Greater than
8. Equality	>=	Greater than or equal to
	==	Equal to
	!=	Not equal to
9. Bitwise	&	Bitwise AND
	^	Bitwise XOR
		Bitwise OR
10. Logical	&&	Logical AND
		Logical OR
11. Conditional	?:	(a ? x : y means "if a then x, else y")
12. Assignment	=	Simple assignment
	*=	Assign product
	/=	Assign quotient
	%=	Assign remainder (modulus)
	+=	Assign sum
	-=	Assign difference
	&=	Assign bitwise AND
	^=	Assign bitwise XOR
	=	Assign bitwise OR
	<<=	Assign left shift
13. Comma	>>=	Assign right shift
	,	Evaluate

APPENDIX D

American Standard Code for Information Interchange

ASCII Value	Symbol	ASCII Value	Symbol	ASCII Value	Symbol	ASCII Value	Symbol
000	NUL	036	\$	072	H	107	k
001	SOH	037	%	073	I	108	l
002	STX	038	&	074	J	109	m
003	ETX	039	'	075	K	110	n
004	EOT	040	(	076	L	111	o
005	ENQ	041	)	077	M	112	p
006	ACK	042	*	078	N	113	q
007	BEL	043	+	079	O	114	r
008	BS	044	,	080	P	115	s
009	HT	045	-	081	Q	116	t
010	LF	046	.	082	R	117	u
011	VT	047	/	083	S	118	v
012	FF	048	0	084	T	119	w
013	CR	049	1	085	U	120	x
014	SO	050	2	086	V	121	y
015	SI	051	3	087	W	122	z
016	DLE	052	4	088	X	123	{
017	DC1	053	5	089	Y	124	
018	DC2	054	6	090	Z	125	}
019	DC3	055	7	091	[	126	~
020	DC4	056	8	092	\	127	DEL
021	NAK	057	9	093	]		
022	SYN	058	:	094	^		
023	ETB	059	;	095	-		
024	CAN	060	<	096	'		
025	EM	061	=	097	a		
026	SUB	063	?	098	b		
027	ESC	064	@	099	c		
028	FS	065	A	100	d		
029	GS	066	B	101	e		
031	US	067	C	102	f		
032	BLANK	068	D	103	g		
033	!	069	E	104	h		
034	"	070	F	105	i		
035	#	071	G	106	j		

APPENDIX E**Scan Codes of Keyboard Keys**

Key	Normal	Shift	Ctrl	Alt
A	1E61	1E41	1E01	1E00
B	3062	3042	3002	3000
C	2E63	2E42	2E03	2E00
D	2064	2044	2004	2000
E	1265	1245	1205	1200
F	2166	2146	2106	2100
G	2267	2247	2207	2200
H	2368	2348	2308	2300
I	1769	1749	1709	1700
J	246A	244A	240A	2400
K	256B	254B	250B	2500
L	266C	264C	260C	2600
M	326D	324D	320D	3200
N	316E	314E	310E	3100
O	186F	184F	180F	1800
P	1970	1950	1910	1900
Q	1071	1051	1011	1000
R	1372	1352	1312	1300
S	1F73	1F53	1F13	1F00
T	1474	1454	1414	1400
U	1675	1655	1615	1600
V	2F76	2F56	2F16	2F00
W	1177	1157	1117	1100
X	2D78	2D58	2D18	2D00
Y	1579	1559	1519	1500
Z	2C7A	2C5A	2C1A	2C00
1	0231	0221	7800	
2	0332	0340	0300	7900
3	0433	0423	7A00	
4	0534	0524	7B00	
5	0635	0625	7C00	
6	0736	075E	071E	7D00
7	0837	0826	7E00	
8	0938	092A	7F00	
9	0A39	0A28	8000	

(Continued)

Key	Normal	Shift	Ctrl	Alt
0	0B30	0B29	8100	
-	0C2D	0C5F	OC1F	8200
=	0D3D	0D2B	8300	
[	1A5B	1A7B	1A1B	1A00
]	1B5D	1B7D	1B1D	1B00
;	273B	273A	2700	
"	2827	2822		
'	2960	297E		
\	2B5C	2B7C	2B1C	2600
,	332C	333C		
.	342E	343E		
/	352F	353F		
F1	3B00	5400	5E00	6800
F2	3C00	5500	5F00	6900
F3	3D00	5600	6000	6A00
F4	3E00	5700	6100	6B00
F5	3F00	5800	6200	6C00
F6	4000	5900	6300	6D00
F7	4100	5A00	6400	6E00
F8	4200	5B00	6500	6F00
F9	4300	5C00	6600	7000
F10	4400	5D00	6700	7100
F11	8500	8700	8900	8B00
F12	8600	8800	8A00	8C00
BackSpace	0E08	0E08	0E7F	OE00
Del	5300	532E	9300	A300
Down Arrow	5000	5032	9100	A000
End	4F00	4F31	7500	9F00
Enter	1C0D	1C0D	1C0A	A600
Esc	011B	011B	011B	0100
Home	4700	4737	7700	9700
Ins	5200	5230	9200	A200
Keypad 5	4C35	8F00		
Keypad *	372A	9600	3700	
Keypad –	4A2D	4A2D	8E00	4A00
Keypad +	4E2B	4E2B	4E00	
Keypad /	352F	352F	9500	A400

(Continued)

Key	Normal	Shift	Ctrl	Alt
Left Arrow	4B00	4B34	7300	9B00
Page Down	5100	5133	7600	A100
Page Up	4900	4939	8400	9900
Prn scr	7200			
Right Arrow	4D00	4D36	7400	9D00
SpaceBar	3920	3920	3920	3920
Tab	0F09	0F00	9400	A500
Up Arrow	4800	4838	8D00	9800

Index

A

- `abort()` function, 773
- `abs()` function, 238–239
- Abstract classes, 489, 619–621
- Abstract data type, 262, 274, 346
- `accept()` function, 335–336
- Accessing private members with pointers, 561–562
- Access scope in classes, 453
- Access scope of class members, 451
- Actual and formal arguments, 207
- addition (+) operator, 434–435
- Algorithms, 789–794
- American National Standards Institute (ANSI), 2
- anonymous namespace, 830
- Anonymous objects, 372–374
- Anonymous union, 127
- ANSI and TURBO-C++ header files, 839–840
- ANSI and TURBO-C++ keywords, 836–839
- ANSI C++ standard, 698
- `arc()` function, 846, 850
- Arithmetic function object, 812
- Arithmetic operations with pointers, 536–538
- Array, 511–529
 - Array characteristics, 512–515
 - Array elements, accessing, 515–516
 - Array elements, passing to function, 517–519
 - Array of objects, 290–292
 - Array of objects using constructors, 359–360
 - Arrays, derived data type, 119–120
 - Arrays initialization using functions, 519–520
 - Arrays of classes, 525–528
 - Arrays of pointers, 516–517, 615–617
 - Arrays (two-dimensional), 520–522
 - `*arrp[]`, 517
- `assign()` function, 595
- Assignment operator (=) overloading, 412–414
- Associative array, 810
- Associative containers, 789

B

- `Bar()` function, 847
- Base and derived classes with constructors, 476–477
- Base and derived classes without constructors, 474
- Base and derived classes without default constructors, 478–479
- Base and derived class pointers, 609–612
- Base class with constructors and derived class without constructors, 475
- Base class without constructors and derived class with constructors, 475–476
- Base class with various constructors and derived class with one constructor, 477–478
- Basic to class type conversion, 415–416
- `begin()` function, 797
- Binary and ASCII files, 666–670
- Binary and ASCII formats, representation, 666
- Binary operators overloading, 405–409
- Bit fields, 67–70
- Bit fields and classes, 317–319
- Bit fields, flags without, 70–71
- Bits, 532
- `bool` data type, 162, 818–820
- Borland C++, 24–27. *See also* C++ environments
- `break` statement, 176, 198
- Bubble sort using function templates, 717–718
- Buffer, 35–36

C

- Call by address (pass by address), 210–212
- Call by reference (pass by reference), 212–215
- Call by value (pass by value), 209–210
- Calling array elements, 512
- Calling convention, 599–600
- C and C++ common keywords, 100
- C++ and C differences, 1–2
- C and C++ I/O functions, difference, 49–56

C and C++ operators, 129–132
 capacity() function, 745
 Cascaded input operation, 39
 catch, 761
 catch block, 761, 765, 777
 Catching multiple exceptions, 767–768
 C++ basic data types, 116
 C++ binding, 604
 C++ constants, 102–108
 C++ data types, 115
 Central processing unit (CPU), 666
 C++ environments, 23–27
 C++ evolution, 2
 C++ exception-handling mechanism, 761–762
 c() function, 520
 C++ function advantages, 204–205
 C++ functions, 203–251
 C++ graphics, 843
 change() function, 300
 Charm++, 17
 check() function, 770–771
 cin.get() function, 59
 C++ input/output (I/O) function, 33–34
 cin statement, 41, 73
 Circle() function, 846, 850
 C++ keywords, 100–101
 Class, 8–9
 class, 258
 Class byte, 368–369
 class declaration, 262
 class father, 495–499
 class in C++, 258, 262–263
 class instantiation, 263
 class members, access limits, 267–268
 Class noname, 373
 class physique, 462
 Class templates, 698–701, 704
 Class templates and inheritance, 715–716
 Class templates revisited, 713–715
 Class templates with exception handling, 777–778
 Class templates with more parameters, 704–705
 Class templates with overloaded operators, 711–712
 Class type to basic data type conversion, 416–418
 Class word, 565–566
 clear() function, 810
 Closed subroutines, 273
 close() function, 660
 Closegraph() function, 847
 COBOL, 4, 5
 Command-line arguments, 679–680
 Comma operator, 142–143
 Communication between program, file,
 and output device, 640
 compare() function, 750–751
 compile time polymorphism, 604
 complex() function, 240–241
 Computer display modes, 844
 concrete classes, 618
 Constant pointers, 106–108

const arguments, 222–224
 const_cast operator, 822–823
 const keyword, 304
 const member functions, 304–305
 const objects, 363–364
 Constructor, 348–349, 468, 490
 Constructor and destructor, execution sequence, 595–597
 Constructor and destructor with static members, 383–384
 Constructor, applications with, 350–353
 Constructor, characteristics, 349
 Constructors and destructors, calling, 367–369
 Constructors and destructors, exceptions, 771–772
 Constructors and destructors in base and derived
 classes, 469–473
 Constructors and destructors, private, 374–375
 Constructors and multiple inheritance, 479–480
 Constructors and virtual functions, 631–632
 Constructors in derived class, 490–491
 Constructors in multilevel inheritance, 482–483
 Constructors in multiple inheritance with explicit
 calls, 480–481
 Constructors with arguments, 353–355
 Constructors with default arguments, 360–361
 container class, 483, 715
 containership, 483
 continue statement, 176–177, 198–199
 Conventional programming, disadvantage, 4–5
 convert() function, 332–333
 C++ operator keywords, 840
 Copy constructors, 361–363
 Counter-controlled repetition, 188
 cout.fill() statement, 65
 cout.put() statement, 50
 cout statement, 40–41, 47–48, 51, 55, 57, 110, 125, 149,
 154, 220, 580
 cout width() statement, 61, 64, 80, 82
 cppstyle() function, 148–149
 C++ pre-defined streams, 34–35
 C++ program create and execute, 21–22
 C++ program flowchart for creating, 23
 C++ program structure, 27–28
 C++ program without class, 28–29
 C++ stream classes, 34, 36–37
 C string to C++ string, 734–735
 C-style string, 732
 cube() function, 228–229
 Curly braces {}, 143–144
 C++ usage, 18

D

data, 641
 Data abstraction, 11
 Data hiding/encapsulation, 11, 258, 274–277
 Data items in STL sequence container, 787
 Decision-making statements, 161–183
 Default arguments, 218–222, 224–225, 246–247
 Default constructor, 349–350
 default keyword, 162

- #define preprocessor directive, 105
- Define repetitive actions. *See* Counter-controlled repetition
- Delegation, 14–15
- delegation, 483
- delete operator, 139–140, 582–586, 584, 596–597
- deque class, 788
- Dereferencing operator (*), 121
- derived classes, 442, 609
- destination stream, 641
- Destructors, 348–349, 364–367
- Destructors and virtual functions, 634
- Destructors, characteristics, 349–350
- Direct access to private members, 562–564
- Disk I/O functions, 640
- display() function, 225, 246–247, 348, 377, 606–607
- do-nothing functions, 618
- Dot operator, 124
- do-while loop, 188, 195–196
- do-while statement with while loop, 196–197
- dummy functions, 618
- Dynamic binding, 13, 621
- dynamic_cast operator, 824
- Dynamic initialization, 112–115
- Dynamic initialization using constructors, 375–377
- Dynamic (late) binding, 607–609
- Dynamic memory allocation, 581
- Dynamic objects, 598–599
- Dynamic operators and constructors, 377–379

E

- early binding, 604, 621
- Ellipse() function, 846–847
- else-if ladder, 171–175
- else keyword, 167
- empty() function, 746–747
- empty, static, and const classes, 310
- Encapsulation, 11–12
- enum, 128
- Enumerated data type, 128–129
- eof() function, 653–654
- Error checking techniques in file operation, 675–677
- Error-free program, 759
- Error handling functions in file operation, 674–678
- Errors checking in file operation, 650–652
- Error trapping functions, 675
- Exception handling goal, 760
- Exception handling, guidelines, 761–765, 778–779
- Exception handling, principles, 760
- Exceptions and inheritance, 776–777
- Exceptions and operator overloading, 774–776
- exchange() function, 725–726

F

- Fabs() function, 238–239
- far pointer, 580–581
- Filebuf, 642

- File modes, 655
- File opening modes, 655–657
- File operations steps, 643–650
- File pointers and manipulators, 657–661
- File stream classes, 641–643
- Fill() algorithm, 791–792
- Fillellipse() function, 854–855
- Filling patterns, 858–862
- Finding end of file, 653–655
- Flags without bit fields, 70–71
- float add() function, 234
- for_each() operator, 790–791
- for loop, 188–191
- Formatted and unformatted data, 37–38
- Formatted console I/O operations, 59–67
- FORTRAN, 4, 5
- free() function, 137
- free store, 582
- friend classes, 302–304
- friend functions, 295–302, 406, 409–412
- fstreambase class, 642
- fstream class, 643
- fstream class derivation, 659
- fstream object and disk file, interaction, 644
- Function call, 206
- function definition, 207
- Function object, 811–813
- Function overloading, 118, 228–230
- Function overloading, precautions with, 234–235
- Function overloading, principles, 230–234
- Function prototype declaration, 205–206
- functions, 204
- Functions, derived data type, 118–119
- function show(), 244–245
- Function templates, 698, 704
- Function templates with more arguments, 705–708

G

- generic function, 698
- Genericity, 15
- Getcolor() function, 848
- getdata() function, 460
- get() function, 49, 52, 321, 468, 664–665
- geth() function, 277
- getline() function, 54–55, 665, 688
- Getmaxx() function, 847
- Getmaxy() function, 847
- getnum() function, 403–404
- get pointer (input pointer), 657
- Get pointer in read mode, 657
- gettrys() function, 294
- getvalue() function, 298–300
- goto statement, 175–176
- Graphics, 843
- Graphics functions, 846–848
- Graphics initialization, 844–846
- Graphresult() function, 847

H

Header files and libraries, 29–30
 heap, 586
 Heap consumption, 586–588
 Hex value, 731
 Hierarchical inheritance, 454, 460–462
 hierarchy of classes, 489
 huge pointer, 580
 Hybrid inheritance, 454, 462–465, 501–502

I

Identifier, 101–102
 if-else-if staircase. *See* else-if ladder
 if-else statement, 167–169
 if statement, 162–165, 523
 ifstream object, 683
 ifstream object and disk file, interaction, 645
 #include pre-processor directive, 205
 Increment/decrement operators, 404–405
 Indefinite repetitive actions. *See* Sentinel-controlled repetition
 in() function, 243–244
 Inheritance, 12, 442
 Inheritance, advantages and disadvantages, 494
 inheritance pathway, 457
 Inheritance types, 454–455
 Initgraph() function, 845–846
 inline functions, 225–228, 272–273
 Inline functions, rules, 274
 Innovative data types, 818–821
 Input and output streams, 38, 641
 input-output arguments, 209
 insert() function, 740
 instance of the template, 698
 instantiating, 704
 Integer constants, 103
 ios class, 37, 67
 iostream class, 642
 iostream library, 33
 istream class, 37, 56–59, 642
 istrstream class, 680–682
 Iterator hierarchy, 794
 Iterators, 786, 794–795
 Iterator status, 794
 itoa() function, 437–438, 861

J

Java, 17
 JVM (Java virtual machine), 17

K

Key and values in maps, 808
 Keyword cdecl, 838
 Keyword explicit, 826–827
 Keyword mutable, 827–828
 Keyword namespace, 828–829

Keyword operator, 398–400
 Keyword pascal, 837
 Keyword using, 830–834
 kind of relationship, 442

L

late binding. *See* Dynamic binding
 late/dynamic binding. *See* run time
 Library functions, 205, 237–241
 Line() function, 247–248, 846, 850–851
 Linked lists with templates, 720–722
 List, 801–808
 List class functions, 801
 Local classes, 307–310
 Local/temporary objects, 349
 Local *versus* global object, 384–385
 Logical function object, 812
 ! (logical negation operator), 650
 Loop, 187
 loop() function, 338–339
 Loop variable, 188
 lvalue kk, 212
 Lvalues (left values), 215

M

main() as constructor and destructor, 379–380
 main() function, 27–28, 204–205, 233, 308, 310–312, 314, 495
 main() function as member function, 311–312
 malloc() function, 587, 589–590
 Manipulators, 71–74
 Manipulators, user-defined, 74–76
 Manipulators with arguments, 661–664
 Manipulator with one and multiple parameter, 76–87
 Map, 808–811
 Map class functions, 811
 Mapped values, 789
 matrix, 520
 max() template, 724–725
 Member and non-member function, 310–311
 Member function, 268
 Member function, characteristics, 272
 Member function outside class, 271–272, 328–330
 Member function templates, 709–710
 memory address, 532
 Memory management operators, 137–141
 Memory models, 577–579
 Memory organization, 578
 merge() function, 807–808
 Message passing, 13–14
 Method, 10
 Miscellaneous functions, 752–754
 modf and modfl function, 238
 modular programming, 204
 Monolithic program, 5, 161
 Mouse programming, 862–866
 Moveto() function, 847

`mult()` function, 726
 Multilevel inheritance, 454, 457–459, 500
 Multipath inheritance, 455, 465
 Multiple `catch` statements, 765–766
 Multiple constructors, 355–359
 Multiple `ifs`, 165–166
 Multiple inheritance, 454, 459–460
 Multiple inheritance and virtual class, 481–482
 Mutating sequence operation (algorithm), 793

N

Namespace, 133–137
 Namespace alias, 835
`namespace std`, 786
 near pointer, 580
 Nested class, 319–320, 370–372
 Nested `for` loops, 191–192
 Nested `if-else` statement, 169–171
 nested namespace, 829
 Nested `switch` case, 182–183
 Nesting member function, 270
 new and `malloc()`, difference, 582
`_new_handleri`, 587
 new operators, 137–138, 582–586, 596–597
`nocreate` mode, 685
 Noncommon figures, 866
 Non-mutating sequence operations, 789–790
 Non-template function, working, 700
 Normal function templates, 701–704
`not` operator, 650
 Numeric algorithms, 794

O

Object address, 597–598
 Object as class member, 483–488
 Object oriented languages, 16–17
 Object oriented programming, 6–7
 Object oriented technology, 2–4, 15
 Objects as function arguments, 292–295
 Object slicing, 629–631
`ofstream` object, 683
 One-argument constructor and operator function, 424–425
 One class type to another class type conversion, 418–422
 One-dimensional array declaration and initialization, 511–512
`open()` function, 648
 Open subroutines. *See* Inline functions
 * operator, 131–132
 Operator +, 396, 423–424
 [] operator, 131–132, 722
`operator()` function, 403, 406
`operator++()` function, 401–402, 404
`operator-()` function, 403
 Operator function name, 398
 Operator overloading, 395–396, 398
 Operator return type, 403–404
 Operators in C and C++, 129–132

`ostream`, 426, 642, 681
`ostream` class, 680–681
 Outside member function as `inline`, 272–274
`Outtextxy()` function, 848
 Overloading assignment operator (=), 412–414
 Overloading binary operators, 405–409
 Overloading constructors, 355–359
 Overloading `main()` functions, 313–314
 Overloading member functions, 312–313, 492–494
 Overloading new and delete in classes, 592–595
 Overloading new and delete operators, 588–592
 Overloading of template functions, 708–709
 Overloading operators rules, 422–424
 Overloading stream operators, 425–427
 Overloading unary operators, 400–403
 Overloading with `friend` function, 409–412

P

Palindrome number, 195
 Parameterized constructor, 353–355
 Passing arguments, 209–215
`period()` function, 294–295
 Pointer *p, 583
 Pointers, 531–573
 Pointers, accessing private members, 561–562
 Pointers and inheritance, 491–492
 Pointers and two-dimensional arrays, 523
 Pointers, arithmetic operations with, 536–538
 Pointers declaration, 533–536
 Pointers, derived data type, 118
 Pointers feature, 532–533
 Pointer to class, 542–543
 Pointer to derived classes and base class, 550–554
 Pointer to members, 554–561
 Pointer to object, 544–546
 Pointer to pointer, 538–539
 polymorphism, 12–13, 603–604, 615
 Polymorphisms in C++, 604
`pop_front()` function, 804
`pop()` operation, 715
 Pre-defined manipulators, 71
`printf()` functions, 2, 41
`printf()` statement, 41, 47, 49, 513, 527, 535, 565
 private access, 443–444
 Private constructors and destructors, 374–375
 Private inheritance, 447–449
 Private inheritance, protected data, 449–453
 private keyword, 265–266
 Private member function, 270–271, 330–339
 Procedural/structured programming, 5–6
 Procedure oriented languages, 4
 Program execution before `main()`, 381–383
 Programs using library functions, 848–856
 Program with/without function, 204
 protected access specifier, 443–444
 protected keyword, 266–267
 Public inheritance, 444–447
 public keyword, 264–265

pure abstract classes, 618
 Pure virtual functions, 618–619
 push_back() function, 804–805
 push() operation, 715
 put() function, 49–50, 52, 664–665
 Put pointer in append mode, 658
 Put pointer in write mode, 658
 put pointer (output pointer), 657

Q

Qualifier classes and inheritance, 489–490

R

Random access memory (RAM), 112
 Random access operation, 670–674
 range of array, 521
 rdbuf() copies, 688
 readdata() member function, 327
 Read-only data member, 364
 Real constants, 103–104
 Rectangle() function, 847, 852–853
 Recursion, 235–236
 Recursion with template functions, 710–711
 Recursive constructors, 380–381
 Recursive member function, 306–307
 References, derived data type, 120–121
 Referencing operator (&), 121
 reinterpret_cast operator, 823–824
 Relational function object, 811
 Relational operators, 738–739
 Restorecrtmode() function, 847
 Re-throwing exception, 768–769
 Return by reference, 216–217
 return statement, 207–209, 217, 307
 Return type operator, 403–404
 Return values of functions, 690
 Reusability, 14
 Reusability, 442
 run time, 604
 runtime type information (RTTI), 824–825
 Rvalues (right values), 216

S

scanf() functions, 2
 Scope access operator, 133
 Secondary memory, 581
 seekg() function, 686
 seekp() and seekg() functions, 661–662
 Segment and offset address, 579–581
 Sending output (file) to devices, 682–683
 Sentinel-controlled repetition, 188
 Sequential access files, 664–665
 Setbkcolor() function, 859
 Setcolor() function, 848
 setdata() function, 433–434

setf() function, 67–68, 70
 Setfillstyle() function, 854–855, 859, 861
 Setlinestyle() function, 857–858
 set_new_handler, 587–588
 set_terminate() function, 773–774
 Settextjustify() function, 848
 Settextstyle() function, 848, 856
 setv() function, 569
 show() function, 242–243, 244–246, 347, 371, 375, 388, 416, 449, 459, 493–494, 519, 572–573
 signed modifiers, 117
 Single-character constants, 104
 Single inheritance, 454, 456–457
 sizeof() operator, 141, 670
 Sleep() function, 855
 Smalltalk, 17
 sort() function, 805
 Sorting operation (algorithm), 792–793
 source stream, 641
 Specifying exceptions, 769–771
 square() function, 227, 232
 Standard namespace std, 835–836
 Standard template library (STL), 735, 785–813
 static binding. *See* early binding
 static_cast operator, 821–822
 Static (early) binding, 604–607
 static member variables, 280–287
 static members in memory, 281
 static object, 289–290
 static private and public member function, 287–289
 Status bits, 675
 STL containers classification, 787
 STL programming model, 786
 STL sequence containers, 787–788
 STL (standard template library). *See* Standard template library (STL)
 strcpy() function, 824
 streambuf class, 60, 641
 Stream objects with multiple files, 650
 stream operators, 666
 Stream operators overloading, 425–427
 string, 643, 731–756
 String attributes, 743–747
 String constants, 104, 108–109
 String constructors, 735
 String library functions, 732
 String manipulating functions, 737
 String objects, handling, 740–743
 Strings, accessing elements of, 747–750
 strlen() function, 115, 165, 733
 stringstream class, 680–682
 struct, 123
 struct declaration, 261–262
 Structure and classes, user-defined data types, 123–125
 Structure in C, 259–261
 Structure in C++, 261–262
 sum() function, 243, 379
 Swap() function, 210, 212

switch statement, 177–182
Syntaxes of various loops, 188

T

tellg() function, 663, 686
Template, 698
Templates and macros, differences, 719–720
Templates, guidelines, 718–719
Text mode, 844
this pointer, 546–549, 568
Three- or multi-dimensional arrays, 524–525
throw, 760–761
Token, 100
Tree, 789
tri_num() function, 307, 381
try, 760–761
try block, 761, 765
Turbo C++, 25, 100–101, 280, 350, 646, 817
Two-dimensional arrays, 520–522
Type casting formats, 45
Type casting operators, 821–826
Type conversion, 414–422, 425
typeid() operator, 825
Type modifiers, 117

U

Unary operators overloading, 400–403
Uncaught exceptions, controlling, 773–774
Unconditional control transfer statements, 175–177
#undef abs statement, 239
Unformatted console I/O operations, 38–48
Union, user-defined data types, 125–127
unsigned modifiers, 117
upcasting, 612
User-defined data types, 123–129, 395

User-defined functions, 205
using declaration, 832
using directive, 835

V

Variable, 108
Variable declaration, 108–109
VDU memory, 844
Vector, 795–800
Vector class functions, 796
Video display and display adapters, 844
Virtual base classes, 466–468
Virtual destructors, 632–633
virtual function, 607, 612–613
Virtual functions in derived classes, 627–629
Virtual functions, rules, 613–615
Virtual functions, working, 621–627
void data type, 117
void pointers, 539–540
void pointers and addresses of objects, 564–565
Volatile member function, 305–306
VPTR, 623, 626–627
VTABLE, 623, 626–628

W

warning() function, 588–589, 621
wchar_t data type, 820–821
while loop, 192–195
Wide_character literal, 820
wild pointers, 540–542
Working with texts, 856–858
write() statements, 55–56

Z

Zero-argument constructor, 468, 478